



IEC 61691-7

Edition 1.0 2009-12

# INTERNATIONAL STANDARD

IEEE Std 1666™

Behavioural languages –  
Part 7: SystemC® Language Reference Manual





## THIS PUBLICATION IS COPYRIGHT PROTECTED

Copyright © 2009 IEEE

All rights reserved. IEEE is a registered trademark in the U.S. Patent & Trademark Office, owned by the Institute of Electrical and Electronics Engineers, Inc.

Unless otherwise specified, no part of this publication may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying and microfilm, without permission in writing from the IEC Central Office.

Any questions about IEEE copyright should be addressed to the IEEE. Enquiries about obtaining additional rights to this publication and other information requests should be addressed to the IEC or your local IEC member National Committee.

IEC Central Office  
3, rue de Varembé  
CH-1211 Geneva 20  
Switzerland  
Email: [inmail@iec.ch](mailto:inmail@iec.ch)  
Web: [www.iec.ch](http://www.iec.ch)

Institute of Electrical and Electronics Engineers, Inc  
3 Park Avenue  
New York, NY10016-5997  
USA  
Email: [stds.ipr@ieee.org](mailto:stds.ipr@ieee.org)  
Web: [www.ieee.org](http://www.ieee.org)

### About the IEC

The International Electrotechnical Commission (IEC) is the leading global organization that prepares and publishes International Standards for all electrical, electronic and related technologies.

### About IEC publications

The technical content of IEC publications is kept under constant review by the IEC. Please make sure that you have the latest edition, a corrigenda or an amendment might have been published.

- Catalogue of IEC publications: [www.iec.ch/searchpub](http://www.iec.ch/searchpub)

The IEC on-line Catalogue enables you to search by a variety of criteria (reference number, text, technical committee,...). It also gives information on projects, withdrawn and replaced publications.

- IEC Just Published: [www.iec.ch/online\\_news/justpub](http://www.iec.ch/online_news/justpub)

Stay up to date on all new IEC publications. Just Published details twice a month all new publications released. Available on-line and also by email.

- Electropedia: [www.electropedia.org](http://www.electropedia.org)

The world's leading online dictionary of electronic and electrical terms containing more than 20 000 terms and definitions in English and French, with equivalent terms in additional languages. Also known as the International Electrotechnical Vocabulary online.

- Customer Service Centre: [www.iec.ch/webstore/custserv](http://www.iec.ch/webstore/custserv)

If you wish to give us your feedback on this publication or need further assistance, please visit the Customer Service Centre FAQ or contact us:

Email: [csc@iec.ch](mailto:csc@iec.ch)  
Tel.: +41 22 919 02 11



IEC 61691-7

Edition 1.0 2009-12

# INTERNATIONAL STANDARD

IEEE Std 1666™

---

**Behavioural languages –  
Part 7: SystemC® Language Reference Manual**

INTERNATIONAL  
ELECTROTECHNICAL  
COMMISSION

PRICE CODE **XP**

ICS 25.040, 35.060

ISBN 2-8318-1069-4

# CONTENTS

|                                                                                   |     |
|-----------------------------------------------------------------------------------|-----|
| Foreword.....                                                                     | xi  |
| IEEE Introduction .....                                                           | xvi |
| 1. Overview.....                                                                  | 1   |
| 1.1 Scope.....                                                                    | 1   |
| 1.2 Purpose.....                                                                  | 1   |
| 1.3 Subsets .....                                                                 | 1   |
| 1.4 Relationship with C++.....                                                    | 1   |
| 1.5 Guidance for readers .....                                                    | 2   |
| 2. References.....                                                                | 3   |
| 3. Terminology and conventions used in this standard.....                         | 4   |
| 3.1 Terminology.....                                                              | 4   |
| 3.1.1 Shall, should, may, can.....                                                | 4   |
| 3.1.2 Implementation, application.....                                            | 4   |
| 3.1.3 Call, called from, derived from .....                                       | 4   |
| 3.1.4 Specific technical terms .....                                              | 4   |
| 3.2 Syntactical conventions .....                                                 | 6   |
| 3.2.1 Implementation-defined.....                                                 | 6   |
| 3.2.2 Disabled .....                                                              | 6   |
| 3.2.3 Ellipsis (...).....                                                         | 6   |
| 3.2.4 Class names.....                                                            | 6   |
| 3.2.5 Embolded text .....                                                         | 7   |
| 3.3 Semantic conventions .....                                                    | 7   |
| 3.3.1 Class definitions and the inheritance hierarchy .....                       | 7   |
| 3.3.2 Function definitions and side-effects .....                                 | 7   |
| 3.3.3 Functions whose return type is a reference or a pointer.....                | 7   |
| 3.3.4 Namespaces and internal naming .....                                        | 9   |
| 3.3.5 Non-compliant applications and errors .....                                 | 9   |
| 3.4 Notes and examples .....                                                      | 10  |
| 4. Elaboration and simulation semantics .....                                     | 11  |
| 4.1 Elaboration.....                                                              | 11  |
| 4.1.1 Instantiation .....                                                         | 11  |
| 4.1.2 Process macros.....                                                         | 13  |
| 4.1.3 Port binding and export binding .....                                       | 13  |
| 4.1.4 Setting the time resolution .....                                           | 14  |
| 4.2 Simulation.....                                                               | 14  |
| 4.2.1 The scheduling algorithm .....                                              | 15  |
| 4.2.2 Cycles in the scheduling algorithm.....                                     | 17  |
| 4.3 Running elaboration and simulation .....                                      | 18  |
| 4.3.1 Function declarations.....                                                  | 18  |
| 4.3.2 Function sc_elab_and_sim.....                                               | 18  |
| 4.3.3 Functions sc_argc and sc_argv .....                                         | 19  |
| 4.3.4 Running under application control using functions sc_main and sc_start..... | 19  |
| 4.3.5 Running under control of the kernel .....                                   | 20  |
| 4.4 Elaboration and simulation callbacks .....                                    | 20  |
| 4.4.1 before_end_of_elaboration .....                                             | 21  |
| 4.4.2 end_of_elaboration .....                                                    | 22  |
| 4.4.3 start_of_simulation .....                                                   | 23  |

|        |                                                                                                |    |
|--------|------------------------------------------------------------------------------------------------|----|
| 4.4.4  | end_of_simulation .....                                                                        | 23 |
| 4.5    | Other functions related to the scheduler .....                                                 | 24 |
| 4.5.1  | Function declarations .....                                                                    | 24 |
| 4.5.2  | Function sc_stop, sc_set_stop_mode, and sc_get_stop_mode .....                                 | 24 |
| 4.5.3  | Function sc_time_stamp .....                                                                   | 25 |
| 4.5.4  | Function sc_delta_count .....                                                                  | 26 |
| 4.5.5  | Function sc_is_running .....                                                                   | 26 |
| 5.     | Core language class definitions .....                                                          | 27 |
| 5.1    | Class header files .....                                                                       | 27 |
| 5.1.1  | #include "systemc" .....                                                                       | 27 |
| 5.1.2  | #include "systemc.h" .....                                                                     | 27 |
| 5.2    | sc_module .....                                                                                | 29 |
| 5.2.1  | Description .....                                                                              | 29 |
| 5.2.2  | Class definition .....                                                                         | 29 |
| 5.2.3  | Constraints on usage .....                                                                     | 31 |
| 5.2.4  | kind .....                                                                                     | 31 |
| 5.2.5  | SC_MODULE .....                                                                                | 31 |
| 5.2.6  | Constructors .....                                                                             | 32 |
| 5.2.7  | SC_CTOR .....                                                                                  | 32 |
| 5.2.8  | SC_HAS_PROCESS .....                                                                           | 33 |
| 5.2.9  | SC_METHOD, SC_THREAD, SC_CTHREAD .....                                                         | 33 |
| 5.2.10 | Method process .....                                                                           | 34 |
| 5.2.11 | Thread and clocked thread processes .....                                                      | 35 |
| 5.2.12 | Clocked thread processes and reset_signal_is .....                                             | 36 |
| 5.2.13 | sensitive .....                                                                                | 37 |
| 5.2.14 | dont_initialize .....                                                                          | 38 |
| 5.2.15 | set_stack_size .....                                                                           | 39 |
| 5.2.16 | next_trigger .....                                                                             | 39 |
| 5.2.17 | wait .....                                                                                     | 41 |
| 5.2.18 | Positional port binding .....                                                                  | 43 |
| 5.2.19 | before_end_of_elaboration, end_of_elaboration, start_of_simulation,<br>end_of_simulation ..... | 44 |
| 5.2.20 | get_child_objects .....                                                                        | 45 |
| 5.2.21 | sc_gen_unique_name .....                                                                       | 45 |
| 5.2.22 | sc_behavior and sc_channel .....                                                               | 46 |
| 5.3    | sc_module_name .....                                                                           | 47 |
| 5.3.1  | Description .....                                                                              | 47 |
| 5.3.2  | Class definition .....                                                                         | 47 |
| 5.3.3  | Constraints on usage .....                                                                     | 47 |
| 5.3.4  | Module hierarchy .....                                                                         | 48 |
| 5.3.5  | Member functions .....                                                                         | 48 |
| 5.4    | sc_sensitive .....                                                                             | 50 |
| 5.4.1  | Description .....                                                                              | 50 |
| 5.4.2  | Class definition .....                                                                         | 50 |
| 5.4.3  | Constraints on usage .....                                                                     | 50 |
| 5.4.4  | operator<< .....                                                                               | 50 |
| 5.5    | sc_spawn_options and sc_spawn .....                                                            | 52 |
| 5.5.1  | Description .....                                                                              | 52 |
| 5.5.2  | Class definition .....                                                                         | 52 |
| 5.5.3  | Constraints on usage .....                                                                     | 53 |
| 5.5.4  | Constructors .....                                                                             | 53 |
| 5.5.5  | Member functions .....                                                                         | 53 |

|        |                                                                                               |    |
|--------|-----------------------------------------------------------------------------------------------|----|
| 5.5.6  | sc_spawn .....                                                                                | 54 |
| 5.5.7  | SC_FORK and SC_JOIN .....                                                                     | 56 |
| 5.6    | sc_process_handle .....                                                                       | 58 |
| 5.6.1  | Description .....                                                                             | 58 |
| 5.6.2  | Class definition.....                                                                         | 58 |
| 5.6.3  | Constraints on usage .....                                                                    | 59 |
| 5.6.4  | Constructors .....                                                                            | 59 |
| 5.6.5  | Member functions .....                                                                        | 59 |
| 5.6.6  | sc_get_current_process_handle.....                                                            | 61 |
| 5.7    | sc_event_finder and sc_event_finder_t .....                                                   | 62 |
| 5.7.1  | Description .....                                                                             | 62 |
| 5.7.2  | Class definition.....                                                                         | 62 |
| 5.7.3  | Constraints on usage.....                                                                     | 62 |
| 5.8    | sc_event_and_list† and sc_event_or_list† .....                                                | 65 |
| 5.8.1  | Description .....                                                                             | 65 |
| 5.8.2  | Class definition.....                                                                         | 65 |
| 5.8.3  | Constraints on usage.....                                                                     | 65 |
| 5.8.4  | Event lists .....                                                                             | 65 |
| 5.9    | sc_event .....                                                                                | 66 |
| 5.9.1  | Description .....                                                                             | 66 |
| 5.9.2  | Class definition.....                                                                         | 66 |
| 5.9.3  | Constraints on usage.....                                                                     | 66 |
| 5.9.4  | notify and cancel .....                                                                       | 66 |
| 5.9.5  | Event lists .....                                                                             | 67 |
| 5.9.6  | Multiple event notifications .....                                                            | 67 |
| 5.10   | sc_time.....                                                                                  | 68 |
| 5.10.1 | Description .....                                                                             | 68 |
| 5.10.2 | Class definition.....                                                                         | 68 |
| 5.10.3 | Time resolution .....                                                                         | 69 |
| 5.10.4 | Functions and operators .....                                                                 | 69 |
| 5.10.5 | SC_ZERO_TIME.....                                                                             | 69 |
| 5.11   | sc_port.....                                                                                  | 71 |
| 5.11.1 | Description.....                                                                              | 71 |
| 5.11.2 | Class definition .....                                                                        | 71 |
| 5.11.3 | Template parameters .....                                                                     | 72 |
| 5.11.4 | Constraints on usage .....                                                                    | 73 |
| 5.11.5 | Constructors .....                                                                            | 74 |
| 5.11.6 | kind .....                                                                                    | 74 |
| 5.11.7 | Named port binding .....                                                                      | 74 |
| 5.11.8 | Member functions for bound ports and port-to-port binding.....                                | 75 |
| 5.11.9 | before_end_of_elaboration, end_of_elaboration, start_of_simulation,<br>end_of_simulation..... | 79 |
| 5.12   | sc_export.....                                                                                | 80 |
| 5.12.1 | Description .....                                                                             | 80 |
| 5.12.2 | Class definition.....                                                                         | 80 |
| 5.12.3 | Template parameters .....                                                                     | 81 |
| 5.12.4 | Constraints on usage.....                                                                     | 81 |
| 5.12.5 | Constructors .....                                                                            | 81 |
| 5.12.6 | kind.....                                                                                     | 81 |
| 5.12.7 | Export binding.....                                                                           | 82 |
| 5.12.8 | Member functions for bound exports and export-to-export binding.....                          | 83 |
| 5.12.9 | before_end_of_elaboration, end_of_elaboration, start_of_simulation,<br>end_of_simulation..... | 84 |

|        |                                                                                               |     |
|--------|-----------------------------------------------------------------------------------------------|-----|
| 5.13   | sc_interface .....                                                                            | 85  |
| 5.13.1 | Description .....                                                                             | 85  |
| 5.13.2 | Class definition.....                                                                         | 85  |
| 5.13.3 | Constraints on usage.....                                                                     | 85  |
| 5.13.4 | register_port .....                                                                           | 86  |
| 5.13.5 | default_event .....                                                                           | 86  |
| 5.14   | sc_prim_channel .....                                                                         | 88  |
| 5.14.1 | Description .....                                                                             | 88  |
| 5.14.2 | Class definition.....                                                                         | 88  |
| 5.14.3 | Constraints on usage .....                                                                    | 89  |
| 5.14.4 | Constructors .....                                                                            | 89  |
| 5.14.5 | kind.....                                                                                     | 89  |
| 5.14.6 | request_update and update .....                                                               | 89  |
| 5.14.7 | next_trigger and wait.....                                                                    | 90  |
| 5.14.8 | before_end_of_elaboration, end_of_elaboration, start_of_simulation,<br>end_of_simulation..... | 90  |
| 5.15   | sc_object .....                                                                               | 92  |
| 5.15.1 | Description .....                                                                             | 92  |
| 5.15.2 | Class definition.....                                                                         | 92  |
| 5.15.3 | Constraints on usage.....                                                                     | 93  |
| 5.15.4 | Constructors and hierarchical names.....                                                      | 93  |
| 5.15.5 | name, basename, and kind.....                                                                 | 94  |
| 5.15.6 | print and dump .....                                                                          | 95  |
| 5.15.7 | Functions for object hierarchy traversal.....                                                 | 95  |
| 5.15.8 | Member functions for attributes .....                                                         | 97  |
| 5.16   | sc_attr_base.....                                                                             | 99  |
| 5.16.1 | Description .....                                                                             | 99  |
| 5.16.2 | Class definition .....                                                                        | 99  |
| 5.16.3 | Member functions .....                                                                        | 99  |
| 5.17   | sc_attribute.....                                                                             | 100 |
| 5.17.1 | Description .....                                                                             | 100 |
| 5.17.2 | Class definition .....                                                                        | 100 |
| 5.17.3 | Template parameters.....                                                                      | 100 |
| 5.17.4 | Member functions and data members.....                                                        | 100 |
| 5.18   | sc_attr_cltn.....                                                                             | 101 |
| 5.18.1 | Description .....                                                                             | 101 |
| 5.18.2 | Class definition.....                                                                         | 101 |
| 5.18.3 | Constraints on usage.....                                                                     | 101 |
| 5.18.4 | Iterators.....                                                                                | 101 |
| 6.     | Predefined channel class definitions.....                                                     | 103 |
| 6.1    | sc_signal_in_if.....                                                                          | 103 |
| 6.1.1  | Description .....                                                                             | 103 |
| 6.1.2  | Class definition.....                                                                         | 103 |
| 6.1.3  | Member functions .....                                                                        | 103 |
| 6.2    | sc_signal_in_if<bool> and sc_signal_in_if<sc_dt::sc_logic>.....                               | 104 |
| 6.2.1  | Description .....                                                                             | 104 |
| 6.2.2  | Class definition.....                                                                         | 104 |
| 6.2.3  | Member functions .....                                                                        | 105 |
| 6.3    | sc_signal_inout_if.....                                                                       | 106 |
| 6.3.1  | Description .....                                                                             | 106 |
| 6.3.2  | Class definition.....                                                                         | 106 |
| 6.3.3  | write.....                                                                                    | 106 |

|        |                                                |     |
|--------|------------------------------------------------|-----|
| 6.4    | sc_signal                                      | 107 |
| 6.4.1  | Description                                    | 107 |
| 6.4.2  | Class definition                               | 107 |
| 6.4.3  | Template parameter T                           | 108 |
| 6.4.4  | Reading and writing signals                    | 108 |
| 6.4.5  | Constructors                                   | 109 |
| 6.4.6  | register_port                                  | 109 |
| 6.4.7  | Member functions for reading                   | 109 |
| 6.4.8  | Member functions for writing                   | 110 |
| 6.4.9  | Member functions for events                    | 110 |
| 6.4.10 | Diagnostic member functions                    | 110 |
| 6.4.11 | operator<<                                     | 111 |
| 6.5    | sc_signal<bool> and sc_signal<sc_dt::sc_logic> | 113 |
| 6.5.1  | Description                                    | 113 |
| 6.5.2  | Class definition                               | 113 |
| 6.5.3  | Member functions                               | 114 |
| 6.6    | sc_buffer                                      | 116 |
| 6.6.1  | Description                                    | 116 |
| 6.6.2  | Class definition                               | 116 |
| 6.6.3  | Constructors                                   | 116 |
| 6.6.4  | Member functions                               | 117 |
| 6.7    | sc_clock                                       | 119 |
| 6.7.1  | Description                                    | 119 |
| 6.7.2  | Class definition                               | 119 |
| 6.7.3  | Characteristic properties                      | 120 |
| 6.7.4  | Constructors                                   | 120 |
| 6.7.5  | write                                          | 120 |
| 6.7.6  | Diagnostic member functions                    | 120 |
| 6.7.7  | before_end_of_elaboration                      | 121 |
| 6.7.8  | sc_in_clk                                      | 121 |
| 6.8    | sc_in                                          | 122 |
| 6.8.1  | Description                                    | 122 |
| 6.8.2  | Class definition                               | 122 |
| 6.8.3  | Member functions                               | 123 |
| 6.8.4  | Function sc_trace                              | 123 |
| 6.8.5  | end_of_elaboration                             | 123 |
| 6.9    | sc_in<bool> and sc_in<sc_dt::sc_logic>         | 124 |
| 6.9.1  | Description                                    | 124 |
| 6.9.2  | Class definition                               | 124 |
| 6.9.3  | Member functions                               | 126 |
| 6.10   | sc_inout                                       | 127 |
| 6.10.1 | Description                                    | 127 |
| 6.10.2 | Class definition                               | 127 |
| 6.10.3 | Member functions                               | 128 |
| 6.10.4 | initialize                                     | 128 |
| 6.10.5 | Function sc_trace                              | 128 |
| 6.10.6 | end_of_elaboration                             | 129 |
| 6.10.7 | Binding                                        | 129 |
| 6.11   | sc_inout<bool> and sc_inout<sc_dt::sc_logic>   | 130 |
| 6.11.1 | Description                                    | 130 |
| 6.11.2 | Class definition                               | 130 |
| 6.11.3 | Member functions                               | 132 |
| 6.12   | sc_out                                         | 133 |
| 6.12.1 | Description                                    | 133 |



|        |                                      |     |
|--------|--------------------------------------|-----|
| 6.12.2 | Class definition.....                | 133 |
| 6.12.3 | Member functions .....               | 133 |
| 6.13   | sc_signal_resolved .....             | 134 |
| 6.13.1 | Description .....                    | 134 |
| 6.13.2 | Class definition .....               | 134 |
| 6.13.3 | Constructors .....                   | 134 |
| 6.13.4 | Resolution semantics .....           | 135 |
| 6.13.5 | Member functions .....               | 136 |
| 6.14   | sc_in_resolved .....                 | 137 |
| 6.14.1 | Description .....                    | 137 |
| 6.14.2 | Class definition .....               | 137 |
| 6.14.3 | Member functions .....               | 137 |
| 6.15   | sc_inout_resolved .....              | 138 |
| 6.15.1 | Description .....                    | 138 |
| 6.15.2 | Class definition.....                | 138 |
| 6.15.3 | Member functions .....               | 138 |
| 6.16   | sc_out_resolved .....                | 139 |
| 6.16.1 | Description .....                    | 139 |
| 6.16.2 | Class definition.....                | 139 |
| 6.16.3 | Member functions.....                | 139 |
| 6.17   | sc_signal_rv .....                   | 140 |
| 6.17.1 | Description.....                     | 140 |
| 6.17.2 | Class definition .....               | 140 |
| 6.17.3 | Semantics and member functions ..... | 140 |
| 6.18   | sc_in_rv.....                        | 142 |
| 6.18.1 | Description.....                     | 142 |
| 6.18.2 | Class definition .....               | 142 |
| 6.18.3 | Member functions .....               | 142 |
| 6.19   | sc_inout_rv.....                     | 143 |
| 6.19.1 | Description .....                    | 143 |
| 6.19.2 | Class definition.....                | 143 |
| 6.19.3 | Member functions .....               | 143 |
| 6.20   | sc_out_rv.....                       | 145 |
| 6.20.1 | Description .....                    | 145 |
| 6.20.2 | Class definition.....                | 145 |
| 6.20.3 | Member functions .....               | 145 |
| 6.21   | sc_fifo_in_if.....                   | 146 |
| 6.21.1 | Description .....                    | 146 |
| 6.21.2 | Class definition.....                | 146 |
| 6.21.3 | Member functions .....               | 146 |
| 6.22   | sc_fifo_out_if.....                  | 148 |
| 6.22.1 | Description.....                     | 148 |
| 6.22.2 | Class definition .....               | 148 |
| 6.22.3 | Member functions .....               | 148 |
| 6.23   | sc_fifo .....                        | 150 |
| 6.23.1 | Description .....                    | 150 |
| 6.23.2 | Class definition.....                | 150 |
| 6.23.3 | Template parameter T .....           | 151 |
| 6.23.4 | Constructors .....                   | 151 |
| 6.23.5 | register_port .....                  | 152 |
| 6.23.6 | Member functions for reading.....    | 152 |
| 6.23.7 | Member functions for writing .....   | 152 |
| 6.23.8 | The update phase.....                | 153 |
| 6.23.9 | Member functions for events.....     | 153 |

|         |                                                                     |     |
|---------|---------------------------------------------------------------------|-----|
| 6.23.10 | Member functions for available values and free slots.....           | 153 |
| 6.23.11 | Diagnostic member functions.....                                    | 154 |
| 6.23.12 | operator<<.....                                                     | 154 |
| 6.24    | sc_fifo_in.....                                                     | 156 |
| 6.24.1  | Description.....                                                    | 156 |
| 6.24.2  | Class definition.....                                               | 156 |
| 6.24.3  | Member functions.....                                               | 156 |
| 6.25    | sc_fifo_out.....                                                    | 157 |
| 6.25.1  | Description.....                                                    | 157 |
| 6.25.2  | Class definition.....                                               | 157 |
| 6.25.3  | Member functions.....                                               | 157 |
| 6.26    | sc_mutex_if.....                                                    | 160 |
| 6.26.1  | Description.....                                                    | 160 |
| 6.26.2  | Class definition.....                                               | 160 |
| 6.26.3  | Member functions.....                                               | 160 |
| 6.27    | sc_mutex.....                                                       | 161 |
| 6.27.1  | Description.....                                                    | 161 |
| 6.27.2  | Class definition.....                                               | 161 |
| 6.27.3  | Constructors.....                                                   | 161 |
| 6.27.4  | Member functions.....                                               | 162 |
| 6.28    | sc_semaphore_if.....                                                | 163 |
| 6.28.1  | Description.....                                                    | 163 |
| 6.28.2  | Class definition.....                                               | 163 |
| 6.28.3  | Member functions.....                                               | 163 |
| 6.29    | sc_semaphore.....                                                   | 164 |
| 6.29.1  | Description.....                                                    | 164 |
| 6.29.2  | Class definition.....                                               | 164 |
| 6.29.3  | Constructors.....                                                   | 164 |
| 6.29.4  | Member functions.....                                               | 165 |
| 6.30    | sc_event_queue.....                                                 | 166 |
| 6.30.1  | Description.....                                                    | 166 |
| 6.30.2  | Class definition.....                                               | 166 |
| 6.30.3  | Constraints on usage.....                                           | 166 |
| 6.30.4  | Constructors.....                                                   | 166 |
| 6.30.5  | kind.....                                                           | 167 |
| 6.30.6  | Member functions.....                                               | 167 |
| 7.      | Data types.....                                                     | 169 |
| 7.1     | Introduction.....                                                   | 169 |
| 7.2     | Common characteristics.....                                         | 172 |
| 7.2.1   | Initialization and assignment operators.....                        | 172 |
| 7.2.2   | Precision of arithmetic expressions.....                            | 173 |
| 7.2.3   | Base class default word length.....                                 | 174 |
| 7.2.4   | Word length.....                                                    | 175 |
| 7.2.5   | Bit-select.....                                                     | 175 |
| 7.2.6   | Part-select.....                                                    | 175 |
| 7.2.7   | Concatenation.....                                                  | 176 |
| 7.2.8   | Reduction operators.....                                            | 178 |
| 7.2.9   | Integer conversion.....                                             | 178 |
| 7.2.10  | String input and output.....                                        | 178 |
| 7.2.11  | Conversion of application-defined types in integer expressions..... | 179 |
| 7.3     | String literals.....                                                | 180 |
| 7.4     | sc_value_base†.....                                                 | 182 |

|         |                                                   |     |
|---------|---------------------------------------------------|-----|
| 7.4.1   | Description .....                                 | 182 |
| 7.5     | Limited-precision integer types .....             | 184 |
| 7.5.1   | Type definitions .....                            | 184 |
| 7.5.2   | sc_int_base .....                                 | 185 |
| 7.5.3   | sc_uint_base .....                                | 190 |
| 7.5.4   | sc_int .....                                      | 194 |
| 7.5.5   | sc_uint .....                                     | 197 |
| 7.5.6   | Bit-selects .....                                 | 199 |
| 7.5.7   | Part-selects .....                                | 203 |
| 7.6     | Finite-precision integer types .....              | 209 |
| 7.6.1   | Type definitions .....                            | 209 |
| 7.6.2   | Constraints on usage .....                        | 209 |
| 7.6.3   | sc_signed .....                                   | 209 |
| 7.6.4   | sc_unsigned .....                                 | 216 |
| 7.6.5   | sc_bigint .....                                   | 223 |
| 7.6.6   | sc_biguint .....                                  | 225 |
| 7.6.7   | Bit-selects .....                                 | 227 |
| 7.6.8   | Part-selects .....                                | 231 |
| 7.7     | Integer concatenations .....                      | 236 |
| 7.7.1   | Description .....                                 | 236 |
| 7.7.2   | Class definition .....                            | 236 |
| 7.7.3   | Constraints on usage .....                        | 237 |
| 7.7.4   | Assignment operators .....                        | 238 |
| 7.7.5   | Implicit type conversion .....                    | 238 |
| 7.7.6   | Explicit type conversion .....                    | 238 |
| 7.7.7   | Other member functions .....                      | 238 |
| 7.8     | Generic base proxy class .....                    | 239 |
| 7.8.1   | Description .....                                 | 239 |
| 7.8.2   | Class definition .....                            | 239 |
| 7.8.3   | Constraints on usage .....                        | 239 |
| 7.9     | Logic and vector types .....                      | 240 |
| 7.9.1   | Type definitions .....                            | 240 |
| 7.9.2   | sc_logic .....                                    | 240 |
| 7.9.3   | sc_bv_base .....                                  | 244 |
| 7.9.4   | sc_lv_base .....                                  | 251 |
| 7.9.5   | sc_bv .....                                       | 257 |
| 7.9.6   | sc_lv .....                                       | 259 |
| 7.9.7   | Bit-selects .....                                 | 261 |
| 7.9.8   | Part-selects .....                                | 264 |
| 7.9.9   | Concatenations .....                              | 270 |
| 7.10    | Fixed-point types .....                           | 278 |
| 7.10.1  | Fixed-point representation .....                  | 278 |
| 7.10.2  | Fixed-point type conversion .....                 | 279 |
| 7.10.3  | Fixed-point data types .....                      | 279 |
| 7.10.4  | Fixed-point expressions and operations .....      | 281 |
| 7.10.5  | Bit and part selection .....                      | 284 |
| 7.10.6  | Variable-precision fixed-point value limits ..... | 284 |
| 7.10.7  | Fixed-point word length and mode .....            | 284 |
| 7.10.8  | Conversions to character string .....             | 287 |
| 7.10.9  | Finite word-length effects .....                  | 289 |
| 7.10.10 | sc_fxnum .....                                    | 313 |
| 7.10.11 | sc_fxnum_fast .....                               | 317 |
| 7.10.12 | sc_fxval .....                                    | 322 |
| 7.10.13 | sc_fxval_fast .....                               | 326 |

|         |                                                  |     |
|---------|--------------------------------------------------|-----|
| 7.10.14 | sc_fix .....                                     | 331 |
| 7.10.15 | sc_ufix .....                                    | 334 |
| 7.10.16 | sc_fix_fast.....                                 | 337 |
| 7.10.17 | sc_ufix_fast.....                                | 340 |
| 7.10.18 | sc_fixed.....                                    | 343 |
| 7.10.19 | sc_ufixed.....                                   | 345 |
| 7.10.20 | sc_fixed_fast.....                               | 347 |
| 7.10.21 | sc_ufixed_fast.....                              | 350 |
| 7.10.22 | Bit-selects .....                                | 352 |
| 7.10.23 | Part-selects.....                                | 354 |
| 7.11    | Contexts .....                                   | 361 |
| 7.11.1  | sc_length_param.....                             | 361 |
| 7.11.2  | sc_length_context.....                           | 362 |
| 7.11.3  | sc_fxtype_params.....                            | 363 |
| 7.11.4  | sc_fxtype_context.....                           | 366 |
| 7.11.5  | sc_fxcast_switch.....                            | 367 |
| 7.11.6  | sc_fxcast_context .....                          | 368 |
| 7.12    | Control of string representation .....           | 370 |
| 7.12.1  | Description .....                                | 370 |
| 7.12.2  | Class definition .....                           | 370 |
| 7.12.3  | Functions.....                                   | 370 |
| 8.      | Utility class definitions .....                  | 371 |
| 8.1     | Trace files .....                                | 371 |
| 8.1.1   | Class definition and function declarations ..... | 371 |
| 8.1.2   | sc_trace_file .....                              | 371 |
| 8.1.3   | sc_create_vcd_trace_file .....                   | 372 |
| 8.1.4   | sc_close_vcd_trace_file .....                    | 372 |
| 8.1.5   | sc_write_comment .....                           | 372 |
| 8.1.6   | sc_trace.....                                    | 372 |
| 8.2     | sc_report.....                                   | 375 |
| 8.2.1   | Description.....                                 | 375 |
| 8.2.2   | Class definition .....                           | 375 |
| 8.2.3   | Constraints on usage .....                       | 375 |
| 8.2.4   | sc_severity.....                                 | 376 |
| 8.2.5   | Copy constructor and assignment .....            | 376 |
| 8.2.6   | Member functions .....                           | 376 |
| 8.3     | sc_report_handler.....                           | 378 |
| 8.3.1   | Description .....                                | 378 |
| 8.3.2   | Class definition.....                            | 378 |
| 8.3.3   | Constraints on usage.....                        | 380 |
| 8.3.4   | sc_actions .....                                 | 380 |
| 8.3.5   | report .....                                     | 380 |
| 8.3.6   | set_actions .....                                | 381 |
| 8.3.7   | stop_after.....                                  | 381 |
| 8.3.8   | get_count.....                                   | 382 |
| 8.3.9   | suppress and force .....                         | 383 |
| 8.3.10  | set_handler .....                                | 383 |
| 8.3.11  | get_new_action_id .....                          | 384 |
| 8.3.12  | sc_interrupt_here and sc_stop_here .....         | 384 |
| 8.3.13  | get_cached_report and clear_cached_report .....  | 384 |
| 8.3.14  | set_log_file_name and get_log_file_name.....     | 385 |
| 8.4     | sc_exception.....                                | 386 |

|                                                                            |                             |     |
|----------------------------------------------------------------------------|-----------------------------|-----|
| 8.4.1                                                                      | Description .....           | 386 |
| 8.4.2                                                                      | Definition .....            | 386 |
| 8.5                                                                        | Utility functions .....     | 387 |
| 8.5.1                                                                      | Function declarations ..... | 387 |
| 8.5.2                                                                      | sc_abs .....                | 387 |
| 8.5.3                                                                      | sc_max .....                | 387 |
| 8.5.4                                                                      | sc_min .....                | 387 |
| 8.5.5                                                                      | sc_copyright .....          | 387 |
| 8.5.6                                                                      | sc_version .....            | 388 |
| 8.5.7                                                                      | sc_release .....            | 388 |
| Annex A (informative) Introduction to SystemC .....                        |                             | 389 |
| Annex B (informative) Glossary .....                                       |                             | 393 |
| Annex C (informative) Deprecated features .....                            |                             | 403 |
| Annex D (informative) Changes between the different SystemC versions ..... |                             | 405 |
| Annex E (informative) List of Participants .....                           |                             | 407 |
| Index .....                                                                |                             | 409 |

## INTERNATIONAL ELECTROTECHNICAL COMMISSION

## BEHAVIOURAL LANGUAGES –

Part 7: SystemC<sup>®</sup> Language Reference Manual

## FOREWORD

- 1) The International Electrotechnical Commission (IEC) is a worldwide organization for standardization comprising all national electrotechnical committees (IEC National Committees). The object of IEC is to promote international co-operation on all questions concerning standardization in the electrical and electronic fields. To this end and in addition to other activities, IEC publishes International Standards, Technical Specifications, Technical Reports, Publicly Available Specifications (PAS) and Guides (hereafter referred to as "IEC Publication(s)"). Their preparation is entrusted to technical committees; any IEC National Committee interested in the subject dealt with may participate in this preparatory work. International, governmental and non-governmental organizations liaising with the IEC also participate in this preparation. IEC collaborates closely with the International Organization for Standardization (ISO) in accordance with conditions determined by agreement between the two organizations.
- 2) The formal decisions or agreements of IEC on technical matters express, as nearly as possible, an international consensus of opinion on the relevant subjects since each technical committee has representation from all interested IEC National Committees.
- 3) IEC Publications have the form of recommendations for international use and are accepted by IEC National Committees in that sense. While all reasonable efforts are made to ensure that the technical content of IEC Publications is accurate, IEC cannot be held responsible for the way in which they are used or for any misinterpretation by any end user.
- 4) In order to promote international uniformity, IEC National Committees undertake to apply IEC Publications transparently to the maximum extent possible in their national and regional publications. Any divergence between any IEC Publication and the corresponding national or regional publication shall be clearly indicated in the latter.
- 5) IEC itself does not provide any attestation of conformity. Independent certification bodies provide conformity assessment services and, in some areas, access to IEC marks of conformity. IEC is not responsible for any services carried out by independent certification bodies.
- 6) Attention is drawn to the possibility that some of the elements of this IEC Publication may be the subject of patent rights. IEC shall not be held responsible for identifying any or all such patent rights.

International Standard IEC 61691-7/IEEE Std 1666 has been processed through IEC technical committee 93: *Design automation*.

The text of this standard is based on the following documents:

| IEEE Std    | FDIS        | Report on voting |
|-------------|-------------|------------------|
| 1666 (2005) | 93/279/FDIS | 93/285/RVD       |

Full information on the voting for the approval of this standard can be found in the report on voting indicated in the above table.

A list of parts of the IEC 61691 series can be found on the IEC web site.

The committee has decided that the contents of this publication will remain unchanged until the maintenance result date indicated on the IEC web site under "<http://webstore.iec.ch>" in the data related to the specific publication. At this date, the publication will be

- reconfirmed,
- withdrawn,
- replaced by a revised edition, or
- amended.

## IEC/IEEE Dual Logo International Standards

This Dual Logo International Standard is the result of an agreement between the IEC and the Institute of Electrical and Electronics Engineers, Inc. (IEEE). The original IEEE Standard was submitted to the IEC for consideration under the agreement, and the resulting IEC/IEEE Dual Logo International Standard has been published in accordance with the ISO/IEC Directives.

IEEE Standards documents are developed within the IEEE Societies and the Standards Coordinating Committees of the IEEE Standards Association (IEEE-SA) Standards Board. The IEEE develops its standards through a consensus development process, approved by the American National Standards Institute, which brings together volunteers representing varied viewpoints and interests to achieve the final product. Volunteers are not necessarily members of the Institute and serve without compensation. While the IEEE administers the process and establishes rules to promote fairness in the consensus development process, the IEEE does not independently evaluate, test, or verify the accuracy of any of the information contained in its standards.

Use of an IEC/IEEE Dual Logo International Standard is wholly voluntary. The IEC and IEEE disclaim liability for any personal injury, property or other damage, of any nature whatsoever, whether special, indirect, consequential, or compensatory, directly or indirectly resulting from the publication, use of, or reliance upon this, or any other IEC or IEEE Standard document.

The IEC and IEEE do not warrant or represent the accuracy or content of the material contained herein, and expressly disclaim any express or implied warranty, including any implied warranty of merchantability or fitness for a specific purpose, or that the use of the material contained herein is free from patent infringement. IEC/IEEE Dual Logo International Standards documents are supplied "AS IS".

The existence of an IEC/IEEE Dual Logo International Standard does not imply that there are no other ways to produce, test, measure, purchase, market, or provide other goods and services related to the scope of the IEC/IEEE Dual Logo International Standard. Furthermore, the viewpoint expressed at the time a standard is approved and issued is subject to change brought about through developments in the state of the art and comments received from users of the standard.

Every IEEE Standard is subjected to review at least every five years for revision or reaffirmation. When a document is more than five years old and has not been reaffirmed, it is reasonable to conclude that its contents, although still of some value, do not wholly reflect the present state of the art. Users are cautioned to check to determine that they have the latest edition of any IEEE Standard.

In publishing and making this document available, the IEC and IEEE are not suggesting or rendering professional or other services for, or on behalf of, any person or entity. Neither the IEC nor IEEE is undertaking to perform any duty owed by any other person or entity to another. Any person utilizing this, and any other IEC/IEEE Dual Logo International Standards or IEEE Standards document, should rely upon the advice of a competent professional in determining the exercise of reasonable care in any given circumstances.

Interpretations – Occasionally questions may arise regarding the meaning of portions of standards as they relate to specific applications. When the need for interpretations is brought to the attention of IEEE, the Institute will initiate action to prepare appropriate responses. Since IEEE Standards represent a consensus of concerned interests, it is important to ensure that any interpretation has also received the concurrence of a balance of interests. For this reason, IEEE and the members of its societies and Standards Coordinating Committees are not able to provide an instant response to interpretation requests except in those cases where the matter has previously received formal consideration.

Comments for revision of IEC/IEEE Dual Logo International Standards are welcome from any interested party, regardless of membership affiliation with the IEC or IEEE. Suggestions for changes in documents should be in the form of a proposed change of text, together with appropriate supporting comments. Comments on standards and requests for interpretations should be addressed to:

Secretary, IEEE-SA Standards Board, 445 Hoes Lane, Piscataway, NJ 08854, USA and/or General Secretary, IEC, 3, rue de Varembé, PO Box 131, 1211 Geneva 20, Switzerland.

Authorization to photocopy portions of any individual standard for internal or personal use is granted by the Institute of Electrical and Electronics Engineers, Inc., provided that the appropriate fee is paid to Copyright Clearance Center. To arrange for payment of licensing fee, please contact Copyright Clearance Center, Customer Service, 222 Rosewood Drive, Danvers, MA 01923 USA; +1 978 750 8400. Permission to photocopy portions of any individual standard for educational classroom use can also be obtained through the Copyright Clearance Center.

NOTE – Attention is called to the possibility that implementation of this standard may require use of subject matter covered by patent rights. By publication of this standard, no position is taken with respect to the existence or validity of any patent rights in connection therewith. The IEEE shall not be responsible for identifying patents for which a license may be required by an IEEE standard or for conducting inquiries into the legal validity or scope of those patents that are brought to its attention.





# IEEE Standard SystemC<sup>®</sup> Language Reference Manual

Sponsor

**Design Automation Standards Committee**  
of the  
**IEEE Computer Society**

Approved 28 March 2006

**American National Standards Institute**

Approved 6 December 2005

**IEEE-SA Standards Board**

Grateful acknowledgment is made to Open SystemC Initiative for the permission to use the following source material:

SystemC<sup>®</sup> Language Reference Manual Version 2.1

**Abstract:** SystemC<sup>®1</sup> is defined in this standard. SystemC is an ANSI standard C++ class library for system and hardware design for use by designers and architects who need to address complex systems that are a hybrid between hardware and software. This standard provides a precise and complete definition of the SystemC class library so that a SystemC implementation can be developed with reference to this standard alone. The primary audiences for this standard are the implementors of the SystemC class library, the implementors of tools supporting the class library, and users of the class library.

**Keywords:** C++, computer languages, digital systems, discrete event simulation, electronic design automation, electronic systems, electronic system level, embedded software, fixed-point, hardware description language, hardware design, hardware verification, SystemC, system modeling, system-on-chip, transaction level

---

<sup>1</sup>SystemC<sup>®</sup> is a registered trademark of Open SystemC Initiative.

## IEEE introduction

This document defines SystemC, which is a C++ class library.

As the electronics industry builds more complex systems involving large numbers of components including software, there is an increasing need for a modeling language that can manage the complexity and size of these systems. SystemC provides a mechanism for managing this complexity with its facility for modeling hardware and software together at multiple levels of abstraction. This capability is not available in traditional hardware description languages.

Stakeholders in SystemC include Electronic Design Automation (EDA) companies who implement SystemC class libraries and tools, Integrated Circuit (IC) suppliers who extend those class libraries and use SystemC to model their intellectual property, and end users who use SystemC to model their systems.

Before the publication of this standard, SystemC was defined by an open source proof-of-concept C++ library, also known as *the reference simulator*, available from the Open SystemC Initiative (OSCI). In the event of discrepancies between the behavior of the reference simulator and statements made in this standard, this standard shall be taken to be definitive.

This standard is not intended to serve as a users' guide or to provide an introduction to SystemC. Readers requiring a SystemC tutorial or information on the intended use of SystemC should consult the OSCI Web site ([www.systemc.org](http://www.systemc.org)) to locate the many books and training classes available.

## Notice to users

### Errata

Errata, if any, for this and all other standards can be accessed at the following URL: <http://standards.ieee.org/reading/ieee/updates/errata/index.html>. Users are encouraged to check this URL for errata periodically.

### Interpretations

Current interpretations can be accessed at the following URL: <http://standards.ieee.org/reading/ieee/interp/index.html>.

### Patents

Attention is called to the possibility that implementation of this standard may require use of subject matter covered by patent rights. By publication of this standard, no position is taken with respect to the existence or validity of any patent rights in connection therewith. The IEEE is not responsible for identifying Essential Patent Claims for which a license may be required, for conducting inquiries into the legal validity or scope of Patents Claims or determining whether any licensing terms or conditions provided in connection with submission of a Letter of Assurance, if any, or in any licensing agreements are reasonable or non-discriminatory. Users of this standard are expressly advised that determination of the validity of any patent rights, and the risk of infringement of such rights, is entirely their own responsibility. Further information may be obtained from the IEEE Standards Association.

***IMPORTANT NOTICE: This standard is not intended to ensure safety, security, health, or environmental protection in all circumstances. Implementers of the standard are responsible for determining appropriate safety, security, environmental, and health practices or regulatory requirements.***

***This IEEE document is made available for use subject to important notices and legal disclaimers. These notices and disclaimers appear in all publications containing this document and may be found under the heading "Important Notice" or "Important Notices and Disclaimers Concerning IEEE Documents." They can also be obtained on request from IEEE or viewed at <http://standards.ieee.org/TPR/disclaimers.html>.***



# BEHAVIOURAL LANGUAGES –

## Part 7: SystemC<sup>®</sup> Language Reference Manual

### 1. Overview

#### 1.1 Scope

This standard defines SystemC<sup>®</sup><sup>1</sup> as an ANSI standard C++ class library for system and hardware design.

#### 1.2 Purpose

The general purpose of SystemC is to provide a C++-based standard for designers and architects who need to address complex systems that are a hybrid between hardware and software.

The specific purpose of this standard is to provide a precise and complete definition of the SystemC class library so that a SystemC implementation can be developed with reference to this standard alone. This standard is not intended to serve as a users' guide or to provide an introduction to SystemC, but does contain useful information for end users.

#### 1.3 Subsets

It is anticipated that tool vendors will create implementations that support only a subset of this standard or that impose further constraints on the use of this standard. Such implementations are not fully compliant with this standard but may nevertheless claim partial compliance with this standard and may use the name SystemC.

#### 1.4 Relationship with C++

This standard is closely related to the C++ programming language and adheres to the terminology used in ISO/IEC 14882:2003. This standard does not seek to restrict the usage of the C++ programming language; a SystemC application may use any of the facilities provided by C++, which in turn may use any of the facilities provided by C. However, where the facilities provided by this standard are used, they shall be used in accordance with the rules and constraints set out in this standard.

This standard defines the public interface to the SystemC class library and the constraints on how those classes may be used. The SystemC class library may be implemented in any manner whatsoever, provided only that the obligations imposed by this standard are honored.

A C++ class library may be extended using the mechanisms provided by the C++ language. Implementors and users are free to extend SystemC in this way, provided that they do not violate this standard.

---

<sup>1</sup>SystemC<sup>®</sup> is a registered trademark of Open SystemC Initiative.

NOTE—It is possible to create a well-formed C++ program that is legal according to the C++ programming language standard but that violates this standard. An implementation is not obliged to detect every violation of this standard.<sup>2</sup>

## 1.5 Guidance for readers

Readers who are not entirely familiar with SystemC should start with Annex A, “Introduction to SystemC,” which provides a brief informal summary of the subject intended to aid in the understanding of the normative definitions. Such readers may also find it helpful to scan the examples embedded in the normative definitions and to see Annex B, “Glossary.”

Readers should pay close attention to Clause 3, “Terminology and conventions used in this standard.” An understanding of the terminology defined in Clause 3 is necessary for a precise interpretation of this standard.

Clause 4, “Elaboration and simulation semantics,” defines the behavior of the SystemC kernel and is central to an understanding of SystemC. The semantic definitions given in the subsequent clauses detailing the individual classes are built upon the foundations laid in Clause 4.

The clauses from Clause 5 onward define the public interface to the SystemC class library. The following information is listed for each class:

- a) A C++ source code listing of the class definition
- b) A statement of any constraints on the use of the class and its members
- c) A statement of the semantics of the class and its members
- d) For certain classes, a description of functions, typedefs, and macros associated with the class.
- e) Informative examples illustrating both typical and atypical uses of the class

Readers should bear in mind that the primary obligation of a tool vendor is to implement the abstract semantics defined in Clause 4, using the framework and constraints provided by the class definitions starting in Clause 5.

Annex A is intended to aid the reader in the understanding of the structure and intent of the SystemC class library.

Annex B is a glossary giving informal descriptions of the terms used in this standard.

Annex C lists the deprecated features, that is, features that were present in version 2.0.1 of the Open SystemC Initiative (OSCI) open source proof-of-concept SystemC implementation but are not part of this standard.

Annex D lists the changes between SystemC version 2.0.1 and version 2.1 Beta Oct 12 2004, and the changes between SystemC 2.1 Beta Oct 12 2004 and this standard.

---

<sup>2</sup>Notes in text, tables, and figures are given for information only, and do not contain requirements needed to implement the standard.

## 2. References

The following documents are indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the document (including any amendments or corrigenda) applies.

This standard shall be used in conjunction with the following publications:

ISO/IEC 14882:2003, Programming Languages—C++.<sup>3</sup>

IEC 61691-4:2004, Behavioural languages - Part 4: Verilog® hardware description language | IEEE Std 1364™-2001, IEEE Standard Verilog® Hardware Description Language.<sup>4, 5, 6</sup>

---

<sup>3</sup>IEC publications are available from the Sales Department of The International Electrotechnical Commission, Case Postale 131, 3, rue de Varembe, CH-1211, Genève 20, Switzerland/Suisse (<http://www.iec.ch>). ISO publications are available from the ISO Central Secretariat, 1 chemin de la Voie-Creuse, CP 56, CH-1211, Genève 20, Switzerland/Suisse (<http://www.iso.ch>). ISO/IEC publications are also available in the United States from the Sales Department, American National Standards Institute, 25 West 43rd Street, 4th Floor, New York, NY 10036, USA (<http://www.ansi.org/>).

<sup>4</sup>IEEE publications are available from the Institute of Electrical and Electronics Engineers, 445 Hoes Lane, P.O. Box 1331, Piscataway, NJ 08855-1331, USA (<http://standards.ieee.org/>).

<sup>5</sup>The IEEE standards or products referred to in this clause are trademarks of the Institute of Electrical and Electronics Engineers, Inc.

<sup>6</sup>IEEE Std 1364-2001 was adopted as IEC 61691-4:2004

### 3. Terminology and conventions used in this standard

#### 3.1 Terminology

##### 3.1.1 Shall, should, may, can

The word *shall* is used to indicate a mandatory requirement.

The word *should* is used to recommend a particular course of action, but does not impose any obligation.

The word *may* is used to mean shall be permitted (in the sense of being legally allowed).

The word *can* is used to mean shall be able to (in the sense of being technically possible).

In some cases, word usage is qualified to indicate on whom the obligation falls, such as *an application may* or *an implementation shall*.

##### 3.1.2 Implementation, application

The word *implementation* is used to mean any specific implementation of the full SystemC class library as defined in this standard, only the public interface of which need be exposed to the application.

The word *application* is used to mean a C++ program, written by an end user, that uses the SystemC class library, that is, uses classes, functions, or macros defined in this standard.

##### 3.1.3 Call, called from, derived from

The term *call* is taken to mean call directly or indirectly. Call indirectly means call an intermediate function which in turn calls the function in question, where the chain of function calls may be extended indefinitely.

Similarly, *called from* means called from directly or indirectly.

Except where explicitly qualified, the term *derived from* is taken to mean derived directly or indirectly from. Derived indirectly from means derived from one or more intermediate base classes.

##### 3.1.4 Specific technical terms

The following terms are sometimes used to refer to classes and sometimes used to refer to objects of those classes. When the distinction is important, the usage of the term may be qualified. For example, a *port instance* is an object of a class derived from the class **sc\_port**, whereas a *port class* is a class derived from class **sc\_port**.

A *module* is a class derived from the class **sc\_module**.

A *port* is either a class derived from the class **sc\_port** or an object of class **sc\_port**.

An *export* is an object of class **sc\_export**.

An *interface* is a class derived from the class **sc\_interface**.

An *interface proper* is an abstract class derived from the class **sc\_interface** but not derived from the class **sc\_object**.



A *primitive channel* is a non-abstract class derived from one or more interfaces and also derived from the class **sc\_prim\_channel**.

A *hierarchical channel* is a non-abstract class derived from one or more interfaces and also derived from the class **sc\_module**.

A *channel* is a non-abstract class derived from one or more interfaces. A channel may be a primitive channel or a hierarchical channel. If not, it is strongly recommended that a channel be derived from the class **sc\_object**.

An *event* is an object of the class **sc\_event**.

A *signal* is an object of the class **sc\_signal**.

A *process instance* is an object of an implementation-defined class derived from the class **sc\_object** and created by one of the three macros SC\_METHOD, SC\_THREAD, or SC\_CTHREAD or by calling the function **sc\_spawn**.

The term *process* refers to either a process instance or to the member function that is associated with a process instance when it is created. The meaning is made clear by the context.

A *static process* is a process created during the construction of the module hierarchy or from the **before\_end\_of\_elaboration** callback.

A *dynamic process* is a process created from the **end\_of\_elaboration** callback or during simulation.

An *unspawned process* is a process created by invoking one of the three macros SC\_METHOD, SC\_THREAD, or SC\_CTHREAD. An unspawned process is typically a static process, but would be a dynamic process if invoked from the **end\_of\_elaboration** callback.

A *spawned process* is a process created by calling the function **sc\_spawn**. A spawned process is typically a dynamic process, but would be a static process if **sc\_spawn** is called before the end of elaboration.

A *process handle* is an object of the class **sc\_process\_handle**.

The *module hierarchy* is the total set of module instances constructed during elaboration. The term is sometimes used to include all of the objects instantiated within those modules during elaboration. The module hierarchy is a subset of the *object hierarchy*.

The *object hierarchy* is the total set of objects of class **sc\_object**. Part of the object hierarchy is constructed during elaboration (the *module hierarchy*) and includes module, port, primitive channel, and static process instances. Part is constructed dynamically and destroyed dynamically during simulation and includes dynamic process instances (see 5.15).

A given instance is *within* module M if the constructor of the instance is called (explicitly or implicitly) from the constructor of module M and if the instance is not within another module instance that is itself within module M.

A given module is said to *contain* a given instance if the instance is within that module.

A *child* of a given module is an instance that is within that module.

A *parent* of a given instance is a module having that instance as a child.

A *top-level module* is a module that is not instantiated within any other module.

The concepts of *elaboration* and *simulation* are defined in Clause 4. The terms *during elaboration* and *during simulation* indicate that an action may happen at that time. The implementation makes a number of callbacks to the application during elaboration and simulation. Whether a particular action is allowed within a particular callback cannot be inferred from the terms *during elaboration* and *during simulation* alone but is defined in detail in 4.4. For example, a number of actions that are permitted *during elaboration* are explicitly forbidden during the **end\_of\_elaboration** callback.

## 3.2 Syntactical conventions

### 3.2.1 Implementation-defined

The italicized term *implementation-defined* is used where part of a C++ definition is omitted from this standard. In such cases, an implementation shall provide an appropriate definition that honors the semantics defined in this standard.

### 3.2.2 Disabled

The italicized term *disabled* is used within a C++ class definition to indicate a group of member functions that shall be disabled by the implementation so that they cannot be called by an application. The disabled member functions are typically the default constructor, the copy constructor, or the assignment operator.

### 3.2.3 Ellipsis (...)

An ellipsis, which consists of three consecutive dots (...), is used to indicate that irrelevant or repetitive parts of a C++ code listing or example have been omitted for clarity.

### 3.2.4 Class names

Class names italicized and annotated with a superscript dagger (<sup>†</sup>) should not be used explicitly within an application. Moreover, an application shall not create an object of such a class. It is strongly recommended that the given class name be used. However, an implementation may substitute an alternative class name in place of every occurrence of a particular daggered class name.

Only the class name is considered here. Whether any part of the definition of the class is implementation-defined is a separate issue.

The class names are the following:

|                                       |                                          |                                        |                                          |
|---------------------------------------|------------------------------------------|----------------------------------------|------------------------------------------|
| <i>sc_bind_proxy</i> <sup>†</sup>     | <i>sc_fxnum_bitref</i> <sup>†</sup>      | <i>sc_signed_bitref</i> <sup>†</sup>   | <i>sc_uint_subref</i> <sup>†</sup>       |
| <i>sc_bitref</i> <sup>†</sup>         | <i>sc_fxnum_fast_bitref</i> <sup>†</sup> | <i>sc_signed_bitref_r</i> <sup>†</sup> | <i>sc_uint_subref_r</i> <sup>†</sup>     |
| <i>sc_bitref_r</i> <sup>†</sup>       | <i>sc_fxnum_fast_subref</i> <sup>†</sup> | <i>sc_signed_subref</i> <sup>†</sup>   | <i>sc_unsigned_bitref</i> <sup>†</sup>   |
| <i>sc_concatref</i> <sup>†</sup>      | <i>sc_fxnum_subref</i> <sup>†</sup>      | <i>sc_signed_subref_r</i> <sup>†</sup> | <i>sc_unsigned_bitref_r</i> <sup>†</sup> |
| <i>sc_concref</i> <sup>†</sup>        | <i>sc_int_bitref</i> <sup>†</sup>        | <i>sc_subref</i> <sup>†</sup>          | <i>sc_unsigned_subref</i> <sup>†</sup>   |
| <i>sc_concref_r</i> <sup>†</sup>      | <i>sc_int_bitref_r</i> <sup>†</sup>      | <i>sc_subref_r</i> <sup>†</sup>        | <i>sc_unsigned_subref_r</i> <sup>†</sup> |
| <i>sc_context_begin</i> <sup>†</sup>  | <i>sc_int_subref</i> <sup>†</sup>        | <i>sc_switch</i> <sup>†</sup>          | <i>sc_value_base</i> <sup>†</sup>        |
| <i>sc_event_and_list</i> <sup>†</sup> | <i>sc_int_subref_r</i> <sup>†</sup>      | <i>sc_uint_bitref</i> <sup>†</sup>     |                                          |
| <i>sc_event_or_list</i> <sup>†</sup>  | <i>sc_sensitive</i> <sup>†</sup>         | <i>sc_uint_bitref_r</i> <sup>†</sup>   |                                          |

### 3.2.5 Embolded text

Embolding is used to enhance readability in this standard but has no significance in SystemC itself. Embolding is used for names of types, classes, functions, and operators in running text and in code fragments where these names are defined. Embolding is never used for upper-case names of macros, constants, and enum literals.

## 3.3 Semantic conventions

### 3.3.1 Class definitions and the inheritance hierarchy

An implementation may differ from this standard in that an implementation may introduce additional base classes, class members, and friends to the classes defined in this standard. An implementation may modify the inheritance hierarchy by moving class members defined by this standard into base classes not defined by this standard. Such additions and modifications may be made as necessary in order to implement the semantics defined by this standard or in order to introduce additional functionality not defined by this standard.

### 3.3.2 Function definitions and side-effects

This standard explicitly defines the semantics of the C++ functions in the SystemC class library. Such functions shall not have any side-effects that would contradict the behavior explicitly mandated by this standard. In general, the reader should assume the common-sense rule that if it is explicitly stated that a function shall perform action A, that function shall not perform any action other than A, either directly or by calling another function defined in this standard. However, a function may, and indeed in certain circumstances shall, perform any tasks necessary for resource management, performance optimization, or to support any ancillary features of an implementation. As an example of resource management, it is assumed that a destructor will perform any tasks necessary to release the resources allocated by the corresponding constructor. As an example of an ancillary feature, an implementation could have the constructor for class **sc\_module** increment a count of the number of module instances in the module hierarchy.

### 3.3.3 Functions whose return type is a reference or a pointer

Many functions in this standard return a reference to an object or a pointer to an object, that is, the return type of the function is a reference or a pointer. This subclause gives some general rules defining the lifetime and the validity of such objects.

An object returned from a function by pointer or by reference is said to be valid during any period in which the object is not deleted and the value or behavior of the object remains accessible to the application. If an application refers to the returned object after it ceases to be valid, the behavior of the implementation shall be undefined.

#### 3.3.3.1 Functions that return **\*this** or an actual argument

In certain cases, the object returned is either an object (**\*this**) returned by reference from its own member function (for example, the assignment operators), or is an object that was passed by reference as an actual argument to the function being called (for example, **std::ostream& operator<< (std::ostream&, const T&)** ). In either case, the function call itself places no additional obligations on the implementation concerning the lifetime and validity of the object following return from the function call.

### 3.3.3.2 Functions that return `char*`

Certain functions have the return type `char*`, that is, they return a pointer to a null-terminated character string. Such strings shall remain valid until the end of the program with the exception of member function `sc_process_handle::name` and member functions of class `sc_report`, where the implementation is only required to keep the string valid while the process handle or report object itself is valid.

### 3.3.3.3 Functions that return a reference or pointer to an object in the module hierarchy

Certain functions return a reference or pointer to an object that forms part of the module hierarchy or a property of such an object. The return types of these functions include the following:

- a) `sc_interface *` // Returns a channel
- b) `sc_event&` // Returns an event
- c) `sc_event_finder&` // Returns an event finder
- d) `sc_time&` // Returns a property of primitive channel `sc_clock`

The implementation is obliged to ensure that the returned object is valid until either the channel, event, or event finder is deleted explicitly by the application or until the destruction of the module hierarchy, whichever is sooner.

### 3.3.3.4 Functions that return a reference or pointer to a transient object

Certain functions return a reference or pointer to an object that may be deleted by the application or the implementation before the destruction of the module hierarchy. The return types of these functions include the following:

- a) `sc_object *`
- b) `sc_attr_base *`
- c) `std::string&` // Property of an attribute object

The functions concerned are the following:

```
sc_object* sc_process_handle::get_parent_object() const;
sc_object* sc_process_handle::get_process_object() const;
sc_object* sc_object::get_parent_object() const;
sc_object* sc_find_object( const char* );
sc_attr_base* sc_object::get_attribute( const std::string& );
const sc_attr_base* sc_object::get_attribute( const std::string& ) const;
sc_attr_base* sc_object::remove_attribute( const std::string& );
const std::string& sc_attr_base::name() const;
```

The implementation is only obliged to ensure that the returned reference is valid until the `sc_object`, `sc_attr_base`, or `std::string` object itself is deleted.

Certain functions return a reference to an object that represents a transient collection of other objects, where the application may add or delete objects before the destruction of the module hierarchy such that the contents of the collection would be modified. The return types of these functions include the following:

- a) `std::vector< sc_object * > &`
- b) `sc_attr_cltn *`

The functions concerned are the following:

```
virtual const std::vector<sc_object*>& sc_module::get_child_objects() const;  
const std::vector<sc_object*>& sc_process_handle::get_child_objects() const;  
virtual const std::vector<sc_object*>& sc_object::get_child_objects() const;  
const std::vector<sc_object*>& sc_get_top_level_objects();  
sc_attr_cltn& sc_object::attr_cltn();  
const sc_attr_cltn& sc_object::attr_cltn() const;
```

The implementation is only obliged to ensure that the returned object (the vector or collection) is itself valid until an **sc\_object** or an attribute is added or deleted that would affect the collection returned by the function if it were to be called again.

### 3.3.3.5 Functions **sc\_time\_stamp** and **sc\_signal::read**

The implementation is obliged to keep the object returned from function **sc\_time\_stamp** valid until the start of the next timed notification phase.

The implementation is obliged to keep the object returned from function **sc\_signal::read** valid until the end of the current evaluation phase.

For both functions, it is strongly recommended that the application be written in such a way that it would have identical behavior, whether these functions return a reference to an object or return the same object by value.

### 3.3.4 Namespaces and internal naming

An implementation shall place every declaration and every macro definition specified by this standard within one of the two namespaces **sc\_core** and **sc\_dt**. The core language and predefined channels shall be placed in the namespace **sc\_core**. The SystemC data types proper shall be placed in the namespace **sc\_dt**. The utilities are divided between the two namespaces.

It is recommended that an implementation use nested namespaces within **sc\_core** and **sc\_dt** in order to reduce to a minimum the number of implementation-defined names in these two namespaces. The names of any such nested namespaces shall be implementation-defined.

In general, the choice of internal, implementation-specific names within an implementation can cause naming conflicts within an application. It is up to the implementor to choose names that are unlikely to cause naming conflicts within an application.

### 3.3.5 Non-compliant applications and errors

In the case where an application fails to meet an obligation imposed by this standard, the behavior of the SystemC implementation shall be undefined in general. When this results in the violation of a diagnosable rule of the C++ standard, the C++ implementation will issue a diagnostic message in conformance with the C++ standard.

When this standard explicitly states that the failure of an application to meet a specific obligation is an *error* or a *warning*, the SystemC implementation shall generate a diagnostic message by calling the function **sc\_report\_handler::report**. In the case of an *error*, the implementation shall call function **report** with a severity of **SC\_ERROR**. In the case of a *warning*, the implementation shall call function **report** with a severity of **SC\_WARNING**.

An implementation or an application may choose to suppress run-time error checking and diagnostic messages because of considerations of efficiency or practicality. For example, an application may call member function **set\_actions** of class **sc\_report\_handler** to take no action for certain categories of report. An application that fails to meet the obligations imposed by this standard remains in error.

There are cases where this standard states explicitly that a certain behavior or result is *undefined*. This standard places no obligations on the implementation in such a circumstance. In particular, such a circumstance may or may not result in an *error* or a *warning*.

### 3.4 Notes and examples

Notes appear at the end of certain subclauses, designated by the upper-case word NOTE. Notes often describe consequences of rules defined elsewhere in this standard. Certain subclauses include examples consisting of fragments of C++ source code. Such notes and examples are informative to help the reader but are not an official part of this standard.

## 4. Elaboration and simulation semantics

An implementation of the SystemC class library includes a public *shell* consisting of those predefined classes, functions, macros, and so forth that can be used directly by an application. Such features are defined in Clause 5, Clause 6, Clause 7, and Clause 8 of this standard. An implementation also includes a private *kernel* that implements the core functionality of the class library. The underlying semantics of the kernel are defined in this clause.

The execution of a SystemC application consists of *elaboration* followed by *simulation*. Elaboration results in the creation of the *module hierarchy*. Elaboration involves the execution of application code, the public shell of the implementation (as mentioned in the preceding paragraph), and the private kernel of the implementation. Simulation involves the execution of the *scheduler*, part of the kernel, which in turn may execute *processes* within the application.

In addition to providing support for elaboration and implementing the scheduler, the kernel may also provide implementation-specific functionality beyond the scope of this standard. As an example of such functionality, the kernel may save the state of the module hierarchy after elaboration and run or restart simulation from that point, or it may support the graphical display of state variables on-the-fly during simulation.

The phases of elaboration and simulation shall run in the following sequence:

- a) Elaboration—Construction of the module hierarchy
- b) Elaboration—Callbacks to function **before\_end\_of\_elaboration**
- c) Elaboration—Callbacks to function **end\_of\_elaboration**
- d) Simulation—Callbacks to function **start\_of\_simulation**
- e) Simulation—Initialization phase
- f) Simulation—Evaluation, update, delta notification, and timed notification phases (repeated)
- g) Simulation—Callbacks to function **end\_of\_simulation**
- h) Simulation—Destruction of the module hierarchy

### 4.1 Elaboration

The primary purpose of elaboration is to create internal data structures within the kernel as required to support the semantics of simulation. During elaboration, the parts of the module hierarchy (modules, ports, primitive channels, and processes) are created, and ports and exports are bound to channels.

The actions stated in the following subclauses can occur during elaboration and only during elaboration.

NOTE 1—Because these actions can only occur during elaboration, SystemC does not support the dynamic creation or modification of the module hierarchy during simulation, although it does support dynamic processes.

NOTE 2—Other actions besides those listed below may occur during elaboration, provided that they do not contradict any statement made in this standard. For example, objects of class **sc\_dt::sc\_logic** may be created during elaboration and spawned processes may be created during elaboration, but the function **notify** of class **sc\_event** shall not be called during elaboration.

#### 4.1.1 Instantiation

Instances of the following classes (or classes derived from these classes) may be created during elaboration and only during elaboration. Such instances shall not be deleted before the destruction of the module hierarchy at the end of simulation.

sc\_module (see 5.2)  
sc\_port (see 5.11)  
sc\_export (see 5.12)  
sc\_prim\_channel (see 5.14)

An implementation shall permit an application to have zero or one top-level modules and may permit more than one top-level module (see 4.3.4.1 and 4.3.5).

Instances of class **sc\_module** and class **sc\_prim\_channel** may only be created within a module or within function **sc\_main**. Instances of class **sc\_port** and class **sc\_export** can only be created within a module. It shall be an error to instantiate a module or primitive channel other than within a module or within function **sc\_main**, or to instantiate a port or export other than within a module.

The instantiation of a module also implies the construction of objects of class **sc\_module\_name** and class **sc\_sensitive**<sup>†</sup> (see 5.4).

Although these rules allow for considerable flexibility in instantiating the module hierarchy, it is strongly recommended that, wherever possible, module, port, export, and primitive channel instances be data members of a module or their addresses be stored in data members of a module. Moreover, the names of those data members should match the string names of the instances wherever possible.

NOTE 1—The four classes **sc\_module**, **sc\_port**, **sc\_export**, and **sc\_prim\_channel** are derived from a common base class **sc\_object**, and thus have some member functions in common (see 5.15).

NOTE 2—Objects of classes derived from **sc\_object** but not derived from one of these four classes may be instantiated during elaboration or during simulation, as may objects of user-defined classes.

*Example:*

```
#include "systemc.h"

struct Mod: sc_module
{
    SC_CTOR(Mod) {}
};

struct S
{
    Mod m; // Unusual coding style - module instance within struct
    S(char* name_) : m(name_) {}
};

struct Top: sc_module // Five instances of module Mod exist within module Top.
{
    Mod m1; // Recommended coding style
    Mod *m2; // Recommended coding style
    S s1;

    SC_CTOR(Top)
    : m1("m1"), // m1.name() returns "top.m1"
      s1("s1") // s1.m.name() returns "top.s1"
    {
        m2 = new Mod("m2"); // m2->name() returns "top.m2"
        f();
        S *s2 = new S("s2"); // s2->m.name() returns "top.s2"
    }
};
```



```
    }  
    void f() {  
        Mod *m3 = new Mod("m3"); // Unusual coding style - not recommended  
    }                                     // m3->name() returns "top.m3"  
};  
  
int sc_main(int argc, char* argv[])  
{  
    Top top("top");  
    sc_start();  
    return 0;  
}
```

#### 4.1.2 Process macros

An *unspawned process instance* is a process created by invoking one of the following three process macros:

```
SC_METHOD  
SC_THREAD  
SC_CTHREAD
```

The name of a member function belonging to a class derived from class **sc\_module** shall be passed as an argument to the macro. This member function shall become the function *associated* with the process instance.

Unspawned processes can be created during elaboration or from the **end\_of\_elaboration** callback. Spawned processes may be created by calling the function **sc\_spawn** during elaboration or during simulation.

The purpose of the process macros is to register the associated function with the kernel such that the scheduler can call back that member function during simulation. It is also possible to use spawned processes for this same purpose. The process macros are provided for backward compatibility with earlier versions of SystemC and to provide clocked threads for hardware synthesis.

#### 4.1.3 Port binding and export binding

Port instances can be *bound* to channel instances, to other port instances, or to export instances. Export instances can be *bound* to channel instances or to other export instances, but not to port instances. Port binding is an asymmetrical relationship, and export binding is an asymmetrical relationship. If a port is *bound* to a channel, it is not true to say that the channel is *bound* to the port. Rather, it is true to say that the channel is the channel to which the port is *bound*.

Ports can be bound by name or by position. Named port binding is performed by a member function of class **sc\_port** (see 5.11.7). Positional port binding is performed by a member function of class **sc\_module** (see 5.2.18). Exports can only be bound by name. Export binding is performed by a member function of class **sc\_export** (see 5.12.7).

A given port instance shall not be bound both by name and by position.

A port should typically be bound within the parent of the module instance containing that port. Hence, when port A is bound to port B, the module containing port A will typically be instantiated within the module containing port B. An export should typically be bound within the module containing the export. A port should typically be bound to a channel or a port that lies within the same module in which the port is bound or to an export within a child module. An export should typically be bound to a channel that lies within the same module in which the export is bound or to an export within a child module.

When port A is bound to port B, and port B is bound to channel C, the effect shall be the same as if port A were bound directly to channel C. Wherever this standard refers to port A being bound to channel C, it shall be assumed this means that port A is bound either directly to channel C or to another port that is itself bound to channel C according to this very same rule. This same rule shall apply when binding exports.

Port and export binding can occur during elaboration and only during elaboration. Whether a port need be bound is dependent upon the port policy argument of the port instance, whereas every export shall be bound exactly once. A module may have zero or more ports and zero or more exports. If a module has no ports, no (positional) port bindings are necessary or permitted for instances of that module. Ports may be bound (by name) in any sequence. The binding of ports belonging to different module instances may be interleaved. Since a port may be bound to another port that has not yet itself been bound, the implementation may defer the completion of port binding until a later time during elaboration, whereas exports shall be bound immediately. Such deferred port binding shall be completed by the implementation before the callbacks to function **end\_of\_elaboration**.

The channel to which a port is bound shall not be deleted before the destruction of the module hierarchy at the end of simulation.

Where permitted in the definition of the port object, a single port can be bound to multiple channel or port instances. Such ports are known as *multiports* (see 5.11.3). An export can only be bound once. It shall be an error to bind a given port instance to a given channel instance more than once, even if the port is a multiport.

When a port is bound to a channel, the kernel shall call the member function **register\_port** of the channel. There is no corresponding function called when an export is bound (see 5.13).

The purpose of port and export binding is to enable a port or export to forward interface method calls made during simulation to the channel instances to which that port was bound during elaboration. This forwarding is performed during simulation by member functions of the class **sc\_port** and class **sc\_export**, such as **operator->**. A port *requires* the services defined by an interface (that is, the type of the port), whereas an export *provides* the services defined by an interface (that is, the type of the export).

NOTE 1—A phrase such as *bind a channel to a port* is not used in this standard. However, it is recognized that such a phrase may be used informally to mean *bind a port to a channel*.

NOTE 2—A port of a child module instance can be bound to an export of that same child module instance.

NOTE 3—Member function **register\_port** is defined in the class **sc\_interface** from which every channel is derived.

#### 4.1.4 Setting the time resolution

The simulation time resolution can be set during elaboration and only during elaboration. Time resolution is set by calling the function **sc\_set\_time\_resolution** (see 5.10.3).

NOTE—Time resolution can only be set globally. There is no concept of a local time resolution.

## 4.2 Simulation

This subclause defines the behavior of the scheduler and the semantics of simulated time and process execution.

The primary purpose of the scheduler is to trigger or resume the execution of the processes that the user supplies as part of the application. The scheduler is event-driven, meaning that processes are executed in response to the occurrence of events. Events occur (are *notified*) at precise points in simulation time. Events are represented by objects of the class **sc\_event**, and by this class alone (see 5.9).

Simulation time is an integer quantity. Simulation time is initialized to zero at the start of simulation and increases monotonically during simulation. The physical significance of the integer value representing time within the kernel is determined by the simulation time resolution. Simulation time and time intervals are represented by class **sc\_time**. Certain functions allow time to be expressed as a value pair having the signature **double,sc\_time\_unit** (see 5.10.1).

The scheduler can execute a spawned or unspawned process instance as a consequence of one of the following four causes, and these alone:

- In response to the process instance having been made runnable during the initialization phase (see 4.2.1.1)
- In response to a call to function **sc\_spawn** during simulation
- In response to the occurrence of an event to which the process instance is sensitive
- In response to a time-out having occurred

The *sensitivity* of a process instance is the set of events and time-outs that can potentially cause the process to be resumed or triggered. The *static sensitivity* of an unspawned process instance is fixed during elaboration. The *static sensitivity* of a spawned process instance is fixed when the function **sc\_spawn** is called. The *dynamic sensitivity* of a process instance may vary over time under the control of the process itself. A process instance is said to be *sensitive* to an event if the event has been added to the static sensitivity or dynamic sensitivity of the process instance. A *time-out* occurs when a given time interval has elapsed.

The scheduler shall also manage event notifications and primitive channel update requests.

#### 4.2.1 The scheduling algorithm

The semantics of the scheduling algorithm are defined in the following subclauses. For the sake of clarity, imperative language is used in this description. The description of the scheduling algorithm uses the following four sets:

- The set of runnable processes
- The set of update requests
- The set of delta notifications and time-outs
- The set of timed notifications and time-outs

An implementation may substitute an alternative scheme, provided the scheduling semantics given here are retained.

A process instance shall not appear more than once in the set of runnable processes. An attempt to add to this set a process instance that is already runnable shall be ignored.

An *update request* results from, and only from, a call to member function **request\_update** of class **sc\_prim\_channel** (see 5.14.6).

An *immediate notification* results from, and only from, a call to member function **notify** of class **sc\_event** with no arguments (see 5.9.4).

A *delta notification* results from, and only from, a call to member function **notify** of class **sc\_event** with a zero-valued time argument.

A *timed notification* results from, and only from, a call to member function **notify** of class **sc\_event** with a non-zero-valued time argument. The time argument determines the time of the notification, relative to the time when function **notify** is called.

A *time-out* results from, and only from, certain calls to functions **wait** or **next\_trigger**, which are member functions of class **sc\_module**, member functions of class **sc\_prim\_channel**, and non-member functions. A time-out resulting from a call with a zero-valued time argument is added to the set of delta notifications and time-outs. A time-out resulting from a call with a non-zero-valued time argument is added to the set of timed notifications and time-outs (see 5.2.16 and 5.2.17).

The scheduler starts by executing the initialization phase.

#### 4.2.1.1 Initialization phase

Perform the following three steps in the order given:

- a) Run the update phase as defined in 4.2.1.3 but without continuing to the delta notification phase.
- b) Add every method and thread process instance in the object hierarchy to the set of runnable processes, but exclude those process instances for which the function **dont\_initialize** has been called, and exclude clocked thread processes.
- c) Run the delta notification phase, as defined in 4.2.1.4. At the end of the delta notification phase, go to the evaluation phase.

NOTE—The update and delta notification phases are necessary because function **request\_update** can be called during elaboration in order to set initial values for primitive channels, for example, from function **initialize** of class **sc\_inout**.

#### 4.2.1.2 Evaluation phase

From the set of runnable processes, select a process instance and trigger or resume its execution. Run the process instance immediately and without interruption up to the point where it either returns or calls the function **wait**.

Since process instances execute without interruption, only a single process instance can be running at any one time, and no other process instance can execute until the currently executing process instance has yielded control to the kernel. A process shall not pre-empt or interrupt the execution of another process. This is known as *co-routine semantics* or *co-operative multitasking*.

The order in which process instances are selected from the set of runnable processes is implementation-defined. However, if a specific version of a specific implementation runs a specific application using a specific input data set, the order of process execution shall not vary from run to run.

A process may execute an immediate notification, in which case determine which process instances are currently sensitive to the notified event and add all such process instances to the set of runnable processes. Such processes shall be executed subsequently in this very same evaluation phase.

A process may call function **sc\_spawn** to create a spawned process instance, in which case the new process instance shall be added to the set of runnable processes (unless function **sc\_spawn\_options::dont\_initialize** is called) and subsequently executed in this very same evaluation phase.

A process may call the member function **request\_update** of a primitive channel, which will cause the member function **update** of that same primitive channel to be called back during the very next update phase.

Repeat this step until the set of runnable processes is empty, then go on to the update phase.

NOTE 1—The scheduler is not pre-emptive. An application can assume that a method process will execute in its entirety without interruption, and a thread or clocked thread process will execute the code between two consecutive calls to function **wait** without interruption.

NOTE 2—Because the order in which processes are run within the evaluation phase is not under the control of the application, access to shared storage should be explicitly synchronized to avoid non-deterministic behavior.

NOTE 3—An implementation running on a machine that provides hardware support for concurrent processes may permit two or more processes to run concurrently, provided that the behavior appears identical to the co-routine semantics defined in this subclause. In other words, the implementation would be obliged to analyze any dependencies between processes and constrain their execution to match the co-routine semantics.

NOTE 4—When an immediate notification occurs, only processes that are currently sensitive to the notified event shall be made runnable. This excludes processes that are only made dynamically sensitive to the notified event later in the same evaluation phase.

#### 4.2.1.3 Update phase

Execute any and all pending calls to function **update** resulting from calls to function **request\_update** made in the immediately preceding evaluation phase or made during elaboration if the update phase is executed as part of the initialization phase.

If no remaining pending calls to function **update** exist, go on to the delta notification phase (except when executed from the initialization phase).

#### 4.2.1.4 Delta notification phase

If pending delta notifications or time-outs exist (which can only result from calls to function **notify** or function **wait** in the immediately preceding evaluation phase or update phase):

- a) Determine which process instances are sensitive to these events or time-outs.
- b) Add all such process instances to the set of runnable processes.
- c) Remove all such notifications and time-outs from the set of delta notifications and time-outs.

If, at the end of the delta notification phase, the set of runnable processes is non-empty, go back to the evaluation phase.

#### 4.2.1.5 Timed notification phase

If pending timed notifications or time-outs exist:

- a) Advance simulation time to the time of the earliest pending timed notification or time-out.
- b) Determine which process instances are sensitive to the events notified and time-outs lapsing at this precise time.
- c) Add all such process instances to the set of runnable processes.
- d) Remove all such notifications and time-outs from the set of timed notifications and time-outs.

If no pending timed notifications or time-outs exist, the end of simulation has been reached. So, exit the scheduler.

If, at the end of the timed notification phase, the set of runnable processes is non-empty, go back to the evaluation phase.

### 4.2.2 Cycles in the scheduling algorithm

A *delta cycle* is a sequence of steps in the scheduling algorithm consisting of the following steps in the order given:

- a) An evaluation phase
- b) An update phase
- c) A delta notification phase

The initialization phase does not include a delta cycle.

NOTE 1—The scheduling algorithm implies the existence of three causal loops resulting from immediate notification, delta notification, and timed notification, as follows:

- The immediate notification loop is restricted to a single evaluation phase.
- The delta notification loop takes the path of evaluation phase, followed by update phase, followed by delta notification phase and back to evaluation phase. This loop advances simulation by one delta cycle.
- The timed notification loop takes the path of evaluation phase, followed by update phase, followed by delta notification phase, followed by timed notification phase and back to evaluation phase. This loop advances simulation time.

NOTE 2—The immediate notification loop is non-deterministic in the sense that process execution can be interleaved with immediate notification, and the order in which runnable processes are executed is undefined.

NOTE 3—The delta notification and timed notification loops are deterministic in the sense that process execution alternates with primitive channel updates. If, within a particular application, inter-process communication is confined to using only deterministic primitive channels, the behavior of the application will be independent of the order in which the processes are executed within the evaluation phase (assuming no other explicit dependencies on process order such as external input or output exist).

### 4.3 Running elaboration and simulation

An implementation shall provide either or both of the following two mechanisms for running elaboration and simulation:

- Under application control using functions **sc\_main** and **sc\_start**
- Under control of the kernel

Both mechanisms are defined in the following subclauses. An implementation is not obliged to provide both mechanisms.

#### 4.3.1 Function declarations

```
namespace sc_core {

    int sc_elab_and_sim( int argc, char* argv[] );
    int sc_argc();
    const char* const* sc_argv();
    void sc_start();
    void sc_start( const sc_time& );
    void sc_start( double, sc_time_unit );
}
```

#### 4.3.2 Function **sc\_elab\_and\_sim**

The function **main** that is the entry point of the C++ program may be provided by the implementation or by the application. If function **main** is provided by the implementation, function **main** shall initiate the mechanisms for elaboration and simulation as described in this subclause. If function **main** is provided by the application, function **main** shall call the function **sc\_elab\_and\_sim**, which is the entry point into the SystemC implementation.

The implementation shall provide a function **sc\_elab\_and\_sim** with the following declaration:

```
int sc_elab_and_sim( int argc, char* argv[] );
```

Function **sc\_elab\_and\_sim** shall initiate the mechanisms for running elaboration and simulation. The application should pass the values of the parameters from function **main** as arguments to function

**sc\_elab\_and\_sim**. Whether the application may call function **sc\_elab\_and\_sim** more than once is implementation-defined.

A return value of 0 from function **sc\_elab\_and\_sim** shall indicate successful completion. An implementation may use other return values to indicate other termination conditions.

NOTE—Function **sc\_elab\_and\_sim** was named **sc\_main\_main** in an earlier version of SystemC.

#### 4.3.3 Functions **sc\_argc** and **sc\_argv**

The implementation shall provide functions **sc\_argc** and **sc\_argv** with the following declarations:

```
int sc_argc();  
const char* const* sc_argv();
```

These two functions shall return the values of the arguments passed to function **main** or function **sc\_elab\_and\_sim**.

#### 4.3.4 Running under application control using functions **sc\_main** and **sc\_start**

The application provides a function **sc\_main** and calls the function **sc\_start**, as defined in 4.3.4.1 and 4.3.4.2.

##### 4.3.4.1 Function **sc\_main**

An application shall provide a function **sc\_main** with the following declaration. The order and types of the arguments and the return type shall be as shown here:

```
int sc_main( int argc, char* argv[] );
```

This function shall be called once from the kernel and is the only entry point into the application. The arguments **argc** and **argv[]** are command-line arguments. The implementation may pass the values of C++ command-line arguments (as passed to function **main**) through to function **sc\_main**. The choice of which C++ command-line arguments to pass is implementation-defined.

Elaboration consists of the execution of the **sc\_main** function from the start of **sc\_main** to the point immediately before the first call to the function **sc\_start**.

A return value of 0 from function **sc\_main** shall indicate successful completion. An application may use other return values to indicate other termination conditions.

NOTE 1—As a consequence of the rules defined in 4.1, before calling function **sc\_start** for the first time, the function **sc\_main** may instantiate modules, instantiate primitive channels, bind the ports and exports of module instances to channels, and set the time resolution. More than one top-level module may exist.

NOTE 2—Throughout this standard the term *call* is taken to mean call directly or indirectly. Hence function **sc\_start** may be called indirectly from function **sc\_main** by another function or functions.

##### 4.3.4.2 Function **sc\_start**

The implementation shall provide a function **sc\_start**, overloaded with the following signatures:

```
void sc_start();  
void sc_start( const sc_time& );  
void sc_start( double, sc_time_unit );
```

The behavior of the latter function shall be equivalent to the following definition:

```
void sc_start( double d, sc_time_unit t ) { sc_start( sc_time(d, t) ); }
```

When called for the first time, function **sc\_start** shall start the scheduler, which shall run up to the simulation time passed as an argument (if an argument was passed), unless otherwise interrupted.

When called on the second and subsequent occasions, function **sc\_start** shall resume the scheduler from the time it had reached at the end of the previous call to **sc\_start**. The scheduler shall run for the time passed as an argument (if an argument was passed), relative to the current simulation time, unless otherwise interrupted.

When a time is passed as an argument, the scheduler shall execute up to and including the timed notification phase that advances simulation time to the end time (calculated by adding the time given as an argument to the simulation time when function **sc\_start** is called).

When function **sc\_start** is called without any arguments, the scheduler shall run until it reaches completion, unless otherwise interrupted.

When function **sc\_start** is called with a zero-valued time argument, the scheduler shall run for one delta cycle.

Once started, the scheduler shall run until either it reaches completion, or the application calls the function **sc\_stop**, or an exception occurs, or simulation is stopped or aborted by the report handler (see 8.3). Once the function **sc\_stop** has been called, function **sc\_start** shall not be called again.

Function **sc\_start** may be called from function **sc\_main**, and only from function **sc\_main**.

On completion, function **sc\_start** returns control to the function from which it was called.

NOTE—When the scheduler is paused between successive calls to function **sc\_start**, the set of runnable processes need not be empty.

#### 4.3.5 Running under control of the kernel

Elaboration and simulation may be initiated under the direct control of the kernel, in which case the implementation shall not call the function **sc\_main**, and the implementation is not obliged to provide a function **sc\_start**.

An implementation may permit more than one top-level module but is not obliged to do so.

NOTE 1—In this case, the mechanisms used to initiate elaboration and simulation and to identify top-level modules are implementation-defined.

NOTE 2—In this case, an implementation shall honor all obligations set out in this standard with the exception of those in 4.3.4.

#### 4.4 Elaboration and simulation callbacks

Four callback functions are called by the kernel at various stages during elaboration and simulation. They have the following declarations:

```
virtual void before_end_of_elaboration();  
virtual void end_of_elaboration();  
virtual void start_of_simulation();  
virtual void end_of_simulation();
```



The implementation shall define each of these four callback functions as member functions of the classes **sc\_module**, **sc\_port**, **sc\_export**, and **sc\_prim\_channel**, and each of these definitions shall have empty function bodies. The implementation also overrides various of these functions as member functions of various predefined channel and port classes and having specific behaviors (see Clause 6). An application may override any of these four functions in any class derived from any of the classes mentioned in this paragraph. If an application overrides any such callback function of a predefined class and the callback has implementation-defined behavior (for example, **sc\_in::end\_of\_elaboration**), the application-defined member function in the derived class may or may not call the implementation-defined function of the base class, and the behavior will differ, depending on whether the member function of the base class is called.

Within each of the four categories of callback functions, the order in which the callbacks are made for objects of class **sc\_module**, **sc\_port**, **sc\_export**, and **sc\_prim\_channel** shall be implementation-defined.

The implementation shall make callbacks to all such functions for every instance in the module hierarchy, as defined in the following subclauses.

The implementation shall provide the following two functions:

```
namespace sc_core {

    bool sc_start_of_simulation_invoked();
    bool sc_end_of_simulation_invoked();
}
```

Function **sc\_start\_of\_simulation\_invoked** shall return **true** after and only after all the callbacks to function **start\_of\_simulation** have executed to completion. Function **sc\_end\_of\_simulation\_invoked** shall return **true** after and only after all the callbacks to function **end\_of\_simulation** have executed to completion.

#### 4.4.1 before\_end\_of\_elaboration

The implementation shall make callbacks to member function **before\_end\_of\_elaboration** after the construction of the module hierarchy defined in 4.3 is complete. Function **before\_end\_of\_elaboration** may extend the construction of the module hierarchy by instantiating further modules (and other objects) within the module hierarchy.

The purpose of member function **before\_end\_of\_elaboration** is to allow an application to perform actions during elaboration that depend on global properties of the module hierarchy and which also need to modify the module hierarchy. Examples include the instantiation of top-level modules to monitor events buried within the hierarchy.

The following actions may be performed directly or indirectly from the member function **before\_end\_of\_elaboration**.

- a) The instantiation of objects of class **sc\_module**, **sc\_port**, **sc\_export**, **sc\_prim\_channel**
- b) The instantiation of objects of other classes derived from class **sc\_object**
- c) Port binding
- d) Export binding
- e) The macros **SC\_METHOD**, **SC\_THREAD**, **SC\_CTHREAD**, **SC\_HAS\_PROCESS**
- f) The member **sensitive** and member functions **dont\_initialize**, **set\_stack\_size**, and **reset\_signal\_is** of the class **sc\_module**
- g) Calls to event finder functions
- h) Calls to function **sc\_spawn** to create static spawned processes
- i) Calls to member function **request\_update** of class **sc\_prim\_channel** to create update requests (for example by calling member function **initialize** of class **sc\_inout**)

The following constructs shall not be used directly within member function **before\_end\_of\_elaboration**, but may be used where permitted within module instances nested within callbacks to **before\_end\_of\_elaboration**:

- a) The macro **SC\_CTOR**
- b) Calls to member function **notify** of the class **sc\_event**

**operator->** and **operator[]** of class **sc\_port** should not be called from the function **before\_end\_of\_elaboration**, because the implementation may not have completed port binding at the time of this callback and hence these operators may return null pointers. The member function **size** may return a value less than its final value.

Any **sc\_object** instances created from callback **before\_end\_of\_elaboration** shall be placed at a location in the module hierarchy as if those instances had been created from the constructor of the module to which the callback belongs, or to the parent module if the callback belongs to a port, export, or primitive channel. In other words, it shall be as if the instances were created from the constructor of the object whose callback is called.

Objects instantiated from the member function **before\_end\_of\_elaboration** may themselves override any of the four callback functions, including the member function **before\_end\_of\_elaboration** itself. The implementation shall make all such nested callbacks. An application can assume that every such member function will be called back by the implementation, whatever the context in which the object is instantiated.

#### 4.4.2 end\_of\_elaboration

The implementation shall call member function **end\_of\_elaboration** at the very end of elaboration after all callbacks to **before\_end\_of\_elaboration** have completed and after the completion of any instantiation or port binding performed by those callbacks and before starting simulation.

The purpose of member function **end\_of\_elaboration** is to allow an application to perform housekeeping actions at the end of elaboration that do not need to modify the module hierarchy. Examples include design rule checking, actions that depend on the number of times a port is bound, and printing diagnostic messages concerning the module hierarchy.

The following actions may be performed directly or indirectly from the callback **end\_of\_elaboration**:

- a) The instantiation of objects of classes derived from class **sc\_object** but excluding classes **sc\_module**, **sc\_port**, **sc\_export**, and **sc\_prim\_channel**
- b) The macros **SC\_METHOD**, **SC\_THREAD**, and **SC\_HAS\_PROCESS**
- c) The member **sensitive** and member functions **dont\_initialize** and **set\_stack\_size** of the class **sc\_module**
- d) Calls to function **sc\_spawn** to create dynamic spawned processes
- e) Calls to member function **request\_update** of class **sc\_prim\_channel** to create update requests (for example by calling member function **write** of class **sc\_inout**)
- f) Interface method calls using **operator->** and **operator[]** of class **sc\_port**, provided that those calls do not attempt to perform actions prohibited outside simulation such as event notification

The following constructs shall not be used directly or indirectly within callback **end\_of\_elaboration**:

- a) The instantiation of objects of class **sc\_module**, **sc\_port**, **sc\_export**, **sc\_prim\_channel**
- b) Port binding
- c) Export binding
- d) The macros **SC\_CTOR**, **SC\_CTHREAD**

- e) The member function **reset\_signal\_is** of the class **sc\_module**
- f) Calls to event finder functions
- g) Calls to member function **notify** of the class **sc\_event**

#### 4.4.3 start\_of\_simulation

The implementation shall call member function **start\_of\_simulation** immediately when the application calls function **sc\_start** for the first time or at the very start of simulation, if simulation is initiated under the direct control of the kernel. If an application makes multiple calls to **sc\_start**, the implementation shall only make the callbacks to **start\_of\_simulation** on the first such call to **sc\_start**. The implementation shall call function **start\_of\_simulation** after the callbacks to **end\_of\_elaboration** and before invoking the initialization phase of the scheduler.

The purpose of member function **start\_of\_simulation** is to allow an application to perform housekeeping actions at the start of simulation. Examples include opening stimulus and response files and printing diagnostic messages. The intention is that an implementation that initiates elaboration and simulation under direct control of the kernel (in the absence of functions **sc\_main** and **sc\_start**) shall make the callbacks to **end\_of\_elaboration** at the end of elaboration and the callbacks to **start\_of\_simulation** at the start of simulation.

The following actions may be performed directly or indirectly from the callback **start\_of\_simulation**:

- a) The instantiation of objects of classes derived from class **sc\_object** but excluding classes **sc\_module**, **sc\_port**, **sc\_export**, and **sc\_prim\_channel**
- b) Calls to function **sc\_spawn** to create dynamic spawned processes
- c) Calls to member function **request\_update** of class **sc\_prim\_channel** to create update requests (for example by calling member function **write** of class **sc\_inout**)
- d) Interface method calls using **operator->** and **operator[]** of class **sc\_port**, provided that those calls do not attempt to perform actions prohibited outside simulation such as event notification

The following constructs shall not be used directly or indirectly within callback **start\_of\_simulation**:

- a) The instantiation of objects of class **sc\_module**, **sc\_port**, **sc\_export**, **sc\_prim\_channel**
- b) Port binding
- c) Export binding
- d) The macros **SC\_CTOR**, **SC\_METHOD**, **SC\_THREAD**, **SC\_CTHREAD**, **SC\_HAS\_PROCESS**
- e) The member **sensitive** and member functions **dont\_initialize**, **set\_stack\_size**, and **reset\_signal\_is** of the class **sc\_module**
- f) Calls to event finder functions
- g) Calls to member function **notify** of the class **sc\_event**

#### 4.4.4 end\_of\_simulation

The implementation shall call member function **end\_of\_simulation** at the point when the scheduler halts because of the function **sc\_stop** having been called during simulation (see 4.5.2) or at the very end of simulation if simulation is initiated under the direct control of the kernel. The **end\_of\_simulation** callbacks shall only be called once even if function **sc\_stop** is called multiple times.

The purpose of member function **end\_of\_simulation** is to allow an application to perform housekeeping actions at the end of simulation. Examples include closing stimulus and response files and printing diagnostic messages. The intention is that an implementation that initiates elaboration and simulation under

direct control of the kernel (in the absence of functions **sc\_main** and **sc\_start**) shall make the callbacks to **end\_of\_simulation** at the very end of simulation whether or not function **sc\_stop** has been called.

As a consequence of the language mechanisms of C++, the destructors of any objects in the module hierarchy will be called as these objects are deleted at the end of program execution. Any callbacks to function **end\_of\_simulation** shall be made before the destruction of the module hierarchy. The function **sc\_end\_of\_simulation\_invoked** may be called by the application within a destructor to determine whether the callback has been made.

The implementation is not obliged to support any of the following actions when made directly or indirectly from the member function **end\_of\_simulation** or from the destructors of any objects in the module hierarchy. Whether any of these actions cause an error is implementation-defined.

- a) The instantiation of objects of classes derived from class **sc\_object**
- b) Calls to function **sc\_spawn** to create dynamic spawned processes
- c) Calls to member function **request\_update** of class **sc\_prim\_channel** to create update requests (for example by calling member function **write** of class **sc\_inout**)
- d) Calls to member function **notify** of the class **sc\_event**

## 4.5 Other functions related to the scheduler

### 4.5.1 Function declarations

```
namespace sc_core {

    enum sc_stop_mode
    {
        SC_STOP_FINISH_DELTA ,
        SC_STOP_IMMEDIATE
    };

    extern void sc_set_stop_mode( sc_stop_mode mode );
    extern sc_stop_mode sc_get_stop_mode();
    void sc_stop();

    const sc_time& sc_time_stamp();
    const sc_dt::uint64 sc_delta_count();
    bool sc_is_running();
}
```

### 4.5.2 Function **sc\_stop**, **sc\_set\_stop\_mode**, and **sc\_get\_stop\_mode**

The implementation shall provide functions **sc\_set\_stop\_mode**, **sc\_get\_stop\_mode**, and **sc\_stop** with the following declarations:

```
enum sc_stop_mode
{
    SC_STOP_FINISH_DELTA ,
    SC_STOP_IMMEDIATE
};

extern void sc_set_stop_mode( sc_stop_mode mode );
extern sc_stop_mode sc_get_stop_mode();
```

```
void sc_stop();
```

The function **sc\_set\_stop\_mode** shall set the current stop mode to the value passed as an argument. The function **sc\_get\_stop\_mode** shall return the current stop mode.

The function **sc\_stop** may be called by the application from an elaboration or simulation callback, from a process, from the member function **update** of class **sc\_prim\_channel**, or from function **sc\_main**. The implementation may call the function **sc\_stop** from member function **report** of class **sc\_report\_handler**.

A call to function **sc\_stop** shall cause elaboration or simulation to halt as described below and control to return to function **sc\_main** or to the kernel. The implementation shall print out a message from function **sc\_stop** to standard output to indicate that simulation has been halted by this means. The implementation shall make the **end\_of\_simulation** callbacks as described in 4.4.4.

If the function **sc\_stop** is called from one of the callbacks **before\_end\_of\_elaboration**, **end\_of\_elaboration**, **start\_of\_simulation**, or **end\_of\_simulation**, elaboration or simulation shall halt after the current callback phase is complete, that is, after all callbacks of the given kind have been made.

If the function **sc\_stop** is called during the evaluation phase or the update phase, the scheduler shall halt as determined by the current stop mode but in any case before the delta notification phase of the current delta cycle. If the current stop mode is **SC\_STOP\_FINISH\_DELTA**, the scheduler shall complete both the current evaluation phase and the current update phase before halting simulation. If the current stop mode is **SC\_STOP\_IMMEDIATE** and function **sc\_stop** is called during the evaluation phase, the scheduler shall complete the execution of the current process and shall then halt without executing any further processes and without executing the update phase. If function **sc\_stop** is called during the update phase, the scheduler shall complete the update phase before halting. Whatever the stop mode, simulation shall not halt until the currently executing process has yielded control to the scheduler (such as by calling function **wait** or by executing a return statement).

It shall be an error for the application to call function **sc\_start** after function **sc\_stop** has been called.

If function **sc\_stop** is called a second time before or after elaboration or simulation has halted, the implementation shall issue a warning. If function **stop\_after** of class **sc\_report\_handler** has been used to cause **sc\_stop** to be called on the occurrence of a warning, the implementation shall override this report-handling mechanism and shall not make further calls to **sc\_stop**, preventing an infinite regression.

NOTE 1—A function **sc\_stop** shall be provided by the implementation, whether or not the implementors choose to provide a function **sc\_start**.

NOTE 2—Throughout this standard, the term *call* is taken to mean call directly or indirectly. Hence function **sc\_stop** may be called indirectly, for example, by an interface method call.

#### 4.5.3 Function **sc\_time\_stamp**

The implementation shall provide a function **sc\_time\_stamp** with the following declaration:

```
const sc_time& sc_time_stamp();
```

The function **sc\_time\_stamp** shall return the current simulation time. During elaboration and initialization the function shall return a value of zero.

NOTE—The simulation time can only be modified by the scheduler.

#### 4.5.4 Function **sc\_delta\_count**

The implementation shall provide a function **sc\_delta\_count** with the following declaration:

```
const sc_dt::uint64 sc_delta_count();
```

The function **sc\_delta\_count** shall return an integer value that is incremented exactly once in each delta cycle, and thus returns a count of the absolute number of delta cycles that have occurred during simulation, starting from zero. When the delta count reaches the maximum value of type **sc\_dt::uint64**, the count shall start again from zero. Hence the delta count in successive delta cycles might be maxvalue-1, maxvalue, 0, 1, 2, and so on.

NOTE—This function is intended for use in primitive channels to detect whether an event has occurred by comparing the delta count with the delta count stored in a variable from an earlier delta cycle. The following code fragment will test whether a process has been executed in two consecutive delta cycles:

```
if (sc_delta_count() == stored_delta_count + 1) { /* consecutive */ }
stored_delta_count = sc_delta_count();
```

#### 4.5.5 Function **sc\_is\_running**

The implementation shall provide a function **sc\_is\_running** with the following declaration:

```
bool sc_is_running();
```

The function **sc\_is\_running** shall return the value **true** while the scheduler is running, including the initialization phase, and shall return the value **false** during elaboration, during the callbacks **start\_of\_simulation** and **end\_of\_simulation** and when called from the destructor of any object in the module hierarchy.

## 5. Core language class definitions

### 5.1 Class header files

To use the SystemC class library features, an application shall include either of the C++ header files specified in this subclause at appropriate positions in the source code as required by the scope and linkage rules of C++.

#### 5.1.1 `#include "systemc"`

The header file named **systemc** shall add the names **sc\_core** and **sc\_dt** to the declarative region in which it is included, and these two names only. The header file **systemc** shall not introduce into the declarative region in which it is included any other names from this standard or any names from the standard C or C++ libraries.

It is recommended that applications include the header file **systemc** rather than the header file **systemc.h**.

*Example:*

```
#include "systemc"
using sc_core::sc_module;
using sc_core::sc_signal;
using sc_core::SC_NS;
using sc_core::sc_start;
using sc_dt::sc_logic;
```

```
#include <iostream>
using std::ofstream;
using std::cout;
using std::endl;
```

#### 5.1.2 `#include "systemc.h"`

The header file named **systemc.h** shall add all of the names from the namespaces **sc\_core** and **sc\_dt** to the declarative region in which it is included, together with selected names from the standard C or C++ libraries as defined in this subclause. It is recommended that an implementation keep to a minimum the number of additional implementation-specific names introduced by this header file.

The header file **systemc.h** is provided for backward compatibility with earlier versions of SystemC and may be deprecated in future versions of this standard.

The header file **systemc.h** shall include at least the following:

```
#include "systemc"

// Using declarations for all the names in the sc_core namespace specified in this standard
using sc_core::sc_module;
...

// Using declarations for all the names in the sc_dt namespace specified in this standard
using sc_dt::sc_int;
...
```

```
// Using declarations for selected names in the standard libraries
using std::ios;
using std::streambuf;
using std::streampos;
using std::streamsize;
using std::iostream;
using std::istream;
using std::ostream;
using std::cin;
using std::cout;
using std::cerr;
using std::endl;
using std::flush;
using std::dec;
using std::hex;
using std::oct;
using std::fstream;
using std::ifstream;
using std::ofstream;
using std::size_t;
using std::memchr;
using std::memcmp;
using std::memcpy;
using std::memmove;
using std::memset;
using std::strcat;
using std::strncat;
using std::strchr;
using std::strrchr;
using std::strcmp;
using std::strncmp;
using std::strcpy;
using std::strncpy;
using std::strcspn;
using std::strspn;
using std::strlen;
using std::strpbrk;
using std::strstr;
using std::strtok;
```



## 5.2 sc\_module

### 5.2.1 Description

Class **sc\_module** is the base class for modules. Modules are the principle structural building blocks of SystemC.

### 5.2.2 Class definition

```
namespace sc_core {

class sc_bind_proxy† { implementation-defined };
const sc_bind_proxy† SC_BIND_PROXY_NIL;

class sc_module
: public sc_object
{
public:
    virtual ~sc_module();

    virtual const char* kind() const;

    void operator() ( const sc_bind_proxy†& p001,
                     const sc_bind_proxy†& p002 = SC_BIND_PROXY_NIL,
                     const sc_bind_proxy†& p003 = SC_BIND_PROXY_NIL,
                     ...
                     const sc_bind_proxy†& p063 = SC_BIND_PROXY_NIL,
                     const sc_bind_proxy†& p064 = SC_BIND_PROXY_NIL );

    virtual const std::vector<sc_object*>& get_child_objects() const;

protected:
    sc_module( const sc_module_name& );
    sc_module();

    void reset_signal_is( const sc_in<bool>& , bool );
    void reset_signal_is( const sc_signal<bool>& , bool );

    sc_sensitive† sensitive;

    void dont_initialize();
    void set_stack_size( size_t );

    void next_trigger();
    void next_trigger( const sc_event& );
    void next_trigger( sc_event_or_list†& );
    void next_trigger( sc_event_and_list†& );
    void next_trigger( const sc_time& );
    void next_trigger( double , sc_time_unit );
    void next_trigger( const sc_time& , const sc_event& );
    void next_trigger( double , sc_time_unit , const sc_event& );
    void next_trigger( const sc_time& , sc_event_or_list†& );
    void next_trigger( double , sc_time_unit , sc_event_or_list†& );
};
```

```

void next_trigger( const sc_time& , const sc_event_and_list†& );
void next_trigger( double , sc_time_unit , sc_event_and_list†& );

void wait();
void wait( int );
void wait( const sc_event& );
void wait( sc_event_or_list†& );
void wait( sc_event_and_list†& );
void wait( const sc_time& );
void wait( double , sc_time_unit );
void wait( const sc_time& , const sc_event& );
void wait( double , sc_time_unit , const sc_event& );
void wait( const sc_time& , sc_event_or_list†& );
void wait( double , sc_time_unit , sc_event_or_list†& );
void wait( const sc_time& , sc_event_and_list†& );
void wait( double , sc_time_unit , sc_event_and_list†& );

virtual void before_end_of_elaboration();
virtual void end_of_elaboration();
virtual void start_of_simulation();
virtual void end_of_simulation();
};

void next_trigger();
void next_trigger( const sc_event& );
void next_trigger( sc_event_or_list†& );
void next_trigger( sc_event_and_list†& );
void next_trigger( const sc_time& );
void next_trigger( double , sc_time_unit );
void next_trigger( const sc_time& , const sc_event& );
void next_trigger( double , sc_time_unit , const sc_event& );
void next_trigger( const sc_time& , sc_event_or_list†& );
void next_trigger( double , sc_time_unit , sc_event_or_list†& );
void next_trigger( const sc_time& , const sc_event_and_list†& );
void next_trigger( double , sc_time_unit , sc_event_and_list†& );

void wait();
void wait( int );
void wait( const sc_event& );
void wait( sc_event_or_list†& );
void wait( sc_event_and_list†& );
void wait( const sc_time& );
void wait( double , sc_time_unit );
void wait( const sc_time& , const sc_event& );
void wait( double , sc_time_unit , const sc_event& );
void wait( const sc_time& , sc_event_or_list†& );
void wait( double , sc_time_unit , sc_event_or_list†& );
void wait( const sc_time& , sc_event_and_list†& );
void wait( double , sc_time_unit , sc_event_and_list†& );

#define SC_MODULE(name)      struct name : sc_module
#define SC_CTOR(name)        implementation-defined; name(sc_module_name)
#define SC_HAS_PROCESS(name) implementation-defined
#define SC_METHOD(name)      implementation-defined

```

```
#define SC_THREAD(name)      implementation-defined
#define SC_CTHREAD(name,clk) implementation-defined

const char* sc_gen_unique_name( const char* );

typedef sc_module sc_behavior;
typedef sc_module sc_channel;

}      // namespace sc_core
```

### 5.2.3 Constraints on usage

Objects of class **sc\_module** can only be constructed during elaboration. It shall be an error to instantiate a module during simulation.

Every class derived (directly or indirectly) from class **sc\_module** shall have at least one constructor. Every such constructor shall have one and only one parameter of class **sc\_module\_name** but may have further parameters of classes other than **sc\_module\_name**. That parameter is not required to be the first parameter of the constructor.

A string-valued argument shall be passed to the constructor of every module instance. It is good practice to make this string name the same as the C++ variable name through which the module is referenced, if such a variable exists.

Inter-module communication should typically be accomplished using interface method calls, that is, a module should communicate with its environment through its ports. Other communication mechanisms are permissible, for example, for debugging or diagnostic purposes.

NOTE 1—Because the constructors are protected, class **sc\_module** cannot be instantiated directly but may be used as a base class.

NOTE 2—A module should be *publicly* derived from class **sc\_module**.

NOTE 3—It is permissible to use class **sc\_module** as an indirect base class. In other words, a module can be derived from another module. This can be a useful coding idiom.

### 5.2.4 kind

Member function **kind** shall return the string "**sc\_module**".

### 5.2.5 SC\_MODULE

The macro SC\_MODULE may be used to prefix the definition of a module, but the use of this macro is not obligatory.

*Example:*

// The following two class definitions are equally acceptable.

```
SC_MODULE(M)
{
    M(sc_module_name) {}
    ...
};
```

```
class M: public sc_module
{
    public:
        M(sc_module_name) {}
        ...
};
```

### 5.2.6 Constructors

```
sc_module( const sc_module_name& );
sc_module();
```

Module names are managed by class **sc\_module\_name**, not by class **sc\_module**. The string name of the module instance is initialized using the value of the string name passed as an argument to the constructor of the class **sc\_module\_name** (see 5.3).

### 5.2.7 SC\_CTOR

This macro is provided for convenience when declaring or defining a constructor of a module. Macro SC\_CTOR shall only be used at a place where the rules of C++ permit a constructor to be declared and can be used as the declarator of a constructor declaration or a constructor definition. The name of the module class being constructed shall be passed as the argument to the macro.

*Example:*

```
SC_MODULE(M1)
{
    SC_CTOR(M1) // Constructor definition
    : i(0)
    {}
    int i;
    ...
};

SC_MODULE(M2)
{
    SC_CTOR(M2); // Constructor declaration
    int i;
    ...
};

M2::M2(sc_module_name) : i(0) {}
```

The use of macro SC\_CTOR is not obligatory. Using SC\_CTOR, it is not possible to add user-defined arguments to the constructor. If an application needs to pass additional arguments, the constructor shall be provided explicitly. This is a useful coding idiom.

NOTE 1—The macros SC\_CTOR and SC\_MODULE may be used in conjunction or may be used separately.

NOTE 2—Since macro SC\_CTOR is equivalent to declaring a constructor for a module, an implementation shall ensure that the constructor so declared has a parameter of type **sc\_module\_name**.

NOTE 3—If process macros are invoked but macro SC\_CTOR is not used, macro SC\_HAS\_PROCESS shall be used instead (see 5.2.8).

*Example:*

```
SC_MODULE(M)
{
    M(sc_module_name n, int a, int b)    // Additional constructor parameters
    : sc_module(n)
    {}
    ...
};
```

### 5.2.8 SC\_HAS\_PROCESS

Macro `SC_CTOR` includes definitions used by the macros `SC_METHOD`, `SC_THREAD` and `SC_CTHREAD`. These same definitions are introduced by the macro `SC_HAS_PROCESS`. If a process macro is invoked from the constructor body of a module but macro `SC_CTOR` is not used within the module class definition, macro `SC_HAS_PROCESS` shall be invoked within the class definition or the constructor body of the module. If a process macro is invoked from the **before\_end\_of\_elaboration** or **end\_of\_elaboration** callbacks of a module but macro `SC_CTOR` is not used within the module class definition, macro `SC_HAS_PROCESS` shall be invoked within the class definition of the module or from that same callback.

Macro `SC_HAS_PROCESS` shall only be used within the class definition, constructor body, or member function body of a module. The name of the module class being constructed shall be passed as the argument to the macro.

NOTE—The use of the macros `SC_CTOR` and `SC_HAS_PROCESS` is not required in order to call the function `sc_spawn`.

*Example:*

```
SC_MODULE(M)
{
    M(sc_module_name n)    // SC_CTOR not used
    : sc_module(n)
    {
        SC_THREAD(T);    // Process macro
    }
    void T();

    SC_HAS_PROCESS(M);    // Necessary
    ...
};
```

### 5.2.9 SC\_METHOD, SC\_THREAD, SC\_CTHREAD

The argument passed to the macro `SC_METHOD` or `SC_THREAD` or the first argument passed to `SC_CTHREAD` shall be the name of a member function. The macro shall associate that function with a *method process instance*, a *thread process instance*, or a *clocked thread process instance*, respectively. This shall be the only way in which an unspawned process instance can be created (see 4.1.2).

The second argument passed to the macro `SC_CTHREAD` shall be an expression of the type `sc_event_finder`.

These three macros shall only be invoked in the body of the constructor, in the **before\_end\_of\_elaboration** or **end\_of\_elaboration** callbacks of a module, or in a member function called from the constructor or callback. Macro **SC\_CTHREAD** shall not be invoked from the **end\_of\_elaboration** callback. The first argument shall be the name of a member function of that same module.

A member function associated with an unspawned process instance shall have a return type of **void**, and shall have no arguments. (Note that a function associated with a spawned process instance may have a return type and may have arguments.)

A single member function can be associated with multiple process instances within the same module. Each process instance is a distinct object of a class derived from class **sc\_object**, and each macro shall use the member function name (in quotation marks) as the string name ultimately passed as an argument to the constructor of the base class sub-object of class **sc\_object**. Each process instance can have its own static sensitivity and shall be triggered or resumed independently of other process instances.

Associating a member function with a process instance does not impose any explicit restrictions on how that member function may be used by the application. For example, such a function may be called directly by the application, as well as being called by the kernel.

*Example:*

```
SC_MODULE(M)
{
    sc_in<bool> clk;

    SC_CTOR(M)
    {
        SC_METHOD(a_method);
        SC_THREAD(a_thread);
        SC_CTHREAD(a_ctypead, clk.pos());
    }
    void a_method();
    void a_thread();
    void a_ctypead();
    ...
};
```

### 5.2.10 Method process

This subclause shall apply to both spawned and unspawned process instances.

A method process is said to be *triggered* when the kernel calls the function associated with the process instance. When a method process is triggered, the associated function executes from beginning to end, then returns control to the kernel. A method process cannot be *terminated*.

A method process instance may have static sensitivity. A method process, and only a method process, may call the function **next\_trigger** to create dynamic sensitivity. Function **next\_trigger** is a member function of class **sc\_module**, a member function of class **sc\_prim\_channel**, and a non-member function.

An implementation is not obliged to run a method process in a separate software thread. A method process may run in the same execution context as the simulation kernel.

NOTE 1—Any local variables declared within the process will be destroyed on return from the process. Data members of the module should be used to store persistent state associated with the method process.

NOTE 2—Function **next\_trigger** can be called from a member function of the module itself, from a member function of a channel, or from any function subject only to the rules of C++, provided that the function is ultimately called from a method process.

### 5.2.11 Thread and clocked thread processes

This subclause shall apply to both spawned and unspawned process instances.

A function associated with a thread or clocked thread process instance is called once and only once by the kernel, except when a clocked thread process is reset, in which case the associated function may be called again (see 5.2.12).

A thread or clocked thread process, and only such a process, may call the function **wait**. Such a call causes the calling process to suspend execution. Function **wait** is a member function of class **sc\_module**, a member function of class **sc\_prim\_channel**, and a non-member function.

A thread or clocked thread process instance is said to be *resumed* when the kernel causes the process to continue execution, starting with the statement immediately following the most recent call to function **wait**. When a thread or clocked thread process is resumed, the process executes until it reaches the next call to function **wait**. Then, the process is suspended once again.

A thread process instance may have static sensitivity. A thread process instance may call function **wait** to create dynamic sensitivity. A clocked thread process instance is statically sensitive only to a single clock.

Each thread or clocked thread process requires its own execution stack. As a result, context switching between thread processes may impose a simulation overhead when compared with method processes.

If the thread or clocked thread process executes the entire function body or executes a return statement and thus returns control to the kernel, the associated function shall not be called again for that process instance. The process instance is then said to be *terminated*.

NOTE 1—It is a common coding idiom to include an infinite loop containing a call to function **wait** within a thread or clocked thread process in order to prevent the process from terminating prematurely.

NOTE 2—When a process instance is resumed, any local variables defined within the process will retain the values they had when the process was suspended.

NOTE 3—If a thread or clocked thread process executes an infinite loop that does not call function **wait**, the process will never suspend. Since the scheduler is not pre-emptive, no other process will be able to execute.

NOTE 4—Function **wait** can be called from a member function of the module itself, from a member function of a channel, or from any function subject only to the rules of C++, provided that the function is ultimately called from a thread or clocked thread process.

*Example:*

```
SC_MODULE(synchronous_module)
{
    sc_in<bool> clock;

    SC_CTOR(synchronous_module)
    {
        SC_THREAD(thread);
        sensitive << clock.pos();
    }
    void thread()      // Member function called once only
    {
        for (;;)
        {
            wait();    // Resume on positive edge of clock
            ...
        }
    }
    ...
};
```

## 5.2.12 Clocked thread processes and reset\_signal\_is

A clocked thread process shall be a static process; clocked threads cannot be spawned processes.

A clocked thread process shall be statically sensitive to a single clock, as determined by the event finder passed as the second argument to macro `SC_CTHREAD`. The clocked thread process shall be statically sensitive to the event returned from the given event finder.

A clocked thread process may call either of the following functions:

```
void wait();
void wait( int );
```

It shall be an error for a clocked thread process to call any other overloaded form of the function **wait**.

```
void reset_signal_is( const sc_in<bool>& , bool );
void reset_signal_is( const sc_signal<bool>& , bool );
```

Member function **reset\_signal\_is** of class **sc\_module** shall determine the reset signal of a clocked thread process.

**reset\_signal\_is** shall only be called in the body of the constructor, in the **before\_end\_of\_elaboration** callback of a module, or in a member function called from the constructor or callback, and only after having created a clocked thread process instance within that same constructor or callback.

The order of execution of the statements within the body of the constructor or the **before\_end\_of\_elaboration** callback shall be used to associate the call to **reset\_signal\_is** with a particular process instance; it is associated with the most recently created process instance. If a module is instantiated within the constructor or callback between the process being created and function **reset\_signal\_is** being called, the effect of calling **reset\_signal\_is** shall be undefined. It shall be an error to associate function **reset\_signal\_is** with a process instance that is not a clocked thread process.



The first argument passed to function **reset\_signal\_is** shall be the signal instance to be used as the reset (the signal may be identified indirectly by passing a port instance). The second argument shall be the active level of the reset, meaning that the clocked thread process shall be reset only when the value of the reset signal is equal to the value of this second argument.

A clocked thread process instance shall be reset when and only when the clock event to which the process instance is statically sensitive is notified and the reset signal is active. Resetting a clocked thread process instance shall consist of abandoning the current execution of the process instance, which shall have been suspended at a call to function **wait**, and calling the associated function again from the start of the function. A process instance being reset shall become runnable in the evaluation phase immediately following the delta notification phase or timed notification phase in which the clock event notification occurs. An active reset signal shall not cause the process to be reset in the absence of a clock event notification; in other words, the reset is synchronous with respect to the clock.

The first time the clock event is notified, the function associated with a clocked thread process shall be called whether or not the reset signal is active. If a clocked thread process instance has been terminated, the clock event shall be ignored for that process instance. A terminated process cannot be reset.

*Example:*

```
sc_in<bool> clock;
sc_in<bool> reset;

SC_CTOR(M)
{
    SC_CTHREAD(CT, clock.pos());
    reset_signal_is(reset, true);
}

void CT()
{
    if (reset)
    {
        ...           // Reset actions
    }
    while(true)
    {
        wait(1);       // Wait for 1 clock cycle
        ...           // Clocked actions
    }
}
```

### 5.2.13 sensitive

*sc\_sensitive*<sup>†</sup> sensitive;

This subclause describes the static sensitivity of an unspawned process. Static sensitivity for a spawned process is created using member function **set\_sensitivity** of class **sc\_spawn\_options** (see 5.5).

Data member **sensitive** of class **sc\_module** can be used to create the static sensitivity of an unspawned process instance using **operator<<** of class *sc\_sensitive*<sup>†</sup> (see 5.4). This shall be the only way to create static sensitivity for an unspawned process instance. However, static sensitivity may be enabled or disabled by calling function **next\_trigger** (see 5.2.16) or function **wait** (see 5.2.17).

Static sensitivity shall only be created in the body of the constructor, in the **before\_end\_of\_elaboration** or **end\_of\_elaboration** callbacks of a module, or in a member function called from the constructor or callback, and only after having created an unspawned process instance within that same constructor or callback. It shall be an error to modify the static sensitivity of a unspawned process during simulation.

The order of execution of the statements within the body of the constructor or the **before\_end\_of\_elaboration** or **end\_of\_elaboration** callbacks is used to associate static sensitivity with a particular unspawned process instance; sensitivity is associated with the process instance most recently created within the body of the current constructor or callback.

A clocked thread process cannot have static sensitivity other than to the clock itself. Using data member **sensitive** to create static sensitivity for a clocked thread process shall have no effect.

NOTE 1—Unrelated statements may be executed between creating an unspawned process instance and creating the static sensitivity for that same process instance. Static sensitivity may be created in a different function body from the one in which the process instance was created.

NOTE 2—Data member **sensitive** can be used more than once to add to the static sensitivity of any particular unspawned process instance; each call to **operator<<** adds further events to the static sensitivity of the most recently created process instance.

### 5.2.14 dont\_initialize

void **dont\_initialize**();

This subclause describes member function **dont\_initialize** of class **sc\_module**, which determines the behavior of an unspawned process instance during initialization. The initialization behavior of a spawned process is determined by the member function **dont\_initialize** of class **sc\_spawn\_options** (see 5.5).

Member function **dont\_initialize** of class **sc\_module** shall prevent a particular unspawned process instance from being made runnable during the initialization phase of the scheduler. In other words, the member function associated with the given process instance shall not be called by the scheduler until the process instance is triggered or resumed because of the occurrence of an event.

**dont\_initialize** shall only be called in the body of the constructor, in the **before\_end\_of\_elaboration** or **end\_of\_elaboration** callbacks of a module, or in a member function called from the constructor or callback, and only after having created an unspawned process instance within that same constructor or callback.

The order of execution of the statements within the body of the constructor or the **before\_end\_of\_elaboration** or **end\_of\_elaboration** callbacks is used to associate the call to **dont\_initialize** with a particular unspawned process instance; it is associated with the most recently created process instance. If a module is instantiated within the constructor or callback between the process being created and function **dont\_initialize** being called, the effect of calling **dont\_initialize** shall be undefined.

**dont\_initialize** shall have no effect if called for a clocked thread process, which is not made runnable during the initialization phase in any case. An implementation may generate a warning but is not obliged to do so.

*Example:*

```
SC_MODULE(Mod)
{
    sc_signal<bool> A, B, C, D, E;

    SC_CTOR(Mod)
    {
```

```

    sensitive << A;          // Has no effect. Poor coding style

    SC_THREAD(T);
    sensitive << B << C; // Thread process T is made sensitive to B and C.

    SC_METHOD(M);
    f();                  // Method process M is made sensitive to D.
    sensitive << E;        // Method process M is made sensitive to E as well as D.
    dont_initialize();     // Method process M is not made runnable during initialization.
}

void f() { sensitive << D; } // An unusual coding style

void T();
void M();
...
};

```

### 5.2.15 set\_stack\_size

```
void set_stack_size( size_t );
```

This subclause describes member function **set\_stack\_size** of class **sc\_module**, which sets the stack size of an unspawned process instance during initialization. The stack size of a spawned process is set by the member function **set\_stack\_size** of class **sc\_spawn\_options** (see 5.5).

An application may call member function **set\_stack\_size** to request a change to the size of the execution stack for the thread or clocked thread process instance for which the function is called. The effect of this function is implementation-defined.

**set\_stack\_size** shall only be called in the body of the constructor, in the **before\_end\_of\_elaboration** or **end\_of\_elaboration** callbacks of a module, or in a member function called from the constructor or callback, and only after having created an unspawned process instance within that same constructor or callback. It shall be an error to call **set\_stack\_size** at other times or to call **set\_stack\_size** for a method process instance.

The order of execution of the statements within the body of the constructor or the **before\_end\_of\_elaboration** or **end\_of\_elaboration** callbacks is used to associate the call to **set\_stack\_size** with a particular unspawned process instance; it is associated with the most recently created unspawned process instance.

### 5.2.16 next\_trigger

This subclause shall apply to both spawned and unspawned process instances.

This subclause shall apply to member function **next\_trigger** of class **sc\_module**, member function **next\_trigger** of class **sc\_prim\_channel**, and non-member function **next\_trigger**.

The function **next\_trigger** shall set the dynamic sensitivity of the method process instance from which it is called for the very next occasion on which that process instance is triggered, and for that occasion only. The dynamic sensitivity is determined by the arguments passed to function **next\_trigger**.

If function **next\_trigger** is called more than once during a single execution of a particular method process instance, the last call to be executed shall prevail. The effects of earlier calls to function **next\_trigger** for that particular process instance shall be cancelled.

If function **next\_trigger** is not called during a particular execution of a method process instance, the method process instance shall next be triggered according to its static sensitivity.

A call to the function **next\_trigger** with one or more arguments shall override the static sensitivity of the process instance.

It shall be an error to call function **next\_trigger** from a thread or clocked thread process.

NOTE—The function **next\_trigger** does not suspend the method process instance; a method process cannot be suspended but always executes to completion before returning control to the kernel.

void **next\_trigger**();

The process shall be triggered on the static sensitivity. In the absence of static sensitivity for this particular process instance, the process shall not be triggered again during the current simulation.

void **next\_trigger**( const sc\_event& );

The process shall be triggered when the event passed as an argument is notified.

void **next\_trigger**( *sc\_event\_or\_list*<sup>†</sup>& );

The argument shall take the form of a list of events separated by the **operator|** of classes **sc\_event** and *sc\_event\_or\_list*<sup>†</sup>. The process shall be triggered when any one of the given events is notified. The occurrence or non-occurrence of the other events in the list shall have no effect on that particular triggering of the process.

void **next\_trigger**( *sc\_event\_and\_list*<sup>†</sup>& );

The argument shall take the form of a list of events separated by the **operator&** of classes **sc\_event** and *sc\_event\_and\_list*<sup>†</sup>. In order for the process to be triggered, every single one of the given events shall be notified, with no explicit constraints on the time or order of those notifications. The process is triggered when the last such event is notified, last in the sense of being at the latest point in simulation time, not last in the list. An event in the list may be notified more than once before the last event is notified.

void **next\_trigger**( const sc\_time& );

The process shall be triggered after the time given as an argument has elapsed. The time shall be taken to be relative to the time at which function **next\_trigger** is called. When a process is triggered in this way, a *time-out* is said to have occurred.

void **next\_trigger**( double v , sc\_time\_unit tu );

is equivalent to the following:

void next\_trigger( sc\_time( v , tu ) );

void **next\_trigger**( const sc\_time& , const sc\_event& );

The process shall be triggered after the given time or when the given event is notified, whichever occurs first.

void **next\_trigger**( double , sc\_time\_unit , const sc\_event& );

void **next\_trigger**( const sc\_time& , *sc\_event\_or\_list*<sup>†</sup>& );

void **next\_trigger**( double , sc\_time\_unit , *sc\_event\_or\_list*<sup>†</sup>& );

void **next\_trigger**( const sc\_time& , const *sc\_event\_and\_list*<sup>†</sup>& );

```
void next_trigger( double , sc_time_unit , sc_event_and_list†& );
```

Each of these compound forms combines a time with an event or event list. The semantics of these compound forms shall be deduced from the rules given for the simple forms. In each case, the process shall be triggered after the given time-out or in response to the given event or event list, whichever is satisfied first.

*Example:*

```
SC_MODULE(M)
{
    SC_CTOR(M)
    {
        SC_METHOD(entry);
        sensitive << sig;
    }
    void entry()                                // Run first at initialization.
    {
        if (sig == 0)    next_trigger(e1 | e2);    // Trigger on event e1 or event e2 next time
        else if (sig == 1) next_trigger(1, SC_NS); // Time-out after 1 nanosecond.
        else            next_trigger();           // Trigger on signal sig next time.
    }
    sc_signal<int> sig;
    sc_event e1, e2;
    ...
};
```

### 5.2.17 wait

This subclause shall apply to both spawned and unspawned process instances.

In addition to causing the process instance to suspend, the function **wait** may set the dynamic sensitivity of the thread or clocked thread process instance from which it is called for the very next occasion on which that process instance is resumed, and for that occasion only. The dynamic sensitivity is determined by the arguments passed to function **wait**.

A call to the function **wait** with an empty argument list or with a single integer argument shall use the static sensitivity of the process instance. This is the only form of **wait** permitted within a clocked thread process.

A call to the function **wait** with one or more non-integer arguments shall override the static sensitivity of the process instance.

When calling function **wait** with a passed-by-reference parameter, the application shall be obliged to ensure that the lifetimes of any actual arguments passed by reference extend from the time the function is called to the time the function call reaches completion, and moreover in the case of a parameter of type **sc\_time**, the application shall not modify the value of the actual argument during that period.

It shall be an error to call function **wait** from a method process.

```
void wait();
```

The process shall be resumed on the static sensitivity. In the absence of static sensitivity for this particular process, the process shall not be resumed again during the current simulation.

void **wait**( int );

A call to this function shall be equivalent to calling the function **wait** with an empty argument list for a number of times in immediate succession, the number of times being passed as the value of the argument. It shall be an error to pass an argument value less than or equal to zero. The implementation is expected to optimize the execution speed of this function for clocked thread processes.

void **wait**( const sc\_event& );

The process shall be resumed when the event passed as an argument is notified.

void **wait**( sc\_event\_or\_list<sup>†</sup>& );

The argument shall take the form of a list of events separated by the **operator|** of classes **sc\_event** and **sc\_event\_or\_list<sup>†</sup>**. The process shall be resumed when any one of the given events is notified. The occurrence or non-occurrence of the other events in the list shall have no effect on the resumption of that particular process. If a particular event appears more than once in the list, the behavior shall be the same as if it appeared only once (see 5.8).

void **wait**( sc\_event\_and\_list<sup>†</sup>& );

The argument shall take the form of a list of events separated by the **operator&** of classes **sc\_event** and **sc\_event\_and\_list<sup>†</sup>**. In order for the process to be resumed, every single one of the given events shall be notified, with no explicit constraints on the time or order of those notifications. The process is resumed when the last such event is notified, last in the sense of being at the latest point in simulation time, not last in the list. An event in the list may be notified more than once before the last event is notified. If a particular event appears more than once in the list, the behavior shall be the same as if it appeared only once (see 5.8).

void **wait**( const sc\_time& );

The process shall be resumed after the time given as an argument has elapsed. The time shall be taken to be relative to the time at which function **wait** is called. When a process is resumed in this way, a *time-out* is said to have occurred.

void **wait**( double v , sc\_time\_unit tu );

is equivalent to the following:

void wait( sc\_time( v, tu ) );

void **wait**( const sc\_time& , const sc\_event& );

The process shall be resumed after the given time or when the given event is notified, whichever occurs first.

void **wait**( double , sc\_time\_unit , const sc\_event& );

void **wait**( const sc\_time& , sc\_event\_or\_list<sup>†</sup>& );

void **wait**( double , sc\_time\_unit , sc\_event\_or\_list<sup>†</sup>& );

void **wait**( const sc\_time& , const sc\_event\_and\_list<sup>†</sup>& );

void **wait**( double , sc\_time\_unit , sc\_event\_and\_list<sup>†</sup>& );

Each of these compound forms combines a time with an event or event list. The semantics of these compound forms shall be deduced from the rules given for the simple forms. In each case, the

process shall be resumed after the given time-out or in response to the given event or event list, whichever is satisfied first.

### 5.2.18 Positional port binding

Ports can be bound using either positional binding or named binding. Positional binding is performed using the **operator()** defined in the current subclause. Named binding is performed using the **operator()** or the function **bind** of the class **sc\_port** (see 5.11).

```
void operator() (
    const sc_bind_proxy† & p001,
    const sc_bind_proxy† & p002 = SC_BIND_PROXY_NIL,
    ...
    const sc_bind_proxy† & p063 = SC_BIND_PROXY_NIL,
    const sc_bind_proxy† & p064 = SC_BIND_PROXY_NIL );
```

This operator shall bind the port instances within the module instance for which the operator is called to the channel instances and port instances passed as actual arguments to the operator, the port order being determined by the order in which the ports were constructed. The first port to be constructed shall be bound to the first argument, the second port to the second argument, and so forth. It shall be an error if the number of actual arguments is greater than the number of ports to be bound.

A multiport instance (see 5.11.3) shall be treated as a single port instance when positional binding is used and may only be bound once, to a single channel instance or port instance. However, if a multiport instance P is bound by position to another multiport instance Q, the child multiport P may be bound indirectly to more than one channel through the parent multiport Q. A given multiport shall not be bound both by position and by name.

This operator shall only bind ports, not exports. Any export instances contained within the module instance shall be ignored by this operator.

An implementation may permit more than 64 ports to be bound in a single call to **operator()** by allowing more than 64 arguments but is not obliged to do so. **operator()** shall not be called more than once for a given module instance.

The following objects, and these alone, can be used as actual arguments to **operator()**:

- a) A channel, which is an object of a class derived from class **sc\_interface**
- b) A port, which is an object of a class derived from class **sc\_port**

The *type of a port* is the name of the interface passed as a template argument to class **sc\_port** when the port is instantiated. The interface implemented by the channel in case a) or the type of the port in case b) shall be the same as or derived from the type of the port being bound.

An implementation may defer the completion of port binding until a later time during elaboration because the port to which a port is bound may not yet itself have been bound. Such deferred port binding shall be completed by the implementation before the callbacks to function **end\_of\_elaboration**.

NOTE 1—To bind more than 64 ports of a single module instance, named binding should be used.

NOTE 2—Class *sc\_bind\_proxy†*, the parameter type of **operator()**, may provide user-defined conversions in the form of two constructors, one having a parameter type of **sc\_interface**, and the other a parameter type of **sc\_port\_base**.

NOTE 3—The actual argument cannot be an export, because this would require the C++ compiler to perform two implicit conversions. However, it is possible to pass an export as an actual argument by explicitly calling the user-defined conversion **sc\_export::operator IF&**. It is also possible to bind a port to an export using named port binding.

*Example:*

```
SC_MODULE(M1)
{
    sc_inout<int> P, Q, R; // Ports
    ...
};

SC_MODULE(Top1)
{
    sc_inout <int> A, B;
    sc_signal<int> C;
    M1 m1;           // Module instance
    SC_CTOR(Top1)
    : m1("m1")
    {
        m1(A, B, C); // Binds P-to-A, Q-to-B, R-to-C
    }
    ...
};

SC_MODULE(M2)
{
    sc_inout<int> S;
    sc_inout<int> *T; // Pointer-to-port (an unusual coding style)
    sc_inout<int> U;
    SC_CTOR(M2) { T = new sc_inout<int>; }
    ...
};

SC_MODULE(Top2)
{
    sc_inout <int> D, E;
    sc_signal<int> F;
    M2 m2;           // Module instance
    SC_CTOR(Top2)
    : m2("m2")
    {
        m2(D, E, F); // Binds S-to-D, U-to-E, (*T)-to-F
                      // Note that binding order depends on the order of port construction
    }
    ...
};
```

### 5.2.19 before\_end\_of\_elaboration, end\_of\_elaboration, start\_of\_simulation, end\_of\_simulation

See 4.4.



### 5.2.20 get\_child\_objects

```
virtual const std::vector<sc_object*>& get_child_objects() const;
```

Member function **get\_child\_objects** shall return a **std::vector** containing a pointer to every instance of class **sc\_object** that lies within the module in the object hierarchy. This shall include pointers to all module, port, primitive channel, unspawned process, and spawned process instances within the module and any other application-defined objects derived from class **sc\_object** within the module.

NOTE 1—The phrase *within a module* does not include instances nested within modules instances but only includes the immediate children of the given module.

NOTE 2—An application can identify the instances by calling the member functions **name** and **kind** of class **sc\_object** or can determine their types using a dynamic cast.

*Example:*

```
int sc_main (int argc, char* argv[])
{
    Top_level_module top("top");

    std::vector<sc_object*> children = top.get_child_objects();

    // Print out names and kinds of top-level objects
    for (unsigned i = 0; i < children.size(); i++)
        std::cout << children[i]->name() << " " << children[i]->kind() << std::endl;

    sc_start();
    return 0;
}
```

### 5.2.21 sc\_gen\_unique\_name

```
const char* sc_gen_unique_name( const char* seed );
```

The function **sc\_gen\_unique\_name** shall return a unique character string that depends on the context from which the function is called. For this purpose, each module instance shall have a separate space of unique string names, and there shall be a single global space of unique string names for calls to **sc\_gen\_unique\_name** not made from within any module. These spaces of unique string names shall be maintained by function **sc\_gen\_unique\_name** and are only visible outside this function in so far as they affect the value of the strings returned from this function. Function **sc\_gen\_unique\_name** shall only guarantee the uniqueness of strings within each space of unique string names. There shall be no guarantee that the generated name does not clash with a string that was not generated by function **sc\_gen\_unique\_name**.

The unique string shall be constructed by appending a string of two or more characters as a suffix to the character string passed as argument **seed**, subject to the rules given in the remainder of this subclause. The appended suffix shall take the form of a single underscore character, followed by a series of one or more decimal digits from the character set 0-9. The number and choice of digits shall be implementation-defined.

There shall be no restrictions on the character set of the **seed** argument to function **sc\_gen\_unique\_name**. The **seed** argument may be the empty string.

String names are case-sensitive, and every character in a string name is significant. For example, “a”, “A”, “a\_”, and “A\_” are each unique string names with respect to one another.

NOTE—The intended use of **sc\_gen\_unique\_name** is to generate unique string names for objects of class **sc\_object**. Class **sc\_object** does impose restrictions on the character set of string names passed as constructor arguments. The value returned from function **sc\_gen\_unique\_name** may be used for other unrelated purposes.

### 5.2.22 sc\_behavior and sc\_channel

```
typedef sc_module sc_behavior;  
typedef sc_module sc_channel;
```

The typedefs **sc\_behavior** and **sc\_channel** are provided for users to express their intent.

NOTE—There is no distinction between a *behavior* and a hierarchical channel other than a difference of intent. Either may include both ports and public member functions.

*Example:*

```
class bus_interface  
: virtual public sc_interface  
{  
    public:  
        virtual void write(int addr, int data) = 0;  
        virtual void read (int addr, int& data) = 0;  
};  
  
class bus_adapter  
: public bus_interface, public sc_channel  
{  
    public:  
        virtual void write(int addr, int data);           // Interface methods implemented in channel  
        virtual void read (int addr, int& data);  
  
        sc_in<bool> clock;                                // Ports  
        sc_out<bool> wr, rd;  
        sc_out<int>  addr_bus;  
        sc_out<int>  data_out;  
        sc_in <int>  data_in;  
  
        SC_CTOR(bus_adapter) { ... }                     // Module constructor  
  
    private:  
        ...  
};
```

## 5.3 **sc\_module\_name**

### 5.3.1 Description

Class **sc\_module\_name** acts as a container for the string name of a module and provides the mechanism for building the hierarchical names of instances in the module hierarchy during elaboration.

When an application creates an object of a class derived directly or indirectly from class **sc\_module**, the application typically passes an argument of type **char\*** to the module constructor, which itself has a single parameter of class **sc\_module\_name** and thus the constructor **sc\_module\_name( const char\* )** is called as an implicit conversion. On the other hand, when an application derives a new class directly or indirectly from class **sc\_module**, the derived class constructor calls the base class constructor with an argument of class **sc\_module\_name** and thus the copy constructor **sc\_module\_name( const sc\_module\_name& )** is called.

### 5.3.2 Class definition

```
namespace sc_core {  
  
class sc_module_name  
{  
    public:  
        sc_module_name( const char* );  
        sc_module_name( const sc_module_name& );  
  
        ~sc_module_name();  
  
        operator const char*() const;  
  
    private:  
        // Disabled  
        sc_module_name();  
        sc_module_name& operator=( const sc_module_name& );  
};  
  
} // namespace sc_core
```

### 5.3.3 Constraints on usage

Class **sc\_module\_name** shall only be used as the type of a parameter of a constructor of a class derived from class **sc\_module**. Moreover, every such constructor shall have exactly one parameter of type **sc\_module\_name**, which need not be the first parameter of the constructor.

In the case that the constructor of a class C derived directly or indirectly from class **sc\_module** is called from the constructor of a class D derived directly from class C, the parameter of type **sc\_module\_name** of the constructor of class D shall be passed directly through as an argument to the constructor of class C. In other words, the derived class constructor shall pass the **sc\_module\_name** through to the base class constructor as a constructor argument.

NOTE 1—The macro **SC\_CTOR** defines such a constructor.

NOTE 2—In the case of a class C derived directly from class **sc\_module**, the constructor for class C is not obliged to pass the **sc\_module\_name** through to the constructor for class **sc\_module**. The default constructor for class **sc\_module** may be called explicitly or implicitly from the constructor for class C.

### 5.3.4 Module hierarchy

To keep track of the module hierarchy during elaboration, the implementation may maintain an internal stack of pointers to objects of class **sc\_module\_name**, referred to below as *the stack*. For the purpose of building hierarchical names, when objects of class **sc\_module**, **sc\_port**, **sc\_export**, or **sc\_prim\_channel** are constructed or when spawned or unspawned processes instances are created, they are assumed to exist within the module identified by the **sc\_module\_name** object on the top of the stack. In other words, each instance in the module hierarchy is named as if it were a child of the module identified by the item on the top of the stack at the point when the instance is created.

NOTE 1—The *hierarchical name* of an instance in the object hierarchy is returned from member function **name** of class **sc\_object**, which is the base class of all such instances.

NOTE 2—The implementation is not obliged to use these particular mechanisms (a stack of pointers), but if not, the implementation shall substitute an alternative mechanism that is semantically equivalent.

### 5.3.5 Member functions

**sc\_module\_name**( const char\* );

This constructor shall push a pointer to the object being constructed onto the top of the stack. The constructor argument shall be used as the string name of the module being instantiated within the module hierarchy by ultimately being passed as an argument to the constructor of class **sc\_object**.

**sc\_module\_name**( const sc\_module\_name& );

This constructor shall copy the constructor argument but shall not modify the stack.

**~sc\_module\_name**();

If and only if the object being destroyed was constructed by **sc\_module\_name**( const char\* ), the destructor shall remove the **sc\_module\_name** pointer from the top of the stack.

**operator const char\***() const;

This conversion function shall return the string name (not the hierarchical name) associated with the **sc\_module\_name**.

NOTE 1—When a *complete object* of a class derived from **sc\_module** is constructed, the constructor for that derived class shall be passed an argument of type **char\***. The first constructor above will be called to perform an implicit conversion from type **char\*** to type **sc\_module\_name**, thus pushing the newly created module name onto the stack and signifying the entry into a new level in the module hierarchy. On return from the constructor for the class of the complete object, the destructor for class **sc\_module\_name** will be called and will remove the module name from the stack.

NOTE 2—When an **sc\_module\_name** is passed as an argument to the constructor of a base class, the above copy constructor is called. The **sc\_module\_name** parameter of the base class may be unused. The reason for mandating that every such constructor have a parameter of class **sc\_module\_name** (even if the parameter is unused) is to ensure that every such derived class can be instantiated as a module in its own right.

*Example:*

```
struct A: sc_module
{
    A(sc_module_name) {} // Calls sc_module()
};

struct B: sc_module
{
    B(sc_module_name n)
    : sc_module(n) {} // Calls sc_module(sc_module_name&)
};

struct C: B // One module derived from another
{
    C(sc_module_name n)
    : B(n) {} // Calls sc_module_name(sc_module_name&) then
              // B(sc_module_name)
};

struct Top: sc_module
{
    A a;
    C c;

    Top(sc_module_name n)
    : sc_module(n), // Calls sc_module(sc_module_name&)
      a("a"),      // Calls sc_module_name(char*) then calls A(sc_module_name)
      c("c") {}    // Calls sc_module_name(char*) then calls C(sc_module_name)
};
```

## 5.4 *sc\_sensitive*<sup>†</sup>

### 5.4.1 Description

Class *sc\_sensitive*<sup>†</sup> provides the operators used to build the static sensitivity of an unspawned process instance. To create static sensitivity for a spawned process, use the member function **set\_sensitivity** of the class **sc\_spawn\_options** (see 5.5).

### 5.4.2 Class definition

```
namespace sc_core {

class sc_sensitive†
{
public:
    sc_sensitive†& operator<< ( const sc_event& );
    sc_sensitive†& operator<< ( const sc_interface& );
    sc_sensitive†& operator<< ( const sc_port_base& );
    sc_sensitive†& operator<< ( sc_event_finder& );

    // Other members
    implementation-defined
};

} // namespace sc_core
```

### 5.4.3 Constraints on usage

An application shall not explicitly create an object of class *sc\_sensitive*<sup>†</sup>.

Class **sc\_module** shall have a data member named **sensitive** of type *sc\_sensitive*<sup>†</sup>. The use of **sensitive** to create static sensitivity is described in 5.2.13.

### 5.4.4 **operator**<<

*sc\_sensitive*<sup>†</sup>& **operator**<< ( const sc\_event& );

The event passed as an argument shall be added to the static sensitivity of the process instance.

*sc\_sensitive*<sup>†</sup>& **operator**<< ( const sc\_interface& );

The event returned by member function **default\_event** of the channel instance passed as an argument to **operator**<< shall be added to the static sensitivity of the process instance.

NOTE 1—If the channel passed as an argument does not override function **default\_event**, the member function **default\_event** of class **sc\_interface** is called through inheritance.

NOTE 2—An export can be passed as an actual argument to this operator because of the existence of the user-defined conversion **sc\_export<IF>::operator**.

*sc\_sensitive*<sup>†</sup>& **operator**<< ( const sc\_port\_base& );

The event returned by member function **default\_event** of the channel instance to which the port instance passed as an argument to **operator**<< is bound shall be added to the static sensitivity of the process instance. In other words, the process is made sensitive to the given port, calling function

**default\_event** to determine to which particular event it should be made sensitive. If the port instance is a multiport (see 5.11.3), the events returned by calling member function **default\_event** for each and every channel instance to which the multiport is bound shall be added to the static sensitivity of the process instance.

*sc\_sensitive*<sup>†</sup> & **operator**<< ( *sc\_event\_finder*& );

The event found by the event finder passed as an argument to **operator**<< shall be added to the static sensitivity of the process instance (see 5.7).

NOTE—An event finder is necessary to create static sensitivity when the application needs to select between multiple events defined in the channel. In a such a case the **default\_event** mechanism is inadequate.

## 5.5 **sc\_spawn\_options** and **sc\_spawn**

### 5.5.1 Description

Function **sc\_spawn** is used to create a static or dynamic spawned process instance.

Class **sc\_spawn\_options** is used to create an object that is passed as an argument to function **sc\_spawn** when creating a spawned process instance. The spawn options determine certain properties of the spawned process instance when used in this way. Calling the member functions of an **sc\_spawn\_options** object shall have no effect on any process instance unless the object is passed as an argument to **sc\_spawn**.

### 5.5.2 Class definition

```
namespace sc_core {

class sc_spawn_options
{
    public:
        sc_spawn_options();

        void spawn_method();
        void dont_initialize();
        void set_stack_size( int );

        void set_sensitivity( const sc_event* );
        void set_sensitivity( sc_port_base* );
        void set_sensitivity( sc_export_base* );
        void set_sensitivity( sc_interface* );
        void set_sensitivity( sc_event_finder* );

    private:
        // Disabled
        sc_spawn_options( const sc_spawn_options& );
        sc_spawn_options& operator=( const sc_spawn_options& );
};

template <typename T>
sc_process_handle sc_spawn(
    T object ,
    const char* name_p = 0 ,
    const sc_spawn_options* opt_p = 0 );

template <typename T>
sc_process_handle sc_spawn(
    typename T::result_type* r_p ,
    T object ,
    const char* name_p = 0 ,
    const sc_spawn_options* opt_p = 0 );

#define sc_bind    boost::bind
#define sc_ref(r)  boost::ref(r)
#define sc_cref(r) boost::cref(r)

#define SC_FORK implementation-defined
```



```
#define SC_JOIN implementation-defined

}      // namespace sc_core
```

### 5.5.3 Constraints on usage

Function **sc\_spawn** may be called during elaboration or from a static, dynamic, spawned, or unspawned process during simulation. Similarly, objects of class **sc\_spawn\_options** may be created or modified during elaboration or during simulation.

### 5.5.4 Constructors

**sc\_spawn\_options** ();

The default constructor shall create an object having the default values for the properties set by the functions **spawn\_method**, **dont\_initialize**, **set\_stack\_size**, and **set\_sensitivity**.

### 5.5.5 Member functions

void **spawn\_method**();

Member function **spawn\_method** shall set a property of the spawn options to indicate that the spawned process shall be a method process. The default is a thread process.

void **dont\_initialize**();

Member function **dont\_initialize** shall set a property of the spawn options to indicate that the spawned process instance shall not be made runnable during the initialization phase or when it is created. By default, this property is not set, and thus by default the spawned process instance shall be made runnable during the initialization phase of the scheduler if spawned during elaboration, or it shall be made runnable in the current or next evaluation phase if spawned during simulation irrespective of the static sensitivity of the spawned process instance. If the process is spawned during elaboration, member function **dont\_initialize** of class **sc\_spawn\_options** shall provide the same behavior for spawned processes as the member function **dont\_initialize** of class **sc\_module** provides for unspawned processes.

void **set\_stack\_size**( int );

Member function **set\_stack\_size** shall set a property of the spawn options to set the stack size of the spawned process. This member function shall provide the same behavior for spawned processes as the member function **set\_stack\_size** of class **sc\_module** provides for unspawned processes. The effect of calling this function is implementation-defined.

It shall be an error to call **set\_stack\_size** for a method process.

```
void set_sensitivity( const sc_event* );
void set_sensitivity( sc_port_base* );
void set_sensitivity( sc_export_base* );
void set_sensitivity( sc_interface* );
void set_sensitivity( sc_event_finder* );
```

Member function **set\_sensitivity** shall set a property of the spawn options to add the object passed as an argument to **set\_sensitivity** to the static sensitivity of the spawned process, as described for **operator<<** in 5.4.4, or if the argument is the address of an export, the process is made sensitive to the channel instance to which that export is bound. If the argument is the address of a multiport, the

process shall be made sensitive to the events returned by calling member function **default\_event** for each and every channel instance to which the multiport is bound. By default, the static sensitivity is empty. Calls to **set\_sensitivity** are cumulative: each call to **set\_sensitivity** extends the static sensitivity as set in the spawn options. Calls to the four different overloaded member functions can be mixed.

NOTE 1—There are no member functions to set the spawn options to spawn a thread process or to make a process runnable during initialization. This functionality is reliant on the default values of the **sc\_spawn\_options** object.

NOTE 2—It is not possible to spawn a dynamic clocked thread process.

### 5.5.6 sc\_spawn

```
template <typename T>
sc_process_handle sc_spawn(
    T object ,
    const char* name_p = 0 ,
    const sc_spawn_options* opt_p = 0 );

template <typename T>
sc_process_handle sc_spawn(
    typename T::result_type* r_p ,
    T object ,
    const char* name_p = 0 ,
    const sc_spawn_options* opt_p = 0 );

#define sc_bind    boost::bind
#define sc_ref(r)  boost::ref(r)
#define sc_cref(r) boost::cref(r)
```

Function **sc\_spawn** shall create a static or dynamic spawned process instance.

Function **sc\_spawn** may be called during elaboration, in which case the spawned process is a *child* of the module instance within which function **sc\_spawn** is called or is a top-level object if function **sc\_spawn** is called from function **sc\_main**.

Function **sc\_spawn** may be called during simulation, in which case the spawned process is a child of the process that called function **sc\_spawn**. Function **sc\_spawn** may be called from a method process, a thread process, or a clocked thread process.

The process or module from which **sc\_spawn** is called is the *parent* of the spawned process. Thus a set of dynamic process instances may have a hierarchical relationship, similar to the module hierarchy, which will be reflected in the hierarchical names of the process instances.

If function **sc\_spawn** is called during the evaluation phase, the spawned process shall be made runnable in the current evaluation phase (unless **dont\_initialize** has been called for this process instance). If function **sc\_spawn** is called during the update phase, the spawned process shall be made runnable in the very next evaluation phase (unless **dont\_initialize** has been called for this process instance).

The argument of type **T** shall be either a function pointer or a function object, that is, an object of a class that overloads **operator()** as a member function and shall specify the function associated with the spawned process instance, that is, the function to be spawned. This shall be the only mandatory argument to function **sc\_spawn**.

If present, the argument of type **T::result\_type\*** shall pass a pointer to a memory location that shall receive the value returned from the function associated with the process instance. In this case, the argument of type **T** shall be a function object of a class that exposes a nested type named **result\_type**. Furthermore, **operator()** of the function object shall have the return type **result\_type**. See the example below.

The macros **sc\_bind**, **sc\_ref**, and **sc\_cref** are provided for convenience when using the free Boost C++ libraries to bind arguments to spawned functions. Passing arguments to spawned processes is a powerful mechanism that allows processes to be parameterized when they are spawned and permits processes to update variables over time through reference arguments. **boost::bind** provides a convenient way to pass value arguments, reference arguments, and const reference arguments to spawned functions but its use is not mandatory. See the examples below and the Boost documentation available on the internet.

The argument of type **const char\*** shall give the string name of the spawned process instance and shall be passed by the implementation to the constructor for the **sc\_object** that forms the base class sub-object of the spawned process instance. If no such argument is given or if the argument is an empty string, the implementation shall create a string name for the process instance by calling function **sc\_gen\_unique\_name** with the seed string **"thread\_p"** in the case of a thread process or **"method\_p"** in the case of a method process.

The argument of type **sc\_spawn\_options\*** shall set the spawn options for the spawned process instance. If no such argument is provided, the spawned process instance shall take the default values as defined for the member functions of class **sc\_spawn\_options**. The application is not obliged to keep the **sc\_spawn\_options** object valid after the return from function **sc\_spawn**.

Function **sc\_spawn** shall return a process handle to the spawned process instance.

NOTE 1—Function **sc\_spawn** provides a superset of the functionality of the macros **SC\_THREAD** and **SC\_METHOD**. In addition to the functionality provided by these macros, function **sc\_spawn** provides the passing of arguments and return values to and from processes spawned during elaboration or simulation. The macros are retained for compatibility with earlier versions of SystemC.

NOTE 2—If a spawn options argument is given, a process string name argument shall also be given, although that string name argument may be an empty string.

*Example:*

```
int f();

struct Functor
{
    typedef int result_type;
    result_type operator() ();
};

Functor::result_type Functor::operator() () { return f(); }

int h(int a, int& b, const int& c);

struct MyMod: sc_module
{
    sc_signal<int> sig;
    void g();

    SC_CTOR(MyMod)
    {
```

```

    SC_THREAD(T);
}
int ret;
void T()
{
    sc_spawn(&f);           // Spawn a function without arguments and discard any return value.
                           // Spawn a similar process and create a process handle.
    sc_process_handle handle = sc_spawn(&f);

    Functor fr;
    sc_spawn(&ret, fr);      // Spawn a function object and catch the return value.

    sc_spawn_options opt;
    opt.spawn_method();
    opt.set_sensitivity(&sig);
    opt.dont_initialize();
    sc_spawn(f, "f1", &opt); // Spawn a method process named "f1", sensitive to sig, not initialized.
                           // Spawn a similar process named "f2" and catch the return value.
    sc_spawn(&ret, fr, "f2", &opt);

                           // Spawn a member function using Boost bind.
    sc_spawn(sc_bind(&MyMod::g, this));

    int A = 0, B, C;
                           // Spawn a function using Boost bind, pass arguments
                           // and catch the return value.
    sc_spawn(&ret, sc_bind(&h, A, sc_ref(B), sc cref(C)));
}
};

```

### 5.5.7 SC\_FORK and SC\_JOIN

```

#define SC_FORK implementation-defined
#define SC_JOIN implementation-defined

```

The macros SC\_FORK and SC\_JOIN can only be used as a pair to bracket a set of calls to function **sc\_spawn** from within a thread or clocked thread process. It is an error to use the fork-join construct in a method process. The implementation shall make each call to **sc\_spawn** immediately control enters the fork-join construct and shall spawn a separate process instance for each such call. In other words, the child processes shall be spawned without delay and may potentially all become runnable in the current evaluation phase (depending on their spawn options). The spawned process instances shall be thread processes. It is an error to spawn a method process within a fork-join construct. Control leaves the fork-join construct when all the spawned process instances have terminated.

The text between SC\_FORK and SC\_JOIN shall consist of a series of one or more calls to function **sc\_spawn** separated by commas. The comma after the final call to **sc\_spawn** and immediately before SC\_JOIN shall be optional. There shall be no other characters other than white space separating SC\_FORK, the function calls, the commas, and SC\_JOIN. If an application violates these rules, the effect shall be undefined.

*Example:*

```
SC_FORK
    sc_spawn( arguments ),
    sc_spawn( arguments ),
    sc_spawn( arguments )
SC_JOIN
```

## 5.6 sc\_process\_handle

### 5.6.1 Description

Class **sc\_process\_handle** provides a process handle to an underlying spawned or unspawned process instance. A process handle can be in one of two states: *valid* or *invalid*. A *valid* process handle shall be associated with a single underlying process instance, which may or may not be in the terminated state. An *invalid* process handle shall not be associated with any underlying process instance. A process instance may be associated with zero, one or many process handles, and the number and identity of such process handles may change over time.

Since dynamic process instances can be created and destroyed dynamically during simulation, it is in general unsafe to manipulate a process instance through a raw pointer to the process instance (or to the base class sub-object of class **sc\_object**). The purpose of class **sc\_process\_handle** is to provide a safe and uniform mechanism for manipulating both spawned and unspawned process instances without reliance on raw pointers. If control returns from the function associated with a thread process instance (that is, the process terminates), the underlying process instance may be deleted, but the process handle will continue to exist.

### 5.6.2 Class definition

```
namespace sc_core {

enum sc_curr_proc_kind
{
    SC_NO_PROC_,
    SC_METHOD_PROC_,
    SC_THREAD_PROC_,
    SC_CTHREAD_PROC_
};

class sc_process_handle
{
public:
    sc_process_handle();
    sc_process_handle( const sc_process_handle& );
    explicit sc_process_handle( sc_object* );
    ~sc_process_handle();

    bool valid() const;

    sc_process_handle& operator= ( const sc_process_handle& );
    bool operator== ( const sc_process_handle& ) const;
    bool operator!= ( const sc_process_handle& ) const;

    const char* name() const;
    sc_curr_proc_kind proc_kind() const;
    const std::vector<sc_object*>& get_child_objects() const;
    sc_object* get_parent_object() const;
    sc_object* get_process_object() const;
    bool dynamic() const;
    bool terminated() const;
    const sc_event& terminated_event() const;
};
```

```
sc_process_handle sc_get_current_process_handle();  
  
} // namespace sc_core
```

### 5.6.3 Constraints on usage

None. A process handle may be created, copied, or deleted at any time during elaboration or simulation. The handle may be valid or invalid.

### 5.6.4 Constructors

**sc\_process\_handle();**

The default constructor shall create an invalid process handle.

**sc\_process\_handle( const sc\_process\_handle& );**

The copy constructor shall duplicate the process handle passed as an argument. The result will be two handles to the same underlying process instance or two invalid handles.

**explicit sc\_process\_handle( sc\_object\* );**

If the argument is a pointer to a process instance, this constructor shall create a valid process handle to the given process instance. Otherwise, this constructor shall create an invalid process handle.

### 5.6.5 Member functions

**bool valid() const;**

Member function **valid** shall return **true** if and only if the process handle is valid.

**sc\_process\_handle& operator= ( const sc\_process\_handle& );**

The assignment operator shall duplicate the process handle passed as an argument. The result will be two handles to the same underlying process instance or two invalid handles.

**bool operator== ( const sc\_process\_handle& ) const;**

The equality operator shall return **true** if and only if the two process handles are both valid and share the same underlying process instance.

**bool operator!= ( const sc\_process\_handle& ) const;**

The inequality operator shall return **false** if and only if the two process handles are both valid and share the same underlying process instance.

**const char\* name() const;**

Member function **name** shall return the hierarchical name of the underlying process instance. If the process handle is invalid, member function **name** shall return an empty string. The implementation is only obliged to keep the returned string valid while the process handle is valid.

**sc\_curr\_proc\_kind proc\_kind() const;**

For a valid process handle, member function **proc\_kind** shall return one of the three values **SC\_METHOD\_PROC\_**, **SC\_THREAD\_PROC\_**, or **SC\_CTHREAD\_PROC\_**, depending on the

kind of the underlying process instance, that is, method process, thread process, or clocked thread process, respectively. For an invalid process handle, member function **proc\_kind** shall return the value **SC\_NO\_PROC\_**.

const std::vector<sc\_object\*>& **get\_child\_objects**() const;

Member function **get\_child\_objects** shall return a **std::vector** containing a pointer to every instance of class **sc\_object** that is a child of the underlying process instance. This shall include every dynamic process instance that was spawned during the execution of the underlying process instance and any other application-defined objects derived from class **sc\_object** created from the underlying process instance. Processes that are spawned from child processes are not included (grandchildren, as it were). If the process handle is invalid, member function **get\_child\_objects** shall return an empty **std::vector**.

This same function shall be overridden in any implementation-defined classes derived from **sc\_object** and associated with spawned and unspawned process instances. Such functions shall have identical behavior provided that the process handle is valid.

sc\_object\* **get\_parent\_object**() const;

Member function **get\_parent\_object** shall return a pointer to the module instance or process instance from which the underlying process instance was spawned. If the process handle is invalid, member function **get\_parent\_object** shall return the null pointer.

sc\_object\* **get\_process\_object**() const;

Member function **get\_process\_object** shall return a pointer to the process instance associated with the process handle. If the process handle is invalid, member function **get\_process\_object** shall return the null pointer. An application should test for a null pointer before dereferencing the pointer. Moreover, an application should assume that the pointer remains valid only until the calling process suspends.

bool **dynamic**() const;

Member function **dynamic** shall return **true** if the underlying process instance is a dynamic process and false if the underlying process instance is a static process. If the process handle is invalid, member function **dynamic** shall return the value **false**.

bool **terminated**() const;

Member function **terminated** shall return **true** if and only if the underlying process instance has *terminated*. A thread or clocked thread process is *terminated* after the point when control is returned from the associated function. A method process is never terminated, so member function **terminated** shall always return **false** for a method process. If the process handle is invalid, member function **terminated** shall return the value **false**.

When the underlying process instance terminates, an implementation may choose to invalidate any associated process handles but is not obliged to do so. In other words, when a process terminates, an implementation is neither obliged to keep the handle valid nor to invalidate the handle. If the process handle is valid, function **terminated** will return **true**, or if invalid, **terminated** will return **false**.

const sc\_event& **terminated\_event**() const;

Member function **terminated\_event** shall return a reference to an event that is notified when the underlying process instance terminates. If member function **terminated\_event** is called for a method process, the implementation shall generate a warning and the event shall never be notified. It



shall be an error to call member function **terminated\_event** for an invalid process handle and the event shall never be notified.

### 5.6.6 **sc\_get\_current\_process\_handle**

sc\_process\_handle **sc\_get\_current\_process\_handle**();

The value returned from function **sc\_get\_current\_process\_handle** shall depend on the context in which it is called. When called during elaboration from the body of a module constructor or from a function called from the body of a module constructor, **sc\_get\_current\_process\_handle** shall return a handle to the spawned or unspawned process instance most recently created within that module, if any. If the most recently created process instance was not within the current module, or if function **sc\_get\_current\_process\_handle** is called from one of the callbacks **before\_end\_of\_elaboration** or **end\_of\_elaboration**, an implementation may return either a handle to the most recently created process instance or an invalid handle. When called during simulation, **sc\_get\_current\_process\_handle** shall return a handle to the currently executing spawned or unspawned process instance, if any. If there is no such process instance, **sc\_get\_current\_process\_handle** shall return an invalid handle.

*Example:*

```
SC_MODULE(Mod)
{
    ...
    SC_CTOR(Mod)
    {
        SC_METHOD(Run);
        sensitive << in;
        sc_process_handle h1 = sc_get_current_process_handle(); // Returns a handle to process Run
    }
    void Run()
    {
        sc_process_handle h2 = sc_get_current_process_handle(); // Returns a handle to process Run
        if (h2.proc_kind() == SC_METHOD_PROC_)
            ... // Running a method process
        sc_object* parent = h2.get_parent_object(); // Returns a pointer to the
                                                    // module instance
        if (parent)
        {
            handle = sc_process_handle(parent); // Invalid handle - parent is not a process
            if (handle.valid())
                ... // Executed if parent were a
                  // valid process
        }
    }
    ...
};
```

## 5.7 **sc\_event\_finder** and **sc\_event\_finder\_t**

### 5.7.1 Description

An *event finder* is a member function of a port class with a return type of **sc\_event\_finder&**. When a port instance is bound to a channel instance containing multiple events, an event finder permits a specific event from the channel to be retrieved through the port instance and added to the static sensitivity of a process instance. **sc\_event\_finder\_t** is a templated wrapper for class **sc\_event\_finder**, where the template parameter is the interface type of the port.

An event finder function is called when creating static sensitivity to events through a port. Because port binding may be deferred, it may not be possible for the implementation to retrieve an event to which a process is to be made sensitive at the time the process instance is created. Instead, an application should call an event finder function, in which case the implementation shall defer the adding of events to the static sensitivity of the process until port binding has been completed. These deferred actions shall be completed by the implementation before the callbacks to function **end\_of\_elaboration**.

If an event finder function is called for a multiport bound to more than one channel instance, the events for all such channel instances shall be added to the static sensitivity of the process.

### 5.7.2 Class definition

```
namespace sc_core {

class sc_event_finder implementation-defined;

template <class IF>
class sc_event_finder_t
: public sc_event_finder
{
public:
    sc_event_finder_t( const sc_port_base& port_, const sc_event& (IF::*event_method_) () const );

    // Other members
    implementation-defined
};

} // namespace sc_core
```

### 5.7.3 Constraints on usage

An application shall only use class **sc\_event\_finder** as the return type (passed by reference) of a member function of a port class, or as the base class for an application-specific event finder class template that may possess additional template parameters and event method parameters.

An application shall only use class **sc\_event\_finder\_t<interface>** in constructing the object returned from an event finder.

An event finder shall have a return type of **sc\_event\_finder&** and shall return an object of class **sc\_event\_finder\_t<interface>** or an application-specific event finder class template, where:

- interface* shall be the name of an interface to which said port can be bound, and
- the first argument passed to the constructor for said object shall be the port object itself, and

- c) the second argument shall be the address of a member function of said interface. The event *found by* the event finder is the event returned by this function.

An event finder member function may only be called when creating the static sensitivity of a process using **operator<<**, function **set\_sensitivity**, or macro **SC\_CTHREAD**. An event finder member function shall only be called during elaboration, either from a constructor or from the **before\_end\_of\_elaboration** callback. An event finder member function shall not be called from the **end\_of\_elaboration** callback or during simulation. Instead, an application may make a process directly sensitive to an event.

In the case of a multiport, an event finder member function cannot find an event from an individual channel instance to which the multiport is bound using an index number. An application can work around this restriction by getting the events from the individual channel instances in the **end\_of\_elaboration** callback after port binding is complete (see example below).

*Example:*

```
#include "systemc.h"

class if_class
: virtual public sc_interface
{
    public:
        virtual const sc_event& ev_func() const = 0;
        ...
};

class chan_class
: public if_class, public sc_prim_channel
{
    public:
        virtual const sc_event& ev_func() const { return an_event; }
        ...
    private:
        sc_event an_event;
};

template<int N = 1>
class port_class
: public sc_port<if_class,N>
{
    public:
        sc_event_finder& event_finder() const
        {
            return *new sc_event_finder_t<if_class>( *this , &if_class::ev_func );
        }
        ...
};

SC_MODULE(mod_class)
{
    port_class<1> port_var;
    port_class<0> multiport;
```

```

SC_CTOR(mod_class)
{
    SC_METHOD(method);
    sensitive << port_var.event_finder();      // Sensitive to chan_class::an_event
}
void method();
...

void end_of_elaboration()
{
    SC_METHOD(method2);
    for (int i = 0; i < multiport.size(); i++)
        sensitive << multiport[i]->ev_func();      // Sensitive to chan_class::an_event
}
void method2();
...
};

```

NOTE—For particular examples of event finders, refer to the functions **pos** and **neg** of class **sc\_in<bool>** (see 6.9).

## 5.8 *sc\_event\_and\_list<sup>†</sup>* and *sc\_event\_or\_list<sup>†</sup>*

### 5.8.1 Description

The classes *sc\_event\_and\_list<sup>†</sup>* and *sc\_event\_or\_list<sup>†</sup>* provide the **&** and **|** operators used to construct the event lists passed as arguments to the functions **wait** (see 5.2.16) and **next\_trigger** (see 5.2.17).

### 5.8.2 Class definition

```
namespace sc_core {

class sc_event_and_list†
{
    public:
        sc_event_and_list† & operator& ( const sc_event& );

        // Other members
        implementation-defined
};

class sc_event_or_list†
{
    public:
        sc_event_or_list† & operator| ( const sc_event& );

        // Other members
        implementation-defined
};

} // namespace sc_core
```

### 5.8.3 Constraints on usage

An application shall not explicitly create an object of class *sc\_event\_and\_list<sup>†</sup>* or *sc\_event\_or\_list<sup>†</sup>*.

Classes *sc\_event\_and\_list<sup>†</sup>* and *sc\_event\_or\_list<sup>†</sup>* are the return types of **operator&** and **operator|**, respectively, of class **sc\_event**, and are parameter types of the functions **wait** and **next\_trigger**.

### 5.8.4 Event lists

```
sc_event_and_list† & operator& ( const sc_event& );
sc_event_or_list† & operator| ( const sc_event& );
```

A call to either operator shall add the event passed as an argument to the event list from which the operator is called.

## 5.9 sc\_event

### 5.9.1 Description

An event is an object of class **sc\_event**, used for process synchronization. A process instance may be triggered or resumed on the *occurrence* of an event, that is, when the event is notified. Any given event may be notified on many separate occasions.

### 5.9.2 Class definition

```
namespace sc_core {

class sc_event
{
public:
    sc_event();
    ~sc_event();

    void notify();
    void notify( const sc_time& );
    void notify( double , sc_time_unit );
    void cancel();

    sc_event_or_list†& operator| ( const sc_event& ) const;
    sc_event_and_list†& operator& ( const sc_event& ) const;

private:
    // Disabled
    sc_event( const sc_event& );
    sc_event& operator= ( const sc_event& );
};

} // namespace sc_core
```

### 5.9.3 Constraints on usage

Objects of class **sc\_event** may be constructed during elaboration or simulation but events shall only be notified during simulation. It shall be an error to notify an event during elaboration

### 5.9.4 notify and cancel

void **notify**();

A call to member function **notify** with an empty argument list shall create an immediate notification. Any and all process instances sensitive to the event shall be made runnable before control is returned from function **notify**.

NOTE 1—Process instances sensitive to the event will not be resumed or triggered until the process that called **notify** has suspended or returned.

NOTE 2—All process instances sensitive to the event will be run in the current evaluation phase and in an order that is implementation-defined. The presence of immediate notification can introduce non-deterministic behavior.

NOTE 3—Member function **update** of class **sc\_prim\_channel** shall not call **notify** to create an immediate notification.

```
void notify( const sc_time& );
void notify( double , sc_time_unit );
```

A call to member function **notify** with an argument that represents a zero time shall create a delta notification.

A call to function **notify** with an argument that represents a non-zero time shall create a timed notification at the given time, expressed relative to the simulation time when function **notify** is called. In other words, the value of the time argument is added to the current simulation time to determine the time at which the event will be notified.

NOTE—In the case of a delta notification, all processes that are sensitive to the event in the delta notification phase will be made runnable in the subsequent evaluation phase. In the case of a timed notification, all processes sensitive to the event at the time the event occurs will be made runnable at the time, which will be a future simulation time.

```
void cancel();
```

Member function **cancel** shall delete any pending notification for this event.

NOTE 1—At most one pending notification can exist for any given event.

NOTE 2—Immediate notification cannot be cancelled.

### 5.9.5 Event lists

```
sc_event_or_list†& operator| ( const sc_event& ) const;
sc_event_and_list†& operator& ( const sc_event& ) const;
```

A call to either operator shall add the event passed as an argument to the event list from which the operator is called.

NOTE—Event lists are used as arguments to functions **wait** (see 5.2.17) and **next\_trigger** (see 5.2.16).

### 5.9.6 Multiple event notifications

A given event shall have no more than one pending notification.

If function **notify** is called for an event that already has a notification pending, only the notification scheduled to occur at the earliest time shall survive. The notification scheduled to occur at the later time shall be cancelled (or never be scheduled in the first place). An immediate notification is taken to occur earlier than a delta notification, and a delta notification earlier than a timed notification. This is irrespective of the order in which function **notify** is called.

*Example:*

```
sc_event e;
e.notify(SC_ZERO_TIME); // Delta notification
e.notify(1, SC_NS);      // Timed notification ignored due to pending delta notification
e.notify();              // Immediate notification cancels pending delta notification. e is notified

e.notify(2, SC_NS);      // Timed notification
e.notify(3, SC_NS);      // Timed notification ignored due to earlier pending timed notification
e.notify(1, SC_NS);      // Timed notification cancels pending timed notification
e.notify(SC_ZERO_TIME); // Delta notification cancels pending timed notification
                        // e is notified in the next delta cycle
```

## 5.10 sc\_time

### 5.10.1 Description

Class **sc\_time** is used to represent simulation time and time intervals, including delays and time-outs. An object of class **sc\_time** is constructed from a **double** and an **sc\_time\_unit**. Time shall be represented internally as an unsigned integer of at least 64 bits. For implementations using more than 64 bits, the return value of member function **value** need not be of type **sc\_dt::uint64** (see member function **value** in 5.10.2).

### 5.10.2 Class definition

```
namespace sc_core {

enum sc_time_unit {SC_FS = 0, SC_PS, SC_NS, SC_US, SC_MS, SC_SEC};

class sc_time
{
public:
    sc_time();
    sc_time( double , sc_time_unit );
    sc_time( const sc_time& );

    sc_time& operator= ( const sc_time& );

    sc_dt::uint64 value() const;
    double to_double() const;
    double to_seconds() const;
    const std::string to_string() const;

    bool operator== ( const sc_time& ) const;
    bool operator!= ( const sc_time& ) const;
    bool operator< ( const sc_time& ) const;
    bool operator<= ( const sc_time& ) const;
    bool operator> ( const sc_time& ) const;
    bool operator>= ( const sc_time& ) const;

    sc_time& operator+= ( const sc_time& );
    sc_time& operator-= ( const sc_time& );
    sc_time& operator*= ( double );
    sc_time& operator/= ( double );

    void print( std::ostream& = std::cout ) const;
};

const sc_time operator+ ( const sc_time&, const sc_time& );
const sc_time operator- ( const sc_time&, const sc_time& );

const sc_time operator* ( const sc_time&, double );
const sc_time operator* ( double, const sc_time& );
const sc_time operator/ ( const sc_time&, double );
double operator/ ( const sc_time&, const sc_time& );

std::ostream& operator<< ( std::ostream&, const sc_time& );
```



```
const sc_time SC_ZERO_TIME;

void sc_set_time_resolution( double, sc_time_unit );
sc_time sc_get_time_resolution();

}      // namespace sc_core
```

### 5.10.3 Time resolution

Time shall be represented internally as an integer multiple of the time resolution. The default time resolution is 1 picosecond. Every object of class **sc\_time** shall share a single common global time resolution.

The time resolution can only be changed by calling the function **sc\_set\_time\_resolution**. This function shall only be called during elaboration, shall not be called more than once, and shall not be called after constructing an object of type **sc\_time** with a non-zero time value. The value of the **double** argument shall be positive and shall be a power of 10. It shall be an error for an application to break the rules given in this paragraph.

The constructor for **sc\_time** shall scale and round the given time value to the nearest multiple of the time resolution. Whether the value is rounded up or down is implementation-defined. The default constructor shall create an object having a time value of zero.

The values of enum **sc\_time\_unit** shall be taken to have their standard physical meanings, for example, SC\_FS = femtosecond = 10E-15 seconds.

The function **sc\_get\_time\_resolution** shall return the time resolution.

### 5.10.4 Functions and operators

All arithmetic, relational, equality, and assignment operators declared in 5.10.2 shall be taken to have their natural meanings when performing integer arithmetic on the underlying representation of time. The results of integer underflow and divide-by-zero shall be implementation-defined.

```
sc_dt::uint64 value() const;
double to_double() const;
double to_seconds() const;
```

These functions shall return the underlying representation of the time value, first converting the value to a **double** in each of the two cases **to\_double** and **to\_seconds**, and then also scaling the resultant value to units of 1 second in the case of **to\_seconds**.

```
const std::string to_string() const;
void print( std::ostream& = std::cout ) const;
std::ostream& operator<<( std::ostream& , const sc_time& );
```

These functions shall return the time value converted to a string or print that string to the given stream. The format of the string is implementation-defined.

### 5.10.5 SC\_ZERO\_TIME

Constant SC\_ZERO\_TIME represents a time value of zero. It is good practice to use this constant whenever writing a time value of zero, for example, when creating a delta notification or a delta time-out.

*Example:*

```
sc_event e;  
e.notify(SC_ZERO_TIME); // Delta notification  
wait(SC_ZERO_TIME);     // Delta time-out
```

## 5.11 sc\_port

### 5.11.1 Description

Ports provide the means by which a module can be written such that it is independent of the context in which it is instantiated. A port forwards interface method calls to the channel to which the port is bound. A port defines a set of services (as identified by the type of the port) that are required by the module containing the port.

If a module is to call a member function belonging to a channel that is outside the module itself, that call should be made using an interface method call through a port of the module. To do otherwise is considered bad coding style. However, a call to a member function belonging to a channel instantiated within the current module may be made directly. This is known as *portless* channel access. If a module is to call a member function belonging to a channel instance within a child module, that call should be made through an export of the child module (see 5.12).

### 5.11.2 Class definition

```
namespace sc_core {

enum sc_port_policy
{
    SC_ONE_OR_MORE_BOUND ,           // Default
    SC_ZERO_OR_MORE_BOUND ,
    SC_ALL_BOUND
};

class sc_port_base
: public sc_object { implementation-defined };

template <class IF, int N = 1, sc_port_policy POL = SC_ONE_OR_MORE_BOUND>
class sc_port
: public sc_port_base
{
public:
    sc_port();
    explicit sc_port( const char* );
    virtual ~sc_port();

    virtual const char* kind() const;

    void operator() ( IF& );
    void operator() ( sc_port<IF,N>& );

    void bind( IF& );
    void bind( sc_port<IF,N>& );

    int size() const;

    IF* operator-> ();
    const IF* operator-> () const;

    IF* operator[] ( int );
    const IF* operator[] ( int ) const;
};
```

```

virtual sc_interface* get_interface();
virtual const sc_interface* get_interface() const;

protected:
    virtual void before_end_of_elaboration();
    virtual void end_of_elaboration();
    virtual void start_of_simulation();
    virtual void end_of_simulation();

private:
    // Disabled
    sc_port( const sc_port<IF,N>& );
    sc_port<IF,N>& operator= ( const sc_port<IF,N>& );
};

} // namespace sc_core

```

### 5.11.3 Template parameters

The first argument to template **sc\_port** shall be the name of an *interface proper*. This interface is said to be the *type of the port*. A port can only be bound to a channel derived from the type of the port or to another port or export with a type derived from the type of the port.

The second argument to template **sc\_port** is an optional integer value. If present, this argument shall specify the maximum number of channel instances to which any one instance of the port belonging to any specific module instance may be bound. If the value of this argument is zero, the port may be bound to an arbitrary number of channel instances. It shall be an error to bind a port to more channel instances than the number permitted by the second template argument.

The default value of the second argument is 1. If the value of the second argument is not 1, the port is said to be a *multiport*. If a port is bound to another port, the value of this argument may differ between the two ports.

The third argument to template **sc\_port** is an optional port policy of type **sc\_port\_policy**. The port policy argument determines the rules for binding multiports and the rules for unbound ports.

The policy SC\_ONE\_OR\_MORE\_BOUND means that the port instance shall be bound to one or more channel instances, the maximum number being determined by the value of the second template argument. It shall be an error for the port instance to remain unbound at the end of elaboration.

The policy SC\_ZERO\_OR\_MORE\_BOUND means that the port instance shall be bound to zero or more channel instances, the maximum number being determined by the value of the second template argument. The port instance may remain unbound at the end of elaboration.

The policy SC\_ALL\_BOUND means that the port instance shall be bound to exactly the number of channel instances given by value of the second template argument, no more and no less, provided that value is greater than zero. If the value of the second template argument is zero, policy SC\_ALL\_BOUND shall have the same meaning as policy SC\_ONE\_OR\_MORE\_BOUND. It shall be an error for the port instance to remain unbound at the end of elaboration, or to be bound to fewer channel instances than the number required by the second template argument.

It shall be an error to bind a given port instance to a given channel instance more than once, whether directly or through another port.

The port policy shall apply independently to each port instance, even when a port is bound to another port. For example, if a port on a child module with a type **sc\_port<IF>** is bound to a port on a parent module with a type **sc\_port<IF,2,SC\_ALL\_BOUND>**, the two port policies are contradictory and one or other will inevitably result in an error at the end of elaboration.

The port policies shall hold when port binding is completed by the implementation just before the callbacks to function **end\_of\_elaboration** but are not required to hold any earlier. For example, a port of type **sc\_port<IF,2,SC\_ALL\_BOUND>** could be bound once in a module constructor and once in the callback function **before\_end\_of\_elaboration**.

*Example:*

```
sc_port<IF>                // Bound to exactly 1 channel instance
sc_port<IF,0>              // Bound to 1 or more channel instances
                           // with no upper limit
sc_port<IF,3>              // Bound to 1, 2 or 3 channel instances
sc_port<IF,0,SC_ZERO_OR_MORE_BOUND> // Bound to 0 or more channel instances
                           // with no upper limit
sc_port<IF,1,SC_ZERO_OR_MORE_BOUND> // Bound to 0 or 1 channel instances
sc_port<IF,3,SC_ZERO_OR_MORE_BOUND> // Bound to 0, 1, 2 or 3 channel instances
sc_port<IF,3,SC_ALL_BOUND>  // Bound to exactly 3 channel instances
```

NOTE—A port may be bound indirectly to a channel by being bound to another port or export (see 4.1.3).

#### 5.11.4 Constraints on usage

An implementation shall derive class **sc\_port\_base** from class **sc\_object**.

Ports shall only be instantiated during elaboration and only from within a module. It shall be an error to instantiate a port other than within a module. It shall be an error to instantiate a port during simulation.

The member functions **size** and **get\_interface** can be called during elaboration or simulation, whereas **operator->** and **operator[]** should only be called from **end\_of\_elaboration** or during simulation.

It is strongly recommended that a port within a given module be bound at the point where the given module is instantiated, that is, within the constructor from which the module is instantiated. Furthermore, it is strongly recommended that the port be bound to a channel or another port that is itself instantiated within the module containing the instance of the given module or to an export that is instantiated within a child module. This recommendation may be violated on occasion. For example, it is convenient to bind an otherwise unbound port from the **before\_end\_of\_elaboration** callback of the port instance itself.

The constraint that a port be instantiated *within a module* allows for considerable flexibility. However, it is strongly recommended that a port instance be a data member of a module wherever practical; otherwise, the syntax necessary for named port binding becomes somewhat arcane in that it requires more than simple class member access using the dot operator.

Suppose a particular port is instantiated within module C, and module C is itself instantiated within module P. It is permissible for the port to be bound at some point in the code remote from the point at which module C is instantiated, it is permissible for the port to be bound to a channel (or another port) that is itself instantiated in a module other than the module P, and it is permissible for the port to be bound to an export that is instantiated somewhere other than in a child module of module P. However, all such cases would result in a breakdown of the normal discipline of the module hierarchy and are strongly discouraged in typical usage.

### 5.11.5 Constructors

```
sc_port();
explicit sc_port( const char* );
```

The constructor for class **sc\_port** shall pass the character string argument (if such argument exists) through to the constructor belonging to the base class **sc\_object** to set the string name of the instance in the module hierarchy.

The default constructor shall call function **sc\_gen\_unique\_name("port")** to generate a unique string name that it shall then pass through to the constructor for the base class **sc\_object**.

NOTE—A port instance need not be given an explicit string name within the application when it is constructed.

### 5.11.6 kind

Member function **kind** shall return the string **"sc\_port"**.

### 5.11.7 Named port binding

Ports can be bound either using the functions listed in this subclause for named binding or using the **operator()** from class **sc\_module** for positional binding. An implementation may defer the completion of port binding until a later time during elaboration because the port to which a port is bound may not yet itself have been bound. Such deferred port binding shall be completed by the implementation before the callbacks to function **end\_of\_elaboration**.

```
void operator() ( IF& );
void bind( IF& );
```

Each of these two functions shall bind the port instance for which the function is called to the channel instance passed as an argument to the function. The actual argument can be an export, in which case the C++ compiler will call the implicit conversion **sc\_export<IF>::operator&**.

```
void operator() ( sc_port<IF,N>& );
void bind( sc_port<IF,N>& );
```

Each of these two functions shall bind the port instance for which the function is called to the port instance passed as an argument to the function.

*Example:*

```
SC_MODULE(M)
{
    sc_inout<int> P, Q, R, S; // Ports
    sc_inout<int> *T;        // Pointer-to-port (not a recommended coding style)

    SC_CTOR(M) { T = new sc_inout<int>; }
    ...
};

SC_MODULE(Top)
{
    sc_inout <int> A, B;
```

```
sc_signal<int> C, D;  
M m;           // Module instance  
SC_CTOR(Top)  
: m("m")  
{  
    m.P(A);      // Binds P-to-A  
    m.Q.bind(B); // Binds Q-to-B  
    m.R(C);      // Binds R-to-C  
    m.S.bind(D); // Binds S-to-D  
    m.T->bind(E); // Binds T-to-E  
}  
...  
};
```

### 5.11.8 Member functions for bound ports and port-to-port binding

The member functions described in this subclause return information about ports that have been bound during elaboration. These functions return information concerning the ordered set of channel instances to which a particular port instance (which may or may not be a multiport) is bound.

The ordered set *S* of channel instances to which a given port is bound (for the purpose of defining the semantics of the functions given in this subclause) is determined as follows.

- a) When the port or export is bound to a channel instance, that channel instance shall be added to the end of the ordered set *S*.
- b) When the port or export is bound to an export, rules a) and b) shall be applied recursively to the export.
- c) When the port is bound to another port, rules a), b), and c) shall be applied recursively to the other port.

Because an implementation may defer the completion of port binding until a later time during elaboration, the number and order of the channel instances as returned from the member functions described in this subclause may change during elaboration and the final order is implementation-defined, but shall not change during the **end\_of\_elaboration** callback or during simulation.

NOTE—As a consequence of the above rules, a given channel instance may appear to lie at a different position in the ordered set of channel instances when viewed from ports at different positions in the module hierarchy. For example, a given channel instance may be the first channel instance to which a port of a parent module is bound but the third channel instance to which a port of a child module is bound.

#### 5.11.8.1 size

```
int size() const;
```

Member function **size** shall return the number of channel instances to which the port instance for which it is called has been bound.

If member function **size** is called during elaboration and before the callback **end\_of\_elaboration**, the value returned is implementation-defined because the time at which port binding is completed is implementation-defined.

NOTE—The value returned by **size** will be 1 for a typical port but may be 0 if the port is unbound or greater than 1 for a multiport.

### 5.11.8.2 operator->

IF\* **operator->** ();  
const IF\* **operator->** () const;

**operator->** shall return a pointer to the first channel instance to which the port was bound during elaboration.

It shall be an error to call **operator->** for an unbound port. If **operator->** is called during elaboration and before the callback **end\_of\_elaboration**, the behavior is implementation-defined because the time at which port binding is completed is implementation-defined.

NOTE—**operator->** is key to the interface method call paradigm in that it permits a process to call a member function, defined in a channel, through a port bound to that channel.

*Example:*

```
struct iface
: virtual sc_interface
{
    virtual int read() const = 0;
};

struct chan
: iface, sc_prim_channel
{
    virtual int read() const;
};

int chan::read() const { ... }

SC_MODULE(modu)
{
    sc_port<iface> P;

    SC_CTOR(modu)
    {
        SC_THREAD(thread);
    }
    void thread()
    {
        int i = P->read();    // Interface method call
    }
};

SC_MODULE(top)
{
    modu *mo;
    chan *ch;

    SC_CTOR(top)
    {
        ch = new chan;
        mo = new modu("mo");
    }
};
```



```

        mo->P(*ch);          // Port P bound to channel *ch
    }
};

```

### 5.11.8.3 operator[]

```

IF* operator[] ( int );
const IF* operator[] ( int ) const;

```

**operator[]** shall return a pointer to a channel instance to which a port is bound. The argument identifies which channel instance shall be returned. The instances are numbered starting from zero in the order in which the port binding was completed, the order being implementation-defined.

The value of the argument shall lie in the range 0 to N-1, where N is the number of instances to which the port is bound. It shall be an error to call **operator[]** with an argument value that lies outside this range. If **operator[]** is called during elaboration and before the callback **end\_of\_elaboration**, the behavior is implementation-defined because the time at which port binding is completed is implementation-defined.

**operator[]** may be called for a port that is not a multiport, in which case the value of the argument should be 0.

*Example:*

```

class bus_interface;

class slave_interface
: virtual public sc_interface
{
    public:
        virtual void slave_write(int addr, int data) = 0;
        virtual void slave_read (int addr, int& data) = 0;
};

class bus_channel
: public bus_interface, public sc_module
{
    public:
        ...
        sc_port<slave_interface, 0> slave_port;    // Multiport for attaching slaves to bus

        SC_CTOR(bus_channel)
        {
            SC_THREAD(action);
        }
    private:
        void action()
        {
            for (int i = 0; i < slave_port.size(); i++)    // Function size() returns number of slaves
                slave_port[i]->slave_write(0,0);          // Operator[] indexes slave port
        }
};

```

```
class memory
: public slave_interface, public sc_module
{
    public:
        virtual void slave_write(int addr, int data);
        virtual void slave_read (int addr, int& data);
        ...
};

SC_MODULE(top_level)
{
    bus_channel bus;
    memory    ram0, ram1, ram2, ram3;

    SC_CTOR(top_level)
    : bus("bus"), ram0("ram0"), ram1("ram1"), ram2("ram2"), ram3("ram3")
    {
        bus.slave_port(ram0);
        bus.slave_port(ram1);
        bus.slave_port(ram2);
        bus.slave_port(ram3);                // One multiport bound to four memory channels
    }
};
```

#### 5.11.8.4 get\_interface

```
virtual sc_interface* get_interface();
virtual const sc_interface* get_interface() const;
```

Member function **get\_interface** shall return a pointer to the first channel instance to which the port is bound. If the port is unbound, a null pointer shall be returned. This member function may be called during elaboration to test whether a port has yet been bound. Because the time at which deferred port binding is completed is implementation-defined, it is implementation-defined whether **get\_interface** returns a pointer to a channel instance or a null pointer when called during construction or from the callback **before\_end\_of\_elaboration**.

**get\_interface** is intended for use in implementing specialized port classes derived from **sc\_port**. In general, an application should call **operator->** instead. However, **get\_interface** permits an application to call a member function of the class of the channel to which the port is bound, even if such a function is not a member of the interface type of the port.

NOTE—Function **get\_interface** cannot return channels beyond the first channel instance to which a multiport is bound; use **operator[]** instead.

*Example:*

```
SC_MODULE(Top)
{
    sc_in<bool> clock;

    void before_end_of_elaboration()
    {
        sc_interface* i_f = clock.get_interface();
    }
};
```

```
sc_clock* clk = dynamic_cast<sc_clock*>(i_f);  
sc_time t = clk->period();           // Call method of clock object to which port is bound  
...
```

#### **5.11.9 before\_end\_of\_elaboration, end\_of\_elaboration, start\_of\_simulation, end\_of\_simulation**

See 4.4.

## 5.12 sc\_export

### 5.12.1 Description

Class **sc\_export** allows a module to provide an interface to its parent module. An export forwards interface method calls to the channel to which the export is bound. An export defines a set of services (as identified by the type of the export) that are provided by the module containing the export.

Providing an interface through an export is an alternative to a module simply implementing the interface. The use of an explicit export allows a single module instance to provide multiple interfaces in a structured manner.

If a module is to call a member function belonging to a channel instance within a child module, that call should be made through an export of the child module.

### 5.12.2 Class definition

```
namespace sc_core {

class sc_export_base
: public sc_object { implementation-defined };

template<class IF>
class sc_export
: public sc_export_base
{
public:
    sc_export();
    explicit sc_export( const char* );
    virtual ~sc_export();

    virtual const char* kind() const;

    void operator() ( IF& );
    void bind( IF& );
    operator IF& ();

    IF* operator-> ();
    const IF* operator-> () const;

    virtual sc_interface* get_interface();
    virtual const sc_interface* get_interface() const;

protected:
    virtual void before_end_of_elaboration();
    virtual void end_of_elaboration();
    virtual void start_of_simulation();
    virtual void end_of_simulation();
};
}
```

```
private
    // Disabled
    sc_export( const sc_export<IF>& );
    sc_export<IF>& operator= ( const sc_export<IF>& );
};

} // namespace sc_core
```

### 5.12.3 Template parameters

The argument to template **sc\_export** shall be the name of an interface proper. This interface is said to be the *type of the export*. An export can only be bound to a channel derived from the type of the export or to another export with a type derived from the type of the export.

NOTE—An export may be bound indirectly to a channel by being bound to another export (see 4.1.3).

### 5.12.4 Constraints on usage

An implementation shall derive class **sc\_export\_base** from class **sc\_object**.

Exports shall only be instantiated during elaboration and only from within a module. It shall be an error to instantiate an export other than within a module. It shall be an error to instantiate an export during simulation.

Every export of every module instance shall be bound once and once only during elaboration. It shall be an error to have an export remaining unbound at the end of elaboration. It shall be an error to bind an export to more than one channel.

The member function **get\_interface** can be called during elaboration or simulation, whereas **operator->** should only be called during simulation.

It is strongly recommended that an export within a given module be bound within that same module. Furthermore, it is strongly recommended that the export be bound to a channel that is itself instantiated within the current module or implemented by the current module or bound to an export that is instantiated within a child module. Any other usage would result in a breakdown of the normal discipline of the module hierarchy and is strongly discouraged (see 5.11.4).

### 5.12.5 Constructors

```
sc_export();
explicit sc_export( const char* );
```

The constructor for class **sc\_export** shall pass the character string argument (if there is one) through to the constructor belonging to the base class **sc\_object** in order to set the string name of the instance in the module hierarchy.

The default constructor shall call function **sc\_gen\_unique\_name("export")** in order to generate a unique string name that it shall then pass through to the constructor for the base class **sc\_object**.

NOTE—An export instance need not be given an explicit string name within the application when it is constructed.

### 5.12.6 kind

Member function **kind** shall return the string "**sc\_export**".

### 5.12.7 Export binding

Exports can be bound using either of the two functions defined here. The notion of positional binding is not applicable to exports. Each of these functions shall bind the export immediately, in contrast to ports for which the implementation may need to defer the binding.

```
void operator() ( IF& );  
void bind( IF& );
```

Each of these two functions shall bind the export instance for which the function is called to the channel instance passed as an argument to the function.

NOTE—The actual argument could be an export, in which case **operator IF&** would be called as an implicit conversion.

*Example:*

```
struct i_f: virtual sc_interface  
{  
    virtual void print() = 0;  
};  
  
struct Chan: sc_channel, i_f  
{  
    SC_CTOR(Chan) {}  
    void print() { std::cout << "I'm Chan, name=" << name() << std::endl; }  
};  
  
struct Caller: sc_module  
{  
    sc_port<i_f> p;  
    ...  
};  
  
struct Bottom: sc_module  
{  
    sc_export<i_f> xp;  
    Chan ch;  
    SC_CTOR(Bottom) : ch("ch")  
    {  
        xp.bind(ch);           // Bind export xp to channel ch  
    }  
};  
  
struct Middle: sc_module  
{  
    sc_export<i_f> xp;  
    Bottom* b;  
    SC_CTOR(Middle)  
    {  
        b = new Bottom ("b");  
        xp.bind(b->xp);       // Bind export xp to export b->xp  
  
        b->xp->print();        // Call method of export within child module  
    }  
};
```

```

    }
};

struct Top: sc_module
{
    Caller* c;
    Middle* m;

    SC_CTOR(Top)
    {
        c = new Caller ("c");
        m = new Middle ("m");
        c->p(m->xp);      // Bind port c->p to export m->xp
    }
};

```

### 5.12.8 Member functions for bound exports and export-to-export binding

The member functions described in this subclause return information about exports that have been bound during elaboration, and hence these member functions should only be called after the export has been bound during elaboration or during simulation. These functions return information concerning the channel instance to which a particular export instance has been bound.

It shall be an error to bind an export more than once. It shall be an error for an export to be unbound at the end of elaboration.

The channel instance to which a given export is bound (for the purpose of defining the semantics of the functions given in this subclause) is determined as follows:

- a) If the export is bound to a channel instance, that is the channel instance in question.
- b) If the export is bound to another export, rules a) and b) shall be applied recursively to the other export.

#### 5.12.8.1 operator-> and operator IF&

```

IF* operator-> ();
const IF* operator-> () const;
operator IF& ();

```

**operator->** and **operator IF&** shall both return a pointer to the channel instance to which the export was bound during elaboration.

It shall be an error for an application to call this operator if the export is unbound.

NOTE 1—**operator->** is intended for use during simulation when making an interface method call through an export instance from a parent module of the module containing the export.

NOTE 2—**operator IF&** is intended for use during elaboration as an implicit conversion when passing an object of class **sc\_export** in a context that requires an **sc\_interface**, for example, when binding a port to an export or when adding an export to the static sensitivity of a process.

NOTE 3—There is no **operator[]** for class **sc\_export**, and there is no notion of a multi-export. Each export can only be bound to a single channel.

### 5.12.8.2 **get\_interface**

```
virtual sc_interface* get_interface();  
virtual const sc_interface* get_interface() const;
```

Member function **get\_interface** shall return a pointer to the channel instance to which the export is bound. If the export is unbound, a null pointer shall be returned. This member function may be called during elaboration to test whether an export has yet been bound.

### 5.12.9 **before\_end\_of\_elaboration, end\_of\_elaboration, start\_of\_simulation, end\_of\_simulation**

See 4.4.



## 5.13 sc\_interface

### 5.13.1 Description

**sc\_interface** is the abstract base class for all interfaces.

An *interface* is a class derived from the class **sc\_interface**. An *interface proper* is an abstract class derived from class **sc\_interface** but not derived from class **sc\_object**. An interface proper contains a set of pure virtual functions that shall be defined in one or more channels derived from that interface proper. Such a channel is said to *implement* the interface.

NOTE 1—The term *interface proper* is used to distinguish an interface proper from a channel. A channel is a class derived indirectly from class **sc\_interface** and in that sense a channel is an interface. However, a channel is not an interface proper.

NOTE 2—As a consequence of the rules of C++, an instance of a channel derived from an interface **IF** or a pointer to such an instance can be passed as the argument to a function with a parameter of type **IF&** or **IF\***, respectively, or a port of type **IF** can be bound to such a channel.

### 5.13.2 Class definition

```
namespace sc_core {

class sc_interface
{
    public:
        virtual void register_port( sc_port_base& , const char* );
        virtual const sc_event& default_event() const;
        virtual ~sc_interface();

    protected:
        sc_interface();

    private:
        // Disabled
        sc_interface( const sc_interface& );
        sc_interface& operator=( const sc_interface& );
};

} // namespace sc_core
```

### 5.13.3 Constraints on usage

An application should not use class **sc\_interface** as the direct base class for any class other than an interface proper.

An interface proper shall obey the following rules:

- It shall be publicly derived directly or indirectly from class **sc\_interface**
- If directly derived from class **sc\_interface**, it shall use the **virtual** specifier
- It shall not be derived directly or indirectly from class **sc\_object**

An interface proper should typically obey the following rules:

- It should contain one or more pure virtual functions
- It should not be derived from any other class that is not itself an interface proper

- c) It should not contain any function declarations or function definitions apart from the pure virtual functions
- d) It should not contain any data members

NOTE 1—An interface proper may be derived from another interface proper or from two or more other interfaces proper, thus creating a multiple inheritance hierarchy.

NOTE 2—A channel class may be derived from any number of interfaces proper.

#### 5.13.4 register\_port

virtual void **register\_port**( sc\_port\_base& , const char\* );

The definition of this function in class **sc\_interface** does nothing. An application may override this function in a channel.

The purpose of function **register\_port** is to enable an application to perform actions that depend on port binding during elaboration, such as checking connectivity errors.

Member function **register\_port** of a channel shall be called by the implementation whenever a port is bound to a channel instance. The first argument shall be a reference to the port instance being bound. The second argument shall be the value returned from the expression **typeid( IF ).name()**, where **IF** is the interface type of the port.

Member function **register\_port** shall not be called when an export is bound to a channel.

If a port P is bound to another port Q, and port Q is in turn bound to a channel instance, the first argument to member function **register\_port** shall be the port P. In other words, **register\_port** is *not* passed a reference to a port on a parent module if a port on a child module is in turn bound to that port; instead, it is passed as a reference to the port on the child module, and so on recursively down the module hierarchy.

In the case that multiple ports are bound to the same single channel instance or port instance, member function **register\_port** shall be called once for each port so bound.

*Example:*

```
void register_port( sc_port_base& port_, const char* if_typename_ )
{
    std::string nm( if_typename_ );
    if( nm == typeid( my_interface ).name() )
        std::cout << " channel " << name() << " bound to port " << port_.name() << std::endl;
}
```

#### 5.13.5 default\_event

virtual const sc\_event& **default\_event**() const;

Member function **default\_event** shall be called by the implementation in every case where a port or channel instance is used to define the static sensitivity of a process instance by being passed directly as an argument to **operator<<** of class *sc\_sensitive*<sup>†</sup>. In such a case the application shall override this function in the channel in question to return a reference to an event to which the process instance will be made sensitive.

If this function is called by the implementation but not overridden by the application, the implementation may generate a warning.

*Example:*

```
struct my_if
: virtual sc_interface
{
    virtual int read() = 0;
};

class my_ch
: public my_if, public sc_module
{
    public:
        virtual int read() { return m_val; }
        virtual const sc_event& default_event() const { return m_ev; }
    private:
        int m_val;
        sc_event m_ev;
        ...
};
```

## 5.14 sc\_prim\_channel

### 5.14.1 Description

**sc\_prim\_channel** is the base class for all primitive channels and provides such channels with unique access to the update phase of the scheduler. In common with hierarchical channels, a primitive channel may provide public member functions that can be called using the interface method call paradigm.

This standard provides a number of predefined primitive channels to model common communication mechanisms (see Clause 6).

### 5.14.2 Class definition

```
namespace sc_core {

class sc_prim_channel
: public sc_object
{
public:
    virtual const char* kind() const;

protected:
    sc_prim_channel();
    explicit sc_prim_channel( const char* );
    virtual ~sc_prim_channel();

    void request_update();
    virtual void update();

    void next_trigger();
    void next_trigger( const sc_event& );
    void next_trigger( sc_event_or_list†& );
    void next_trigger( sc_event_and_list†& );
    void next_trigger( const sc_time& );
    void next_trigger( double , sc_time_unit );
    void next_trigger( const sc_time& , const sc_event& );
    void next_trigger( double , sc_time_unit , const sc_event& );
    void next_trigger( const sc_time& , sc_event_or_list†& );
    void next_trigger( double , sc_time_unit , sc_event_or_list†& );
    void next_trigger( const sc_time& , sc_event_and_list†& );
    void next_trigger( double , sc_time_unit , sc_event_and_list†& );

    void wait();
    void wait( int );
    void wait( const sc_event& );
    void wait( sc_event_or_list†& );
    void wait( sc_event_and_list†& );
    void wait( const sc_time& );
    void wait( double , sc_time_unit );
    void wait( const sc_time& , const sc_event& );
    void wait( double , sc_time_unit , const sc_event& );
    void wait( const sc_time& , sc_event_or_list†& );
    void wait( double , sc_time_unit , sc_event_or_list†& );
    void wait( const sc_time& , sc_event_and_list†& );
    void wait( double , sc_time_unit , sc_event_and_list†& );
};

}
```

```
void wait( double , sc_time_unit , sc_event_and_list†& );

virtual void before_end_of_elaboration();
virtual void end_of_elaboration();
virtual void start_of_simulation();
virtual void end_of_simulation();

private:
    // Disabled
    sc_prim_channel( const sc_prim_channel& );
    sc_prim_channel& operator= ( const sc_prim_channel& );
};

} // namespace sc_core
```

### 5.14.3 Constraints on usage

Objects of class **sc\_prim\_channel** can only be constructed during elaboration. It shall be an error to instantiate a primitive channel during simulation.

A primitive channel should be *publicly* derived from class **sc\_prim\_channel**.

A primitive channel shall implement one or more interfaces.

NOTE—Because the constructors are protected, class **sc\_prim\_channel** cannot be instantiated directly but may be used as a base class for a primitive channel.

### 5.14.4 Constructors

```
sc_prim_channel();
explicit sc_prim_channel( const char* );
```

The constructor for class **sc\_prim\_channel** shall pass the character string argument (if such argument exists) through to the constructor belonging to the base class **sc\_object** to set the string name of the instance in the module hierarchy.

NOTE—A class derived from class **sc\_prim\_channel** is not obliged to have a constructor, in which case the default constructor for class **sc\_object** will generate a unique string name. As a consequence, a primitive channel instance need not be given an explicit string name within the application when it is constructed.

### 5.14.5 kind

Member function **kind** shall return the string "**sc\_prim\_channel**".

### 5.14.6 request\_update and update

```
void request_update();
```

Member function **request\_update** shall cause the scheduler to queue an update request for the specific primitive channel instance making the call (see 4.2.1.3).

virtual void **update**();

Member function **update** shall be called back by the scheduler during the update phase in response to a call to **request\_update**. An application may override this member function in a primitive channel. The definition of this function in class **sc\_prim\_channel** itself does nothing.

When overridden in a derived class, member function **update** shall not perform any of the following actions:

- a) Call any member function of class **sc\_prim\_channel** with the exception of member function **update** itself if overridden within a base class of the current object
- b) Call member function **notify()** of class **sc\_event** with no arguments to create an immediate notification

If the application violates the two rules just given, the behavior of the implementation shall be undefined.

Member function **update** should not change the state of any storage except for data members of the current object. Doing so may result in non-deterministic behavior.

Member function **update** should not read the state of any primitive channel instance other than the current object. Doing so may result in non-deterministic behavior.

Member function **update** may call function **sc\_spawn** to create a dynamic process instance. Such a process shall not become runnable until the next evaluation phase.

NOTE 1—The purpose of the member functions **request\_update** and **update** is to permit simultaneous requests to a channel made during the evaluation phase to be resolved or arbitrated during the update phase. The nature of the arbitration is the responsibility of the application; for example, the behavior of member function **update** may be deterministic or random.

NOTE 2—**update** will typically only read and modify data members of the current object and create delta notifications.

#### 5.14.7 next\_trigger and wait

The behavior of the member functions **wait** and **next\_trigger** of class **sc\_prim\_channel** shall be identical to that of the member functions of class **sc\_module** with the same function names and signatures. Aside from the fact that they are members of different classes and so have different scopes, the restrictions concerning the context in which the member functions may be called is also identical. For example, the member function **next\_trigger** shall only be called from a method process.

#### 5.14.8 before\_end\_of\_elaboration, end\_of\_elaboration, start\_of\_simulation, end\_of\_simulation

See 4.4.

*Example:*

```
struct my_if
: virtual sc_interface                                // An interface proper
{
    virtual int  read() = 0;
    virtual void write(int) = 0;
};

struct my_prim
: sc_prim_channel, my_if                            // A primitive channel
```

```

{
    my_prim()                                // Default constructor
    :
        sc_prim_channel( sc_gen_unique_name("my_prim") ),
        m_req(false),
        m_written(false),
        m_cur_val(0) {}

    virtual void write(int val)
    {
        if (!m_req)                          // Only keeps the 1st value written in any one delta
        {
            m_new_val = val;
            request_update();                 // Schedules an update request
            m_req = true;
        }
    }

    virtual void update()                    // Called back by the scheduler in the update phase
    {
        m_cur_val = m_new_val;
        m_req = false;
        m_written = true;
        m_write_event.notify(SC_ZERO_TIME); // A delta notification
    }

    virtual int read()
    {
        if (!m_written) wait(m_write_event); // Blocked until update() is called
        m_written = false;
        return m_cur_val;
    }

    bool m_req, m_written;
    sc_event m_write_event;
    int m_new_val, m_cur_val;
};

```

## 5.15 **sc\_object**

### 5.15.1 Description

Class **sc\_object** is the common base class for classes **sc\_module**, **sc\_port**, **sc\_export**, and **sc\_prim\_channel**, and for the implementation-defined classes associated with spawned and unspawned process instances. The set of **sc\_objects** shall be organized into an *object hierarchy*, where each **sc\_object** has no more than one parent but may have multiple siblings and multiple children. Only module objects and process objects can have children.

An **sc\_object** is a *child* of a module instance if and only if that object lies *within* the module instance, as defined in 3.1.4. An **sc\_object** is a *child* of a process instance if and only if that object was created during the execution of the function associated with that process instance. Object P is a *parent* of object C if and only if C is a child of P.

An **sc\_object** that has no parent object is said to be a *top-level* object. Module instances, spawned process instances, and objects of an application-defined class derived from class **sc\_object** may be top-level objects.

Each call to function **sc\_spawn** shall create a spawned process instance that is either a child of the caller or a top-level object. The parent of the spawned process instance so created may be another spawned process instance, an unspawned process instance, or a module instance. Alternatively, the spawned process instance may be a top-level object.

Each **sc\_object** shall have a unique hierarchical name reflecting its position in the object hierarchy.

Attributes may be added to each **sc\_object**.

NOTE—An implementation may permit multiple top-level **sc\_objects** (see 4.3).

### 5.15.2 Class definition

```
namespace sc_core {

class sc_object
{
    public:
        const char* name() const;
        const char* basename() const;
        virtual const char* kind() const;

        virtual void print( std::ostream& = std::cout ) const;
        virtual void dump( std::ostream& = std::cout ) const;

        virtual const std::vector<sc_object*>& get_child_objects() const;
        sc_object* get_parent_object() const;

        bool add_attribute( sc_attr_base& );
        sc_attr_base* get_attribute( const std::string& );
        const sc_attr_base* get_attribute( const std::string& ) const;
        sc_attr_base* remove_attribute( const std::string& );
        void remove_all_attributes();
        int num_attributes() const;
        sc_attr_cltn& attr_cltn();
        const sc_attr_cltn& attr_cltn() const;
}
```



```
protected:
    sc_object();
    sc_object(const char*);
    virtual ~sc_object();
};

const std::vector<sc_object*>& sc_get_top_level_objects();
sc_object* sc_find_object( const char* );

} // namespace sc_core
```

### 5.15.3 Constraints on usage

An application may use class **sc\_object** as a base class for other classes besides modules, ports, exports, primitive channels, and processes. An application may access the hierarchical name of such an object, or may add attributes to such an object.

An application shall not define a class that has two or more base class sub-objects of class **sc\_object**.

Objects of class **sc\_object** may be instantiated during elaboration or may be instantiated during simulation. However, modules, ports, exports, and primitive channels can only be instantiated during elaboration. It is permitted to create a channel that is neither a hierarchical channel nor a primitive channel but is nonetheless derived from class **sc\_object**, and to instantiate such a channel either during elaboration or during simulation. Portless channel access is permitted for any channel but a port or export cannot be bound to a channel that is instantiated during simulation.

NOTE 1—Because the constructors are protected, class **sc\_object** cannot be instantiated directly.

NOTE 2—Since the classes having **sc\_object** as a direct base class (that is, **sc\_module**, **sc\_port**, **sc\_export**, and **sc\_prim\_channel**) have class **sc\_object** as a non-virtual base class, any class derived from these classes shall have at most one direct base class derived from class **sc\_object**. In other words, multiple inheritance from the classes derived from class **sc\_object** is not permitted.

### 5.15.4 Constructors and hierarchical names

```
sc_object();
sc_object(const char*);
```

Both constructors shall register the **sc\_object** as part of the object hierarchy and shall construct a hierarchical name for the object using the string name passed as an argument. Calling the constructor **sc\_object(const char\*)** with an empty string shall have the same behavior as the default constructor, that is, the string name shall be set to "**object**".

A hierarchical name shall be composed of a set of string names separated by the period character '.', starting with the string name of a top-level **sc\_object** instance and including the string name of each module instance or process instance descending down through the object hierarchy until the current **sc\_object** is reached. The hierarchical name shall end with the string name of the **sc\_object** itself.

Hierarchical names are case-sensitive.

It shall be an error if a string name includes the period character (.) or any white-space characters. It is strongly recommended that an application limit the character set of a string name to the following:

- a) The lower-case letters a-z
- b) The upper-case letters A-Z

- c) The decimal digits 0-9
- d) The underscore character \_

An implementation may generate a warning if a string name contains characters outside this set but is not obliged to do so.

There shall be a single global namespace for hierarchical names. Each **sc\_object** shall have a unique non-empty hierarchical name. An implementation shall not add any names to this namespace other than the hierarchical names of **sc\_objects** explicitly constructed by an application.

The constructor shall build a hierarchical name from the string name (either passed in as an argument or the default name "**object**") and test whether that hierarchical name is unique. If it is unique, that hierarchical name shall become the hierarchical name of the object. If not, the constructor shall call function **sc\_gen\_unique\_name**, passing the string name as a seed. It shall use the value returned as a replacement for the string name and shall repeat this process until a unique hierarchical name is generated.

If function **sc\_gen\_unique\_name** is called more than once in the course of constructing any given **sc\_object**, the choice of seed passed to **sc\_gen\_unique\_name** on the second and subsequent calls shall be implementation-defined but shall in any case be either the string name passed as the seed on the first such call or shall be one of the string names returned from **sc\_gen\_unique\_name** in the course of constructing the given **sc\_object**. In other words, the final string name shall have the original string name as a prefix.

If the constructor needs to substitute a new string name in place of the original string name as the result of a name clash, the constructor shall generate a single warning.

NOTE—If an implementation were to create internal objects of class **sc\_object**, the implementation would be obliged by the rules of this subclause to exclude those objects from the object hierarchy and from the namespace of hierarchical names. This would necessitate an extension to the semantics of class **sc\_object**, and the implementation would be obliged to make such an extension transparent to the application.

### 5.15.5 name, basename, and kind

const char\* **name**() const;

Member function **name** shall return the *hierarchical name* of the **sc\_object** instance in the object hierarchy.

const char\* **basename**() const;

Member function **basename** shall return the string name of the **sc\_object** instance. This is the string name created when the **sc\_object** instance was constructed.

virtual const char\* **kind**() const;

Member function **kind** returns a character string identifying the *kind* of the **sc\_object**. Member function **kind** of class **sc\_object** shall return the string "**sc\_object**". Every class that is part of the implementation and that is derived from class **sc\_object** shall override member function **kind** to return an appropriate string.

*Example:*

```
#include "systemc.h"
SC_MODULE(Mod)
{
    sc_port<sc_signal_in_if<int>> p;

    SC_CTOR(Mod)          // p.name() returns "top.mod.p"
    : p("p")              // p.basename() returns "p"
    {}                    // p.kind() returns "sc_port"
};

SC_MODULE(Top)
{
    Mod *mod;              // mod->name() returns "top.mod"
    sc_signal<int> sig;     // sig.name() returns "top.sig"

    SC_CTOR(Top)
    : sig("sig")
    {
        mod = new Mod("mod");
        mod->p(sig);
    }
};

int sc_main(int argc, char* argv[])
{
    Top top("top");        // top.name() returns "top"
    sc_start();
    return 0;
}
```

### 5.15.6 print and dump

virtual void **print**( std::ostream& = std::cout ) const;

Member function **print** shall print the character string returned by member function **name** to the stream passed as an argument. No additional characters shall be printed.

virtual void **dump**( std::ostream& = std::cout ) const;

Member function **dump** shall print at least the name and the kind of the **sc\_object** to the stream passed as an argument. The formatting shall be implementation-dependent. The purpose of **dump** is to allow an implementation to print diagnostic information to help the user debug an application.

### 5.15.7 Functions for object hierarchy traversal

The four functions in this subclause return information that supports the traversal of the object hierarchy. An implementation shall allow each of these four functions to be called at any stage during elaboration or simulation. If called before elaboration is complete, they shall return information concerning the partially constructed object hierarchy as it exists at the time the functions are called. In other words, a function shall return pointers to any objects that have been constructed before the time the function is called but will exclude any objects constructed after the function is called.

```
virtual const std::vector<sc_object*>& get_child_objects() const;
```

Member function **get\_child\_objects** shall return a **std::vector** containing a pointer to every instance of class **sc\_object** that is a child of the current **sc\_object** in the object hierarchy. The virtual function **sc\_object::get\_child\_objects** shall return an empty vector but shall be overridden by the implementation in those classes derived from class **sc\_object** that do have children, that is, class **sc\_module** and the implementation-defined classes associated with spawned and unspawned process instances.

```
sc_object* get_parent_object() const;
```

Member function **get\_parent\_object** shall return a pointer to the **sc\_object** that is the parent of the current object in the object hierarchy. If the current object is a top-level object, member function **get\_parent\_object** shall return the null pointer.

```
const std::vector<sc_object*>& sc_get_top_level_objects();
```

Function **sc\_get\_top\_level\_objects** shall return a **std::vector** containing pointers to all of the top-level **sc\_objects**.

```
sc_object* sc_find_object( const char* );
```

Function **sc\_find\_object** shall return a pointer to the **sc\_object** that has a hierarchical name that exactly matches the value of the string argument or shall return the null pointer if there is no **sc\_object** having a matching name.

*Examples:*

```
void scan_hierarchy(sc_object* obj)                // Traverse the entire object subhierarchy
                                                    // below a given object
{
    std::vector<sc_object*> children = obj->get_child_objects();
    for ( unsigned i = 0; i < children.size(); i++ )
        if ( children[i] )
            scan_hierarchy( children[i] );
}

std::vector<sc_object*> tops = sc_get_top_level_objects();
for ( unsigned i = 0; i < tops.size(); i++ )
    if ( tops[i] )
        scan_hierarchy( tops[i] );                // Traverse the object hierarchy below
                                                    // each top-level object

sc_object* obj = sc_find_object("foo.foobar");     // Find an object given its hierarchical name

sc_module* m;
if ( m = dynamic_cast<sc_module*>(obj) )           // Test whether the given object is a module
    ...                                             // The given object is a module

sc_object* parent = obj->get_parent_object();       // Get the parent of the given object
if (parent)                                         // parent is a null pointer for a top-level object
    std::cout << parent->name() << " " << parent->kind(); // Print the name and kind
```

### 5.15.8 Member functions for attributes

bool **add\_attribute**( sc\_attr\_base& );

Member function **add\_attribute** shall attempt to attach to the object of class **sc\_object** the attribute passed as an argument. If an attribute having the same name as the new attribute is already attached to this object, member function **add\_attribute** shall not attach the new attribute and shall return the value **false**. Otherwise, member function **add\_attribute** shall attach the new attribute and shall return the value **true**. The argument should be an object of class **sc\_attribute**, not **sc\_attr\_base**.

The lifetime of an attribute shall extend until the attribute has been completely removed from all objects. If an application deletes an attribute that is still attached to an object, the behavior of the implementation shall be undefined.

sc\_attr\_base\* **get\_attribute**( const std::string& );  
const sc\_attr\_base\* **get\_attribute**( const std::string& ) const;

Member function **get\_attribute** shall attempt to retrieve from the object of class **sc\_object** an attribute having the name passed as an argument. If an attribute with the given name is attached to this object, member function **get\_attribute** shall return a pointer to that attribute. Otherwise, member function **get\_attribute** shall return the null pointer.

sc\_attr\_base\* **remove\_attribute**( const std::string& );

Member function **remove\_attribute** shall attempt to remove from the object of class **sc\_object** an attribute having the name passed as an argument. If an attribute with the given name is attached to this object, member function **remove\_attribute** shall return a pointer to that attribute and remove the attribute from this object. Otherwise, member function **remove\_attribute** shall return the null pointer.

void **remove\_all\_attributes**();

Member function **remove\_all\_attributes** shall remove all attributes from the object of class **sc\_object**.

int **num\_attributes**() const;

Member function **num\_attributes** shall return the number of attributes attached to the object of class **sc\_object**.

sc\_attr\_cltn& **attr\_cltn**();  
const sc\_attr\_cltn& **attr\_cltn**() const;

Member function **attr\_cltn** shall return the collection of attributes attached to the object of class **sc\_object** (see 5.18).

NOTE—A pointer returned from function **get\_attribute** needs to be cast to type **sc\_attribute<T>\*** in order to access data member **value** of class **sc\_attribute**.

*Example:*

```
sc_signal<int> sig;  
...  
// Add an attribute to an sc_object  
sc_attribute<int> a("number", 1);  
sig.add_attribute(a);  
  
// Retrieve the attribute by name and modify the value  
sc_attribute<int>* ap;  
ap = (sc_attribute<int>*)sig.get_attribute("number");  
++ ap->value;
```

## 5.16 sc\_attr\_base

### 5.16.1 Description

Class **sc\_attr\_base** is the base class for attributes, storing only the name of the attribute. The name is used as a key when retrieving an attribute from an object. Every attribute attached to a specific object shall have a unique name but two or more attributes with identical names may be attached to distinct objects.

### 5.16.2 Class definition

```
namespace sc_core {  
  
class sc_attr_base  
{  
    public:  
        sc_attr_base( const std::string& );  
        sc_attr_base( const sc_attr_base& );  
        virtual ~sc_attr_base();  
  
        const std::string& name() const;  
  
    private:  
        // Disabled  
        sc_attr_base();  
        sc_attr_base& operator=( const sc_attr_base& );  
};  
  
} // namespace sc_core
```

### 5.16.3 Member functions

The constructors for class **sc\_attr\_base** shall set the name of the attribute to the string passed as an argument to the constructor.

Member function **name** shall return the name of the attribute.

## 5.17 **sc\_attribute**

### 5.17.1 Description

Class **sc\_attribute** stores the value of an attribute. It is derived from class **sc\_attr\_base**, which stores the name of the attribute. An attribute can be attached to an object of class **sc\_object**.

### 5.17.2 Class definition

```
namespace sc_core {

template <class T>
class sc_attribute
: public sc_attr_base
{
public:
    sc_attribute( const std::string& );
    sc_attribute( const std::string&, const T& );
    sc_attribute( const sc_attribute<T>& );
    virtual ~sc_attribute();
    T value;

private:
    // Disabled
    sc_attribute();
    sc_attribute<T>& operator=( const sc_attribute<T>& );
};

} // namespace sc_core
```

### 5.17.3 Template parameters

The argument passed to template **sc\_attribute** shall be of a *copy-constructible* type.

### 5.17.4 Member functions and data members

The constructors shall set the name and value of the attribute using the name (of type **std::string**) and value (of type **T**) passed as arguments to the constructor. If no value is passed to the constructor, the default constructor (of type **T**) shall be called to construct the value.

Data member **value** is the value of the attribute. An application may read or assign this public data member.



## 5.18 sc\_attr\_cltn

### 5.18.1 Description

Class **sc\_attr\_cltn** is a container class for attributes, as used in the implementation of class **sc\_object**. It provides iterators for traversing all of the attributes in an attribute collection.

### 5.18.2 Class definition

```
namespace sc_core {  
  
class sc_attr_cltn  
{  
    public:  
        typedef sc_attr_base* elem_type;  
        typedef elem_type* iterator;  
        typedef const elem_type* const_iterator;  
  
        iterator begin();  
        const_iterator begin() const;  
        iterator end();  
        const_iterator end() const;  
  
        // Other members  
        Implementation-defined  
  
    private:  
        // Disabled  
        sc_attr_cltn( const sc_attr_cltn& );  
        sc_attr_cltn& operator=( const sc_attr_cltn& );  
};  
  
} // namespace sc_core
```

### 5.18.3 Constraints on usage

An application shall not explicitly create an object of class **sc\_attr\_cltn**. An application may use the iterators to traverse the attribute collection returned by member function **attr\_cltn** of class **sc\_object**.

An implementation is only obliged to keep an attribute collection valid until a new attribute is attached to the **sc\_object** or an existing attribute is removed from the **sc\_object** in question. Hence an application should traverse the attribute collection immediately on return from member function **attr\_cltn**.

### 5.18.4 Iterators

```
iterator begin();  
const_iterator begin() const;  
iterator end();  
const_iterator end() const;
```

Member function **begin** shall return a pointer to the first element of the collection. Each element of the collection is itself a pointer to an attribute.

Member function **end** shall return a pointer to the element following the last element of the collection.

*Example:*

```
sc_signal<int> sig;  
...  
  
// Iterate through all the attributes of an sc_object  
sc_attr_cltn& c = sig.attr_cltn();  
for (sc_attr_cltn::iterator i = c.begin(); i < c.end(); i++)  
{  
    sc_attribute<int>* ap = dynamic_cast<sc_attribute<int>*>(*i);  
    if (ap) std::cout << ap->name() << "=" << ap->value << std::endl;  
}
```

## 6. Predefined channel class definitions

### 6.1 **sc\_signal\_in\_if**

#### 6.1.1 Description

Class **sc\_signal\_in\_if** is an interface proper used by predefined channels, including **sc\_signal**. Interface **sc\_signal\_in\_if** gives read access to the value of a signal.

#### 6.1.2 Class definition

```
namespace sc_core {

template <class T>
class sc_signal_in_if
: virtual public sc_interface
{
    public:
        virtual const T& read() const = 0;
        virtual const sc_event& value_changed_event() const = 0;
        virtual bool event() const = 0;

    protected:
        sc_signal_in_if();

    private:
        // Disabled
        sc_signal_in_if( const sc_signal_in_if<T>& );
        sc_signal_in_if<T>& operator=( const sc_signal_in_if<T>& );
};

} // namespace sc_core
```

#### 6.1.3 Member functions

The following member functions are all pure virtual functions. The descriptions refer to the expected definitions of the functions when overridden in a channel that implements this interface. The precise semantics will be channel-specific.

Member function **read** shall return a reference to the current value of the channel.

Member function **value\_changed\_event** shall return a reference to an event that is notified whenever the value of the channel is written or modified.

Member function **event** shall return the value **true** if and only if the value of the channel was written or modified in the immediately preceding delta cycle.

NOTE—The value of the channel may have been modified in the evaluation phase or in the update phase of the immediately preceding delta cycle, depending on whether it is a hierarchical channel or a primitive channel (for example, **sc\_signal**).

## 6.2 **sc\_signal\_in\_if<bool> and sc\_signal\_in\_if<sc\_dt::sc\_logic>**

### 6.2.1 Description

Classes **sc\_signal\_in\_if<bool>** and **sc\_signal\_in\_if<sc\_dt::sc\_logic>** are interfaces proper that provide additional member functions appropriate for two-valued signals.

### 6.2.2 Class definition

```
namespace sc_core {

template <>
class sc_signal_in_if<bool>
: virtual public sc_interface
{
    public:
        virtual const T& read() const = 0;

        virtual const sc_event& value_changed_event() const = 0;
        virtual const sc_event& posedge_event() const = 0;
        virtual const sc_event& negedge_event() const = 0;

        virtual bool event() const = 0;
        virtual bool posedge() const = 0;
        virtual bool negedge() const = 0;

    protected:
        sc_signal_in_if();

    private:
        // Disabled
        sc_signal_in_if( const sc_signal_in_if<bool>& );
        sc_signal_in_if<bool>& operator=( const sc_signal_in_if<bool>& );
};

template <>
class sc_signal_in_if<sc_dt::sc_logic>
: virtual public sc_interface
{
    public:
        virtual const T& read() const = 0;

        virtual const sc_event& value_changed_event() const = 0;
        virtual const sc_event& posedge_event() const = 0;
        virtual const sc_event& negedge_event() const = 0;

        virtual bool event() const = 0;
        virtual bool posedge() const = 0;
        virtual bool negedge() const = 0;

    protected:
        sc_signal_in_if();
};
```

```
private:
    // Disabled
    sc_signal_in_if( const sc_signal_in_if<sc_dt::sc_logic>& );
    sc_signal_in_if<sc_dt::sc_logic>& operator= ( const sc_signal_in_if<sc_dt::sc_logic>& );
};

} // namespace sc_core
```

### 6.2.3 Member functions

The following list is incomplete. For the remaining member functions, refer to the definitions of the member functions for class **sc\_signal\_in\_if** (see 6.1.3).

Member function **posedge\_event** shall return a reference to an event that is notified whenever the value of the channel (as returned by member function **read**) changes and the new value of the channel is **true** or '1'.

Member function **negedge\_event** shall return a reference to an event that is notified whenever the value of the channel (as returned by member function **read**) changes and the new value of the channel is **false** or '0'.

Member function **posedge** shall return the value **true** if and only if the value of the channel changed in the update phase of the immediately preceding delta cycle and the new value of the channel is **true** or '1'.

Member function **negedge** shall return the value **true** if and only if the value of the channel changed in the update phase of the immediately preceding delta cycle and the new value of the channel is **false** or '0'.

## 6.3 sc\_signal\_inout\_if

### 6.3.1 Description

Class **sc\_signal\_inout\_if** is an interface proper that is used by predefined channels, including **sc\_signal**. Interface **sc\_signal\_inout\_if** gives both read and write access to the value of a signal, and is derived from a further interface proper **sc\_signal\_write\_if**.

### 6.3.2 Class definition

```
namespace sc_core {

template <class T>
class sc_signal_write_if
{
public:
    virtual void write( const T& ) = 0;

protected:
    sc_signal_write_if();

private:
    // Disabled
    sc_signal_write_if( const sc_signal_write_if<T>& );
    sc_signal_write_if<T>& operator= ( const sc_signal_write_if<T>& );
};

template <class T>
class sc_signal_inout_if
: public sc_signal_in_if<T> , public sc_signal_write_if<T>
{
protected:
    sc_signal_inout_if();

private:
    // Disabled
    sc_signal_inout_if( const sc_signal_inout_if<T>& );
    sc_signal_inout_if<T>& operator= ( const sc_signal_inout_if<T>& );
};

} // namespace sc_core
```

### 6.3.3 write

Member function **write** shall modify the value of the channel such that the channel appears to have the new value (as returned by member function **read**) in the next delta cycle but not before then. The new value is passed as an argument to the function.

## 6.4 **sc\_signal**

### 6.4.1 Description

Class **sc\_signal** is a predefined primitive channel intended to model the behavior of a single piece of wire carrying a digital electronic signal.

### 6.4.2 Class definition

```
namespace sc_core {

template <class T>
class sc_signal
: public sc_signal_inout_if<T>, public sc_prim_channel
{
public:
    sc_signal();
    explicit sc_signal( const char* );
    virtual ~sc_signal();

    virtual void register_port( sc_port_base&, const char* );

    virtual const T& read() const;
    operator const T& () const;

    virtual void write( const T& );
    sc_signal<T>& operator=( const T& );
    sc_signal<T>& operator=( const sc_signal<T>& );

    virtual const sc_event& default_event() const;
    virtual const sc_event& value_changed_event() const;
    virtual bool event() const;

    virtual void print( std::ostream& = std::cout ) const;
    virtual void dump( std::ostream& = std::cout ) const;
    virtual const char* kind() const;

protected:
    virtual void update();

private:
    // Disabled
    sc_signal( const sc_signal<T>& );
};

template <class T>
inline std::ostream& operator<<( std::ostream&, const sc_signal<T>& );

} // namespace sc_core
```

### 6.4.3 Template parameter **T**

The argument passed to template **sc\_signal** shall be either a C++ type for which the predefined semantics for assignment and equality are adequate (for example, a fundamental type or a pointer), or a type **T** that obeys each of the following rules:

- a) The following equality operator shall be defined for the type **T** and should return the value **true** if and only if the two values being compared are to be regarded as indistinguishable for the purposes of signal propagation (that is, an event occurs only if the values are different). The implementation shall use this operator within the implementation of the signal to determine whether an event has occurred.

```
bool T::operator==( const T& );
```

- b) The following stream operator shall be defined and should copy the state of the object given as the second argument to the stream given as the first argument. The way in which the state information is formatted is undefined by this standard. The implementation shall use this operator in implementing the behavior of the member functions **print** and **dump**.

```
std::ostream& operator<< ( std::ostream&, const T& );
```

- c) If the default assignment semantics are inadequate (in the sense given in this subclause), the following assignment operator should be defined for the type **T**. In either case (default assignment or explicit operator), the semantics of assignment should be sufficient to assign the state of an object of type **T** such that the value of the left operand is indistinguishable from the value of the right operand using the equality operator mentioned in this subclause. The implementation shall use this assignment operator within the implementation of the signal when assigning or copying values of type **T**.

```
const T& operator= ( const T& );
```

- d) If any constructor for type **T** exists, a default constructor for type **T** shall be defined.
- e) If the class template is used to define a signal to which a port of type **sc\_in**, **sc\_inout**, or **sc\_out** is bound, the following function shall be defined:

```
void sc_trace( sc_trace_file*, const T&, const std::string& );
```

NOTE 1—The equality and assignment operators are not obliged to compare and assign the complete state of the object, although they should typically do so. For example, diagnostic information may be associated with an object that is not to be propagated through the signal.

NOTE 2—The SystemC data types proper (**sc\_dt::sc\_int**, **sc\_dt::sc\_logic**, and so forth) all conform to the rule set just given.

NOTE 3—It is illegal to pass class **sc\_module** (for example) as a template argument to class **sc\_signal**, because **sc\_module::operator==** does not exist. It is legal to pass type **sc\_module\*** through a signal, although this would be regarded as an abuse of the module hierarchy and thus bad practice.

### 6.4.4 Reading and writing signals

A signal is *read* by calling member function **read** or **operator const T& ()**.

A signal is *written* by calling member function **write** or **operator=** of the given signal object. It shall be an error to write a given signal instance from more than one process instance. A signal may be written during elaboration to initialize the value of the signal.



Signals are typically read and written during the evaluation phase but the value of the signal is only modified during the subsequent update phase. If and only if the value of the signal actually changes as a result of being written, an event (the *value-changed event*) shall be notified in the delta notification phase that immediately follows.

If a given signal is written on multiple occasions within a particular evaluation phase, the value to which the signal changes in the immediately following update phase shall be determined by the most recent write, that is, *the last write wins*.

NOTE 1—The specialized ports **sc\_inout** and **sc\_out** have a member function **initialize** for the purpose of initializing the value of a signal during elaboration.

NOTE 2—If the value of a signal is read during elaboration, the value returned will be the initial value of the signal as created by the default constructor for type **T**.

NOTE 3—If a given signal is written and read during the same evaluation phase, the old value will be read. The value written will not be available to be read until the subsequent evaluation phase.

### 6.4.5 Constructors

**sc\_signal();**

This constructor shall call the base class constructor from its initializer list as follows:

```
sc_prim_channel( sc_gen_unique_name( "signal" ) )
```

explicit **sc\_signal( const char\* name\_ );**

This constructor shall call the base class constructor from its initializer list as follows:

```
sc_prim_channel( name_ )
```

Both constructors shall initialize the value of the signal by calling the default constructor for type **T** from their initializer lists.

### 6.4.6 register\_port

virtual void **register\_port( sc\_port\_base&, const char\* );**

Member function **register\_port** of class **sc\_interface** shall be overridden in class **sc\_signal**, and shall perform the following error check. It is an error if more than one port of type **sc\_signal\_inout\_if** is bound to a given signal.

### 6.4.7 Member functions for reading

virtual const **T& read() const;**

Member function **read** shall return a reference to the current value of the signal but shall not modify the state of the signal.

**operator const T& () const;**

**operator const T& ()** shall return a reference to the current value of the signal (as returned by member function **read**).

#### 6.4.8 Member functions for writing

virtual void **write**( const T& );

Member function **write** shall modify the value of the signal such that the signal appears to have the new value (as returned by member function **read**) in the next delta cycle but not before then. This shall be accomplished using the update request mechanism of the primitive channel. The new value is passed as an argument to member function **write**.

**operator=**

The behavior of **operator=** shall be equivalent to the following definitions:

```
sc_signal<T>& operator= ( const T& arg ) { write( arg ); return *this; }
sc_signal<T>& operator= ( const sc_signal<T>& arg ) { write( arg.read() ); return *this; }
```

virtual void **update**();

Member function **update** of class **sc\_prim\_channel** shall be overridden by the implementation in class **sc\_signal** to implement the updating of the signal value that occurs as a result of the signal being written. Member function **update** shall modify the current value of the signal such that it gets the new value (as passed as an argument to member function **write**), and shall cause the value-changed event to be notified in the immediately following delta notification phase if the value of the signal has changed.

NOTE—Member function **update** is called by the scheduler but typically is not called by an application. However, member function **update** of class **sc\_signal** may be called from member function **update** of a class derived from class **sc\_signal**.

#### 6.4.9 Member functions for events

virtual const sc\_event& **default\_event**() const;

virtual const sc\_event& **value\_changed\_event**() const;

Member functions **default\_event** and **value\_changed\_event** shall both return a reference to the value-changed event.

virtual bool **event**() const;

Member function **event** shall return the value **true** if and only if the value of the signal changed in the update phase of the immediately preceding delta cycle; that is, a member function **write** or **operator=** was called in the immediately preceding evaluation phase, and the value written or assigned was different from the previous value of the signal.

NOTE—Member function **event** returns **true** when called from a process that was executed as a direct result of the value-changed event of that same signal instance being notified.

#### 6.4.10 Diagnostic member functions

virtual void **print**( std::ostream& = std::cout ) const;

Member function **print** shall print the current value of the signal to the stream passed as an argument by calling **operator<<** (**std::ostream&**, **T&**). No additional characters shall be printed.

virtual void **dump**( std::ostream& = std::cout ) const;

Member function **dump** shall print at least the hierarchical name, the current value, and the new value of the signal to the stream passed as an argument. The formatting shall be implementation-defined.

virtual const char\* **kind**() const;

Member function **kind** shall return the string "sc\_signal".

#### 6.4.11 operator<<

template <class T>

inline std::ostream& **operator<<**( std::ostream& , const sc\_signal<T>& );

**operator<<** shall print the current value of the signal passed as the second argument to the stream passed as the first argument by calling **operator<<**( std::ostream& , T& ).

*Example:*

```
SC_MODULE(M)
{
    sc_signal<int> sig;

    SC_CTOR(M)
    {
        SC_THREAD(writer);
        SC_THREAD(reader);
        SC_METHOD(writer2);
        sensitive << sig;           // Sensitive to the default event
    }
    void writer()
    {
        wait(50, SC_NS);
        sig.write(1);
        sig.write(2);
        wait(50, SC_NS);
        sig = 3;                    // Calls operator= ( const T& )
    }
    void reader()
    {
        wait(sig.value_changed_event());
        int i = sig.read();         // Reads a value of 2
        wait(sig.value_changed_event());
        i = sig;                    // Calls operator const T& () which returns a value of 3
    }
    void writer2()
    {
        sig.write(sig + 1);        // An error. A signal shall not have multiple writers
    }
};
```

NOTE—The following classes are related to class **sc\_signal**:

- The classes **sc\_signal<bool>** and **sc\_signal<sc\_dt::sc\_logic>** provide additional member functions appropriate for two-valued signals.
- The class **sc\_buffer** is derived from **sc\_signal** but differs in that the value-changed event is notified whenever the buffer is written whether or not the value of the buffer has changed.
- The class **sc\_clock** is derived from **sc\_signal** and generates a periodic clock signal.
- The class **sc\_signal\_resolved** allows multiple writers.
- The classes **sc\_in**, **sc\_out**, and **sc\_inout** are specialized ports that may be bound to signals, and which provide functions to conveniently access the member functions of the signal through the port.

## 6.5 **sc\_signal<bool> and sc\_signal<sc\_dt::sc\_logic>**

### 6.5.1 Description

Classes **sc\_signal<bool>** and **sc\_signal<sc\_dt::sc\_logic>** are predefined primitive channels that provide additional member functions appropriate for two-valued signals.

### 6.5.2 Class definition

```
namespace sc_core {  
  
template <>  
class sc_signal<bool>  
: public sc_signal_inout_if<bool>, public sc_prim_channel  
{  
    public:  
        sc_signal();  
        explicit sc_signal( const char* );  
        virtual ~sc_signal();  
  
        virtual void register_port( sc_port_base&, const char* );  
  
        virtual const bool& read() const;  
        operator const bool& () const;  
  
        virtual void write( const bool& );  
        sc_signal<bool>& operator= ( const bool& );  
        sc_signal<bool>& operator= ( const sc_signal<bool>& );  
  
        virtual const sc_event& default_event() const;  
  
        virtual const sc_event& value_changed_event() const;  
        virtual const sc_event& posedge_event() const;  
        virtual const sc_event& negedge_event() const;  
  
        virtual bool event() const;  
        virtual bool posedge() const;  
        virtual bool negedge() const;  
  
        virtual void print( std::ostream& = std::cout ) const;  
        virtual void dump( std::ostream& = std::cout ) const;  
        virtual const char* kind() const;  
  
    protected:  
        virtual void update();  
  
    private:  
        // Disabled  
        sc_signal( const sc_signal<bool>& );  
};  
  
template <>  
class sc_signal<sc_dt::sc_logic>
```

```

: public sc_signal_inout_if<sc_dt::sc_logic>, public sc_prim_channel
{
    public:
        sc_signal();
        explicit sc_signal( const char* );
        virtual ~sc_signal();

        virtual void register_port( sc_port_base&, const char* );

        virtual const sc_dt::sc_logic& read() const;
        operator const sc_dt::sc_logic& () const;

        virtual void write( const sc_dt::sc_logic& );
        sc_signal<sc_dt::sc_logic>& operator=( const sc_dt::sc_logic& );
        sc_signal<sc_dt::sc_logic>& operator=( const sc_signal<sc_dt::sc_logic>& );

        virtual const sc_event& default_event() const;

        virtual const sc_event& value_changed_event() const;
        virtual const sc_event& posedge_event() const;
        virtual const sc_event& negedge_event() const;

        virtual bool event() const;
        virtual bool posedge() const;
        virtual bool negedge() const;

        virtual void print( std::ostream& = std::cout ) const;
        virtual void dump( std::ostream& = std::cout ) const;
        virtual const char* kind() const;

    protected:
        virtual void update();

    private:
        // Disabled
        sc_signal( const sc_signal<sc_dt::sc_logic>& );
};

// namespace sc_core

```

### 6.5.3 Member functions

The following list is incomplete. For the remaining member functions, refer to the definitions of the member functions for class **sc\_signal** (see 6.4).

```
virtual const sc_event& posedge_event () const;
```

Member function **posedge\_event** shall return a reference to an event that is notified whenever the value of the signal (as returned by member function **read**) changes and the new value of the signal is **true** or '1'.

virtual const sc\_event& **negedge\_event**() const;

Member function **negedge\_event** shall return a reference to an event that is notified whenever the value of the signal (as returned by member function **read**) changes and the new value of the signal is **false** or '0'.

virtual bool **posedge** () const;

Member function **posedge** shall return the value **true** if and only if the value of the signal changed in the update phase of the immediately preceding delta cycle and the new value of the signal is **true** or '1'.

virtual bool **negedge**() const;

Member function **negedge** shall return the value **true** if and only if the value of the signal changed in the update phase of the immediately preceding delta cycle and the new value of the signal is **false** or '0'.

*Example:*

```
sc_signal<bool> clk;  
...  
void thread_process()  
{  
    for (;;)   
    {  
        if (clk.posedge())  
            wait(clk.negedge_event());  
        ...  
    }  
}
```

## 6.6 sc\_buffer

### 6.6.1 Description

Class **sc\_buffer** is a predefined primitive channel derived from class **sc\_signal**. Class **sc\_buffer** differs from class **sc\_signal** in that a value-changed event is notified whenever the buffer is written rather than only when the value of the signal is changed. A *buffer* is an object of the class **sc\_buffer**.

### 6.6.2 Class definition

```
namespace sc_core {

template <class T>
class sc_buffer
: public sc_signal<T>
{
public:
    sc_buffer();
    explicit sc_buffer( const char* );

    virtual void write( const T& );

    sc_buffer<T>& operator= ( const T& );
    sc_buffer<T>& operator= ( const sc_signal<T>& );
    sc_buffer<T>& operator= ( const sc_buffer<T>& );

    virtual const char* kind() const;

protected:
    virtual void update();

private:
    // Disabled
    sc_buffer( const sc_buffer<T>& );
};

// namespace sc_core
```

### 6.6.3 Constructors

**sc\_buffer**();

This constructor shall call the base class constructor from its initializer list as follows:

```
sc_signal<T>( sc_gen_unique_name( "buffer" ) )
```

explicit **sc\_buffer**( const char\* name\_ );

This constructor shall call the base class constructor from its initializer list as follows:

```
sc_signal<T>( name_ )
```



#### 6.6.4 Member functions

virtual void **write**( const T& );

Member function **write** shall modify the value of the buffer such that the buffer appears to have the new value (as returned by member function **read**) in the next delta cycle but not before then. This shall be accomplished using the update request mechanism of the primitive channel. The new value is passed as an argument to member function **write**.

**operator=**

The behavior of **operator=** shall be equivalent to the following definitions:

```
sc_buffer<T>& operator= ( const T& arg ) { write( arg ); return *this; }
```

```
sc_buffer<T>& operator= ( const sc_signal<T>& arg ) { write( arg.read() ); return *this; }
```

```
sc_buffer<T>& operator= ( const sc_buffer<T>& arg ) { write( arg.read() ); return *this; }
```

virtual void **update**();

Member function **update** of class **sc\_signal** shall be overridden by the implementation in class **sc\_buffer** to implement the updating of the buffer value that occurs as a result of the buffer being written. Member function **update** shall modify the current value of the buffer such that it gets the new value (as passed as an argument to member function **write**) and shall cause the value-changed event to be notified in the immediately following delta notification phase, regardless of whether the value of the buffer has changed (see 6.4.4 and 6.4.8). (This is in contrast to member function **update** of the base class **sc\_signal**, which only causes the value-changed event to be notified if the new value is different from the old value.)

In other words, suppose the current value of the buffer is V, and member function **write** is called with argument value V. Function **write** will store the new value V (in some implementation-defined storage area distinct from the current value of the buffer) and will call **request\_update**. Member function **update** will be called back during the update phase and will set the current value of the buffer to the new value V. The current value of the buffer will not change, because the old value is equal to the new value but the value-changed event will be notified nonetheless.

virtual const char\* **kind**() const;

Member function **kind** shall return the string "**sc\_buffer**".

*Example:*

```
SC_MODULE(M)
{
    sc_buffer<int> buf;

    SC_CTOR(M)
    {
        SC_THREAD(writer);
        SC_METHOD(reader);
        sensitive << buf;
    }
    void writer()
    {
        buf.write(1);
        wait(SC_ZERO_TIME);
        buf.write(1);
    }
    void reader()
    {
        // Executed during initialization and then twice more with buf = 0, 1, 1
        std::cout << buf << std::endl;
    }
};
```

## 6.7 sc\_clock

### 6.7.1 Description

Class **sc\_clock** is a predefined primitive channel derived from the class **sc\_signal** and intended to model the behavior of a digital clock signal. A *clock* is an object of the class **sc\_clock**. The value and events associated with the clock are accessed through the interface **sc\_signal\_in\_if<bool>**.

### 6.7.2 Class definition

```
namespace sc_core {  
  
class sc_clock  
: public sc_signal<bool>  
{  
    public:  
        sc_clock();  
        explicit sc_clock( const char* name_ );  
  
        sc_clock( const char* name_,  
                  const sc_time& period_,  
                  double duty_cycle_ = 0.5,  
                  const sc_time& start_time_ = SC_ZERO_TIME,  
                  bool posedge_first_ = true );  
  
        sc_clock( const char* name_,  
                  double period_v_,  
                  sc_time_unit period_tu_,  
                  double duty_cycle_ = 0.5 );  
  
        sc_clock( const char* name_,  
                  double period_v_,  
                  sc_time_unit period_tu_,  
                  double duty_cycle_,  
                  double start_time_v_,  
                  sc_time_unit start_time_tu_,  
                  bool posedge_first_ = true );  
  
        virtual ~sc_clock();  
  
        virtual void write( const bool& );  
  
        const sc_time& period() const;  
        double duty_cycle() const;  
        const sc_time& start_time() const;  
        bool posedge_first() const;  
  
        virtual const char* kind() const;  
  
    protected:  
        virtual void before_end_of_elaboration();  
  
    private:  
        // Disabled
```

```

    sc_clock( const sc_clock& );
    sc_clock& operator=( const sc_clock& );
};

typedef sc_in<bool> sc_in_clk ;

}      // namespace sc_core

```

### 6.7.3 Characteristic properties

A clock is characterized by the following properties:

- Period*—The time interval between two consecutive transitions from value **false** to value **true**, which shall be equal to the time interval between two consecutive transitions from value **true** to value **false**. The period shall be greater than zero. The default period is 1 nanosecond.
- Duty cycle*—The proportion of the period during which the clock has the value **true**. The duty cycle shall lie between the limits 0.0 and 1.0, exclusive. The default duty cycle is 0.5.
- Start time*—The absolute time of the first transition of the value of the clock (**false** to **true** or **true** to **false**). The default start time is zero.
- Posedge\_first*—If posedge\_first is **true**, the clock is initialized to the value **false**, and changes from **false** to **true** at the start time. If posedge\_first is **false**, the clock is initialized to the value **true**, and changes from **true** to **false** at the start time. The default value of posedge\_first is **true**.

NOTE—A clock does not have a stop time but will stop in any case when function **sc\_stop** is called.

### 6.7.4 Constructors

The constructors shall set the characteristic properties of the clock as defined by the constructor arguments. Any characteristic property not defined by the constructor arguments shall take a default value as defined in 6.7.3.

The default constructor shall call the base class constructor from its initializer list as follows:

```
sc_signal<bool>( sc_gen_unique_name( "clock" ) )
```

### 6.7.5 write

```
virtual void write( const bool& );
```

It shall be an error for an application to call member function **write**. The member function **write** of the base class **sc\_signal** is not applicable to clocks.

### 6.7.6 Diagnostic member functions

```
const sc_time& period() const;
```

Member function **period** shall return the period of the clock.

```
double duty_cycle() const;
```

Member function **duty\_cycle** shall return the duty cycle of the clock.

```
const sc_time& start_time() const;
```

Member function **start\_time** shall return the start time of the clock.

bool **posedge\_first**() const;

Member function **posedge\_first** shall return the value of the **posedge\_first** property of the clock.

virtual const char\* **kind**() const;

Member function **kind** shall return the string "**sc\_clock**".

### 6.7.7 before\_end\_of\_elaboration

virtual void **before\_end\_of\_elaboration**();

Member function **before\_end\_of\_elaboration**, which is defined in the class **sc\_prim\_channel**, shall be overridden by the implementation in the current class with a behavior that is implementation-defined.

NOTE 1—An implementation may use **before\_end\_of\_elaboration** to spawn one or more static processes to generate the clock.

NOTE 2—If this member function is overridden in a class derived from the current class, function **before\_end\_of\_elaboration** as overridden in the current class should be called explicitly from the overridden member function of the derived class in order to invoke the implementation-defined behavior.

### 6.7.8 sc\_in\_clk

typedef sc\_in<bool> **sc\_in\_clk**;

The typedef **sc\_in\_clk** is provided for convenience when adding clock inputs to a module and for backward compatibility with earlier versions of SystemC. An application may use **sc\_in\_clk** or **sc\_in<bool>** interchangeably.

## 6.8 sc\_in

### 6.8.1 Description

Class **sc\_in** is a specialized port class for use with signals. It provides functions to conveniently access certain member functions of the channel to which the port is bound. It may be used to model an input pin on a module.

### 6.8.2 Class definition

```
namespace sc_core {

template <class T>
class sc_in
: public sc_port<sc_signal_in_if<T>,1>
{
public:
    sc_in();
    explicit sc_in( const char* );
    virtual ~sc_in();

    void bind ( const sc_signal_in_if<T>& );
    void operator() ( const sc_signal_in_if<T>& );

    void bind ( sc_port<sc_signal_in_if<T>, 1>& );
    void operator() ( sc_port<sc_signal_in_if<T>, 1>& );

    void bind ( sc_port<sc_signal_inout_if<T>, 1>& );
    void operator() ( sc_port<sc_signal_inout_if<T>, 1>& );

    virtual void end_of_elaboration();

    const T& read() const;
    operator const T& () const;

    const sc_event& default_event() const;
    const sc_event& value_changed_event() const;
    bool event() const;
    sc_event_finder& value_changed() const;

    virtual const char* kind() const;

private:
    // Disabled
    sc_in( const sc_in<T>& );
    sc_in<T>& operator=( const sc_in<T>& );
};

template <class T>
inline void sc_trace( sc_trace_file*, const sc_in<T>&, const std::string& );

} // namespace sc_core
```

### 6.8.3 Member functions

The constructors shall pass their arguments to the corresponding constructor for the base class **sc\_port**.

Member function **bind** and **operator()** shall each call member function **bind** of the base class **sc\_port**, passing through their parameters as arguments to function **bind**, in order to bind the object of class **sc\_in** to the channel or port instance passed as an argument.

Member function **read** and **operator const T&()** shall each call member function **read** of the object to which the port is bound using **operator->** of class **sc\_port**, that is:

`(*this)->read()`

Member functions **default\_event**, **value\_changed\_event**, and **event** shall each call the corresponding member function of the object to which the port is bound using **operator->** of class **sc\_port**, for example:

`(*this)->event()`

Member function **value\_changed** shall return a reference to class **sc\_event\_finder**, where the event finder object itself shall be constructed using the member function **value\_changed\_event** (see 5.7).

Member function **kind** shall return the string "**sc\_in**".

### 6.8.4 Function **sc\_trace**

template <class T>

inline void **sc\_trace**( sc\_trace\_file\*, const sc\_in<T>&, const std::string& );

Function **sc\_trace** shall trace the channel to which the port passed as the second argument is bound (see 8.1) by calling function **sc\_trace** with a second argument of type **const T&** (see 6.4.3). The port need not have been bound at the point during elaboration when function **sc\_trace** is called. In this case, the implementation shall defer the call to trace the signal until after the port has been bound and the identity of the signal is known.

### 6.8.5 end\_of\_elaboration

virtual void **end\_of\_elaboration**();

Member function **end\_of\_elaboration**, which is defined in the class **sc\_port**, shall be overridden by the implementation in the current class with a behavior that is implementation-defined.

NOTE 1—An implementation may use **end\_of\_elaboration** to implement the deferred call to **sc\_trace**.

NOTE 2—If this member function is overridden in a class derived from the current class, function **end\_of\_elaboration** as overridden in the current class should be called explicitly from the overridden member function of the derived class in order to invoke the implementation-defined behavior.

## 6.9 sc\_in<bool> and sc\_in<sc\_dt::sc\_logic>

### 6.9.1 Description

Class **sc\_in<bool>** and **sc\_in<sc\_dt::sc\_logic>** are specialized port classes that provide additional member functions for two-valued signals.

### 6.9.2 Class definition

```
namespace sc_core {

template <>
class sc_in<bool>
: public sc_port<sc_signal_in_if<bool>,1>
{
public:
    sc_in();
    explicit sc_in( const char* );
    virtual ~sc_in();

    void bind ( const sc_signal_in_if<bool>& );
    void operator() ( const sc_signal_in_if<bool>& );

    void bind ( sc_port<sc_signal_in_if<bool>, 1>& );
    void operator() ( sc_port<sc_signal_in_if<bool>, 1>& );

    void bind ( sc_port<sc_signal_inout_if<bool>, 1>& );
    void operator() ( sc_port<sc_signal_inout_if<bool>, 1>& );

    virtual void end_of_elaboration();

    const bool& read() const;
    operator const bool& () const;

    const sc_event& default_event() const;
    const sc_event& value_changed_event() const;
    const sc_event& posedge_event() const;
    const sc_event& negedge_event() const;

    bool event() const;
    bool posedge() const;
    bool negedge() const;

    sc_event_finder& value_changed() const;
    sc_event_finder& pos() const;
    sc_event_finder& neg() const;

    virtual const char* kind() const;

private:
    // Disabled
    sc_in( const sc_in<bool>& );
    sc_in<bool>& operator=( const sc_in<bool>& );
};
```



```

template <
inline void sc_trace<bool>( sc_trace_file*, const sc_in<bool>&, const std::string& );

template <
class sc_in<sc_dt::sc_logic>
: public sc_port<sc_signal_in_if<sc_dt::sc_logic>,1>
{
    public:
        sc_in();
        explicit sc_in( const char* );
        virtual ~sc_in();

        void bind ( const sc_signal_in_if<sc_dt::sc_logic>& );
        void operator() ( const sc_signal_in_if<sc_dt::sc_logic>& );

        void bind ( sc_port<sc_signal_in_if<sc_dt::sc_logic>, 1>& );
        void operator() ( sc_port<sc_signal_in_if<sc_dt::sc_logic>, 1>& );

        void bind ( sc_port<sc_signal_inout_if<sc_dt::sc_logic>, 1>& );
        void operator() ( sc_port<sc_signal_inout_if<sc_dt::sc_logic>, 1>& );

        virtual void end_of_elaboration();

        const sc_dt::sc_logic& read() const;
        operator const sc_dt::sc_logic& () const;

        const sc_event& default_event() const;
        const sc_event& value_changed_event() const;
        const sc_event& posedge_event() const;
        const sc_event& negedge_event() const;

        bool event() const;
        bool posedge() const;
        bool negedge() const;

        sc_event_finder& value_changed() const;
        sc_event_finder& pos() const;
        sc_event_finder& neg() const;

        virtual const char* kind() const;

    private:
        // Disabled
        sc_in( const sc_in<sc_dt::sc_logic>& );
        sc_in<sc_dt::sc_logic>& operator= ( const sc_in<sc_dt::sc_logic>& );
};

template <
inline void
sc_trace<sc_dt::sc_logic>( sc_trace_file*, const sc_in<sc_dt::sc_logic>&, const std::string& );

} // namespace sc_core

```

### 6.9.3 Member functions

The following list is incomplete. For the remaining member functions and for the function **sc\_trace**, refer to the definitions of the member functions for class **sc\_in** (see 6.8.3).

Member functions **posedge\_event**, **negedge\_event**, **posedge**, and **negedge** shall each call the corresponding member function of the object to which the port is bound using **operator->** of class **sc\_port**, for example:

(\*this)->negedge()

Member functions **pos** and **neg** shall return a reference to class **sc\_event\_finder**, where the event finder object itself shall be constructed using the member function **posedge\_event** or **negedge\_event**, respectively (see 5.7).

## 6.10 sc\_inout

### 6.10.1 Description

Class **sc\_inout** is a specialized port class for use with signals. It provides functions to conveniently access certain member functions of the channel to which the port is bound. It may be used to model an output pin or a bidirectional pin on a module.

### 6.10.2 Class definition

```
namespace sc_core {

template <class T>
class sc_inout
: public sc_port<sc_signal_inout_if<T>,1>
{
public:
    sc_inout();
    explicit sc_inout( const char* );
    virtual ~sc_inout();

    void initialize( const T& );
    void initialize( const sc_signal_in_if<T>& );

    virtual void end_of_elaboration();

    const T& read() const;
    operator const T& () const;

    void write( const T& );
    sc_inout<T>& operator=( const T& );
    sc_inout<T>& operator=( const sc_signal_in_if<T>& );
    sc_inout<T>& operator=( const sc_port< sc_signal_in_if<T>, 1>& );
    sc_inout<T>& operator=( const sc_port< sc_signal_inout_if<T>, 1>& );
    sc_inout<T>& operator=( const sc_inout<T>& );

    const sc_event& default_event() const;
    const sc_event& value_changed_event() const;
    bool event() const;
    sc_event_finder& value_changed() const;

    virtual const char* kind() const;

private:
    // Disabled
    sc_inout( const sc_inout<T>& );
};

template <class T>
inline void sc_trace( sc_trace_file*, const sc_inout<T>&, const std::string& );

} // namespace sc_core
```

### 6.10.3 Member functions

The constructors shall pass their arguments to the corresponding constructor for the base class **sc\_port**.

Member function **read** and **operator const T&()** shall each call member function **read** of the object to which the port is bound using **operator->** of class **sc\_port**, that is:

```
(*this)->read()
```

Member function **write** and **operator=** shall each call the member function **write** of the object to which the port is bound using **operator->** of class **sc\_port**, calling member function **read** to get the value of the parameter, where the parameter is an interface or a port, for example:

```
sc_inout<T>& operator= ( const sc_inout<T>& port_ )  
{ (*this)->write( port_->read() ); return *this; }
```

Member function **write** shall not be called during elaboration before the port has been bound (see 6.10.4).

Member functions **default\_event**, **value\_changed\_event**, and **event** shall each call the corresponding member function of the object to which the port is bound using **operator->** of class **sc\_port**, for example:

```
(*this)->event()
```

Member function **value\_changed** shall return a reference to class **sc\_event\_finder**, where the event finder object itself shall be constructed using the member function **value\_changed\_event** (see 5.7).

Member function **kind** shall return the string "**sc\_inout**".

### 6.10.4 initialize

Member function **initialize** shall set the initial value of the signal to which the port is bound by calling member function **write** of that signal using the value passed as an argument to member function **initialize**. If the actual argument is a channel, the initial value shall be determined by reading the value of the channel. The port need not have been bound at the point during elaboration when member function **initialize** is called. In this case, the implementation shall defer the call to **write** until after the port has been bound and the identity of the signal is known.

NOTE 1—A port of class **sc\_in** will be bound to exactly one signal but the binding may be performed indirectly through a port of the parent module.

NOTE 2—The purpose of member function **initialize** is to allow the value of a port to be initialized during elaboration before the port being bound. However, member function **initialize** may be called during elaboration or simulation.

### 6.10.5 Function **sc\_trace**

```
template <class T>
```

```
inline void sc_trace( sc_trace_file*, const sc_in<T>&, const std::string& );
```

Function **sc\_trace** shall trace the channel to which the port passed as the second argument is bound (see 8.1) by calling function **sc\_trace** with a second argument of type **const T&** (see 6.4.3). The port need not have been bound at the point during elaboration when function **sc\_trace** is called. In this case, the implementation shall defer the call to trace the signal until after the port has been bound and the identity of the signal is known.

### 6.10.6 end\_of\_elaboration

virtual void **end\_of\_elaboration**();

Member function **end\_of\_elaboration**, which is defined in the class **sc\_port**, shall be overridden by the implementation in the current class with a behavior that is implementation-defined.

NOTE 1—An implementation may use **end\_of\_elaboration** to implement the deferred calls for **initialize** and **sc\_trace**.

NOTE 2—If this member function is overridden in a class derived from the current class, function **end\_of\_elaboration** as overridden in the current class should be called explicitly from the overridden member function of the derived class in order to invoke the implementation-defined behavior.

### 6.10.7 Binding

Because interface **sc\_signal\_inout\_if** is derived from interface **sc\_signal\_in\_if**, a port of class **sc\_in** of a child module may be bound to a port of class **sc\_inout** of a parent module but a port of class **sc\_inout** of a child module cannot be bound to a port of class **sc\_in** of a parent module.

## 6.11 **sc\_inout<bool> and sc\_inout<sc\_dt::sc\_logic>**

### 6.11.1 Description

Class **sc\_inout<bool>** and **sc\_inout<sc\_dt::sc\_logic>** are specialized port classes that provide additional member functions for two-valued signals.

### 6.11.2 Class definition

```
namespace sc_core {

template <>
class sc_inout<bool>
: public sc_port<sc_signal_inout_if<bool>,1>
{
public:
    sc_inout();
    explicit sc_inout( const char* );
    virtual ~sc_inout();

    void initialize( const bool& );
    void initialize( const sc_signal_in_if<bool>& );

    virtual void end_of_elaboration();

    const bool& read() const;
    operator const bool& () const;

    void write( const bool& );
    sc_inout<bool>& operator=( const bool& );
    sc_inout<bool>& operator=( const sc_signal_in_if<bool>& );
    sc_inout<bool>& operator=( const sc_port< sc_signal_in_if<bool>, 1>& );
    sc_inout<bool>& operator=( const sc_port< sc_signal_inout_if<bool>, 1>& );
    sc_inout<bool>& operator=( const sc_inout<bool>& );

    const sc_event& default_event() const;
    const sc_event& value_changed_event() const;
    const sc_event& posedge_event() const;
    const sc_event& negedge_event() const;

    bool event() const;
    bool posedge() const;
    bool negedge() const;

    sc_event_finder& value_changed() const;
    sc_event_finder& pos() const;
    sc_event_finder& neg() const;

    virtual const char* kind() const;

private:
    // Disabled
    sc_inout( const sc_inout<bool>& );
};
```

```

template <>
inline void sc_trace<bool>( sc_trace_file*, const sc_inout<bool>&, const std::string& );

template <>
class sc_inout<sc_dt::sc_logic>
: public sc_port<sc_signal_inout_if<sc_dt::sc_logic>,1>
{
    public:
        sc_inout();
        explicit sc_inout( const char* );
        virtual ~sc_inout();

        void initialize( const sc_dt::sc_logic& );
        void initialize( const sc_signal_in_if<sc_dt::sc_logic>& );

        virtual void end_of_elaboration();

        const sc_dt::sc_logic& read() const;
        operator const sc_dt::sc_logic& () const;

        void write( const sc_dt::sc_logic& );
        sc_inout<sc_dt::sc_logic>& operator=( const sc_dt::sc_logic& );
        sc_inout<sc_dt::sc_logic>& operator=( const sc_signal_in_if<sc_dt::sc_logic>& );
        sc_inout<sc_dt::sc_logic>& operator=( const sc_port< sc_signal_in_if<sc_dt::sc_logic>, 1>& );
        sc_inout<sc_dt::sc_logic>& operator=( const sc_port< sc_signal_inout_if<sc_dt::sc_logic>, 1>& );
        sc_inout<sc_dt::sc_logic>& operator=( const sc_inout<sc_dt::sc_logic>& );

        const sc_event& default_event() const;
        const sc_event& value_changed_event() const;
        const sc_event& posedge_event() const;
        const sc_event& negedge_event() const;

        bool event() const;
        bool posedge() const;
        bool negedge() const;

        sc_event_finder& value_changed() const;
        sc_event_finder& pos() const;
        sc_event_finder& neg() const;

        virtual const char* kind() const;

    private:
        // Disabled
        sc_inout( const sc_inout<sc_dt::sc_logic>& );
};

template <>
inline void
sc_trace<sc_dt::sc_logic>( sc_trace_file*, const sc_inout<sc_dt::sc_logic>&, const std::string& );

} // namespace sc_core

```

### 6.11.3 Member functions

The following list is incomplete. For the remaining member functions and for the function **sc\_trace**, refer to the definitions of the member functions for class **sc\_inout**.

Member functions **posedge\_event**, **negedge\_event**, **posedge**, and **negedge** shall each call the corresponding member function of the object to which the port is bound using **operator->** of class **sc\_port**, for example:

(\*this)->negedge()

Member functions **pos** and **neg** shall return a reference to class **sc\_event\_finder**, where the event finder object itself shall be constructed using the member function **posedge\_event** or **negedge\_event**, respectively (see 5.7).

Member function **kind** shall return the string **"sc\_inout"**.



## 6.12 sc\_out

### 6.12.1 Description

Class **sc\_out** is derived from class **sc\_inout** and is identical to class **sc\_inout** except for differences inherent in it being a derived class, for example, constructors and assignment operators. The purpose of having both classes is to allow users to express their intent, that is, **sc\_out** for output pins and **sc\_inout** for bidirectional pins.

### 6.12.2 Class definition

```
namespace sc_core {

template <class T>
class sc_out
: public sc_inout<T>
{
    public:

        sc_out();
        explicit sc_out( const char* );
        virtual ~sc_out();

        sc_out<T>& operator= ( const T& );
        sc_out<T>& operator= ( const sc_signal_in_if<T>& );
        sc_out<T>& operator= ( const sc_port< sc_signal_in_if<T>, 1>& );
        sc_out<T>& operator= ( const sc_port< sc_signal_inout_if<T>, 1>& );
        sc_out<T>& operator= ( const sc_out<T>& );

        virtual const char* kind() const;

    private:
        // Disabled
        sc_out( const sc_out<T>& );
};

} // namespace sc_core
```

### 6.12.3 Member functions

The constructors shall pass their arguments to the corresponding constructors for the base class **sc\_inout<T>**.

The behavior of the assignment operators shall be identical to that of class **sc\_inout** but with the class name **sc\_out** substituted in place of the class name **sc\_inout** wherever appropriate.

Member function **kind** shall return the string "**sc\_out**".

## 6.13 sc\_signal\_resolved

### 6.13.1 Description

Class **sc\_signal\_resolved** is a predefined primitive channel derived from class **sc\_signal**. A resolved signal is an object of class **sc\_signal\_resolved** or class **sc\_signal\_rv**. Class **sc\_signal\_resolved** differs from class **sc\_signal** in that a resolved signal may be written by multiple processes, conflicting values being resolved within the channel.

### 6.13.2 Class definition

```
namespace sc_core {

class sc_signal_resolved
: public sc_signal<sc_dt::sc_logic>
{
    public:
        sc_signal_resolved();
        explicit sc_signal_resolved( const char* );
        virtual ~sc_signal_resolved();

        virtual void register_port( sc_port_base&, const char* );

        virtual void write( const sc_dt::sc_logic& );
        sc_signal_resolved& operator=( const sc_dt::sc_logic& );
        sc_signal_resolved& operator=( const sc_signal_resolved& );

        virtual const char* kind() const;

    protected:
        virtual void update();

    private:
        // Disabled
        sc_signal_resolved( const sc_signal_resolved& );
};

} // namespace sc_core
```

### 6.13.3 Constructors

**sc\_signal\_resolved**();

This constructor shall call the base class constructor from its initializer list as follows:

```
sc_signal<sc_dt::sc_logic>( sc_gen_unique_name( "signal_resolved" ) )
```

explicit **sc\_signal\_resolved**( const char\* name\_ );

This constructor shall call the base class constructor from its initializer list as follows:

```
sc_signal<sc_dt::sc_logic>( name_ )
```

#### 6.13.4 Resolution semantics

A resolved signal is written by calling member function **write** or **operator=** of the given signal object. Like class **sc\_signal**, **operator=** shall call member function **write**.

Each resolved signal shall maintain a *list of written values* containing one value for each distinct process instance that writes to the resolved signal object. This list shall store the value most recently written to the resolved signal object by each such process instance.

If and only if the written value is different from the previous written value or this is the first occasion on which the particular process instance has written to the particular signal object, the member function **write** shall then call the member function **request\_update**.

During the update phase, member function **update** shall first use the list of written values to calculate a single *resolved value* for the resolved signal, and then perform update semantics similar to class **sc\_signal** but using the resolved value just calculated.

A value shall be added to the list of written values on the first occasion that each particular process instance writes to the resolved signal object. Values shall not be removed from the list of written values. Before the first occasion on which a given process instance writes to a given resolved signal, that process instance shall not contribute to the calculation of the resolved value for that signal.

The resolved value shall be calculated from the list of written values using the following algorithm:

- 1) Take a copy of the list.
- 2) Take any two values from the copy of the list and replace them with one value according to the truth table shown in Table 1.
- 3) Repeat 2 until only a single value remains. This is the resolved value.

**Table 1—Resolution table for `sc_signal_resolved`**

|     | '0' | '1' | 'Z' | 'X' |
|-----|-----|-----|-----|-----|
| '0' | '0' | 'X' | '0' | 'X' |
| '1' | 'X' | '1' | '1' | 'X' |
| 'Z' | '0' | '1' | 'Z' | 'X' |
| 'X' | 'X' | 'X' | 'X' | 'X' |

Before the first occasion on which a given process instance writes to a given resolved signal, the value written by that process instance is effectively '**Z**' in terms of its effect on the resolution calculation. On the other hand, the default initial value for a resolved signal (as would be returned by member function **read** before the first **write**) is '**X**'. Thus it is strongly recommended that each process instance that writes to a given resolved signal perform a write to that signal at time zero.

NOTE 1—The order in which values are passed to the function defined by the truth table in Table 1 does not affect the result of the calculation.

NOTE 2—The calculation of the resolved value is performed using the value most recently written by each and every process that writes to that particular signal object, regardless of whether the most recent write occurred in the current delta cycle, in a previous delta cycle, or at an earlier time.

NOTE 3—These same resolution semantics apply, whether the resolved signal is accessed directly by a process or is accessed indirectly through a port bound to the resolved signal.

### 6.13.5 Member functions

Member function **register\_port** of class **sc\_signal** shall be overridden in class **sc\_signal\_resolved**, such that the error check for multiple output ports performed by **sc\_signal::register\_port** is disabled for channel objects of class **sc\_signal\_resolved**.

Member function **write**, **operator=**, and member function **update** shall have the same behavior as the corresponding members of class **sc\_signal**, except where the behavior differs for multiple writers as defined in 6.13.4.

Member function **kind** shall return the string "**sc\_signal\_resolved**".

*Example:*

```
SC_MODULE(M)
{
    sc_signal_resolved sig;

    SC_CTOR(M)
    {
        SC_THREAD(T1);
        SC_THREAD(T2);
        SC_THREAD(T3);
    }
    void T1()
    {
        // Time=0 ns, no written values    sig=X
        wait(10, SC_NS);
        sig = sc_dt::SC_LOGIC_0; // Time=10 ns, written values=0    sig=0
        wait(20, SC_NS);
        sig = sc_dt::SC_LOGIC_Z; // Time=30 ns, written values=Z,Z    sig=Z
    }
    void T2()
    {
        wait(20, SC_NS);
        sig = sc_dt::SC_LOGIC_Z; // Time=20 ns, written values=0,Z    sig=0
        wait(30, SC_NS);
        sig = sc_dt::SC_LOGIC_0; // Time=50 ns, written values=Z,0,1    sig=X
    }
    void T3()
    {
        wait(40, SC_NS);
        sig = sc_dt::SC_LOGIC_1; // Time=40 ns, written values=Z,Z,1    sig=1
    }
};
```

## 6.14 **sc\_in\_resolved**

### 6.14.1 Description

Class **sc\_in\_resolved** is a specialized port class for use with resolved signals. It is similar in behavior to port class **sc\_in<sc\_dt::sc\_logic>** from which it is derived. The only difference is that a port of class **sc\_in\_resolved** shall be bound to a channel of class **sc\_signal\_resolved**, whereas a port of class **sc\_in<sc\_dt::sc\_logic>** may be bound to a channel of class **sc\_signal<sc\_dt::sc\_logic>** or class **sc\_signal\_resolved**.

### 6.14.2 Class definition

```
namespace sc_core {  
  
class sc_in_resolved  
: public sc_in<sc_dt::sc_logic>  
{  
    public:  
        sc_in_resolved();  
        explicit sc_in_resolved( const char* );  
        virtual ~sc_in_resolved();  
  
        virtual void end_of_elaboration();  
  
        virtual const char* kind() const;  
  
    private:  
        // Disabled  
        sc_in_resolved( const sc_in_resolved& );  
        sc_in_resolved& operator=( const sc_in_resolved& );  
};  
  
} // namespace sc_core
```

### 6.14.3 Member functions

The constructors shall pass their arguments to the corresponding constructors for the base class **sc\_in<sc\_dt::sc\_logic>**.

Member function **end\_of\_elaboration** shall perform an error check. It is an error if the port is not bound to a channel of class **sc\_signal\_resolved**.

Member function **kind** shall return the string "**sc\_in\_resolved**".

NOTE—As always, the port may be bound indirectly through a port of a parent module.

## 6.15 sc\_inout\_resolved

### 6.15.1 Description

Class **sc\_inout\_resolved** is a specialized port class for use with resolved signals. It is similar in behavior to port class **sc\_inout<sc\_dt::sc\_logic>** from which it is derived. The only difference is that a port of class **sc\_inout\_resolved** shall be bound to a channel of class **sc\_signal\_resolved**, whereas a port of class **sc\_inout<sc\_dt::sc\_logic>** may be bound to a channel of class **sc\_signal<sc\_dt::sc\_logic>** or class **sc\_signal\_resolved**.

### 6.15.2 Class definition

```
namespace sc_core {

class sc_inout_resolved
: public sc_inout<sc_dt::sc_logic>
{
public:
    sc_inout_resolved();
    explicit sc_inout_resolved( const char* );
    virtual ~sc_inout_resolved();

    virtual void end_of_elaboration();

    sc_inout_resolved& operator= ( const sc_dt::sc_logic& );
    sc_inout_resolved& operator= ( const sc_signal_in_if<sc_dt::sc_logic>& );
    sc_inout_resolved& operator= ( const sc_port<sc_signal_in_if<sc_dt::sc_logic>, 1>& );
    sc_inout_resolved& operator= ( const sc_port<sc_signal_inout_if<sc_dt::sc_logic>, 1>& );
    sc_inout_resolved& operator= ( const sc_inout_resolved& );

    virtual const char* kind() const;

private:
    // Disabled
    sc_inout_resolved( const sc_inout_resolved& );
};

} // namespace sc_core
```

### 6.15.3 Member functions

The constructors shall pass their arguments to the corresponding constructors for the base class **sc\_inout<sc\_dt::sc\_logic>**.

Member function **end\_of\_elaboration** shall perform an error check. It is an error if the port is not bound to a channel of class **sc\_signal\_resolved**.

The behavior of the assignment operators shall be identical to that of class **sc\_inout<sc\_dt::sc\_logic>** but with the class name **sc\_inout\_resolved** substituted in place of the class name **sc\_inout<sc\_dt::sc\_logic>** wherever appropriate.

Member function **kind** shall return the string **"sc\_inout\_resolved"**.

NOTE—As always, the port may be bound indirectly through a port of a parent module.

## 6.16 sc\_out\_resolved

### 6.16.1 Description

Class **sc\_out\_resolved** is derived from class **sc\_inout\_resolved**, and is identical to class **sc\_inout\_resolved** except for differences inherent in it being a derived class, for example, constructors and assignment operators. The purpose of having both classes is to allow users to express their intent, that is, **sc\_out\_resolved** for output pins connected to resolved signals and **sc\_inout\_resolved** for bidirectional pins connected to resolved signals.

### 6.16.2 Class definition

```
namespace sc_core {  
  
class sc_out_resolved  
: public sc_inout_resolved  
{  
    public:  
        sc_out_resolved();  
        explicit sc_out_resolved( const char* );  
        virtual ~sc_out_resolved();  
  
        sc_out_resolved& operator= ( const sc_dt::sc_logic& );  
        sc_out_resolved& operator= ( const sc_signal_in_if<sc_dt::sc_logic>& );  
        sc_out_resolved& operator= ( const sc_port<sc_signal_in_if<sc_dt::sc_logic>, 1>& );  
        sc_out_resolved& operator= ( const sc_port<sc_signal_inout_if<sc_dt::sc_logic>, 1>& );  
        sc_out_resolved& operator= ( const sc_out_resolved& );  
  
        virtual const char* kind() const;  
  
    private:  
        // Disabled  
        sc_out_resolved( const sc_out_resolved& );  
};  
  
} // namespace sc_core
```

### 6.16.3 Member functions

The constructors shall pass their arguments to the corresponding constructors for the base class **sc\_inout\_resolved**.

The behavior of the assignment operators shall be identical to that of class **sc\_inout\_resolved** but with the class name **sc\_out\_resolved** substituted in place of the class name **sc\_inout\_resolved** wherever appropriate.

Member function **kind** shall return the string "**sc\_out\_resolved**".

## 6.17 sc\_signal\_rv

### 6.17.1 Description

Class **sc\_signal\_rv** is a predefined primitive channel derived from class **sc\_signal**. Class **sc\_signal\_rv** is similar to class **sc\_signal\_resolved**. The difference is that the argument to the base class template **sc\_signal** is type **sc\_dt::sc\_lv<W>** instead of type **sc\_dt::sc\_logic**.

### 6.17.2 Class definition

```
namespace sc_core {

template <int W>
class sc_signal_rv
: public sc_signal<sc_dt::sc_lv<W>>
{
public:
    sc_signal_rv();
    explicit sc_signal_rv( const char* );
    virtual ~sc_signal_rv();

    virtual void register_port( sc_port_base&, const char* );

    virtual void write( const sc_dt::sc_lv<W>& );
    sc_signal_rv<W>& operator=( const sc_dt::sc_lv<W>& );
    sc_signal_rv<W>& operator=( const sc_signal_rv<W>& );

    virtual const char* kind() const;

protected:
    virtual void update();

private:
    // Disabled
    sc_signal_rv( const sc_signal_rv<W>& );
};

// namespace sc_core
```

### 6.17.3 Semantics and member functions

The semantics of class **sc\_signal\_rv** shall be identical to the semantics of class **sc\_signal\_resolved** except for differences due to the fact that the value to be resolved is of type **sc\_dt::sc\_lv** (see 6.13.4).

The value shall be propagated through the resolved signal as an atomic value; that is, an event shall be notified, and the entire value of the vector shall be resolved and updated whenever any bit of the vector written by any process changes.

The *list of written values* shall contain values of type **sc\_dt::sc\_lv**, and each value of type **sc\_dt::sc\_lv** shall be treated atomically for the purpose of building and updating the list.

If and only if the written value differs from the previous written value (in one or more bit positions) or this is the first occasion on which the particular process has written to the particular signal object, the member function **write** shall then call the member function **request\_update**.



The resolved value shall be calculated for the entire vector by applying the rule described in 6.13.4 to each bit position within the vector in turn.

The default constructor shall call the base class constructor from its initializer list as follows:

```
sc_signal<sc_dt::sc_lv<W>> ( sc_gen_unique_name( "signal_rv" ) )
```

Member function **kind** shall return the string "**sc\_signal\_rv**".

## 6.18 sc\_in\_rv

### 6.18.1 Description

Class **sc\_in\_rv** is a specialized port class for use with resolved signals. It is similar in behavior to port class **sc\_in<sc\_dt::sc\_lv<W>>** from which it is derived. The only difference is that a port of class **sc\_in\_rv** shall be bound to a channel of class **sc\_signal\_rv**, whereas a port of class **sc\_in<sc\_dt::sc\_lv<W>>** may be bound to a channel of class **sc\_signal<sc\_dt::sc\_lv<W>>** or class **sc\_signal\_rv**.

### 6.18.2 Class definition

```
namespace sc_core {

template <int W>
class sc_in_rv
: public sc_in<sc_dt::sc_lv<W>>
{
public:
    sc_in_rv();
    explicit sc_in_rv( const char* );
    virtual ~sc_in_rv();

    virtual void end_of_elaboration();

    virtual const char* kind() const;

private:
    // Disabled
    sc_in_rv( const sc_in_rv<W>& );
    sc_in_rv<W>& operator=( const sc_in_rv<W>& );
};

} // namespace sc_core
```

### 6.18.3 Member functions

The constructors shall pass their arguments to the corresponding constructors for the base class **sc\_in<sc\_dt::sc\_lv<W>>**.

Member function **end\_of\_elaboration** shall perform an error check. It is an error if the port is not bound to a channel of class **sc\_signal\_rv**.

Member function **kind** shall return the string "**sc\_in\_rv**".

NOTE—As always, the port may be bound indirectly through a port of a parent module.

## 6.19 sc\_inout\_rv

### 6.19.1 Description

Class **sc\_inout\_rv** is a specialized port class for use with resolved signals. It is similar in behavior to port class **sc\_inout<sc\_dt::sc\_lv<W> >** from which it is derived. The only difference is that a port of class **sc\_inout\_rv** shall be bound to a channel of class **sc\_signal\_rv**, whereas a port of class **sc\_inout<sc\_dt::sc\_lv<W> >** may be bound to a channel of class **sc\_signal<sc\_dt::sc\_lv<W> >** or class **sc\_signal\_rv**.

### 6.19.2 Class definition

```
namespace sc_core {

template <int W>
class sc_inout_rv
: public sc_inout<sc_dt::sc_lv<W> >
{
public:
    sc_inout_rv();
    explicit sc_inout_rv( const char* );
    virtual ~sc_inout_rv();

    sc_inout_rv<W>& operator= ( const sc_dt::sc_lv<W>& );
    sc_inout_rv<W>& operator= ( const sc_signal_in_if<sc_dt::sc_lv<W> >& );
    sc_inout_rv<W>& operator= ( const sc_port<sc_signal_in_if<sc_dt::sc_lv<W> >, 1>& );
    sc_inout_rv<W>& operator= ( const sc_port<sc_signal_inout_if<sc_dt::sc_lv<W> >, 1>& );
    sc_inout_rv<W>& operator= ( const sc_inout_rv<W>& );

    virtual void end_of_elaboration();

    virtual const char* kind() const;

private:
    // Disabled
    sc_inout_rv( const sc_inout_rv<W>& );
};

// namespace sc_core
```

### 6.19.3 Member functions

The constructors shall pass their arguments to the corresponding constructors for the base class **sc\_inout<sc\_dt::sc\_lv<W> >**.

Member function **end\_of\_elaboration** shall perform an error check. It is an error if the port is not bound to a channel of class **sc\_signal\_rv**.

The behavior of the assignment operators shall be identical to that of class **sc\_inout<sc\_dt::sc\_lv<W> >**, but with the class name **sc\_inout\_rv** substituted in place of the class name **sc\_inout<sc\_dt::sc\_lv<W> >** wherever appropriate.

Member function **kind** shall return the string "**sc\_inout\_rv**".

NOTE—The port may be bound indirectly through a port of a parent module.

## 6.20 sc\_out\_rv

### 6.20.1 Description

Class **sc\_out\_rv** is derived from class **sc\_inout\_rv**, and is identical to class **sc\_inout\_rv** except for differences inherent in it being a derived class, for example, constructors and assignment operators. The purpose of having both classes is to allow users to express their intent, that is, **sc\_out\_rv** for output pins connected to resolved vectors and **sc\_inout\_rv** for bidirectional pins connected to resolved vectors.

### 6.20.2 Class definition

```
namespace sc_core {  
  
template <int W>  
class sc_out_rv  
: public sc_inout_rv<W>  
{  
    public:  
        sc_out_rv();  
        explicit sc_out_rv( const char* );  
        virtual ~sc_out_rv();  
  
        sc_out_rv<W>& operator= ( const sc_dt::sc_lv<W>& );  
        sc_out_rv<W>& operator= ( const sc_signal_in_if<sc_dt::sc_lv<W>>& );  
        sc_out_rv<W>& operator= ( const sc_port<sc_signal_in_if<sc_dt::sc_lv<W>>, 1>& );  
        sc_out_rv<W>& operator= ( const sc_port<sc_signal_inout_if<sc_dt::sc_lv<W>>, 1>& );  
        sc_out_rv<W>& operator= ( const sc_out_rv<W>& );  
  
        virtual const char* kind() const;  
  
    private:  
        // Disabled  
        sc_out_rv( const sc_out_rv<W>& );  
};  
  
} // namespace sc_core
```

### 6.20.3 Member functions

The constructors shall pass their arguments to the corresponding constructors for the base class **sc\_inout\_rv<W>**.

The behavior of the assignment operators shall be identical to that of class **sc\_inout\_rv<W>**, but with the class name **sc\_out\_rv<W>** substituted in place of the class name **sc\_inout\_rv<W>** wherever appropriate.

Member function **kind** shall return the string "**sc\_out\_rv**".

## 6.21 sc\_fifo\_in\_if

### 6.21.1 Description

Class **sc\_fifo\_in\_if** is an interface proper and is implemented by the predefined channel **sc\_fifo**. Interface **sc\_fifo\_in\_if** gives read access to a fifo channel, and is derived from two further interfaces proper, **sc\_fifo\_nonblocking\_in\_if** and **sc\_fifo\_blocking\_in\_if**.

### 6.21.2 Class definition

```
namespace sc_core {

template <class T>
class sc_fifo_nonblocking_in_if
: virtual public sc_interface
{
    public:
        virtual bool nb_read( T& ) = 0;
        virtual const sc_event& data_written_event() const = 0;
};

template <class T>
class sc_fifo_blocking_in_if
: virtual public sc_interface
{
    public:
        virtual void read( T& ) = 0;
        virtual T read() = 0;
};

template <class T>
class sc_fifo_in_if : public sc_fifo_nonblocking_in_if<T>, public sc_fifo_blocking_in_if<T>
{
    public:
        virtual int num_available() const = 0;

    protected:
        sc_fifo_in_if();

    private:
        // Disabled
        sc_fifo_in_if( const sc_fifo_in_if<T>& );
        sc_fifo_in_if<T>& operator=( const sc_fifo_in_if<T>& );
};

// namespace sc_core
```

### 6.21.3 Member functions

The following member functions are all pure virtual functions. The descriptions refer to the expected definitions of the functions when overridden in a channel that implements this interface. The precise semantics will be channel-specific.

Member functions **read** and **nb\_read** shall return the value least recently written into the fifo, and shall remove that value from the fifo such that it cannot be read again. If the fifo is empty, member function **read** shall suspend until a value has been written to the fifo, whereas member function **nb\_read** shall return immediately. The return value of the function **nb\_read** shall indicate whether a value was read.

When calling member function **void read(T&)** of class **sc\_fifo\_blocking\_in\_if**, the application shall be obliged to ensure that the lifetime of the actual argument extends from the time the function is called to the time the function call reaches completion. Moreover, the application shall not modify the value of the actual argument during that period.

Member function **data\_written\_event** shall return a reference to an event that is notified whenever a value is written into the fifo.

Member function **num\_available** shall return the number of values currently available in the fifo to be read.

## 6.22 sc\_fifo\_out\_if

### 6.22.1 Description

Class **sc\_fifo\_out\_if** is an interface proper and is implemented by the predefined channel **sc\_fifo**. Interface **sc\_fifo\_out\_if** gives write access to a fifo channel and is derived from two further interfaces proper, **sc\_fifo\_nonblocking\_out\_if** and **sc\_fifo\_blocking\_out\_if**.

### 6.22.2 Class definition

```
namespace sc_core {

template <class T>
class sc_fifo_nonblocking_out_if
: virtual public sc_interface
{
    public:
        virtual bool nb_write( const T& ) = 0;
        virtual const sc_event& data_read_event() const = 0;
};

template <class T>
class sc_fifo_blocking_out_if
: virtual public sc_interface
{
    public:
        virtual void write( const T& ) = 0;
};

template <class T>
class sc_fifo_out_if : public sc_fifo_nonblocking_out_if<T>, public sc_fifo_blocking_out_if<T>
{
    public:
        virtual int num_free() const = 0;

    protected:
        sc_fifo_out_if();

    private:
        // Disabled
        sc_fifo_out_if( const sc_fifo_out_if<T>& );
        sc_fifo_out_if<T>& operator=( const sc_fifo_out_if<T>& );
};

} // namespace sc_core
```

### 6.22.3 Member functions

The following member functions are all pure virtual functions. The descriptions refer to the expected definitions of the functions when overridden in a channel that implements this interface. The precise semantics will be channel-specific.



Member functions **write** and **nb\_write** shall write the value passed as an argument into the fifo. If the fifo is full, member function **write** shall suspend until a value has been read from the fifo, whereas member function **nb\_write** shall return immediately. The return value of the function **nb\_write** shall indicate whether a value was written into an empty slot.

When calling member function **void write(const T&)** of class **sc\_fifo\_blocking\_out\_if**, the application shall be obliged to ensure that the lifetime of the actual argument extends from the time the function is called to the time the function call reaches completion, and moreover the application shall not modify the value of the actual argument during that period.

Member function **data\_read\_event** shall return a reference to an event that is notified whenever a value is read from the fifo.

Member function **num\_free** shall return the number of unoccupied slots in the fifo available to accept written values.

## 6.23 sc\_fifo

### 6.23.1 Description

Class **sc\_fifo** is a predefined primitive channel intended to model the behavior of a fifo, that is, a first-in-first-out buffer. A *fifo* is an object of class **sc\_fifo**. Each fifo has a number of *slots* for storing values. The number of slots is fixed when the object is constructed.

### 6.23.2 Class definition

```
namespace sc_core {

template <class T>
class sc_fifo
: public sc_fifo_in_if<T>, public sc_fifo_out_if<T>, public sc_prim_channel
{
public:
    explicit sc_fifo( int size_ = 16 );
    explicit sc_fifo( const char* name_, int size_ = 16);
    virtual ~sc_fifo();

    virtual void register_port( sc_port_base&, const char* );

    virtual void read( T& );
    virtual T read();
    virtual bool nb_read( T& );
    operator T ();

    virtual void write( const T& );
    virtual bool nb_write( const T& );
    sc_fifo<T>& operator= ( const T& );

    virtual const sc_event& data_written_event() const;
    virtual const sc_event& data_read_event() const;

    virtual int num_available() const;
    virtual int num_free() const;

    virtual void print( std::ostream& = std::cout ) const;
    virtual void dump( std::ostream& = std::cout ) const;
    virtual const char* kind() const;

protected:
    virtual void update();

private:
    // Disabled
    sc_fifo( const sc_fifo<T>& );
    sc_fifo& operator= ( const sc_fifo<T>& );
};
```

```
template <class T>
inline std::ostream& operator<< ( std::ostream&, const sc_fifo<T>& );

}      // namespace sc_core
```

### 6.23.3 Template parameter T

The argument passed to template **sc\_fifo** shall be either a C++ type for which the predefined semantics for assignment are adequate (for example, a fundamental type or a pointer), or a type **T** that obeys each of the following rules:

- a) The following stream operator shall be defined and should copy the state of the object given as the second argument to the stream given as the first argument. The way in which the state information is formatted is undefined by this standard. The implementation shall use this operator in implementing the behavior of the member functions **print** and **dump**.

```
std::ostream& operator<< ( std::ostream&, const T& );
```

- b) If the default assignment semantics are inadequate to assign the state of the object, the following assignment operator should be defined for the type **T**. The implementation shall use this operator to copy the value being written into a fifo slot or the value being read out of a fifo slot.

```
const T& operator= ( const T& );
```

- c) If any constructor for type **T** exists, a default constructor for type **T** shall be defined.

NOTE 1—The assignment operator is not obliged to assign the complete state of the object, although it should typically do so. For example, diagnostic information may be associated with an object that is not to be propagated through the fifo.

NOTE 2—The SystemC data types proper (**sc\_dt::sc\_int**, **sc\_dt::sc\_logic**, and so forth) all conform to the above rule set.

NOTE 3—It is legal to pass type **sc\_module\*** through a fifo, although this would be regarded as an abuse of the module hierarchy and thus bad practice.

### 6.23.4 Constructors

```
explicit sc_fifo( int size_ = 16 );
```

This constructor shall call the base class constructor from its initializer list as follows:

```
sc_prim_channel( sc_gen_unique_name( "fifo" ) )
```

```
explicit sc_fifo( const char* name_, int size_ = 16 );
```

This constructor shall call the base class constructor from its initializer list as follows:

```
sc_prim_channel( name_ )
```

Both constructors shall initialize the number of slots in the fifo to the value given by the parameter **size\_**. The number of slots shall be greater than zero.

### 6.23.5 register\_port

virtual void **register\_port**( sc\_port\_base&, const char\* );

Member function **register\_port** of class **sc\_interface** shall be overridden in class **sc\_fifo** and shall perform an error check. It is an error if more than one port of type **sc\_fifo\_in\_if** is bound to a given fifo, and an error if more than one port of type **sc\_fifo\_out\_if** is bound to a given fifo.

### 6.23.6 Member functions for reading

virtual void **read**( T& );

virtual T **read**();

virtual bool **nb\_read**( T& );

Member functions **read** and **nb\_read** shall return the value least recently written into the fifo and shall remove that value from the fifo such that it cannot be read again. Multiple values may be read within a single delta cycle. The order in which values are read from the fifo shall precisely match the order in which values were written into the fifo. Values written into the fifo during the current delta cycle are not available for reading in that delta cycle but become available for reading in the immediately following delta cycle.

The value read from the fifo shall be returned as the value of the member function or as an argument passed by reference, as appropriate.

If the fifo is empty (that is, no values are available for reading), member function **read** shall suspend until the *data-written event* is notified. At that point, it shall resume (in the immediately following evaluation phase) and complete the reading of the value least recently written into the fifo before returning.

If the fifo is empty, member function **nb\_read** shall return immediately without modifying the state of the fifo, without calling **request\_update**, and with a return value of **false**. Otherwise, if a value is available for reading, the return value of member function **nb\_read** shall be **true**.

**operator T ();**

The behavior of **operator T()** shall be equivalent to the following definition:

```
operator T () { return read(); }
```

### 6.23.7 Member functions for writing

virtual void **write**( const T& );

virtual bool **nb\_write**( const T& );

Member functions **write** and **nb\_write** shall write the value passed as an argument into the fifo. Multiple values may be written within a single delta cycle. If values are read from the fifo during the current delta cycle, the empty slots in the fifo so created do not become free for the purposes of writing until the immediately following delta cycle.

If the fifo is full (that is, no free slots exist for the purposes of writing), member function **write** shall suspend until the *data-read event* is notified. At which point, it shall resume (in the immediately following evaluation phase) and complete the writing of the argument value into the fifo before returning.

If the fifo is full, member function **nb\_write** shall return immediately without modifying the state of the fifo, without calling **request\_update**, and with a return value of **false**. Otherwise, if a slot is free, the return value of member function **nb\_write** shall be **true**.

#### **operator=**

The behavior of **operator=** shall be equivalent to the following definition:

```
sc_fifo<T>& operator= ( const T& a ) { write( a ); return *this; }
```

### **6.23.8 The update phase**

Member functions **read**, **nb\_read**, **write**, and **nb\_write** shall complete the act of reading or writing the fifo by calling member function **request\_update** of class **sc\_prim\_channel**.

virtual void **update**();

Member function **update** of class **sc\_prim\_channel** shall be overridden in class **sc\_fifo** to update the number of values available for reading and the number of free slots for writing and shall cause the *data-written event* or the *data-read event* to be notified in the immediately following delta notification phase as necessary.

NOTE—If a fifo is empty and member functions **write** and **read** are both called (from the same process or from two different processes) during the evaluation phase of the same delta cycle, the write will complete in that delta cycle, but the read will suspend because the fifo is empty. The number of values available for reading will be incremented to one during the update phase, and the read will complete in the following delta cycle, returning the value just written.

### **6.23.9 Member functions for events**

virtual const sc\_event& **data\_written\_event**() const;

Member function **data\_written\_event** shall return a reference to an event, the *data-written event*, that is notified in the delta notification phase that occurs at the end of the delta cycle in which a value is written into the fifo.

virtual const sc\_event& **data\_read\_event**() const;

Member function **data\_read\_event** shall return a reference to an event, the *data-read event*, that is notified in the delta notification phase that occurs at the end of the delta cycle in which a value is read from the fifo.

### **6.23.10 Member functions for available values and free slots**

virtual int **num\_available**() const;

Member function **num\_available** shall return the number of values that are available for reading in the current delta cycle. The calculation shall deduct any values read during the current delta cycle but shall not add any values written during the current delta cycle.

virtual int **num\_free**() const;

Member function **num\_free** shall return the number of empty slots that are free for writing in the current delta cycle. The calculation shall deduct any slots written during the current delta cycle but shall not add any slots made free by reading in the current delta cycle.

### 6.23.11 Diagnostic member functions

virtual void **print**( std::ostream& = std::cout ) const;

Member function **print** shall print a list of the values stored in the fifo and that are available for reading. They will be printed in the order they were written to the fifo and are printed to the stream passed as an argument by calling **operator<<** (**std::ostream&**, **T&**). The formatting shall be implementation-defined.

virtual void **dump**( std::ostream& = std::cout ) const;

Member function **dump** shall print at least the hierarchical name of the fifo and a list of the values stored in the fifo that are available for reading. They are printed to the stream passed as an argument. The formatting shall be implementation-defined.

virtual const char\* **kind**() const;

Member function **kind** shall return the string "**sc\_fifo**".

### 6.23.12 operator<<

template <class T>

inline std::ostream& **operator<<** ( std::ostream&, const sc\_fifo<T>& );

**operator<<** shall call member function **print** to print the contents of the fifo passed as the second argument to the stream passed as the first argument by calling **operator<<** (**std::ostream&**, **T&**)

*Example:*

```
SC_MODULE(M)
{
    sc_fifo<int> fifo;
    SC_CTOR(M) : fifo(4)
    {
        SC_THREAD(T);
    }
    void T()
    {
        int d;
        fifo.write(1);
        fifo.print(std::cout);           // 1
        fifo.write(2);
        fifo.print(std::cout);           // 1 2
        fifo.write(3);
        fifo.print(std::cout);           // 1 2 3
        std::cout << fifo.num_available(); // 0 values available to read
        std::cout << fifo.num_free();      // 1 free slot
        fifo.read(d);                    // read suspends and returns in the next delta cycle
        fifo.print(std::cout);           // 2 3
        std::cout << fifo.num_available(); // 2 values available to read
        std::cout << fifo.num_free();      // 1 free slot
        fifo.read(d);
        fifo.print(std::cout);           // 3
    }
}
```

```
        fifo.read(d);  
        fifo.print(std::cout);           // Empty  
        std::cout << fifo.num_available(); // 0 values available to read  
        std::cout << fifo.num_free();     // 1 free slot  
        wait(SC_ZERO_TIME);  
        std::cout << fifo.num_free();     // 4 free slots  
    }  
};
```

## 6.24 sc\_fifo\_in

### 6.24.1 Description

Class **sc\_fifo\_in** is a specialized port class for use when reading from a fifo. It provides functions to conveniently access certain member functions of the fifo to which the port is bound.

### 6.24.2 Class definition

```
namespace sc_core {

template <class T>
class sc_fifo_in
: public sc_port<sc_fifo_in_if<T>,0>
{
public:
    sc_fifo_in();
    explicit sc_fifo_in( const char* );
    virtual ~sc_fifo_in();

    void read( T& );
    T read();
    bool nb_read( T& );
    const sc_event& data_written_event() const;
    sc_event_finder& data_written() const;
    int num_available() const;
    virtual const char* kind() const;

private:
    // Disabled
    sc_fifo_in( const sc_fifo_in<T>& );
    sc_fifo_in<T>& operator=( const sc_fifo_in<T>& );
};

} // namespace sc_core
```

### 6.24.3 Member functions

The constructors shall pass their arguments to the corresponding constructor for the base class **sc\_port**.

Member functions **read**, **nb\_read**, **data\_written\_event**, and **num\_available** shall each call the corresponding member function of the object to which the port is bound using **operator->** of class **sc\_port**, for example:

```
T read() { return (*this)->read(); }
```

Member function **data\_written** shall return a reference to class **sc\_event\_finder**, where the event finder object itself shall be constructed using the member function **data\_written\_event** (see 5.7).

Member function **kind** shall return the string "**sc\_fifo\_in**".



## 6.25 sc\_fifo\_out

### 6.25.1 Description

Class **sc\_fifo\_out** is a specialized port class for use when writing to a fifo. It provides functions to conveniently access certain member functions of the fifo to which the port is bound.

### 6.25.2 Class definition

```
namespace sc_core {

template <class T>
class sc_fifo_out
: public sc_port<sc_fifo_out_if<T>,0>
{
    public:
        sc_fifo_out();
        explicit sc_fifo_out( const char* );
        virtual ~sc_fifo_out();

        void write( const T& );
        bool nb_write( const T& );
        const sc_event& data_read_event() const;
        sc_event_finder& data_read() const;
        int num_free() const;
        virtual const char* kind() const;

    private:
        // Disabled
        sc_fifo_out( const sc_fifo_out<T>& );
        sc_fifo_out<T>& operator=( const sc_fifo_out<T>& );
};

} // namespace sc_core
```

### 6.25.3 Member functions

The constructors shall pass their arguments to the corresponding constructor for the base class **sc\_port**.

Member functions **write**, **nb\_write**, **data\_read\_event**, and **num\_free** shall each call the corresponding member function of the object to which the port is bound using **operator->** of class **sc\_port**, for example:

```
void write( const T& a ) { (*this)->write( a ); }
```

Member function **data\_read** shall return a reference to class **sc\_event\_finder**, where the event finder object itself shall be constructed using the member function **data\_read\_event** (see 5.7).

Member function **kind** shall return the string "**sc\_fifo\_out**".

*Example:*

```
// Type passed as template argument to sc_fifo<>
class U
{
public:
    U(int val = 0)                // If any constructor exists, a default constructor is required.
    {
        ptr = new int;
        *ptr = val;
    }
    int get() const { return *ptr; }
    void set(int i) { *ptr = i; }
    // Default assignment semantics are inadequate
    const U& operator= (const U& arg) { *(this->ptr) = *(arg.ptr); return *this; }
private:
    int *ptr;
};

// operator<< required
std::ostream& operator<< (std::ostream& os, const U& arg) { return (os << arg.get()); }

SC_MODULE(M1)
{
    sc_fifo_out<U> fifo_out;

    SC_CTOR(M1)
    {
        SC_THREAD(producer);
    }

    void producer()
    {
        U u;
        for (int i = 0; i < 4; i++)
        {
            u.set(i);
            bool status;
            do {
                wait(1, SC_NS);
                status = fifo_out.nb_write(u);        // Non-blocking write
            } while (!status);
        }
    }
};

SC_MODULE(M2)
{
    sc_fifo_in<U> fifo_in;

    SC_CTOR(M2)
    {
        SC_THREAD(consumer);
        sensitive << fifo_in.data_written();
    }
};
```

```
    }

    void consumer()
    {
        for (;;)
        {
            wait(fifo_in.data_written_event());
            U u;
            bool status = fifo_in.nb_read(u);
            std::cout << u << " ";           // 0 1 2 3
        }
    }
};

SC_MODULE(Top)
{
    sc_fifo<U> fifo;
    M1 m1;
    M2 m2;

    SC_CTOR(Top)
    : m1("m1"), m2("m2")
    {
        m1.fifo_out(fifo);
        m2.fifo_in (fifo);
    }
};
```

## 6.26 **sc\_mutex\_if**

### 6.26.1 Description

Class **sc\_mutex\_if** is an interface proper, and is implemented by the predefined channel **sc\_mutex**.

### 6.26.2 Class definition

```
namespace sc_core {  
  
class sc_mutex_if  
: virtual public sc_interface  
{  
    public:  
        virtual int lock() = 0;  
        virtual int trylock() = 0;  
        virtual int unlock() = 0;  
  
    protected:  
        sc_mutex_if();  
  
    private:  
        // Disabled  
        sc_mutex_if( const sc_mutex_if& );  
        sc_mutex_if& operator=( const sc_mutex_if& );  
};  
  
} // namespace sc_core
```

### 6.26.3 Member functions

The behavior of the member functions of class **sc\_mutex\_if** is defined in class **sc\_mutex**.

## 6.27 **sc\_mutex**

### 6.27.1 Description

Class **sc\_mutex** is a predefined primitive channel intended to model the behavior of a mutual exclusion lock used to control access to a resource shared by concurrent processes. A *mutex* is an object of class **sc\_mutex**. A mutex shall be in one of two exclusive states: *unlocked* or *locked*. Only one process can lock a given mutex at one time. A mutex can only be unlocked by the particular process instance that locked the mutex but may be locked subsequently by a different process.

NOTE—Although **sc\_mutex** is derived from **sc\_prim\_channel**, **sc\_mutex** does not use the request update mechanism.

### 6.27.2 Class definition

```
namespace sc_core {  
  
class sc_mutex  
: public sc_mutex_if, public sc_prim_channel  
{  
    public:  
        sc_mutex();  
        explicit sc_mutex( const char* );  
  
        virtual int lock();  
        virtual int trylock();  
        virtual int unlock();  
  
        virtual const char* kind() const;  
  
    private:  
        // Disabled  
        sc_mutex( const sc_mutex& );  
        sc_mutex& operator=( const sc_mutex& );  
};  
  
} // namespace sc_core
```

### 6.27.3 Constructors

**sc\_mutex**();

This constructor shall call the base class constructor from its initializer list as follows:

**sc\_prim\_channel**( **sc\_gen\_unique\_name**( "mutex" ) )

explicit **sc\_mutex**( const char\* name\_ );

This constructor shall call the base class constructor from its initializer list as follows:

**sc\_prim\_channel**( name\_ )

Both constructors shall unlock the mutex.

#### 6.27.4 Member functions

virtual int **lock**();

If the mutex is unlocked, member function **lock** shall lock the mutex and return.

If the mutex is locked, member function **lock** shall suspend until the mutex is unlocked (by another process). At that point, it shall resume and attempt to lock the mutex by applying these same rules again.

Member function **lock** shall unconditionally return the value **0**.

If multiple processes attempt to lock the mutex in the same delta cycle, the choice of which process instance is given the lock in that delta cycle shall be non-deterministic; that is, it will rely on the order in which processes are resumed within the evaluation phase.

virtual int **trylock**();

If the mutex is unlocked, member function **trylock** shall lock the mutex and shall return the value **0**.

If the mutex is locked, member function **trylock** shall immediately return the value **-1**. The mutex shall remain locked.

virtual int **unlock**();

If the mutex is unlocked, member function **unlock** shall return the value **-1**. The mutex shall remain unlocked.

If the mutex was locked by a process instance other than the calling process, member function **unlock** shall return the value **-1**. The mutex shall remain locked.

If the mutex was locked by the calling process, member function **unlock** shall unlock the mutex and shall return the value **0**. If processes are suspended and are waiting for the mutex to be unlocked, the lock shall be given to exactly one of these processes (the choice of process instance being non-deterministic) while the remaining processes shall suspend again. This shall be accomplished within a single evaluation phase; that is, an implementation shall use immediate notification to signal the act of unlocking a mutex to other processes.

virtual const char\* **kind**() const;

Member function **kind** shall return the string **"sc\_mutex"**.

## 6.28 sc\_semaphore\_if

### 6.28.1 Description

Class **sc\_semaphore\_if** is an interface proper and is implemented by the predefined channel **sc\_semaphore**.

### 6.28.2 Class definition

```
namespace sc_core {  
  
class sc_semaphore_if  
: virtual public sc_interface  
{  
    public:  
        virtual int wait() = 0;  
        virtual int trywait() = 0;  
        virtual int post() = 0;  
        virtual int get_value() const = 0;  
  
    protected:  
        sc_semaphore_if();  
  
    private:  
        // Disabled  
        sc_semaphore_if( const sc_semaphore_if& );  
        sc_semaphore_if& operator=( const sc_semaphore_if& );  
};  
  
} // namespace sc_core
```

### 6.28.3 Member functions

The behavior of the member functions of class **sc\_semaphore\_if** is defined in class **sc\_semaphore**.

## 6.29 sc\_semaphore

### 6.29.1 Description

Class **sc\_semaphore** is a predefined primitive channel intended to model the behavior of a software semaphore used to provide limited concurrent access to a shared resource. A semaphore has an integer value, the *semaphore value*, which is set to the permitted number of concurrent accesses when the semaphore is constructed.

NOTE—Although **sc\_semaphore** is derived from **sc\_prim\_channel**, **sc\_semaphore** does not use the request update mechanism.

### 6.29.2 Class definition

```
namespace sc_core {

class sc_semaphore
: public sc_semaphore_if, public sc_prim_channel
{
public:
    explicit sc_semaphore( int );
    sc_semaphore( const char*, int );

    virtual int wait();
    virtual int trywait();
    virtual int post();
    virtual int get_value() const;

    virtual const char* kind() const;

private:
    // Disabled
    sc_semaphore( const sc_semaphore& );
    sc_semaphore& operator=( const sc_semaphore& );
};

} // namespace sc_core
```

### 6.29.3 Constructors

explicit **sc\_semaphore**( int );

This constructor shall call the base class constructor from its initializer list as follows:

```
sc_prim_channel( sc_gen_unique_name( "semaphore" ) )
```

**sc\_semaphore**( const char\* name\_, int );

This constructor shall call the base class constructor from its initializer list as follows:

```
sc_prim_channel( name_ )
```

Both constructors shall set the *semaphore value* to the value of the **int** parameter, which shall be non-negative.



#### 6.29.4 Member functions

virtual int **wait**();

If the semaphore value is greater than **0**, member function **wait** shall decrement the semaphore value and return.

If the semaphore value is equal to **0**, member function **wait** shall suspend until the semaphore value is incremented (by another process). At that point, it shall resume and attempt to decrement the semaphore value by applying these same rules again.

Member function **wait** shall unconditionally return the value **0**.

The semaphore value shall not become negative. If multiple processes attempt to decrement the semaphore value in the same delta cycle, the choice of which process instance decrements the semaphore value and which processes suspend shall be non-deterministic; that is, it will rely on the order in which processes are resumed within the evaluation phase.

virtual int **trywait**();

If the semaphore value is greater than **0**, member function **trywait** shall decrement the semaphore value and shall return the value **0**.

If the semaphore value is equal to **0**, member function **trywait** shall immediately return the value **-1** without modifying the semaphore value.

virtual int **post**();

Member function **post** shall increment the semaphore value. If processes exist that are suspended and are waiting for the semaphore value to be incremented, exactly one of these processes shall be permitted to decrement the semaphore value (the choice of process instance being non-deterministic) while the remaining processes shall suspend again. This shall be accomplished within a single evaluation phase; that is, an implementation shall use immediate notification to signal the act incrementing the semaphore value to any waiting processes.

Member function **post** shall unconditionally return the value **0**.

virtual int **get\_value**() const;

Member function **get\_value** shall return the semaphore value.

virtual const char\* **kind**() const;

Member function **kind** shall return the string "**sc\_semaphore**".

NOTE 1—The semaphore value may be decremented and incremented by different processes.

NOTE 2—The semaphore value may exceed the value set by the constructor.

## 6.30 sc\_event\_queue

### 6.30.1 Description

Class **sc\_event\_queue** represents an event queue. Like class **sc\_event**, an event queue has a member function **notify**. Unlike an **sc\_event**, an event queue is a hierarchical channel and can have multiple notifications pending.

### 6.30.2 Class definition

```
namespace sc_core {

class sc_event_queue_if
: public virtual sc_interface
{
    public:
        virtual void notify( double , sc_time_unit ) = 0;
        virtual void notify( const sc_time& ) = 0;
        virtual void cancel_all() = 0;
};

class sc_event_queue
: public sc_event_queue_if , public sc_module
{
    public:
        sc_event_queue();
        explicit sc_event_queue( sc_module_name );
        ~sc_event_queue();

        virtual const char* kind() const;

        virtual void notify( double , sc_time_unit );
        virtual void notify( const sc_time& );
        virtual void cancel_all();

        virtual const sc_event& default_event() const;
};

} // namespace sc_core
```

### 6.30.3 Constraints on usage

Class **sc\_event\_queue** is a hierarchical channel, and thus **sc\_event\_queue** objects can only be constructed during elaboration.

NOTE—An object of class **sc\_event\_queue** cannot be used in most contexts requiring an **sc\_event** but can be used to create static sensitivity because it implements member function **sc\_interface::default\_event**.

### 6.30.4 Constructors

**sc\_event\_queue**();

The default constructor shall call the base class constructor from its initializer list as follows:

```
sc_module( sc_gen_unique_name( "event_queue" ) )
```

explicit **sc\_event\_queue**( sc\_module\_name );

This constructor shall pass the module name argument through to the constructor for the base class **sc\_module**.

### 6.30.5 kind

Member function **kind** shall return the string "**sc\_event\_queue**".

### 6.30.6 Member functions

virtual void **notify**( double , sc\_time\_unit );

virtual void **notify**( const sc\_time& );

A call to member function **notify** with an argument that represents a zero time shall cause a delta notification on the default event.

A call to function **notify** with an argument that represents a non-zero time shall cause a timed notification on the default event at the given time, expressed relative to the simulation time when function **notify** is called. In other words, the value of the time argument is added to the current simulation time to determine the time at which the event will be notified.

If function **notify** is called when there is already one or more notifications pending, the new notification shall be queued in addition to the pending notifications. Each queued notification shall occur at the time determined by the semantics of function **notify**, irrespective of the order in which the calls to **notify** are made.

The default event shall not be notified more than once in any one delta cycle. If multiple notifications are pending for the same delta cycle, those notifications shall occur in successive delta cycles. If multiple timed notification are pending for the same simulation time, those notifications shall occur in successive delta cycles starting with the first delta cycle at that simulation time step and with no gaps in the delta cycle sequence.

virtual void **cancel\_all**();

Member function **cancel\_all** shall immediately delete every pending notification for this event queue object including both delta and timed notifications, but shall have no effect on other event queue objects.

virtual const sc\_event& **default\_event**() const;

Member function **default\_event** shall return a reference to the default event.

The mechanism used to queue notifications shall be implementation-defined, with the proviso that an event queue object must provide a single default event that is notified once for every call to member function **notify**.

NOTE—Event queue notifications are anonymous in the sense that the only information carried by the default event is the time of notification. A process instance sensitive to the default event cannot tell which call to function **notify** caused the notification.

*Example:*

```
sc_event_queue EQ;
```

```
SC_CTOR(Mod)
```

```
{  
    SC_THREAD(T);  
    SC_METHOD(M);  
    sensitive << EQ;  
    dont_initialize();  
}
```

```
void T()
```

```
{  
    EQ.notify(2, SC_NS);           // M runs at time 2ns  
    EQ.notify(1, SC_NS);           // M runs at time 1ns, 1st or 2nd delta cycle  
    EQ.notify(SC_ZERO_TIME);       // M runs at time 0ns  
    EQ.notify(1, SC_NS);           // M runs at time 1ns, 2nd or 1st delta cycle  
}
```

## 7. Data types

### 7.1 Introduction

All native C++ types are supported within a SystemC application. SystemC provides additional data type classes within the `sc_dt` namespace to represent values with application-specific word lengths applicable to digital hardware. These data types are referred to as *SystemC data types*.

The SystemC data type classes consist of the following:

- *limited-precision integers*, which are classes derived from class `sc_int_base`, class `sc_uint_base`, or instances of such classes. A limited-precision integer shall represent a signed or unsigned integer value at a precision limited by its underlying native C++ representation and its specified word length.
- *finite-precision integers*, which are classes derived from class `sc_signed`, class `sc_unsigned`, or instances of such classes. A finite-precision integer shall represent a signed or unsigned integer value at a precision limited only by its specified word length.
- *finite-precision fixed-point types*, which are classes derived from class `sc_fxnum` or instances of such classes. A finite-precision fixed-point type shall represent a signed or unsigned fixed-point value at a precision limited only by its specified word length, integer word length, quantization mode, and overflow mode.
- *limited-precision fixed-point types*, which are classes derived from class `sc_fxnum_fast` or instances of such classes. A limited-precision fixed-point type shall represent a signed or unsigned fixed-point value at a precision limited by its underlying native C++ floating-point representation and its specified word length, integer word length, quantization mode, and overflow mode.
- *variable-precision fixed-point type*, which is the class `sc_fxval`. A variable-precision fixed-point type shall represent a fixed-point value with a precision that may vary over time and is not subject to quantization or overflow.
- *limited variable-precision fixed-point type*, which is the class `sc_fxval_fast`. A limited variable-precision fixed-point type shall represent a fixed-point value with a precision that is limited by its underlying C++ floating-point representation and that may vary over time and is not subject to quantization or overflow.
- *single-bit logic types* implement a four-valued logic data type with states *logic 0*, *logic 1*, *high-impedance*, and *unknown* and shall be represented by the symbols '0', '1', 'X', and 'Z', respectively. The lower-case symbols 'x' and 'z' are acceptable alternatives for 'X' and 'Z', respectively, as character literals assigned to single-bit logic types.
- *bit vectors*, which are classes derived from class `sc_bv_base`, or instances of such classes. A bit vector shall implement a multiple bit data type, where each bit has a state of *logic 0* or *logic 1* and is represented by the symbols '0' or '1', respectively.
- *logic vectors*, which are classes derived from class `sc_lv_base`, or instances of such classes. A logic vector shall implement a multiple-bit data type, where each bit has a state of *logic 0*, *logic 1*, *high-impedance*, or *unknown* and is represented by the symbols '0', '1', 'X', or 'Z'. The lower-case symbols 'x' and 'z' are acceptable alternatives for 'X' and 'Z', respectively, within string literals assigned to logic vectors.

Apart from the single-bit logic types, the variable-precision fixed-point types, and the limited variable-precision fixed-point types, the classes within each category are organized as an object-oriented hierarchy with common behavior defined in base classes. A class template shall be derived from each base class by the implementation such that applications can specify word lengths as template arguments.

The term *fixed-point type* is used in this standard to refer to any finite-precision fixed-point type or limited-precision fixed-point type. The variable-precision and limited variable-precision fixed-point types are fixed-point types only in the restricted sense that they store a representation of a fixed-point value and can be

mixed with other fixed-point types in expressions, but they are not fixed-point types in the sense that they do not model quantization or overflow effects and are not intended to be used directly by an application.

The term *numeric type* is used in this standard to refer to any limited-precision integer, finite-precision integer, finite-precision fixed-point type, or limited-precision fixed-point type. The term *vector* is used to refer to any bit vector or logic vector. The word length of a numeric type or vector object shall be set when the object is initialized and shall not subsequently be altered. Each bit within a word shall have an index. The right-hand bit shall have index **0** and is the least-significant bit for numeric types. The index of the left-hand bit shall be the word length minus **1**.

The limited-precision signed integer base class is **sc\_int\_base**. The limited-precision unsigned integer base class is **sc\_uint\_base**. The corresponding class templates are **sc\_int** and **sc\_uint**, respectively.

The finite-precision signed integer base class is **sc\_signed**. The finite-precision unsigned integer base class is **sc\_unsigned**. The corresponding class templates are **sc\_bigint** and **sc\_biguint**, respectively.

The signed finite-precision fixed-point base class is **sc\_fix**. The unsigned finite-precision fixed-point base class is **sc\_ufix**. Both base classes are derived from class **sc\_fxnum**. The corresponding class templates are **sc\_fixed** and **sc\_ufixed**, respectively.

The signed limited-precision fixed-point base class is **sc\_fix\_fast**. The unsigned limited-precision fixed-point base class is **sc\_ufix\_fast**. Both base classes are derived from class **sc\_fxnum\_fast**. The corresponding class templates are **sc\_fixed\_fast** and **sc\_ufixed\_fast**, respectively.

The variable-precision fixed-point class is **sc\_fxval**. The limited variable-precision fixed-point class is **sc\_fxval\_fast**. These two classes are used as the operand types and return types of many fixed-point operations.

The bit vector base class is **sc\_bv\_base**. The corresponding class template is **sc\_bv**.

The logic vector base class is **sc\_lv\_base**. The corresponding class template is **sc\_lv**.

The single-bit logic type is **sc\_logic**.

It is recommended that applications create SystemC data type objects using the class templates given in this clause (for example, **sc\_int**) rather than the untemplated base classes (for example, **sc\_int\_base**).

The relationships between the SystemC data type classes are shown in Table 2.

**Table 2—SystemC data types**

| <b>Class template</b> | <b>Base class</b> | <b>Generic base class</b> | <b>Representation</b> | <b>Precision</b> |
|-----------------------|-------------------|---------------------------|-----------------------|------------------|
| sc_int                | sc_int_base       | sc_value_base             | signed integer        | limited          |
| sc_uint               | sc_uint_base      | sc_value_base             | unsigned integer      | limited          |
| sc_bigint             | sc_signed         | sc_value_base             | signed integer        | finite           |
| sc_biguint            | sc_unsigned       | sc_value_base             | unsigned integer      | finite           |
| sc_fixed              | sc_fix            | sc_fxnum                  | signed fixed-point    | finite           |
| sc_ufixed             | sc_ufix           | sc_fxnum                  | unsigned fixed-point  | finite           |
| sc_fixed_fast         | sc_fix_fast       | sc_fxnum_fast             | signed fixed-point    | limited          |
| sc_ufixed_fast        | sc_ufix_fast      | sc_fxnum_fast             | unsigned fixed-point  | limited          |
|                       |                   | sc_fxval                  | fixed-point           | variable         |
|                       |                   | sc_fxval_fast             | fixed-point           | limited-variable |
|                       | sc_logic          |                           | single bit            |                  |
| sc_bv                 | sc_bv_base        |                           | bit vector            |                  |
| sc_lv                 | sc_lv_base        |                           | logic vector          |                  |

## 7.2 Common characteristics

This subclause specifies some common characteristics of the SystemC data types such as common operators and functions. This subclause should be taken as specifying a set of obligations on the implementation to provide operators and functions with the given behavior. In some cases the implementation has some flexibility with regard to how the given behavior is implemented. The remainder of Clause 7 gives a detailed definition of the SystemC data type classes.

An underlying principle is that native C++ integer and floating-point types, C++ string types, and SystemC data types may be mixed in expressions.

Equality and bitwise operators can be used for all SystemC data types. Arithmetic and relational operators can be used with the numeric types only. The semantics of the equality operators, bitwise operators, arithmetic operators, and relational operators are the same in SystemC as in C++.

User-defined conversions supplied by the implementation support translation from SystemC types to C++ native types and other SystemC types.

Bit-select, part-select, and concatenation operators return an instance of a proxy class. The term *proxy class* is used in this standard to refer to a class whose purpose is to represent a SystemC data type object within an expression and which provides additional operators or features not otherwise present in the represented object. An example is a proxy class that allows an `sc_int` variable to be used as if it were a C++ array of `bool` and to distinguish between its use as an rvalue or an lvalue within an expression. Instances of proxy classes are only intended to be used within the expressions that create them. An application should not call a proxy class constructor to create a named object and should not declare a pointer or reference to a proxy class. It is strongly recommended that an application avoid the use of a proxy class as the return type of a function because the lifetime of the object to which the proxy class refers may not extend beyond the function return statement.

NOTE 1—The bitwise shift left or shift right operation has no meaning for a single-bit logic type and is undefined.

NOTE 2—The term *user-defined conversions* in this context has the same meaning as in the C++ standard. It applies to type conversions of class objects by calling constructors and conversion functions that are used for implicit type conversions and explicit type conversions.

NOTE 3—Care should be taken when mixing signed and unsigned numeric types in expressions that use implicit type conversions, since an implementation is not required to issue a warning if the polarity of a converted value is changed.

### 7.2.1 Initialization and assignment operators

Overloaded constructors shall be provided by the implementation for all integer (limited-precision integer and finite-precision integer) class templates that allow initialization with an object of any SystemC data type.

Overloaded constructors shall be provided for all vector (bit vector and logic vector) class templates that allow initialization with an object of any SystemC integer or vector data type.

Overloaded constructors shall be provided for all finite-precision fixed-point and limited precision fixed-point class templates that allow initialization with an object of any SystemC integer data type.

All SystemC data type classes shall define a copy constructor that creates a copy of the specified object with the same value and the same word length.

Overloaded assignment operators and constructors shall perform direct or indirect conversion between types. The data type base classes may define a restricted set of constructors and assignment operators that only permit direct initialization from a subset of the SystemC data types. As a general principal, data type



class template constructors may be called implicitly by an application to perform conversion from other types since their word length is specified by a template argument. On the other hand, the data type base class constructors with a single parameter of a different type should only be called explicitly since the required word length is not specified.

If the target of an assignment operation has a word length that is insufficient to hold the value assigned to it, the left-hand bits of the value stored shall be truncated to fit the target word length. If truncation occurs, an implementation may generate a warning but is not obliged to do so, and an application can in any case disable such a warning (see 3.3.5).

If a data type object or string literal is assigned to a target having a greater word length, the value shall be extended with additional bits at its left-hand side to match the target word length. Extension of a signed numeric type shall preserve both its sign and magnitude and is referred to as *sign extension*. Extension of all other types shall insert bits with a value of logic 0 and is referred to as *zero extension*.

Assignment of a fixed-point type to an integer type shall use the integer component only; any fractional component is discarded.

Assignment of a value with a word length greater than 1 to a single-bit logic type shall be an error.

NOTE—An integer literal is always treated as unsigned unless prefixed by a minus symbol. An unsigned integer literal will always be extended with leading zeros when assigned to a data type object having a larger word length, regardless of whether the object itself is signed or unsigned.

## 7.2.2 Precision of arithmetic expressions

The type of the value returned by any arithmetic expression containing only limited-precision integers or limited-precision integers and native C++ integer types shall be an implementation-defined C++ integer type with a maximum word length of 64 bits. The action taken by an implementation if the precision required by the return value exceeds 64 bits is undefined and the value is implementation-dependent.

The value returned by any arithmetic expression containing only finite-precision integers or finite-precision integers and any combination of limited-precision or native C++ integer types shall be a finite-precision integer with a word-length sufficient to contain the value with no loss of accuracy.

The value returned by any arithmetic expression containing any fixed-point type shall be a variable-precision or limited variable-precision fixed-point type (see 7.10.4).

Applications should use explicit type casts within expressions combining multiple types where an implementation does not provide overloaded operators with signatures exactly matching the operand types.

*Example:*

```
int i = 10;
sc_dt::int64 i64 = 100;           // long long int
sc_int<16> sci = 2;
sc_bigint<16> bi = 20;
float f = 2.5;
sc_fixed<16,8> scf = 2.5;

( i * sci );                      // Ambiguous
( i * static_cast<sc_dt::int_type>(sci) ); // Implementation-defined C++ integer
( i * bi );                      // 48-bit finite-precision integer (assumes int = 32 bits)
```

```

(i64 * bi);           // 80-bit finite-precision integer
(f * bi);             // Ambiguous
(static_cast<int>(f) * bi); // 48-bit finite-precision integer (assumes int = 32 bits)
(scf * sci);          // Variable-precision fixed-point type

```

### 7.2.3 Base class default word length

The default word length of a data type base class shall be used where its default constructor is called (implicitly or explicitly). The default word length shall be set by the *length parameter* in context at the point of construction. A length parameter may be brought into context by creating a length context object. Length contexts shall have local scope and by default be activated immediately. Once activated, they shall remain in effect for as long as they are in scope, or until another length context is activated. Activation of a length context shall be deferred if its second constructor argument is `SC_LATER` (the default value is `SC_NOW`). A deferred length context can be activated by calling its member function **begin**.

Length contexts shall be managed by a global length context stack. When a length context is activated, it shall be placed at the top of the stack. A length context may be deactivated and removed from the top of the stack by calling its member function **end**. The **end** method shall only be called for the length context currently at the top of the context stack. A length context is also deactivated and removed from the stack when it goes out of scope. The current context shall always be the length context at the top of the stack.

A length context shall only be activated once. An active length context shall only be deactivated once.

The classes `sc_length_param` and `sc_length_context` shall be used to create length parameters and length contexts, respectively, for SystemC integers and vectors.

In addition to the word length, the fixed-point types shall have default integer word length and mode attributes. These shall be set by the *fixed-point type parameter* in context at the point of construction. A fixed-point type parameter shall be brought into context by creating a *fixed-point type context object*. The use of a fixed-point type context shall follow the same rules as a length context. A stack for fixed-point type contexts with the same characteristics as the length context stack shall exist.

The classes `sc_fxtype_params` and `sc_fxtype_context` shall be used to create fixed-point type parameters and fixed-point type contexts, respectively.

*Example:*

```

sc_length_param length10(10);
sc_length_context cntxt10(length10);           // length10 now in context
sc_int_base int_array[2];                     // Array of 10-bit integers
sc_core::sc_signal<sc_int_base> S1;           // Signal of 10-bit integer
{
    sc_length_param length12(12);
    sc_length_context cntxt12(length12,SC_LATER); // cntxt12 deferred
    sc_length_param length14(14);
    sc_length_context cntxt14(length14,SC_LATER); // cntxt14 deferred
    sc_uint_base var1;                          // length 10
    cntxt12.begin();                            // Bring length12 into context
    sc_uint_base var2;                          // length 12
    cntxt14.begin();                            // Bring length14 into context
    sc_uint_base var3;                          // length 14
    cntxt14.end();                              // end cntx14, cntx12 restored
    sc_bv_base var4;                            // length 12
}

```

```
}                                // cntxt12 out of scope, cntx10 restored  
sc_bv_base var5;                // length 10
```

NOTE 1—The context stacks allow a default context to be locally replaced by an alternative context and subsequently restored.

NOTE 2—An activated context remains active for the lifetime of the context object or until it is explicitly deactivated. A context can therefore affect the default parameters of data type objects created outside of the function in which it is activated. An application should ensure that any contexts created or activated within functions whose execution order is non-deterministic do not result in temporal ordering dependencies in other parts of the application. Failure to meet this condition could result in behavior that is implementation-dependent.

## 7.2.4 Word length

The word length (a positive integer indicating the number of bits) of a SystemC integer, vector, part-select, or concatenation shall be returned by the member function **length**.

## 7.2.5 Bit-select

*Bit-selects* are instances of a proxy class that reference the bit at the specified position within an associated object that is a SystemC numeric type or vector.

The C++ subscript operator (**operator[]**) shall be overloaded by the implementation to create a bit-select when called with a single non-negative integer argument specifying the bit position. It shall be an error if the specified bit position is outside the bounds of its numeric type or vector object.

User-defined conversions shall allow bit-selects to be used in expressions where a **bool** object operand is expected. A bit-select of an lvalue may be used as an rvalue or an lvalue. A bit-select of an rvalue shall only be used as an rvalue.

A bit-select or a **bool** value may be assigned to an lvalue bit-select. The assignment shall modify the state of the selected bit within the associated numeric type or vector object represented by the lvalue. An application shall not assign a value to an rvalue bit-select.

Bit-selects for integer, bit vector, and logic vector types shall have an explicit **to\_bool** conversion function that returns the state of the selected bit.

*Example:*

```
sc_int<4> I1;                    // 4 bit signed integer  
I1[1] = true;                    // Selected bit used as lvalue  
bool b0 = I1[0].to_bool();       // Selected bit used as rvalue
```

NOTE 1—Bit-selects corresponding to lvalues and rvalues of a particular type are themselves objects of two distinct classes.

NOTE 2—A bit-select class can contain user-defined conversions for both implicit and explicit conversion of the selected bit value to **bool**.

## 7.2.6 Part-select

*Part-selects* are instances of a proxy class that provide access to a contiguous subset of bits within an associated object that is a numeric type or vector.

The member function **range( int , int )** of a numeric type, bit vector, or logic vector shall create a part-select. The two non-negative integer arguments specify the left- and right-hand index positions. A part-select shall

provide a reference to a word within its associated object, starting at the left-hand index position and extending to, and including, the right-hand index position. It shall be an error if the left-hand index position or right-hand index position lies outside the bounds of the object.

The C++ function call operator (**operator()**) shall be overloaded by the implementation to create a part-select and may be used as a direct replacement for the **range** function.

User-defined conversions shall allow a part-select to be used in expressions where the expected operand is an object of the numeric type or vector type associated with the part-select, subject to certain constraints (see 7.5.7.3, 7.6.8.3, 7.9.8.3). A part-select of an lvalue may be used as an rvalue or an lvalue. A part-select of an rvalue shall only be used as an rvalue.

Integer part-selects may be directly assigned to an object of any other SystemC data type, with the exception of bit-selects. Fixed-point part-selects may be directly assigned to any SystemC integer or vector, any part-select or any concatenation. Vector part-selects may only be directly assigned to a vector, vector part-select, or vector concatenation (assignments to other types are ambiguous or require an explicit conversion).

The bits within a part-select do not reflect the sign of their associated object and shall be taken as representing an unsigned binary number when converted to a numeric value. Assignments of part-selects to a target having a greater word length shall be zero extended, regardless of the type of their associated object.

*Example:*

```
sc_int<8> I2 = 2;           // "0b00000010"
I2.range(3,2) = I2.range(1,0); // "0b00001010"
sc_int<8> I3 = I2.range(3,0); // "0b00001010"
                           // Zero-extended to 8 bits
sc_bv<8> b1 = "0b11110000";
b1.range(5,2) = b1.range(2,5); // "0b11001100"
                           // Reversed bit-order between position 5 and 2
```

NOTE 1—A part-select cannot be used to reverse the bit-order of a limited-precision integer type.

NOTE 2—Part-selects corresponding to lvalues and rvalues of a particular type are themselves objects of two distinct classes.

NOTE 3—A part-select is not required to be an acceptable replacement where an object reference operand is expected. If an implementation provides a mechanism to allow such replacements (for example, by defining the appropriate overloaded member functions), it is not required to do so for all data types.

## 7.2.7 Concatenation

*Concatenations* are instances of a proxy class that reference the bits within multiple objects as if they were part of a single aggregate object.

The **concat**( arg0 , arg1 ) function shall create a concatenation. The *concatenation arguments* (arg0 and arg1) may be two SystemC integer, vector, bit-select, part-select, or concatenation objects. The C++ comma operator (**operator,**) shall also be overloaded to create a concatenation and may be used as a direct replacement for the **concat** function.

The type of a concatenation argument shall be a *concatenation base type*, or it shall be derived from a concatenation base type. An implementation shall provide a common concatenation base type for all SystemC integers and a common concatenation base type for all vectors. The concatenation base type of bit-select and part-select concatenation arguments is the same as their associated integer or vector objects. The concatenation arguments may be any combination of two objects having the same concatenation base type.

A concatenation object shall have the same concatenation base type as the concatenation arguments passed to the function that created the object. The set of permissible concatenation arguments for a given concatenation base type consists of the following:

- a) Objects whose base class or concatenation base type matches the given concatenation base type
- b) Bit-selects of a)
- c) Part-selects of a)
- d) Concatenations of a) and/or b) and/or c) in any combination

When both concatenation arguments are lvalues, the concatenation shall be an lvalue. If any concatenation argument is an rvalue, the concatenation shall be an rvalue.

A single concatenation argument may be a **bool** value when the other argument is a SystemC integer, vector, bit-select, part-select, or concatenation object. The resulting concatenation shall be an rvalue.

An expression may be assigned to an lvalue concatenation if the base type of the expression return value is the same as the base type of the lvalue concatenation. If the word length of a value assigned to a concatenation with a signed base type is smaller than the word length of the concatenation, the value shall be sign-extended to match the word length of the concatenation. Assignments to concatenations of all other numeric types and vectors shall be zero-extended (if required). Assignment to a concatenation shall update the values of the objects specified by its concatenation arguments.

A concatenation may be assigned to an object whose base class is the same as the concatenation base type. Where a concatenation is assigned to a target having a greater word length than the concatenation, it is zero-extended to the target length. When a concatenation is assigned to a target having a shorter word length than the concatenation, the left-hand bits of the value shall be truncated to fit the target word length. If truncation occurs, an implementation may generate a warning but is not obliged to do so, and an application can in any case disable such a warning (see 3.3.5).

*Example:*

The following concatenations are well-formed:

```
sc_uint<8> U1 = 2;                // "0b00000010"
sc_uint<2> U2 = 1;                // "0b01"
sc_uint<8> U3 = (true,U1.range(3,0),U2,U2[0]); // U3 = "0b10010011"
// Base class same as concatenation base type
(U2[0],U1[0],U1.range(7,1)) = (U1[7],U1); // Copies U1[7] to U2[0], U1 rotated left
concat(U2[0],concat(U1[0],U1.range(7,1))) = concat(U1[7],U1);
// Same as previous example but using concat
```

The following concatenations are ill-formed:

```
sc_bv<8> Bv1;
(Bv1,U1) = "0xffff";             // Bv1 and U1 do not share common base type

bool C1=true; bool C2 = false;
U2 = (C1,C1);                    // Cannot concatenate 2 bool objects
(C1,I1) = "0x1ff";              // Bool concatenation argument creates rvalue
```

NOTE 1—Parentheses are required around the concatenation arguments when using the C++ comma operator because of its low operator precedence.

NOTE 2—An implementation is not required to support bit-selects and part-selects of concatenations.

NOTE 3—Concatenations corresponding to lvalues and rvalues of a particular type are themselves objects of two distinct classes.

### 7.2.8 Reduction operators

The reduction operators shall perform a sequence of bitwise operations on a SystemC integer or vector to produce a **bool** result. The first step shall be a boolean operation applied to the first and second bits of the object. The boolean operation shall then be re-applied using the previous result and the next bit of the object. This process shall be repeated until every bit of the object has been processed. The value returned shall be the result of the final boolean operation. The following reduction operators shall be provided:

- a) **and\_reduce** performs a bitwise AND between all bits.
- b) **nand\_reduce** performs a bitwise NAND between all bits.
- c) **or\_reduce** performs a bitwise OR between all bits.
- d) **nor\_reduce** performs a bitwise NOR between all bits.
- e) **xor\_reduce** performs a bitwise XOR between all bits.
- f) **xnor\_reduce** performs a bitwise XNOR between all bits.

### 7.2.9 Integer conversion

All SystemC data types shall provide an assignment operator that can accept a C++ integer value. A signed value shall be sign-extended to match the length of the SystemC data type target.

SystemC data types shall provide member functions for explicit type conversion to C++ integer types as follows:

- a) **to\_int** converts to native C++ **int** type.
- b) **to\_uint** converts to native C++ **unsigned** type.
- c) **to\_long** converts to native C++ **long** type.
- d) **to\_ulong** converts to native C++ **unsigned long** type.
- e) **to\_uint64()** converts to a native C++ unsigned integer type having a word length of 64 bits.
- f) **to\_int64()** converts to native C++ integer type having a word length of 64 bits.

These member functions shall interpret the bits within a SystemC integer, fixed-point type or vector, or any part-select or concatenation thereof, as representing an unsigned binary value, with the exception of signed integers and signed fixed-point types.

Truncation shall be performed where necessary for the value to be represented as a C++ integer.

Attempting to convert a logic vector containing 'X' or 'Z' values to an integer shall be an error.

### 7.2.10 String input and output

```
void scan( std::istream& is = std::cin );
void print( std::ostream& os = std::cout ) const;
```

All SystemC data types shall provide a member function **scan** that allows an object value to be set by reading a string from the specified C++ input stream. The string content may use any of the representations permitted by 7.3.

All SystemC data types shall provide a member function **print** that allows an object value to be written to a C++ output stream.

SystemC numeric types shall be printed as signed or unsigned decimal values. SystemC vector types shall be printed as a string of bit values.

All SystemC data types shall support the output stream inserter (**operator<<**) for formatted printing to a C++ stream. The format shall be the same as for the member function **print**.

The C++ ostream manipulators **dec**, **oct**, and **hex** shall have the same effect for limited-precision and finite-precision integers and vector types as they do for standard C++ integers: that is, they shall cause the values of such objects to be printed in decimal, octal or hexadecimal formats, respectively. The formats used shall be those described in 7.3 with the exception that vectors shall be printed as a bit-pattern string when the **dec** manipulator is active.

All SystemC data types shall support the input stream inserter (**operator>>**) for formatted input from a C++ input stream. The permitted formats shall be the same as those permitted for the member function **scan**.

void **dump** ( std::ostream& os = std::cout ) const;

All fixed-point types shall additionally provide a member function **dump** that shall print at least the type name and value to the stream passed as an argument. The purpose of **dump** is to allow an implementation to dump out diagnostic information to help the user debug an application.

#### 7.2.11 Conversion of application-defined types in integer expressions

The generic base proxy class template **sc\_generic\_base** shall be provided by the implementation and may be used as a base class for application-defined classes.

All SystemC integer, integer part-select, and integer concatenation classes shall provide an assignment operator that accepts an object derived from the generic base proxy class template. All SystemC integer classes shall additionally provide an overloaded constructor with a single argument that is a constant reference to a generic base proxy object.

NOTE—The generic base proxy class is not included in the collection of classes described by the term “SystemC data types” as used in this standard.

### 7.3 String literals

A string literal representation may be used as the value of a SystemC numeric or vector type object. It shall consist of a standard prefix followed by a magnitude expressed as one or more digits.

The magnitude representation for SystemC integer types shall be based on that of C++ integer literals.

The magnitude representation for SystemC vector types shall be based on that of C++ unsigned integer literals.

The magnitude representation for SystemC fixed-point types shall be based on that of C++ floating literals but without the optional floating suffix.

The permitted representations are identified with a symbol from the enumerated type **sc\_numrep** as specified in Table 3.

**Table 3—String literal representation**

| <b>sc_numrep</b> | <b>Prefix</b> | <b>Magnitude format</b>      |
|------------------|---------------|------------------------------|
| SC_DEC           | 0d            | decimal                      |
| SC_BIN           | 0b            | binary                       |
| SC_BIN_US        | 0bus          | binary unsigned              |
| SC_BIN_SM        | 0bsm          | binary sign & magnitude      |
| SC_OCT           | 0o            | octal                        |
| SC_OCT_US        | 0ous          | octal unsigned               |
| SC_OCT_SM        | 0osm          | octal sign & magnitude       |
| SC_HEX           | 0x            | hexadecimal                  |
| SC_HEX_US        | 0xus          | hexadecimal unsigned         |
| SC_HEX_SM        | 0xsm          | hexadecimal sign & magnitude |
| SC_CSD           | 0csd          | canonical signed digit       |

An implementation shall provide overloaded constructors and assignment operators that permit the value of any SystemC numeric type or vector to be set by a character string having one of the prefixes specified in Table 3. The character ‘+’ or ‘-’ may optionally be placed before the prefix for decimal and “sign & magnitude” formats to indicate polarity. The prefix shall be followed by an unsigned integer value, except in the cases of the binary, octal, and hexadecimal formats, where the prefix shall be followed by a two’s complement value expressed as a binary, octal, or hexadecimal integer, respectively. An implementation shall sign-extend any integer string literal used to set the value of an object having a longer word length.

The canonical signed digit representation shall use the character ‘-’ to represent the bit value **-1**.



A bit-pattern string (containing bit or logic character values with no prefix) may be assigned to a vector. If the number of characters in the bit-pattern string is less than the vector word length, the string shall be zero extended at its left-hand side to the vector word length. The result of assigning such a string to a numeric type is undefined.

An instance of a SystemC numeric type, vector, part-select, or concatenation may be converted to a C++ **std::string** object by calling its member function **to\_string**. The signature of **to\_string** shall be as follows:

```
std::string to_string( sc_numrep numrep , bool with_prefix );
```

The **numrep** argument shall be one of the **sc\_numrep** values given in Table 3. The magnitude representation in a string created from an unsigned integer or vector shall be prefixed by a single zero, except where **numrep** is **SC\_DEC**. If the **with\_prefix** argument is **true**, the prefix corresponding to the **numrep** value in Table 3 shall be appended to the left-hand side of the resulting string. The default value of **with\_prefix** shall be **true**.

It shall be an error to call the member function **to\_string** of a logic-vector object if any of its elements have the value '**X**' or '**Z**'.

The value of an instance of a single-bit logic type may be converted to a single character by calling its member function **to\_char**.

*Example:*

```
sc_int<4> I1;                // 4-bit signed integer
I1 = "0b10100";             // 5-bit signed binary literal truncated to 4 bits
std::string S1 = I1.to_string(SC_BIN,true); // The contents of S1 will be the string "0b0100"
sc_int<10> I2;               // 10-bit integer
I2 = "0d478";                // Decimal equivalent of "0b0111011110"
std::string S2 = I2.to_string(SC_CSD,false); // The contents of S2 will be the string "1000-000-0"
sc_uint<8> I3;               // 8-bit unsigned integer
I3 = "0x7";                  // Zero-extended to 8-bit value "0x07"
std::string S3 = I3.to_string(SC_HEX);      // The contents of S3 will be the string "0x007"
sc_lv<16> lv;                // 16-bit logic vector
lv = "0xff";                 // Sign-extended to 16-bit value "0xffff"
std::string S4 = lv.to_string(SC_HEX);      // The contents of S4 will be the string "0x0ffff"
sc_bv<8> bv;                 // 8-bit bit vector
bv = "11110000";             // Bit-pattern string
std::string S5 = bv.to_string(SC_BIN);      // The contents of S5 will be the string "0b011110000"
```

NOTE—SystemC data types may provide additional overloaded **to\_string** functions that require a different number of arguments.

## 7.4 *sc\_value\_base*<sup>†</sup>

### 7.4.1 Description

Class *sc\_value\_base*<sup>†</sup> provides a common base class for all SystemC limited-precision integers and finite-precision integers. It provides a set of virtual methods that may be called by an implementation to perform concatenation operations.

#### 7.4.1.1 Class definition

```
namespace sc_dt {

class sc_value_base†
{
    friend class sc_concatref†;
private:
    virtual void concat_clear_data( bool to_ones=false );
    virtual bool concat_get_ctrl( unsigned long* dst_p , int low_i ) const;
    virtual bool concat_get_data( unsigned long* dst_p , int low_i ) const;
    virtual uint64 concat_get_uint64() const;
    virtual int concat_length( bool* xz_present_p=0 ) const;
    virtual void concat_set( int64 src , int low_i );
    virtual void concat_set( const sc_signed& src , int low_i );
    virtual void concat_set( const sc_unsigned& src , int low_i );
    virtual void concat_set( uint64 src , int low_i );
};

} // namespace sc_dt
```

#### 7.4.1.2 Constraints on usage

An application should not create an object of type *sc\_value\_base*<sup>†</sup> and should not directly call any member function inherited by a derived class from an *sc\_value\_base*<sup>†</sup> parent.

If an application-defined class derived from the generic base proxy class template **sc\_generic\_base** is also derived from *sc\_value\_base*<sup>†</sup>, objects of this class may be used as arguments to an integer concatenation. Such a class shall override the virtual member functions of *sc\_value\_base*<sup>†</sup> as private members to provide the concatenation operations permitted for objects of that type.

It shall be an error for any member function of *sc\_value\_base*<sup>†</sup> that is not overridden in a derived class to be called for an object of the derived class.

#### 7.4.1.3 Member functions

virtual void **concat\_clear\_data**( bool to\_ones=false );

Member function **concat\_clear\_data** shall set every bit in the *sc\_value\_base*<sup>†</sup> object to the state provided by the argument.

virtual bool **concat\_get\_ctrl**( unsigned long\* dst\_p , int low\_i ) const;

Member function **concat\_get\_ctrl** shall copy control data to the packed-array given as the first argument, starting at the bit position within the packed-array given by the second argument. The return value shall always be **false**.

virtual bool **concat\_get\_data**( unsigned long\* dst\_p , int low\_i ) const;

Member function **concat\_get\_data** shall copy data to the packed-array given as the first argument, starting at the bit position within the packed-array given by the second argument. The return value shall be **true** if the data is non-zero; otherwise, it shall be **false**.

virtual uint64 **concat\_get\_uint64**() const;

Member function **concat\_get\_uint64** shall return the value of the *sc\_value\_base*<sup>†</sup> object as a C++ unsigned integer having a word length of exactly 64-bits.

virtual int **concat\_length**( bool\* xz\_present\_p=0 ) const;

Member function **concat\_length** shall return the number of bits in the *sc\_value\_base*<sup>†</sup> object. The value of the object associated with the optional argument shall be set to **true** if any bits have the value 'X' or 'Z'.

virtual void **concat\_set**( int64 src , int low\_i );

virtual void **concat\_set**( const sc\_signed& src , int low\_i );

virtual void **concat\_set**( const sc\_unsigned& src , int low\_i );

virtual void **concat\_set**( uint64 src , int low\_i );

Member function **concat\_set** shall set the value of the *sc\_value\_base*<sup>†</sup> object to the bit-pattern of the integer given by the first argument. The bit-pattern shall be read as a contiguous sequence of bits starting at the position given by the second argument.

## 7.5 Limited-precision integer types

### 7.5.1 Type definitions

The following type definitions are used in the limited-precision integer type classes:

```
namespace sc_dt {  
  
    typedef implementation-defined int_type;  
    typedef implementation-defined uint_type;  
    typedef implementation-defined int64;  
    typedef implementation-defined uint64;  
  
} // namespace sc_dt
```

**int\_type** is an implementation-dependent native C++ integer type. An implementation shall provide a minimum representation size of 64 bits.

**uint\_type** is an implementation-dependent native C++ unsigned integer type. An implementation shall provide a minimum representation size of 64 bits.

**int64** is a native C++ integer type having a word length of exactly 64 bits.

**uint64** is a native C++ unsigned integer type having a word length of exactly 64 bits.

## 7.5.2 **sc\_int\_base**

### 7.5.2.1 Description

Class **sc\_int\_base** represents a limited word-length integer. The word length is specified by a constructor argument or, by default, by the **sc\_length\_context** object currently in scope. The word length of an **sc\_int\_base** object shall be fixed during instantiation and shall not subsequently be changed.

The integer value shall be held in an implementation-dependent native C++ integer type. A minimum representation size of 64 bits is required.

**sc\_int\_base** is the base class for the **sc\_int** class template.

### 7.5.2.2 Class definition

```
namespace sc_dt {

class sc_int_base
: public sc_value_base†
{
    friend class sc_uint_bitref†r;
    friend class sc_uint_bitref†r;
    friend class sc_uint_subref†r;
    friend class sc_uint_subref†r;

public:
    // Constructors
    explicit sc_int_base( int w = sc_length_param().len() );
    sc_int_base( int_type v , int w );
    sc_int_base( const sc_int_base& a );

    template< typename T >
    explicit sc_int_base( const sc_generic_base<T>& a );
    explicit sc_int_base( const sc_int_subref†r& a );
    explicit sc_int_base( const sc_signed& a );
    explicit sc_int_base( const sc_unsigned& a );
    explicit sc_int_base( const sc_bv_base& v );
    explicit sc_int_base( const sc_lv_base& v );
    explicit sc_int_base( const sc_uint_subref†r& v );
    explicit sc_int_base( const sc_signed_subref†r& v );
    explicit sc_int_base( const sc_unsigned_subref†r& v );

    // Destructor
    ~sc_int_base();

    // Assignment operators
    sc_int_base& operator=( int_type v );
    sc_int_base& operator=( const sc_int_base& a );
    sc_int_base& operator=( const sc_int_subref†r& a );
    template<class T>
    sc_int_base& operator=( const sc_generic_base<T>& a );
    sc_int_base& operator=( const sc_signed& a );
    sc_int_base& operator=( const sc_unsigned& a );
    sc_int_base& operator=( const sc_fxval& a );
```

```

sc_int_base& operator= ( const sc_fxval_fast& a );
sc_int_base& operator= ( const sc_fxnum& a );
sc_int_base& operator= ( const sc_fxnum_fast& a );
sc_int_base& operator= ( const sc_bv_base& a );
sc_int_base& operator= ( const sc_lv_base& a );
sc_int_base& operator= ( const char* a );
sc_int_base& operator= ( unsigned long a );
sc_int_base& operator= ( long a );
sc_int_base& operator= ( unsigned int a );
sc_int_base& operator= ( int a );
sc_int_base& operator= ( uint64 a );
sc_int_base& operator= ( double a );

// Prefix and postfix increment and decrement operators
sc_int_base& operator++ ();           // Prefix
const sc_int_base operator++ ( int ); // Postfix
sc_int_base& operator-- ();          // Prefix
const sc_int_base operator-- ( int ); // Postfix

// Bit selection
sc_int_bitref† operator[] ( int i );
sc_int_bitref† operator[] ( int i ) const;

// Part selection
sc_int_subref† operator() ( int left , int right );
sc_int_subref† operator() ( int left , int right ) const;
sc_int_subref† range( int left , int right );
sc_int_subref† range( int left , int right ) const;

// Capacity
int length() const;

// Reduce methods
bool and_reduce() const;
bool nand_reduce() const;
bool or_reduce() const;
bool nor_reduce() const;
bool xor_reduce() const;
bool xnor_reduce() const;

// Implicit conversion to int_type
operator int_type() const;

// Explicit conversions
int to_int() const;
unsigned int to_uint() const;
long to_long() const;
unsigned long to_ulong() const;
int64 to_int64() const;
uint64 to_uint64() const;
double to_double() const;

// Explicit conversion to character string
const std::string to_string( sc_numrep numrep = SC_DEC ) const;

```

```

const std::string to_string( sc_numrep numrep , bool w_prefix ) const;

// Other methods
void print( std::ostream& os = std::cout ) const;
void scan( std::istream& is = std::cin );

};

} // namespace sc_dt

```

### 7.5.2.3 Constraints on usage

The word length of an **sc\_int\_base** object shall not be greater than the maximum size of the integer representation used to hold its value.

### 7.5.2.4 Constructors

explicit **sc\_int\_base**( int w = sc\_length\_param().len() );

Constructor **sc\_int\_base** shall create an object of word length specified by **w**. It is the default constructor when **w** is not specified (in which case its value shall be set by the current length context). The initial value of the object shall be 0.

**sc\_int\_base**( int\_type v , int w );

Constructor **sc\_int\_base** shall create an object of word length specified by **w** with initial value specified by **v**. Truncation of most significant bits shall occur if the value cannot be represented in the specified word length.

template< class T >

**sc\_int\_base**( const sc\_generic\_base<T>& a );

Constructor **sc\_int\_base** shall create an **sc\_int\_base** object with a word length matching the constructor argument. The constructor shall set the initial value of the object to the value returned from the member function **to\_int64** of the constructor argument.

The other constructors shall create an **sc\_int\_base** object whose size and value matches that of the argument. The size of the argument shall not be greater than the maximum word length of an **sc\_int\_base** object.

### 7.5.2.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to **sc\_int\_base**, using truncation or sign-extension as described in 7.2.1.

### 7.5.2.6 Implicit type conversion

**operator int\_type**() const;

Operator **int\_type** can be used for implicit type conversion from **sc\_int\_base** to the native C++ integer representation.

NOTE 1—This operator enables the use of standard C++ bitwise logical and arithmetic operators with **sc\_int\_base** objects.

NOTE 2—This operator is used by the C++ output stream operator and by the member functions of other data type classes that are not explicitly overload for **sc\_int\_base**.

### 7.5.2.7 Explicit type conversion

```
const std::string to_string( sc_numrep numrep = SC_DEC ) const;
const std::string to_string( sc_numrep numrep, bool w_prefix ) const;
```

Member function **to\_string** shall perform the conversion to an **std::string**, as described in 7.2.11. Calling the **to\_string** function with a single argument is equivalent to calling the **to\_string** function with two arguments where the second argument is **true**. Calling the **to\_string** function with no arguments is equivalent to calling the **to\_string** function with two arguments, where the first argument is **SC\_DEC** and the second argument is **true**.

### 7.5.2.8 Arithmetic, bitwise, and comparison operators

Operations specified in Table 4 are permitted. The following applies:

- **n** represents an object of type **sc\_int\_base**.
- **i** represents an object of integer type **int\_type**.

The arguments of the comparison operators may also be of any other class that is derived from **sc\_int\_base**.

**Table 4—sc\_int\_base arithmetic, bitwise, and comparison operations**

| Expression           | Return type             | Operation                                      |
|----------------------|-------------------------|------------------------------------------------|
| <b>n += i</b>        | <b>sc_int_base&amp;</b> | <b>sc_int_base</b> assign sum                  |
| <b>n -= i</b>        | <b>sc_int_base&amp;</b> | <b>sc_int_base</b> assign difference           |
| <b>n *= i</b>        | <b>sc_int_base&amp;</b> | <b>sc_int_base</b> assign product              |
| <b>n /= i</b>        | <b>sc_int_base&amp;</b> | <b>sc_int_base</b> assign quotient             |
| <b>n %= i</b>        | <b>sc_int_base&amp;</b> | <b>sc_int_base</b> assign remainder            |
| <b>n &amp;= i</b>    | <b>sc_int_base&amp;</b> | <b>sc_int_base</b> assign bitwise and          |
| <b>n  = i</b>        | <b>sc_int_base&amp;</b> | <b>sc_int_base</b> assign bitwise or           |
| <b>n ^= i</b>        | <b>sc_int_base&amp;</b> | <b>sc_int_base</b> assign bitwise exclusive or |
| <b>n &lt;&lt;= i</b> | <b>sc_int_base&amp;</b> | <b>sc_int_base</b> assign left-shift           |
| <b>n &gt;&gt;= i</b> | <b>sc_int_base&amp;</b> | <b>sc_int_base</b> assign right-shift          |
| <b>n == n</b>        | <b>bool</b>             | test equal                                     |
| <b>n != n</b>        | <b>bool</b>             | test not equal                                 |
| <b>n &lt; n</b>      | <b>bool</b>             | test less than                                 |
| <b>n &lt;= n</b>     | <b>bool</b>             | test less than or equal                        |
| <b>n &gt; n</b>      | <b>bool</b>             | test greater than                              |
| <b>n &gt;= n</b>     | <b>bool</b>             | test greater than or equal                     |



Arithmetic and bitwise operations permitted for C++ integer types shall be permitted for **sc\_int\_base** objects using implicit type conversions. The return type of these operations is an implementation-dependent C++ integer type.

NOTE—An implementation is required to supply overloaded operators on **sc\_int\_base** objects to satisfy the requirements of this subclause. It is unspecified whether these operators are members of **sc\_int\_base**, global operators, or provided in some other way.

#### 7.5.2.9 Other member functions

void **scan**( std::istream& is = std::cin );

Member function **scan** shall set the value by reading the next formatted character string from the specified input stream (see 7.2.10).

void **print**( std::ostream& os = std::cout ) const;

Member function **print** shall write the value as a formatted character string to the specified output stream (see 7.2.10).

int **length**() const;

Member function **length** shall return the word length (see 7.2.4).

### 7.5.3 sc\_uint\_base

#### 7.5.3.1 Description

Class **sc\_uint\_base** represents a limited word-length unsigned integer. The word length shall be specified by a constructor argument or, by default, by the **sc\_length\_context** object currently in scope. The word length of an **sc\_uint\_base** object shall be fixed during instantiation and shall not subsequently be changed.

The integer value shall be held in an implementation-dependent native C++ unsigned integer type. A minimum representation size of 64 bits is required.

**sc\_uint\_base** is the base class for the **sc\_uint** class template.

#### 7.5.3.2 Class definition

```
namespace sc_dt {

class sc_uint_base
: public sc_value_base†
{
    friend class sc_uint_bitrefr†;
    friend class sc_uint_bitrefr†;
    friend class sc_uint_subrefr†;
    friend class sc_uint_subrefr†;

public:
    // Constructors
    explicit sc_uint_base( int w = sc_length_param().len() );
    sc_uint_base( uint_type v , int w );
    sc_uint_base( const sc_uint_base& a );
    explicit sc_uint_base( const sc_uint_subrefr†& a );

    template <class T>
    explicit sc_uint_base( const sc_generic_base<T>& a );
    explicit sc_uint_base( const sc_bv_base& v );
    explicit sc_uint_base( const sc_lv_base& v );
    explicit sc_uint_base( const sc_int_subrefr†& v );
    explicit sc_uint_base( const sc_signed_subrefr†& v );
    explicit sc_uint_base( const sc_unsigned_subrefr†& v );
    explicit sc_uint_base( const sc_signed& a );
    explicit sc_uint_base( const sc_unsigned& a );

    // Destructor
    ~sc_uint_base();

    // Assignment operators
    sc_uint_base& operator= ( uint_type v );
    sc_uint_base& operator= ( const sc_uint_base& a );
    sc_uint_base& operator= ( const sc_uint_subrefr†& a );
    template <class T>
    sc_uint_base& operator= ( const sc_generic_base<T>& a );
    sc_uint_base& operator= ( const sc_signed& a );
    sc_uint_base& operator= ( const sc_unsigned& a );
    sc_uint_base& operator= ( const sc_fxval& a );
```

```

sc_uint_base& operator= ( const sc_fxval_fast& a );
sc_uint_base& operator= ( const sc_fxnum& a );
sc_uint_base& operator= ( const sc_fxnum_fast& a );
sc_uint_base& operator= ( const sc_bv_base& a );
sc_uint_base& operator= ( const sc_lv_base& a );
sc_uint_base& operator= ( const char* a );
sc_uint_base& operator= ( unsigned long a );
sc_uint_base& operator= ( long a );
sc_uint_base& operator= ( unsigned int a );
sc_uint_base& operator= ( int a );
sc_uint_base& operator= ( int64 a );
sc_uint_base& operator= ( double a );

// Prefix and postfix increment and decrement operators
sc_uint_base& operator++ ();           // Prefix
const sc_uint_base operator++ ( int ); // Postfix
sc_uint_base& operator-- ();          // Prefix
const sc_uint_base operator-- ( int ); // Postfix

// Bit selection
sc_uint_bitref† operator[] ( int i );
sc_uint_bitref_r† operator[] ( int i ) const;

// Part selection
sc_uint_subref† operator() ( int left, int right );
sc_uint_subref_r† operator() ( int left, int right ) const;
sc_uint_subref† range( int left, int right );
sc_uint_subref_r† range( int left, int right ) const;

// Capacity
int length() const;

// Reduce methods
bool and_reduce() const;
bool nand_reduce() const;
bool or_reduce() const;
bool nor_reduce() const;
bool xor_reduce() const;
bool xnor_reduce() const;

// Implicit conversion to uint_type
operator uint_type() const;

// Explicit conversions
int to_int() const;
unsigned int to_uint() const;
long to_long() const;
unsigned long to_ulong() const;
int64 to_int64() const;
uint64 to_uint64() const;
double to_double() const;

// Explicit conversion to character string
const std::string to_string( sc_numrep numrep = SC_DEC ) const;

```

```

const std::string to_string( sc_numrep numrep , bool w_prefix ) const;

// Other methods
void print( std::ostream& os = std::cout ) const;
void scan( std::istream& is = std::cin );
};

} // namespace sc_dt

```

### 7.5.3.3 Constraints on usage

The word length of an **sc\_uint\_base** object shall not be greater than the maximum size of the unsigned integer representation used to hold its value.

### 7.5.3.4 Constructors

```
explicit sc_uint_base( int w = sc_length_param().len() );
```

Constructor **sc\_uint\_base** shall create an object of word length specified by **w**. This is the default constructor when **w** is not specified (in which case its value is set by the current length context). The initial value of the object shall be **0**.

```
sc_uint_base( uint_type v , int w );
```

Constructor **sc\_uint\_base** shall create an object of word length specified by **w** with initial value specified by **v**. Truncation of most significant bits shall occur if the value cannot be represented in the specified word length.

```
template< class T >
```

```
sc_uint_base( const sc_generic_base<T>& a );
```

Constructor **sc\_uint\_base** shall create an **sc\_uint\_base** object with a word length matching the constructor argument. The constructor shall set the initial value of the object to the value returned from the member function **to\_uint64** of the constructor argument.

The other constructors shall create an **sc\_uint\_base** object whose size and value matches that of the argument. The size of the argument shall not be greater than the maximum word length of an **sc\_uint\_base** object.

### 7.5.3.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to **sc\_uint\_base**, using truncation or sign-extension as described in 7.2.1.

### 7.5.3.6 Implicit type conversion

```
operator uint_type() const;
```

Operator **uint\_type** can be used for implicit type conversion from **sc\_uint\_base** to the native C++ unsigned integer representation.

NOTE 1—This operator enables the use of standard C++ bitwise logical and arithmetic operators with **sc\_uint\_base** objects.

NOTE 2—This operator is used by the C++ output stream operator and by the member functions of other data type classes that are not explicitly overload for **sc\_uint\_base**.

### 7.5.3.7 Explicit type conversion

```
const std::string to_string( sc_numrep numrep = SC_DEC ) const;
const std::string to_string( sc_numrep numrep , bool w_prefix ) const;
```

Member function **to\_string** shall perform the conversion to an **std::string**, as described in 7.2.11. Calling the **to\_string** function with a single argument is equivalent to calling the **to\_string** function with two arguments, where the second argument is **true**. Calling the **to\_string** function with no arguments is equivalent to calling the **to\_string** function with two arguments, where the first argument is **SC\_DEC** and the second argument is **true**.

### 7.5.3.8 Arithmetic, bitwise, and comparison operators

Operations specified in Table 5 are permitted. The following applies:

- **U** represents an object of type **sc\_uint\_base**.
- **u** represents an object of integer type **uint\_type**.

The arguments of the comparison operators may also be of any other class that is derived from **sc\_uint\_base**.

**Table 5—sc\_uint\_base arithmetic, bitwise, and comparison operations**

| Expression           | Return type              | Operation                                       |
|----------------------|--------------------------|-------------------------------------------------|
| <b>U += u</b>        | <b>sc_uint_base&amp;</b> | <b>sc_uint_base</b> assign sum                  |
| <b>U -= u</b>        | <b>sc_uint_base&amp;</b> | <b>sc_uint_base</b> assign difference           |
| <b>U *= u</b>        | <b>sc_uint_base&amp;</b> | <b>sc_uint_base</b> assign product              |
| <b>U /= u</b>        | <b>sc_uint_base&amp;</b> | <b>sc_uint_base</b> assign quotient             |
| <b>U %= u</b>        | <b>sc_uint_base&amp;</b> | <b>sc_uint_base</b> assign remainder            |
| <b>U &amp;= u</b>    | <b>sc_uint_base&amp;</b> | <b>sc_uint_base</b> assign bitwise and          |
| <b>U  = u</b>        | <b>sc_uint_base&amp;</b> | <b>sc_uint_base</b> assign bitwise or           |
| <b>U ^= u</b>        | <b>sc_uint_base&amp;</b> | <b>sc_uint_base</b> assign bitwise exclusive or |
| <b>U &lt;&lt;= u</b> | <b>sc_uint_base&amp;</b> | <b>sc_uint_base</b> assign left-shift           |
| <b>U &gt;&gt;= u</b> | <b>sc_uint_base&amp;</b> | <b>sc_uint_base</b> assign right-shift          |
| <b>U == U</b>        | <b>bool</b>              | test equal                                      |
| <b>U != U</b>        | <b>bool</b>              | test not equal                                  |
| <b>U &lt; U</b>      | <b>bool</b>              | test less than                                  |
| <b>U &lt;= U</b>     | <b>bool</b>              | test less than or equal                         |
| <b>U &gt; U</b>      | <b>bool</b>              | test greater than                               |
| <b>U &gt;= U</b>     | <b>bool</b>              | test greater than or equal                      |

Arithmetic and bitwise operations permitted for C++ integer types shall be permitted for **sc\_uint\_base** objects using implicit type conversions. The return type of these operations is an implementation-dependent C++ integer type.

NOTE—An implementation is required to supply overloaded operators on **sc\_uint\_base** objects to satisfy the requirements of this subclause. It is unspecified whether these operators are members of **sc\_uint\_base**, global operators, or provided in some other way.

### 7.5.3.9 Other member functions

void **scan**( std::istream& is = std::cin );

Member function **scan** shall set the value by reading the next formatted character string from the specified input stream (see 7.2.10).

void **print**( std::ostream& os = std::cout ) const;

Member function **print** shall write the value as a formatted character string to the specified output stream (see 7.2.10).

int **length**() const;

Member function **length** shall return the word length (see 7.2.4).

## 7.5.4 sc\_int

### 7.5.4.1 Description

Class template **sc\_int** represents a limited word-length signed integer. The word length shall be specified by a template argument.

Any public member functions of the base class **sc\_int\_base** that are overridden in class **sc\_int** shall have the same behavior in the two classes. Any public member functions of the base class not overridden in this way shall be publicly inherited by class **sc\_int**.

### 7.5.4.2 Class definition

```
namespace sc_dt {

template <int W>
class sc_int
: public sc_int_base
{
public:
    // Constructors
    sc_int();
    sc_int( int_type v );
    sc_int( const sc_int<W>& a );
    sc_int( const sc_int_base& a );
    sc_int( const sc_int_subref_r7& a );

    template <class T>
    sc_int( const sc_generic_base<T>& a );
    sc_int( const sc_signed& a );
    sc_int( const sc_unsigned& a );
    explicit sc_int( const sc_fxval& a );
```

```

explicit sc_int( const sc_fxval_fast& a );
explicit sc_int( const sc_fxnum& a );
explicit sc_int( const sc_fxnum_fast& a );
sc_int( const sc_bv_base& a );
sc_int( const sc_lv_base& a );
sc_int( const char* a );
sc_int( unsigned long a );
sc_int( long a );
sc_int( unsigned int a );
sc_int( int a );
sc_int( uint64 a );
sc_int( double a );

// Assignment operators
sc_int<W>& operator= ( int_type v );
sc_int<W>& operator= ( const sc_int_base& a );
sc_int<W>& operator= ( const sc_int_subref_r†& a );
sc_int<W>& operator= ( const sc_int<W>& a );
template <class T>
sc_int<W>& operator= ( const sc_generic_base<T>& a );
sc_int<W>& operator= ( const sc_signed& a );
sc_int<W>& operator= ( const sc_unsigned& a );
sc_int<W>& operator= ( const sc_fxval& a );
sc_int<W>& operator= ( const sc_fxval_fast& a );
sc_int<W>& operator= ( const sc_fxnum& a );
sc_int<W>& operator= ( const sc_fxnum_fast& a );
sc_int<W>& operator= ( const sc_bv_base& a );
sc_int<W>& operator= ( const sc_lv_base& a );
sc_int<W>& operator= ( const char* a );
sc_int<W>& operator= ( unsigned long a );
sc_int<W>& operator= ( long a );
sc_int<W>& operator= ( unsigned int a );
sc_int<W>& operator= ( int a );
sc_int<W>& operator= ( uint64 a );
sc_int<W>& operator= ( double a );

// Prefix and postfix increment and decrement operators
sc_int<W>& operator++ (); // Prefix
const sc_int<W> operator++ ( int ); // Postfix
sc_int<W>& operator-- (); // Prefix
const sc_int<W> operator-- ( int ); // Postfix
};

} // namespace sc_dt

```

### 7.5.4.3 Constraints on usage

The word length of an **sc\_int** object shall not be greater than the maximum word length of an **sc\_int\_base**.

#### 7.5.4.4 Constructors

**sc\_int**();

Default constructor **sc\_int** shall create an **sc\_int** object of word length specified by the template argument **W**. The initial value of the object shall be **0**.

template< class T >

**sc\_int**( const sc\_generic\_base<T>& a );

Constructor **sc\_int** shall create an **sc\_int** object of word length specified by the template argument. The constructor shall set the initial value of the object to the value returned from the member function **to\_int64** of the constructor argument.

The other constructors shall create an **sc\_int** object of word length specified by the template argument **W** and value corresponding to the integer magnitude of the constructor argument. If the word length of the specified initial value differs from the template argument, truncation or sign-extension shall be used as described in 7.2.1.

#### 7.5.4.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to **sc\_int**, using truncation or sign-extension as described in 7.2.1.

#### 7.5.4.6 Arithmetic and bitwise operators

Operations specified in Table 6 are permitted. The following applies:

- **n** represents an object of type **sc\_int**.
- **i** represents an object of integer type **int\_type**.

**Table 6—sc\_int arithmetic and bitwise operations**

| Expression            | Return type | Operation                          |
|-----------------------|-------------|------------------------------------|
| <b>n</b> += <b>i</b>  | sc_int<W>&  | sc_int assign sum                  |
| <b>n</b> -= <b>i</b>  | sc_int<W>&  | sc_int assign difference           |
| <b>n</b> *= <b>i</b>  | sc_int<W>&  | sc_int assign product              |
| <b>n</b> /= <b>i</b>  | sc_int<W>&  | sc_int assign quotient             |
| <b>n</b> %= <b>i</b>  | sc_int<W>&  | sc_int assign remainder            |
| <b>n</b> &= <b>i</b>  | sc_int<W>&  | sc_int assign bitwise and          |
| <b>n</b>  = <b>i</b>  | sc_int<W>&  | sc_int assign bitwise or           |
| <b>n</b> ^= <b>i</b>  | sc_int<W>&  | sc_int assign bitwise exclusive or |
| <b>n</b> <<= <b>i</b> | sc_int<W>&  | sc_int assign left-shift           |
| <b>n</b> >>= <b>i</b> | sc_int<W>&  | sc_int assign right-shift          |



Arithmetic and bitwise operations permitted for C++ integer types shall be permitted for **sc\_int** objects using implicit type conversions. The return type of these operations is an implementation-dependent C++ integer type.

NOTE—An implementation is required to supply overloaded operators on **sc\_int** objects to satisfy the requirements of this subclause. It is unspecified whether these operators are members of **sc\_int**, global operators, or provided in some other way.

## 7.5.5 sc\_uint

### 7.5.5.1 Description

Class template **sc\_uint** represents a limited word-length unsigned integer. The word length shall be specified by a template argument. Any public member functions of the base class **sc\_uint\_base** that are overridden in class **sc\_uint** shall have the same behavior in the two classes. Any public member functions of the base class not overridden in this way shall be publicly inherited by class **sc\_uint**.

### 7.5.5.2 Class definition

```
namespace sc_dt {  
  
template <int W>  
class sc_uint  
: public sc_uint_base  
{  
    public:  
        // Constructors  
  
        sc_uint();  
        sc_uint( uint_type v );  
        sc_uint( const sc_uint<W>& a );  
        sc_uint( const sc_uint_base& a );  
        sc_uint( const sc_uint_subref_r†& a );  
        template <class T>  
        sc_uint( const sc_generic_base<T>& a );  
        sc_uint( const sc_signed& a );  
        sc_uint( const sc_unsigned& a );  
        explicit sc_uint( const sc_fxval& a );  
        explicit sc_uint( const sc_fxval_fast& a );  
        explicit sc_uint( const sc_fxnum& a );  
        explicit sc_uint( const sc_fxnum_fast& a );  
        sc_uint( const sc_bv_base& a );  
        sc_uint( const sc_lv_base& a );  
        sc_uint( const char* a );  
        sc_uint( unsigned long a );  
        sc_uint( long a );  
        sc_uint( unsigned int a );  
        sc_uint( int a );  
        sc_uint( int64 a );  
        sc_uint( double a );  
  
        // Assignment operators  
        sc_uint<W>& operator= ( uint_type v );  
        sc_uint<W>& operator= ( const sc_uint_base& a );
```

```

sc_uint<W>& operator= ( const sc_uint_subref_r†& a );
sc_uint<W>& operator= ( const sc_uint<W>& a );
template <class T>
sc_uint<W>& operator= ( const sc_generic_base<T>& a );
sc_uint<W>& operator= ( const sc_signed& a );
sc_uint<W>& operator= ( const sc_unsigned& a );
sc_uint<W>& operator= ( const sc_fxval& a );
sc_uint<W>& operator= ( const sc_fxval_fast& a );
sc_uint<W>& operator= ( const sc_fxnum& a );
sc_uint<W>& operator= ( const sc_fxnum_fast& a );
sc_uint<W>& operator= ( const sc_bv_base& a );
sc_uint<W>& operator= ( const sc_lv_base& a );
sc_uint<W>& operator= ( const char* a );
sc_uint<W>& operator= ( unsigned long a );
sc_uint<W>& operator= ( long a );
sc_uint<W>& operator= ( unsigned int a );
sc_uint<W>& operator= ( int a );
sc_uint<W>& operator= ( int64 a );
sc_uint<W>& operator= ( double a );

// Prefix and postfix increment and decrement operators
sc_uint<W>& operator++ ();           // Prefix
const sc_uint<W> operator++ ( int ); // Postfix
sc_uint<W>& operator-- ();          // Prefix
const sc_uint<W> operator-- ( int ); // Postfix
};

} // namespace sc_dt

```

### 7.5.5.3 Constraints on usage

The word length of an **sc\_uint** object shall not be greater than the maximum word length of an **sc\_uint\_base**.

### 7.5.5.4 Constructors

**sc\_uint**();

Default constructor **sc\_uint** shall create an **sc\_uint** object of word length specified by the template argument **W**. The initial value of the object shall be **0**.

template< class T >

**sc\_uint**( const sc\_generic\_base<T>& a );

Constructor **sc\_uint** shall create an **sc\_uint** object of word length specified by the template argument. The constructor shall set the initial value of the object to the value returned from the member function **to\_uint64** of the constructor argument.

The other constructors shall create an **sc\_uint** object of word length specified by the template argument **W** and value corresponding to the integer magnitude of the constructor argument. If the word length of the specified initial value differs from the template argument, truncation or sign-extension shall be used as described in 7.2.1.

### 7.5.5.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to **sc\_uint**. If the size of a data type or string literal operand differs from the **sc\_uint** word length, truncation or sign-extension shall be used as described in 7.2.1.

### 7.5.5.6 Arithmetic and bitwise operators

Operations specified in Table 7 are permitted. The following applies:

- **U** represents an object of type **sc\_uint**.
- **u** represents an object of integer type **uint\_type**.

**Table 7—sc\_uint arithmetic and bitwise operations**

| Expression           | Return type                  | Operation                                  |
|----------------------|------------------------------|--------------------------------------------|
| <b>U += u</b>        | <b>sc_uint&lt;W&gt;&amp;</b> | <b>sc_uint</b> assign sum                  |
| <b>U -= u</b>        | <b>sc_uint&lt;W&gt;&amp;</b> | <b>sc_uint</b> assign difference           |
| <b>U *= u</b>        | <b>sc_uint&lt;W&gt;&amp;</b> | <b>sc_uint</b> assign product              |
| <b>U /= u</b>        | <b>sc_uint&lt;W&gt;&amp;</b> | <b>sc_uint</b> assign quotient             |
| <b>U %= u</b>        | <b>sc_uint&lt;W&gt;&amp;</b> | <b>sc_uint</b> assign remainder            |
| <b>U &amp;= u</b>    | <b>sc_uint&lt;W&gt;&amp;</b> | <b>sc_uint</b> assign bitwise and          |
| <b>U  = u</b>        | <b>sc_uint&lt;W&gt;&amp;</b> | <b>sc_uint</b> assign bitwise or           |
| <b>U ^= u</b>        | <b>sc_uint&lt;W&gt;&amp;</b> | <b>sc_uint</b> assign bitwise exclusive or |
| <b>U &lt;&lt;= u</b> | <b>sc_uint&lt;W&gt;&amp;</b> | <b>sc_uint</b> assign left-shift           |
| <b>U &gt;&gt;= u</b> | <b>sc_uint&lt;W&gt;&amp;</b> | <b>sc_uint</b> assign right-shift          |

Arithmetic and bitwise operations permitted for C++ integer types shall be permitted for **sc\_uint** objects using implicit type conversions. The return type of these operations is an implementation-dependent C++ integer.

NOTE—An implementation is required to supply overloaded operators on **sc\_uint** objects to satisfy the requirements of this subclause. It is unspecified whether these operators are members of **sc\_uint**, global operators, or provided in some other way.

## 7.5.6 Bit-selects

### 7.5.6.1 Description

Class *sc\_int\_bitref<sub>r</sub>*<sup>†</sup> represents a bit selected from an **sc\_int\_base** used as an rvalue.

Class *sc\_int\_bitref<sub>l</sub>*<sup>†</sup> represents a bit selected from an **sc\_int\_base** used as an lvalue.

Class *sc\_uint\_bitref<sub>r</sub>*<sup>†</sup> represents a bit selected from an **sc\_uint\_base** used as an rvalue.

Class *sc\_uint\_bitref<sup>†</sup>* represents a bit selected from an **sc\_uint\_base** used as an lvalue.

### 7.5.6.2 Class definition

```
namespace sc_dt {

class sc_int_bitref†
: public sc_value_base†
{
    friend class sc_int_base;

public:
    // Copy constructor
    sc_int_bitref†( const sc_int_bitref†& a );

    // Destructor
    virtual ~sc_int_bitref†();

    // Capacity
    int length() const;

    // Implicit conversion to uint64
    operator uint64 () const;
    bool operator! () const;
    bool operator~ () const;

    // Explicit conversions
    bool to_bool() const;

    // Other methods
    void print( std::ostream& os = std::cout ) const;

protected:
    sc_int_bitref†();

private:
    // Disabled
    sc_int_bitref†& operator= ( const sc_int_bitref†& );
};

// -----

class sc_int_bitref†
: public sc_int_bitref†
{
    friend class sc_int_base;

public:
    // Copy constructor
    sc_int_bitref†( const sc_int_bitref†& a );

    // Assignment operators
    sc_int_bitref†& operator= ( const sc_int_bitref†& b );
```

```
    sc_int_bitref† & operator= ( const sc_int_bitref† & b );
    sc_int_bitref† & operator= ( bool b );
    sc_int_bitref† & operator&= ( bool b );
    sc_int_bitref† & operator|= ( bool b );
    sc_int_bitref† & operator^= ( bool b );

    // Other methods
    void scan( std::istream& is = std::cin );

private:
    sc_int_bitref†();
};

// -----

class sc_uint_bitref_r†
: public sc_value_base†
{
    friend class sc_uint_base;

public:
    // Copy constructor
    sc_uint_bitref_r†( const sc_uint_bitref_r† & a );

    // Destructor
    virtual ~sc_uint_bitref_r†();

    // Capacity
    int length() const;

    // Implicit conversion to uint64
    operator uint64 () const;
    bool operator! () const;
    bool operator~ () const;

    // Explicit conversions
    bool to_bool() const;

    // Other methods
    void print( std::ostream& os = std::cout ) const;

protected:
    sc_uint_bitref_r†();

private:
    // Disabled
    sc_uint_bitref_r† & operator= ( const sc_uint_bitref_r† & );
};

// -----

class sc_uint_bitref†
: public sc_uint_bitref_r†
{
```

```

friend class sc_uint_base;

public:
    // Copy constructor
    sc_uint_bitref†( const sc_uint_bitref†& a );

    // Assignment operators
    sc_uint_bitref†& operator= ( const sc_uint_bitref† r†& b );
    sc_uint_bitref†& operator= ( const sc_uint_bitref†& b );
    sc_uint_bitref†& operator= ( bool b );
    sc_uint_bitref†& operator&= ( bool b );
    sc_uint_bitref†& operator|= ( bool b );
    sc_uint_bitref†& operator^= ( bool b );

    // Other methods
    void scan( std::istream& is = std::cin );

private:
    sc_uint_bitref†();
};

} // namespace sc_dt

```

### 7.5.6.3 Constraints on usage

Bit-select objects shall only be created using the bit-select operators of an **sc\_int\_base** or **sc\_uint\_base** object (or an instance of a class derived from **sc\_int\_base** or **sc\_uint\_base**).

An application shall not explicitly create an instance of any bit-select class.

An application should not declare a reference or pointer to any bit-select object.

It is strongly recommended that an application avoid the use of a bit-select as the return type of a function because the lifetime of the object to which the bit-select refers may not extend beyond the function return statement.

*Example:*

```

sc_dt::sc_int_bitref get_bit_n(sc_int_base i, int n) {
    return i[n]; // Unsafe: returned bit-select references local variable
}

```

### 7.5.6.4 Assignment operators

Overloaded assignment operators for the lvalue bit-selects shall provide conversion from **bool** values. Assignment operators for rvalue bit-selects shall be declared as private to prevent their use by an application.

### 7.5.6.5 Implicit type conversion

**operator uint64()** const;

Operator **uint64** can be used for implicit type conversion from a bit-select to the native C++ unsigned integer having exactly 64 bits. If the selected bit has the value '1' (true), the conversion shall return the value 1; otherwise, it shall return 0.

bool **operator!** () const;

bool **operator~** () const;

**operator!** and **operator~** shall return a C++ **bool** value that is the inverse of the selected bit.

### 7.5.6.6 Other member functions

void **scan**( std::istream& is = std::cin );

Member function **scan** shall set the value of the bit referenced by an lvalue bit-select. The value shall correspond to the C++ bool value obtained by reading the next formatted character string from the specified input stream (see 7.2.10).

void **print**( std::ostream& os = std::cout ) const;

Member function **print** shall print the value of the bit referenced by the bit-select to the specified output stream (see 7.2.10). The formatting shall be implementation-defined but shall be equivalent to printing the value returned by member function **to\_bool**.

int **length**() const;

Member function **length** shall unconditionally return a word length of 1 (see 7.2.4).

## 7.5.7 Part-selects

### 7.5.7.1 Description

Class *sc\_int\_subref<sub>r</sub><sup>†</sup>* represents a signed integer part-select from an **sc\_int\_base** used as an rvalue.

Class *sc\_int\_subref<sup>†</sup>* represents a signed integer part-select from an **sc\_int\_base** used as an lvalue.

Class *sc\_uint\_subref<sub>r</sub><sup>†</sup>* represents an unsigned integer part-select from an **sc\_uint\_base** used as an rvalue.

Class *sc\_uint\_subref<sup>†</sup>* represents an unsigned integer part-select from an **sc\_uint\_base** used as an lvalue.

### 7.5.7.2 Class definition

```
namespace sc_dt {
```

```
class sc_int_subrefr†
{
    friend class sc_int_base;
    friend class sc_int_subref†;

public:
    // Copy constructor
    sc_int_subrefr†( const sc_int_subrefr†& a );
```

```

// Destructor
virtual ~sc_int_subref_r†();

// Capacity
int length() const;

// Reduce methods
bool and_reduce() const;
bool nand_reduce() const;
bool or_reduce() const;
bool nor_reduce() const;
bool xor_reduce() const;
bool xnor_reduce() const;

// Implicit conversion to uint_type
operator uint_type() const;

// Explicit conversions
int to_int() const;
unsigned int to_uint() const;
long to_long() const;
unsigned long to_ulong() const;
int64 to_int64() const;
uint64 to_uint64() const;
double to_double() const;

// Explicit conversion to character string
const std::string to_string( sc_numrep numrep = SC_DEC ) const;
const std::string to_string( sc_numrep numrep , bool w_prefix ) const;

// Other methods
void print( std::ostream& os = std::cout ) const;

protected:
    sc_int_subref_r†();

private:
    // Disabled
    sc_int_subref_r†& operator=( const sc_int_subref_r†& );
};

// -----

class sc_int_subref†
: public sc_int_subref_r†
{
    friend class sc_int_base;

public:
    // Copy constructor
    sc_int_subref†( const sc_int_subref†& a );

    // Assignment operators
    sc_int_subref†& operator=( int_type v );

```



```
    sc_int_subref†& operator= ( const sc_int_base& a );  
    sc_int_subref†& operator= ( const sc_int_subref_r†& a );  
    sc_int_subref†& operator= ( const sc_int_subref†& a );  
    template< class T >  
    sc_int_subref†& operator= ( const sc_generic_base<T>& a );  
    sc_int_subref†& operator= ( const char* a );  
    sc_int_subref†& operator= ( unsigned long a );  
    sc_int_subref†& operator= ( long a );  
    sc_int_subref†& operator= ( unsigned int a );  
    sc_int_subref†& operator= ( int a );  
    sc_int_subref†& operator= ( uint64 a );  
    sc_int_subref†& operator= ( double a );  
    sc_int_subref†& operator= ( const sc_signed& );  
    sc_int_subref†& operator= ( const sc_unsigned& );  
    sc_int_subref†& operator= ( const sc_bv_base& );  
    sc_int_subref†& operator= ( const sc_lv_base& );  
  
    // Other methods  
    void scan( std::istream& is = std::cin );  
  
protected:  
    sc_int_subref†();  
  
};  
  
// -----  
  
class sc_uint_subref_r†  
{  
    friend class sc_uint_base;  
    friend class sc_uint_subref†;  
  
public:  
    // Copy constructor  
    sc_uint_subref_r†( const sc_uint_subref_r†& a );  
  
    // Destructor  
    virtual ~sc_uint_subref_r();  
  
    // Capacity  
    int length() const;  
  
    // Reduce methods  
    bool and_reduce() const;  
    bool nand_reduce() const;  
    bool or_reduce() const;  
    bool nor_reduce() const;  
    bool xor_reduce() const;  
    bool xnor_reduce() const;  
  
    // Implicit conversion to uint_type  
    operator uint_type() const;
```

```

// Explicit conversions
int to_int() const;
unsigned int to_uint() const;
long to_long() const;
unsigned long to_ulong() const;
int64 to_int64() const;
uint64 to_uint64() const;
double to_double() const;

// Explicit conversion to character string
const std::string to_string( sc_numrep numrep = SC_DEC ) const;
const std::string to_string( sc_numrep numrep , bool w_prefix ) const;

// Other methods
void print( std::ostream& os = std::cout ) const;

protected:
    sc_uint_subref_r†();

private:
    // Disabled
    sc_uint_subref_r& operator=( const sc_uint_subref_r& );
};

// -----

class sc_uint_subref†
: public sc_uint_subref_r†
{
    friend class sc_uint_base;

public:
    // Copy constructor
    sc_uint_subref†( const sc_uint_subref†& a );

    // Assignment operators
    sc_uint_subref†& operator=( uint_type v );
    sc_uint_subref†& operator=( const sc_uint_base& a );
    sc_uint_subref†& operator=( const sc_uint_subref_r& a );
    sc_uint_subref†& operator=( const sc_uint_subref& a );
    template<class T>
    sc_uint_subref†& operator=( const sc_generic_base<T>& a );
    sc_uint_subref†& operator=( const char* a );
    sc_uint_subref†& operator=( unsigned long a );
    sc_uint_subref†& operator=( long a );
    sc_uint_subref†& operator=( unsigned int a );
    sc_uint_subref†& operator=( int a );
    sc_uint_subref†& operator=( int64 a );
    sc_uint_subref†& operator=( double a );
    sc_uint_subref†& operator=( const sc_signed& );
    sc_uint_subref†& operator=( const sc_unsigned& );
    sc_uint_subref†& operator=( const sc_bv_base& );
    sc_uint_subref†& operator=( const sc_lv_base& );

```

```

        // Other methods
        void scan( std::istream& is = std::cin );

    protected:
        sc_uint_subref†();
};

} // namespace sc_dt

```

### 7.5.7.3 Constraints on usage

Integer part-select objects shall only be created using the part-select operators of an **sc\_int\_base** or **sc\_uint\_base** object (or an instance of a class derived from **sc\_int\_base** or **sc\_uint\_base**), as described in 7.2.6.

An application shall not explicitly create an instance of any integer part-select class.

An application should not declare a reference or pointer to any integer part-select object.

It shall be an error if the left-hand index of a limited-precision integer part-select is less than the right-hand index.

It is strongly recommended that an application avoid the use of a part-select as the return type of a function because the lifetime of the object to which the part-select refers may not extend beyond the function return statement.

*Example:*

```

sc_dt::sc_int_subref get_byte(sc_int_base ib, int pos) {
    return ib(pos+7,pos);    // Unsafe: returned part-select references local variable
}

```

### 7.5.7.4 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to lvalue integer part-selects. If the size of a data type or string literal operand differs from the integer part-select word length, truncation, zero-extension, or sign-extension shall be used as described in 7.2.1.

Assignment operators for rvalue integer part-selects shall be declared as private to prevent their use by an application.

### 7.5.7.5 Implicit type conversion

```

sc_int_subref†::operator uint_type() const;
sc_uint_subref†::operator uint_type() const;

```

**operator int\_type** and **operator uint\_type** can be used for implicit type conversion from integer part-selects to the native C++ unsigned integer representation.

NOTE 1—These operators enable the use of standard C++ bitwise logical and arithmetic operators with integer part-select objects.

NOTE 2—These operators are used by the C++ output stream operator and by member functions of other data type classes that are not explicitly overload for integer part-selects.

### 7.5.7.6 Explicit type conversion

```
const std::string to_string( sc_numrep numrep = SC_DEC ) const;  
const std::string to_string( sc_numrep numrep , bool w_prefix ) const;
```

Member function **to\_string** shall perform the conversion to an **std::string**, as described in 7.2.11. Calling the **to\_string** function with a single argument is equivalent to calling the **to\_string** function with two arguments, where the second argument is **true**. Calling the **to\_string** function with no arguments is equivalent to calling the **to\_string** function with two arguments, where the first argument is SC\_DEC and the second argument is **true**.

### 7.5.7.7 Other member functions

```
void scan( std::istream& is = std::cin );
```

Member function **scan** shall set the values of the bits referenced by an lvalue part-select by reading the next formatted character string from the specified input stream (see 7.2.10).

```
void print( std::ostream& os = std::cout ) const;
```

Member function **print** shall print the values of the bits referenced by the part-select to the specified output stream (see 7.2.10).

```
int length() const;
```

Member function **length** shall return the word length of the part-select (see 7.2.4).

## 7.6 Finite-precision integer types

### 7.6.1 Type definitions

The following type definitions are used in the finite-precision integer type classes:

```
namespace sc_dt{  
  
typedef implementation-defined int64;  
typedef implementation-defined uint64;  
  
} // namespace sc_dt
```

**int64** is a native C++ integer type having a word length of exactly 64 bits.

**uint64** is a native C++ unsigned integer type having a word length of exactly 64 bits.

### 7.6.2 Constraints on usage

Overloaded arithmetic and comparison operators allow finite-precision integer objects to be used in expressions following similar but not identical rules to standard C++ integer types. The differences from the standard C++ integer operator behavior are the following:

- a) Where one operand is unsigned and the other is signed, the unsigned operand shall be converted to signed and the return type shall be signed.
- b) The return type of a subtraction shall always be signed.
- c) The word length of the return type of an arithmetic operator shall depend only on the nature of the operation and the word length of its operands.
- d) A floating-point variable or literal shall not be directly used as an operand. It should first be converted to an appropriate signed or unsigned integer type.

### 7.6.3 **sc\_signed**

#### 7.6.3.1 Description

Class **sc\_signed** represents a finite word-length integer. The word length shall be specified by a constructor argument or, by default, by the length context object currently in scope. The word length of an **sc\_signed** object shall be fixed during instantiation and shall not subsequently be changed.

The integer value shall be stored with a finite precision determined by the specified word length. The precision shall not depend on the limited resolution of any standard C++ integer type.

**sc\_signed** is the base class for the **sc\_bigint** class template.

**7.6.3.2 Class definition**

```

namespace sc_dt {

class sc_signed
: public sc_value_base†
{
    friend class sc_concatref†;
    friend class sc_signed_bitref†;
    friend class sc_signed_bitref‡;
    friend class sc_signed_subref†;
    friend class sc_signed_subref‡;
    friend class sc_unsigned;
    friend class sc_unsigned_subref;

public:
    // Constructors
    explicit sc_signed( int nb = sc_length_param().len() );
    sc_signed( const sc_signed& v );
    sc_signed( const sc_unsigned& v );
    template<class T>
    explicit sc_signed( const sc_generic_base<T>& v );
    explicit sc_signed( const sc_bv_base& v );
    explicit sc_signed( const sc_lv_base& v );
    explicit sc_signed( const sc_int_subref_r& v );
    explicit sc_signed( const sc_uint_subref_r& v );
    explicit sc_signed( const sc_signed_subref_r& v );
    explicit sc_signed( const sc_unsigned_subref_r& v );

    // Assignment operators
    sc_signed& operator= ( const sc_signed& v );
    sc_signed& operator= ( const sc_signed_subref_r†& a );
    template< class T >
    sc_signed& operator= ( const sc_generic_base<T>& a );
    sc_signed& operator= ( const sc_unsigned& v );
    sc_signed& operator= ( const sc_unsigned_subref_r†& a );
    sc_signed& operator= ( const char* v );
    sc_signed& operator= ( int64 v );
    sc_signed& operator= ( uint64 v );
    sc_signed& operator= ( long v );
    sc_signed& operator= ( unsigned long v );
    sc_signed& operator= ( int v );
    sc_signed& operator= ( unsigned int v );
    sc_signed& operator= ( double v );
    sc_signed& operator= ( const sc_int_base& v );
    sc_signed& operator= ( const sc_uint_base& v );
    sc_signed& operator= ( const sc_bv_base& );
    sc_signed& operator= ( const sc_lv_base& );
    sc_signed& operator= ( const sc_fxval& );
    sc_signed& operator= ( const sc_fxval_fast& );
    sc_signed& operator= ( const sc_fxnum& );
    sc_signed& operator= ( const sc_fxnum_fast& );

```

```
// Destructor
~sc_signed();

// Increment operators.
sc_signed& operator++ ();
const sc_signed operator++ ( int );

// Decrement operators.
sc_signed& operator-- ();
const sc_signed operator-- ( int );

// Bit selection
sc_signed_bitref† operator[] ( int i );
sc_signed_bitref_r† operator[] ( int i ) const;

// Part selection
sc_signed_subref† range( int i , int j );
sc_signed_subref_r† range( int i , int j ) const;
sc_signed_subref† operator() ( int i , int j );
sc_signed_subref_r† operator() ( int i , int j ) const;

// Explicit conversions
int to_int() const;
unsigned int to_uint() const;
long to_long() const;
unsigned long to_ulong() const;
int64 to_int64() const;
uint64 to_uint64() const;
double to_double() const;

// Explicit conversion to character string
const std::string to_string( sc_numrep numrep = SC_DEC ) const;
const std::string to_string( sc_numrep numrep, bool w_prefix ) const;

// Print functions
void print( std::ostream& os = std::cout ) const;
void scan( std::istream& is = std::cin );

// Capacity
int length() const;

// Reduce methods
bool and_reduce() const;
bool nand_reduce() const;
bool or_reduce() const;
bool nor_reduce() const;
bool xor_reduce() const;
bool xnor_reduce() const;

// Overloaded operators

};

} // namespace sc_dt
```

### 7.6.3.3 Constraints on usage

An object of type **sc\_signed** shall not be used as a direct replacement for a C++ integer type, since no implicit type conversion member functions are provided. An explicit type conversion is required to pass the value of an **sc\_signed** object as an argument to a function expecting a C++ integer value argument.

### 7.6.3.4 Constructors

explicit **sc\_signed**( int nb = sc\_length\_param().len() );

Constructor **sc\_signed** shall create an **sc\_signed** object of word length specified by **nb**. This is the default constructor when **nb** is not specified (in which case its value is set by the current length context). The initial value of the object shall be **0**.

template< class T >

**sc\_signed**( const sc\_generic\_base<T>& a );

Constructor **sc\_signed** shall create an **sc\_signed** object with a word length matching the constructor argument. The constructor shall set the initial value of the object to the value returned from the member function **to\_sc\_signed** of the constructor argument.

The other constructors create an **sc\_signed** object with the same word length and value as the constructor argument.

### 7.6.3.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to **sc\_signed**, using truncation or sign-extension as described in 7.2.1.

### 7.6.3.6 Explicit type conversion

const std::string **to\_string**( sc\_numrep numrep = SC\_DEC ) const;

const std::string **to\_string**( sc\_numrep numrep, bool w\_prefix ) const;

Member function **to\_string** shall perform conversion to an **std::string** representation, as described in 7.2.11. Calling the **to\_string** function with a single argument is equivalent to calling the **to\_string** function with two arguments, where the second argument is **true**. Calling the **to\_string** function with no arguments is equivalent to calling the **to\_string** function with two arguments, where the first argument is SC\_DEC and the second argument is **true**.

### 7.6.3.7 Arithmetic, bitwise, and comparison operators

Operations specified in Table 8, Table 9, and Table 10 are permitted. The following applies:

- **S** represents an object of type **sc\_signed**.
- **U** represents an object of type **sc\_unsigned**.
- **i** represents an object of integer type **int**, **long**, **unsigned int**, **unsigned long**, **sc\_signed**, **sc\_unsigned**, **sc\_int\_base**, or **sc\_uint\_base**.
- **s** represents an object of signed integer type **int**, **long**, **sc\_signed**, or **sc\_int\_base**.



The operands may also be of any other class that is derived from those just given.

**Table 8—sc\_signed arithmetic operations**

| Expression | Return type | Operation                               |
|------------|-------------|-----------------------------------------|
| $S + i$    | sc_signed   | sc_signed addition                      |
| $i + S$    | sc_signed   | sc_signed addition                      |
| $U + s$    | sc_signed   | addition of sc_unsigned and signed      |
| $s + U$    | sc_signed   | addition of signed and sc_unsigned      |
| $S += i$   | sc_signed&  | sc_signed assign sum                    |
| $S - i$    | sc_signed   | sc_signed subtraction                   |
| $i - S$    | sc_signed   | sc_signed subtraction                   |
| $U - i$    | sc_signed   | sc_unsigned subtraction                 |
| $i - U$    | sc_signed   | sc_unsigned subtraction                 |
| $S -= i$   | sc_signed&  | sc_signed assign difference             |
| $S * i$    | sc_signed   | sc_signed multiplication                |
| $i * S$    | sc_signed   | sc_signed multiplication                |
| $U * s$    | sc_signed   | multiplication of sc_unsigned by signed |
| $s * U$    | sc_signed   | multiplication of signed by sc_unsigned |
| $S *= i$   | sc_signed&  | sc_signed assign product                |
| $S / i$    | sc_signed   | sc_signed division                      |
| $i / S$    | sc_signed   | sc_signed division                      |
| $U / s$    | sc_signed   | division of sc_unsigned by signed       |
| $s / U$    | sc_signed   | division of signed by sc_unsigned       |
| $S /= i$   | sc_signed&  | sc_signed assign quotient               |
| $S \% i$   | sc_signed   | sc_signed remainder                     |
| $i \% S$   | sc_signed   | sc_signed remainder                     |
| $U \% s$   | sc_signed   | remainder of sc_unsigned with signed    |
| $s \% U$   | sc_signed   | remainder of signed with sc_unsigned    |
| $S \% = i$ | sc_signed&  | sc_signed assign remainder              |
| $+S$       | sc_signed   | sc_signed unary plus                    |
| $-S$       | sc_signed   | sc_signed unary minus                   |
| $-U$       | sc_signed   | sc_unsigned unary minus                 |

If the result of any arithmetic operation is zero, the word length of the return value shall be set by the **sc\_length\_context** in scope. Otherwise, the following rules apply:

- Addition shall return a result with a word length that is equal to the word length of the longest operand plus one.
- Multiplication shall return a result with a word length that is equal to the sum of the word lengths of the two operands.
- Remainder shall return a result with a word length that is equal to the word length of the shortest operand.
- All other arithmetic operators shall return a result with a word length that is equal to the word length of the longest operand.

**Table 9—sc\_signed bitwise operations**

| Expression | Return type | Operation                               |
|------------|-------------|-----------------------------------------|
| S & i      | sc_signed   | sc_signed bitwise and                   |
| i & S      | sc_signed   | sc_signed bitwise and                   |
| U & s      | sc_signed   | sc_unsigned bitwise and signed          |
| s & U      | sc_signed   | signed bitwise and sc_unsigned          |
| S &= i     | sc_signed&  | sc_signed assign bitwise and            |
| S   i      | sc_signed   | sc_signed bitwise or                    |
| i   S      | sc_signed   | sc_signed bitwise or                    |
| U   s      | sc_signed   | sc_unsigned bitwise or signed           |
| s   U      | sc_signed   | signed bitwise or sc_unsigned           |
| S  = i     | sc_signed&  | sc_signed assign bitwise or             |
| S ^ i      | sc_signed   | sc_signed bitwise exclusive or          |
| i ^ S      | sc_signed   | sc_signed bitwise exclusive or          |
| U ^ s      | sc_signed   | sc_unsigned bitwise exclusive or signed |
| s ^ U      | sc_signed   | sc_unsigned bitwise exclusive or signed |
| S ^= i     | sc_signed&  | sc_signed assign bitwise exclusive or   |
| S << i     | sc_signed   | sc_signed left-shift                    |
| U << S     | sc_unsigned | sc_unsigned left-shift                  |
| S <<= i    | sc_signed&  | sc_signed assign left-shift             |
| S >> i     | sc_signed   | sc_signed right-shift                   |
| U >> S     | sc_unsigned | sc_unsigned right-shift                 |
| S >>= i    | sc_signed&  | sc_signed assign right-shift            |
| ~S         | sc_signed   | sc_signed bitwise complement            |

Binary bitwise operators shall return a result with a word length that is equal to the word length of the longest operand.

The left shift operator shall return a result with a word length that is equal to the word length of its **sc\_signed** operand plus the right (integer) operand. Bits added on the right-hand side of the result shall be set to zero.

The right shift operator shall return a result with a word length that is equal to the word length of its **sc\_signed** operand. Bits added on the left-hand side of the result shall be set to the same value as the left-hand bit of the **sc\_signed** operand (a right-shift preserves the sign).

The behavior of a shift operator is undefined if the right operand is negative.

**Table 10—sc\_signed comparison operations**

| Expression | Return type | Operation                  |
|------------|-------------|----------------------------|
| $S == i$   | bool        | test equal                 |
| $i == S$   | bool        | test equal                 |
| $S != i$   | bool        | test not equal             |
| $i != S$   | bool        | test not equal             |
| $S < i$    | bool        | test less than             |
| $i < S$    | bool        | test less than             |
| $S <= i$   | bool        | test less than or equal    |
| $i <= S$   | bool        | test less than or equal    |
| $S > i$    | bool        | test greater than          |
| $i > S$    | bool        | test greater than          |
| $S >= i$   | bool        | test greater than or equal |
| $i >= S$   | bool        | test greater than or equal |

NOTE—An implementation is required to supply overloaded operators on **sc\_signed** objects to satisfy the requirements of this subclause. It is unspecified whether these operators are members of **sc\_signed**, global operators, or provided in some other way.

#### 7.6.3.8 Other member functions

void **scan**( std::istream& is = std::cin );

Member function **scan** shall set the value by reading the next formatted character string from the specified input stream (see 7.2.10).

void **print**( std::ostream& os = std::cout ) const;

Member function **print** shall write the value as a formatted character string to the specified output stream (see 7.2.10).

int **length**() const;

Member function **length** shall return the word length (see 7.2.4).

## 7.6.4 **sc\_unsigned**

### 7.6.4.1 Description

Class **sc\_unsigned** represents a finite word-length unsigned integer. The word length shall be specified by a constructor argument or, by default, by the length context currently in scope. The word length of an **sc\_unsigned** object is fixed during instantiation and shall not be subsequently changed.

The integer value shall be stored with a finite precision determined by the specified word length. The precision shall not depend on the limited resolution of any standard C++ integer type.

**sc\_unsigned** is the base class for the **sc\_biguint** class template.

### 7.6.4.2 Class definition

```
namespace sc_dt {

class sc_unsigned
: public sc_value_base†
{
    friend class sc_concatref†;
    friend class sc_unsigned_bitref†r;
    friend class sc_unsigned_bitref†;
    friend class sc_unsigned_subref†r;
    friend class sc_unsigned_subref†;
    friend class sc_signed;
    friend class sc_signed_subref†;

public:
    // Constructors
    explicit sc_unsigned( int nb = sc_length_param().len() );
    sc_unsigned( const sc_unsigned& v );
    sc_unsigned( const sc_signed& v );
    template<class T>
    explicit sc_unsigned( const sc_generic_base<T>& v );
    explicit sc_unsigned( const sc_bv_base& v );
    explicit sc_unsigned( const sc_lv_base& v );
    explicit sc_unsigned( const sc_int_subref_r& v );
    explicit sc_unsigned( const sc_uint_subref_r& v );
    explicit sc_unsigned( const sc_signed_subref_r& v );
    explicit sc_unsigned( const sc_unsigned_subref_r& v );

    // Assignment operators
    sc_unsigned& operator= ( const sc_unsigned& v );
    sc_unsigned& operator= ( const sc_unsigned_subref_r†& a );
    template<class T>
    sc_unsigned& operator= ( const sc_generic_base<T>& a );
    sc_unsigned& operator= ( const sc_signed& v );
    sc_unsigned& operator= ( const sc_signed_subref_r†& a );
```

```
sc_unsigned& operator= ( const char* v );
sc_unsigned& operator= ( int64 v );
sc_unsigned& operator= ( uint64 v );
sc_unsigned& operator= ( long v );
sc_unsigned& operator= ( unsigned long v );
sc_unsigned& operator= ( int v );
sc_unsigned& operator= ( unsigned int v );
sc_unsigned& operator= ( double v );
sc_unsigned& operator= ( const sc_int_base& v );
sc_unsigned& operator= ( const sc_uint_base& v );
sc_unsigned& operator= ( const sc_bv_base& );
sc_unsigned& operator= ( const sc_lv_base& );
sc_unsigned& operator= ( const sc_fxval& );
sc_unsigned& operator= ( const sc_fxval_fast& );
sc_unsigned& operator= ( const sc_fxnum& );
sc_unsigned& operator= ( const sc_fxnum_fast& );

// Destructor
~sc_unsigned();

// Increment operators
sc_unsigned& operator++ ();
const sc_unsigned operator++ ( int );

// Decrement operators
sc_unsigned& operator-- ();
const sc_unsigned operator-- ( int );

// Bit selection
sc_unsigned_bitref† operator[] ( int i );
sc_unsigned_bitref_r† operator[] ( int i ) const;

// Part selection
sc_unsigned_subref† range ( int i , int j );
sc_unsigned_subref_r† range( int i , int j ) const;
sc_unsigned_subref† operator() ( int i , int j );
sc_unsigned_subref_r† operator() ( int i , int j ) const;

// Explicit conversions
int to_int() const;
unsigned int to_uint() const;
long to_long() const;
unsigned long to_ulong() const;
int64 to_int64() const;
uint64 to_uint64() const;
double to_double() const;

// Explicit conversion to character string
const std::string to_string( sc_numrep numrep = SC_DEC ) const;
const std::string to_string( sc_numrep numrep, bool w_prefix ) const;

// Print functions
void print( std::ostream& os = std::cout ) const;
void scan( std::istream& is = std::cin );
```

```

// Capacity
int length() const;

// Bit width

// Reduce methods
bool and_reduce() const;
bool nand_reduce() const;
bool or_reduce() const;
bool nor_reduce() const;
bool xor_reduce() const;
bool xnor_reduce() const;

// Overloaded operators

};

} // namespace sc_dt

```

#### 7.6.4.3 Constraints on usage

An object of type **sc\_unsigned** may not be used as a direct replacement for a C++ integer type since no implicit type conversion member functions are provided. An explicit type conversion is required to pass the value of an **sc\_unsigned** object as an argument to a function expecting a C++ integer value argument.

#### 7.6.4.4 Constructors

explicit **sc\_unsigned**( int nb = sc\_length\_param().len() );

Constructor **sc\_unsigned** shall create an **sc\_unsigned** object of word length specified by **nb**. This is the default constructor when **nb** is not specified (in which case its value is set by the current length context). The initial value shall be **0**.

```
template< class T >
```

```
sc_unsigned( const sc_generic_base<T>& a );
```

Constructor **sc\_unsigned** shall create an **sc\_unsigned** object with a word length matching the constructor argument. The constructor shall set the initial value of the object to the value returned from the member function **to\_sc\_unsigned** of the constructor argument.

The other constructors create an **sc\_unsigned** object with the same word length and value as the constructor argument.

#### 7.6.4.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to **sc\_unsigned**, using truncation or sign-extension as described in 7.2.1.

#### 7.6.4.6 Explicit type conversion

```
const std::string to_string( sc_numrep numrep = SC_DEC ) const;
```

```
const std::string to_string( sc_numrep numrep, bool w_prefix ) const;
```

Member function **to\_string** shall perform the conversion to an **std::string**, as described in 7.2.11. Calling the **to\_string** function with a single argument is equivalent to calling the **to\_string** function with two arguments, where the second argument is **true**. Calling the **to\_string** function with no

arguments is equivalent to calling the **to\_string** function with two arguments, where the first argument is **SC\_DEC** and the second argument is **true**.

#### 7.6.4.7 Arithmetic, bitwise, and comparison operators

Operations specified in Table 11, Table 12, and Table 13 are permitted. The following applies:

- **S** represents an object of type **sc\_signed**.
- **U** represents an object of type **sc\_unsigned**.
- **i** represents an object of integer type **int**, **long**, **unsigned int**, **unsigned long**, **sc\_signed**, **sc\_unsigned**, **sc\_int\_base**, or **sc\_uint\_base**.
- **s** represents an object of signed integer type **int**, **long**, **sc\_signed**, or **sc\_int\_base**.
- **u** represents an object of unsigned integer type **unsigned int**, **unsigned long**, **sc\_unsigned**, or **sc\_uint\_base**.

The operands may also be of any other class that is derived from those just given.

**Table 11—sc\_unsigned arithmetic operations**

| Expression | Return type  | Operation                               |
|------------|--------------|-----------------------------------------|
| $U + u$    | sc_unsigned  | sc_unsigned addition                    |
| $u + U$    | sc_unsigned  | sc_unsigned addition                    |
| $U + s$    | sc_signed    | addition of sc_unsigned and signed      |
| $s + U$    | sc_signed    | addition of signed and sc_unsigned      |
| $U += i$   | sc_unsigned& | sc_unsigned assign sum                  |
| $U - i$    | sc_signed    | sc_unsigned subtraction                 |
| $i - U$    | sc_signed    | sc_unsigned subtraction                 |
| $U -= i$   | sc_unsigned& | sc_unsigned assign difference           |
| $U * u$    | sc_unsigned  | sc_unsigned multiplication              |
| $u * U$    | sc_unsigned  | sc_unsigned multiplication              |
| $U * s$    | sc_signed    | multiplication of sc_unsigned by signed |
| $s * U$    | sc_signed    | multiplication of signed by sc_unsigned |
| $U *= i$   | sc_unsigned& | sc_unsigned assign product              |
| $U / u$    | sc_unsigned  | sc_unsigned division                    |
| $u / U$    | sc_unsigned  | sc_unsigned division                    |

**Table 11—sc\_unsigned arithmetic operations (*continued*)**

| Expression | Return type  | Operation                            |
|------------|--------------|--------------------------------------|
| $U / s$    | sc_signed    | division of sc_unsigned by signed    |
| $s / U$    | sc_signed    | division of signed by sc_unsigned    |
| $U /= i$   | sc_unsigned& | sc_unsigned assign quotient          |
| $U \% u$   | sc_unsigned  | sc_unsigned remainder                |
| $u \% U$   | sc_unsigned  | sc_unsigned remainder                |
| $U \% s$   | sc_signed    | remainder of sc_unsigned with signed |
| $s \% U$   | sc_signed    | remainder of signed with sc_unsigned |
| $U \% = i$ | sc_unsigned& | sc_unsigned assign remainder         |
| $+U$       | sc_unsigned  | sc_unsigned unary plus               |
| $-U$       | sc_signed    | sc_unsigned unary minus              |

If the result of any arithmetic operation is zero, the word length of the return value shall be set by the **sc\_length\_context** in scope. Otherwise, the following rules apply:

- Addition shall return a result with a word length that is equal to the word length of the longest operand plus one.
- Multiplication shall return a result with a word length that is equal to the sum of the word lengths of the two operands.
- Remainder shall return a result with a word length that is equal to the word length of the shortest operand.
- All other arithmetic operators shall return a result with a word length that is equal to the word length of the longest operand.



**Table 12—sc\_unsigned bitwise operations**

| Expression    | Return type  | Operation                               |
|---------------|--------------|-----------------------------------------|
| $U \& u$      | sc_unsigned  | sc_unsigned bitwise and                 |
| $u \& U$      | sc_unsigned  | sc_unsigned bitwise and                 |
| $U \& s$      | sc_signed    | sc_unsigned bitwise and signed          |
| $s \& U$      | sc_signed    | signed bitwise and sc_unsigned          |
| $U \&= i$     | sc_unsigned& | sc_unsigned assign bitwise and          |
| $U   u$       | sc_unsigned  | sc_unsigned bitwise or                  |
| $u   U$       | sc_unsigned  | sc_unsigned bitwise or                  |
| $U   s$       | sc_signed    | sc_unsigned bitwise or signed           |
| $s   U$       | sc_signed    | signed bitwise or sc_unsigned           |
| $U  = i$      | sc_unsigned& | sc_unsigned assign bitwise or           |
| $U \wedge u$  | sc_unsigned  | sc_unsigned bitwise exclusive or        |
| $u \wedge U$  | sc_unsigned  | sc_unsigned bitwise exclusive or        |
| $U \wedge s$  | sc_signed    | sc_unsigned bitwise exclusive or signed |
| $s \wedge U$  | sc_signed    | sc_unsigned bitwise exclusive or signed |
| $U \wedge= i$ | sc_unsigned& | sc_unsigned assign bitwise exclusive or |
| $U \ll i$     | sc_unsigned  | sc_unsigned left-shift                  |
| $S \ll U$     | sc_signed    | sc_signed left-shift                    |
| $U \ll= i$    | sc_unsigned& | sc_unsigned assign left-shift           |
| $U \gg i$     | sc_unsigned  | sc_unsigned right-shift                 |
| $S \gg U$     | sc_signed    | sc_signed right-shift                   |
| $U \gg= i$    | sc_unsigned& | sc_unsigned assign right-shift          |
| $\sim U$      | sc_unsigned  | sc_unsigned bitwise complement          |

Binary bitwise operators shall return a result with a word length that is equal to the word length of the longest operand.

The left shift operator shall return a result with a word length that is equal to the word length of its **sc\_unsigned** operand plus one. The bit on the right-hand side of the result shall be set to zero.

The right shift operator shall return a result with a word length that is equal to the word length of its **sc\_unsigned** operand. The bit on the left-hand side of the result shall be set to zero.

**Table 13—sc\_unsigned comparison operations**

| Expression | Return type | Operation                  |
|------------|-------------|----------------------------|
| $U == i$   | bool        | test equal                 |
| $i == U$   | bool        | test equal                 |
| $U != i$   | bool        | test not equal             |
| $i != U$   | bool        | test not equal             |
| $U < i$    | bool        | test less than             |
| $i < U$    | bool        | test less than             |
| $U <= i$   | bool        | test less than or equal    |
| $i <= U$   | bool        | test less than or equal    |
| $U > i$    | bool        | test greater than          |
| $i > U$    | bool        | test greater than          |
| $U >= i$   | bool        | test greater than or equal |
| $i >= U$   | bool        | test greater than or equal |

NOTE—An implementation is required to supply overloaded operators on **sc\_unsigned** objects to satisfy the requirements of this subclause. It is unspecified whether these operators are members of **sc\_unsigned**, global operators, or provided in some other way.

#### 7.6.4.8 Other member functions

void **scan**( std::istream& is = std::cin );

Member function **scan** shall set the value by reading the next formatted character string from the specified input stream (see 7.2.10).

void **print**( std::ostream& os = std::cout ) const;

Member function **print** shall write the value as a formatted character string to the specified output stream (see 7.2.10).

int **length**() const;

Member function **length** shall return the word length (see 7.2.4).

## 7.6.5 **sc\_bigint**

### 7.6.5.1 Description

Class template **sc\_bigint** represents a finite word-length signed integer. The word length shall be specified by a template argument. The integer value shall be stored with a finite precision determined by the specified word length. The precision shall not depend on the limited resolution of any standard C++ integer type.

Any public member functions of the base class **sc\_signed** that are overridden in class **sc\_bigint** shall have the same behavior in the two classes. Any public member functions of the base class not overridden in this way shall be publicly inherited by class **sc\_bigint**. The operations specified in 7.6.3.7 are permitted for objects of type **sc\_bigint**.

### 7.6.5.2 Class definition

```
namespace sc_dt {  
  
template< int W >  
class sc_bigint  
: public sc_signed  
{  
    public:  
        // Constructors  
        sc_bigint();  
        sc_bigint( const sc_bigint<W>& v );  
        sc_bigint( const sc_signed& v );  
        sc_bigint( const sc_signed_subref†& v );  
        template< class T >  
        sc_bigint( const sc_generic_base<T>& a );  
        sc_bigint( const sc_unsigned& v );  
        sc_bigint( const sc_unsigned_subref†& v );  
        sc_bigint( const char* v );  
        sc_bigint( int64 v );  
        sc_bigint( uint64 v );  
        sc_bigint( long v );  
        sc_bigint( unsigned long v );  
        sc_bigint( int v );  
        sc_bigint( unsigned int v );  
        sc_bigint( double v );  
        sc_bigint( const sc_bv_base& v );  
        sc_bigint( const sc_lv_base& v );  
        explicit sc_bigint( const sc_fxval& v );  
        explicit sc_bigint( const sc_fxval_fast& v );  
        explicit sc_bigint( const sc_fxnum& v );  
        explicit sc_bigint( const sc_fxnum_fast& v );  
  
        // Destructor  
        ~sc_bigint();  
  
        // Assignment operators  
        sc_bigint<W>& operator= ( const sc_bigint<W>& v );  
        sc_bigint<W>& operator= ( const sc_signed& v );  
        sc_bigint<W>& operator= ( const sc_signed_subref†& v );  
        template< class T >
```

```

    sc_bigint<W>& operator= ( const sc_generic_base<T>& a );
    sc_bigint<W>& operator= ( const sc_unsigned& v );
    sc_bigint<W>& operator= ( const sc_unsigned_subref†& v );
    sc_bigint<W>& operator= ( const char* v );
    sc_bigint<W>& operator= ( int64 v );
    sc_bigint<W>& operator= ( uint64 v );
    sc_bigint<W>& operator= ( long v );
    sc_bigint<W>& operator= ( unsigned long v );
    sc_bigint<W>& operator= ( int v );
    sc_bigint<W>& operator= ( unsigned int v );
    sc_bigint<W>& operator= ( double v );
    sc_bigint<W>& operator= ( const sc_bv_base& v );
    sc_bigint<W>& operator= ( const sc_lv_base& v );
    sc_bigint<W>& operator= ( const sc_int_base& v );
    sc_bigint<W>& operator= ( const sc_uint_base& v );
    sc_bigint<W>& operator= ( const sc_fxval& v );
    sc_bigint<W>& operator= ( const sc_fxval_fast& v );
    sc_bigint<W>& operator= ( const sc_fxnum& v );
    sc_bigint<W>& operator= ( const sc_fxnum_fast& v );

};

} // namespace sc_dt

```

### 7.6.5.3 Constraints on usage

An object of type **sc\_bigint** may not be used as a direct replacement for a C++ integer type, since no implicit type conversion member functions are provided. An explicit type conversion is required to pass the value of an **sc\_bigint** object as an argument to a function expecting a C++ integer value argument.

### 7.6.5.4 Constructors

**sc\_bigint**();

Default constructor **sc\_bigint** shall create an **sc\_bigint** object of word length specified by the template argument **W** and shall set the initial value to **0**.

template< class T >

**sc\_bigint**( const sc\_generic\_base<T>& a );

Constructor **sc\_bigint** shall create an **sc\_bigint** object of word length specified by the template argument. The constructor shall set the initial value of the object to the value returned from the member function **to\_sc\_signed** of the constructor argument.

Other constructors shall create an **sc\_bigint** object of word length specified by the template argument **W** and value corresponding to the integer magnitude of the constructor argument. If the word length of the specified initial value differs from the template argument, truncation or sign-extension shall be used as described in 7.2.1.

NOTE—Most of the constructors can be used as implicit conversions from fundamental types or SystemC data types to **sc\_bigint**. Hence a function having an **sc\_bigint** parameter can be passed a floating-point argument, for example, and the argument will be implicitly converted. The exceptions are the conversions from fixed-point types to **sc\_bigint**, which must be called explicitly.

### 7.6.5.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to **sc\_bigint**, using truncation or sign-extension as described in 7.2.1.

### 7.6.6 sc\_biguint

#### 7.6.6.1 Description

Class template **sc\_biguint** represents a finite word-length unsigned integer. The word length shall be specified by a template argument. The integer value shall be stored with a finite precision determined by the specified word length. The precision shall not depend on the limited resolution of any standard C++ integer type.

Any public member functions of the base class **sc\_unsigned** that are overridden in class **sc\_biguint** shall have the same behavior in the two classes. Any public member functions of the base class not overridden in this way shall be publicly inherited by class **sc\_biguint**. The operations specified in 7.6.4.7 are permitted for objects of type **sc\_biguint**.

#### 7.6.6.2 Class definition

```
namespace sc_dt {

template< int W >
class sc_biguint
: public sc_unsigned
{
public:
    // Constructors
    sc_biguint();
    sc_biguint( const sc_biguint<W>& v );
    sc_biguint( const sc_unsigned& v );
    sc_biguint( const sc_unsigned_subref†& v );
    template< class T >
    sc_biguint( const sc_generic_base<T>& a );
    sc_biguint( const sc_signed& v );
    sc_biguint( const sc_signed_subref†& v );
    sc_biguint( const char* v );
    sc_biguint( int64 v );
    sc_biguint( uint64 v );
    sc_biguint( long v );
    sc_biguint( unsigned long v );
    sc_biguint( int v );
    sc_biguint( unsigned int v );
    sc_biguint( double v );
    sc_biguint( const sc_bv_base& v );
    sc_biguint( const sc_lv_base& v );
    explicit sc_biguint( const sc_fxval& v );
    explicit sc_biguint( const sc_fxval_fast& v );
    explicit sc_biguint( const sc_fxnum& v );
    explicit sc_biguint( const sc_fxnum_fast& v );

    // Destructor
    ~sc_biguint();
};
```

```

// Assignment operators
sc_biguint<W>& operator= ( const sc_biguint<W>& v );
sc_biguint<W>& operator= ( const sc_unsigned& v );
sc_biguint<W>& operator= ( const sc_unsigned_subref†& v );
template< class T >
sc_biguint<W>& operator= ( const sc_generic_base<T>& a );
sc_biguint<W>& operator= ( const sc_signed& v );
sc_biguint<W>& operator= ( const sc_signed_subref†& v );
sc_biguint<W>& operator= ( const char* v );
sc_biguint<W>& operator= ( int64 v );
sc_biguint<W>& operator= ( uint64 v );
sc_biguint<W>& operator= ( long v );
sc_biguint<W>& operator= ( unsigned long v );
sc_biguint<W>& operator= ( int v );
sc_biguint<W>& operator= ( unsigned int v );
sc_biguint<W>& operator= ( double v );
sc_biguint<W>& operator= ( const sc_bv_base& v );
sc_biguint<W>& operator= ( const sc_lv_base& v );
sc_biguint<W>& operator= ( const sc_int_base& v );
sc_biguint<W>& operator= ( const sc_uint_base& v );
sc_biguint<W>& operator= ( const sc_fxval& v );
sc_biguint<W>& operator= ( const sc_fxval_fast& v );
sc_biguint<W>& operator= ( const sc_fxnum& v );
sc_biguint<W>& operator= ( const sc_fxnum_fast& v );
};

} // namespace sc_dt

```

### 7.6.6.3 Constraints on usage

An object of type **sc\_biguint** may not be used as a direct replacement for a C++ integer type, since no implicit type conversion member functions are provided. An explicit type conversion is required to pass the value of an **sc\_biguint** object as an argument to a function expecting a C++ integer value argument.

### 7.6.6.4 Constructors

**sc\_biguint**();

Default constructor **sc\_biguint** shall create an **sc\_biguint** object of word length specified by the template argument **W** and shall set the initial value to **0**.

template< class T >

**sc\_biguint**( const sc\_generic\_base<T>& a );

Constructor shall create an **sc\_biguint** object of word length specified by the template argument. The constructor shall set the initial value of the object to the value returned from the member function **to\_sc\_unsigned** of the constructor argument.

The other constructors shall create an **sc\_biguint** object of word length specified by the template argument **W** and value corresponding to the integer magnitude of the constructor argument. If the word length of the specified initial value differs from the template argument, truncation or sign-extension shall be used as described in 7.2.1.

NOTE—Most of the constructors can be used as implicit conversions from fundamental types or SystemC data types to **sc\_biguint**. Hence a function having an **sc\_biguint** parameter can be passed a floating-point argument, for example, and the argument will be implicitly converted. The exceptions are the conversions from fixed-point types to **sc\_biguint**, which must be called explicitly.

#### 7.6.6.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to **sc\_biguint**, using truncation or sign-extension, as described in 7.2.1.

#### 7.6.7 Bit-selects

##### 7.6.7.1 Description

Class *sc\_signed\_bitref<sup>r†</sup>* represents a bit selected from an **sc\_signed** used as an rvalue.

Class *sc\_signed\_bitref<sup>†</sup>* represents a bit selected from an **sc\_signed** used as an lvalue.

Class *sc\_unsigned\_bitref<sup>r†</sup>* represents a bit selected from an **sc\_unsigned** used as an rvalue.

Class *sc\_unsigned\_bitref<sup>†</sup>* represents a bit selected from an **sc\_unsigned** used as an lvalue.

##### 7.6.7.2 Class definition

```
namespace sc_dt {  
  
class sc_signed_bitrefr†  
: public sc_value_base†  
{  
    friend class sc_signed;  
    friend class sc_signed_bitref†;  
  
public:  
    // Copy constructor  
    sc_signed_bitrefr†( const sc_signed_bitrefr†& a );  
  
    // Destructor  
    virtual ~sc_signed_bitrefr†();  
  
    // Capacity  
    int length() const;  
  
    // Implicit conversion to uint64  
    operator uint64 () const;  
    bool operator! () const;  
    bool operator~ () const;  
  
    // Explicit conversions  
    bool to_bool() const;  
  
    // Other methods  
    void print( std::ostream& os = std::cout ) const;
```

```
protected:
    sc_signed_bitref_r†();

private:
    // Disabled
    sc_signed_bitref_r† & operator= ( const sc_signed_bitref_r† & );
};

// -----

class sc_signed_bitref†
: public sc_signed_bitref_r†
{
    friend class sc_signed;

public:
    // Copy constructor
    sc_signed_bitref†( const sc_signed_bitref† & a );

    // Assignment operators
    sc_signed_bitref† & operator= ( const sc_signed_bitref_r† & );
    sc_signed_bitref† & operator= ( const sc_signed_bitref† & );
    sc_signed_bitref† & operator= ( bool );

    sc_signed_bitref† & operator&= ( bool );
    sc_signed_bitref† & operator|= ( bool );
    sc_signed_bitref† & operator^= ( bool );

    // Other methods
    void scan( std::istream& is = std::cin );

protected:
    sc_signed_bitref†();
};

// -----

class sc_unsigned_bitref_r†
: public sc_value_base†
{
    friend class sc_unsigned;

public:
    // Copy constructor
    sc_unsigned_bitref_r†( const sc_unsigned_bitref_r† & a );

    // Destructor
    virtual ~sc_unsigned_bitref_r†();

    // Capacity
    int length() const;

    // Implicit conversion to uint64
    operator uint64 () const;
```



```

    bool operator! () const;
    bool operator~ () const;

    // Explicit conversions
    bool to_bool() const;

    // Other methods
    void print( std::ostream& os = std::cout ) const;

protected:
    sc_unsigned_bitref†();

private:
    // Disabled
    sc_unsigned_bitref† & operator= ( const sc_unsigned_bitref† & );
};

// -----

class sc_unsigned_bitref†
: public sc_unsigned_bitref†
{
    friend class sc_unsigned;

public:
    // Copy constructor
    sc_unsigned_bitref†( const sc_unsigned_bitref† & a );

    // Assignment operators
    sc_unsigned_bitref† & operator= ( const sc_unsigned_bitref† & );
    sc_unsigned_bitref† & operator= ( const sc_unsigned_bitref† & );
    sc_unsigned_bitref† & operator= ( bool );

    sc_unsigned_bitref† & operator&= ( bool );
    sc_unsigned_bitref† & operator|= ( bool );
    sc_unsigned_bitref† & operator^= ( bool );

    // Other methods
    void scan( std::istream& is = std::cin );

protected:
    sc_unsigned_bitref†();
};

} // namespace sc_dt

```

### 7.6.7.3 Constraints on usage

Bit-select objects shall only be created using the bit-select operators of an **sc\_signed** or **sc\_unsigned** object (or an instance of a class derived from **sc\_signed** or **sc\_unsigned**).

An application shall not explicitly create an instance of any bit-select class.

An application should not declare a reference or pointer to any bit-select object.

It is strongly recommended that an application avoid the use of a bit-select as the return type of a function because the lifetime of the object to which the bit-select refers may not extend beyond the function return statement.

*Example:*

```
sc_dt::sc_signed_bitref get_bit_n(sc_signed iv, int n) {
    return iv[n];    // Unsafe: returned bit-select references local variable
}
```

#### 7.6.7.4 Assignment operators

Overloaded assignment operators for the lvalue bit-selects shall provide conversion from **bool** values. Assignment operators for rvalue bit-selects shall be declared as private to prevent their use by an application.

#### 7.6.7.5 Implicit type conversion

**operator uint64** () const;

**operator uint64** can be used for implicit type conversion from a bit-select to a native C++ unsigned integer having exactly 64 bits. If the selected bit has the value '1' (true), the conversion shall return the value 1; otherwise, it shall return 0.

**bool operator!** () const;

**bool operator~** () const;

**operator!** and **operator~** shall return a C++ bool value that is the inverse of the selected bit.

#### 7.6.7.6 Other member functions

**void scan**( std::istream& is = std::cin );

Member function **scan** shall set the value of the bit referenced by an lvalue bit-select. The value shall correspond to the C++ bool value obtained by reading the next formatted character string from the specified input stream (see 7.2.10).

**void print**( std::ostream& os = std::cout ) const;

Member function **print** shall print the value of the bit referenced by the bit-select to the specified output stream (see 7.2.10). The formatting shall be implementation-defined but shall be equivalent to printing the value returned by member function **to\_bool**.

**int length**() const;

Member function **length** shall unconditionally return a word length of 1 (see 7.2.4).

## 7.6.8 Part-selects

### 7.6.8.1 Description

Class *sc\_signed\_subref<sub>r<sup>†</sup></sub>* represents a signed integer part-select from an **sc\_signed** used as an rvalue.

Class *sc\_signed\_subref<sub>r<sup>†</sup></sub>* represents a signed integer part-select from an **sc\_signed** used as an lvalue.

Class *sc\_unsigned\_subref<sub>r<sup>†</sup></sub>* represents an unsigned integer part-select from an **sc\_unsigned** used as an rvalue.

Class *sc\_unsigned\_subref<sub>r<sup>†</sup></sub>* represents an unsigned integer part-select from an **sc\_unsigned** used as an lvalue.

### 7.6.8.2 Class definition

```
namespace sc_dt {  
  
class sc_signed_subrefr†  
: public sc_value_baser†  
{  
    friend class sc_signed;  
    friend class sc_unsigned;  
  
public:  
    // Copy constructor  
    sc_signed_subrefr†( const sc_signed_subrefr†& a );  
  
    // Destructor  
    virtual ~sc_unsigned_subrefr†();  
  
    // Capacity  
    int length() const;  
  
    // Implicit conversion to sc_unsigned  
    operator sc_unsigned () const;  
  
    // Explicit conversions  
    int to_int() const;  
    unsigned int to_uint() const;  
    long to_long() const;  
    unsigned long to_ulong() const;  
    int64 to_int64() const;  
    uint64 to_uint64() const;  
    double to_double() const;  
  
    // Explicit conversion to character string  
    const std::string to_string( sc_numrep numrep = SC_DEC ) const;  
    const std::string to_string( sc_numrep numrep, bool w_prefix ) const;  
  
    // Reduce methods  
    bool and_reduce() const;  
    bool nand_reduce() const;  
    bool or_reduce() const;
```

```

    bool nor_reduce() const;
    bool xor_reduce() const;
    bool xnor_reduce() const;

    // Other methods
    void print( std::ostream& os = std::cout ) const;

protected:
    sc_signed_subref†();

private:
    // Disabled
    sc_signed_subref† & operator=( const sc_signed_subref† & );
};

// -----

class sc_signed_subref†
: public sc_signed_subref†
{
    friend class sc_signed;

public:
    // Copy constructor
    sc_signed_subref†( const sc_signed_subref† & a );

    // Assignment operators
    sc_signed_subref† & operator=( const sc_signed_subref† & a );
    sc_signed_subref† & operator=( const sc_signed_subref† & a );
    sc_signed_subref† & operator=( const sc_signed & a );
    template< class T >
    sc_signed_subref† & operator=( const sc_generic_base<T>& a );
    sc_signed_subref† & operator=( const sc_unsigned_subref† & a );
    sc_signed_subref† & operator=( const sc_unsigned & a );
    sc_signed_subref† & operator=( const char* a );
    sc_signed_subref† & operator=( unsigned long a );
    sc_signed_subref† & operator=( long a );
    sc_signed_subref† & operator=( unsigned int a );
    sc_signed_subref† & operator=( int a );
    sc_signed_subref† & operator=( uint64 a );
    sc_signed_subref† & operator=( int64 a );
    sc_signed_subref† & operator=( double a );
    sc_signed_subref† & operator=( const sc_int_base & a );
    sc_signed_subref† & operator=( const sc_uint_base & a );

    // Other methods
    void scan( std::istream& is = std::cin );

private:
    // Disabled
    sc_signed_subref†();
};

// -----

```

```

class sc_unsigned_subref_r†
: public sc_value_base†
{
    friend class sc_signed;
    friend class sc_unsigned;

public:
    // Copy constructor
    sc_unsigned_subref_r†( const sc_unsigned_subref_r†& a );

    // Destructor
    virtual ~sc_unsigned_subref_r†();

    // Capacity
    int length() const;

    // Implicit conversion to sc_unsigned
    operator sc_unsigned () const;

    // Explicit conversions
    int to_int() const;
    unsigned int to_uint() const;
    long to_long() const;
    unsigned long to_ulong() const;
    int64 to_int64() const;
    uint64 to_uint64() const;
    double to_double() const;

    // Explicit conversion to character string
    const std::string to_string( sc_numrep numrep = SC_DEC ) const;
    const std::string to_string( sc_numrep numrep , bool w_prefix ) const;

    // Reduce methods
    bool and_reduce() const;
    bool nand_reduce() const;
    bool or_reduce() const;
    bool nor_reduce() const;
    bool xor_reduce() const;
    bool xnor_reduce() const;

    // Other methods
    void print( std::ostream& os = std::cout ) const;

protected:
    sc_unsigned_subref_r†();

private:
    // Disabled
    sc_unsigned_subref_r& operator=( const sc_unsigned_subref_r†& );
};

// -----

```

```

class sc_unsigned_subref†
: public sc_unsigned_subref_r†
{
    friend class sc_unsigned;

public:
    // Copy constructor
    sc_unsigned_subref†( const sc_unsigned_subref†& a );

    // Assignment operators
    sc_unsigned_subref†& operator= ( const sc_unsigned_subref_r†& a );
    sc_unsigned_subref†& operator= ( const sc_unsigned_subref†& a );
    sc_unsigned_subref†& operator= ( const sc_unsigned& a );
    template<class T>
    sc_unsigned_subref†& operator= ( const sc_generic_base<T>& a );
    sc_unsigned_subref†& operator= ( const sc_signed_subref_r& a );
    sc_unsigned_subref†& operator= ( const sc_signed& a );
    sc_unsigned_subref†& operator= ( const char* a );
    sc_unsigned_subref†& operator= ( unsigned long a );
    sc_unsigned_subref†& operator= ( long a );
    sc_unsigned_subref†& operator= ( unsigned int a );
    sc_unsigned_subref†& operator= ( int a );
    sc_unsigned_subref†& operator= ( uint64 a );
    sc_unsigned_subref†& operator= ( int64 a );
    sc_unsigned_subref†& operator= ( double a );
    sc_unsigned_subref†& operator= ( const sc_int_base& a );
    sc_unsigned_subref†& operator= ( const sc_uint_base& a );

    // Other methods
    void scan( std::istream& is = std::cin );

protected:
    sc_unsigned_subref†();
};

} // namespace sc_dt

```

### 7.6.8.3 Constraints on usage

Integer part-select objects shall only be created using the part-select operators of an **sc\_signed** or **sc\_unsigned** object (or an instance of a class derived from **sc\_signed** or **sc\_unsigned**), as described in 7.2.6.

An application shall not explicitly create an instance of any integer part-select class.

An application should not declare a reference or pointer to any integer part-select object.

It is strongly recommended that an application avoid the use of a part-select as the return type of a function because the lifetime of the object to which the part-select refers may not extend beyond the function return statement.

*Example:*

```
sc_dt::sc_signed_subref get_byte(sc_signed s, int pos) {
    return s(pos+7,pos);    // Unsafe: returned part-select references local variable
}
```

NOTE—The left-hand index of a finite-precision integer part-select may be less than the right-hand index. The bit order in the part-select is then the reverse of that in the original integer.

#### 7.6.8.4 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to lvalue integer part-selects. If the size of a data type or string literal operand differs from the integer part-select word length, truncation, zero-extension, or sign-extension shall be used, as described in 7.2.1.

Assignment operators for rvalue integer part-selects shall be declared as private to prevent their use by an application.

#### 7.6.8.5 Implicit type conversion

```
sc_signed_subref r†::operator sc_unsigned() const;
sc_unsigned_subref r†::operator sc_unsigned() const;

    operator sc_unsigned can be used for implicit type conversion from integer part-selects to
    sc_unsigned.
```

NOTE—These operators are used by the output stream operator and by member functions of other data type classes that are not explicitly overloaded for finite-precision integer part-selects.

#### 7.6.8.6 Explicit type conversion

```
const std::string to_string( sc_numrep numrep = SC_DEC ) const;
const std::string to_string( sc_numrep numrep , bool w_prefix ) const;
```

Member function **to\_string** shall perform a conversion to an **std::string** representation, as described in 7.2.11. Calling the **to\_string** function with a single argument is equivalent to calling the **to\_string** function with two arguments, where the second argument is **true**. Calling the **to\_string** function with no arguments is equivalent to calling the **to\_string** function with two arguments where the first argument is **SC\_DEC** and the second argument is **true**.

#### 7.6.8.7 Other member functions

```
void scan( std::istream& is = std::cin );
```

Member function **scan** shall set the values of the bits referenced by an lvalue part-select by reading the next formatted character string from the specified input stream (see 7.2.10).

```
void print( std::ostream& os = std::cout ) const;
```

Member function **print** shall print the values of the bits referenced by the part-select to the specified output stream (see 7.2.10).

```
int length() const;
```

Member function **length** shall return the word length of the part-select (see 7.2.4).

## 7.7 Integer concatenations

### 7.7.1 Description

Class *sc\_concatref<sup>†</sup>* represents a concatenation of bits from one or more objects whose concatenation base types are SystemC integers.

### 7.7.2 Class definition

```
namespace sc_dt {

class sc_concatref†
: public sc_generic_base<sc_concatref†>, public sc_value_base†
{
public:
    // Destructor
    virtual ~sc_concatref†();

    // Capacity
    unsigned int length() const;

    // Explicit conversions
    int to_int() const;
    unsigned int to_uint() const;
    long to_long() const;
    unsigned long to_ulong() const;
    int64 to_int64() const;
    uint64 to_uint64() const;
    double to_double() const;
    void to_sc_signed( sc_signed& target ) const;
    void to_sc_unsigned( sc_unsigned& target ) const;

    // Implicit conversions
    operator uint64() const;
    operator const sc_unsigned&() const;

    // Unary operators
    sc_unsigned operator+ () const;
    sc_unsigned operator- () const;
    sc_unsigned operator~ () const;

    // Explicit conversion to character string
    const std::string to_string( sc_numrep numrep = SC_DEC ) const;
    const std::string to_string( sc_numrep numrep , bool w_prefix ) const;

    // Assignment operators
    const sc_concatref† & operator= ( int v );
    const sc_concatref† & operator= ( unsigned int v );
    const sc_concatref† & operator= ( long v );
    const sc_concatref† & operator= ( unsigned long v );
    const sc_concatref† & operator= ( int64 v );
    const sc_concatref† & operator= ( uint64 v );
    const sc_concatref† & operator= ( const sc_concatref†& v );
    const sc_concatref† & operator= ( const sc_signed& v );
};
```



```

    const sc_concatref† & operator= ( const sc_unsigned& v );
    const sc_concatref† & operator= ( const char* v_p );
    const sc_concatref† & operator= ( const sc_bv_base& v );
    const sc_concatref† & operator= ( const sc_lv_base& v );

    // Reduce methods
    bool and_reduce() const;
    bool nand_reduce() const;
    bool or_reduce() const;
    bool nor_reduce() const;
    bool xor_reduce() const;
    bool xnor_reduce() const;

    // Other methods
    void print( std::ostream& os = std::cout ) const;
    void scan( std::istream& is );

private:
    sc_concatref†( const sc_concatref† & );
    ~sc_concatref†();
};

sc_concatref† & concat( sc_value_base† & a , sc_value_base† & b );
const sc_concatref† & concat( const sc_value_base† & a , const sc_value_base† & b );
const sc_concatref† & concat( const sc_value_base† & a , bool b );
const sc_concatref† & concat( bool a , const sc_value_base† & b );
sc_concatref† & operator, ( sc_value_base† & a , sc_value_base† & b );
const sc_concatref† & operator, ( const sc_value_base† & a , const sc_value_base† & b );
const sc_concatref† & operator, ( const sc_value_base† & a , bool b );
const sc_concatref† & operator, ( bool a , const sc_value_base† & b );

} // namespace sc_dt

```

### 7.7.3 Constraints on usage

Integer concatenation objects shall only be created using the **concat** function (or **operator**;) according to the rules in 7.2.7.

At least one of the concatenation arguments shall be an object with a SystemC integer concatenation base type, that is, an instance of a class derived directly or indirectly from class *sc\_value\_base<sup>†</sup>*.

A single concatenation argument (that is, one of the two arguments to the **concat** function or **operator**;) may be a **bool** value, a reference to a **sc\_core::sc\_signal<bool>** channel, or a reference to a **sc\_core::sc\_in<bool>**, **sc\_core::sc\_inout<bool>**, or **sc\_core::sc\_out<bool>** port.

An application shall not explicitly create an instance of any integer concatenation class. An application shall not implicitly create an instance of any integer concatenation class by using it as a function argument or as a function return value.

An application should not declare a reference or pointer to any integer concatenation object.

### 7.7.4 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to lvalue integer concatenations. If the size of a data type or string literal operand differs from the integer concatenation word length, truncation, zero-extension, or sign-extension shall be used, as described in 7.2.1.

Assignment operators for rvalue integer concatenations shall not be called by an application.

### 7.7.5 Implicit type conversion

**operator uint64** () const;  
**operator const sc\_unsigned&** () const;

Operators **uint64** and **sc\_unsigned** shall provide implicit unsigned type conversion from an integer concatenation to a native C++ unsigned integer having exactly 64 bits or a **sc\_unsigned** object with a length equal to the total number of bits contained within the objects referenced by the concatenation.

NOTE—Enables the use of standard C++ and SystemC bitwise logical and arithmetic operators with integer concatenation objects.

### 7.7.6 Explicit type conversion

const std::string **to\_string**( sc\_numrep numrep = SC\_DEC ) const;  
const std::string **to\_string**( sc\_numrep numrep , bool w\_prefix ) const;

Member function **to\_string** shall convert the object to an **std::string** representation, as described in 7.2.11. Calling the **to\_string** function with a single argument is equivalent to calling the **to\_string** function with two arguments, where the second argument is true. Calling the **to\_string** function with no arguments is equivalent to calling the **to\_string** function with two arguments, where the first argument is SC\_DEC and the second argument is true.

### 7.7.7 Other member functions

void **scan**( std::istream& is = std::cin );

Member function **scan** shall set the values of the bits referenced by an lvalue concatenation by reading the next formatted character string from the specified input stream (see 7.2.10).

void **print**( std::ostream& os = std::cout ) const;

Member function **print** shall print the values of the bits referenced by the concatenation to the specified output stream (see 7.2.10).

int **length**() const;

Member function **length** shall return the word length of the concatenation (see 7.2.4).

## 7.8 Generic base proxy class

### 7.8.1 Description

Class template **sc\_generic\_base** provides a common proxy base class for application-defined data types that are required to be converted to a SystemC integer.

### 7.8.2 Class definition

```
namespace sc_dt {  
  
template< class T >  
class sc_generic_base  
{  
    public:  
        inline const T* operator-> () const;  
        inline T* operator-> ();  
};  
  
} // namespace sc_dt
```

### 7.8.3 Constraints on usage

An application shall not explicitly create an instance of **sc\_generic\_base**.

Any application-defined type derived from **sc\_generic\_base** shall provide the following public const member functions:

int **length**() const;

Member function **length** shall return the number of bits required to hold the integer value.

uint64 **to\_uint64**() const;

Member function **to\_uint64** shall return the value as a native C++ unsigned integer having exactly 64 bits.

int64 **to\_int64**() const;

Member function **to\_int64** shall return the value as a native C++ signed integer having exactly 64 bits.

void **to\_sc\_unsigned**( sc\_unsigned& ) const;

Member function **to\_sc\_unsigned** shall return the value as an unsigned integer using the **sc\_unsigned** argument passed by reference.

void **to\_sc\_signed**( sc\_signed& ) const;

Member function **to\_sc\_signed** shall return the value as a signed integer using the **sc\_signed** argument passed by reference.

## 7.9 Logic and vector types

### 7.9.1 Type definitions

The following enumerated type definition is used by the logic and vector type classes. Its literal values represent (in numerical order) the four possible logic states: *logic 0*, *logic 1*, *high-impedance*, and *unknown*, respectively. This type is not intended to be used directly by an application, which should instead use the character literals '0', '1', 'Z', and 'X' to represent the logic states, or the application may use the constants SC\_LOGIC\_0, SC\_LOGIC\_1, SC\_LOGIC\_Z, SC\_LOGIC\_X in contexts where the character literals would be ambiguous.

```
namespace sc_dt {

enum sc_logic_value_t
{
    Log_0 = 0,
    Log_1,
    Log_Z,
    Log_X
};

} // namespace sc_dt
```

### 7.9.2 sc\_logic

#### 7.9.2.1 Description

Class **sc\_logic** represents a single bit with a value corresponding to any one of the four logic states. Applications should use the character literals '0', '1', 'Z', and 'X' to represent the states logic 0, logic 1, high-impedance, and unknown, respectively. The lower-case character literals 'z' and 'x' are acceptable alternatives to 'Z' and 'X', respectively. Any other character used as an **sc\_logic** literal shall be interpreted as the unknown state.

The C++ bool values **false** and **true** may be used as arguments to **sc\_logic** constructors and operators. They shall be interpreted as logic 0 and logic 1, respectively.

Logic operations shall be permitted for **sc\_logic** values following the truth tables shown in Table 14, Table 15, Table 16, and Table 17.

**Table 14—sc\_logic AND truth table**

|     | '0' | '1' | 'Z' | 'X' |
|-----|-----|-----|-----|-----|
| '0' | '0' | '0' | '0' | '0' |
| '1' | '0' | '1' | 'X' | 'X' |
| 'Z' | '0' | 'X' | 'X' | 'X' |
| 'X' | '0' | 'X' | 'X' | 'X' |

**Table 15—sc\_logic OR truth table**

|     | '0' | '1' | 'Z' | 'X' |
|-----|-----|-----|-----|-----|
| '0' | '0' | '1' | 'X' | 'X' |
| '1' | '1' | '1' | '1' | '1' |
| 'Z' | 'X' | '1' | 'X' | 'X' |
| 'X' | 'X' | '1' | 'X' | 'X' |

**Table 16—sc\_logic exclusive or truth table**

|     | '0' | '1' | 'Z' | 'X' |
|-----|-----|-----|-----|-----|
| '0' | '0' | '1' | 'X' | 'X' |
| '1' | '1' | '0' | 'X' | 'X' |
| 'Z' | 'X' | 'X' | 'X' | 'X' |
| 'X' | 'X' | 'X' | 'X' | 'X' |

**Table 17—sc\_logic complement truth table**

|  | '0' | '1' | 'Z' | 'X' |
|--|-----|-----|-----|-----|
|  | '1' | '0' | 'X' | 'X' |

### 7.9.2.2 Class definition

```
namespace sc_dt {

class sc_logic
{
public:
    // Constructors
    sc_logic();
    sc_logic( const sc_logic& a );
    sc_logic( sc_logic_value_t v );
```

```

explicit sc_logic( bool a );
explicit sc_logic( char a );
explicit sc_logic( int a );

// Destructor
~sc_logic();

// Assignment operators
sc_logic& operator= ( const sc_logic& a );
sc_logic& operator= ( sc_logic_value_t v );
sc_logic& operator= ( bool a );
sc_logic& operator= ( char a );
sc_logic& operator= ( int a );

// Explicit conversions
sc_logic_value_t value() const;
char to_char() const;
bool to_bool() const;
bool is_01() const;

void print( std::ostream& os = std::cout ) const;
void scan( std::istream& is = std::cin );

private:
    // Disabled
    explicit sc_logic( const char* );
    sc_logic& operator= ( const char* );
};

} // namespace sc_dt

```

### 7.9.2.3 Constraints on usage

An integer argument to an **sc\_logic** constructor or operator shall be equivalent to the corresponding **sc\_logic\_value\_t** enumerated value. It shall be an error if any such integer argument is outside the range 0 to 3.

A literal value assigned to an **sc\_logic** object or used to initialize an **sc\_logic** object may be a character literal but not a string literal.

### 7.9.2.4 Constructors

**sc\_logic**();

Default constructor **sc\_logic** shall create an **sc\_logic** object with a value of unknown.

```

sc_logic( const sc_logic& a );
sc_logic( sc_logic_value_t v );
explicit sc_logic( bool a );
explicit sc_logic( char a );
explicit sc_logic( int a );

```

Constructor **sc\_logic** shall create an **sc\_logic** object with the value specified by the argument.

### 7.9.2.5 Explicit type conversion

`sc_logic_value_t value()` const;

Member function **value** shall convert the **sc\_logic** value to the **sc\_logic\_value\_t** equivalent.

`char to_char()` const;

Member function **to\_char** shall convert the **sc\_logic** value to the **char** equivalent.

`bool to_bool()` const;

Member function **to\_bool** shall convert the **sc\_logic** value to **false** or **true**. It shall be an error to call this function if the **sc\_logic** value is not logic 0 or logic 1.

`bool is_01()` const;

Member function **is\_01** shall return **true** if the **sc\_logic** value is logic 0 or logic 1; otherwise, the return value shall be **false**.

### 7.9.2.6 Bitwise and comparison operators

Operations specified in Table 18 shall be permitted. The following applies:

- **L** represents an object of type **sc\_logic**.
- **n** represents an object of type **int**, **sc\_logic**, **sc\_logic\_value\_t**, **bool**, **char**, or **int**.

**Table 18—sc\_logic bitwise and comparison operations**

| Expression              | Return type    | Operation                            |
|-------------------------|----------------|--------------------------------------|
| <code>~L</code>         | const sc_logic | sc_logic bitwise complement          |
| <code>L &amp; n</code>  | const sc_logic | sc_logic bitwise and                 |
| <code>n &amp; L</code>  | const sc_logic | sc_logic bitwise and                 |
| <code>L &amp;= n</code> | sc_logic&      | sc_logic assign bitwise and          |
| <code>L   n</code>      | const sc_logic | sc_logic bitwise or                  |
| <code>n   L</code>      | const sc_logic | sc_logic bitwise or                  |
| <code>L  = n</code>     | sc_logic&      | sc_logic assign bitwise or           |
| <code>L ^ n</code>      | const sc_logic | sc_logic bitwise exclusive or        |
| <code>n ^ L</code>      | const sc_logic | sc_logic bitwise exclusive or        |
| <code>L ^= n</code>     | sc_logic&      | sc_logic assign bitwise exclusive or |
| <code>L == n</code>     | bool           | test equal                           |
| <code>n == L</code>     | bool           | test equal                           |
| <code>L != n</code>     | bool           | test not equal                       |
| <code>n != L</code>     | bool           | test not equal                       |

NOTE—An implementation is required to supply overloaded operators on **sc\_logic** objects to satisfy the requirements of this subclause. It is unspecified whether these operators are members of **sc\_logic**, global operators, or provided in some other way.

### 7.9.2.7 Other member functions

void **scan**( std::istream& is = std::cin );

Member function **scan** shall set the value by reading the next non-white-space character from the specified input stream (see 7.2.10).

void **print**( std::ostream& os = std::cout ) const;

Member function **print** shall write the value as the character literal '0', '1', 'X', or 'Z' to the specified output stream (see 7.2.10).

### 7.9.2.8 sc\_logic constant definitions

A constant of type **sc\_logic** shall be defined for each of the four possible **sc\_logic\_value\_t** states. These constants should be used by applications to assign values to, or compare values with, other **sc\_logic** objects, particularly in those cases where an implicit conversion from a C++ char value would be ambiguous.

```
namespace sc_dt {

const sc_logic SC_LOGIC_0( Log_0 );
const sc_logic SC_LOGIC_1( Log_1 );
const sc_logic SC_LOGIC_Z( Log_Z );
const sc_logic SC_LOGIC_X( Log_X );

} // namespace sc_dt
```

*Example:*

```
sc_core::sc_signal<sc_logic> A;
A = '0'; // Error: ambiguous conversion
A = static_cast<sc_logic>('0'); // Correct but not recommended
A = SC_LOGIC_0; // Recommended representation of logic 0
```

## 7.9.3 sc\_bv\_base

### 7.9.3.1 Description

Class **sc\_bv\_base** represents a finite word-length bit vector. It can be treated as an array of **bool** or an array of **sc\_logic\_value\_t** (with the restriction that only the states logic 0 and logic 1 are legal). The word length shall be specified by a constructor argument or, by default, by the length context object currently in scope. The word length of an **sc\_bv\_base** object shall be fixed during instantiation and shall not subsequently be changed.

**sc\_bv\_base** is the base class for the **sc\_bv** class template.



### 7.9.3.2 Class definition

```

namespace sc_dt {

class sc_bv_base
{
    friend class sc_lv_base;

public:
    // Constructors
    explicit sc_bv_base( int nb = sc_length_param().len() );
    explicit sc_bv_base( bool a, int nb = sc_length_param().len() );
    sc_bv_base( const char* a );
    sc_bv_base( const char* a , int nb );
    template <class X>
    sc_bv_base( const sc_subref_r†<X>& a );
    template <class T1, class T2>
    sc_bv_base( const sc_concref_r†<T1,T2>& a );
    sc_bv_base( const sc_lv_base& a );
    sc_bv_base( const sc_bv_base& a );

    // Destructor
    virtual ~sc_bv_base();

    // Assignment operators
    template <class X>
    sc_bv_base& operator=( const sc_subref_r†<X>& a );
    template <class T1, class T2>
    sc_bv_base& operator=( const sc_concref_r†<T1,T2>& a );
    sc_bv_base& operator=( const sc_bv_base& a );
    sc_bv_base& operator=( const sc_lv_base& a );
    sc_bv_base& operator=( const char* a );
    sc_bv_base& operator=( const bool* a );
    sc_bv_base& operator=( const sc_logic* a );
    sc_bv_base& operator=( const sc_unsigned& a );
    sc_bv_base& operator=( const sc_signed& a );
    sc_bv_base& operator=( const sc_uint_base& a );
    sc_bv_base& operator=( const sc_int_base& a );
    sc_bv_base& operator=( unsigned long a );
    sc_bv_base& operator=( long a );
    sc_bv_base& operator=( unsigned int a );
    sc_bv_base& operator=( int a );
    sc_bv_base& operator=( uint64 a );
    sc_bv_base& operator=( int64 a );

    // Bitwise rotations
    sc_bv_base& lrotate( int n );
    sc_bv_base& rrotate( int n );

    // Bitwise reverse
    sc_bv_base& reverse();

    // Bit selection
    sc_bitref†<sc_bv_base> operator[]( int i );

```

```

    sc_bitref_r†<sc_bv_base> operator[( int i ) const;

    // Part selection
    sc_subref†<sc_bv_base> operator( int hi , int lo );
    sc_subref_r†<sc_bv_base> operator( int hi , int lo ) const;

    sc_subref†<sc_bv_base> range( int hi , int lo );
    sc_subref_r†<sc_bv_base> range( int hi , int lo ) const;

    // Reduce functions
    sc_logic_value_t and_reduce() const;
    sc_logic_value_t nand_reduce() const;
    sc_logic_value_t or_reduce() const;
    sc_logic_value_t nor_reduce() const;
    sc_logic_value_t xor_reduce() const;
    sc_logic_value_t xnor_reduce() const;

    // Common methods
    int length() const;

    // Explicit conversions to character string
    const std::string to_string() const;
    const std::string to_string( sc_numrep ) const;
    const std::string to_string( sc_numrep , bool ) const;

    // Explicit conversions
    int to_int() const;
    unsigned int to_uint() const;
    long to_long() const;
    unsigned long to_ulong() const;
    bool is_01() const;

    // Other methods
    void print( std::ostream& os = std::cout ) const;
    void scan( std::istream& is = std::cin );
};

} // namespace sc_dt

```

### 7.9.3.3 Constraints on usage

Attempting to assign the **sc\_logic\_value\_t** values high-impedance or unknown to any element of an **sc\_bv\_base** object shall be an error.

The result of assigning an array of bool or an array of **sc\_logic** to an **sc\_bv\_base** object having a greater word length than the number of array elements is undefined.

### 7.9.3.4 Constructors

explicit **sc\_bv\_base**( int nb = sc\_length\_param().len() );

Default constructor **sc\_bv\_base** shall create an **sc\_bv\_base** object of word length specified by **nb** and shall set the initial value of each element to logic **0**. This is the default constructor when **nb** is not specified (in which case its value is set by the current length context).

explicit **sc\_bv\_base**( bool a , int nb = sc\_length\_param().len() );

Constructor **sc\_bv\_base** shall create an **sc\_bv\_base** object of word length specified by **nb**. If **nb** is not specified the length shall be set by the current length context. The constructor shall set the initial value of each element to the value of **a**.

**sc\_bv\_base**( const char\* a );

Constructor **sc\_bv\_base** shall create an **sc\_bv\_base** object with an initial value set by the string **a**. The word length shall be set to the number of characters in the string.

**sc\_bv\_base**( const char\* a , int nb );

Constructor **sc\_bv\_base** shall create an **sc\_bv\_base** object with an initial value set by the string and word length **nb**. If the number of characters in the string does not match the value of **nb**, the initial value shall be truncated or zero extended to match the word length.

```
template <class X> sc_bv_base( const sc_subref r†<X>& a );
template <class T1, class T2> sc_bv_base( const sc_concref r†<T1,T2>& a );
sc_bv_base( const sc_lv_base& a );
sc_bv_base( const sc_bv_base& a );
```

Constructor **sc\_bv\_base** shall create an **sc\_bv\_base** object with the same word length and value as **a**.

NOTE—An implementation may provide a different set of constructors to create an **sc\_bv\_base** object from an *sc\_subref* r<sup>†</sup><T>, *sc\_concref* r<sup>†</sup><T1,T2>, or *sc\_lv\_base* object, for example, by providing a class template that is used as a common base class for all these types.

### 7.9.3.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to **sc\_bv\_base**, using truncation or zero-extension, as described in 7.2.1.

### 7.9.3.6 Explicit type conversion

```
const std::string to_string() const;
const std::string to_string( sc_numrep ) const;
const std::string to_string( sc_numrep , bool ) const;
```

Member function **to\_string** shall perform the conversion to an **std::string** representation, as described in 7.2.11. Calling the **to\_string** function with a single argument is equivalent to calling the **to\_string** function with two arguments, where the second argument is **true**.

Calling the **to\_string** function with no arguments shall create a binary string with a single '1' or '0' corresponding to each bit. This string shall not be prefixed by "0b" or a leading zero.

*Example:*

```
sc_bv_base B(4);           // 4-bit vector
B = "0xf";                 // Each bit set to logic 1
std::string S1 = B.to_string(SC_BIN,false); // The contents of S1 will be the string "01111"
std::string S2 = B.to_string(SC_BIN);       // The contents of S2 will be the string "0b01111"
std::string S3 = B.to_string();             // The contents of S3 will be the string "1111"
```

bool **is\_01**() const;

Member function **is\_01** shall always return **true**, since an **sc\_bv\_base** object can only contain elements with a value of logic 0 or logic 1.

Member functions that return the integer equivalent of the bit representation shall be provided to satisfy the requirements of 7.2.9.

### 7.9.3.7 Bitwise and comparison operators

Operations specified in Table 19 and Table 20 are permitted. The following applies:

- **B** represents an object of type **sc\_bv\_base**.
- **Vi** represents an object of logic vector type **sc\_bv\_base**, **sc\_lv\_base**, *sc\_subref<sub>r</sub><sup>†</sup><T>* or *sc\_concref<sub>r</sub><sup>†</sup><T1,T2>* or integer type **int**, **long**, **unsigned int**, **unsigned long**, **sc\_signed**, **sc\_unsigned**, **sc\_int\_base**, or **sc\_uint\_base**.
- **i** represents an object of integer type **int**.
- **A** represents an array object with elements of type **char**, **bool**, or **sc\_logic**.

The operands may also be of any other class that is derived from those just given.

**Table 19—sc\_bv\_base bitwise operations**

| Expression | Return type      | Operation                       |
|------------|------------------|---------------------------------|
| B & Vi     | const sc_lv_base | sc_bv_base bitwise and          |
| Vi & B     | const sc_lv_base | sc_bv_base bitwise and          |
| B & A      | const sc_lv_base | sc_bv_base bitwise and          |
| A & B      | const sc_lv_base | sc_bv_base bitwise and          |
| B &= Vi    | sc_bv_base&      | sc_bv_base assign bitwise and   |
| B &= A     | sc_bv_base&      | sc_bv_base assign bitwise and   |
| B   Vi     | const sc_lv_base | sc_bv_base bitwise or           |
| Vi   B     | const sc_lv_base | sc_bv_base bitwise or           |
| B   A      | const sc_lv_base | sc_bv_base bitwise or           |
| A   B      | const sc_lv_base | sc_bv_base bitwise or           |
| B  = Vi    | sc_bv_base&      | sc_bv_base assign bitwise or    |
| B  = A     | sc_bv_base&      | sc_bv_base assign bitwise or    |
| B ^ Vi     | const sc_lv_base | sc_bv_base bitwise exclusive or |

**Table 19—*sc\_bv\_base* bitwise operations (continued)**

| Expression      | Return type             | Operation                                     |
|-----------------|-------------------------|-----------------------------------------------|
| $V_i \wedge B$  | const <i>sc_lv_base</i> | <i>sc_bv_base</i> bitwise exclusive or        |
| $B \wedge A$    | const <i>sc_lv_base</i> | <i>sc_bv_base</i> bitwise exclusive or        |
| $A \wedge B$    | const <i>sc_lv_base</i> | <i>sc_bv_base</i> bitwise exclusive or        |
| $B \wedge= V_i$ | <i>sc_bv_base</i> &     | <i>sc_bv_base</i> assign bitwise exclusive or |
| $B \wedge= A$   | <i>sc_bv_base</i> &     | <i>sc_bv_base</i> assign bitwise exclusive or |
| $B \ll i$       | const <i>sc_lv_base</i> | <i>sc_bv_base</i> left-shift                  |
| $B \ll= i$      | <i>sc_bv_base</i> &     | <i>sc_bv_base</i> assign left-shift           |
| $B \gg i$       | const <i>sc_lv_base</i> | <i>sc_bv_base</i> right-shift                 |
| $B \gg= i$      | <i>sc_bv_base</i> &     | <i>sc_bv_base</i> assign right-shift          |
| $\sim B$        | const <i>sc_lv_base</i> | <i>sc_bv_base</i> bitwise complement          |

Binary bitwise operators shall return a result with a word length that is equal to the word length of the longest operand.

The left shift operator shall return a result with a word length that is equal to the word length of its **sc\_bv\_base** operand plus the right (integer) operand. Bits added on the right-hand side of the result shall be set to zero.

The right shift operator returns a result with a word length that is equal to the word length of its **sc\_bv\_base** operand. Bits added on the left-hand side of the result shall be set to zero.

It is an error if the right operand of a shift operator is negative.

**Table 20—*sc\_bv\_base* comparison operations**

| Expression | Return type | Operation  |
|------------|-------------|------------|
| $B == V_i$ | bool        | test equal |
| $V_i == B$ | bool        | test equal |
| $B == A$   | bool        | test equal |
| $A == B$   | bool        | test equal |

*sc\_bv\_base*& **lrotate**( int *n* );

Member function **lrotate** shall rotate an **sc\_bv\_base** object *n* places to the left.

`sc_bv_base& rrotate( int n );`

Member function **rrotate** shall rotate an **sc\_bv\_base** object **n** places to the right.

`sc_bv_base& reverse();`

Member function **reverse** shall reverse the bit order in an **sc\_bv\_base** object.

NOTE—An implementation is required to supply overloaded operators on **sc\_bv\_base** objects to satisfy the requirements of this subclause. It is unspecified whether these operators are members of **sc\_bv\_base**, global operators, or provided in some other way.

### 7.9.3.8 Other member functions

`void scan( std::istream& is = std::cin );`

Member function **scan** shall set the value by reading the next formatted character string from the specified input stream (see 7.2.10).

`void print( std::ostream& os = std::cout ) const;`

Member function **print** shall write the value as a formatted character string to the specified output stream (see 7.2.10).

`int length() const;`

Member function **length** shall return the word length (see 7.2.4).

## 7.9.4 sc\_lv\_base

### 7.9.4.1 Description

Class **sc\_lv\_base** represents a finite word-length bit vector. It can be treated as an array of **sc\_logic\_value\_t** values. The word length shall be specified by a constructor argument or, by default, by the length context object currently in scope. The word length of an **sc\_lv\_base** object shall be fixed during instantiation and shall not subsequently be changed.

**sc\_lv\_base** is the base class for the **sc\_lv** class template.

### 7.9.4.2 Class definition

```
namespace sc_dt {  
  
class sc_lv_base  
{  
    friend class sc_bv_base;  
  
public:  
    // Constructors  
    explicit sc_lv_base( int length_ = sc_length_param().len() );  
    explicit sc_lv_base( const sc_logic& a, int length_ = sc_length_param().len() );  
    sc_lv_base( const char* a );  
    sc_lv_base( const char* a , int length_ );  
    template <class X>  
    sc_lv_base( const sc_subref_r<X>& a );  
    template <class T1, class T2>  
    sc_lv_base( const sc_concref_r<T1,T2>& a );  
    sc_lv_base( const sc_bv_base& a );  
    sc_lv_base( const sc_lv_base& a );  
  
    // Destructor  
    virtual ~sc_lv_base();  
  
    // Assignment operators  
    template <class X>  
    sc_lv_base& operator= ( const sc_subref_r<X>& a );  
    template <class T1, class T2>  
    sc_lv_base& operator= ( const sc_concref_r<T1,T2>& a );  
    sc_lv_base& operator= ( const sc_bv_base& a );  
    sc_lv_base& operator= ( const sc_lv_base& a );  
    sc_lv_base& operator= ( const char* a );  
    sc_lv_base& operator= ( const bool* a );  
    sc_lv_base& operator= ( const sc_logic* a );  
    sc_lv_base& operator= ( const sc_unsigned& a );  
    sc_lv_base& operator= ( const sc_signed& a );  
    sc_lv_base& operator= ( const sc_uint_base& a );  
    sc_lv_base& operator= ( const sc_int_base& a );  
    sc_lv_base& operator= ( unsigned long a );  
    sc_lv_base& operator= ( long a );  
    sc_lv_base& operator= ( unsigned int a );  
    sc_lv_base& operator= ( int a );  
    sc_lv_base& operator= ( uint64 a );
```

```

sc_lv_base& operator= ( int64 a );

// Bitwise rotations
sc_lv_base& lrotate( int n );
sc_lv_base& rrotate( int n );

// Bitwise reverse
sc_lv_base& reverse();

// Bit selection
sc_bitref†<sc_bv_base> operator[] ( int i );
sc_bitref_r†<sc_bv_base> operator[] ( int i ) const;

// Part selection
sc_subref†<sc_lv_base> operator() ( int hi , int lo );
sc_subref_r†<sc_lv_base> operator() ( int hi , int lo ) const;

sc_subref†<sc_lv_base> range( int h i , int lo );
sc_subref_r†<sc_lv_base> range( int hi , int lo ) const;

// Reduce functions
sc_logic_value_t and_reduce() const;
sc_logic_value_t nand_reduce() const;
sc_logic_value_t or_reduce() const;
sc_logic_value_t nor_reduce() const;
sc_logic_value_t xor_reduce() const;
sc_logic_value_t xnor_reduce() const;

// Common methods
int length() const;

// Explicit conversions to character string
const std::string to_string() const;
const std::string to_string( sc_numrep ) const;
const std::string to_string( sc_numrep , bool ) const;

// Explicit conversions
int to_int() const;
unsigned int to_uint() const;
long to_long() const;
unsigned long to_ulong() const;
bool is_01() const;

// Other methods
void print( std::ostream& os = std::cout ) const;
void scan( std::istream& is = std::cin );
};

} // namespace sc_dt

```

#### 7.9.4.3 Constraints on usage

The result of assigning an array of **bool** or an array of **sc\_logic** to an **sc\_lv\_base** object having a greater word length than the number of array elements is undefined.



#### 7.9.4.4 Constructors

explicit **sc\_lv\_base**( int nb = sc\_length\_param().len() );

Constructor **sc\_lv\_base** shall create an **sc\_lv\_base** object of word length specified by **nb** and shall set the initial value of each element to logic 0. This is the default constructor when **nb** is not specified (in which case its value shall be set by the current length context).

explicit **sc\_lv\_base**( bool a, int nb = sc\_length\_param().len() );

Constructor **sc\_lv\_base** shall create an **sc\_lv\_base** object of word length specified by **nb** and shall set the initial value of each element to the value of **a**. If **nb** is not specified, the length shall be set by the current length context.

**sc\_lv\_base**( const char\* a );

Constructor **sc\_lv\_base** shall create an **sc\_lv\_base** object with an initial value set by the string literal **a**. The word length shall be set to the number of characters in the string literal.

**sc\_lv\_base**( const char\* a , int nb );

Constructor **sc\_lv\_base** shall create an **sc\_lv\_base** object with an initial value set by the string literal and word length **nb**. If the number of characters in the string literal does not match the value of **nb**, the initial value shall be truncated or zero extended to match the word length.

template <class X> **sc\_lv\_base**( const *sc\_subref\_r†*<X>& a );

template <class T1, class T2> **sc\_lv\_base**( const *sc\_concref\_r†*<T1,T2>& a );

**sc\_lv\_base**( const **sc\_bv\_base**& a );

Constructor **sc\_lv\_base** shall create an **sc\_lv\_base** object with the same word length and value as **a**.

**sc\_lv\_base**( const **sc\_lv\_base**& a );

Constructor **sc\_lv\_base** shall create an **sc\_lv\_base** object with the same word length and value as **a**.

NOTE—An implementation may provide a different set of constructors to create an **sc\_lv\_base** object from an *sc\_subref\_r†*<T>, *sc\_concref\_r†*, or **sc\_bv\_base** object, for example, by providing a class template that is used as a common base class for all these types.

#### 7.9.4.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to **sc\_lv\_base**, using truncation or zero-extension, as described in 7.2.1.

#### 7.9.4.6 Explicit type conversion

const std::string **to\_string**() const;

const std::string **to\_string**( sc\_numrep ) const;

const std::string **to\_string**( sc\_numrep , bool ) const;

Member function **to\_string** shall perform a conversion to an **std::string** representation, as described in 7.2.11. Calling the **to\_string** function with a single argument is equivalent to calling the **to\_string** function with two arguments, where the second argument is **true**. Attempting to call the single or

double argument **to\_string** function for an **sc\_lv\_base** object with one or more elements set to the high-impedance or unknown state shall be an error.

Calling the **to\_string** function with no arguments shall create a logic value string with a single '1', '0', 'Z', or 'X' corresponding to each bit. This string shall not be prefixed by "0b" or a leading zero.

*Example:*

```
sc_lv_base L(4);           // 4-bit vector
L = "0xf";                 // Each bit set to logic 1
std::string S1 = L.to_string(SC_BIN,false); // The contents of S1 will be the string "01111"
std::string S2 = L.to_string(SC_BIN);       // The contents of S2 will be the string "0b01111"
std::string S3 = L.to_string();             // The contents of S3 will be the string "1111"
```

**bool is\_01() const;**

Member function **is\_01** shall return **true** only when every element of an **sc\_lv\_base** object has a value of logic 0 or logic 1. If any element has the value high-impedance or unknown, it shall return **false**.

Member functions that return the integer equivalent of the bit representation shall be provided to satisfy the requirements of subclause 7.2.9. Calling any such integer conversion function for an object having one or more bits set to the high-impedance or unknown state shall be an error.

#### 7.9.4.7 Bitwise and comparison operators

Operations specified in Table 21 and Table 22 are permitted. The following applies:

- **L** represents an object of type **sc\_lv\_base**.
- **Vi** represents an object of logic vector type **sc\_bv\_base**, **sc\_lv\_base**,  $sc\_subref\_r^{\dagger} < T >$  or  $sc\_concref\_r^{\dagger} < T1, T2 >$ , or integer type **int**, **long**, **unsigned int**, **unsigned long**, **sc\_signed**, **sc\_unsigned**, **sc\_int\_base**, or **sc\_uint\_base**.
- **i** represents an object of integer type **int**.
- **A** represents an array object with elements of type **char**, **bool**, or **sc\_logic**.

The operands may also be of any other class that is derived from those just given.

**Table 21—sc\_lv\_base bitwise operations**

| Expression      | Return type      | Operation                              |
|-----------------|------------------|----------------------------------------|
| $L \& V_i$      | const sc_lv_base | sc_lv_base bitwise and                 |
| $V_i \& L$      | const sc_lv_base | sc_lv_base bitwise and                 |
| $L \& A$        | const sc_lv_base | sc_lv_base bitwise and                 |
| $A \& L$        | const sc_lv_base | sc_lv_base bitwise and                 |
| $L \&= V_i$     | sc_lv_base&      | sc_lv_base assign bitwise and          |
| $L \&= A$       | sc_lv_base&      | sc_lv_base assign bitwise and          |
| $L   V_i$       | const sc_lv_base | sc_lv_base bitwise or                  |
| $V_i   L$       | const sc_lv_base | sc_lv_base bitwise or                  |
| $L   A$         | const sc_lv_base | sc_lv_base bitwise or                  |
| $A   L$         | const sc_lv_base | sc_lv_base bitwise or                  |
| $L  = V_i$      | sc_lv_base&      | sc_lv_base assign bitwise or           |
| $L  = A$        | sc_lv_base&      | sc_lv_base assign bitwise or           |
| $L \wedge V_i$  | const sc_lv_base | sc_lv_base bitwise exclusive or        |
| $V_i \wedge L$  | const sc_lv_base | sc_lv_base bitwise exclusive or        |
| $L \wedge A$    | const sc_lv_base | sc_lv_base bitwise exclusive or        |
| $A \wedge L$    | const sc_lv_base | sc_lv_base bitwise exclusive or        |
| $L \wedge= V_i$ | sc_lv_base&      | sc_lv_base assign bitwise exclusive or |
| $L \wedge= A$   | sc_lv_base&      | sc_lv_base assign bitwise exclusive or |
| $L \ll i$       | const sc_lv_base | sc_lv_base left-shift                  |
| $L \ll= i$      | sc_lv_base&      | sc_lv_base assign left-shift           |
| $L \gg i$       | const sc_lv_base | sc_lv_base right-shift                 |
| $L \gg= i$      | sc_lv_base&      | sc_lv_base assign right-shift          |
| $\sim L$        | const sc_lv_base | sc_lv_base bitwise complement          |

Binary bitwise operators shall return a result with a word length that is equal to the word length of the longest operand.

The left shift operator shall return a result with a word length that is equal to the word length of its **sc\_lv\_base** operand plus the right (integer) operand. Bits added on the right-hand side of the result shall be set to zero.

The right shift operator shall return a result with a word length that is equal to the word length of its **sc\_lv\_base** operand. Bits added on the left-hand side of the result shall be set to zero.

It is an error if the right operand of a shift operator is negative.

**Table 22—sc\_lv\_base comparison operations**

| Expression | Return type | Operation  |
|------------|-------------|------------|
| $L == Vi$  | bool        | test equal |
| $Vi == L$  | bool        | test equal |
| $L == A$   | bool        | test equal |
| $A == L$   | bool        | test equal |

**sc\_lv\_base**& **lrotate**( int n );

Member function **lrotate** shall rotate an **sc\_lv\_base** object **n** places to the left.

**sc\_lv\_base**& **rrotate**( int n );

Member function **rrotate** shall rotate an **sc\_lv\_base** object **n** places to the right.

**sc\_lv\_base**& **reverse**();

Member function **reverse** shall reverse the bit order in an **sc\_lv\_base** object.

NOTE—An implementation is required to supply overloaded operators on **sc\_lv\_base** objects to satisfy the requirements of this subclause. It is unspecified whether these operators are members of **sc\_lv\_base**, global operators, or provided in some other way.

#### 7.9.4.8 Other member functions

void **scan**( std::istream& is = std::cin );

Member function **scan** shall set the value by reading the next formatted character string from the specified input stream (see 7.2.10).

void **print**( std::ostream& os = std::cout ) const;

Member function **print** shall write the value as a formatted character string to the specified output stream (see 7.2.10).

int **length**() const;

Member function **length** shall return the word length (see 7.2.4).

## 7.9.5 sc\_bv

### 7.9.5.1 Description

Class template **sc\_bv** represents a finite word-length bit vector. It can be treated as an array of **bool** or an array of **sc\_logic\_value\_t** values (with the restriction that only the states logic 0 and logic 1 are legal). The word length shall be specified by a template argument.

Any public member functions of the base class **sc\_bv\_base** that are overridden in class **sc\_bv** shall have the same behavior in the two classes. Any public member functions of the base class not overridden in this way shall be publicly inherited by class **sc\_bv**.

### 7.9.5.2 Class definition

```
namespace sc_dt {

template <int W>
class sc_bv
: public sc_bv_base
{
public:
    // Constructors
    sc_bv();
    explicit sc_bv( bool init_value );
    explicit sc_bv( char init_value );
    sc_bv( const char* a );
    sc_bv( const bool* a );
    sc_bv( const sc_logic* a );
    sc_bv( const sc_unsigned& a );
    sc_bv( const sc_signed& a );
    sc_bv( const sc_uint_base& a );
    sc_bv( const sc_int_base& a );
    sc_bv( unsigned long a );
    sc_bv( long a );
    sc_bv( unsigned int a );
    sc_bv( int a );
    sc_bv( uint64 a );
    sc_bv( int64 a );
    template <class X>
    sc_bv( const sc_subref_r†<X>& a );
    template <class T1, class T2>
    sc_bv( const sc_concref_r†<T1,T2>& a );
    sc_bv( const sc_bv_base& a );
    sc_bv( const sc_lv_base& a );
    sc_bv( const sc_bv<W>& a );

    // Assignment operators
    template <class X>
    sc_bv<W>& operator= ( const sc_subref_r†<X>& a );
    template <class T1, class T2>
    sc_bv<W>& operator= ( const sc_concref_r†<T1,T2>& a );
    sc_bv<W>& operator= ( const sc_bv_base& a );
    sc_bv<W>& operator= ( const sc_lv_base& a );
    sc_bv<W>& operator= ( const sc_bv<W>& a );
}
```

```

    sc_bv<W>& operator= ( const char* a );
    sc_bv<W>& operator= ( const bool* a );
    sc_bv<W>& operator= ( const sc_logic* a );
    sc_bv<W>& operator= ( const sc_unsigned& a );
    sc_bv<W>& operator= ( const sc_signed& a );
    sc_bv<W>& operator= ( const sc_uint_base& a );
    sc_bv<W>& operator= ( const sc_int_base& a );
    sc_bv<W>& operator= ( unsigned long a );
    sc_bv<W>& operator= ( long a );
    sc_bv<W>& operator= ( unsigned int a );
    sc_bv<W>& operator= ( int a );
    sc_bv<W>& operator= ( uint64 a );
    sc_bv<W>& operator= ( int64 a );
};

} // namespace sc_dt

```

### 7.9.5.3 Constraints on usage

Attempting to assign the **sc\_logic\_value\_t** values high-impedance or unknown to any element of an **sc\_bv** object shall be an error.

The result of assigning an array of **bool** or an array of **sc\_logic** to an **sc\_bv** object having a greater word length than the number of array elements is undefined.

### 7.9.5.4 Constructors

**sc\_bv**();

The default constructor **sc\_bv** shall create an **sc\_bv** object of word length specified by the template argument **W** and it shall set the initial value of every element to logic 0.

The other constructors shall create an **sc\_bv** object of word length specified by the template argument **W** and value corresponding to the constructor argument. If the word length of a data type or string literal argument differs from the template argument, truncation or zero-extension shall be applied, as described in 7.2.1. If the number of elements in an array of **bool** or array of **sc\_logic** used as the constructor argument is less than the word length, the initial value of all elements shall be undefined.

NOTE—An implementation may provide a different set of constructors to create an **sc\_bv** object from an *sc\_subref\_r<sup>T</sup>*<*T*>, *sc\_concref\_r<sup>T</sup>*<*T1*,*T2*>, **sc\_bv\_base**, or **sc\_lv\_base** object, for example, by providing a class template that is used as a common base class for all these types.

### 7.9.5.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to **sc\_bv**, using truncation or zero-extension, as described in 7.2.1. The exception is assignment of an array of **bool** or an array of **sc\_logic** to an **sc\_bv** object, as described in 7.9.5.4.

## 7.9.6 sc\_lv

### 7.9.6.1 Description

Class template **sc\_lv** represents a finite word-length bit vector. It can be treated as an array of **sc\_logic\_value\_t** values. The word length shall be specified by a template argument.

Any public member functions of the base class **sc\_lv\_base** that are overridden in class **sc\_lv** shall have the same behavior in the two classes. Any public member functions of the base class not overridden in this way shall be publicly inherited by class **sc\_lv**.

### 7.9.6.2 Class definition

```
namespace sc_dt {

template <int W>
class sc_lv
: public sc_lv_base
{
public:
    // Constructors
    sc_lv();
    explicit sc_lv( const sc_logic& init_value );
    explicit sc_lv( bool init_value );
    explicit sc_lv( char init_value );
    sc_lv( const char* a );
    sc_lv( const bool* a );
    sc_lv( const sc_logic* a );
    sc_lv( const sc_unsigned& a );
    sc_lv( const sc_signed& a );
    sc_lv( const sc_uint_base& a );
    sc_lv( const sc_int_base& a );
    sc_lv( unsigned long a );
    sc_lv( long a );
    sc_lv( unsigned int a );
    sc_lv( int a );
    sc_lv( uint64 a );
    sc_lv( int64 a );
    template <class X>
    sc_lv( const sc_subref_r†<X>& a );
    template <class T1, class T2>
    sc_lv( const sc_concref_r†<T1,T2>& a );
    sc_lv( const sc_bv_base& a );
    sc_lv( const sc_lv_base& a );
    sc_lv( const sc_lv<W>& a );

    // Assignment operators
    template <class X>
    sc_lv<W>& operator=( const sc_subref_r†<X>& a );
    template <class T1, class T2>
    sc_lv<W>& operator=( const sc_concref_r†<T1,T2>& a );
    sc_lv<W>& operator=( const sc_bv_base& a );
    sc_lv<W>& operator=( const sc_lv_base& a );
    sc_lv<W>& operator=( const sc_lv<W>& a );
}
```

```

    sc_lv<W>& operator= ( const char* a );
    sc_lv<W>& operator= ( const bool* a );
    sc_lv<W>& operator= ( const sc_logic* a );
    sc_lv<W>& operator= ( const sc_unsigned& a );
    sc_lv<W>& operator= ( const sc_signed& a );
    sc_lv<W>& operator= ( const sc_uint_base& a );
    sc_lv<W>& operator= ( const sc_int_base& a );
    sc_lv<W>& operator= ( unsigned long a );
    sc_lv<W>& operator= ( long a );
    sc_lv<W>& operator= ( unsigned int a );
    sc_lv<W>& operator= ( int a );
    sc_lv<W>& operator= ( uint64 a );
    sc_lv<W>& operator= ( int64 a );
};

} // namespace sc_dt

```

### 7.9.6.3 Constraints on usage

The result of assigning an array of **bool** or an array of **sc\_logic** to an **sc\_lv** object having a greater word length than the number of array elements is undefined.

### 7.9.6.4 Constructors

**sc\_lv**();

Default constructor **sc\_lv** shall create an **sc\_lv** object of word length specified by the template argument **W** and shall set the initial value of every element to unknown.

The other constructors shall create an **sc\_lv** object of word length specified by the template argument **W** and value corresponding to the constructor argument. If the word length of a data type or string literal argument differs from the template argument, truncation or zero-extension shall be applied, as described in 7.2.1. If the number of elements in an array of **bool** or array of **sc\_logic** used as the constructor argument is less than the word length, the initial value of all elements shall be undefined.

NOTE—An implementation may provide a different set of constructors to create an **sc\_lv** object from an *sc\_subref<sub>r<sup>†</sup></sub><T>*, *sc\_concref<sub>r<sup>†</sup></sub><T1,T2>*, **sc\_bv\_base**, or **sc\_lv\_base** object, for example, by providing a class template that is used as a common base class for all these types.

### 7.9.6.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to **sc\_lv**, using truncation or zero-extension, as described in 7.2.1. The exception is assignment from an array of **bool** or an array of **sc\_logic** to an **sc\_lv** object, as described in 7.9.6.4.



## 7.9.7 Bit-selects

### 7.9.7.1 Description

Class template  $sc\_bitref\_r^{\dagger}<T>$  represents a bit selected from a vector used as an rvalue.

Class template  $sc\_bitref^{\dagger}<T>$  represents a bit selected from a vector used as an lvalue.

The use of the term vector here includes part-selects and concatenations of bit vectors and logic vectors. The template parameter is the name of the class accessed by the bit-select.

### 7.9.7.2 Class definition

```
namespace sc_dt {

template <class T>
class sc_bitref_r†
{
    friend class sc_bv_base;
    friend class sc_lv_base;

public:
    // Copy constructor
    sc_bitref_r†( const sc_bitref_r†<T>& a );

    // Bitwise complement
    const sc_logic operator~() const;

    // Implicit conversion to sc_logic
    operator const sc_logic() const;

    // Explicit conversions
    bool is_01() const;
    bool to_bool() const;
    char to_char() const;

    // Common methods
    int length() const;

    // Other methods
    void print( std::ostream& os = std::cout ) const;

private:
    // Disabled
    sc_bitref_r†();
    sc_bitref_r†<T>& operator= ( const sc_bitref_r†<T>& );
};

// -----

template <class T>
class sc_bitref†
: public sc_bitref_r†<T>
{
```

```

friend class sc_bv_base;
friend class sc_lv_base;

public:
    // Copy constructor
    sc_bitref†( const sc_bitref†<T>& a );

    // Assignment operators
    sc_bitref†<T>& operator= ( const sc_bitref†<T>& a );
    sc_bitref†<T>& operator= ( const sc_bitref†<T>& a );
    sc_bitref†<T>& operator= ( const sc_logic& a );
    sc_bitref†<T>& operator= ( sc_logic_value_t v );
    sc_bitref†<T>& operator= ( bool a );
    sc_bitref†<T>& operator= ( char a );
    sc_bitref†<T>& operator= ( int a );

    // Bitwise assignment operators
    sc_bitref†<T>& operator&= ( const sc_bitref†<T>& a );
    sc_bitref†<T>& operator&= ( const sc_logic& a );
    sc_bitref†<T>& operator&= ( sc_logic_value_t v );
    sc_bitref†<T>& operator&= ( bool a );
    sc_bitref†<T>& operator&= ( char a );
    sc_bitref†<T>& operator&= ( int a );

    sc_bitref†<T>& operator|= ( const sc_bitref†<T>& a );
    sc_bitref†<T>& operator|= ( const sc_logic& a );
    sc_bitref†<T>& operator|= ( sc_logic_value_t v );
    sc_bitref†<T>& operator|= ( bool a );
    sc_bitref†<T>& operator|= ( char a );
    sc_bitref†<T>& operator|= ( int a );

    sc_bitref†<T>& operator^= ( const sc_bitref†<T>& a );
    sc_bitref†<T>& operator^= ( const sc_logic& a );
    sc_bitref†<T>& operator^= ( sc_logic_value_t v );
    sc_bitref†<T>& operator^= ( bool a );
    sc_bitref†<T>& operator^= ( char a );
    sc_bitref†<T>& operator^= ( int a );

    // Other methods
    void scan( std::istream& is = std::cin );

private:
    // Disabled
    sc_bitref();
};

} // namespace sc_dt

```

### 7.9.7.3 Constraints on usage

Bit-select objects shall only be created using the bit-select operators of an **sc\_bv\_base** or **sc\_lv\_base** object (or an instance of a class derived from **sc\_bv\_base** or **sc\_lv\_base**) or a part-select or concatenation thereof, as described in 7.2.6.

An application shall not explicitly create an instance of any bit-select class.

An application should not declare a reference or pointer to any bit-select object.

It is strongly recommended that an application avoid the use of a bit-select as the return type of a function because the lifetime of the object to which the bit-select refers may not extend beyond the function return statement.

*Example:*

```
sc_dt::sc_bitref<sc_bv_base> get_bit_n(sc_bv_base bv, int n) {  
    return bv[n];    // Unsafe: returned bit-select references local variable  
}
```

#### 7.9.7.4 Assignment operators

Overloaded assignment operators for the lvalue bit-select shall provide conversion to **sc\_logic\_value\_t** values. The assignment operator for the rvalue bit-select shall be declared as private to prevent its use by an application.

#### 7.9.7.5 Implicit type conversion

**operator const sc\_logic()** const;

Operator **sc\_logic** shall create an **sc\_logic** object with the same value as the bit-select.

#### 7.9.7.6 Explicit type conversion

**char to\_char()** const;

Member function **to\_char** shall convert the bit-select value to the **char** equivalent.

**bool to\_bool()** const;

Member function **to\_bool** shall convert the bit-select value to **false** or **true**. It shall be an error to call this function if the **sc\_logic** value is not logic 0 or logic 1.

**bool is\_01()** const;

Member function **is\_01** shall return **true** if the **sc\_logic** value is logic 0 or logic 1; otherwise, the return value shall be **false**.

### 7.9.7.7 Bitwise and comparison operators

Operations specified in Table 23 are permitted. The following applies:

**B** represents an object of type  $sc\_bitref\_r^{\dagger}<T>$  (or any derived class).

**Table 23— $sc\_bitref\_r^{\dagger}<T>$  bitwise and comparison operations**

| Expression   | Return type       | Operation                                         |
|--------------|-------------------|---------------------------------------------------|
| $B \& B$     | const $sc\_logic$ | $sc\_bitref\_r^{\dagger}<T>$ bitwise and          |
| $B   B$      | const $sc\_logic$ | $sc\_bitref\_r^{\dagger}<T>$ bitwise or           |
| $B \wedge B$ | const $sc\_logic$ | $sc\_bitref\_r^{\dagger}<T>$ bitwise exclusive or |
| $B == B$     | bool              | test equal                                        |
| $B != B$     | bool              | test not equal                                    |

NOTE—An implementation is required to supply overloaded operators on  $sc\_bitref\_r^{\dagger}<T>$  objects to satisfy the requirements of this subclause. It is unspecified whether these operators are members of  $sc\_bitref\_r^{\dagger}<T>$ , global operators, or provided in some other way.

### 7.9.7.8 Other member functions

void **scan**( std::istream& is = std::cin );

Member function **scan** shall set the value of the bit referenced by an lvalue bit-select. The value shall correspond to the C++ bool value obtained by reading the next formatted character string from the specified input stream (see 7.2.10).

void **print**( std::ostream& os = std::cout ) const;

Member function **print** shall print the value of the bit referenced by the bit-select to the specified output stream (see 7.2.10). The formatting shall be implementation-defined but shall be equivalent to printing the value returned by member function **to\_bool**.

int **length**() const;

Member function **length** shall unconditionally return a word length of 1 (see 7.2.4).

## 7.9.8 Part-selects

### 7.9.8.1 Description

Class template  $sc\_subref\_r^{\dagger}<T>$  represents a part-select from a vector used as an rvalue.

Class template  $sc\_subref^{\dagger}<T>$  represents a part-select from a vector used as an lvalue.

The use of the term vector here includes part-selects and concatenations of bit vectors and logic vectors. The template parameter is the name of the class accessed by the part-select.

The set of operations that can be performed on a part-select shall be identical to that of its associated vector (subject to the constraints that apply to rvalue objects).

### 7.9.8.2 Class definition

```
namespace sc_dt {

template <class T>
class sc_subref_r†
{
public:
    // Copy constructor
    sc_subref_r†( const sc_subref_r†<T>& a );

    // Bit selection
    sc_bitref_r†<sc_subref_r†<T>> operator[] ( int i ) const;

    // Part selection
    sc_subref_r†<sc_subref_r†<T>> operator() ( int hi , int lo ) const;

    sc_subref_r†<sc_subref_r†<T>> range( int hi , int lo ) const;

    // Reduce functions
    sc_logic_value_t and_reduce() const;
    sc_logic_value_t nand_reduce() const;
    sc_logic_value_t or_reduce() const;
    sc_logic_value_t nor_reduce() const;
    sc_logic_value_t xor_reduce() const;
    sc_logic_value_t xnor_reduce() const;

    // Common methods
    int length() const;

    // Explicit conversions to character string
    const std::string to_string() const;
    const std::string to_string( sc_numrep ) const;
    const std::string to_string( sc_numrep , bool ) const;

    // Explicit conversions
    int to_int() const;
    unsigned int to_uint() const;
    long to_long() const;
    unsigned long to_ulong() const;
    bool is_01() const;

    // Other methods
    void print( std::ostream& os = std::cout ) const;
    bool reversed() const;

private:
    // Disabled
    sc_subref_r†();
    sc_subref_r†<T>& operator=( const sc_subref_r†<T>& );
};
```

```
// -----

template <class T>
class sc_subref†
: public sc_subref_r†<T>
{
public:
    // Copy constructor
    sc_subref†( const sc_subref†<T>& a );

    // Assignment operators

    template <class T>
    sc_subref†<T>& operator= ( const sc_subref_r†<T>& a );
    template <class T1, class T2>
    sc_subref†<T>& operator= ( const sc_concref_r†<T1,T2>& a );
    sc_subref†<T>& operator= ( const sc_bv_base& a );
    sc_subref†<T>& operator= ( const sc_lv_base& a );
    sc_subref†<T>& operator= ( const sc_subref_r†<T>& a );
    sc_subref†<T>& operator= ( const sc_subref†<T>& a );
    sc_subref†<T>& operator= ( const char* a );
    sc_subref†<T>& operator= ( const bool* a );
    sc_subref†<T>& operator= ( const sc_logic* a );
    sc_subref†<T>& operator= ( const sc_unsigned& a );
    sc_subref†<T>& operator= ( const sc_signed& a );
    sc_subref†<T>& operator= ( const sc_uint_base& a );
    sc_subref†<T>& operator= ( const sc_int_base& a );
    sc_subref†<T>& operator= ( unsigned long a );
    sc_subref†<T>& operator= ( long a );
    sc_subref†<T>& operator= ( unsigned int a );
    sc_subref†<T>& operator= ( int a );
    sc_subref†<T>& operator= ( uint64 a );
    sc_subref†<T>& operator= ( int64 a );

    // Bitwise rotations
    sc_subref†<T>& lrotate( int n );
    sc_subref†<T>& rrotate( int n );

    // Bitwise reverse
    sc_subref†<T>& reverse();

    // Bit selection
    sc_bitref†<sc_subref†<T>> operator[] ( int i );

    // Part selection
    sc_subref†<sc_subref†<T>> operator() ( int hi , int lo );

    sc_subref†<sc_subref†<T>> range( int hi , int lo );

    // Other methods
    void scan( std::istream& = std::cin );
```

```
private:
    // Disabled
    sc_subref*();
};

} // namespace sc_dt
```

### 7.9.8.3 Constraints on usage

Part-select objects shall only be created using the part-select operators of an **sc\_bv\_base** or **sc\_lv\_base** object (or an instance of a class derived from **sc\_bv\_base** or **sc\_lv\_base**) or a part-select or concatenation thereof, as described in 7.2.6.

An application shall not explicitly create an instance of any part-select class.

An application should not declare a reference or pointer to any part-select object.

An rvalue part-select shall not be used to modify the vector with which it is associated.

It is strongly recommended that an application avoid the use of a part-select as the return type of a function because the lifetime of the object to which the part-select refers may not extend beyond the function return statement.

*Example:*

```
sc_dt::sc_subref<sc_bv_base> get_byte(sc_bv_base bv, int pos) {
    return bv(pos+7,pos);    // Unsafe: returned part-select references local variable
}
```

### 7.9.8.4 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to lvalue part-selects. If the size of a data type or string literal operand differs from the part-select word length, truncation or zero-extension shall be used, as described in 7.2.1. If an array of **bool** or array of **sc\_logic** is assigned to a part-select and its number of elements is less than the part-select word length, the value of the part-select shall be undefined.

The default assignment operator for an rvalue part-select is private to prevent its use by an application.

### 7.9.8.5 Explicit type conversion

```
const std::string to_string() const;
const std::string to_string( sc_numrep ) const;
const std::string to_string( sc_numrep , bool ) const;
```

Member function **to\_string** shall convert to an **std::string** representation, as described in 7.2.11. Calling the **to\_string** function with a single argument is equivalent to calling the **to\_string** function with two arguments, where the second argument is **true**. Attempting to call the single or double argument **to\_string** function for a part-select with one or more elements set to the high-impedance or unknown state shall be an error.

Calling the **to\_string** function with no arguments shall create a logic value string with a single '1', '0', 'Z', or 'X' corresponding to each bit. This string shall not be prefixed by "0b" or a leading zero.

bool **is\_01**() const;

Member function **is\_01** shall return **true** only when every element of a part-select has a value of logic 0 or logic 1. If any element has the value high-impedance or unknown, it shall return **false**.

Member functions that return the integer equivalent of the bit representation shall be provided to satisfy the requirements of 7.2.9. Calling any such integer conversion function for an object having one or more bits set to the high-impedance or unknown state shall be an error.

### 7.9.8.6 Bitwise and comparison operators

Operations specified in Table 24 and Table 27 are permitted for all vector part-selects. Operations specified in Table 25 are permitted for lvalue vector part-selects only. The following applies:

- **P** represents an lvalue or rvalue vector part-select.
- **L** represents an lvalue vector part-select.
- **Vi** represents an object of logic vector type **sc\_bv\_base**, **sc\_lv\_base**, *sc\_subref<sub>r<sup>†</sup></sub><T>*, or *sc\_concref<sub>r<sup>†</sup></sub><T1,T2>*, or integer type **int**, **long**, **unsigned int**, **unsigned long**, **sc\_signed**, **sc\_unsigned**, **sc\_int\_base**, or **sc\_uint\_base**.
- **i** represents an object of integer type **int**.
- **A** represents an array object with elements of type **char**, **bool**, or **sc\_logic**.

The operands may also be of any other class that is derived from those just given.

**Table 24—*sc\_subref<sub>r<sup>†</sup></sub><T>* bitwise operations**

| Expression          | Return type      | Operation                                                              |
|---------------------|------------------|------------------------------------------------------------------------|
| <b>P &amp; Vi</b>   | const sc_lv_base | <i>sc_subref<sub>r<sup>†</sup></sub>&lt;T&gt;</i> bitwise and          |
| <b>Vi &amp; P</b>   | const sc_lv_base | <i>sc_subref<sub>r<sup>†</sup></sub>&lt;T&gt;</i> bitwise and          |
| <b>P &amp; A</b>    | const sc_lv_base | <i>sc_subref<sub>r<sup>†</sup></sub>&lt;T&gt;</i> bitwise and          |
| <b>A &amp; P</b>    | const sc_lv_base | <i>sc_subref<sub>r<sup>†</sup></sub>&lt;T&gt;</i> bitwise and          |
| <b>P   Vi</b>       | const sc_lv_base | <i>sc_subref<sub>r<sup>†</sup></sub>&lt;T&gt;</i> bitwise or           |
| <b>Vi   P</b>       | const sc_lv_base | <i>sc_subref<sub>r<sup>†</sup></sub>&lt;T&gt;</i> bitwise or           |
| <b>P   A</b>        | const sc_lv_base | <i>sc_subref<sub>r<sup>†</sup></sub>&lt;T&gt;</i> bitwise or           |
| <b>A   P</b>        | const sc_lv_base | <i>sc_subref<sub>r<sup>†</sup></sub>&lt;T&gt;</i> bitwise or           |
| <b>P ^ Vi</b>       | const sc_lv_base | <i>sc_subref<sub>r<sup>†</sup></sub>&lt;T&gt;</i> bitwise exclusive or |
| <b>Vi ^ P</b>       | const sc_lv_base | <i>sc_subref<sub>r<sup>†</sup></sub>&lt;T&gt;</i> bitwise exclusive or |
| <b>P ^ A</b>        | const sc_lv_base | <i>sc_subref<sub>r<sup>†</sup></sub>&lt;T&gt;</i> bitwise exclusive or |
| <b>A ^ P</b>        | const sc_lv_base | <i>sc_subref<sub>r<sup>†</sup></sub>&lt;T&gt;</i> bitwise exclusive or |
| <b>P &lt;&lt; i</b> | const sc_lv_base | <i>sc_subref<sub>r<sup>†</sup></sub>&lt;T&gt;</i> left-shift           |
| <b>P &gt;&gt; i</b> | const sc_lv_base | <i>sc_subref<sub>r<sup>†</sup></sub>&lt;T&gt;</i> right-shift          |
| <b>~P</b>           | const sc_lv_base | <i>sc_subref<sub>r<sup>†</sup></sub>&lt;T&gt;</i> bitwise complement   |



**Table 25— $sc\_subref^{\dagger}<T>$  bitwise operations**

| Expression      | Return type                    | Operation                                                |
|-----------------|--------------------------------|----------------------------------------------------------|
| $L \&= V_i$     | $sc\_subref\_r^{\dagger}<T>\&$ | $sc\_subref\_r^{\dagger}<T>$ assign bitwise and          |
| $L \&= A$       | $sc\_subref\_r^{\dagger}<T>\&$ | $sc\_subref\_r^{\dagger}<T>$ assign bitwise and          |
| $L \mid= V_i$   | $sc\_subref\_r^{\dagger}<T>\&$ | $sc\_subref\_r^{\dagger}<T>$ assign bitwise or           |
| $L \mid= A$     | $sc\_subref\_r^{\dagger}<T>\&$ | $sc\_subref\_r^{\dagger}<T>$ assign bitwise or           |
| $L \wedge= V_i$ | $sc\_subref\_r^{\dagger}<T>\&$ | $sc\_subref\_r^{\dagger}<T>$ assign bitwise exclusive or |
| $L \wedge= A$   | $sc\_subref\_r^{\dagger}<T>\&$ | $sc\_subref\_r^{\dagger}<T>$ assign bitwise exclusive or |
| $L \ll= i$      | $sc\_subref\_r^{\dagger}<T>\&$ | $sc\_subref\_r^{\dagger}<T>$ assign left-shift           |
| $L \gg= i$      | $sc\_subref\_r^{\dagger}<T>\&$ | $sc\_subref\_r^{\dagger}<T>$ assign right-shift          |

Binary bitwise operators shall return a result with a word length that is equal to the word length of the longest operand.

The left shift operator shall return a result with a word length that is equal to the word length of its part-select operand plus the right (integer) operand. Bits added on the right-hand side of the result shall be set to zero.

The right shift operator shall return a result with a word length that is equal to the word length of its part-select operand. Bits added on the left-hand side of the result shall be set to zero.

It is an error if the right operand of a shift operator is negative.

**Table 26— $sc\_subref\_r^{\dagger}<T>$  comparison operations**

| Expression | Return type | Operation  |
|------------|-------------|------------|
| $P == V_i$ | bool        | test equal |
| $V_i == P$ | bool        | test equal |
| $P == A$   | bool        | test equal |
| $A == P$   | bool        | test equal |

$sc\_subref^{\dagger}<T>\& \mathbf{lrotate}(\text{int } n);$

Member function **lrotate** shall rotate an lvalue part-select  $n$  places to the left.

$sc\_subref^{\dagger}<T>\& \mathbf{rrotate}(\text{int } n);$

Member function **rrotate** shall rotate an lvalue part-select  $n$  places to the right.

*sc\_subref<sup>†</sup><T>* & **reverse**();

Member function **reverse** shall reverse the bit order in an lvalue part-select.

NOTE—An implementation is required to supply overloaded operators on *sc\_subref<sup>†</sup><T>* and *sc\_subref<sup>†</sup><T>* objects to satisfy the requirements of this subclause. It is unspecified whether these operators are members of *sc\_subref<sup>†</sup><T>*, members of *sc\_subref<sup>†</sup><T>*, global operators, or provided in some other way.

### 7.9.8.7 Other member functions

void **scan**( std::istream& is = std::cin );

Member function **scan** shall set the values of the bits referenced by an lvalue part-select by reading the next formatted character string from the specified input stream (see 7.2.10).

void **print**( std::ostream& os = std::cout ) const;

Member function **print** shall print the values of the bits referenced by the part-select to the specified output stream (see 7.2.10).

int **length**() const;

Member function **length** shall return the word length of the part-select (see 7.2.4).

bool **reversed**() const;

Member function **reversed** shall return **true** if the elements of a part-select are in the reverse order to those of its associated vector (if the left-hand index used to form the part-select is less than the right-hand index); otherwise, the return value shall be **false**.

## 7.9.9 Concatenations

### 7.9.9.1 Description

Class template *sc\_concref<sup>†</sup><T1,T2>* represents a concatenation of bits from one or more vector used as an rvalue.

Class template *sc\_concref<sup>†</sup><T1,T2>* represents a concatenation of bits from one or more vector used as an lvalue.

The use of the term vector here includes part-selects and concatenations of bit vectors and logic vectors. The template parameters are the class names of the two vectors used to create the concatenation.

The set of operations that can be performed on a concatenation shall be identical to that of its associated vectors (subject to the constraints that apply to rvalue objects).

### 7.9.9.2 Class definition

```
namespace sc_dt {
```

```
template <class T1, class T2>
```

```
class sc_concref†r†
{
```

```
    public:
```

```
        // Copy constructor
```

```
        sc_concref†r†( const sc_concref†r†<T1,T2>& a );
```

```

// Destructor
virtual ~sc_concref_r†();

// Bit selection
sc_bitref_r†<sc_concref_r†<T1,T2>> operator[] ( int i ) const;

// Part selection
sc_subref_r†<sc_concref_r†<T1,T2>> operator() ( int hi , int lo ) const;

sc_subref_r†<sc_concref_r†<T1,T2>> range( int hi , int lo ) const;

// Reduce functions
sc_logic_value_t and_reduce() const;
sc_logic_value_t nand_reduce() const;
sc_logic_value_t or_reduce() const;
sc_logic_value_t nor_reduce() const;
sc_logic_value_t xor_reduce() const;
sc_logic_value_t xnor_reduce() const;

// Common methods
int length() const;

// Explicit conversions to character string
const std::string to_string() const;
const std::string to_string( sc_numrep ) const;
const std::string to_string( sc_numrep , bool ) const;

// Explicit conversions
int to_int() const;
unsigned int to_uint() const;
long to_long() const;
unsigned long to_ulong() const;
bool is_01() const;

// Other methods
void print( std::ostream& os = std::cout ) const;

private:
// Disabled
sc_concref†();
sc_concref_r†<T1,T2>& operator= ( const sc_concref_r†<T1,T2>& );
};

// -----

template <class T1, class T2>
class sc_concref†
: public sc_concref_r†<T1,T2>
{
public:
// Copy constructor
sc_concref†( const sc_concref†<T1,T2>& a );

```

```

// Assignment operators
template <class T>
sc_concref<T1,T2>& operator= ( const sc_subref_r<T>& a );
template <class T1, class T2>
sc_concref<T1,T2>& operator= ( const sc_concref_r<T1,T2>& a );
sc_concref<T1,T2>& operator= ( const sc_bv_base& a );
sc_concref<T1,T2>& operator= ( const sc_lv_base& a );
sc_concref<T1,T2>& operator= ( const sc_concref<T1,T2>& a );
sc_concref<T1,T2>& operator= ( const char* a );
sc_concref<T1,T2>& operator= ( const bool* a );
sc_concref<T1,T2>& operator= ( const sc_logic* a );
sc_concref<T1,T2>& operator= ( const sc_unsigned& a );
sc_concref<T1,T2>& operator= ( const sc_signed& a );
sc_concref<T1,T2>& operator= ( const sc_uint_base& a );
sc_concref<T1,T2>& operator= ( const sc_int_base& a );
sc_concref<T1,T2>& operator= ( unsigned long a );
sc_concref<T1,T2>& operator= ( long a );
sc_concref<T1,T2>& operator= ( unsigned int a );
sc_concref<T1,T2>& operator= ( int a );
sc_concref<T1,T2>& operator= ( uint64 a );
sc_concref<T1,T2>& operator= ( int64 a );

// Bitwise rotations
sc_concref<T1,T2>& lrotate( int n );
sc_concref<T1,T2>& rrotate( int n );

// Bitwise reverse
sc_concref<T1,T2>& reverse();

// Bit selection
sc_bitref<sc_concref<T1,T2>> operator[] ( int i );

// Part selection
sc_subref<sc_concref<T1,T2>> operator() ( int hi , int lo );

sc_subref<sc_concref<T1,T2>> range( int hi , int lo );

// Other methods
void scan( std::istream& = std::cin );

private:
    // Disabled
    sc_concref();
};

// r-value concatenation operators and functions

template <typename C1, typename C2>
sc_concref_r<C1,C2> operator, ( C1 , C2 );

template <typename C1, typename C2>
sc_concref_r<C1,C2> concat( C1 , C2 );

// l-value concatenation operators and functions

```

```
template <typename C1, typename C2>
sc_concref<C1,C2> operator, ( C1 , C2 );
```

```
template <typename C1, typename C2>
sc_concref<C1,C2> concat( C1 , C2 );
```

```
} // namespace sc_dt
```

### 7.9.9.3 Constraints on usage

Concatenation objects shall only be created using the **concat** function (or **operator**,) according to the rules in 7.2.7. The concatenation arguments shall be objects with a common concatenation base type of **sc\_bv\_base** or **sc\_lv\_base** (or an instance of a class derived from **sc\_bv\_base** or **sc\_lv\_base**) or a part-select or concatenation of them.

An application shall not explicitly create an instance of any concatenation class.

An application should not declare a reference or pointer to any concatenation object.

An rvalue concatenation shall be created when any argument to the **concat** function (or **operator**,) is an rvalue. An rvalue concatenation shall not be used to modify any vector with which it is associated.

It is strongly recommended that an application avoid the use of a concatenation as the return type of a function because the lifetime of the objects to which the concatenation refer may not extend beyond the function return statement.

*Example:*

```
sc_dt::sc_concref_r<sc_bv_base,sc_bv_base> pad(sc_bv_base& bv, char pchar) {
    const sc_bv<4> padword(pchar); // Unsafe: returned concatenation references
                                   // a non-static local variable (padword)
    return concat(bv,padword);
}
```

### 7.9.9.4 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to lvalue concatenations. If the size of a data type or string literal operand differs from the concatenation word length, truncation or zero-extension shall be used, as described in 7.2.1. If an array of **bool** or array of **sc\_logic** is assigned to a concatenation and its number of elements is less than the concatenation word length, the value of the concatenation shall be undefined.

The default assignment operator for an rvalue concatenation shall be declared as private to prevent its use by an application.

### 7.9.9.5 Explicit type conversion

```
const std::string to_string() const;
const std::string to_string( sc_numrep ) const;
const std::string to_string( sc_numrep , bool ) const;
```

Member function **to\_string** shall perform the conversion to an **std::string** representation, as described in 7.2.11. Calling the **to\_string** function with a single argument is equivalent to calling the **to\_string** function with two arguments, where the second argument is **true**. Attempting to call the single or double argument **to\_string** function for a concatenation with one or more elements set to the high-impedance or unknown state shall be an error.

Calling the **to\_string** function with no arguments shall create a logic value string with a single '1', '0', 'Z', or 'X' corresponding to each bit. This string shall not be prefixed by "0b" or a leading zero.

**bool is\_01() const;**

Member function **is\_01** shall return **true** only when every element of a concatenation has a value of logic 0 or logic 1. If any element has the value high-impedance or unknown, it shall return **false**.

Member functions that return the integer equivalent of the bit representation shall be provided to satisfy the requirements of 7.2.9. Calling any such integer conversion function for an object having one or more bits set to the high-impedance or unknown state shall be an error.

### 7.9.9.6 Bitwise and comparison operators

Operations specified in Table 27 and Table 29 are permitted for all vector concatenations; operations specified in Table 28 are permitted for lvalue vector concatenations only. The following applies:

- **C** represents an lvalue or rvalue vector concatenation.
- **L** represents an lvalue vector concatenation.
- **Vi** represents an object of logic vector type **sc\_bv\_base**, **sc\_lv\_base**, *sc\_subref\_r<sup>†</sup><T>*, or *sc\_concref\_r<sup>†</sup><T1,T2>*, or integer type **int**, **long**, **unsigned int**, **unsigned long**, **sc\_signed**, **sc\_unsigned**, **sc\_int\_base**, or **sc\_uint\_base**.
- **i** represents an object of integer type **int**.
- **A** represents an array object with elements of type **char**, **bool**, or **sc\_logic**.

The operands may also be of any other class that is derived from those just given.

**Table 27— $sc\_concref\_r^{\dagger}<T1,T2>$  bitwise operations**

| Expression     | Return type      | Operation                                              |
|----------------|------------------|--------------------------------------------------------|
| $C \& V_i$     | const sc_lv_base | $sc\_concref\_r^{\dagger}<T1,T2>$ bitwise and          |
| $V_i \& C$     | const sc_lv_base | $sc\_concref\_r^{\dagger}<T1,T2>$ bitwise and          |
| $C \& A$       | const sc_lv_base | $sc\_concref\_r^{\dagger}<T1,T2>$ bitwise and          |
| $A \& C$       | const sc_lv_base | $sc\_concref\_r^{\dagger}<T1,T2>$ bitwise and          |
| $C   V_i$      | const sc_lv_base | $sc\_concref\_r^{\dagger}<T1,T2>$ bitwise or           |
| $V_i   C$      | const sc_lv_base | $sc\_concref\_r^{\dagger}<T1,T2>$ bitwise or           |
| $C   A$        | const sc_lv_base | $sc\_concref\_r^{\dagger}<T1,T2>$ bitwise or           |
| $A   C$        | const sc_lv_base | $sc\_concref\_r^{\dagger}<T1,T2>$ bitwise or           |
| $C \wedge V_i$ | const sc_lv_base | $sc\_concref\_r^{\dagger}<T1,T2>$ bitwise exclusive or |
| $V_i \wedge C$ | const sc_lv_base | $sc\_concref\_r^{\dagger}<T1,T2>$ bitwise exclusive or |
| $C \wedge A$   | const sc_lv_base | $sc\_concref\_r^{\dagger}<T1,T2>$ bitwise exclusive or |
| $A \wedge C$   | const sc_lv_base | $sc\_concref\_r^{\dagger}<T1,T2>$ bitwise exclusive or |
| $C \ll i$      | const sc_lv_base | $sc\_concref\_r^{\dagger}<T1,T2>$ left-shift           |
| $C \gg i$      | const sc_lv_base | $sc\_concref\_r^{\dagger}<T1,T2>$ right-shift          |
| $\sim C$       | const sc_lv_base | $sc\_concref\_r^{\dagger}<T1,T2>$ bitwise complement   |

**Table 28— $sc\_concref^{\dagger}<T1,T2>$  bitwise operations**

| Expression      | Return type                      | Operation                                                  |
|-----------------|----------------------------------|------------------------------------------------------------|
| $L \&= V_i$     | $sc\_concref^{\dagger}<T1,T2>\&$ | $sc\_concref^{\dagger}<T1,T2>$ assign bitwise and          |
| $L \&= A$       | $sc\_concref^{\dagger}<T1,T2>\&$ | $sc\_concref^{\dagger}<T1,T2>$ assign bitwise and          |
| $L  = V_i$      | $sc\_concref^{\dagger}<T1,T2>\&$ | $sc\_concref^{\dagger}<T1,T2>$ assign bitwise or           |
| $L  = A$        | $sc\_concref^{\dagger}<T1,T2>\&$ | $sc\_concref^{\dagger}<T1,T2>$ assign bitwise or           |
| $L \wedge= V_i$ | $sc\_concref^{\dagger}<T1,T2>\&$ | $sc\_concref^{\dagger}<T1,T2>$ assign bitwise exclusive or |
| $L \wedge= A$   | $sc\_concref^{\dagger}<T1,T2>\&$ | $sc\_concref^{\dagger}<T1,T2>$ assign bitwise exclusive or |
| $L \ll= i$      | $sc\_concref^{\dagger}<T1,T2>\&$ | $sc\_concref^{\dagger}<T1,T2>$ assign left-shift           |
| $L \gg= i$      | $sc\_concref^{\dagger}<T1,T2>\&$ | $sc\_concref^{\dagger}<T1,T2>$ assign right-shift          |

Binary bitwise operators shall return a result with a word length that is equal to the word length of the longest operand.

The left shift operator shall return a result with a word length that is equal to the word length of its concatenation operand plus the right (integer) operand. Bits added on the right-hand side of the result shall be set to zero.

The right shift operator shall return a result with a word length that is equal to the word length of its concatenation operand. Bits added on the left-hand side of the result shall be set to zero.

**Table 29— $sc\_concref\_r^{\dagger}<T1,T2>$  comparison operations**

| Expression | Return type | Operation  |
|------------|-------------|------------|
| $C == V_i$ | bool        | test equal |
| $V_i == C$ | bool        | test equal |
| $C == A$   | bool        | test equal |
| $A == C$   | bool        | test equal |

$sc\_concref^{\dagger}<T1,T2>\& \mathbf{lrotate}(\text{int } n);$

Member function **lrotate** shall rotate an lvalue part-select  $n$  places to the left.

$sc\_concref^{\dagger}<T1,T2>\& \mathbf{rrotate}(\text{int } n);$

Member function **rrotate** shall rotate an lvalue part-select  $n$  places to the right.

$sc\_concref^{\dagger}<T1,T2>\& \mathbf{reverse}();$

Member function **reverse** shall reverse the bit order in an lvalue part-select.

NOTE—An implementation is required to supply overloaded operators on  $sc\_concref\_r^{\dagger}<T1,T2>$  and  $sc\_concref^{\dagger}<T1,T2>$  objects to satisfy the requirements of this subclause. It is unspecified whether these operators are members of  $sc\_concref\_r^{\dagger}<T1,T2>$ , members of  $sc\_concref^{\dagger}<T1,T2>$ , global operators, or provided in some other way.

### 7.9.9.7 Other member functions

`void scan( std::istream& is = std::cin );`

Member function **scan** shall set the values of the bits referenced by an lvalue concatenation by reading the next formatted character string from the specified input stream (see 7.2.10).

`void print( std::ostream& os = std::cout ) const;`

Member function **print** shall print the values of the bits referenced by the concatenation to the specified output stream (see 7.2.10).

`int length() const;`

Member function **length** shall return the word length of the concatenation (see 7.2.4).



### 7.9.9.8 Function concat and operator,

```
template <typename C1, typename C2>
sc_concref_r†<C1,C2> operator, ( C1 , C2 );
```

```
template <typename C1, typename C2>
sc_concref_r†<C1,C2> concat( C1 , C2 );
```

```
template <typename C1, typename C2>
sc_concref†<C1,C2> operator, ( C1 , C2 );
```

```
template <typename C1, typename C2>
sc_concref†<C1,C2> concat( C1 , C2 );
```

Explicit template specializations of function **concat** and **operator**, shall be provided for all permitted concatenations. Attempting to concatenate any two objects that do not have an explicit template specialization for function **concat** or **operator**, defined shall be an error.

A template specialization for rvalue concatenations shall be provided for all combinations of concatenation argument types C1 and C2. Argument C1 has a type in the following list:

```
sc_bitref_r†<T>
sc_subref_r†<T>
sc_concref_r†<T1,T2>
const sc_bv_base&
const sc_lv_base&
```

Argument C2 has one of the types just given or a type in the following list:

```
sc_bitref†<T>
sc_subref†<T>
sc_concref†<T1,T2>
sc_bv_base&
sc_lv_base&
```

Additional template specializations for rvalue concatenations shall be provided for the cases where a single argument has type **bool**, **const char\***, or **const sc\_logic&**. This argument shall be implicitly converted to an equivalent single-bit **const sc\_lv\_base** object.

A template specialization for lvalue concatenations shall be provided for all combinations of concatenation argument types C1 and C2, where each argument has a type in the following list:

```
sc_bitref†<T>
sc_subref†<T>
sc_concref†<T1,T2>
sc_bv_base&
sc_lv_base&
```

## 7.10 Fixed-point types

This subclause describes the fixed-point types and the operations and conventions imposed by these types.

### 7.10.1 Fixed-point representation

In the SystemC binary fixed-point representation, a number shall be represented by a sequence of bits with a specified position for the binary-point. Bits to the left of the binary point shall represent the integer part of the number, and bits to the right of the binary point shall represent the fractional part of the number.

A SystemC fixed-point type shall be characterized by the following:

- The word length (**wl**), which shall be the total number of bits in the number representation.
- The integer word length (**iwl**), which shall be the number of bits in the integer part (the position of the binary point relative to the left-most bit).
- The bit encoding (which shall be signed, two's compliment, or unsigned).

The right-most bit of the number shall be the least significant bit (LSB), and the left-most bit shall be the most significant bit (MSB).

The binary point may be located outside of the data bits. That is, the binary point may be a number of bit positions to the right of the LSB or it may be a number of bit positions to the left of the MSB.

The fixed-point representation can be interpreted according to the following three cases:

- **wl < iwl**  
There are (**iwl-wl**) zeros between the LSB and the binary point. See index 1 in Table 30 for an example of this case.
- **0 <= iwl <= wl**  
The binary point is contained within the bit representation. See index 2, 3, 4, and 5 in Table 30 for examples of this case.
- **iwl < 0**  
There are (**-iwl**) sign-extended bits between the binary point and the MSB. For an unsigned type, the sign-extended bits are zero. For a signed type, the extended bits repeat the MSB. See index 6 and 7 in Table 30 for examples of this case.

The MSB in the fixed-point representation of a signed type shall be the sign bit. The sign bit may be behind the binary point.

The range of values for a signed fixed-point format shall be given by the following:

$$[-2^{(iwl-1)}_f, 2^{(iwl-1)} - 2^{-(wl-iwl)}]$$

The range of values for a unsigned fixed-point format shall be given by the following:

$$[0, 2^{(iwl)} - 2^{-(wl-iwl)}]$$

**Table 30—Examples of fixed-point formats**

| Index | wl | iwl | Fixed-point representation* | Range signed      | Ranged unsigned |
|-------|----|-----|-----------------------------|-------------------|-----------------|
| 1     | 5  | 7   | xxxxx00.                    | [-64,60]          | [0,124]         |
| 2     | 5  | 5   | xxxxx.                      | [-16,15]          | [0,31]          |
| 3     | 5  | 3   | xxx.xx                      | [-4,3.75]         | [0,7.75]        |
| 4     | 5  | 1   | x.xxxx                      | [-1,0.9375]       | [0,1.9375]      |
| 5     | 5  | 0   | .xxxxx                      | [-0.5,0.46875]    | [0,0.96875]     |
| 6     | 5  | -2  | .ssxxxxx                    | [0.125,0.1171875] | [0,0.2421875]   |
| 7     | 1  | -1  | .sx                         | [-0.25,0]         | [0,0.25]        |

\* x is an arbitrary binary digit, 0, or 1. s is a sign-extended digit, 0, or 1,

### 7.10.2 Fixed-point type conversion

Fixed-point type conversion (conversion of a value to a specific fixed-point representation) shall be performed whenever a value is assigned to a fixed-point type variable (including initialization).

If the magnitude of the value is outside the range of the fixed-point representation, or the value has greater precision than the fixed-point representation provides, it shall be mapped (converted) to a value that can be represented. This conversion shall be performed in two steps:

- If the value is within range but has greater precision (it is between representable values), quantization shall be performed to reduce the precision.
- If the magnitude of the value is outside the range, overflow handling shall be performed to reduce the magnitude.

If the target fixed-point representation has greater precision, the additional least significant bits shall be zero extended. If the target fixed-point representation has a greater range, sign extension or zero extension shall be performed for signed and unsigned fixed-point types, respectively, to extend the representation of their most significant bits.

Multiple quantization modes (distinct quantization characteristics) and multiple overflow modes (distinct overflow characteristics) are defined (see 7.10.9.1 and 7.10.9.9).

### 7.10.3 Fixed-point data types

This subclause describes the classes that are provided to represent fixed-point values.

### 7.10.3.1 Finite-precision fixed-point types

The following finite- and variable-precision fixed-point data types shall be provided:

```
sc_fixed<wl,iwl,q_mode,o_mode,n_bits>
sc_ufixed<wl,iwl,q_mode,o_mode,n_bits>
sc_fix
sc_ufix
sc_fxval
```

These types shall be parameterized as to the fixed-point representation (**wl**, **iwl**) and fixed-point conversion modes (**q\_mode**, **o\_mode**, **n\_bits**). The declaration of a variable of one of these types shall specify the values for these parameters. The type parameter values of a variable shall not be modified after the variable declaration. Any data value assigned to the variable shall be converted to specified representation (with the specified word length and binary point location) with the specified quantization and overflow processing (**q\_mode**, **o\_mode**, **n\_bits**) applied if required.

The finite-precision fixed-point types have a common base class **sc\_fxnum**. An application or implementation shall not directly create an object of type **sc\_fxnum**. A reference or pointer to class **sc\_fxnum** may be used to access an object of any type derived from **sc\_fxnum**.

The type **sc\_fxval** is a variable-precision type. A variable of type **sc\_fxval** may store a fixed-point value of arbitrary width and binary point location. A value assigned to a **sc\_fxval** variable shall be stored without a loss of precision or magnitude (the value shall not be modified by quantization or overflow handling).

Types **sc\_fixed**, **sc\_fix**, and **sc\_fxval** shall have a signed (two's complement) representation. Types **sc\_ufixed** and **sc\_ufix** have an unsigned representation.

A fixed-point variable that is declared without an initial value shall be uninitialized. Uninitialized variables may be used wherever the use of an initialized variable is permitted. The result of an operation on an uninitialized variable shall be undefined.

### 7.10.3.2 Limited-precision fixed-point types

The following limited-precision versions of the fixed-point types shall be provided:

```
sc_fixed_fast<wl,iwl,q_mode,o_mode,n_bits>
sc_ufixed_fast<wl,iwl,q_mode,o_mode,n_bits>
sc_fix_fast
sc_ufix_fast
sc_fxval_fast
```

The limited-precision types shall use the same semantics as the finite-precision fixed-point types. Finite-precision and limited-precision types may be mixed freely in expressions. A variable of a limited-precision type shall be a legal replacement in any expression where a variable of the corresponding finite-precision fixed-point type is expected.

The limited-precision fixed-point value shall be held in an implementation-dependent native C++ floating-point type. An implementation shall provide a minimum length of 53 bits to represent the mantissa.

NOTE—For bit-true behavior with the limited-precision types, the word length of the result of any operation or expression shall not exceed 53 bits.

#### 7.10.4 Fixed-point expressions and operations

Fixed-point operations shall be performed using variable-precision fixed-point values; that is, the evaluation of a fixed-point operator shall proceed as follows (except as noted below for specific operators):

- The operands shall be converted (promoted) to variable-precision fixed-point values.
- The operation shall be performed, computing a variable-precision fixed-point result. The result shall be computed so that there is no loss of precision or magnitude (that is, sufficient bits are computed to precisely represent the result).

The right-hand side of a fixed-point assignment shall be evaluated as a variable-precision fixed-point value that is converted to the fixed-point representation specified by the target of the assignment.

If all the operands of a fixed-point operation are limited-precision types, a limited-precision operation shall be performed. This operation shall use limited variable-precision fixed-point values (**sc\_fxval\_fast**) and the result shall be a limited variable-precision fixed-point value.

The right operand of a fixed-point shift operation (the shift amount) shall be of type **int**. If a fixed-point shift operation is called with a fixed-point value for the right operand, the fractional part of the value shall be truncated (no quantization).

The result of the equality and relational operators shall be type **bool**.

Fixed-point operands of a bitwise operator shall be of a finite- or limited-precision type (they shall not be variable precision). Furthermore, both operands of a binary bitwise operator shall have the same sign representation (both signed or both unsigned). The result of a fixed-point bitwise operation shall be either **sc\_fix**, or **sc\_ufix** (or **sc\_fix\_fast**, **sc\_ufix\_fast**), depending on the sign representation of the operands. For binary operators, the two operands shall be aligned at the binary point. The operands shall be temporarily extended (if necessary) to have the same integer word length and fractional word length. The result shall have the same integer and fractional word lengths as the temporarily extended operands.

The remainder operator (%) is not supported for fixed-point types.

The permitted operators are given in Table 31. The following applies:

- **A** represents a fixed-point object.
- **B** and **C** represent appropriate numeric values or objects.
- **s1**, **s2**, **s3** represent signed finite- or limited-precision fixed-point objects.
- **u1**, **u2**, **u3** represent unsigned finite- or limited-precision fixed-point objects.

**Table 31—Fixed-point arithmetic and bitwise functions**

| Expression     | Operation                      |
|----------------|--------------------------------|
| $A = B + C;$   | Addition with assignment       |
| $A = B - C;$   | Subtraction with assignment    |
| $A = B * C;$   | Multiplication with assignment |
| $A = B / C;$   | Division with assignment       |
| $A = B \ll i;$ | Left shift with assignment     |

**Table 31—Fixed-point arithmetic and bitwise functions (continued)**

| Expression           | Operation                                                  |
|----------------------|------------------------------------------------------------|
| $A = B \gg i;$       | Right shift with assignment                                |
| $s1 = s2 \& s3;$     | Bitwise and with assignment for signed operands            |
| $s1 = s2   s3;$      | Bitwise or with assignment for signed operands             |
| $s1 = s2 \wedge s3;$ | Bitwise exclusive-or with assignment for signed operands   |
| $u1 = u2 \& u3;$     | Bitwise and with assignment for unsigned operands          |
| $u1 = u2   u3;$      | Bitwise or with assignment for unsigned operands           |
| $u1 = u2 \wedge u3;$ | Bitwise exclusive-or with assignment for unsigned operands |

The operands of arithmetic fixed-point operations may be combinations of the types listed in Table 32, Table 33, Table 34, and Table 35.

The addition operations specified in Table 32 are permitted for finite-precision fixed-point objects. The following applies:

- **F, F1, F2** represent objects derived from type **sc\_fxnum**.
- **n** represents an object of numeric type **int**, **long**, **unsigned int**, **unsigned long**, **double**, **sc\_signed**, **sc\_unsigned**, **sc\_int\_base**, **sc\_uint\_base**, **sc\_fxval**, **sc\_fxval\_fast**, or an object derived from **sc\_fxnum\_fast** or a numeric string literal.

The operands may also be of any other class that is derived from those just given.

**Table 32—Finite-precision fixed-point addition operations**

| Expression     | Operation                          |
|----------------|------------------------------------|
| $F = F1 + F2;$ | sc_fxnum addition, sc_fxnum assign |
| $F1 += F2;$    | sc_fxnum assign addition           |
| $F1 = F2 + n;$ | sc_fxnum addition, sc_fxnum assign |
| $F1 = n + F2;$ | sc_fxnum addition, sc_fxnum assign |
| $F += n;$      | sc_fxnum assign addition           |

The addition operations specified in Table 33 are permitted for variable-precision fixed-point objects. The following applies:

- **V, V1, V2** represent objects of type **sc\_fxval**.
- **n** represents an object of numeric type **int**, **long**, **unsigned int**, **unsigned long**, **double**, **sc\_signed**, **sc\_unsigned**, **sc\_int\_base**, **sc\_uint\_base**, **sc\_fxval\_fast**, or an object derived from **sc\_fxnum\_fast** or a numeric string literal.

The operands may also be of any other class that is derived from those just given.

**Table 33—Variable-precision fixed-point addition operations**

| Expression     | Operation                             |
|----------------|---------------------------------------|
| $V = V1 + V2;$ | sc_fxval addition,<br>sc_fxval assign |
| $V1 += V2;$    | sc_fxval assign addition              |
| $V1 = V2 + n;$ | sc_fxval addition,<br>sc_fxval assign |
| $V1 = n + V2;$ | sc_fxval addition,<br>sc_fxval assign |
| $V += n;$      | sc_fxval assign addition              |

The addition operations specified in Table 34 are permitted for limited-precision fixed-point objects. The following applies:

- **F, F1, F2** represent objects derived from type **sc\_fxnum\_fast**.
- **n** represents an object of numeric type **int**, **long**, **unsigned int**, **unsigned long**, **double**, **sc\_signed**, **sc\_unsigned**, **sc\_int\_base**, **sc\_uint\_base**, or **sc\_fxval\_fast**, or a numeric string literal.

The operands may also be of any other class that is derived from those just given.

**Table 34—Limited-precision fixed-point addition operations**

| Expression     | Operation                                       |
|----------------|-------------------------------------------------|
| $F = F1 + F2;$ | sc_fxnum_fast addition,<br>sc_fxnum_fast assign |
| $F1 += F2;$    | sc_fxnum_fast assign addition                   |
| $F1 = F2 + n;$ | sc_fxnum_fast addition,<br>sc_fxnum_fast assign |
| $F1 = n + F2;$ | sc_fxnum_fast addition,<br>sc_fxnum_fast assign |
| $F += n;$      | sc_fxnum_fast assign addition                   |

The addition operations specified in Table 35 are permitted for limited variable-precision fixed-point objects. The following applies:

- **V, V1, V2** represent objects of type **sc\_fxval\_fast**.
- **n** represents an object of numeric type **int**, **long**, **unsigned int**, **unsigned long**, **double**, **sc\_signed**, **sc\_unsigned**, **sc\_int\_base**, or **sc\_uint\_base**, or a numeric string literal.

The operands may also be of any other class that is derived from those just given.

**Table 35—Limited variable-precision fixed-point addition operations**

| Expression     | Operation                                       |
|----------------|-------------------------------------------------|
| $V = V1 + V2;$ | sc_fxval_fast addition,<br>sc_fxval_fast assign |
| $V1 += V2;$    | sc_fxval_fast assign addition                   |
| $V1 = V2 + n;$ | sc_fxval_fast addition,<br>sc_fxval_fast assign |
| $V1 = n + V2;$ | sc_fxval_fast addition,<br>sc_fxval_fast assign |
| $V += n;$      | sc_fxval_fast assign addition                   |

Subtraction, multiplication, and division operations are also permitted with the same combinations of operand types as listed in Table 32, Table 33, Table 34, and Table 35.

#### 7.10.5 Bit and part selection

Bit and part selection shall be supported for the fixed-point types, as described in 7.2.5 and 7.2.6. They are not supported for the variable-precision fixed-point types **sc\_fxval** or **sc\_fxval\_fast**.

If the left-hand index of a part-select is less than the right-hand index, the bit order of the part-select shall be reversed.

A part-select may be created with an unspecified range (the **range** function or **operator()** is called with no arguments). In this case, the part-select shall have the same word length and same value as its associated fixed-point object.

#### 7.10.6 Variable-precision fixed-point value limits

In some cases, such as division, using variable precision could lead to infinite word lengths. An implementation should provide an appropriate mechanism to define the maximum permitted word length of a variable-precision value and to detect when this maximum word length is reached.

The action taken by an implementation when a variable-precision value reaches its maximum word length is undefined. The result of any operation that causes a variable-precision value to reach its maximum word length shall be the implementation-dependent representable value nearest to the ideal (infinite precision) result.

#### 7.10.7 Fixed-point word length and mode

The default word length, quantization mode, and saturation mode of a fixed-point type shall be set by the fixed-point type parameter (**sc\_fxtype\_param**) in context at the point of construction as described in 7.2.3. The fixed-point type parameter shall have a field corresponding to the fixed-point representation (**wl**, **iwl**) and fixed-point conversion modes (**q\_mode**, **o\_mode**, **n\_bits**). Default values for these fields shall be defined according to Table 36.



**Table 36—Built-in default values**

| Parameter | Value   |
|-----------|---------|
| wl        | 32      |
| iwl       | 32      |
| q_mode    | SC_TRN  |
| o_mode    | SC_WRAP |
| n_bits    | 0       |

The behavior of a fixed-point object in arithmetic operations may be set to emulate that of a floating-point variable by the *floating-point cast switch* in context at its point of construction. A floating-point cast switch shall be brought into context by creating a *floating-point cast context* object. **sc\_fxcast\_switch** and **sc\_fxcast\_context** shall be used to create floating-point cast switches and floating-point cast contexts, respectively (see 7.11.5 and 7.11.6).

A global floating-point cast context stack shall manage floating-point cast contexts using the same semantics as the length context stack described in 7.2.3.

A floating-point cast switch may be initialized to the value SC\_ON or SC\_OFF. These shall cause the arithmetic behavior to be fixed-point or floating-point, respectively. A default floating-point context with the value SC\_ON shall be defined.

*Example:*

```
sc_fxtype_params fxt(32,16);
sc_fxtype_context fcxt(fxt);

sc_fix A,B,res;           // wl = 32, iwl = 16
A = 10.0;
B = 0.1;
res = A * B;              // res = .999908447265625

sc_fxcast_switch fxs(SC_OFF);
sc_fxcast_context fcxt(fxs);
sc_fix C,D;              // Floating-point behavior
C = 10.0;
D = 0.1;
res = C * D;              // res = 1
```

### 7.10.7.1 Reading parameter settings

The following functions are defined for every finite-precision fixed-point object and limited-precision fixed-point object and shall return its current parameter settings (at runtime).

const sc\_fxcast\_switch& **cast\_switch()** const;

Member function **cast\_switch** shall return the cast switch parameter.

int **iw()** const;

Member function **iw** shall return the integer word-length parameter.

int **n\_bits()** const;

Member function **n\_bits** shall return the number of saturated bits parameter.

sc\_o\_mode **o\_mode()** const;

Member function **o\_mode** shall return the overflow mode parameter using the enumerated type **sc\_o\_mode**, defined as follows:

```
enum sc_o_mode
{
    SC_SAT,           // Saturation
    SC_SAT_ZERO,      // Saturation to zero
    SC_SAT_SYM,       // Symmetrical saturation
    SC_WRAP,          // Wrap-around (*)
    SC_WRAP_SM        // Sign magnitude wrap-around (*)
};
```

sc\_q\_mode **q\_mode()** const;

Member function **q\_mode** shall return the quantization mode parameter using the enumerated type **sc\_q\_mode**, defined as follows:

```
enum sc_q_mode
{
    SC_RND,           // Rounding to plus infinity
    SC_RND_ZERO,      // Rounding to zero
    SC_RND_MIN_INF,   // Rounding to minus infinity
    SC_RND_INF,       // Rounding to infinity
    SC_RND_CONV,      // Convergent rounding
    SC_TRN,           // Truncation
    SC_TRN_ZERO       // Truncation to zero
};
```

const sc\_fxtype\_params& **type\_params()** const;

Member function **type\_params** shall return the type parameters.

int **w()** const;

Member function **w** shall return the total word-length parameter.

### 7.10.7.2 Value attributes

The following functions are defined for every fixed-point object and shall return its current value attributes.

bool **is\_neg()** const;

Member function **is\_neg** shall return **true** if the object holds a negative value; otherwise, the return value shall be **false**.

bool **is\_zero()** const;

Member function **is\_zero** shall return **true** if the object holds a zero value; otherwise, the return value shall be **false**.

bool **overflow\_flag()** const;

Member function **overflow\_flag** shall return **true** if the last write action on this objects caused overflow; otherwise, the return value shall be **false**.

bool **quantization\_flag()** const;

Member function **quantization\_flag** shall return **true** if the last write action on this object caused quantization; otherwise, the return value shall be **false**.

The following function is defined for every finite-precision fixed-point object and shall return its current value:

const sc\_fxval **value()** const;

The following function is defined for every limited-precision fixed-point object and shall return its current value:

const sc\_fxval\_fast **value()** const;

### 7.10.8 Conversions to character string

Conversion to character string of the fixed-point types shall be supported by the **to\_string** method, as described in 7.3.

The **to\_string** method for fixed-point types may be called with an additional argument to specify the string format. This argument shall be of enumerated type **sc\_fmt** and shall always be at the right-hand side of the argument list.

enum **sc\_fmt** { SC\_F, SC\_E };

The default value for **fmt** shall be SC\_F for the finite- and limited-precision fixed-point types. For types **sc\_fxval** and **sc\_fxval\_fast**, the default value for **fmt** shall be SC\_E.

The selected format shall give different character strings only when the binary point is not located within the *w*/bits. In that case, either sign extension (MSB side) or zero extension (LSB side) shall be done (SC\_F format), or exponents shall be used (SC\_E format).

In conversion to SC\_DEC number representation or conversion from a variable-precision variable, only those characters necessary to uniquely represent the value shall be generated. In converting the value of a finite- or limited-precision variable to a binary, octal, or hex representation, the number of characters used shall be determined by the integer and fractional widths (iwl, fwl) of the variable (with sign or zero extension as needed).

*Example:*

```
sc_fixed<7,4> a = -1.5;
a.to_string(SC_DEC);           // -1.5
a.to_string(SC_BIN);           // 0b1110.100
sc_fxval b = -1.5;
b.to_string(SC_BIN);           // 0b10.1
sc_fixed<4,6> c = 20;
c.to_string(SC_BIN,false,SC_F); // 010100
c.to_string(SC_BIN,false,SC_E); // 0101e+2
```

#### 7.10.8.1 String shortcut methods

Four shortcut methods to the **to\_string** method shall be provided for frequently used combinations of arguments. The shortcut methods are listed in Table 37.

**Table 37—Shortcut methods**

| Shortcut method | Number representation |
|-----------------|-----------------------|
| to_dec()        | SC_DEC                |
| to_bin()        | SC_BIN                |
| to_oct()        | SC_OCT                |
| to_hex()        | SC_HEX                |

The shortcut methods shall use the default string formatting.

*Example:*

```
sc_fixed<4,2> a = -1;
a.to_dec();           // Returns std::string with value "-1"
a.to_bin();           // Returns std::string with value "0b11.00"
```

#### 7.10.8.2 Bit-pattern string conversion

Bit-pattern strings may be assigned to fixed-point part-selects. The result of assigning a bit-pattern string to a fixed-point object (except using a part-select) is undefined.

If the number of characters in the bit-pattern string is less than the part-select word length, the string shall be zero extended at its left-hand side to the part-select word length.

### 7.10.9 Finite word-length effects

The following subclauses describe the overflow and quantization modes of SystemC.

#### 7.10.9.1 Overflow modes

Overflow shall occur when the magnitude of a value being assigned to a limited-precision variable exceeds the fixed-point representation. In SystemC, specific overflow modes shall be available to control the mapping to a representable value.

The mutually exclusive overflow modes listed in Table 38 shall be provided. The default overflow mode shall be SC\_WRAP. When using a wrap-around overflow mode, the number of saturated bits (n\_bits) shall by default be set to 0 but can be modified.

**Table 38—Overflow modes**

| Overflow mode               | Name        |
|-----------------------------|-------------|
| Saturation                  | SC_SAT      |
| Saturation to zero          | SC_SAT_ZERO |
| Symmetrical saturation      | SC_SAT_SYM  |
| Wrap-around *               | SC_WRAP     |
| Sign magnitude wrap-around* | SC_WRAP_SM  |

\* with 0 or n\_bits saturated bits (n\_bits > 0). The default value for n\_bits is 0.

In the following subclauses, each of the overflow modes is explained in more detail. A figure is given to explain the behavior graphically. The x axis shows the input values and the y axis represents the output values. Together they determine the overflow mode.

To facilitate the explanation of each overflow mode, the concepts MIN and MAX are used:

- In the case of signed representation, MIN is the lowest (negative) number that may be represented; MAX is the highest (positive) number that may be represented with a certain number of bits. A value  $x$  shall lie in the range:  
$$-2^{n-1} (= \text{MIN}) \leq x \leq (2^{n-1} - 1) (= \text{MAX})$$
where  $n$  indicates the number of bits.
- In the case of unsigned representation, MIN shall equal 0 and MAX shall equal  $2^n - 1$ , where  $n$  indicates the number of bits.

### 7.10.9.2 Overflow for signed fixed-point numbers

The following template contains a signed fixed-point number before and after an overflow mode has been applied and a number of flags. The flags are explained below the template. The flags between parentheses indicate additional optional properties of a bit.

|        |           |          |          |          |           |           |             |              |          |          |          |          |          |          |          |          |           |
|--------|-----------|----------|----------|----------|-----------|-----------|-------------|--------------|----------|----------|----------|----------|----------|----------|----------|----------|-----------|
| Before | <i>x</i>  | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i>  | <i>x</i>  | <i>x</i>    | <i>x</i>     | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i>  |
| After: |           |          |          |          |           |           | <i>x</i>    | <i>x</i>     | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i>  |
| Flags: | <i>sD</i> | <i>D</i> | <i>D</i> | <i>D</i> | <i>ID</i> | <i>sR</i> | <i>R(N)</i> | <i>R(IN)</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>IR</i> |

The following flags and symbols are used in the template just given and in Table 39:

- *x* represents a binary digit (0 or 1).
- *sD* represents a sign bit before overflow handling.
- *D* represents deleted bits.
- *ID* represents the least significant deleted bit.
- *sR* represents the bit on the MSB position of the result number. For the SC\_WRAP\_SM, 0 and SC\_WRAP\_SM, 1 modes, a distinction is made between the original value (*sRo*) and the new value (*sRn*) of this bit.
- *N* represents the saturated bits. Their number is equal to the *n\_bits* argument minus 1. They are always taken after the sign bit of the result number. The *n\_bits* argument is only taken into account for the SC\_WRAP and SC\_WRAP\_SM overflow modes.
- *IN* represents the least significant saturated bit. This flag is only relevant for the SC\_WRAP and SC\_WRAP\_SM overflow modes. For the other overflow modes, these bits are treated as R-bits. For the SC\_WRAP\_SM, *n\_bits* > 1 mode, *INo* represents the original value of this bit.
- *R* represents the remaining bits.
- *IR* represents the least significant remaining bit.

Overflow shall occur when the value of at least one of the deleted bits (*sD*, *D*, *ID*) is not equal to the original value of the bit on the MSB position of the result (*sRo*).

Table 39 shows how a signed fixed-point number shall be cast (in case there is an overflow) for each of the possible overflow modes. The operators used in the table are “!” for a bitwise negation, and “^” for a bitwise exclusive-OR.

**Table 39—Overflow handling for signed fixed-point numbers**

| Overflow mode                       | Result                                                                                                                                                                                                                                                                                                   |                                         |                                         |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|-----------------------------------------|
|                                     | Sign bit ( <i>sR</i> )                                                                                                                                                                                                                                                                                   | Saturated bits ( <i>N</i> , <i>IN</i> ) | Remaining bits ( <i>R</i> , <i>IR</i> ) |
| SC_SAT                              | <i>sD</i>                                                                                                                                                                                                                                                                                                |                                         | ! <i>sD</i>                             |
|                                     | The result number gets the sign bit of the original number. The remaining bits shall get the inverse value of the sign bit.                                                                                                                                                                              |                                         |                                         |
| SC_SAT_ZERO                         | 0                                                                                                                                                                                                                                                                                                        |                                         | 0                                       |
|                                     | All bits shall be set to zero.                                                                                                                                                                                                                                                                           |                                         |                                         |
| SC_SAT_SYM                          | <i>sD</i>                                                                                                                                                                                                                                                                                                |                                         | ! <i>sD</i> ,                           |
|                                     | The result number shall get the sign bit of the original number. The remaining bits shall get the inverse value of the sign bit, except the least significant remaining bit, which shall be set to one.                                                                                                  |                                         |                                         |
| SC_WRAP, ( <i>n_bits</i> =) 0       | <i>sR</i>                                                                                                                                                                                                                                                                                                |                                         | <i>x</i>                                |
|                                     | All bits except for the deleted bits shall be copied to the result.                                                                                                                                                                                                                                      |                                         |                                         |
| SC_WRAP, ( <i>n_bits</i> =) 1       | <i>sD</i>                                                                                                                                                                                                                                                                                                |                                         | <i>x</i>                                |
|                                     | The result number shall get the sign bit of the original number. The remaining bits shall be copied from the original number.                                                                                                                                                                            |                                         |                                         |
| SC_WRAP, <i>n_bits</i> > 1          | <i>sD</i>                                                                                                                                                                                                                                                                                                | ! <i>sD</i>                             | <i>x</i>                                |
|                                     | The result number shall get the sign bit of the original number. The saturated bits shall get the inverse value of the sign bit of the original number. The remaining bits shall be copied from the original number.                                                                                     |                                         |                                         |
| SC_WRAP_SM,<br>( <i>n_bits</i> =) 0 | <i>ID</i>                                                                                                                                                                                                                                                                                                |                                         | $x \wedge sRo \wedge sRn$               |
|                                     | The sign bit of the result number shall get the value of the least significant deleted bit. The remaining bits shall be XORed with the original and the new value of the sign bit of the result.                                                                                                         |                                         |                                         |
| SC_WRAP_SM,<br>( <i>n_bits</i> =) 1 | <i>sD</i>                                                                                                                                                                                                                                                                                                |                                         | $x \wedge sRo \wedge sRn$               |
|                                     | The result number shall get the sign bit of the original number. The remaining bits shall be XORed with the original and the new value of the sign bit of the result.                                                                                                                                    |                                         |                                         |
| SC_WRAP_SM,<br><i>n_bits</i> > 1    | <i>sD</i>                                                                                                                                                                                                                                                                                                | ! <i>sD</i>                             | $x \wedge INo \wedge ! sD$              |
|                                     | The result number shall get the sign bit of the original number. The saturated bits shall get the inverse value of the sign bit of the original number. The remaining bits shall be XORed with the original value of the least significant saturated bit and the inverse value of the original sign bit. |                                         |                                         |

### 7.10.9.3 Overflow for unsigned fixed-point numbers

The following template contains an unsigned fixed-point number before and after an overflow mode has been applied and a number of flags. The flags are explained below the template.

|        |          |          |          |          |           |             |             |              |          |          |          |          |          |          |          |          |          |
|--------|----------|----------|----------|----------|-----------|-------------|-------------|--------------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
| Before | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i>  | <i>x</i>    | <i>x</i>    | <i>x</i>     | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> |
| After: |          |          |          |          |           |             | <i>x</i>    | <i>x</i>     | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> |
| Flags: | <i>D</i> | <i>D</i> | <i>D</i> | <i>D</i> | <i>ID</i> | <i>R(N)</i> | <i>R(N)</i> | <i>R(IN)</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> |

The following flags and symbols are used in the template just given and in Table 40:

- *x* represents an binary digit (0 or 1).
- *D* represents deleted bits.
- *ID* represents the least significant deleted bit.
- *N* represents the saturated bits. Their number is equal to the *n\_bits* argument. The *n\_bits* argument is only taken into account for the SC\_WRAP and SC\_WRAP\_SM overflow modes.
- *R* represents the remaining bits.

Table 40 shows how an unsigned fixed-point number shall be cast in case there is an overflow for each of the possible overflow modes.

**Table 40—Overflow handling for unsigned fixed-point numbers**

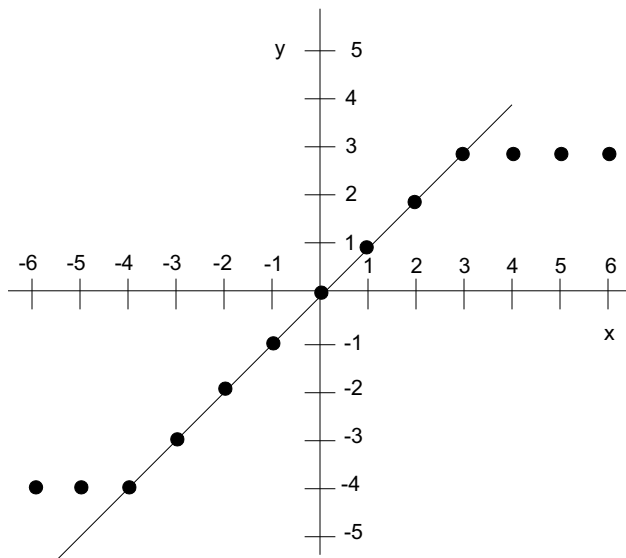
| Overflow mode                 | Result                                                                                                       |                             |
|-------------------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------|
|                               | Saturated bits ( <i>N</i> )                                                                                  | Remaining bits ( <i>R</i> ) |
| SC_SAT                        |                                                                                                              | 1 (overflow) 0 (underflow)  |
|                               | The remaining bits shall be set to 1 (overflow) or 0 (underflow).                                            |                             |
| SC_SAT_ZERO                   |                                                                                                              | 0                           |
|                               | The remaining bits shall be set to 0.                                                                        |                             |
| SC_SAT_SYM                    |                                                                                                              | 1 (overflow) 0 (underflow)  |
|                               | The remaining bits shall be set to 1 (overflow) or 0 (underflow).                                            |                             |
| SC_WRAP, ( <i>n_bits</i> =) 0 |                                                                                                              | <i>x</i>                    |
|                               | All bits except for the deleted bits shall be copied to the result number.                                   |                             |
| SC_WRAP, <i>n_bits</i> > 0    | 1                                                                                                            | <i>x</i>                    |
|                               | The saturated bits of the result number shall be set to 1. The remaining bits shall be copied to the result. |                             |
| SC_WRAP_SM                    | Not defined for unsigned numbers.                                                                            |                             |



During the conversion from signed to unsigned, sign extension shall occur before overflow handling, while in the unsigned to signed conversion, zero extension shall occur first.

#### 7.10.9.4 SC\_SAT

The SC\_SAT overflow mode shall be used to indicate that the output is saturated to MAX in case of overflow, or to MIN in the case of negative overflow. Figure 1 illustrates the SC\_SAT overflow mode for a word length of three bits. The x axis represents the word length before rounding; the y axis represents the word length after rounding. The ideal situation is represented by the diagonal dashed line.



**Figure 1—Saturation for signed numbers**

*Examples* (signed, 3-bit number):

before saturation: **0110 (6)**

after saturation: **011 (3)**

There is an overflow because the decimal number 6 is outside the range of values that can be represented exactly by means of three bits. The result is then rounded to the highest positive representable number, which is 3.

before saturation: **1011 (-5)**

after saturation: **100 (-4)**

There is an overflow because the decimal number -5 is outside the range of values that can be represented exactly by means of three bits. The result is then rounded to the lowest negative representable number, which is -4.

*Example* (unsigned, 3-bit number):

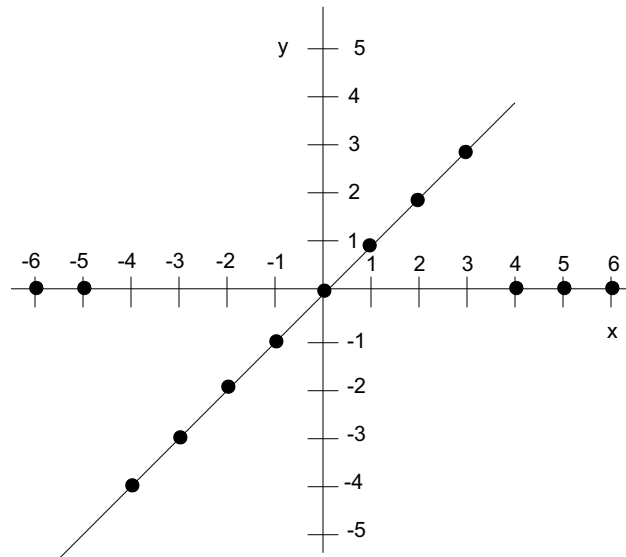
before saturation: **01110 (14)**

after saturation: **111 (7)**

The SC\_SAT mode corresponds to the SC\_WRAP and SC\_WRAP\_SM modes with the number of bits to be saturated equal to the number of kept bits.

### 7.10.9.5 SC\_SAT\_ZERO

The SC\_SAT\_ZERO overflow mode shall be used to indicate that the output is forced to zero in case of an overflow, that is, if MAX or MIN is exceeded. Figure 2 illustrates the SC\_SAT\_ZERO overflow mode for a word length of three bits. The x axis represents the word length before rounding; the y axis represents the word length after rounding.



**Figure 2—Saturation to zero for signed numbers**

*Examples* (signed, 3-bit number):

before saturation to zero: **0110 (6)**

after saturation to zero: **000 (0)**

There is an overflow because the decimal number 6 is outside the range of values that can be represented exactly by means of three bits. The result is saturated to zero.

before saturation to zero: **1011 (-5)**

after saturation to zero: **000 (0)**

There is an overflow because the decimal number -5 is outside the range of values that can be represented exactly by means of three bits. The result is saturated to zero.

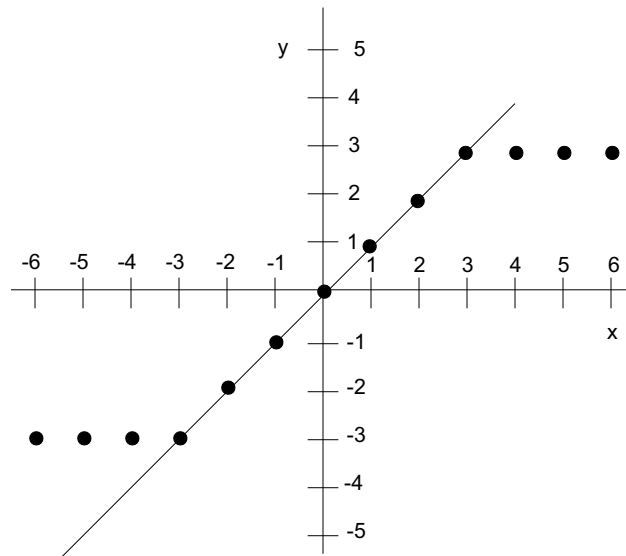
*Example* (unsigned, 3-bit number):

before saturation to zero: **01110 (14)**

after saturation to zero: **000 (0)**

### 7.10.9.6 SC\_SAT\_SYM

The SC\_SAT\_SYM overflow mode shall be used to indicate that the output is saturated to MAX in case of overflow, to -MAX (signed) or MIN (unsigned) in the case of negative overflow. Figure 3 illustrates the SC\_SAT\_SYM overflow mode for a word length of three bits. The x axis represents the word length before rounding; the y axis represents the word length after rounding. The ideal situation is represented by the diagonal dashed line.



**Figure 3—Symmetrical saturation for signed numbers**

*Examples* (signed, 3-bit number):

after symmetrical saturation: **0110 (6)**

after symmetrical saturation: **011 (3)**

There is an overflow because the decimal number 6 is outside the range of values that can be represented exactly by means of three bits. The result is then rounded to the highest positive representable number, which is 3.

after symmetrical saturation: **1011 (-5)**

after symmetrical saturation: **101 (-3)**

There is an overflow because the decimal number -5 is outside the range of values that can be represented exactly by means of three bits. The result is then rounded to minus the highest positive representable number, which is -3.

*Example* (unsigned, 3-bit number):

after symmetrical saturation: **01110 (14)**

after symmetrical saturation: **111 (7)**

#### 7.10.9.7 SC\_WRAP

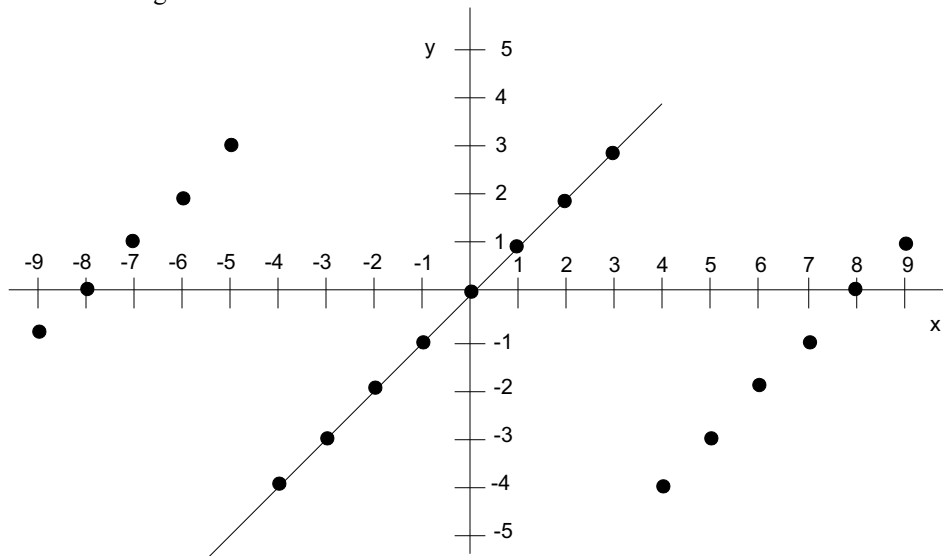
The SC\_WRAP overflow mode shall be used to indicate that the output is wrapped around in the case of overflow.

Two different cases are possible:

- SC\_WRAP with parameter  $n\_bits = 0$
- SC\_WRAP with parameter  $n\_bits > 0$

SC\_WRAP, 0

This shall be the default overflow mode. All bits except for the deleted bits shall be copied to the result number. Figure 4 illustrates the SC\_WRAP overflow mode for a word length of three bits with the  $n\_bits$  parameter set to 0. The x axis represents the word length before rounding; the y axis represents the word length after rounding.



**Figure 4—Wrap-around with  $n\_bits = 0$  for signed numbers**

*Examples* (signed, 3-bit number):

before wrapping around with 0 bits: **0100 (4)**

after wrapping around with 0 bits: **100 (-4)**

There is an overflow because the decimal number 4 is outside the range of values that can be represented exactly by means of three bits. The MSB is truncated and the result becomes negative: -4.

before wrapping around with 0 bits: **1011 (-5)**

after wrapping around with 0 bits: **011 (3)**

There is an overflow because the decimal number -5 is outside the range of values that can be represented exactly by means of three bits. The MSB is truncated and the result becomes positive: 3

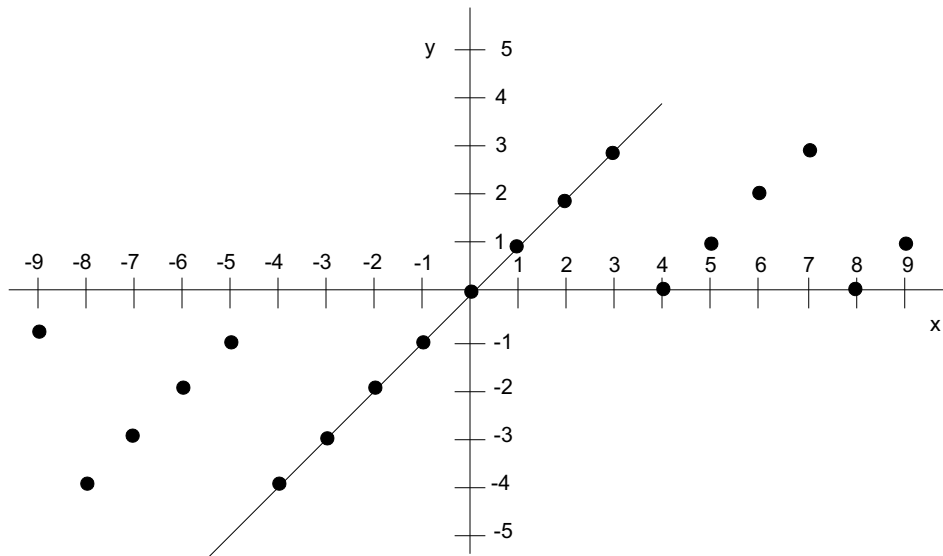
*Example* (unsigned, 3-bit number):

before wrapping around with 0 bits: **11011 (27)**

after wrapping around with 0 bits: **011 (3)**

SC\_WRAP,  $n\_bits > 0$ : SC\_WRAP, 1

Whenever  $n\_bits$  is greater than 0, the specified number of bits on the MSB side of the result shall be saturated with preservation of the original sign; the other bits shall be copied from the original. Positive numbers shall remain positive; negative numbers shall remain negative. Figure 5 illustrates the SC\_WRAP overflow mode for a word length of three bits with the  $n\_bits$  parameter set to 1. The x axis represents the word length before rounding; the y axis represents the word length after rounding.



**Figure 5—Wrap-around with  $n\_bits = 1$  for signed numbers**

*Examples* (signed, 3-bit number):

before wrapping around with 1 bit: **0101 (5)**

after wrapping around with 1 bit: **001 (1)**

There is an overflow because the decimal number 5 is outside the range of values that can be represented exactly by means of three bits. The sign bit is kept, so that positive numbers remain positive.

before wrapping around with 1 bit: **1011 (-5)**

after wrapping around with 1 bit: **111 (-1)**

There is an overflow because the decimal number -5 is outside the range of values that can be represented exactly by means of three bits. The MSB is truncated, but the sign bit is kept, so that negative numbers remain negative.

*Example* (unsigned, 5-bit number):

before wrapping around with 3 bits: **0110010 (50)**

after wrapping around with 3 bits: **11110 (30)**

For this example the SC\_WRAP, 3 mode is applied. The result number is five bits wide. The 3 bits at the MSB side are set to 1; the remaining bits are copied.

#### 7.10.9.8 SC\_WRAP\_SM

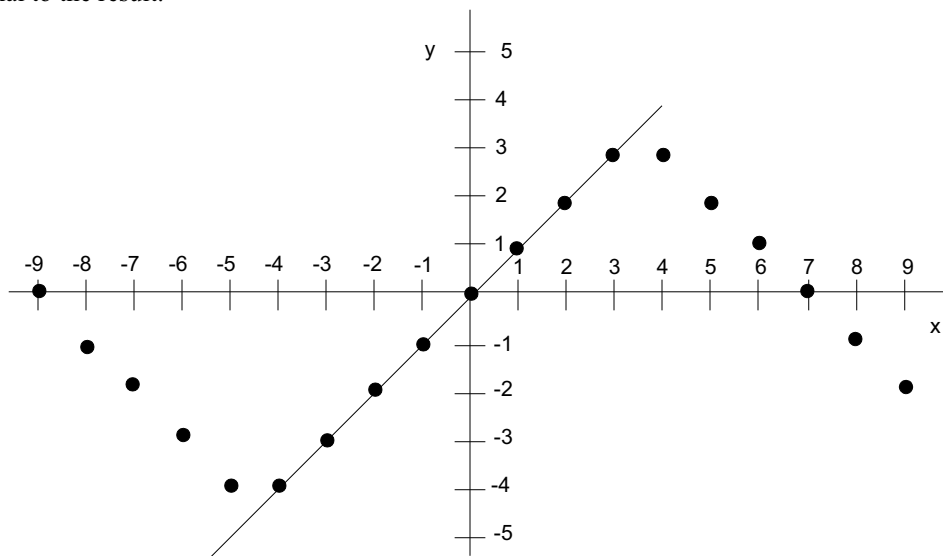
The SC\_WRAP\_SM overflow mode shall be used to indicate that the output is sign-magnitude wrapped around in the case of overflow. The  **$n\_bits$**  parameter shall indicate the number of bits (for example, 1) on the MSB side of the cast number that are saturated with preservation of the original sign.

Two different cases are possible:

- SC\_WRAP\_SM with parameter  $n\_bits = 0$
- SC\_WRAP\_SM with parameter  $n\_bits > 0$

SC\_WRAP\_SM, 0

The MSBs outside the required word length shall be deleted. The sign bit of the result shall get the value of the least significant of the deleted bits. The other bits shall be inverted in case where the original and the new values of the most significant of the kept bits differ. Otherwise, the other bits shall be copied from the original to the result.



**Figure 6—Sign Magnitude Wrap-Around with n\_bits = 0**

*Example:*

The sequence of operations to cast a decimal number 4 into three bits and use the overflow mode SC\_WRAP\_SM, 0, as shown in Figure 6, is as follows:

0100 (4)

The original representation is truncated to be put in a three-bit number:

100 (-4)

The new sign bit is 0. This is the value of least significant deleted bit.

Because the original and the new value of the new sign bit differ, the values of the remaining bits are inverted:

011 (3)

This principle shall be applied to all numbers that cannot be represented exactly by means of three bits, as shown in Table 41.

**Table 41—Sign magnitude wrap-around with  $n\_bits = 0$  for a three-bit number**

| Decimal | Binary |
|---------|--------|
| 8       | 111    |
| 7       | 000    |
| 6       | 001    |
| 5       | 010    |
| 4       | 011    |
| 3       | 011    |
| 2       | 010    |
| 1       | 001    |
| 0       | 000    |
| -1      | 111    |
| -2      | 110    |
| -3      | 101    |
| -4      | 100    |
| -5      | 100    |
| -6      | 101    |
| -7      | 110    |

$SC\_WRAP\_SM, n\_bits > 0$

The first  $n\_bits$  bits on the MSB side of the result number shall be as follows:

- Saturated to MAX in case of a positive number
- Saturated to MIN in case of a negative number

All numbers shall retain their sign.

In case where  $n\_bits$  equals 1, the other bits shall be copied and XORed with the original and the new value of the sign bit of the result. In the case where  $n\_bits$  is greater than 1, the remaining bits shall be XORed with the original value of the least significant saturated bit and the inverse value of the original sign bit.

*Example:*

SC\_WRAP\_SM, n\_bits > 0: SC\_WRAP\_SM, 3

The first three bits on the MSB side of the cast number are saturated to MAX or MIN.

If the decimal number 234 is cast into five bits using the overflow mode SC\_WRAP\_SM, 3, the following happens:

011101010 (234)

The original representation is truncated to five bits:

01010

The original sign bit is copied to the new MSB (bit position 4, starting from bit position 0):

01010

The bits at position 2, 3, and 4 are saturated; they are converted to the maximum value that can be expressed with three bits without changing the sign bit:

01110

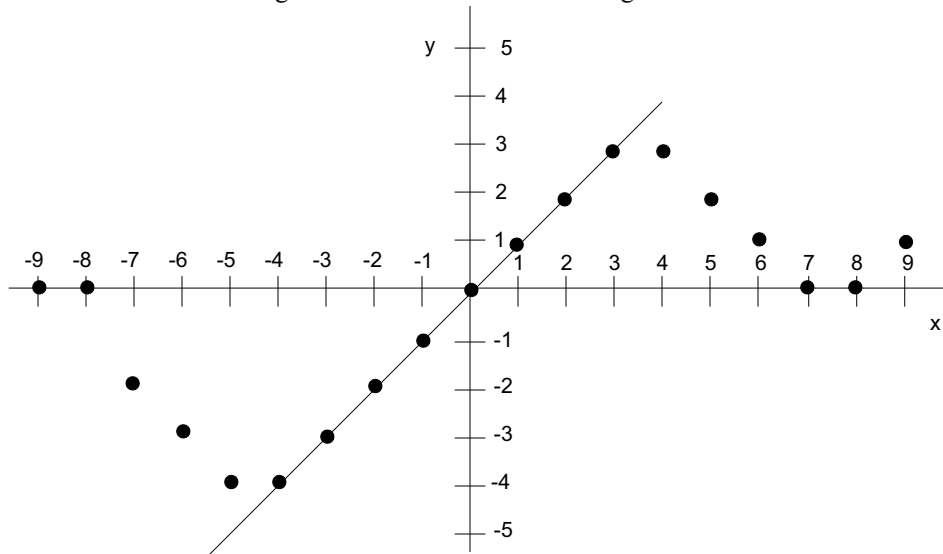
The original value of the bit on position 2 was 0. The remaining bits at the LSB side (10) are XORed with this value and with the inverse value of the original sign bit, that is, with 0 and 1, respectively.

01101 (13)

*Example:*

SC\_WRAP\_SM, n\_bits > 0: SC\_WRAP\_SM, 1

The first bit on the MSB side of the cast number gets the value of the original sign bit. The other bits are copied and XORed with the original and the new value of the sign bit of the result number.



**Figure 7—Sign magnitude wrap-around with n\_bits = 1**

The sequence of operations to cast the decimal number 12 into three bits using the overflow mode SC\_WRAP\_SM, 1, as shown in Figure 7, is as follows:



01100 (12)

The original representation is truncated to three bits.  
100

The original sign bit is copied to the new MSB (bit position 2, starting from bit position 0).  
000

The two remaining bits at the LSB side are XORed with the original (1) and the new value (0) of the new sign bit.  
011

This principle shall be applied to all numbers that cannot be represented exactly by means of three bits, as shown in Table 42.

**Table 42—Sign-magnitude wrap-around with n\_bits=1 for a three-bit number**

| Decimal | Binary |
|---------|--------|
| 9       | 001    |
| 8       | 000    |
| 7       | 000    |
| 6       | 001    |
| 5       | 010    |
| 4       | 011    |
| 3       | 011    |
| 2       | 010    |
| 1       | 001    |
| 0       | 000    |
| -1      | 111    |
| -2      | 110    |
| -3      | 101    |
| -4      | 100    |
| -5      | 100    |
| -6      | 101    |
| -7      | 110    |
| -8      | 111    |
| -9      | 111    |

### 7.10.9.9 Quantization modes

Quantization shall be applied when the precision of the value assigned to a fixed-point variable exceeds the precision of the fixed-point variable. In SystemC, specific quantization modes shall be available to control the mapping to a representable value.

The mutually exclusive quantization modes listed in Table 43 shall be provided. The default quantization mode shall be SC\_TRN.

**Table 43—Quantization modes**

| Quantization mode          | Name           |
|----------------------------|----------------|
| Rounding to plus infinity  | SC_RND         |
| Rounding to zero           | SC_RND_ZERO    |
| Rounding to minus infinity | SC_RND_MIN_INF |
| Rounding to infinity       | SC_RND_INF     |
| Convergent rounding        | SC_RND_CONV    |
| Truncation                 | SC_TRN         |
| Truncation to zero         | SC_TRN_ZERO    |

Quantization is the mapping of a value that may not be precisely represented in a specific fixed-point representation to a value that can be represented with arbitrary magnitude. If a value can be precisely represented, quantization shall not change the value. All the rounding modes shall map a value to the nearest value that is representable. When there are two nearest representable values (the value is halfway between them), the rounding modes shall provide different criteria for selection between the two. Both of the truncate modes shall map a positive value to the nearest representable value that is less than the value. SC\_TRN mode shall map a negative value to the nearest representable value that is less than the value, while SC\_TRN\_ZERO shall map a negative value to the nearest representable value that is greater than the value.

Each of the following quantization modes is followed by a figure. The input values are given on the x axis and the output values on the y axis. Together they determine the quantization mode. In each figure, the quantization mode specified by the respective keyword is combined with the ideal characteristic. This ideal characteristic is represented by the diagonal dashed line.

Before each quantization mode is discussed in detail, an overview is given of how the different quantization modes deal with quantization for signed and unsigned fixed-point numbers.

**7.10.9.10 Quantization for signed fixed-point numbers**

The following template contains a signed fixed-point number in two's complement representation before and after a quantization mode has been applied, and a number of flags. The flags are explained below the template.

|        |           |          |          |          |          |          |          |          |           |           |          |          |          |          |          |
|--------|-----------|----------|----------|----------|----------|----------|----------|----------|-----------|-----------|----------|----------|----------|----------|----------|
| Before | <i>x</i>  | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i>  | <i>x</i>  | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> |
| After: | <i>x</i>  | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i>  |           |          |          |          |          |          |
| Flags: | <i>sR</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>lR</i> | <i>mD</i> | <i>D</i> | <i>D</i> | <i>D</i> | <i>D</i> | <i>D</i> |

The following flags and symbols are used in the template just given and in Table 44:

- *x* represents a binary digit (0 or 1).
- *sR* represents a sign bit.
- *R* represents the remaining bits.
- *lR* represents the least significant remaining bit.
- *mD* represents the most significant deleted bit.
- *D* represents the deleted bits.
- *r* represents the logical or of the deleted bits except for the *mD* bit in the template just given. When there are no remaining bits, *r* is **false**. This means that *r* is **false** when the two nearest numbers are at equal distance.

Table 44 shows how a signed fixed-point number shall be cast for each of the possible quantization modes in cases where there is quantization. If the two nearest representable numbers are not at equal distance, the result shall be the nearest representable number. This shall be found by applying the SC\_RND mode, that is, by adding the most significant of the deleted bits to the remaining bits.

The second column in Table 44 contains the expression that shall be added to the remaining bits. It shall evaluate to a one or a zero. The operators used in the table are “!” for a bitwise negation, “|” for a bitwise OR, and “&” for a bitwise AND.

**Table 44—Quantization handling for signed fixed-point numbers**

| Quantization mode | Expression to be added                                                                                                                                               |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SC_RND            | mD                                                                                                                                                                   |
|                   | Add the most significant deleted bit to the remaining bits.                                                                                                          |
| SC_RND_ZERO       | mD & (sR   r)                                                                                                                                                        |
|                   | If the most significant deleted bit is 1 and either the sign bit or at least one other deleted bit is 1, add 1 to the remaining bits.                                |
| SC_RND_MIN_INF    | mD & r                                                                                                                                                               |
|                   | If the most significant deleted bit is 1 and at least one other deleted bit is 1, add 1 to the remaining bits.                                                       |
| SC_RND_INF        | mD & (! sR   r)                                                                                                                                                      |
|                   | If the most significant deleted bit is 1 and either the inverted value of the sign bit or at least one other deleted bit is 1, add 1 to the remaining bits.          |
| SC_RND_CONV       | mD & (IR   r)                                                                                                                                                        |
|                   | If the most significant deleted bit is 1 and either the least significant of the remaining bits or at least one other deleted bit is 1, add 1 to the remaining bits. |
| SC_TRN            | 0                                                                                                                                                                    |
|                   | Copy the remaining bits.                                                                                                                                             |
| SC_TRN_ZERO       | sR & (mD   r)                                                                                                                                                        |
|                   | If the sign bit is 1 and either the most significant deleted bit or at least one other deleted bit is 1, add 1 to the remaining bits.                                |

#### 7.10.9.11 Quantization for unsigned fixed-point numbers

The following template contains an unsigned fixed-point number before and after a quantization mode has been applied, and a number of flags. The flags are explained below the template.

|        |          |          |          |          |          |          |          |          |           |           |          |          |          |          |
|--------|----------|----------|----------|----------|----------|----------|----------|----------|-----------|-----------|----------|----------|----------|----------|
| Before | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i>  | <i>x</i>  | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> |
| After: | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i> | <i>x</i>  |           |          |          |          |          |
| Flags: | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>R</i> | <i>IR</i> | <i>mD</i> | <i>D</i> | <i>D</i> | <i>D</i> | <i>D</i> |

The following flags and symbols are used in the template just given and in Table 45:

- $x$  represents a binary digit (0 or 1).
- $R$  represents the remaining bits.
- $lR$  represents the least significant remaining bit.
- $mD$  represents the most significant deleted bit.
- $D$  represents the deleted bits.
- $r$  represents the logical or of the deleted bits except for the  $mD$  bit in the template just given. When there are no remaining bits,  $r$  is **false**. This means that  $r$  is **false** when the two nearest numbers are at equal distance.

Table 45 shows how an unsigned fixed-point number shall be cast for each of the possible quantization modes in cases where there is quantization. If the two nearest representable numbers are not at equal distance, the result shall be the nearest representable number. This shall be found for all the rounding modes by applying the SC\_RND mode, that is, by adding the most significant of the deleted bits to the remaining bits.

The second column in Table 45 contains the expression that shall be added to the remaining bits. It shall evaluate to a one or a zero. The “&” operator used in the table represents a bitwise AND, and the “|” a bitwise OR.

**Table 45—Quantization handling for unsigned fixed-point numbers**

| Quantization mode | Expression to be added                                                                                                                                               |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SC_RND            | $mD$                                                                                                                                                                 |
|                   | Add the most significant deleted bit to the left bits.                                                                                                               |
| SC_RND_ZERO       | 0                                                                                                                                                                    |
|                   | Copy the remaining bits.                                                                                                                                             |
| SC_RND_MIN_INF    | 0                                                                                                                                                                    |
|                   | Copy the remaining bits.                                                                                                                                             |
| SC_RND_INF        | $mD$                                                                                                                                                                 |
|                   | Add the most significant deleted bit to the left bits.                                                                                                               |
| SC_RND_CONV       | $mD \& (lR \mid r)$                                                                                                                                                  |
|                   | If the most significant deleted bit is 1 and either the least significant of the remaining bits or at least one other deleted bit is 1, add 1 to the remaining bits. |
| SC_TRN            | 0                                                                                                                                                                    |
|                   | Copy the remaining bits.                                                                                                                                             |
| SC_TRN_ZERO       | 0                                                                                                                                                                    |
|                   | Copy the remaining bits.                                                                                                                                             |

NOTE—For all rounding modes, overflow can occur. One extra bit on the MSB side is needed to represent the result in full precision.

### 7.10.9.12 SC\_RND

The result shall be rounded to the nearest representable number by adding the most significant of the deleted LSBs to the remaining bits. This rule shall be used for all rounding modes when the two nearest representable numbers are not at equal distance. When the two nearest representable numbers are at equal distance, this rule implies that there is rounding towards plus infinity, as shown in Figure 8.

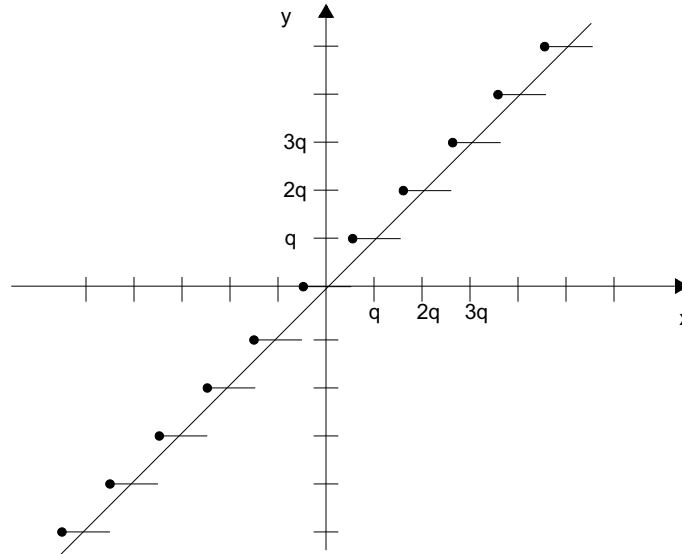


Figure 8—Rounding to plus infinity

In Figure 8, the symbol **q** refers to the quantization step, that is, the resolution of the data type.

*Example (signed):*

Numbers of type **sc\_fixed<4,2>** are assigned to numbers of type **sc\_fixed<3,2,SC\_RND>**

before rounding to plus infinity: **(1.25)**

after rounding to plus infinity: **01.1 (1.5)**

There is quantization because the decimal number 1.25 is outside the range of values that can be represented exactly by means of a **sc\_fixed<3,2,SC\_RND>** number. The most significant of the deleted LSBs (1) is added to the new LSB.

before rounding to plus infinity: **10.11 (-1.25)**

after rounding to plus infinity: **11.0 (-1)**

There is quantization because the decimal number -1.25 is outside the range of values that can be represented exactly by means of a **sc\_fixed<3,2,SC\_RND>** number. The most significant of the deleted LSBs (1) is added to the new LSB.

*Example (unsigned):*

Numbers of type **sc\_ufixed<16,8>** are assigned to numbers of type **sc\_ufixed<12,8,SC\_RND>**

before rounding to plus infinity: **00100110.01001111 (38.30859375)**

after rounding to plus infinity: **00100110.0101 (38.3125)**

### 7.10.9.13 SC\_RND\_ZERO

If the two nearest representable numbers are not at equal distance, the SC\_RND\_ZERO mode shall be applied.

If the two nearest representable numbers are at equal distance, the output shall be rounded towards 0, as shown in Figure 9. For positive numbers, the redundant bits on the LSB side shall be deleted. For negative numbers, the most significant of the deleted LSBs shall be added to the remaining bits.

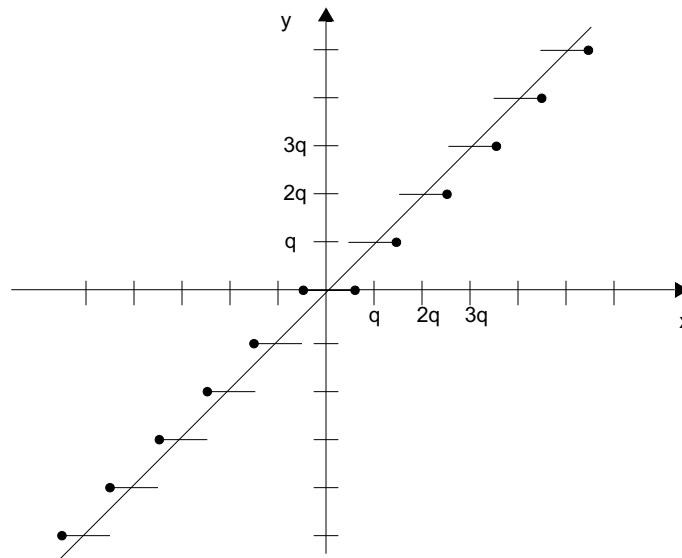


Figure 9—Rounding to Zero

*Example (signed):*

Numbers of type **sc\_fixed<4,2>** are assigned to numbers of type **sc\_fixed<3,2,SC\_RND\_ZERO>**  
before rounding to zero: **(1.25)**  
after rounding to zero: **01.0 (1)**

There is quantization because the decimal number 1.25 is outside the range of values that can be represented exactly by means of a **sc\_fixed<3,2,SC\_RND\_ZERO>** number. The redundant bits are omitted.

before rounding to zero: **10.11 (-1.25)**

after rounding to zero: **11.0 (-1)**

There is quantization because the decimal number -1.25 is outside the range of values that can be represented exactly by means of a **sc\_fixed<3,2,SC\_RND\_ZERO>** number. The most significant of the omitted LSBs (1) is added to the new LSB.

*Example (unsigned):*

Numbers of type **sc\_ufixed<16,8>** are assigned to numbers of type **sc\_ufixed<12,8,SC\_RND\_ZERO>**  
before rounding to zero: **000100110.01001 (38.28125)**  
after rounding to zero: **000100110.0100 (38.25)**

#### 7.10.9.14 SC\_RND\_MIN\_INF

If the two nearest representable numbers are not at equal distance, the SC\_RND\_MIN\_INF mode shall be applied.

If the two nearest representable numbers are at equal distance, there shall be rounding towards minus infinity, as shown in Figure 10, by omitting the redundant bits on the LSB side.

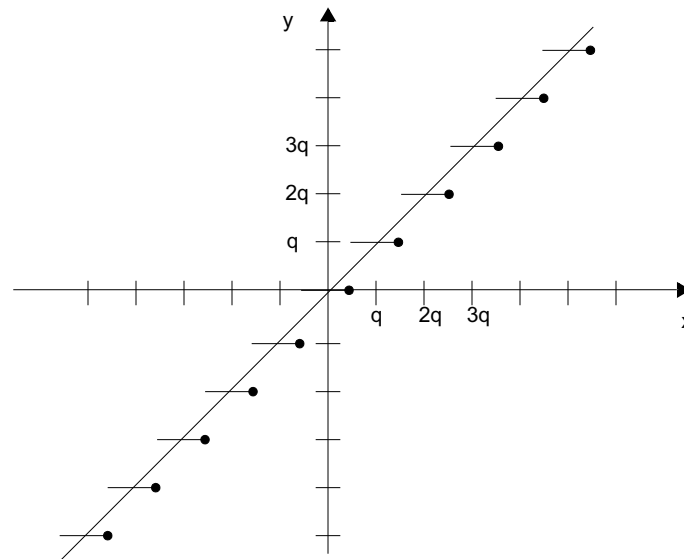


Figure 10—Rounding to minus infinity

*Example (signed):*

Numbers of type **sc\_fixed<4,2>** are assigned to numbers of type **sc\_fixed<3,2,SC\_RND\_MIN\_INF>**

before rounding to minus infinity: **01.01 (1.25)**

after rounding to minus infinity: **01.0 (1)**

There is quantization because the decimal number 1.25 is outside the range of values that can be represented exactly by means of a **sc\_fixed<3,2,SC\_RND\_MIN\_INF>** number. The surplus bits are truncated.

before rounding to minus infinity: **10.11 (-1.25)**

after rounding to minus infinity: **10.1 (-1.5)**

There is quantization because the decimal number -1.25 is outside the range of values that can be represented exactly by means of a **sc\_fixed<3,2,SC\_RND\_MIN\_INF>** number. The surplus bits are truncated.

*Example (unsigned):*

Numbers of type **sc\_ufixed<16,8>** are assigned to numbers of type **sc\_ufixed<12,8,SC\_RND\_MIN\_INF>**

before rounding to minus infinity: **000100110.01001 (38.28125)**

after rounding to minus infinity: **000100110.0100 (38.25)**



### 7.10.9.15 SC\_RND\_INF

Rounding shall be performed if the two nearest representable numbers are at equal distance.

For positive numbers, there shall be rounding towards plus infinity if the LSB of the remaining bits is 1 and towards minus infinity if the LSB of the remaining bits is 0, as shown in Figure 11.

For negative numbers, there shall be rounding towards minus infinity if the LSB of the remaining bits is 1 and towards plus infinity if the LSB of the remaining bits is 0, as shown in Figure 11.

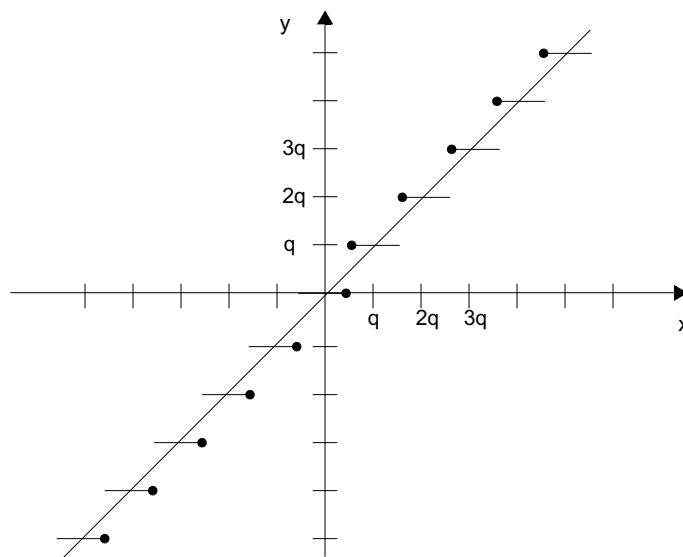


Figure 11—Rounding to infinity

*Example (signed):*

Numbers of type **sc\_fixed<4,2>** are assigned to numbers of type **sc\_fixed<3,2,SC\_RND\_INF>**

before rounding to infinity: **01.01 (1.25)**

after rounding to infinity: **01.1 (1.5)**

There is quantization because the decimal number 1.25 is outside the range of values that can be represented exactly by means of a **sc\_fixed<3,2,SC\_RND\_INF>** number. The most significant of the deleted LSBs (1) is added to the new LSB.

before rounding to infinity: **10.11 (-1.25)**

after rounding to infinity: **10.1 (-1.5)**

There is quantization because the decimal number -1.25 is outside the range of values that can be represented exactly by means of a **sc\_fixed<3,2,SC\_RND\_INF>** number. The surplus bits are truncated.

*Example (unsigned):*

Numbers of type **sc\_ufixed<16,8>** are assigned to numbers of type **sc\_ufixed<12,8,SC\_RND\_INF>**

before rounding to infinity: **000100110.01001 (38.28125)**

after rounding to infinity: **000100110.0101 (38.3125)**

### 7.10.9.16 SC\_RND\_CONV

If the two nearest representable numbers are not at equal distance, the SC\_RND\_CONV mode shall be applied.

If the two nearest representable numbers are at equal distance, there shall be rounding towards plus infinity if the LSB of the remaining bits is 1. There shall be rounding towards minus infinity if the LSB of the remaining bits is 0. The characteristics are shown in Figure 12.

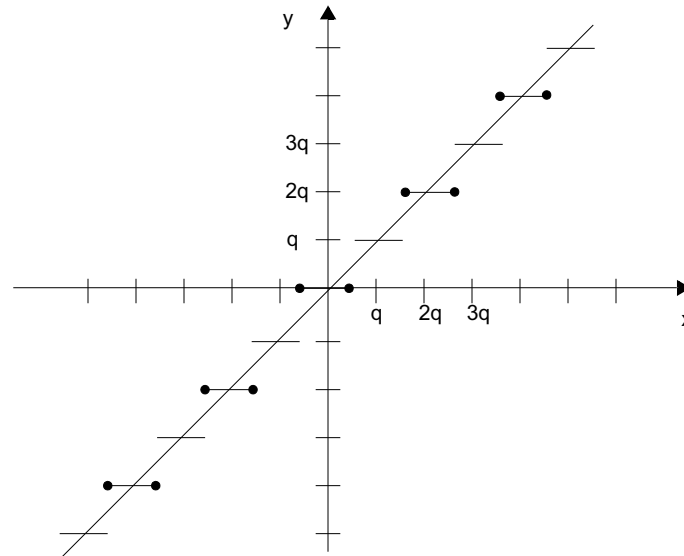


Figure 12—Convergent rounding

*Example (signed):*

Numbers of type **sc\_fixed<4,2>** are assigned to numbers of type **sc\_fixed<3,2,SC\_RND\_CONV>**

before convergent rounding: **00.11 (0.75)**

after convergent rounding: **01.0 (1)**

There is quantization because the decimal number 0.75 is outside the range of values that can be represented exactly by means of a **sc\_fixed<3,2,SC\_RND\_CONV>** number. The surplus bits are truncated and the result is rounded towards plus infinity.

before convergent rounding: **10.11 (-1.25)**

after convergent rounding: **11.0 (-1)**

There is quantization because the decimal number -1.25 is outside the range of values that can be represented exactly by means of a **sc\_fixed<3,2,SC\_RND\_CONV>** number. The surplus bits are truncated and the result is rounded towards plus infinity.

*Example (unsigned):*

Numbers of type **sc\_ufixed<16,8>** are assigned to numbers of type **sc\_ufixed<12,8,SC\_RND\_CONV>**

before convergent rounding: 000100110.01001 (**38.28125**)

after convergent rounding: 000100110.0100 (**38.25**)

before convergent rounding: **000100110.01011 (38.34375)**

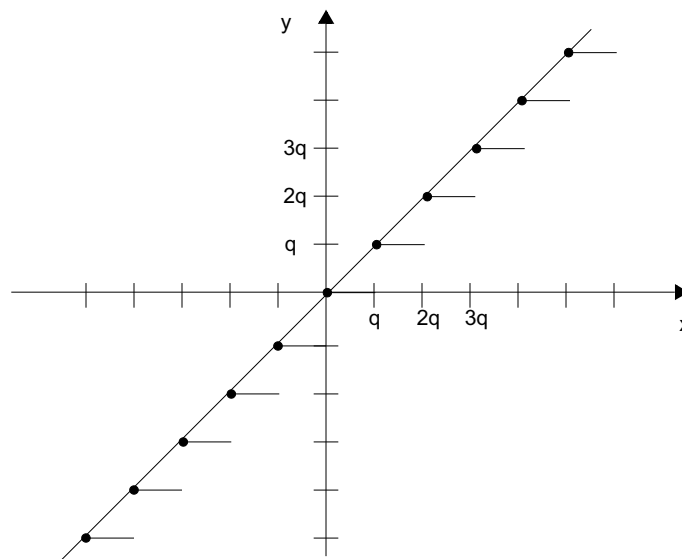
after convergent rounding: **000100110.0110 (38.375)**

#### 7.10.9.17 SC\_TRN

SC\_TRN shall be the default quantization mode. The result shall be rounded towards minus infinity, that is, the superfluous bits on the LSB side shall be deleted. A quantized number shall be approximated by the first representable number below its original value within the required bit range.

NOTE—In scientific literature this mode is usually called “value truncation.”

The required characteristics are shown in Figure 13.



**Figure 13—Truncation**

*Example (signed):*

Numbers of type **sc\_fixed<4,2>** are assigned to numbers of type **sc\_fixed<3,2,SC\_TRN>**

before truncation: **01.01 (1.25)**

after truncation: **01.0 (1)**

There is quantization because the decimal number 1.25 is outside the range of values that can be represented exactly by means of a **sc\_fixed<3,2,SC\_TRN>** number. The LSB is truncated.

before truncation: **10.11 (-1.25)**

after truncation: **10.1 (-1.5)**

There is quantization because the decimal number -1.25 is outside the range of values that can be represented exactly by means of a **sc\_fixed<3,2,SC\_TRN>** number. The LSB is truncated.

*Example (unsigned):*

Numbers of type **sc\_ufixed<16,8>** are assigned to numbers of type **sc\_ufixed<12,8,SC\_TRN>**

before truncation: **00100110.01001111 (38.30859375)**

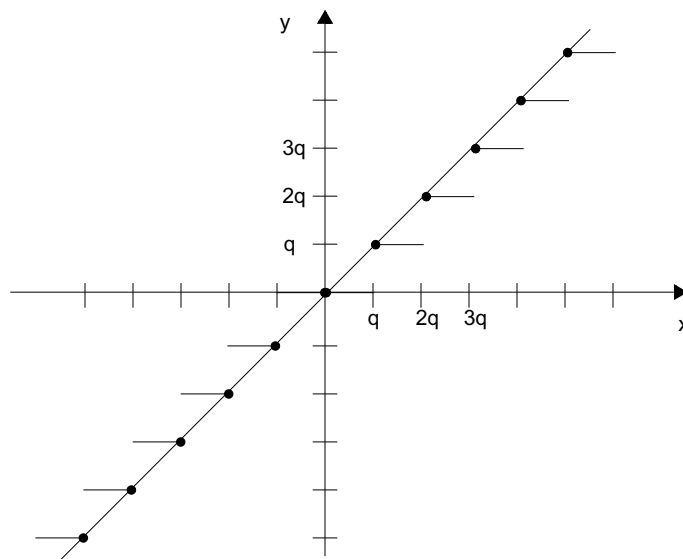
after truncation: **00100110.0100 (38.25)**

### 7.10.9.18 SC\_TRN\_ZERO

For positive numbers, this quantization mode shall correspond to SC\_TRN. For negative numbers, the result shall be rounded towards zero (SC\_RND\_ZERO); that is, the superfluous bits on the right-hand side shall be deleted and the sign bit added to the left LSBs, provided at least one of the deleted bits differs from zero. A quantized number shall be approximated by the first representable number that is lower in absolute value.

NOTE—In scientific literature this mode is usually called “magnitude truncation.”

The required characteristics are shown in Figure 14.



**Figure 14—Truncation to zero**

*Example (signed):*

A number of type **sc\_fixed<4,2>** is assigned to a number of type **sc\_fixed<3,2,SC\_TRN\_ZERO>**

before truncation to zero: **10.11 (-1.25)**

after truncation to zero: **11.0 (-1)**

There is quantization because the decimal number -1.25 is outside the range of values that can be represented exactly by means of a **sc\_fixed<3,2,SC\_TRN\_ZERO>** number. The LSB is truncated and then the sign bit (1) is added at the LSB side.

*Example (unsigned):*

Numbers of type **sc\_ufixed<16,8>** are assigned to numbers of type **sc\_ufixed<12,8,SC\_TRN\_ZERO>**

before truncation to zero: **00100110.01001111 (38.30859375)**

after truncation to zero: **00100110.0100 (38.25)**

### 7.10.10 sc\_fxnum

#### 7.10.10.1 Description

Class **sc\_fxnum** is the base class for finite-precision fixed-point types. It shall be provided in order to define functions and overloaded operators that will work with any derived class.

#### 7.10.10.2 Class definition

```
namespace sc_dt {  
  
class sc_fxnum  
{  
    friend class sc_fxval;  
  
    friend class sc_fxnum_bitref†;  
    friend class sc_fxnum_subref†;  
    friend class sc_fxnum_fast_bitref†;  
    friend class sc_fxnum_fast_subref†;  
  
public:  
    // Unary operators  
    const sc_fxval operator- () const;  
    const sc_fxval operator+ () const;  
  
    // Binary operators  
#define DECL_BIN_OP_T( op , tp ) \  
    friend const sc_fxval operator op ( const sc_fxnum& , tp ); \  
    friend const sc_fxval operator op ( tp , const sc_fxnum& );  
#define DECL_BIN_OP_OTHER( op ) \  
    DECL_BIN_OP_T( op , int64 ) \  
    DECL_BIN_OP_T( op , uint64 ) \  
    DECL_BIN_OP_T( op , const sc_int_base& ) \  
    DECL_BIN_OP_T( op , const sc_uint_base& ) \  
    DECL_BIN_OP_T( op , const sc_signed& ) \  
    DECL_BIN_OP_T( op , const sc_unsigned& )  
#define DECL_BIN_OP( op , dummy ) \  
    friend const sc_fxval operator op ( const sc_fxnum& , const sc_fxnum& ); \  
    DECL_BIN_OP_T( op , int ) \  
    DECL_BIN_OP_T( op , unsigned int ) \  
    DECL_BIN_OP_T( op , long ) \  
    DECL_BIN_OP_T( op , unsigned long ) \  
    DECL_BIN_OP_T( op , double ) \  
    DECL_BIN_OP_T( op , const char* ) \  
    DECL_BIN_OP_T( op , const sc_fxval& ) \  
    DECL_BIN_OP_T( op , const sc_fxval_fast& ) \  
    DECL_BIN_OP_T( op , const sc_fxnum_fast& ) \  
    DECL_BIN_OP_OTHER( op )  
  
    DECL_BIN_OP( * , mult )  
    DECL_BIN_OP( + , add )  
    DECL_BIN_OP( - , sub )  
    DECL_BIN_OP( / , div )
```

```
#undef DECL_BIN_OP_T
#undef DECL_BIN_OP_OTHER
#undef DECL_BIN_OP

friend const sc_fxval operator<< ( const sc_fxnum& , int );
friend const sc_fxval operator>> ( const sc_fxnum& , int );

// Relational (including equality) operators
#define DECL_REL_OP_T( op , tp ) \
    friend bool operator op ( const sc_fxnum& , tp ); \
    friend bool operator op ( tp , const sc_fxnum& ); \
    DECL_REL_OP_T( op , int64 ) \
    DECL_REL_OP_T( op , uint64 ) \
    DECL_REL_OP_T( op , const sc_int_base& ) \
    DECL_REL_OP_T( op , const sc_uint_base& ) \
    DECL_REL_OP_T( op , const sc_signed& ) \
    DECL_REL_OP_T( op , const sc_unsigned& )
#define DECL_REL_OP( op ) \
    friend bool operator op ( const sc_fxnum& , const sc_fxnum& ); \
    DECL_REL_OP_T( op , int ) \
    DECL_REL_OP_T( op , unsigned int ) \
    DECL_REL_OP_T( op , long ) \
    DECL_REL_OP_T( op , unsigned long ) \
    DECL_REL_OP_T( op , double ) \
    DECL_REL_OP_T( op , const char* ) \
    DECL_REL_OP_T( op , const sc_fxval& ) \
    DECL_REL_OP_T( op , const sc_fxval_fast& ) \
    DECL_REL_OP_T( op , const sc_fxnum_fast& ) \
    DECL_REL_OP_OTHER( op )
DECL_REL_OP( < )
DECL_REL_OP( <= )
DECL_REL_OP( > )
DECL_REL_OP( >= )
DECL_REL_OP( == )
DECL_REL_OP( != )

#undef DECL_REL_OP_T
#undef DECL_REL_OP_OTHER
#undef DECL_REL_OP

// Assignment operators
#define DECL_ASN_OP_T( op , tp ) \
    sc_fxnum& operator op( tp ); \
    DECL_ASN_OP_T( op , int64 ) \
    DECL_ASN_OP_T( op , uint64 ) \
    DECL_ASN_OP_T( op , const sc_int_base& ) \
    DECL_ASN_OP_T( op , const sc_uint_base& ) \
    DECL_ASN_OP_T( op , const sc_signed& ) \
    DECL_ASN_OP_T( op , const sc_unsigned& )
#define DECL_ASN_OP( op ) \
    DECL_ASN_OP_T( op , int ) \
    DECL_ASN_OP_T( op , unsigned int ) \
    DECL_ASN_OP_T( op , long ) \
    DECL_ASN_OP_T( op , unsigned long ) \
```

```
DECL_ASN_OP_T( op , double ) \
DECL_ASN_OP_T( op , const char* ) \
DECL_ASN_OP_T( op , const sc_fxval& ) \
DECL_ASN_OP_T( op , const sc_fxval_fast& ) \
DECL_ASN_OP_T( op , const sc_fxnum& ) \
DECL_ASN_OP_T( op , const sc_fxnum_fast& ) \
DECL_ASN_OP_OTHER( op )
DECL_ASN_OP( = )
DECL_ASN_OP( *= )
DECL_ASN_OP( /= )
DECL_ASN_OP( += )
DECL_ASN_OP( -= )
DECL_ASN_OP_T( <=< , int )
DECL_ASN_OP_T( >=> , int )

#undef DECL_ASN_OP_T
#undef DECL_ASN_OP_OTHER
#undef DECL_ASN_OP

// Auto-increment and auto-decrement
const sc_fxval operator++ ( int );
const sc_fxval operator-- ( int );
sc_fxnum& operator++ ();
sc_fxnum& operator-- ();

// Bit selection
const sc_fxnum_bitref† operator[] ( int ) const;
sc_fxnum_bitref† operator[] ( int );

// Part selection
const sc_fxnum_subref† operator() ( int , int ) const;
sc_fxnum_subref† operator() ( int , int );
const sc_fxnum_subref† range( int , int ) const;
sc_fxnum_subref† range( int , int );
const sc_fxnum_subref† operator() () const;
sc_fxnum_subref† operator() ();
const sc_fxnum_subref† range() const;
sc_fxnum_subref† range();

// Implicit conversion
operator double() const;

// Explicit conversion to primitive types
short to_short() const;
unsigned short to_ushort() const;
int to_int() const;
unsigned int to_uint() const;
long to_long() const;
unsigned long to_ulong() const;
int64 to_int64() const;
uint64 to_uint64() const;
float to_float() const;
double to_double() const;
```

```

// Explicit conversion to character string
const std::string to_string() const;
const std::string to_string( sc_numrep ) const;
const std::string to_string( sc_numrep , bool ) const;
const std::string to_string( sc_fmt ) const;
const std::string to_string( sc_numrep , sc_fmt ) const;
const std::string to_string( sc_numrep , bool , sc_fmt ) const;
const std::string to_dec() const;
const std::string to_bin() const;
const std::string to_oct() const;
const std::string to_hex() const;

// Query value
bool is_neg() const;
bool is_zero() const;
bool quantization_flag() const;
bool overflow_flag() const;
const sc_fxval value() const;

// Query parameters
int wl() const;
int iwl() const;
sc_q_mode q_mode() const;
sc_o_mode o_mode() const;
int n_bits() const;
const sc_fxtype_params& type_params() const;
const sc_fxcast_switch& cast_switch() const;

// Print or dump content
void print( std::ostream& = std::cout ) const;
void scan( std::istream& = std::cin );
void dump( std::ostream& = std::cout ) const;

private:
    // Disabled
    sc_fxnum();
    sc_fxnum( const sc_fxnum& );
};

} // namespace sc_dt

```

### 7.10.10.3 Constraints on usage

An application shall not directly create an instance of type **sc\_fxnum**. An application may use a pointer to **sc\_fxnum** or a reference to **sc\_fxnum** to refer to an object of a class derived from **sc\_fxnum**.

### 7.10.10.4 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ numeric representation to **sc\_fxnum**, using truncation or sign-extension, as described in 7.10.4.



#### 7.10.10.5 Implicit type conversion

operator **double**() const;

Operator **double** shall provide implicit type conversion from **sc\_fxnum** to double.

#### 7.10.10.6 Explicit type conversion

short **to\_short**() const;  
 unsigned short **to\_ushort**() const;  
 int **to\_int**() const;  
 unsigned int **to\_uint**() const;  
 long **to\_long**() const;  
 unsigned long **to\_ulong**() const;  
 int64 **to\_int64**() const;  
 uint64 **to\_uint64**() const;  
 float **to\_float**() const;  
 double **to\_double**() const;

These member functions shall perform conversion to C++ numeric types.

const std::string **to\_string**() const;  
 const std::string **to\_string**( sc\_numrep ) const;  
 const std::string **to\_string**( sc\_numrep , bool ) const;  
 const std::string **to\_string**( sc\_fmt ) const;  
 const std::string **to\_string**( sc\_numrep , sc\_fmt ) const;  
 const std::string **to\_string**( sc\_numrep , bool , sc\_fmt ) const;  
 const std::string **to\_dec**() const;  
 const std::string **to\_bin**() const;  
 const std::string **to\_oct**() const;  
 const std::string **to\_hex**() const;

These member functions shall perform the conversion to a **string** representation, as described in 7.2.11, 7.10.8, and 7.10.8.1.

#### 7.10.11 sc\_fxnum\_fast

##### 7.10.11.1 Description

Class **sc\_fxnum\_fast** is the base class for limited-precision fixed-point types. It shall be provided in order to define functions and overloaded operators that will work with any derived class.

##### 7.10.11.2 Class definition

```
namespace sc_dt {

class sc_fxnum_fast
{
    friend class sc_fxval_fast;

    friend class sc_fxnum_bitref†;
    friend class sc_fxnum_subref†;
    friend class sc_fxnum_fast_bitref†;
    friend class sc_fxnum_fast_subref†;
}
```

```

public:
    // Unary operators
    const sc_fxval_fast operator- () const;
    const sc_fxval_fast operator+ () const;

    // Binary operators
    #define DECL_BIN_OP_T( op , tp ) \
        friend const sc_fxval_fast operator op ( const sc_fxnum_fast& , tp ); \
        friend const sc_fxval_fast operator op ( tp , const sc_fxnum_fast& );
    #define DECL_BIN_OP_OTHER( op ) \
        DECL_BIN_OP_T( op , int64 ) \
        DECL_BIN_OP_T( op , uint64 ) \
        DECL_BIN_OP_T( op , const sc_int_base& ) \
        DECL_BIN_OP_T( op , const sc_uint_base& ) \
        DECL_BIN_OP_T( op , const sc_signed& ) \
        DECL_BIN_OP_T( op , const sc_unsigned& )
    #define DECL_BIN_OP( op , dummy ) \
        friend const sc_fxval_fast operator op ( const sc_fxnum_fast& , const sc_fxnum_fast& ); \
        DECL_BIN_OP_T( op , int ) \
        DECL_BIN_OP_T( op , unsigned int ) \
        DECL_BIN_OP_T( op , long ) \
        DECL_BIN_OP_T( op , unsigned long ) \
        DECL_BIN_OP_T( op , double ) \
        DECL_BIN_OP_T( op , const char* ) \
        DECL_BIN_OP_T( op , const sc_fxval_fast& ) \
        DECL_BIN_OP_OTHER( op )

    DECL_BIN_OP( * , mult )
    DECL_BIN_OP( + , add )
    DECL_BIN_OP( - , sub )
    DECL_BIN_OP( / , div )

    #undef DECL_BIN_OP_T
    #undef DECL_BIN_OP_OTHER
    #undef DECL_BIN_OP

    friend const sc_fxval operator<< ( const sc_fxnum_fast& , int );
    friend const sc_fxval operator>> ( const sc_fxnum_fast& , int );

    // Relational (including equality) operators
    #define DECL_REL_OP_T( op , tp ) \
        friend bool operator op ( const sc_fxnum_fast& , tp ); \
        friend bool operator op ( tp , const sc_fxnum_fast& );
    #define DECL_REL_OP_OTHER( op ) \
        DECL_REL_OP_T( op , int64 ) \
        DECL_REL_OP_T( op , uint64 ) \
        DECL_REL_OP_T( op , const sc_int_base& ) \
        DECL_REL_OP_T( op , const sc_uint_base& ) \
        DECL_REL_OP_T( op , const sc_signed& ) \
        DECL_REL_OP_T( op , const sc_unsigned& )
    #define DECL_REL_OP( op ) \
        friend bool operator op ( const sc_fxnum_fast& , const sc_fxnum_fast& ); \
        DECL_REL_OP_T( op , int ) \
        DECL_REL_OP_T( op , unsigned int ) \
        DECL_REL_OP_T( op , long ) \

```

```

DECL_REL_OP_T( op , unsigned long ) \
DECL_REL_OP_T( op , double ) \
DECL_REL_OP_T( op , const char* ) \
DECL_REL_OP_T( op , const sc_fxval_fast& ) \
DECL_REL_OP_OTHER( op )
DECL_REL_OP( < )
DECL_REL_OP( <= )
DECL_REL_OP( > )
DECL_REL_OP( >= )
DECL_REL_OP( == )
DECL_REL_OP( != )

#undef DECL_REL_OP_T
#undef DECL_REL_OP_OTHER
#undef DECL_REL_OP

// Assignment operators
#define DECL_ASN_OP_T( op , tp ) \
    sc_fxnum& operator op( tp ); \
DECL_ASN_OP_T( op , int64 ) \
DECL_ASN_OP_T( op , uint64 ) \
DECL_ASN_OP_T( op , const sc_int_base& ) \
DECL_ASN_OP_T( op , const sc_uint_base& ) \
DECL_ASN_OP_T( op , const sc_signed& ) \
DECL_ASN_OP_T( op , const sc_unsigned& )
#define DECL_ASN_OP( op ) \
DECL_ASN_OP_T( op , int ) \
DECL_ASN_OP_T( op , unsigned int ) \
DECL_ASN_OP_T( op , long ) \
DECL_ASN_OP_T( op , unsigned long ) \
DECL_ASN_OP_T( op , double ) \
DECL_ASN_OP_T( op , const char* ) \
DECL_ASN_OP_T( op , const sc_fxval& ) \
DECL_ASN_OP_T( op , const sc_fxval_fast& ) \
DECL_ASN_OP_T( op , const sc_fxnum& ) \
DECL_ASN_OP_T( op , const sc_fxnum_fast& ) \
DECL_ASN_OP_OTHER( op )
DECL_ASN_OP( = )
DECL_ASN_OP( *= )
DECL_ASN_OP( /= )
DECL_ASN_OP( += )
DECL_ASN_OP( -= )
DECL_ASN_OP_T( <<= , int )
DECL_ASN_OP_T( >>= , int )

#undef DECL_ASN_OP_T
#undef DECL_ASN_OP_OTHER
#undef DECL_ASN_OP

// Auto-increment and auto-decrement
const sc_fxval_fast operator++( int );
const sc_fxval_fast operator--( int );
sc_fxnum_fast& operator++();
sc_fxnum_fast& operator--();

```

```

// Bit selection
const sc_fxnum_bitref† operator[] ( int ) const;
sc_fxnum_bitref† operator[] ( int );

// Part selection
const sc_fxnum_fast_subref† operator() ( int , int ) const;
sc_fxnum_fast_subref† operator() ( int , int );
const sc_fxnum_fast_subref† range( int , int ) const;
sc_fxnum_fast_subref† range( int , int );
const sc_fxnum_fast_subref† operator() () const;
sc_fxnum_fast_subref† operator() ();
const sc_fxnum_fast_subref† range() const;
sc_fxnum_fast_subref† range();

// Implicit conversion
operator double() const;

// Explicit conversion to primitive types
short to_short() const;
unsigned short to_ushort() const;
int to_int() const;
unsigned int to_uint() const;
long to_long() const;
unsigned long to_ulong() const;
int64 to_int64() const;
uint64 to_uint64() const;
float to_float() const;
double to_double() const;

// Explicit conversion to character string
const std::string to_string() const;
const std::string to_string( sc_numrep ) const;
const std::string to_string( sc_numrep , bool ) const;
const std::string to_string( sc_fmt ) const;
const std::string to_string( sc_numrep , sc_fmt ) const;
const std::string to_string( sc_numrep , bool , sc_fmt ) const;
const std::string to_dec() const;
const std::string to_bin() const;
const std::string to_oct() const;
const std::string to_hex() const;

// Query value
bool is_neg() const;
bool is_zero() const;
bool quantization_flag() const;
bool overflow_flag() const;
const sc_fxval_fast value() const;

// Query parameters
int wl() const;
int iwl() const;
sc_q_mode q_mode() const;
sc_o_mode o_mode() const;

```

```

    int n_bits() const;
    const sc_fxtype_params& type_params() const;
    const sc_fxcast_switch& cast_switch() const;

    // Print or dump content
    void print( std::ostream& = std::cout ) const;
    void scan( std::istream& = std::cin );
    void dump( std::ostream& = std::cout ) const;

private:
    // Disabled
    sc_fxnum_fast();
    sc_fxnum_fast( const sc_fxnum_fast& );
};

} // namespace sc_dt

```

### 7.10.11.3 Constraints on usage

An application shall not directly create an instance of type **sc\_fxnum\_fast**. An application may use a pointer to **sc\_fxnum\_fast** or a reference to **sc\_fxnum\_fast** to refer to an object of a class derived from **sc\_fxnum\_fast**.

### 7.10.11.4 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ numeric representation to **sc\_fxnum\_fast**, using truncation or sign-extension, as described in 7.10.4.

### 7.10.11.5 Implicit type conversion

operator **double**() const;

Operator **double** shall provide implicit type conversion from **sc\_fxnum\_fast** to double.

### 7.10.11.6 Explicit type conversion

```

short to_short() const;
unsigned short to_ushort() const;
int to_int() const;
unsigned int to_uint() const;
long to_long() const;
unsigned long to_ulong() const;
int64 to_int64() const;
uint64 to_uint64() const;
float to_float() const;
double to_double() const;

```

These member functions shall perform conversion to C++ numeric types.

```

const std::string to_string() const;
const std::string to_string( sc_numrep ) const;
const std::string to_string( sc_numrep , bool ) const;
const std::string to_string( sc_fmt ) const;
const std::string to_string( sc_numrep , sc_fmt ) const;
const std::string to_string( sc_numrep , bool , sc_fmt ) const;

```

```
const std::string to_dec() const;
const std::string to_bin() const;
const std::string to_oct() const;
const std::string to_hex() const;
```

These member functions shall perform the conversion to a **string** representation, as described in 7.2.11, 7.10.8, and 7.10.8.1.

### 7.10.12 sc\_fxval

#### 7.10.12.1 Description

Class **sc\_fxval** is the variable-precision fixed-point type. It may hold the value of any of the fixed-point types and performs the variable-precision fixed-point arithmetic operations. Type casting shall be performed by the fixed-point types themselves. Limited variable-precision type **sc\_fxval\_fast** and variable-precision type **sc\_fxval** may be mixed freely.

#### 7.10.12.2 Class definition

```
namespace sc_dt {

class sc_fxval
{
public:
    // Constructors and destructor
    sc_fxval();
    sc_fxval( int );
    sc_fxval( unsigned int );
    sc_fxval( long );
    sc_fxval( unsigned long );
    sc_fxval( double );
    sc_fxval( const char* );
    sc_fxval( const sc_fxval& );
    sc_fxval( const sc_fxval_fast& );
    sc_fxval( const sc_fxnum& );
    sc_fxval( const sc_fxnum_fast& );
    sc_fxval( int64 );
    sc_fxval( uint64 );
    sc_fxval( const sc_int_base& );
    sc_fxval( const sc_uint_base& );
    sc_fxval( const sc_signed& );
    sc_fxval( const sc_unsigned& );
    ~sc_fxval();

    // Unary operators
    const sc_fxval operator- () const;
    const sc_fxval& operator+ () const;
    friend void neg( sc_fxval& , const sc_fxval& );

    // Binary operators
    #define DECL_BIN_OP_T( op , tp ) \
        friend const sc_fxval operator op ( const sc_fxval& , tp ); \
        friend const sc_fxval operator op ( tp , const sc_fxval& );
    #define DECL_BIN_OP_OTHER( op ) \
```

```

DECL_BIN_OP_T( op , int64 ) \
DECL_BIN_OP_T( op , uint64 ) \
DECL_BIN_OP_T( op , const sc_int_base& ) \
DECL_BIN_OP_T( op , const sc_uint_base& ) \
DECL_BIN_OP_T( op , const sc_signed& ) \
DECL_BIN_OP_T( op , const sc_unsigned& )
#define DECL_BIN_OP( op , dummy ) \
    friend const sc_fxval operator op ( const sc_fxval& , const sc_fxval& ); \
    DECL_BIN_OP_T( op , int ) \
    DECL_BIN_OP_T( op , unsigned int ) \
    DECL_BIN_OP_T( op , long ) \
    DECL_BIN_OP_T( op , unsigned long ) \
    DECL_BIN_OP_T( op , double ) \
    DECL_BIN_OP_T( op , const char* ) \
    DECL_BIN_OP_T( op , const sc_fxval_fast& ) \
    DECL_BIN_OP_T( op , const sc_fxnum_fast& ) \
    DECL_BIN_OP_OTHER( op )

DECL_BIN_OP( * , mult )
DECL_BIN_OP( + , add )
DECL_BIN_OP( - , sub )
DECL_BIN_OP( / , div )

friend const sc_fxval operator<< ( const sc_fxval& , int );
friend const sc_fxval operator>> ( const sc_fxval& , int );

// Relational (including equality) operators
#define DECL_REL_OP_T( op , tp ) \
    friend bool operator op ( const sc_fxval& , tp ); \
    friend bool operator op ( tp , const sc_fxval& );
#define DECL_REL_OP_OTHER( op ) \
    DECL_REL_OP_T( op , int64 ) \
    DECL_REL_OP_T( op , uint64 ) \
    DECL_REL_OP_T( op , const sc_int_base& ) \
    DECL_REL_OP_T( op , const sc_uint_base& ) \
    DECL_REL_OP_T( op , const sc_signed& ) \
    DECL_REL_OP_T( op , const sc_unsigned& )
#define DECL_REL_OP( op ) \
    friend bool operator op ( const sc_fxval& , const sc_fxval& ); \
    DECL_REL_OP_T( op , int ) \
    DECL_REL_OP_T( op , unsigned int ) \
    DECL_REL_OP_T( op , long ) \
    DECL_REL_OP_T( op , unsigned long ) \
    DECL_REL_OP_T( op , double ) \
    DECL_REL_OP_T( op , const char* ) \
    DECL_REL_OP_T( op , const sc_fxval_fast& ) \
    DECL_REL_OP_T( op , const sc_fxnum_fast& ) \
    DECL_REL_OP_OTHER( op )

DECL_REL_OP( < )
DECL_REL_OP( <= )
DECL_REL_OP( > )
DECL_REL_OP( >= )
DECL_REL_OP( == )

```

```
DECL_REL_OP( != )

// Assignment operators
#define DECL_ASN_OP_T( op , tp ) \
    sc_fxval& operator op( tp );
#define DECL_ASN_OP_OTHER( op ) \
    DECL_ASN_OP_T( op , int64 ) \
    DECL_ASN_OP_T( op , uint64 ) \
    DECL_ASN_OP_T( op , const sc_int_base& ) \
    DECL_ASN_OP_T( op , const sc_uint_base& ) \
    DECL_ASN_OP_T( op , const sc_signed& ) \
    DECL_ASN_OP_T( op , const sc_unsigned& )
#define DECL_ASN_OP( op ) \
    DECL_ASN_OP_T( op , int ) \
    DECL_ASN_OP_T( op , unsigned int ) \
    DECL_ASN_OP_T( op , long ) \
    DECL_ASN_OP_T( op , unsigned long ) \
    DECL_ASN_OP_T( op , double ) \
    DECL_ASN_OP_T( op , const char* ) \
    DECL_ASN_OP_T( op , const sc_fxval& ) \
    DECL_ASN_OP_T( op , const sc_fxval_fast& ) \
    DECL_ASN_OP_T( op , const sc_fxnum& ) \
    DECL_ASN_OP_T( op , const sc_fxnum_fast& ) \
    DECL_ASN_OP_OTHER( op )

DECL_ASN_OP( = )
DECL_ASN_OP( *= )
DECL_ASN_OP( /= )
DECL_ASN_OP( += )
DECL_ASN_OP( -= )

DECL_ASN_OP_T( <<= , int )
DECL_ASN_OP_T( >>= , int )

// Auto-increment and auto-decrement
const sc_fxval operator++( int );
const sc_fxval operator--( int );
sc_fxval& operator++();
sc_fxval& operator--();

// Implicit conversion
operator double() const;

// Explicit conversion to primitive types
short to_short() const;
unsigned short to_ushort() const;
int to_int() const;
unsigned int to_uint() const;
long to_long() const;
unsigned long to_ulong() const;
int64 to_int64() const;
uint64 to_uint64() const;
float to_float() const;
double to_double() const;
```



```

    // Explicit conversion to character string
    const std::string to_string() const;
    const std::string to_string( sc_numrep ) const;
    const std::string to_string( sc_numrep , bool ) const;
    const std::string to_string( sc_fmt ) const;
    const std::string to_string( sc_numrep , sc_fmt ) const;
    const std::string to_string( sc_numrep , bool , sc_fmt ) const;
    const std::string to_dec() const;
    const std::string to_bin() const;
    const std::string to_oct() const;
    const std::string to_hex() const;

    // Methods
    bool is_neg() const;
    bool is_zero() const;
    void print( std::ostream& = std::cout ) const;
    void scan( std::istream& = std::cin );
    void dump( std::ostream& = std::cout ) const;
};

} // namespace sc_dt

```

### 7.10.12.3 Constraints on usage

A **sc\_fxval** object that is declared without an initial value shall be uninitialized (unless it is declared as static, in which case it shall be initialized to zero). Uninitialized objects may be used wherever an initialized object is permitted. The result of an operation on an uninitialized object is undefined.

### 7.10.12.4 Public constructors

The constructor argument shall be taken as the initial value of the **sc\_fxval** object. The default constructor shall not initialize the value.

### 7.10.12.5 Operators

The operators that shall be defined for **sc\_fxval** are given in Table 46.

**Table 46—Operators for **sc\_fxval****

| Operator class | Operators in class    |
|----------------|-----------------------|
| Arithmetic     | * / - << >> ++ --     |
| Equality       | == !=                 |
| Relational     | <<= >>=               |
| Assignment     | = *= /= += -= <<= >>= |

**operator<<** and **operator>>** define arithmetic shifts that perform sign extension.

The types of the operands shall be as defined in 7.10.4.

#### 7.10.12.6 Implicit type conversion

operator **double**() const;

Operator **double** can be used for implicit type conversion to the C++ type **double**.

#### 7.10.12.7 Explicit type conversion

short **to\_short**() const;  
 unsigned short **to\_ushort**() const;  
 int **to\_int**() const;  
 unsigned int **to\_uint**() const;  
 long **to\_long**() const;  
 unsigned long **to\_ulong**() const;  
 int64 **to\_int64**() const;  
 uint64 **to\_uint64**() const;  
 float **to\_float**() const;  
 double **to\_double**() const;

These member functions shall perform the conversion to the respective C++ numeric types.

const std::string **to\_string**() const;  
 const std::string **to\_string**( sc\_numrep ) const;  
 const std::string **to\_string**( sc\_numrep , bool ) const;  
 const std::string **to\_string**( sc\_fmt ) const;  
 const std::string **to\_string**( sc\_numrep , sc\_fmt ) const;  
 const std::string **to\_string**( sc\_numrep , bool , sc\_fmt ) const;  
 const std::string **to\_dec**() const;  
 const std::string **to\_bin**() const;  
 const std::string **to\_oct**() const;  
 const std::string **to\_hex**() const;

These member functions shall perform the conversion to a **string** representation, as described in 7.2.11, 7.10.8, and 7.10.8.1.

#### 7.10.13 sc\_fxval\_fast

##### 7.10.13.1 Description

Type **sc\_fxval\_fast** is the limited variable-precision fixed-point type and shall be limited to a mantissa of 53 bits. It may hold the value of any of the fixed-point types and shall be used to perform the limited variable-precision fixed-point arithmetic operations. Limited variable-precision fixed-point type **sc\_fxval\_fast** and variable-precision fixed-point type **sc\_fxval** may be mixed freely.

### 7.10.13.2 Class definition

```
namespace sc_dt {

class sc_fxval_fast
{
    public:
        sc_fxval_fast();
        sc_fxval_fast( int );
        sc_fxval_fast( unsigned int );
        sc_fxval_fast( long );
        sc_fxval_fast( unsigned long );
        sc_fxval_fast( double );
        sc_fxval_fast( const char* );
        sc_fxval_fast( const sc_fxval& );
        sc_fxval_fast( const sc_fxval_fast& );
        sc_fxval_fast( const sc_fxnum& );
        sc_fxval_fast( const sc_fxnum_fast& );
        sc_fxval_fast( int64 );
        sc_fxval_fast( uint64 );
        sc_fxval_fast( const sc_int_base& );
        sc_fxval_fast( const sc_uint_base& );
        sc_fxval_fast( const sc_signed& );
        sc_fxval_fast( const sc_unsigned& );
        ~sc_fxval_fast();

        // Unary operators
        const sc_fxval_fast operator- () const;
        const sc_fxval_fast& operator+ () const;

        // Binary operators
        #define DECL_BIN_OP_T( op , tp ) \
            friend const sc_fxval_fast operator op ( const sc_fxval_fast& , tp ); \
            friend const sc_fxval_fast operator op ( tp , const sc_fxval_fast& );

        #define DECL_BIN_OP_OTHER( op ) \
            DECL_BIN_OP_T( op , int64 ) \
            DECL_BIN_OP_T( op , uint64 ) \
            DECL_BIN_OP_T( op , const sc_int_base& ) \
            DECL_BIN_OP_T( op , const sc_uint_base& ) \
            DECL_BIN_OP_T( op , const sc_signed& ) \
            DECL_BIN_OP_T( op , const sc_unsigned& )

        #define DECL_BIN_OP( op , dummy ) \
            friend const sc_fxval_fast operator op ( const sc_fxval_fast& , const sc_fxval_fast& ); \
            DECL_BIN_OP_T( op , int ) \
            DECL_BIN_OP_T( op , unsigned int ) \
            DECL_BIN_OP_T( op , long ) \
            DECL_BIN_OP_T( op , unsigned long ) \
            DECL_BIN_OP_T( op , double ) \
            DECL_BIN_OP_T( op , const char* ) \
            DECL_BIN_OP_OTHER( op )

```

```

DECL_BIN_OP( *, mult )
DECL_BIN_OP( +, add )
DECL_BIN_OP( -, sub )
DECL_BIN_OP( /, div )
friend const sc_fxval_fast operator<<( const sc_fxval_fast& , int );
friend const sc_fxval_fast operator>>( const sc_fxval_fast& , int );

// Relational (including equality) operators
#define DECL_REL_OP_T( op , tp ) \
    friend bool operator op ( const sc_fxval_fast& , tp );\
    friend bool operator op ( tp , const sc_fxval_fast& );
#define DECL_REL_OP_OTHER( op ) \
    DECL_REL_OP_T( op , int64 ) \
    DECL_REL_OP_T( op , uint64 ) \
    DECL_REL_OP_T( op , const sc_int_base& ) \
    DECL_REL_OP_T( op , const sc_uint_base& ) \
    DECL_REL_OP_T( op , const sc_signed& ) \
    DECL_REL_OP_T( op , const sc_unsigned& )
#define DECL_REL_OP( op ) \
    friend bool operator op ( const sc_fxval_fast& , const sc_fxval_fast& );\
    DECL_REL_OP_T( op , int ) \
    DECL_REL_OP_T( op , unsigned int ) \
    DECL_REL_OP_T( op , long ) \
    DECL_REL_OP_T( op , unsigned long ) \
    DECL_REL_OP_T( op , double ) \
    DECL_REL_OP_T( op , const char* ) \
    DECL_REL_OP_OTHER( op )

DECL_REL_OP( < )
DECL_REL_OP( <= )
DECL_REL_OP( > )
DECL_REL_OP( >= )
DECL_REL_OP( == )
DECL_REL_OP( != )

// Assignment operators
#define DECL_ASN_OP_T( op , tp ) sc_fxval_fast& operator op( tp );
#define DECL_ASN_OP_OTHER( op ) \
    DECL_ASN_OP_T( op , int64 ) \
    DECL_ASN_OP_T( op , uint64 ) \
    DECL_ASN_OP_T( op , const sc_int_base& ) \
    DECL_ASN_OP_T( op , const sc_uint_base& ) \
    DECL_ASN_OP_T( op , const sc_signed& ) \
    DECL_ASN_OP_T( op , const sc_unsigned& )
#define DECL_ASN_OP( op ) \
    DECL_ASN_OP_T( op , int ) \
    DECL_ASN_OP_T( op , unsigned int ) \
    DECL_ASN_OP_T( op , long ) \
    DECL_ASN_OP_T( op , unsigned long ) \
    DECL_ASN_OP_T( op , double ) \
    DECL_ASN_OP_T( op , const char* ) \
    DECL_ASN_OP_T( op , const sc_fxval& ) \
    DECL_ASN_OP_T( op , const sc_fxval_fast& ) \
    DECL_ASN_OP_T( op , const sc_fxnum& ) \

```

```
DECL_ASN_OP_T( op , const sc_fxnum_fast& ) \
DECL_ASN_OP_OTHER( top )

DECL_ASN_OP( = )
DECL_ASN_OP( *= )
DECL_ASN_OP( /= )
DECL_ASN_OP( += )
DECL_ASN_OP( -= )
DECL_ASN_OP_T( <<= , int )
DECL_ASN_OP_T( >>= , int )

// Auto-increment and auto-decrement
const sc_fxval_fast operator++ ( int );
const sc_fxval_fast operator-- ( int );
sc_fxval_fast& operator++ ();
sc_fxval_fast& operator-- ();

// Implicit conversion
operator double() const;

// Explicit conversion to primitive types
short to_short() const;
unsigned short to_ushort() const;
int to_int() const;
unsigned int to_uint() const;
long to_long() const;
unsigned long to_ulong() const;
int64 to_int64() const;
uint64 to_uint64() const;
float to_float() const;
double to_double() const;

// Explicit conversion to character string
const std::string to_string() const;
const std::string to_string( sc_numrep ) const;
const std::string to_string( sc_numrep , bool ) const;
const std::string to_string( sc_fmt ) const;
const std::string to_string( sc_numrep , sc_fmt ) const;
const std::string to_string( sc_numrep , bool , sc_fmt ) const;
const std::string to_dec() const;
const std::string to_bin() const;
const std::string to_oct() const;
const std::string to_hex() const;

// Other methods
bool is_neg() const;
bool is_zero() const;
void print( std::ostream& = std::cout ) const;
void scan( std::istream& = std::cin );
void dump( std::ostream& = std::cout ) const;
};

} // namespace sc_dt
```

### 7.10.13.3 Constraints on usage

A **sc\_fxval\_fast** object that is declared without an initial value shall be uninitialized (unless it is declared as static, in which case it shall be initialized to zero). Uninitialized objects may be used wherever an initialized object is permitted. The result of an operation on an uninitialized object is undefined.

### 7.10.13.4 Public constructors

The constructor argument shall be taken as the initial value of the **sc\_fxval\_fast** object. The default constructor shall not initialize the value.

### 7.10.13.5 Operators

The operators that shall be defined for **sc\_fxval\_fast** are given in Table 47.

**Table 47—Operators for **sc\_fxval\_fast****

| Operator class | Operators in class    |
|----------------|-----------------------|
| Arithmetic     | * / + - << >> ++ --   |
| Equality       | == !=                 |
| Relational     | <<= >>=               |
| Assignment     | = *= /= += -= <<= >>= |

NOTE—**operator<<** and **operator>>** define arithmetic shifts, not bitwise shifts. The difference is that no bits are lost and proper sign extension is done. Hence, these operators are also well-defined for signed types, such as **sc\_fxval\_fast**.

### 7.10.13.6 Implicit type conversion

**operator double()** const;

Operator **double** can be used for implicit type conversion to the C++ type **double**.

### 7.10.13.7 Explicit type conversion

**short to\_short()** const;  
**unsigned short to\_ushort()** const;  
**int to\_int()** const;  
**unsigned int to\_uint()** const;  
**long to\_long()** const;  
**unsigned long to\_ulong()** const;  
**int64 to\_int64()** const;  
**uint64 to\_uint64()** const;  
**float to\_float()** const;  
**double to\_double()** const;

These member functions shall perform the conversion to the respective C++ numeric types.

```

const std::string to_string() const;
const std::string to_string( sc_numrep ) const;
const std::string to_string( sc_numrep , bool ) const;
const std::string to_string( sc_fmt ) const;
const std::string to_string( sc_numrep , sc_fmt ) const;
const std::string to_string( sc_numrep , bool, sc_fmt ) const;
const std::string to_dec() const;
const std::string to_bin() const;
const std::string to_oct() const;
const std::string to_hex() const;

```

These member functions shall perform the conversion to a **string** representation, as described in 7.2.11, 7.10.8, and 7.10.8.1.

## 7.10.14 **sc\_fix**

### 7.10.14.1 Description

Class **sc\_fix** shall represent a signed (two's complement) finite-precision fixed-point value. The fixed-point type parameters **wl**, **iwl**, **q\_mode**, **o\_mode**, and **n\_bits** may be specified as constructor arguments.

### 7.10.14.2 Class definition

```

namespace sc_dt {

class sc_fix
: public sc_fxnum
{
public:
    // Constructors and destructor
    sc_fix();
    sc_fix( int , int );
    sc_fix( sc_q_mode , sc_o_mode );
    sc_fix( sc_q_mode , sc_o_mode, int );
    sc_fix( int , int , sc_q_mode , sc_o_mode );
    sc_fix( int , int , sc_q_mode , sc_o_mode, int );
    sc_fix( const sc_fxcast_switch& );
    sc_fix( int , int , const sc_fxcast_switch& );
    sc_fix( sc_q_mode , sc_o_mode , const sc_fxcast_switch& );
    sc_fix( sc_q_mode , sc_o_mode , int , const sc_fxcast_switch& );
    sc_fix( int , int , sc_q_mode , sc_o_mode , const sc_fxcast_switch& );
    sc_fix( int , int , sc_q_mode , sc_o_mode , int , const sc_fxcast_switch& );
    sc_fix( const sc_fxtype_params& );
    sc_fix( const sc_fxtype_params& , const sc_fxcast_switch& );

#define DECL_CTORS_T( tp ) \
    sc_fix( tp , int , int ); \
    sc_fix( tp , sc_q_mode , sc_o_mode ); \
    sc_fix( tp , sc_q_mode , sc_o_mode, int ); \
    sc_fix( tp , int , int , sc_q_mode , sc_o_mode ); \
    sc_fix( tp , int , int , sc_q_mode , sc_o_mode , int ); \
    sc_fix( tp , const sc_fxcast_switch& ); \
    sc_fix( tp , int , int , const sc_fxcast_switch& ); \
    sc_fix( tp , sc_q_mode , sc_o_mode , const sc_fxcast_switch& ); \

```

```

    sc_fix( tp , sc_q_mode , sc_o_mode , in , const sc_fxcast_switch& ); \
    sc_fix( tp , int , int , sc_q_mode , sc_o_mode , const sc_fxcast_switch& ); \
    sc_fix( tp , int , int , sc_q_mode , sc_o_mode , int , const sc_fxcast_switch& ); \
    sc_fix( tp , const sc_fxtype_params& ); \
    sc_fix( tp , const sc_fxtype_params& , const sc_fxcast_switch& );
#define DECL_CTORS_T_A( tp ) \
    sc_fix( tp ); \
    DECL_CTORS_T( tp )
#define DECL_CTORS_T_B( tp ) \
    explicit sc_fix( tp ); \
    DECL_CTORS_T( tp )

DECL_CTORS_T_A( int )
DECL_CTORS_T_A( unsigned int )
DECL_CTORS_T_A( long )
DECL_CTORS_T_A( unsigned long )
DECL_CTORS_T_A( double )
DECL_CTORS_T_A( const char* )
DECL_CTORS_T_A( const sc_fxval& )
DECL_CTORS_T_A( const sc_fxval_fast& )
DECL_CTORS_T_A( const sc_fxnum& )
DECL_CTORS_T_A( const sc_fxnum_fast& )
DECL_CTORS_T_B( int64 )
DECL_CTORS_T_B( uint64 )
DECL_CTORS_T_B( const sc_int_base& )
DECL_CTORS_T_B( const sc_uint_base& )
DECL_CTORS_T_B( const sc_signed& )
DECL_CTORS_T_B( const sc_unsigned& )
sc_fix( const sc_fix& );

// Unary bitwise operators
const sc_fix operator~( ) const;

// Binary bitwise operators
friend const sc_fix operator&( const sc_fix& , const sc_fix& );
friend const sc_fix operator&( const sc_fix& , const sc_fix_fast& );
friend const sc_fix operator&( const sc_fix_fast& , const sc_fix& );
friend const sc_fix operator|( const sc_fix& , const sc_fix& );
friend const sc_fix operator|( const sc_fix& , const sc_fix_fast& );
friend const sc_fix operator|( const sc_fix_fast& , const sc_fix& );
friend const sc_fix operator^( const sc_fix& , const sc_fix& );
friend const sc_fix operator^( const sc_fix& , const sc_fix_fast& );
friend const sc_fix operator^( const sc_fix_fast& , const sc_fix& );

sc_fix& operator=( const sc_fix& );

#define DECL_ASN_OP_T( op , tp ) \
    sc_fix& operator op ( tp );
#define DECL_ASN_OP_OTHER( op ) \
    DECL_ASN_OP_T( op , int64 ) \
    DECL_ASN_OP_T( op , uint64 ) \
    DECL_ASN_OP_T( op , const sc_int_base& ) \
    DECL_ASN_OP_T( op , const sc_uint_base& ) \
    DECL_ASN_OP_T( op , const sc_signed& ) \

```



```
DECL_ASN_OP_T( op , const sc_unsigned& )
#define DECL_ASN_OP( op ) \
DECL_ASN_OP_T( op , int ) \
DECL_ASN_OP_T( op , unsigned int ) \
DECL_ASN_OP_T( op , long ) \
DECL_ASN_OP_T( op , unsigned long ) \
DECL_ASN_OP_T( op , double ) \
DECL_ASN_OP_T( op , const char* ) \
DECL_ASN_OP_T( op , const sc_fxval& ) \
DECL_ASN_OP_T( op , const sc_fxval_fast& ) \
DECL_ASN_OP_T( op , const sc_fxnum& ) \
DECL_ASN_OP_T( op , const sc_fxnum_fast& ) \
DECL_ASN_OP_OTHER( op )

DECL_ASN_OP( = )
DECL_ASN_OP( *= )
DECL_ASN_OP( /= )
DECL_ASN_OP( += )
DECL_ASN_OP( -= )
DECL_ASN_OP_T( <=<= , int )
DECL_ASN_OP_T( >=>= , int )
DECL_ASN_OP_T( &= , const sc_fix& )
DECL_ASN_OP_T( &= , const sc_fix_fast& )
DECL_ASN_OP_T( |= , const sc_fix& )
DECL_ASN_OP_T( |= , const sc_fix_fast& )
DECL_ASN_OP_T( ^= , const sc_fix& )
DECL_ASN_OP_T( ^= , const sc_fix_fast& )

const sc_fxval operator++ ( int );
const sc_fxval operator-- ( int );
sc_fix& operator++ ();
sc_fix& operator-- ();

};

} // namespace sc_dt
```

#### 7.10.14.3 Constraints on usage

The word length shall be greater than zero. The number of saturated bits, if specified, shall not be less than zero.

#### 7.10.14.4 Public constructors

The constructor arguments may specify the fixed-point type parameters, as described in 7.10.1. The default constructor shall set fixed-point type parameters according to the fixed-point context in scope at the point of construction. An initial value may additionally be specified as a C++ or SystemC numeric object or as a string literal. A fixed-point cast switch may also be passed as a constructor argument to set the fixed-point casting, as described in 7.10.7.

#### 7.10.14.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ numeric representation to **sc\_fix**, using truncation or sign-extension, as described in 7.10.4.

### 7.10.14.6 Bitwise operators

Bitwise operators for all combinations of operands of type **sc\_fix** and **sc\_fix\_fast** shall be defined, as described in 7.10.4.

### 7.10.15 sc\_ufix

#### 7.10.15.1 Description

Class **sc\_ufix** shall represent an unsigned finite-precision fixed-point value. The fixed-point type parameters **wl**, **iwl**, **q\_mode**, **o\_mode**, and **n\_bits** may be specified as constructor arguments.

#### 7.10.15.2 Class definition

```
namespace sc_dt {

class sc_ufix
: public sc_fxnum
{
    public:
        // Constructors
        explicit sc_ufix();
        sc_ufix( int , int );
        sc_ufix( sc_q_mode , sc_o_mode );
        sc_ufix( sc_q_mode , sc_o_mode , int );
        sc_ufix( int , int , sc_q_mode , sc_o_mode );
        sc_ufix( int , int , sc_q_mode , sc_o_mode , int );
        explicit sc_ufix( const sc_fxcast_switch& );
        sc_ufix( int , int , const sc_fxcast_switch& );
        sc_ufix( sc_q_mode , sc_o_mode , const sc_fxcast_switch& );
        sc_ufix( sc_q_mode , sc_o_mode , int , const sc_fxcast_switch& );
        sc_ufix( int , int , sc_q_mode , sc_o_mode , const sc_fxcast_switch& );
        sc_ufix( int , int , sc_q_mode , sc_o_mode , int , const sc_fxcast_switch& );
        explicit sc_ufix( const sc_fxtype_params& );
        sc_ufix( const sc_fxtype_params& , const sc_fxcast_switch& );

#define DECL_CTORS_T( tp ) \
    sc_ufix( tp , int , int ); \
    sc_ufix( tp , sc_q_mode , sc_o_mode ); \
    sc_ufix( tp , sc_q_mode , sc_o_mode , int ); \
    sc_ufix( tp , int , int , sc_q_mode , sc_o_mode ); \
    sc_ufix( tp , int , int , sc_q_mode , sc_o_mode , int ); \
    sc_ufix( tp , const sc_fxcast_switch& ); \
    sc_ufix( tp , int , int , const sc_fxcast_switch& ); \
    sc_ufix( tp , sc_q_mode , sc_o_mode , const sc_fxcast_switch& ); \
    sc_ufix( tp , sc_q_mode , sc_o_mode , int , const sc_fxcast_switch& ); \
    sc_ufix( tp , int , int , sc_q_mode , sc_o_mode , const sc_fxcast_switch& ); \
    sc_ufix( tp , int , int , sc_q_mode , sc_o_mode , int , const sc_fxcast_switch& ); \
    sc_ufix( tp , const sc_fxtype_params& ); \
    sc_ufix( tp , const sc_fxtype_params& , const sc_fxcast_switch& );
#define DECL_CTORS_T_A( tp ) \
    sc_ufix( tp ); \
    DECL_CTORS_T( tp )
}
```

```
#define DECL_CTORS_T_B( tp ) \
    explicit sc_ufix( tp ); \
    DECL_CTORS_T( tp )

DECL_CTORS_T_A( int )
DECL_CTORS_T_A( unsigned int )
DECL_CTORS_T_A( long )
DECL_CTORS_T_A( unsigned long )
DECL_CTORS_T_A( double )
DECL_CTORS_T_A( const char* )
DECL_CTORS_T_A( const sc_fxval& )
DECL_CTORS_T_A( const sc_fxval_fast& )
DECL_CTORS_T_A( const sc_fxnum& )
DECL_CTORS_T_A( const sc_fxnum_fast& )
DECL_CTORS_T_B( int64 )
DECL_CTORS_T_B( uint64 )
DECL_CTORS_T_B( const sc_int_base& )
DECL_CTORS_T_B( const sc_uint_base& )
DECL_CTORS_T_B( const sc_signed& )
DECL_CTORS_T_B( const sc_unsigned& )

#undef DECL_CTORS_T
#undef DECL_CTORS_T_A
#undef DECL_CTORS_T_B

// Copy constructor
sc_ufix( const sc_ufix& );

// Unary bitwise operators
const sc_ufix operator~( ) const;

// Binary bitwise operators
friend const sc_ufix operator&( const sc_ufix& , const sc_ufix& );
friend const sc_ufix operator&( const sc_ufix& , const sc_ufix_fast& );
friend const sc_ufix operator&( const sc_ufix_fast& , const sc_ufix& );
friend const sc_ufix operator|( const sc_ufix& , const sc_ufix& );
friend const sc_ufix operator|( const sc_ufix& , const sc_ufix_fast& );
friend const sc_ufix operator|( const sc_ufix_fast& , const sc_ufix& );
friend const sc_ufix operator^( const sc_ufix& , const sc_ufix& );
friend const sc_ufix operator^( const sc_ufix& , const sc_ufix_fast& );
friend const sc_ufix operator^( const sc_ufix_fast& , const sc_ufix& );

// Assignment operators
sc_ufix& operator=( const sc_ufix& );
```

```
#define DECL_ASN_OP_T( op , tp ) \
    sc_ufix& operator op ( tp );
#define DECL_ASN_OP_OTHER( op ) \
    DECL_ASN_OP_T( op , int64 ) \
    DECL_ASN_OP_T( op , uint64 ) \
    DECL_ASN_OP_T( op , const sc_int_base& ) \
    DECL_ASN_OP_T( op , const sc_uint_base& ) \
    DECL_ASN_OP_T( op , const sc_signed& ) \
    DECL_ASN_OP_T( op , const sc_unsigned& )
#define DECL_ASN_OP( op ) \
    DECL_ASN_OP_T( op , int ) \
    DECL_ASN_OP_T( op , unsigned int ) \
    DECL_ASN_OP_T( op , long ) \
    DECL_ASN_OP_T( op , unsigned long ) \
    DECL_ASN_OP_T( op , double ) \
    DECL_ASN_OP_T( op , const char* ) \
    DECL_ASN_OP_T( op , const sc_fxval& ) \
    DECL_ASN_OP_T( op , const sc_fxval_fast& ) \
    DECL_ASN_OP_T( op , const sc_fxnum& ) \
    DECL_ASN_OP_T( op , const sc_fxnum_fast& ) \
    DECL_ASN_OP_OTHER( op )

DECL_ASN_OP( = )
DECL_ASN_OP( *= )
DECL_ASN_OP( /= )
DECL_ASN_OP( += )
DECL_ASN_OP( -= )
DECL_ASN_OP_T( <=< , int )
DECL_ASN_OP_T( >=> , int )
DECL_ASN_OP_T( &= , const sc_ufix& )
DECL_ASN_OP_T( &= , const sc_ufix_fast& )
DECL_ASN_OP_T( |= , const sc_ufix& )
DECL_ASN_OP_T( |= , const sc_ufix_fast& )
DECL_ASN_OP_T( ^= , const sc_ufix& )
DECL_ASN_OP_T( ^= , const sc_ufix_fast& )

#undef DECL_ASN_OP_T
#undef DECL_ASN_OP_OTHER
#undef DECL_ASN_OP

// Auto-increment and auto-decrement
const sc_fxval operator++ ( int );
const sc_fxval operator-- ( int );
sc_ufix& operator++ ();
sc_ufix& operator-- ();

};

} // namespace sc_dt
```

### 7.10.15.3 Constraints on usage

The word length shall be greater than zero. The number of saturated bits, if specified, shall not be less than zero.

#### 7.10.15.4 Public constructors

The constructor arguments may specify the fixed-point type parameters, as described in 7.10.1. The default constructor shall set fixed-point type parameters according to the fixed-point context in scope at the point of construction. An initial value may additionally be specified as a C++ or SystemC numeric object or as a string literal. A fixed-point cast switch may also be passed as a constructor argument to set the fixed-point casting, as described in 7.10.7.

#### 7.10.15.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ numeric representation to **sc\_ufix**, using truncation or sign-extension, as described in 7.10.4.

#### 7.10.15.6 Bitwise operators

Bitwise operators for all combinations of operands of type **sc\_ufix** and **sc\_ufix\_fast** shall be defined, as described in 7.10.4.

#### 7.10.16 sc\_fix\_fast

##### 7.10.16.1 Description

Class **sc\_fix\_fast** shall represent a signed (two's complement) limited-precision fixed-point value. The fixed-point type parameters **wl**, **iwl**, **q\_mode**, **o\_mode**, and **n\_bits** may be specified as constructor arguments.

##### 7.10.16.2 Class definition

```
namespace sc_dt {  
  
class sc_fix_fast  
: public sc_fxnum_fast  
{  
    public:  
        // Constructors  
        sc_fix_fast();  
        sc_fix_fast( int , int );  
        sc_fix_fast( sc_q_mode , sc_o_mode );  
        sc_fix_fast( sc_q_mode , sc_o_mode , int );  
        sc_fix_fast( int , int , sc_q_mode , sc_o_mode );  
        sc_fix_fast( int , int , sc_q_mode , sc_o_mode , int );  
        sc_fix_fast( const sc_fxcast_switch& );  
        sc_fix_fast( int , int , const sc_fxcast_switch& );  
        sc_fix_fast( sc_q_mode , sc_o_mode , const sc_fxcast_switch& );  
        sc_fix_fast( sc_q_mode , sc_o_mode , int , const sc_fxcast_switch& );  
        sc_fix_fast( int , int , sc_q_mode , sc_o_mode , const sc_fxcast_switch& );  
        sc_fix_fast( int , int , sc_q_mode , sc_o_mode , int , const sc_fxcast_switch& );  
        sc_fix_fast( const sc_fxtype_params& );  
        sc_fix_fast( const sc_fxtype_params& , const sc_fxcast_switch& );  
};
```

```

#define DECL_CTORS_T( tp ) \
    sc_fix_fast( tp , int , int ); \
    sc_fix_fast( tp , sc_q_mode , sc_o_mode ); \
    sc_fix_fast( tp , sc_q_mode , sc_o_mode , int ); \
    sc_fix_fast( tp , int , int , sc_q_mode , sc_o_mode ); \
    sc_fix_fast( tp , int , int , sc_q_mode , sc_o_mode , int ); \
    sc_fix_fast( tp , const sc_fxcast_switch& ); \
    sc_fix_fast( tp , int , int , const sc_fxcast_switch& ); \
    sc_fix_fast( tp , sc_q_mode , sc_o_mode , const sc_fxcast_switch& ); \
    sc_fix_fast( tp , sc_q_mode , sc_o_mode , int , const sc_fxcast_switch& ); \
    sc_fix_fast( tp , int , int , sc_q_mode , sc_o_mode , const sc_fxcast_switch& ); \
    sc_fix_fast( tp , int , int , sc_q_mode , sc_o_mode , int , const sc_fxcast_switch& ); \
    sc_fix_fast( tp , const sc_fxtype_params& ); \
    sc_fix_fast( tp , const sc_fxtype_params& , const sc_fxcast_switch& );
#define DECL_CTORS_T_A( tp ) \
    sc_fix_fast( tp ); \
    DECL_CTORS_T( tp )
#define DECL_CTORS_T_B( tp ) \
    explicit sc_fix_fast( tp ); \
    DECL_CTORS_T( tp )

DECL_CTORS_T_A( int )
DECL_CTORS_T_A( unsigned int )
DECL_CTORS_T_A( long )
DECL_CTORS_T_A( unsigned long )
DECL_CTORS_T_A( double )
DECL_CTORS_T_A( const char* )
DECL_CTORS_T_A( const sc_fxval& )
DECL_CTORS_T_A( const sc_fxval_fast& )
DECL_CTORS_T_A( const sc_fxnum& )
DECL_CTORS_T_A( const sc_fxnum_fast& )
DECL_CTORS_T_B( int64 )
DECL_CTORS_T_B( uint64 )
DECL_CTORS_T_B( const sc_int_base& )
DECL_CTORS_T_B( const sc_uint_base& )
DECL_CTORS_T_B( const sc_signed& )
DECL_CTORS_T_B( const sc_unsigned& )

// Copy constructor
sc_fix_fast( const sc_fix_fast& );

// Operators
const sc_fix_fast operator~( ) const;
friend const sc_fix_fast operator&( const sc_fix_fast& , const sc_fix_fast& );
friend const sc_fix_fast operator^( const sc_fix_fast& , const sc_fix_fast& );
friend const sc_fix_fast operator|( const sc_fix_fast& , const sc_fix_fast& );
sc_fix_fast& operator=( const sc_fix_fast& );

#define DECL_ASN_OP_T( op , tp ) \
    sc_fix_fast& operator op ( tp );
#define DECL_ASN_OP_OTHER( op ) \
    DECL_ASN_OP_T( op , int64 ) \
    DECL_ASN_OP_T( op , uint64 ) \
    DECL_ASN_OP_T( op , const sc_int_base& )

```

```
DECL_ASN_OP_T( op , const sc_uint_base& )\
DECL_ASN_OP_T( op , const sc_signed& )\
DECL_ASN_OP_T( op , const sc_unsigned& )\
#define DECL_ASN_OP( op ) \
DECL_ASN_OP_T( op , int ) \
DECL_ASN_OP_T( op , unsigned int ) \
DECL_ASN_OP_T( op , long ) \
DECL_ASN_OP_T( op , unsigned long ) \
DECL_ASN_OP_T( op , double ) \
DECL_ASN_OP_T( op , const char* ) \
DECL_ASN_OP_T( op , const sc_fxval& )\
DECL_ASN_OP_T( op , const sc_fxval_fast& )\
DECL_ASN_OP_T( op , const sc_fxnum& )\
DECL_ASN_OP_T( op , const sc_fxnum_fast& )\
DECL_ASN_OP_OTHER( op )

DECL_ASN_OP( = )
DECL_ASN_OP( *= )
DECL_ASN_OP( /= )
DECL_ASN_OP( += )
DECL_ASN_OP( -= )
DECL_ASN_OP_T( <=<= , int )
DECL_ASN_OP_T( >=>= , int )
DECL_ASN_OP_T( &= , const sc_fix& )
DECL_ASN_OP_T( &= , const sc_fix_fast& )
DECL_ASN_OP_T( |= , const sc_fix& )
DECL_ASN_OP_T( |= , const sc_fix_fast& )
DECL_ASN_OP_T( ^= , const sc_fix& )
DECL_ASN_OP_T( ^= , const sc_fix_fast& )

const sc_fxval_fast operator++ ( int );
const sc_fxval_fast operator-- ( int );
sc_fix_fast& operator++ ();
sc_fix_fast& operator-- ();
};

} // namespace sc_dt
```

### 7.10.16.3 Constraints on usage

The word length shall be greater than zero. The number of saturated bits, if specified, shall not be less than zero.

**sc\_fix\_fast** shall use double-precision (floating-point) values. The mantissa of a double-precision value is limited to 53 bits, so bit-true behavior cannot be guaranteed with the limited-precision types.

### 7.10.16.4 Public constructors

The constructor arguments may specify the fixed-point type parameters, as described in 7.10.1. The default constructor shall set fixed-point type parameters according to the fixed-point context in scope at the point of construction. An initial value may additionally be specified as a C++ or SystemC numeric object or as a string literal. A fixed-point cast switch may also be passed as a constructor argument to set the fixed-point casting, as described in 7.10.7.

### 7.10.16.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ numeric representation to **sc\_fix\_fast**, using truncation or sign-extension, as described in 7.10.4.

### 7.10.16.6 Bitwise operators

Bitwise operators for operands of type **sc\_fix\_fast** shall be defined, as described in 7.10.4.

### 7.10.17 sc\_ufix\_fast

#### 7.10.17.1 Description

Class **sc\_ufix\_fast** shall represent an unsigned limited-precision fixed-point value. The fixed-point type parameters **wl**, **iwl**, **q\_mode**, **o\_mode**, and **n\_bits** may be specified as constructor arguments.

#### 7.10.17.2 Class definition

```
namespace sc_dt {

class sc_ufix_fast
: public sc_fxnum_fast
{
public:
    // Constructors
    explicit sc_ufix_fast();
    sc_ufix_fast( int , int );
    sc_ufix_fast( sc_q_mode , sc_o_mode );
    sc_ufix_fast( sc_q_mode , sc_o_mode , int );
    sc_ufix_fast( int , int , sc_q_mode , sc_o_mode );
    sc_ufix_fast( int , int , sc_q_mode , sc_o_mode , int );
    explicit sc_ufix_fast( const sc_fxcast_switch& );
    sc_ufix_fast( int , int , const sc_fxcast_switch& );
    sc_ufix_fast( sc_q_mode , sc_o_mode , const sc_fxcast_switch& );
    sc_ufix_fast( sc_q_mode , sc_o_mode , int , const sc_fxcast_switch& );
    sc_ufix_fast( int , int , sc_q_mode , sc_o_mode , const sc_fxcast_switch& );
    sc_ufix_fast( int , int , sc_q_mode , sc_o_mode , int , const sc_fxcast_switch& );
    explicit sc_ufix_fast( const sc_fxtype_params& );
    sc_ufix_fast( const sc_fxtype_params& , const sc_fxcast_switch& );

#define DECL_CTORS_T( tp ) \
    sc_ufix_fast( tp , int , int ); \
    sc_ufix_fast( tp , sc_q_mode , sc_o_mode ); \
    sc_ufix_fast( tp , sc_q_mode , sc_o_mode , int ); \
    sc_ufix_fast( tp , int , int , sc_q_mode , sc_o_mode ); \
    sc_ufix_fast( tp , int , int , sc_q_mode , sc_o_mode , int ); \
    sc_ufix_fast( tp , const sc_fxcast_switch& ); \
    sc_ufix_fast( tp , int , int , const sc_fxcast_switch& ); \
    sc_ufix_fast( tp , sc_q_mode , sc_o_mode , const sc_fxcast_switch& ); \
    sc_ufix_fast( tp , sc_q_mode , sc_o_mode , int , const sc_fxcast_switch& ); \
    sc_ufix_fast( tp , int , int , sc_q_mode , sc_o_mode , const sc_fxcast_switch& ); \
    sc_ufix_fast( tp , int , int , sc_q_mode , sc_o_mode , int , const sc_fxcast_switch& ); \
    sc_ufix_fast( tp , const sc_fxtype_params& ); \
    sc_ufix_fast( tp , const sc_fxtype_params& , const sc_fxcast_switch& );
```



```
#define DECL_CTORS_T_A( tp ) \
    sc_ufix_fast( tp ); \
    DECL_CTORS_T( tp )
#define DECL_CTORS_T_B( tp ) \
    explicit sc_ufix_fast( tp ); \
    DECL_CTORS_T( tp )

DECL_CTORS_T_A( int )
DECL_CTORS_T_A( unsigned int )
DECL_CTORS_T_A( long )
DECL_CTORS_T_A( unsigned long )
DECL_CTORS_T_A( double )
DECL_CTORS_T_A( const char* )
DECL_CTORS_T_A( const sc_fxval& )
DECL_CTORS_T_A( const sc_fxval_fast& )
DECL_CTORS_T_A( const sc_fxnum& )
DECL_CTORS_T_A( const sc_fxnum_fast& )
DECL_CTORS_T_B( int64 )
DECL_CTORS_T_B( uint64 )
DECL_CTORS_T_B( const sc_int_base& )
DECL_CTORS_T_B( const sc_uint_base& )
DECL_CTORS_T_B( const sc_signed& )
DECL_CTORS_T_B( const sc_unsigned& )

#undef DECL_CTORS_T
#undef DECL_CTORS_T_A
#undef DECL_CTORS_T_B

// Copy constructor
sc_ufix_fast( const sc_ufix_fast& );

// Unary bitwise operators
const sc_ufix_fast operator~( ) const;

// Binary bitwise operators
friend const sc_ufix_fast operator&( const sc_ufix_fast& , const sc_ufix_fast& );
friend const sc_ufix_fast operator^( const sc_ufix_fast& , const sc_ufix_fast& );
friend const sc_ufix_fast operator|( const sc_ufix_fast& , const sc_ufix_fast& );

// Assignment operators
sc_ufix_fast& operator=( const sc_ufix_fast& );
#define DECL_ASN_OP_T( op , tp ) \
    sc_ufix_fast& operator op ( tp );
#define DECL_ASN_OP_OTHER( op ) \
    DECL_ASN_OP_T( op , int64 ) \
    DECL_ASN_OP_T( op , uint64 ) \
    DECL_ASN_OP_T( op , const sc_int_base& ) \
    DECL_ASN_OP_T( op , const sc_uint_base& ) \
    DECL_ASN_OP_T( op , const sc_signed& ) \
    DECL_ASN_OP_T( op , const sc_unsigned& )
#define DECL_ASN_OP( op ) \
    DECL_ASN_OP_T( op , int ) \
    DECL_ASN_OP_T( op , unsigned int ) \
```

```
DECL_ASN_OP_T( op , long ) \
DECL_ASN_OP_T( op , unsigned long ) \
DECL_ASN_OP_T( op , double ) \
DECL_ASN_OP_T( op , const char* ) \
DECL_ASN_OP_T( op , const sc_fxval& ) \
DECL_ASN_OP_T( op , const sc_fxval_fast& ) \
DECL_ASN_OP_T( op , const sc_fxnum& ) \
DECL_ASN_OP_T( op , const sc_fxnum_fast& ) \
DECL_ASN_OP_OTHER( op )

DECL_ASN_OP( = )
DECL_ASN_OP( *= )
DECL_ASN_OP( /= )
DECL_ASN_OP( += )
DECL_ASN_OP( -= )
DECL_ASN_OP_T( <<= , int )
DECL_ASN_OP_T( >>= , int )
DECL_ASN_OP_T( &= , const sc_ufix& )
DECL_ASN_OP_T( &= , const sc_ufix_fast& )
DECL_ASN_OP_T( |= , const sc_ufix& )
DECL_ASN_OP_T( |= , const sc_ufix_fast& )
DECL_ASN_OP_T( ^= , const sc_ufix& )
DECL_ASN_OP_T( ^= , const sc_ufix_fast& )

#undef DECL_ASN_OP_T
#undef DECL_ASN_OP_OTHER
#undef DECL_ASN_OP

// Auto-increment and auto-decrement
const sc_fxval_fast operator++ ( int );
const sc_fxval_fast operator-- ( int );
sc_ufix_fast& operator++ ();
sc_ufix_fast& operator-- ();
};

} // namespace sc_dt
```

### 7.10.17.3 Constraints on usage

The word length shall be greater than zero. The number of saturated bits, if specified, shall not be less than zero.

**sc\_ufix\_fast** shall use double-precision (floating-point) values. The mantissa of a double-precision value is limited to 53 bits, so bit-true behavior cannot be guaranteed with the limited-precision types.

### 7.10.17.4 Public constructors

The constructor arguments may specify the fixed-point type parameters, as described in 7.10.1. The default constructor shall set fixed-point type parameters according to the fixed-point context in scope at the point of construction. An initial value may additionally be specified as a C++ or SystemC numeric object or as a string literal. A fixed-point cast switch may also be passed as a constructor argument to set the fixed-point casting, as described in 7.10.7.

### 7.10.17.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ numeric representation to **sc\_ufix\_fast**, using truncation or sign-extension, as described in 7.10.4.

### 7.10.17.6 Bitwise operators

Bitwise operators for operands of type **sc\_ufix\_fast** shall be defined, as described in 7.10.4.

### 7.10.18 sc\_fixed

#### 7.10.18.1 Description

Class template **sc\_fixed** shall represent a signed (two's complement) finite-precision fixed-point value. The fixed-point type parameters **wl**, **iwl**, **q\_mode**, **o\_mode**, and **n\_bits** shall be specified by the template arguments.

Any public member functions of the base class **sc\_fix** that are overridden in class **sc\_fixed** shall have the same behavior in the two classes. Any public member functions of the base class not overridden in this way shall be publicly inherited by class **sc\_fixed**.

#### 7.10.18.2 Class definition

```
namespace sc_dt {

template <int W, int I,
         sc_q_mode Q = SC_DEFAULT_Q_MODE_,
         sc_o_mode O = SC_DEFAULT_O_MODE_, int N = SC_DEFAULT_N_BITS_>
class sc_fixed
: public sc_fix
{
public:
    // Constructors
    sc_fixed();
    sc_fixed( const sc_fxcast_switch& );

#define DECL_CTORS_T_A( tp ) \
    sc_fixed( tp ); \
    sc_fixed( tp , const sc_fxcast_switch& );
#define DECL_CTORS_T_B( tp ) \
    sc_fixed( tp ); \
    sc_fixed( tp , const sc_fxcast_switch& );

    DECL_CTORS_T_A( int )
    DECL_CTORS_T_A( unsigned int )
    DECL_CTORS_T_A( long )
    DECL_CTORS_T_A( unsigned long )
    DECL_CTORS_T_A( double )
    DECL_CTORS_T_A( const char* )
    DECL_CTORS_T_A( const sc_fxval& )
    DECL_CTORS_T_A( const sc_fxval_fast& )
    DECL_CTORS_T_A( const sc_fxnum& )
    DECL_CTORS_T_A( const sc_fxnum_fast& )
    DECL_CTORS_T_B( int64 )

```

```

DECL_CTORS_T_B( uint64 )
DECL_CTORS_T_B( const sc_int_base& )
DECL_CTORS_T_B( const sc_uint_base& )
DECL_CTORS_T_B( const sc_signed& )
DECL_CTORS_T_B( const sc_unsigned& )
sc_fixed( const sc_fixed<W,I,Q,O,N>& );

// Operators
sc_fixed& operator= ( const sc_fixed<W,I,Q,O,N>& );

#define DECL_ASN_OP_T( op , tp ) \
    sc_fixed& operator op ( tp );
#define DECL_ASN_OP_OTHER( op ) \
    DECL_ASN_OP_T( op , int64 ) \
    DECL_ASN_OP_T( op , uint64 ) \
    DECL_ASN_OP_T( op , const sc_int_base& ) \
    DECL_ASN_OP_T( op , const sc_uint_base& ) \
    DECL_ASN_OP_T( op , const sc_signed& ) \
    DECL_ASN_OP_T( op , const sc_unsigned& )
#define DECL_ASN_OP( op ) \
    DECL_ASN_OP_T( op , int ) \
    DECL_ASN_OP_T( op , unsigned int ) \
    DECL_ASN_OP_T( op , long ) \
    DECL_ASN_OP_T( op , unsigned long ) \
    DECL_ASN_OP_T( op , double ) \
    DECL_ASN_OP_T( op , const char* ) \
    DECL_ASN_OP_T( op , const sc_fxval& ) \
    DECL_ASN_OP_T( op , const sc_fxval_fast& ) \
    DECL_ASN_OP_T( op , const sc_fxnum& ) \
    DECL_ASN_OP_T( op , const sc_fxnum_fast& ) \
    DECL_ASN_OP_OTHER( op )

DECL_ASN_OP( = )
DECL_ASN_OP( *= )
DECL_ASN_OP( /= )
DECL_ASN_OP( += )
DECL_ASN_OP( -= )
DECL_ASN_OP_T( <<= , int )
DECL_ASN_OP_T( >>= , int )
DECL_ASN_OP_T( &= , const sc_fix& )
DECL_ASN_OP_T( &= , const sc_fix_fast& )
DECL_ASN_OP_T( |= , const sc_fix& )
DECL_ASN_OP_T( |= , const sc_fix_fast& )
DECL_ASN_OP_T( ^= , const sc_fix& )
DECL_ASN_OP_T( ^= , const sc_fix_fast& )

const sc_fxval operator++ ( int );
const sc_fxval operator-- ( int );
sc_fixed& operator++ ();
sc_fixed& operator-- ();

};

} // namespace sc_dt

```

### 7.10.18.3 Constraints on usage

The word length shall be greater than zero. The number of saturated bits, if specified, shall not be less than zero.

### 7.10.18.4 Public constructors

The initial value of an **sc\_fixed** object may be specified as a constructor argument, that is, a C++ or SystemC numeric object or a string literal. A fixed-point cast switch may also be passed as a constructor argument to set the fixed-point casting, as described in 7.10.7.

### 7.10.18.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ numeric representation to **sc\_fixed**, using truncation or sign-extension, as described in 7.10.4.

## 7.10.19 sc\_ufixed

### 7.10.19.1 Description

Class template **sc\_ufixed** represents an unsigned finite-precision fixed-point value. The fixed-point type parameters **wl**, **iwl**, **q\_mode**, **o\_mode**, and **n\_bits** shall be specified by the template arguments.

Any public member functions of the base class **sc\_ufix** that are overridden in class **sc\_ufixed** shall have the same behavior in the two classes. Any public member functions of the base class not overridden in this way shall be publicly inherited by class **sc\_ufixed**.

### 7.10.19.2 Class definition

```
namespace sc_dt {

template <int W, int I,
         sc_q_mode Q = SC_DEFAULT_Q_MODE_,
         sc_o_mode O = SC_DEFAULT_O_MODE_, int N = SC_DEFAULT_N_BITS_>
class sc_ufixed
: public sc_ufix
{
public:
    // Constructors
    explicit sc_ufixed();
    explicit sc_ufixed( const sc_fxcast_switch& );

    #define DECL_CTORS_T_A( tp ) \
        sc_ufixed( tp ); \
        sc_ufixed( tp , const sc_fxcast_switch& );
    #define DECL_CTORS_T_B( tp ) \
        explicit sc_ufixed( tp ); \
        sc_ufixed( tp , const sc_fxcast_switch& );
```

```

DECL_CTORS_T_A( int )
DECL_CTORS_T_A( unsigned int )
DECL_CTORS_T_A( long )
DECL_CTORS_T_A( unsigned long )
DECL_CTORS_T_A( double )
DECL_CTORS_T_A( const char* )
DECL_CTORS_T_A( const sc_fxval& )
DECL_CTORS_T_A( const sc_fxval_fast& )
DECL_CTORS_T_A( const sc_fxnum& )
DECL_CTORS_T_A( const sc_fxnum_fast& )
DECL_CTORS_T_B( int64 )
DECL_CTORS_T_B( uint64 )
DECL_CTORS_T_B( const sc_int_base& )
DECL_CTORS_T_B( const sc_uint_base& )
DECL_CTORS_T_B( const sc_signed& )
DECL_CTORS_T_B( const sc_unsigned& )

#undef DECL_CTORS_T_A
#undef DECL_CTORS_T_B

// Copy constructor
sc_ufixed( const sc_ufixed<W,I,Q,O,N>& );

// Assignment operators
sc_ufixed& operator= ( const sc_ufixed<W,I,Q,O,N>& );
#define DECL_ASN_OP_T( op , tp ) \
    sc_ufixed& operator op ( tp );
#define DECL_ASN_OP_OTHER( op ) \
    DECL_ASN_OP_T( op , int64 ) \
    DECL_ASN_OP_T( op , uint64 ) \
    DECL_ASN_OP_T( op , const sc_int_base& ) \
    DECL_ASN_OP_T( op , const sc_uint_base& ) \
    DECL_ASN_OP_T( op , const sc_signed& ) \
    DECL_ASN_OP_T( op , const sc_unsigned& )
#define DECL_ASN_OP( op ) \
    DECL_ASN_OP_T( op , int ) \
    DECL_ASN_OP_T( op , unsigned int ) \
    DECL_ASN_OP_T( op , long ) \
    DECL_ASN_OP_T( op , unsigned long ) \
    DECL_ASN_OP_T( op , double ) \
    DECL_ASN_OP_T( op , const char* ) \
    DECL_ASN_OP_T( op , const sc_fxval& ) \
    DECL_ASN_OP_T( op , const sc_fxval_fast& ) \
    DECL_ASN_OP_T( op , const sc_fxnum& ) \
    DECL_ASN_OP_T( op , const sc_fxnum_fast& ) \
    DECL_ASN_OP_OTHER( op )

DECL_ASN_OP( = )
DECL_ASN_OP( *= )
DECL_ASN_OP( /= )
DECL_ASN_OP( += )
DECL_ASN_OP( -= )
DECL_ASN_OP_T( <<= , int )
DECL_ASN_OP_T( >>= , int )

```

```
DECL_ASN_OP_T( &= , const sc_ufix& )
DECL_ASN_OP_T( &= , const sc_ufix_fast& )
DECL_ASN_OP_T( |= , const sc_ufix& )
DECL_ASN_OP_T( |= , const sc_ufix_fast& )
DECL_ASN_OP_T( ^= , const sc_ufix& )
DECL_ASN_OP_T( ^= , const sc_ufix_fast& )

#undef DECL_ASN_OP_T
#undef DECL_ASN_OP_OTHER
#undef DECL_ASN_OP

// Auto-increment and auto-decrement
const sc_fxval operator++ ( int );
const sc_fxval operator-- ( int );
sc_ufixed& operator++ ();
sc_ufixed& operator-- ();
};

} // namespace sc_dt
```

### 7.10.19.3 Constraints on usage

The word length shall be greater than zero. The number of saturated bits, if specified, shall not be less than zero.

### 7.10.19.4 Public constructors

The initial value of an **sc\_ufixed** object may be specified as a constructor argument that is a C++ or SystemC numeric object or a string literal. A fixed-point cast switch may also be passed as a constructor argument to set the fixed-point casting, as described in 7.10.7.

### 7.10.19.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ numeric representation to **sc\_ufixed**, using truncation or sign-extension, as described in 7.10.4.

## 7.10.20 **sc\_fixed\_fast**

### 7.10.20.1 Description

Class template **sc\_fixed\_fast** shall represent a signed (two's complement) limited-precision fixed-point type. The fixed-point type parameters **wl**, **iwl**, **q\_mode**, **o\_mode**, and **n\_bits** shall be specified by the template arguments.

Any public member functions of the base class **sc\_fix\_fast** that are overridden in class **sc\_fixed\_fast** shall have the same behavior in the two classes. Any public member functions of the base class not overridden in this way shall be publicly inherited by class **sc\_fixed\_fast**.

**7.10.20.2 Class definition**

```

namespace sc_dt {

template <int W, int I,
        sc_q_mode Q = SC_DEFAULT_Q_MODE_,
        sc_o_mode O = SC_DEFAULT_O_MODE_, int N = SC_DEFAULT_N_BITS_>
class sc_fixed_fast
: public sc_fix_fast
{
public:
    // Constructors
    sc_fixed_fast();
    sc_fixed_fast( const sc_fxcast_switch& );

#define DECL_CTORS_T_A( tp ) \
    sc_fixed_fast( tp ); \
    sc_fixed_fast( tp , const sc_fxcast_switch& );
#define DECL_CTORS_T_B( tp ) \
    sc_fixed_fast( tp ); \
    sc_fixed_fast( tp , const sc_fxcast_switch& );

    DECL_CTORS_T_A( int )
    DECL_CTORS_T_A( unsigned int )
    DECL_CTORS_T_A( long )
    DECL_CTORS_T_A( unsigned long )
    DECL_CTORS_T_A( double )
    DECL_CTORS_T_A( const char* )
    DECL_CTORS_T_A( const sc_fxval& )
    DECL_CTORS_T_A( const sc_fxval_fast& )
    DECL_CTORS_T_A( const sc_fxnum& )
    DECL_CTORS_T_A( const sc_fxnum_fast& )
    DECL_CTORS_T_B( int64 )
    DECL_CTORS_T_B( uint64 )
    DECL_CTORS_T_B( const sc_int_base& )
    DECL_CTORS_T_B( const sc_uint_base& )
    DECL_CTORS_T_B( const sc_signed& )
    DECL_CTORS_T_B( const sc_unsigned& )

    sc_fixed_fast( const sc_fixed_fast<W,I,Q,O,N>& );

    // Operators
    sc_fixed_fast& operator=( const
        sc_fixed_fast<W,I,Q,O,N>& );
#define DECL_ASN_OP_T( op , tp ) \
    sc_fixed_fast& operator op ( tp );
#define DECL_ASN_OP_OTHER( op ) \
    DECL_ASN_OP_T( op , int64 ) \
    DECL_ASN_OP_T( op , uint64 ) \
    DECL_ASN_OP_T( op , const sc_int_base& ) \
    DECL_ASN_OP_T( op , const sc_uint_base& ) \
    DECL_ASN_OP_T( op , const sc_signed& ) \
    DECL_ASN_OP_T( op , const sc_unsigned& )
#define DECL_ASN_OP( op ) \

```



```
DECL_ASN_OP_T( op , int ) \
DECL_ASN_OP_T( op , unsigned int ) \
DECL_ASN_OP_T( op , long ) \
DECL_ASN_OP_T( op , unsigned long ) \
DECL_ASN_OP_T( op , double ) \
DECL_ASN_OP_T( op , const char* ) \
DECL_ASN_OP_T( op , const sc_fxval& ) \
DECL_ASN_OP_T( op , const sc_fxval_fast& ) \
DECL_ASN_OP_T( op , const sc_fxnum& ) \
DECL_ASN_OP_T( op , const sc_fxnum_fast& ) \
DECL_ASN_OP_OTHER( op )
DECL_ASN_OP( = )
DECL_ASN_OP( *= )
DECL_ASN_OP( /= )
DECL_ASN_OP( += )
DECL_ASN_OP( -= )
DECL_ASN_OP_T( <<= , int )
DECL_ASN_OP_T( >>= , int )
DECL_ASN_OP_T( &= , const sc_fix& )
DECL_ASN_OP_T( &= , const sc_fix_fast& )
DECL_ASN_OP_T( |= , const sc_fix& )
DECL_ASN_OP_T( |= , const sc_fix_fast& )
DECL_ASN_OP_T( ^= , const sc_fix& )
DECL_ASN_OP_T( ^= , const sc_fix_fast& )
const sc_fxval_fast operator++ ( int );
const sc_fxval_fast operator-- ( int );
sc_fixed_fast& operator++ ();
sc_fixed_fast& operator-- ();
};

} // namespace sc_dt
```

### 7.10.20.3 Constraints on usage

The word length shall be greater than zero. The number of saturated bits, if specified, shall not be less than zero.

**sc\_fixed\_fast** shall use double-precision (floating-point) values whose mantissa is limited to 53 bits.

### 7.10.20.4 Public constructors

The initial value of an **sc\_fixed\_fast** object may be specified as a constructor argument that is a C++ or SystemC numeric object or a string literal. A fixed-point cast switch may also be passed as a constructor argument to set the fixed-point casting, as described in 7.10.7.

### 7.10.20.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ numeric representation to **sc\_fixed\_fast**, using truncation or sign-extension, as described in 7.10.4.

## 7.10.21 **sc\_ufixed\_fast**

### 7.10.21.1 Description

Class template **sc\_ufixed\_fast** shall represent an unsigned limited-precision fixed-point type. The fixed-point type parameters **wl**, **iwl**, **q\_mode**, **o\_mode**, and **n\_bits** shall be specified by the template arguments.

Any public member functions of the base class **sc\_ufix\_fast** that are overridden in class **sc\_ufixed\_fast** shall have the same behavior in the two classes. Any public member functions of the base class not overridden in this way shall be publicly inherited by class **sc\_ufixed\_fast**.

### 7.10.21.2 Class definition

```
namespace sc_dt {

template <int W, int I,
         sc_q_mode Q = SC_DEFAULT_Q_MODE_,
         sc_o_mode O = SC_DEFAULT_O_MODE_, int N = SC_DEFAULT_N_BITS_>
class sc_ufixed_fast
: public sc_ufix_fast
{
public:
    // Constructors
    explicit sc_ufixed_fast();
    explicit sc_ufixed_fast( const sc_fxcast_switch& );

#define DECL_CTORS_T_A( tp ) \
    sc_ufixed_fast( tp ); \
    sc_ufixed_fast( tp , const sc_fxcast_switch& );
#define DECL_CTORS_T_B( tp ) \
    explicit sc_ufixed_fast ( tp ); \
    sc_ufixed_fast( tp , const sc_fxcast_switch& );

    DECL_CTORS_T_A( int )
    DECL_CTORS_T_A( unsigned int )
    DECL_CTORS_T_A( long )
    DECL_CTORS_T_A( unsigned long )
    DECL_CTORS_T_A( double )
    DECL_CTORS_T_A( const char* )
    DECL_CTORS_T_A( const sc_fxval& )
    DECL_CTORS_T_A( const sc_fxval_fast& )
    DECL_CTORS_T_A( const sc_fxnum& )
    DECL_CTORS_T_A( const sc_fxnum_fast& )
    DECL_CTORS_T_B( int64 )
    DECL_CTORS_T_B( uint64 )
    DECL_CTORS_T_B( const sc_int_base& )
    DECL_CTORS_T_B( const sc_uint_base& )
    DECL_CTORS_T_B( const sc_signed& )
    DECL_CTORS_T_B( const sc_unsigned& )

#undef DECL_CTORS_T_A
#undef DECL_CTORS_T_B

```

```

// Copy constructor
sc_ufixed_fast( const sc_ufixed_fast<W,I,Q,O,N>& );

// Assignment operators
sc_ufixed_fast& operator= ( const sc_ufixed_fast<W,I,Q,O,N>& );

#define DECL_ASN_OP_T( op , tp ) \
    sc_ufixed_fast& operator op ( tp );
#define DECL_ASN_OP_OTHER( op ) \
    DECL_ASN_OP_T( op , int64 ) \
    DECL_ASN_OP_T( op , uint64 ) \
    DECL_ASN_OP_T( op , const sc_int_base& ) \
    DECL_ASN_OP_T( op , const sc_uint_base& ) \
    DECL_ASN_OP_T( op , const sc_signed& ) \
    DECL_ASN_OP_T( op , const sc_unsigned& )

#define DECL_ASN_OP( op ) \
    DECL_ASN_OP_T( op , int ) \
    DECL_ASN_OP_T( op , unsigned int ) \
    DECL_ASN_OP_T( op , long ) \
    DECL_ASN_OP_T( op , unsigned long ) \
    DECL_ASN_OP_T( op , double ) \
    DECL_ASN_OP_T( op , const char* ) \
    DECL_ASN_OP_T( op , const sc_fxval& ) \
    DECL_ASN_OP_T( op , const sc_fxval_fast& ) \
    DECL_ASN_OP_T( op , const sc_fxnum& ) \
    DECL_ASN_OP_T( op , const sc_fxnum_fast& \
    DECL_ASN_OP_OTHER( op )

DECL_ASN_OP( = )
DECL_ASN_OP( *= )
DECL_ASN_OP( /= )
DECL_ASN_OP( += )
DECL_ASN_OP( -= )
DECL_ASN_OP_T( <<= , int )
DECL_ASN_OP_T( >>= , int )
DECL_ASN_OP_T( &= , const sc_ufix& )
DECL_ASN_OP_T( &= , const sc_ufix_fast& )
DECL_ASN_OP_T( |= , const sc_ufix& )
DECL_ASN_OP_T( |= , const sc_ufix_fast& )
DECL_ASN_OP_T( ^= , const sc_ufix& )
DECL_ASN_OP_T( ^= , const sc_ufix_fast& )

#undef DECL_ASN_OP_T
#undef DECL_ASN_OP_OTHER
#undef DECL_ASN_OP

// Auto-increment and auto-decrement
const sc_fxval_fast operator++ ( int );
const sc_fxval_fast operator-- ( int );
sc_ufixed_fast& operator++ ();
sc_ufixed_fast& operator-- ();
};

```

```
} // namespace sc_dt
```

### 7.10.21.3 Constraints on usage

The word length shall be greater than zero. The number of saturated bits, if specified, shall not be less than zero.

**sc\_ufixed\_fast** shall use double-precision (floating-point) values whose mantissa is limited to 53 bits.

### 7.10.21.4 Public constructors

The initial value of an **sc\_fixed\_fast** object may be specified as a constructor argument, that is, a C++ or SystemC numeric object or a string literal. A fixed-point cast switch may also be passed as a constructor argument to set the fixed-point casting, as described in 7.10.7.

### 7.10.21.5 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ numeric representation to **sc\_fixed\_fast**, using truncation or sign-extension, as described in 7.10.4.

## 7.10.22 Bit-selects

### 7.10.22.1 Description

Class *sc\_fxnum\_bitref<sup>†</sup>* shall represent a bit selected from an **sc\_fxnum**.

Class *sc\_fxnum\_fast\_bitref<sup>†</sup>* shall represent a bit selected from an **sc\_fxnum\_fast**.

No distinction shall be made between a bit-select used as an lvalue or an rvalue.

### 7.10.22.2 Class definition

```
namespace sc_dt {

class sc_fxnum_bitref†
{
    friend class sc_fxnum;
    friend class sc_fxnum_fast_bitref†;

public:
    // Copy constructor
    sc_fxnum_bitref†( const sc_fxnum_bitref†& );

    // Assignment operators
    #define DECL_ASN_OP_T( op , tp ) \
        sc_fxnum_bitref†& operator op ( tp );
    #define DECL_ASN_OP( op ) \
        DECL_ASN_OP_T( op , const sc_fxnum_bitref†& ) \
        DECL_ASN_OP_T( op , const sc_fxnum_fast_bitref†& ) \
        DECL_ASN_OP_T( op , bool )

    DECL_ASN_OP( = )
    DECL_ASN_OP( &= )
    DECL_ASN_OP( |= )
```

```
DECL_ASN_OP( ^= )

#undef DECL_ASN_OP_T
#undef DECL_ASN_OP

// Implicit conversion
operator bool() const;

// Print or dump content
void print( std::ostream& = std::cout ) const;
void scan( std::istream& = std::cin );
void dump( std::ostream& = std::cout ) const;

private:
    // Disabled
    // Constructors
    sc_fxnum_bitref†( sc_fxnum& , int );
    sc_fxnum_bitref†();
};

// -----

class sc_fxnum_fast_bitref†
{
    friend class sc_fxnum_fast;
    friend class sc_fxnum_bitref†;

public:
    // Copy constructor
    sc_fxnum_fast_bitref†( const sc_fxnum_fast_bitref†& );

    // Assignment operators
    #define DECL_ASN_OP_T( op , tp ) \
        sc_fxnum_fast_bitref†& operator op ( tp );
    #define DECL_ASN_OP( op ) \
        DECL_ASN_OP_T( op , const sc_fxnum_bitref†& ) \
        DECL_ASN_OP_T( op , const sc_fxnum_fast_bitref†& ) \
        DECL_ASN_OP_T( op , bool )

    DECL_ASN_OP( = )
    DECL_ASN_OP( &= )
    DECL_ASN_OP( |= )
    DECL_ASN_OP( ^= )

    #undef DECL_ASN_OP_T
    #undef DECL_ASN_OP

    // Implicit conversion
    operator bool() const;

    // Print or dump content
    void print( std::ostream& = std::cout ) const;
    void scan( std::istream& = std::cin );
    void dump( std::ostream& = std::cout ) const;
```

```

private:
    // Disabled
    // Constructor
    sc_fxnum_fast_bitref†( sc_fxnum_fast& , int );
    sc_fxnum_fast_bitref†();
};

} // namespace sc_dt

```

### 7.10.22.3 Constraints on usage

Bit-select objects shall only be created using the bit-select operators of an instance of a class derived from **sc\_fxnum** or **sc\_fxnum\_fast**.

An application shall not explicitly create an instance of any bit-select class.

An application should not declare a reference or pointer to any bit-select object.

### 7.10.22.4 Assignment operators

Overloaded assignment operators shall provide conversion from **bool** values.

### 7.10.22.5 Implicit type conversion

**operator bool()** const;

Operator **bool** can be used for implicit type conversion from a bit-select to the native C++ **bool** representation.

## 7.10.23 Part-selects

### 7.10.23.1 Description

Class *sc\_fxnum\_subref*<sup>†</sup> shall represent a part-select from an **sc\_fx\_num**.

Class *sc\_fxnum\_fast\_subref*<sup>†</sup> shall represent a part-select from an **sc\_fxnum\_fast**.

No distinction shall be made between a part-select used as an lvalue or an rvalue.

### 7.10.23.2 Class definition

```
namespace sc_dt {

class sc_fxnum_subref†
{
    friend class sc_fxnum;
    friend class sc_fxnum_fast_subref†;

public:
    // Copy constructor
    sc_fxnum_subref†( const sc_fxnum_subref†& );

    // Destructor
    ~sc_fxnum_subref†();

    // Assignment operators
    #define DECL_ASN_OP_T( tp ) \
        sc_fxnum_subref†& operator= ( tp );

    DECL_ASN_OP_T( const sc_fxnum_subref†& )
    DECL_ASN_OP_T( const sc_fxnum_fast_subref†& )
    DECL_ASN_OP_T( const sc_bv_base& )
    DECL_ASN_OP_T( const sc_lv_base& )
    DECL_ASN_OP_T( const char* )
    DECL_ASN_OP_T( const bool* )
    DECL_ASN_OP_T( const sc_signed& )
    DECL_ASN_OP_T( const sc_unsigned& )
    DECL_ASN_OP_T( const sc_int_base& )
    DECL_ASN_OP_T( const sc_uint_base& )
    DECL_ASN_OP_T( int64 )
    DECL_ASN_OP_T( uint64 )
    DECL_ASN_OP_T( int )
    DECL_ASN_OP_T( unsigned int )
    DECL_ASN_OP_T( long )
    DECL_ASN_OP_T( unsigned long )
    DECL_ASN_OP_T( char )

    #undef DECL_ASN_OP_T

    #define DECL_ASN_OP_T_A( op , tp ) \
        sc_fxnum_subref†& operator op ## = ( tp );

    #define DECL_ASN_OP_A( op ) \
        DECL_ASN_OP_T_A( op , const sc_fxnum_subref†& ) \
        DECL_ASN_OP_T_A( op , const sc_fxnum_fast_subref†& ) \
        DECL_ASN_OP_T_A( op , const sc_bv_base& ) \
        DECL_ASN_OP_T_A( op , const sc_lv_base& )

    DECL_ASN_OP_A( & )
    DECL_ASN_OP_A( | )
    DECL_ASN_OP_A( ^ )
}
```

```

#undef DECL_ASN_OP_T_A
#undef DECL_ASN_OP_A

// Relational operators
#define DECL_REL_OP_T( op , tp ) \
    friend bool operator op ( const sc_fxnum_subref†& , tp ); \
    friend bool operator op ( tp , const sc_fxnum_subref†& );
#define DECL_REL_OP( op ) \
    friend bool operator op ( const sc_fxnum_subref†& , const sc_fxnum_subref†& ); \
    friend bool operator op ( const sc_fxnum_subref†& , const sc_fxnum_fast_subref†& ); \
    DECL_REL_OP_T( op , const sc_bv_base& ) \
    DECL_REL_OP_T( op , const sc_lv_base& ) \
    DECL_REL_OP_T( op , const char* ) \
    DECL_REL_OP_T( op , const bool* ) \
    DECL_REL_OP_T( op , const sc_signed& ) \
    DECL_REL_OP_T( op , const sc_unsigned& ) \
    DECL_REL_OP_T( op , int ) \
    DECL_REL_OP_T( op , unsigned int ) \
    DECL_REL_OP_T( op , long ) \
    DECL_REL_OP_T( op , unsigned long )

DECL_REL_OP( == )
DECL_REL_OP( != )

#undef DECL_REL_OP_T
#undef DECL_REL_OP

// Reduce functions
bool and_reduce() const;
bool nand_reduce() const;
bool or_reduce() const;
bool nor_reduce() const;
bool xor_reduce() const;
bool xnor_reduce() const;

// Query parameter
int length() const;

// Explicit conversions
int to_int() const;
unsigned int to_uint() const;
long to_long() const;
unsigned long to_ulong() const;
int64 to_int64() const;
uint64 to_uint64() const;

const std::string to_string() const;
const std::string to_string( sc_numrep ) const;
const std::string to_string( sc_numrep , bool ) const;

// Implicit conversion
operator sc_bv_base() const;

```



```
// Print or dump content
void print( std::ostream& = std::cout ) const;
void scan( std::istream& = std::cin );
void dump( std::ostream& = std::cout ) const;

private:
    // Disabled
    // Constructor
    sc_fxnum_subref†( sc_fxnum& , int , int );
    sc_fxnum_subref†();
};

// -----

class sc_fxnum_fast_subref†
{
    friend class sc_fxnum_fast;
    friend class sc_fxnum_subref†;

public:
    // Copy constructor
    sc_fxnum_fast_subref†( const sc_fxnum_fast_subref†& );

    // Destructor
    ~sc_fxnum_fast_subref†();

    // Assignment operators
    #define DECL_ASN_OP_T( tp ) \
        sc_fxnum_fast_subref†& operator= ( tp );

    DECL_ASN_OP_T( const sc_fxnum_subref†& )
    DECL_ASN_OP_T( const sc_fxnum_fast_subref†& )
    DECL_ASN_OP_T( const sc_bv_base& )
    DECL_ASN_OP_T( const sc_lv_base& )
    DECL_ASN_OP_T( const char* )
    DECL_ASN_OP_T( const bool* )
    DECL_ASN_OP_T( const sc_signed& )
    DECL_ASN_OP_T( ( const sc_unsigned& ) )
    DECL_ASN_OP_T( const sc_int_base& )
    DECL_ASN_OP_T( const sc_uint_base& )
    DECL_ASN_OP_T( int64 )
    DECL_ASN_OP_T( uint64 )
    DECL_ASN_OP_T( int )
    DECL_ASN_OP_T( unsigned int )
    DECL_ASN_OP_T( long )
    DECL_ASN_OP_T( unsigned long )
    DECL_ASN_OP_T( char )

    #undef DECL_ASN_OP_T

    #define DECL_ASN_OP_T_A( op , tp ) \
        sc_fxnum_fast_subref†& operator op ## = ( tp );
```

```
#define DECL_ASN_OP_A( op ) \
    DECL_ASN_OP_T_A( op , const sc_fxnum_subref†& ) \
    DECL_ASN_OP_T_A( op , const sc_fxnum_fast_subref†& ) \
    DECL_ASN_OP_T_A( op , const sc_bv_base& ) \
    DECL_ASN_OP_T_A( op , const sc_lv_base& )

    DECL_ASN_OP_A( & )
    DECL_ASN_OP_A( | )
    DECL_ASN_OP_A( ^ )

#undef DECL_ASN_OP_T_A
#undef DECL_ASN_OP_A

// Relational operators
#define DECL_REL_OP_T( op , tp ) \
    friend bool operator op ( const sc_fxnum_fast_subref†& , tp ); \
    friend bool operator op ( tp , const sc_fxnum_fast_subref†& );
#define DECL_REL_OP( op ) \
    friend bool operator op ( const sc_fxnum_fast_subref†& , const sc_fxnum_fast_subref†& ); \
    friend bool operator op ( const sc_fxnum_fast_subref†& , const sc_fxnum_subref†& ); \
    DECL_REL_OP_T( op , const sc_bv_base& ) \
    DECL_REL_OP_T( op , const sc_lv_base& ) \
    DECL_REL_OP_T( op , const char* ) \
    DECL_REL_OP_T( op , const bool* ) \
    DECL_REL_OP_T( op , const sc_signed& ) \
    DECL_REL_OP_T( op , const sc_unsigned& ) \
    DECL_REL_OP_T( op , int ) \
    DECL_REL_OP_T( op , unsigned int ) \
    DECL_REL_OP_T( op , long ) \
    DECL_REL_OP_T( op , unsigned long )

DECL_REL_OP( == )
DECL_REL_OP( != )

#undef DECL_REL_OP_T
#undef DECL_REL_OP

// Reduce functions
bool and_reduce() const;
bool nand_reduce() const;
bool or_reduce() const;
bool nor_reduce() const;
bool xor_reduce() const;
bool xnor_reduce() const;

// Query parameter
int length() const;
```

```
// Explicit conversions
int to_int() const;
unsigned int to_uint() const;
long to_long() const;
unsigned long to_ulong() const;
int64 to_int64() const;
uint64 to_uint64() const;

const std::string to_string() const;
const std::string to_string( sc_numrep ) const;
const std::string to_string( sc_numrep , bool ) const;

// Implicit conversion
operator sc_bv_base() const;

// Print or dump content
void print( std::ostream& = std::cout ) const;
void scan( std::istream& = std::cin );
void dump( std::ostream& = std::cout ) const;

private:
    // Disabled
    // Constructor
    sc_fxnum_fast_subref*( sc_fxnum_fast& , int , int );
    sc_fxnum_fast_subref*();
};

} // namespace sc_dt
```

### 7.10.23.3 Constraints on usage

Fixed-point part-select objects shall only be created using the part-select operators of an instance of a class derived from **sc\_fxnum** or **sc\_fxnum\_fast**.

An application shall not explicitly create an instance of any fixed-point part-select class.

An application should not declare a reference or pointer to any fixed-point part-select object.

No arithmetic operators are provided for fixed-point part-selects.

### 7.10.23.4 Assignment operators

Overloaded assignment operators shall provide conversion from SystemC data types and the native C++ integer representation to fixed-point part-selects. If the size of a data type or string literal operand differs from the fixed-point part-select word length, truncation, zero-extension, or sign-extension shall be used, as described in 7.2.1.

### 7.10.23.5 Bitwise operators

Overloaded bitwise operators shall be provided for fixed-point part-select, bit-vector, and logic-vector operands.

**7.10.23.6 Implicit type conversion**

*sc\_fxnum\_subref*<sup>\*</sup>::**operator sc\_bv\_base**() const;  
*sc\_fxnum\_fast\_subref*<sup>\*</sup>::**operator sc\_bv\_base**() const;

Operator **sc\_bv\_base** can be used for implicit type conversion from integer part-selects to the SystemC bit-vector representation.

**7.10.23.7 Explicit type conversion**

int **to\_int**() const;  
 unsigned int **to\_uint**() const;  
 long **to\_long**() const;  
 unsigned long **to\_ulong**() const;  
 int64 **to\_int64**() const;  
 uint64 **to\_uint64**() const;

These member functions shall perform the conversion to C++ integer types.

const std::string **to\_string**() const;  
 const std::string **to\_string**( sc\_numrep ) const;  
 const std::string **to\_string**( sc\_numrep , bool ) const;

Member function **to\_string** shall perform the conversion to a **string** representation, as described in 7.2.11, 7.10.8, and 7.10.8.1.

## 7.11 Contexts

This subclause describes the classes that are provided to set the contexts for the data types.

### 7.11.1 **sc\_length\_param**

#### 7.11.1.1 Description

Class **sc\_length\_param** shall represent a length parameter and shall be used to create a length context, as described in 7.2.3.

#### 7.11.1.2 Class definition

```
namespace sc_dt {

class sc_length_param
{
    public:
        sc_length_param();
        sc_length_param( int );
        sc_length_param( const sc_length_param& );

        sc_length_param& operator= ( const sc_length_param& );
        friend bool operator== ( const sc_length_param& , const sc_length_param& );
        friend bool operator!= ( const sc_length_param& , const sc_length_param& );

        int len() const;
        void len( int );
        const std::string to_string() const;
        void print( std::ostream& = std::cout ) const;
        void dump( std::ostream& = std::cout ) const;
};

} // namespace sc_dt
```

#### 7.11.1.3 Constraints on usage

The length (where specified) shall be greater than zero.

#### 7.11.1.4 Public constructors

**sc\_length\_param**();

Default constructor **sc\_length\_param** shall create an **sc\_length\_param** object with the default word length of 32.

**sc\_length\_param**( int n );

Constructor **sc\_length\_param** shall create an **sc\_length\_param** with **n** as the word length with **n** > 0.

**sc\_length\_param**( const **sc\_length\_param**& );

Constructor **sc\_length\_param** shall create a copy of the object given as its argument.

**7.11.1.5 Public methods**

int **len**() const;

Member function **len** shall return the word length stored in the **sc\_length\_param**.

void **len**( int n );

Member function **len** shall set the word length of the **sc\_length\_param** to **n**, with **n** > 0.

const std::string **to\_string**() const;

Member function **to\_string** shall convert the **sc\_length\_param** into its string representation.

void **print**( std::ostream& = std::cout ) const;

Member function **print** shall print the contents to a stream.

**7.11.1.6 Public operators**

sc\_length\_param& **operator=**( const sc\_length\_param& a );

**operator=** shall assign the word-length value of **a** to the left-hand side **sc\_length\_param** instance.

friend bool **operator==**( const sc\_length\_param& a , sc\_length\_param& b );

**operator==** shall return **true** if the stored lengths of **a** and **b** are equal.

friend bool **operator!=**( const sc\_length\_param& a , const sc\_length\_param& b );

**operator!=** shall return **true** if the stored lengths of **a** and **b** are not equal.

**7.11.2 sc\_length\_context****7.11.2.1 Description**

Class **sc\_length\_context** shall be used to create a length context for SystemC integer and vector objects.

**7.11.2.2 Class definition**

```
namespace sc_dt {
```

```
class sc_length_context
```

```
{
```

```
public:
```

```
    explicit sc_length_context( const sc_length_param& , sc_context_begin† = SC_NOW );
```

```
    ~sc_length_context();
```

```
    void begin();
```

```
    void end();
```

```
    static const sc_length_param& default_value();
```

```
    const sc_length_param& value() const;
```

```
};
```

```
} // namespace sc_dt
```

### 7.11.2.3 Public constructor

explicit **sc\_length\_context**( const sc\_length\_param& , *sc\_context\_begin*<sup>†</sup> = SC\_NOW );

Constructor **sc\_length\_context** shall create an **sc\_length\_context** object. The first argument shall be the length parameter to use. The second argument, if supplied, shall have the value SC\_NOW or SC\_LATER.

### 7.11.2.4 Public member functions

void **begin**();

Member function **begin** shall set the current length context, as described in 7.2.3.

static const sc\_length\_param& **default\_value**();

Member function **default\_value** shall return the length parameter currently in context.

void **end**();

Member function **end** shall deactivate the length context and shall remove it from the top of the length context stack, as described in 7.2.3.

const sc\_length\_param& **value**() const;

Member function **value** shall return the length parameter.

## 7.11.3 sc\_fxtype\_params

### 7.11.3.1 Description

Class **sc\_fxtype\_params** shall represent a length parameter and shall be used to create a length context for fixed-point objects, as described in 7.2.3.

### 7.11.3.2 Class definition

```
namespace sc_dt {
```

```
class sc_fxtype_params
```

```
{
```

```
    public:
```

```
        // Constructors and destructor
```

```
        sc_fxtype_params();
```

```
        sc_fxtype_params( int , int );
```

```
        sc_fxtype_params( sc_q_mode , sc_o_mode, int = 0 );
```

```
        sc_fxtype_params( int , int , sc_q_mode , sc_o_mode , int = 0 );
```

```
        sc_fxtype_params( const sc_fxtype_params& );
```

```
        sc_fxtype_params( const sc_fxtype_params& , int , int );
```

```
        sc_fxtype_params( const sc_fxtype_params& , sc_q_mode , sc_o_mode , int = 0 );
```

```
        // Operators
```

```
        sc_fxtype_params& operator=( const sc_fxtype_params& );
```

```
        friend bool operator==( const sc_fxtype_params& , const sc_fxtype_params& );
```

```
        friend bool operator!=( const sc_fxtype_params& , const sc_fxtype_params& );
```

```

// Methods
int wl() const;
void wl( int );
int iwl() const;
void iwl( int );
sc_q_mode q_mode() const;
void q_mode( sc_q_mode );
sc_o_mode o_mode() const;
void o_mode( sc_o_mode );
int n_bits() const;
void n_bits( int );
const std::string to_string() const;
void print( std::ostream& = std::cout ) const;
void dump( std::ostream& = std::cout ) const;
};

} // namespace sc_dt

```

### 7.11.3.3 Constraints on usage

The length (where specified) shall be greater than zero.

### 7.11.3.4 Public constructors

```

sc_fxtype_params ( int wl , int iwl );
sc_fxtype_params ( sc_q_mode q_mode , sc_o_mode o_mode );
sc_fxtype_params ( sc_q_mode q_mode , sc_o_mode o_mode , int n_bits );
sc_fxtype_params ( int wl , int iwl , sc_q_mode q_mode , sc_o_mode o_mode , int n_bits );
sc_fxtype_params ( int wl , int iwl , sc_q_mode q_mode , sc_o_mode o_mode );
sc_fxtype_params ();

```

Constructor **sc\_fxtype\_params** shall create an **sc\_fxtype\_params** object.

**wl** shall be the total number of bits in the fixed-point format. **wl** shall be greater than zero. The default value for **wl** shall be obtained from the fixed-point context currently in scope.

**iwl** shall be the number of integer bits in the fixed-point format. **iwl** may be positive or negative. The default value for **iwl** shall be obtained from the fixed-point context currently in scope.

**q\_mode** shall be the quantization mode to use. Valid values for **o\_mode** are given in 7.10.9.9. The default value for **q\_mode** shall be obtained from the fixed-point context currently in scope.

**o\_mode** shall be the overflow mode to use. Valid values for **o\_mode** are given in 7.10.9.1. The default value for **o\_mode** shall be obtained from the fixed-point context currently in scope.

**n\_bits** shall be the number of saturated bits parameter for the selected overflow mode. **n\_bits** shall be greater than or equal to zero. If the overflow mode is specified, the default value shall be zero. If the overflow mode is not specified, the default value shall be obtained from the fixed-point context currently in scope.

If no fixed-point context is currently in scope, the default values for **wl**, **iwl**, **q\_mode**, **o\_mode**, and **n\_bits** shall be those defined in Table 36 (see 7.10.7).

### 7.11.3.5 Public member functions

```
int iwl() const;
```

Member function **iwl** shall return the **iwl** value.



void **iwl**( int val );

Member function **iwl** shall set the **iwl** value to **val**.

int **n\_bits**() const;

Member function **n\_bits** shall return the **n\_bits** value.

void **n\_bits**( int );

Member function **n\_bits** shall set the **n\_bits** value to **val**.

sc\_o\_mode **o\_mode**() const;

Member function **o\_mode** shall return the **o\_mode**.

void **o\_mode**( sc\_o\_mode mode );

Member function **o\_mode** shall set the **o\_mode** to **mode**.

sc\_q\_mode **q\_mode**() const;

Member function **q\_mode** shall return the **q\_mode**.

void **q\_mode**( sc\_q\_mode mode );

Member function **q\_mode** shall set the **q\_mode** to **mode**.

int **wl**() const;

Member function **wl** shall return the **wl** value.

void **wl**( int val );

Member function **wl** shall set the **wl** value to **val**.

### 7.11.3.6 Operators

sc\_fxtype\_params& **operator=**( const sc\_fxtype\_params& param\_ );

**operator=** shall assign the **wl**, **iwl**, **q\_mode**, **o\_mode**, and **n\_bits** of **param\_** of the right hand side to the left-hand side.

friend bool **operator==**( const sc\_fxtype\_params& param\_a , const sc\_fxtype\_params& param\_b );

**operator==** shall return **true** if **wl**, **iwl**, **q\_mode**, **o\_mode**, and **n\_bits** of **param\_a** are equal to the corresponding values of **param\_b**; otherwise, it shall return **false**.

friend bool **operator!=**( const sc\_fxtype\_params& , const sc\_fxtype\_params& );

**operator!=** shall return **true** if **wl**, **iwl**, **q\_mode**, **o\_mode**, and **n\_bits** of **param\_a** are not equal to the corresponding values of **param\_b**; otherwise, it shall return **false**.

## 7.11.4 **sc\_fxtype\_context**

### 7.11.4.1 **Description**

Class **sc\_fxtype\_context** shall be used to create a length context for fixed-point objects.

### 7.11.4.2 **Class definition**

```

namespace sc_dt {

class sc_fxtype_context
{
    public:
        explicit sc_fxtype_context( const sc_fxtype_params& , sc_context_begin† = SC_NOW );
        ~sc_fxtype_context();

        void begin();
        void end();
        static const sc_fxtype_params& default_value();
        const sc_fxtype_params& value() const;
};

} // namespace sc_dt

```

### 7.11.4.3 **Public constructor**

explicit **sc\_fxtype\_context**( const sc\_fxtype\_params& , *sc\_context\_begin*<sup>†</sup> = SC\_NOW );

Constructor **sc\_fxtype\_context** shall create an **sc\_fxtype\_context** object. The first argument shall be the fixed-point length parameter to use. The second argument (if supplied) shall have the value SC\_NOW or SC\_LATER.

### 7.11.4.4 **Public member functions**

void **begin**();

Member function **begin** shall set the current length context, as described in 7.2.3.

static const sc\_fxtype\_params& **default\_value**();

Member function **default\_value** shall return the length parameter currently in context.

void **end**();

Member function **end** shall deactivate the length context and remove it from the top of the length context stack, as described in 7.2.3.

const sc\_fxtype\_params& **value**() const;

Member function **value** shall return the length parameter.

### 7.11.5 **sc\_fxcast\_switch**

#### 7.11.5.1 **Description**

Class **sc\_fxcast\_switch** shall be used to set the floating-point cast context, as described in 7.10.7.

#### 7.11.5.2 **Class definition**

```
namespace sc_dt {  
  
class sc_fxcast_switch  
{  
    public:  
        // Constructors  
        sc_fxcast_switch();  
        sc_fxcast_switch( sc_switch† );  
        sc_fxcast_switch( const sc_fxcast_switch& );  
  
        // Operators  
        sc_fxcast_switch& operator= ( const sc_fxcast_switch& );  
        friend bool operator== ( const sc_fxcast_switch& , const sc_fxcast_switch& );  
        friend bool operator!= ( const sc_fxcast_switch& , const sc_fxcast_switch& );  
  
        // Methods  
        const std::string to_string() const;  
        void print( std::ostream& = std::cout ) const;  
        void dump( std::ostream& = std::cout ) const;  
};  
  
} // namespace sc_dt
```

#### 7.11.5.3 **Public constructors**

```
sc_fxcast_switch ();  
sc_fxcast_switch ( sc_switch† );
```

The argument (if supplied) shall have the value SC\_OFF or SC\_ON, as described in 7.10.7. The default constructor shall use the floating-point cast context currently in scope.

#### 7.11.5.4 **Public member functions**

```
void print( std::ostream& = std::cout ) const;
```

Member function **print** shall print the **sc\_fxcast\_switch** instance value to an output stream.

#### 7.11.5.5 **Explicit conversion**

```
const std::string to_string() const;
```

Member function **to\_string** shall return the switch state as the character string “SC\_OFF” or “SC\_ON”.

### 7.11.5.6 Operators

`sc_fxcast_switch& operator= ( const sc_fxtype_params& cast_switch );`

**operator=** shall assign **cast\_switch** to the **sc\_fxcast\_switch** on its left-hand side.

`friend bool operator== (const sc_fxcast_switch& switch_a , const sc_fxcast_switch& switch_b );`

**operator==** shall return **true** if **switch\_a** is equal to **switch\_b**; otherwise, it shall return **false**.

`friend bool operator!= ( const sc_fxcast_switch& switch_a , const sc_fxcast_switch& switch_b );`

**operator!=** shall return **true** if **switch\_a** is not equal to **switch\_b**; otherwise, it shall return **false**.

`std::ostream& operator<< ( std::ostream& os , const sc_fxcast_switch& a );`

**operator<<** shall print the instance value of **a** to an output stream **os**.

### 7.11.6 sc\_fxcast\_context

#### 7.11.6.1 Description

Class **sc\_fxcast\_context** shall be used to create a floating-point cast context for fixed-point objects.

#### 7.11.6.2 Class definition

```
namespace sc_dt {

class sc_fxcast_context
{
public:
    explicit sc_fxcast_context( const sc_fxcast_switch& , sc_context_begin† = SC_NOW );
    sc_fxcast_context();

    void begin();
    void end();
    static const sc_fxcast_switch& default_value();
    const sc_fxcast_switch& value() const;
};

} // namespace sc_dt
```

#### 7.11.6.3 Public constructor

`explicit sc_fxcast_context( const sc_fxcast_switch& , sc_context_begin† = SC_NOW );`

Constructor **sc\_fxcast\_context** shall create an **sc\_fxcast\_context** object. Its first argument shall be the floating-point cast switch to use. The second argument (if supplied) shall have the value **SC\_NOW** or **SC\_LATER**.

#### 7.11.6.4 Public member functions

void **begin**();

Member function **begin** shall set the current floating-point cast context, as described in 7.10.7.

static const sc\_fxcast\_switch& **default\_value**();

Member function **default\_value** shall return the cast switch currently in context.

void **end**();

Member function **end** shall deactivate the floating-point cast context and remove it from the top of the floating-point cast context stack.

const sc\_fxcast\_switch& **value**() const;

Member function **value** shall return the cast switch.

## 7.12 Control of string representation

### 7.12.1 Description

Type **sc\_numrep** is used to control the formatting of number representations as character strings when passed as an argument to the **to\_string** member function of a data type object.

### 7.12.2 Class definition

```
namespace sc_dt {

    enum sc_numrep
    {
        SC_NOBASE = 0,
        SC_BIN = 2,
        SC_OCT = 8,
        SC_DEC = 10,
        SC_HEX = 16,
        SC_BIN_US,
        SC_BIN_SM,
        SC_OCT_US,
        SC_OCT_SM,
        SC_HEX_US,
        SC_HEX_SM,
        SC_CSD
    };

    const std::string to_string( sc_numrep );

};    // namespace sc_dt
```

### 7.12.3 Functions

```
const std::string to_string( sc_numrep );
```

Function **to\_string** shall return a string consisting of the same sequence of characters as the name of the corresponding constant value of the enumerated type **sc\_numrep**.

*Example:*

```
to_string(SC_HEX) == "SC_HEX" // is true
```

## 8. Utility class definitions

### 8.1 Trace files

A trace file records a time-ordered sequence of value changes during simulation. The VCD trace file format shall be supported.

A VCD trace file can only be created and opened by calling function **sc\_create\_vcd\_trace\_file**. A trace file may be opened during elaboration or at any time during simulation. Values can only be traced by calling function **sc\_trace**. A trace file shall be opened before values can be traced to that file, and values shall not be traced to a given trace file if one or more delta cycles have elapsed since opening the file. A VCD trace file shall be closed by calling function **sc\_close\_vcd\_trace\_file**. A trace file shall not be closed before the final delta cycle of simulation.

An implementation may support other trace file formats by providing alternatives to the functions **sc\_create\_vcd\_trace\_file** and **sc\_close\_vcd\_trace\_file**.

The lifetime of a traced object need not extend throughout the entire time the trace file is open.

NOTE—A trace file can be opened at any time, but no mechanism is available to switch off tracing before the end of simulation.

#### 8.1.1 Class definition and function declarations

```
namespace sc_core {  
  
class sc_trace_file  
{  
public:  
    virtual void set_time_unit( double , sc_time_unit ) = 0;  
    implementation-defined  
};  
  
sc_trace_file* sc_create_vcd_trace_file( const char* name );  
void sc_close_vcd_trace_file( sc_trace_file* tf );  
void sc_write_comment( sc_trace_file* tf, const std::string& comment );  
void sc_trace ...  
  
} // namespace sc_core
```

#### 8.1.2 **sc\_trace\_file**

```
class sc_trace_file  
{  
public:  
    virtual void set_time_unit( double , sc_time_unit ) = 0;  
    implementation-defined  
};
```

Class **sc\_trace\_file** is the abstract base class from which the classes that provide file handles for VCD or other implementation-defined trace file formats are derived. An application shall not construct objects of class **sc\_trace\_file** but may define pointers and references to this type.

Member function **set\_time\_unit** shall be overridden in the derived class to set the time unit for the trace file. The value of the **double** argument shall be positive and shall be a power of 10. In the absence of any call function **set\_time\_unit**, the default trace file time unit shall be 1 picosecond.

### 8.1.3 sc\_create\_vcd\_trace\_file

`sc_trace_file* sc_create_vcd_trace_file( const char* name );`

Function **sc\_create\_vcd\_trace\_file** shall create a new file handle object of class **sc\_trace\_file**, open a new VCD file associated with the file handle, and return a pointer to the file handle. The file name shall be constructed by appending the character string “.vcd” to the character string passed as an argument to the function.

### 8.1.4 sc\_close\_vcd\_trace\_file

`void sc_close_vcd_trace_file( sc_trace_file* tf );`

Function **sc\_close\_vcd\_trace\_file** shall close the VCD file and delete the file handle pointed to by the argument.

### 8.1.5 sc\_write\_comment

`void sc_write_comment( sc_trace_file* tf, const std::string& comment );`

Function **sc\_write\_comment** shall write the string given as the second argument to the trace file given by the first argument, as a comment, at the simulation time at which the function is called.

### 8.1.6 sc\_trace

`void sc_trace( sc_trace_file* , const bool& , const std::string& );`  
`void sc_trace( sc_trace_file* , const bool* , const std::string& );`  
`void sc_trace( sc_trace_file* , const float& , const std::string& );`  
`void sc_trace( sc_trace_file* , const float* , const std::string& );`  
`void sc_trace( sc_trace_file* , const double& , const std::string& );`  
`void sc_trace( sc_trace_file* , const double* , const std::string& );`  
`void sc_trace( sc_trace_file* , const sc_dt::sc_logic& , const std::string& );`  
`void sc_trace( sc_trace_file* , const sc_dt::sc_logic* , const std::string& );`  
`void sc_trace( sc_trace_file* , const sc_dt::sc_int_base& , const std::string& );`  
`void sc_trace( sc_trace_file* , const sc_dt::sc_int_base* , const std::string& );`  
`void sc_trace( sc_trace_file* , const sc_dt::sc_uint_base& , const std::string& );`  
`void sc_trace( sc_trace_file* , const sc_dt::sc_uint_base* , const std::string& );`  
`void sc_trace( sc_trace_file* , const sc_dt::sc_signed& , const std::string& );`  
`void sc_trace( sc_trace_file* , const sc_dt::sc_signed* , const std::string& );`  
`void sc_trace( sc_trace_file* , const sc_dt::sc_unsigned& , const std::string& );`  
`void sc_trace( sc_trace_file* , const sc_dt::sc_unsigned* , const std::string& );`  
`void sc_trace( sc_trace_file* , const sc_dt::sc_bv_base& , const std::string& );`  
`void sc_trace( sc_trace_file* , const sc_dt::sc_bv_base* , const std::string& );`  
`void sc_trace( sc_trace_file* , const sc_dt::sc_lv_base& , const std::string& );`  
`void sc_trace( sc_trace_file* , const sc_dt::sc_lv_base* , const std::string& );`



```

void sc_trace( sc_trace_file* , const sc_dt::sc_fxval& , const std::string& );
void sc_trace( sc_trace_file* , const sc_dt::sc_fxval* , const std::string& );
void sc_trace( sc_trace_file* , const sc_dt::sc_fxval_fast& , const std::string& );
void sc_trace( sc_trace_file* , const sc_dt::sc_fxval_fast* , const std::string& );
void sc_trace( sc_trace_file* , const sc_dt::sc_fxnum& , const std::string& );
void sc_trace( sc_trace_file* , const sc_dt::sc_fxnum* , const std::string& );
void sc_trace( sc_trace_file* , const sc_dt::sc_fxnum_fast& , const std::string& );
void sc_trace( sc_trace_file* , const sc_dt::sc_fxnum_fast* , const std::string& );

void sc_trace( sc_trace_file* , const unsigned char& , const std::string& ,
               int width = 8 * sizeof( unsigned char ) );

void sc_trace( sc_trace_file* , const unsigned char* , const std::string& ,
               int width = 8 * sizeof( unsigned char ) );

void sc_trace( sc_trace_file* , const unsigned short& , const std::string& ,
               int width = 8 * sizeof( unsigned short ) );

void sc_trace( sc_trace_file* , const unsigned short* , const std::string& ,
               int width = 8 * sizeof( unsigned short ) );

void sc_trace( sc_trace_file* , const unsigned int& , const std::string& ,
               int width = 8 * sizeof( unsigned int ) );

void sc_trace( sc_trace_file* , const unsigned int* , const std::string& ,
               int width = 8 * sizeof( unsigned int ) );

void sc_trace( sc_trace_file* , const unsigned long& , const std::string& ,
               int width = 8 * sizeof( unsigned long ) );

void sc_trace( sc_trace_file* , const unsigned long* , const std::string& ,
               int width = 8 * sizeof( unsigned long ) );

void sc_trace( sc_trace_file* , const char& , const std::string& , int width = 8 * sizeof( char ) );

void sc_trace( sc_trace_file* , const char* , const std::string& , int width = 8 * sizeof( char ) );

void sc_trace( sc_trace_file* , const short& , const std::string& , int width = 8 * sizeof( short ) );

void sc_trace( sc_trace_file* , const short* , const std::string& , int width = 8 * sizeof( short ) );

void sc_trace( sc_trace_file* , const int& , const std::string& , int width = 8 * sizeof( int ) );

void sc_trace( sc_trace_file* , const int* , const std::string& , int width = 8 * sizeof( int ) );

void sc_trace( sc_trace_file* , const long& , const std::string& , int width = 8 * sizeof( long ) );

void sc_trace( sc_trace_file* , const long* , const std::string& , int width = 8 * sizeof( long ) );

void sc_trace( sc_trace_file* , const sc_dt::int64& , const std::string& , int width = 8 * sizeof( long ) );

void sc_trace( sc_trace_file* , const sc_dt::int64* , const std::string& , int width = 8 * sizeof( long ) );

void sc_trace( sc_trace_file* , const sc_dt::uint64& , const std::string& , int width = 8 * sizeof( long ) );

```

```
void sc_trace( sc_trace_file* , const sc_dt::uint64* , const std::string& , int width = 8 * sizeof( long ) );
```

```
template <class T>
```

```
void sc_trace( sc_trace_file* , const sc_signal_in_if<T>& , const std::string& );
```

```
void sc_trace( sc_trace_file* , const sc_signal_in_if<char>& , const std::string& , int width );
```

```
void sc_trace( sc_trace_file* , const sc_signal_in_if<short>& , const std::string& , int width );
```

```
void sc_trace( sc_trace_file* , const sc_signal_in_if<int>& , const std::string& , int width );
```

```
void sc_trace( sc_trace_file* , const sc_signal_in_if<long>& , const std::string& , int width );
```

Function **sc\_trace** shall trace the value passed as the second argument to the trace file passed as the first argument, using the string passed as the third argument to identify the value in the trace file. All changes to the value of the second argument that occur between the time the function is called and the time the trace file is closed shall be recorded in the trace file.

NOTE—The function **sc\_trace** is also overloaded elsewhere in this standard to support additional data types (see 6.8.4 and 6.10.5).

## 8.2 **sc\_report**

### 8.2.1 Description

Class **sc\_report** represents an instance of a report as generated by function **sc\_report\_handler::report**. **sc\_report** objects are accessible to the application if the action SC\_CACHE\_REPORT is set for a given severity level and message type. Also, **sc\_report** objects may be caught by the application when thrown by the report handler (see 8.3).

Type **sc\_severity** represents the severity level of a report.

### 8.2.2 Class definition

```
namespace sc_core {  
  
enum sc_severity {  
    SC_INFO = 0 ,  
    SC_WARNING ,  
    SC_ERROR ,  
    SC_FATAL ,  
    SC_MAX_SEVERITY  
};  
  
class sc_report  
: public std::exception  
{  
public:  
  
    sc_report( const sc_report& );  
    sc_report& operator=( const sc_report& );  
    virtual ~sc_report();  
  
    sc_severity get_severity() const;  
    const char* get_msg_type() const;  
    const char* get_msg() const;  
    const char* get_file_name() const;  
    int get_line_number() const;  
  
    const sc_time& get_time() const;  
    const char* get_process_name() const;  
  
    virtual const char* what() const throw();  
};  
  
} // namespace sc_core
```

### 8.2.3 Constraints on usage

Objects of class **sc\_report** are generated by calling the function **sc\_report\_handler::report**. An application shall not directly create a new object of class **sc\_report** other than by calling the copy constructor. The individual attributes of an **sc\_report** object may only be set by function **sc\_report\_handler::report**.

An implementation shall throw an object of class **sc\_report** from function **default\_handler** of class **sc\_report\_handler** in response to the action SC\_THROW. An application may throw an object of class **sc\_report** from an application-specific report handler function. An application may catch an **sc\_report** in a try-block.

### 8.2.4 sc\_severity

There shall be four severity levels. SC\_MAX\_SEVERITY shall not be a severity level. It shall be an error to pass the value SC\_MAX\_SEVERITY to a function requiring an argument of type **sc\_severity**.

Table 48 describes the intended meanings of the four severity levels. The precise meanings can be overridden by the class **sc\_report\_handler**.

**Table 48—Levels for sc\_severity**

| Severity levels | Description                                                        |
|-----------------|--------------------------------------------------------------------|
| SC_INFO         | An informative message                                             |
| SC_WARNING      | A potential problem                                                |
| SC_ERROR        | An actual problem from which an application may be able to recover |
| SC_FATAL        | An actual problem from which an application cannot recover         |

### 8.2.5 Copy constructor and assignment

```
sc_report( const sc_report& );  
sc_report& operator= ( const sc_report& );
```

The copy constructor and the assignment operator shall each create a deep copy of the **sc\_report** object passed as an argument.

### 8.2.6 Member functions

Several of the member functions specified in this subclause return a pointer to a null-terminated character string. The implementation is only obliged to keep the returned string valid during the lifetime of the **sc\_report** object.

```
sc_severity get_severity() const;  
const char* get_msg_type() const;  
const char* get_msg() const;  
const char* get_file_name() const;  
int get_line_number() const;
```

Each of these five member functions shall return the corresponding property of the **sc\_report** object. The properties themselves can only be set by passing their values as arguments to the function **sc\_report\_handler::report**.

```
const sc_time& get_time() const;  
const char* get_process_name() const;
```

Each of these two member functions shall return the corresponding property of the **sc\_report** object. The properties themselves shall be set by function **sc\_report\_handler::report** according to the simulation time at which the report was generated and the process instance within which it was generated.

virtual const char\* **what**() const;

Member function **what** shall return a text string composed from the severity level, message type, message, file name, line number, process name, and time of the **sc\_report** object. An implementation may vary the content of the text string, depending upon the severity level.

*Example:*

```
try {  
    ...  
    SC_REPORT_ERROR("msg_type", "msg");  
    ...  
} catch ( sc_report e ) {  
    std::cout << "Caught " << e.what() << std::endl;  
}
```

## 8.3 sc\_report\_handler

### 8.3.1 Description

Class **sc\_report\_handler** provides features for writing out textual reports on the occurrence of exceptional circumstances and for defining application-specific behavior to be executed when those reports are generated.

Member function **report** is the central feature of the reporting mechanism, and by itself is sufficient for the generation of reports using the default actions and default handler. Other member function of class **sc\_report\_handler** provide for application-specific report handling. Member function **report** shall be called by an implementation whenever it needs to report an exceptional circumstance. Member function **report** may also be called from SystemC applications created by IP vendors, EDA tool vendors, or end users. The intention is that the behavior of reports embedded in an implementation or in precompiled SystemC code distributed as object code may be modified by end users to calling the member functions of class **sc\_report\_handler**.

In order to define application-specific actions to be taken when a report is generated, reports are categorized according to their severity level and message type. Care should be taken when choosing the message types passed to function **report** in order to give the end user adequate control over the definition of actions. It is recommended that each message type take the following general form:

"/originating\_company\_or\_institution/product\_identifier/subcategory/subcategory..."

It is the responsibility of any party who distributes precompiled SystemC code to ensure that any reports that the end user may need to distinguish for the purpose of setting actions are allocated unique message types.

### 8.3.2 Class definition

```
namespace sc_core {

typedef unsigned sc_actions;

enum {
    SC_UNSPECIFIED      = 0x0000 ,
    SC_DO_NOTHING       = 0x0001 ,
    SC_THROW            = 0x0002 ,
    SC_LOG              = 0x0004 ,
    SC_DISPLAY          = 0x0008 ,
    SC_CACHE_REPORT     = 0x0010 ,
    SC_INTERRUPT        = 0x0020 ,
    SC_STOP             = 0x0040 ,
    SC_ABORT            = 0x0080
};

#define SC_DEFAULT_INFO_ACTIONS \
    ( SC_LOG | SC_DISPLAY )

#define SC_DEFAULT_WARNING_ACTIONS \
    ( SC_LOG | SC_DISPLAY )

#define SC_DEFAULT_ERROR_ACTIONS \
    ( SC_LOG | SC_CACHE_REPORT | SC_THROW )
```

```
#define SC_DEFAULT_FATAL_ACTIONS \
    ( SC_LOG | SC_DISPLAY | SC_CACHE_REPORT | SC_ABORT )

typedef void ( *sc_report_handler_proc ) ( const sc_report& , const sc_actions& );

class sc_report_handler
{
public:
    static void report( sc_severity , const char* msg_type , const char* msg , const char* file , int line );

    static sc_actions set_actions( sc_severity , sc_actions = SC_UNSPECIFIED );
    static sc_actions set_actions( const char * msg_type , sc_actions = SC_UNSPECIFIED );
    static sc_actions set_actions( const char * msg_type , sc_severity , sc_actions = SC_UNSPECIFIED );

    static int stop_after( sc_severity , int limit = -1 );
    static int stop_after( const char* msg_type , int limit = -1 );
    static int stop_after( const char* msg_type , sc_severity , int limit = -1 );

    static int get_count( sc_severity );
    static int get_count( const char* msg_type );
    static int get_count( const char* msg_type , sc_severity );

    static sc_actions suppress( sc_actions );
    static sc_actions suppress();
    static sc_actions force( sc_actions );
    static sc_actions force();

    static void set_handler( sc_report_handler_proc );
    static void default_handler( const sc_report& , const sc_actions& );
    static sc_actions get_new_action_id();

    static sc_report* get_cached_report();
    static void clear_cached_report();

    static bool set_log_file_name( const char* );
    static const char* get_log_file_name();
};

#define SC_REPORT_INFO( msg_type , msg ) \
    sc_report_handler::report( SC_INFO , msg_type , msg , __FILE__ , __LINE__ )

#define SC_REPORT_WARNING( msg_type , msg ) \
    sc_report_handler::report( SC_WARNING , msg_type , msg , __FILE__ , __LINE__ )

#define SC_REPORT_ERROR( msg_type , msg ) \
    sc_report_handler::report( SC_ERROR , msg_type , msg , __FILE__ , __LINE__ )

#define SC_REPORT_FATAL( msg_type , msg ) \
    sc_report_handler::report( SC_FATAL , msg_type , msg , __FILE__ , __LINE__ )

#define sc_assert( expr ) \
    ( ( void ) ( ( expr ) ? 0 : ( SC_REPORT_FATAL( implementation-defined , #expr ) , 0 ) ) )
```

```
void sc_interrupt_here( const char* msg_type , sc_severity );
void sc_stop_here( const char* msg_type , sc_severity );

}    // namespace sc_core
```

### 8.3.3 Constraints on usage

The member functions of class **sc\_report\_handler** can be called at any time during elaboration or simulation. Actions can be set for a severity level or a message type both before and after the first use of that severity level or message type as an argument to member function **report**.

### 8.3.4 **sc\_actions**

The typedef **sc\_actions** represents a word where each bit in the word represents a distinct action. More than one bit may be set, in which case all of the corresponding actions shall be executed. The enumeration defines the set of actions recognized and performed by the default handler. An application-specific report handler set by calling function **set\_handler** may modify or extend this set of actions.

The value SC\_UNSPECIFIED is not an action as such but serves as the default value for a variable or argument of type **sc\_actions**, meaning that no action has been set. In contrast, the value SC\_DO\_NOTHING is a specific action and shall inhibit any actions set with a lower precedence according to the rules given in 8.3.6.

Each severity level is associated with a set of default actions chosen to be appropriate for the given name, but those defaults can be overridden by calling member function **set\_actions**. The default actions shall be defined by the macros SC\_DEFAULT\_INFO\_ACTIONS, SC\_DEFAULT\_WARNING\_ACTIONS, SC\_DEFAULT\_ERROR\_ACTIONS, and SC\_DEFAULT\_FATAL\_ACTIONS.

### 8.3.5 **report**

```
static void report( sc_severity , const char* msg_type , const char* msg , const char* file , int line );
```

Member function **report** shall generate a report and cause the appropriate actions to be taken as defined below.

Member function **report** shall use the severity passed as the first argument and the message type passed as the second argument to determine the set of actions to be executed as a result of previous calls to functions **set\_actions**, **stop\_after**, **suppress**, and **force**. Member function **report** shall create an object of class **sc\_report** initialized using all five argument values and shall pass this object to the handler set by the member function **set\_handler**. The object of class **sc\_report** shall not persist beyond the call to member function **report** unless the action SC\_CACHE\_REPORT is set, in which case the object can be retrieved by calling function **get\_cached\_reports**. An implementation shall maintain a separate cache of **sc\_report** objects for each process instance and a single global report cache for calls to function **report** from outside any process. Each such cache shall store only the most recent report.

Member function **report** shall be responsible for determining the set of actions to be executed. The handler function set by function **set\_handler** shall be responsible for executing those actions.

Member function **report** shall maintain counts of the number of reports generated as described in 8.3.7. These counts shall be incremented regardless of whether actions are executed or suppressed.

The macros SC\_REPORT\_INFO, SC\_REPORT\_WARNING, SC\_REPORT\_ERROR, SC\_REPORT\_FATAL, and **sc\_assert** are provided for convenience when calling member function **report**, but there is no obligation on an application to use these macros.

NOTE—Class **sc\_report** may provide a constructor for the exclusive use of class **sc\_report\_handler** in initializing these properties.



### 8.3.6 set\_actions

```
static sc_actions set_actions( sc_severity , sc_actions = SC_UNSPECIFIED );  
static sc_actions set_actions( const char * msg_type , sc_actions = SC_UNSPECIFIED );  
static sc_actions set_actions( const char * msg_type , sc_severity , sc_actions = SC_UNSPECIFIED );
```

Member function **set\_actions** shall set the actions to be taken by member function **report** when **report** is called with the given severity level, message type, or both. In determining which set of actions to take, the message type shall take precedence over the severity level, and the message type and severity level combined shall take precedence over the message type and severity level considered individually. In other words, the three member functions **set\_actions** just listed appear in order of increasing precedence. The actions of any lower precedence match shall be inhibited.

Each call to **set\_actions** shall replace the actions set by the previous call for the given severity, message type, or severity-message type pair. The value returned from the member function **set\_actions** shall be the actions set by the previous call to that very same overloading of the function **set\_actions** for the given severity level, message type, or severity-message type pair. The first call to function **set\_actions**( **sc\_severity** , **sc\_actions** ) shall return the default actions associated with the given severity level. The first call to one of the remaining two functions for a given message type shall return the value SC\_UNSPECIFIED. Each of the three overloaded functions operates independently in this respect. Precedence is only relevant when **report** is called.

*Example:*

```
sc_report_handler::set_actions(SC_WARNING, SC_DO_NOTHING);  
sc_report_handler::set_actions("/Acme_IP", SC_DISPLAY);  
sc_report_handler::set_actions("/Acme_IP", SC_INFO, SC_DISPLAY | SC_CACHE_REPORT);  
...  
SC_REPORT_WARNING("", "1");           // Silence  
SC_REPORT_WARNING("/Acme_IP", "2");    // Written to standard output  
SC_REPORT_INFO("/Acme_IP", "3");       // Written to standard output and cached
```

### 8.3.7 stop\_after

```
static int stop_after( sc_severity , int limit = -1 );  
static int stop_after( const char* msg_type , int limit = -1 );  
static int stop_after( const char* msg_type , sc_severity , int limit = -1 );
```

Member function **report** shall maintain independent counts of the number of reports generated for each severity level, each message type, and each severity-message type pair. Member function **stop\_after** shall set a limit on the number of reports that will be generated in each case. Member function **report** shall call the function **sc\_stop** when exactly the number of reports given by argument **limit** to function **stop\_after** have been generated for the given severity level, message type, or severity-message type pair.

In determining when to call function **sc\_stop**, the message type shall take precedence over the severity level, and the message type and severity level combined shall take precedence over the message type and severity level considered individually. In other words, the three member functions **stop\_after** just listed appear in order of increasing precedence. If function **report** is called with a combination of severity level and message type that matches more than one limit set by calling **stop\_after**, only the higher precedence limit shall have any effect.

The appropriate counts shall be initialized to the value 1 the first time function **report** is called with a particular severity level, message type, or severity-message type pair and shall not be modified or reset when function **stop\_after** is called. All three counts shall be incremented for each call to function **report** whether or not any actions are executed. When a count for a particular severity-message type pair is incremented, the counts for the given severity level and the given message type

shall be incremented also. If the limit being set has already been reached or exceeded by the count at the time **stop\_after** is called, **sc\_stop** shall not be called immediately but shall be called the next time the given count is incremented.

The default limit is -1, which means that no stop limit is set. Calling function **stop\_after** with a limit of -1 for a particular severity level, message type, or severity-message type pair shall remove the stop limit for that particular case.

A limit of 0 shall mean that there is no stop limit for the given severity level, message type, or severity-message type pair, and, moreover an explicit limit of 0 shall override the behavior of any lower precedence case. However, even with an explicit limit of 0, the actions set for the given case (by calling function **sc\_action** or the default actions) may nonetheless result in function **sc\_stop** or **abort** being called or an exception thrown.

If function **report** is called with a severity level of SC\_FATAL, the default behavior in the absence of any calls to either function **set\_actions** or function **stop\_after** is to execute a set of actions, including SC\_ABORT.

The value returned from the member function **stop\_after** shall be the limit set by the previous call to that very same overloading of the function **stop\_after** for the given severity level, message type, or severity-message type pair. Otherwise, the value returned is the default limit of -1.

*Example 1:*

```
sc_report_handler::stop_after(SC_WARNING, 1);
sc_report_handler::stop_after("/Acme_IP", 2);
sc_report_handler::stop_after("/Acme_IP", SC_WARNING, 3);
...
SC_REPORT_WARNING("/Acme_IP", "Overflow");
SC_REPORT_WARNING("/Acme_IP", "Conflict");
SC_REPORT_WARNING("/Acme_IP", "Misuse");           // sc_stop() called
```

*Example 2:*

```
sc_report_handler::stop_after(SC_WARNING, 5);
sc_report_handler::stop_after("/Acme_IP", SC_WARNING, 1);
...
SC_REPORT_WARNING("/Star_IP", "Unexpected");
SC_REPORT_INFO("/Acme_IP", "Invoked");
SC_REPORT_WARNING("/Acme_IP", "Mistimed");         // sc_stop() called
```

### 8.3.8 get\_count

```
static int get_count( sc_severity );
static int get_count( const char* msg_type );
static int get_count( const char* msg_type , sc_severity );
```

Member function **get\_count** shall return the count of the number of reports generated for each severity level, each message type, and each severity-message type pair maintained by member function **report**. If member function **report** has not been called for the given severity level, message type or severity-message type pair, member function **get\_count** shall return the value zero.

### 8.3.9 suppress and force

```
static sc_actions suppress( sc_actions );  
static sc_actions suppress();
```

Member function **suppress** shall suppress the execution of a given set of actions for subsequent calls to function **report**. The actions to be suppressed are passed as an argument to function **suppress**. The return value from function **suppress** shall be the set of actions that were suppressed immediately prior to the call to function **suppress**. The actions passed as an argument shall replace entirely the previously suppressed actions, there being only a single, global set of suppressed actions. By default, there are no suppressed actions. If the argument list is empty, the set of suppressed actions shall be cleared, restoring the default behavior.

The suppression of certain actions shall not hinder the execution of any other actions that are not suppressed.

```
static sc_actions force( sc_actions );  
static sc_actions force();
```

Member function **force** shall force the execution of a given set of actions for subsequent calls to function **report**. The actions to be forced are passed as an argument to function **force**. The return value from function **force** shall be the set of actions that were forced immediately prior to the call to function **force**. The actions passed as an argument shall replace entirely the previously forced actions, there being only a single, global set of forced actions. By default, there are no forced actions. If the argument list is empty, the set of forced actions shall be cleared, restoring the default behavior.

Forced actions shall be executed in addition to the default actions for the given severity level and in addition to any actions set by calling function **set\_actions**.

If the same action is both suppressed and forced, the force shall take precedence.

### 8.3.10 set\_handler

```
typedef void ( * sc_report_handler_proc ) ( const sc_report& , const sc_actions& );  
static void set_handler( sc_report_handler_proc );
```

Member function **set\_handler** shall set the handler function to be called from function **report**. This allows an application-specific report handler to be provided.

```
static void default_handler( const sc_report& , const sc_actions& );
```

Member function **default\_handler** shall be the default handler, that is, member function **default\_handler** shall be called from function **report** in the absence of any call to function **set\_handler**. Member function **default\_handler** shall perform zero, one or more than one of the actions set out in Table 49, as determined by the value of its second argument. In this table, the *composite message* shall be a text string composed from the severity level, message type, message, file name, line number, process name, and time of the **sc\_report** object. An implementation may vary the content of the *composite message*, depending upon the severity level.

**Table 49—Actions by default\_handler**

| Severity levels | Description                                                                        |
|-----------------|------------------------------------------------------------------------------------|
| SC_UNSPECIFIED  | No action (but function <b>report</b> will execute any lower precedence actions).  |
| SC_DO_NOTHING   | No action (but causes function <b>report</b> to inhibit lower precedence actions). |
| SC_THROW        | Throw the <b>sc_report</b> object.                                                 |

**Table 49—Actions by default\_handler (continued)**

|                 |                                                                                                                                                                  |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SC_LOG          | Write the composite message to the log file as set by function <b>set_log_file_name</b> .                                                                        |
| SC_DISPLAY      | Write the composite message to standard output.                                                                                                                  |
| SC_CACHE_REPORT | No action (but causes function <b>report</b> to cache the report).                                                                                               |
| SC_INTERRUPT    | Call function <b>sc_interrupt_here</b> , passing the message type and severity level of the <b>sc_report</b> object as arguments.                                |
| SC_STOP         | Call function <b>sc_stop_here</b> , passing the message type and severity level of the <b>sc_report</b> object as arguments, then call function <b>sc_stop</b> . |
| SC_ABORT        | Call <b>abort()</b> .                                                                                                                                            |

NOTE—To restore the default handler, call `set_handler( &sc_report_handler::default_handler );`

### 8.3.11 get\_new\_action\_id

`static sc_actions get_new_action_id();`

Member function **get\_new\_action\_id** shall return a value of type **sc\_actions** that represents an unused action. The returned value shall be a word with exactly one bit set. The intention is that such a value can be used to extend the set of actions when writing an application-specific report handler. If there are no more unique values available, the function shall return the value **SC\_UNSPECIFIED**. An application shall not call function **get\_new\_action\_id** before the start of elaboration.

### 8.3.12 sc\_interrupt\_here and sc\_stop\_here

`void sc_interrupt_here( const char* msg_type , sc_severity );`

`void sc_stop_here( const char* msg_type , sc_severity );`

Functions **sc\_interrupt\_here** and **sc\_stop\_here** shall be called from member function **default\_handler** in response to action types **SC\_INTERRUPT** and **SC\_STOP**, respectively. These two functions may also be called from application-specific report handlers. The intention is that these two functions serve as a debugging aid by allowing a user to set a breakpoint on or within either function. To this end, an implementation may choose to implement each of these functions with a switch statement dependent on the severity parameter such that a user can set a breakpoint dependent upon the severity level of the report.

### 8.3.13 get\_cached\_report and clear\_cached\_report

`static sc_report* get_cached_report();`

Member function **get\_cached\_report** shall return a pointer to the most recently cached report for the current process instance if called from a process or the global cache otherwise. Previous reports shall not be accessible.

`static void clear_cached_report();`

Member function **clear\_cached\_report** shall empty the report cache for the current process instance if called from a process or the global cache otherwise. A subsequent call to **get\_cached\_report** would return a null pointer until such a time as a further report was cached in the given cache.

### 8.3.14 **set\_log\_file\_name** and **get\_log\_file\_name**

```
static bool set_log_file_name( const char* );  
static const char* get_log_file_name();
```

Member function **set\_log\_file\_name** shall set the log file name. Member function **get\_log\_file\_name** shall return the log file name as set by **set\_log\_file\_name**. The default value for the log file name is a null pointer. If function **set\_log\_file\_name** is called with a non-null pointer and there is no existing log file name, the log file name shall be set by duplicating the string passed as an argument and the function shall return **true**. If called with a non-null pointer and there is already a log file name, function **set\_log\_file\_name** shall not modify the existing name and shall return **false**. If called with a null pointer, any existing log file name shall be deleted and the function shall return **false**.

Opening, writing, and closing the log file shall be the responsibility of the report handler. Member function **default\_handler** shall call function **get\_log\_file\_name** in response to the action SC\_LOG. Function **get\_log\_file\_name** may also be called from an application-specific report handler.

*Example:*

```
sc_report_handler::set_log_file_name("foo");    // Returns true  
sc_report_handler::get_log_file_name();        // Returns "foo"  
sc_report_handler::set_log_file_name("bar");    // Returns false  
sc_report_handler::get_log_file_name();        // Returns "foo"  
sc_report_handler::set_log_file_name(0);        // Returns false  
sc_report_handler::get_log_file_name();        // Returns 0
```

## 8.4 **sc\_exception**

### 8.4.1 Description

Exceptions are represented by the class **sc\_report**. The typedef **sc\_exception** exists to provide a degree of backward compatibility with earlier versions of the SystemC class library (see 8.2).

### 8.4.2 Definition

```
namespace sc_core {  
  
    typedef std::exception sc_exception;  
  
}
```

## 8.5 Utility functions

### 8.5.1 Function declarations

```
namespace sc_dt {  
  
    template <class T>  
    const T sc_abs( const T& );  
  
    template <class T>  
    const T sc_max( const T& a , const T& b ) { return (( a >= b ) ? a : b ); }  
  
    template <class T>  
    const T sc_min( const T& a , const T& b ) { return (( a <= b ) ? a : b ); }  
}  
  
namespace sc_core {  
  
    const char* sc_copyright();  
    const char* sc_version();  
    const char* sc_release();  
}
```

#### 8.5.2 **sc\_abs**

```
template <class T>  
const T sc_abs( const T& );
```

Function **sc\_abs** shall return the absolute value of the argument. This function shall be implemented by calling the operators **bool T::operator>=( const T& )** and **T T::operator-()**, and hence the template argument can be any SystemC numeric type or any fundamental C++ type.

#### 8.5.3 **sc\_max**

```
template <class T>  
const T sc_max( const T& a , const T& b ) { return (( a >= b ) ? a : b ); }
```

Function **sc\_max** shall return the greater of the two values passed as arguments as defined above.

NOTE—The template argument shall be a type for which **operator>=** is defined or for which a user-defined conversion to such a type is defined, such as any SystemC numeric type or any fundamental C++ type.

#### 8.5.4 **sc\_min**

```
template <class T>  
const T sc_min( const T& a , const T& b ) { return (( a <= b ) ? a : b ); }
```

Function **sc\_min** shall return the lesser of the two values passed as arguments as defined above.

NOTE—The template argument shall be a type for which **operator<=** is defined or for which a user-defined conversion to such a type is defined, such as any SystemC numeric type or any fundamental C++ type.

#### 8.5.5 **sc\_copyright**

```
const char* sc_copyright();
```

Function **sc\_copyright** shall return an implementation-defined string. The intent is that this string should contain a legal copyright notice, which an application may print to the console window or to a log file.

### 8.5.6 **sc\_version**

const char\* **sc\_version**();

Function **sc\_version** shall return an implementation-defined string. The intent is that this string should contain information concerning the version of the SystemC class library implementation, which an application may print to the console window or to a log file.

### 8.5.7 **sc\_release**

const char\* **sc\_release**();

Function **sc\_release** shall return an implementation-defined string of the following form:

*<major#>.<minor#>.<patch>-<originator>*

where *<major#>* represents the major release number, *<minor#>* represents the minor release number, *<patch>* represents the patch level, and *<originator>* represents the originator of the SystemC implementation. The intent is that this string should be machine-readable by any SystemC application that has an interest in checking the version or release number of the SystemC implementation.

The character set for each of these four fields shall be as follows:

- a) The lower-case letters a-z
- b) The upper-case letters A-Z
- c) The decimal digits 0-9
- d) The underscore character \_

*Example:*

```
char* release = sc_release();           // release is initialized with the string "2.1.31jul05_beta-OSCI"
```



## Annex A

(informative)

### Introduction to SystemC

This clause is informative and is intended to aid the reader in the understanding of the structure and intent of the SystemC class library.

The SystemC class library supports the functional modeling of systems by providing classes to represent the following:

- The hierarchical decomposition of a system into modules
- The structural connectivity between those modules using ports and exports
- The scheduling and synchronization of concurrent processes using events and sensitivity
- The passing of simulated time
- The separation of computation (processes) from communication (channels)
- The independent refinement of computation and communication using interfaces
- Hardware-oriented data types for modeling digital logic and fixed-point arithmetic

Loosely speaking, SystemC allows a user to write a set of C++ functions (processes) that are executed under control of a scheduler in an order that mimics the passage of simulated time and that are synchronized and communicate in a way that is useful for modeling electronic systems containing hardware and embedded software. The processes are encapsulated in a module hierarchy that captures the structural relationships and connectivity of the system. Inter-process communication uses a mechanism, the interface method call, that facilitates the abstraction and independent refinement of system-level interfaces.

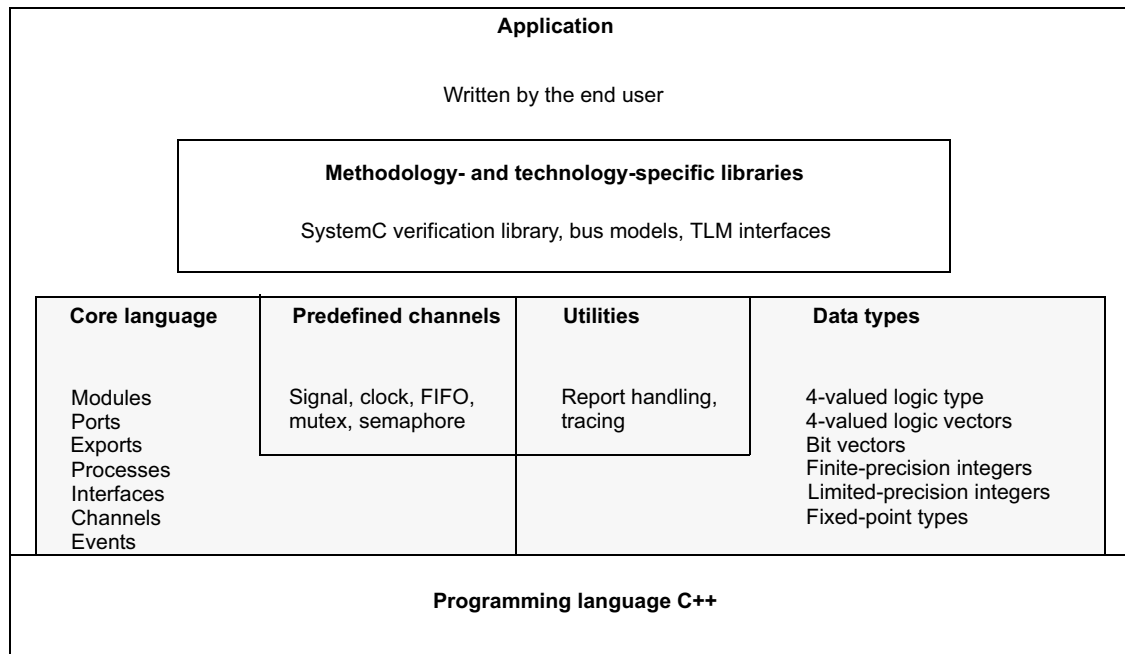


Figure A-1—SystemC language architecture

The architecture of a SystemC application is shown in Figure A-1. The shaded blocks represent the SystemC class library itself. The layer shown immediately above the SystemC class library represents standard or proprietary C++ libraries associated with specific design or verification methodologies or specific communication channels and is outside the scope of this standard.

The classes of the SystemC library fall into four categories: the core language, the SystemC data types, the predefined channels, and the utilities. The core language and the data types may be used independently of one another, although they are more typically used together.

At the core of SystemC is a simulation engine containing a process scheduler. Processes are executed in response to the notification of events. Events are notified at specific points in simulated time. In the case of time-ordered events, the scheduler is deterministic. In the case of events occurring at the same point in simulation time, the scheduler is non-deterministic. The scheduler is non-preemptive (see 4.2.1).

The *module* is the basic structural building block. Systems are represented by a module hierarchy consisting of a set of modules related by instantiation. A module can contain the following:

- Ports (see 5.11)
- Exports (see 5.12)
- Channels (see 5.2 and 5.14)
- Processes (see 5.2.10 and 5.2.11)
- Events (see 5.9)
- Instances of other modules (see 4.1.1)
- Other data members
- Other member functions

Modules, ports, exports, channels, interfaces, events, and times are implemented as C++ classes.

The execution of a SystemC application consists of *elaboration*, during which the module hierarchy is created, followed by *simulation*, during which the scheduler runs. Both elaboration and simulation involve the execution of code both from the application and from the *kernel*. The kernel is the part of a SystemC class library implementation that provides the core functionality for elaboration and the scheduler.

Instances of ports, exports, channels, and modules can only be created during elaboration. Once created during elaboration, this hierarchical structure remains fixed for the remainder of elaboration and simulation (see Clause 4). Process instances can be created statically during elaboration (see 5.2.9) or dynamically during simulation (see 5.5). Modules, channels, ports, exports, and processes are derived from a common base class **sc\_object**, which provides methods for traversing the module hierarchy. Arbitrary attributes (name-value pairs) can be attached to instances of **sc\_object** (see 5.15).

Instances of ports, exports, channels, and modules can only be created within modules. The only exception to this rule is top-level modules.

Processes are used to perform computations and hence to model the functionality of a system. Although notionally concurrent, processes are actually scheduled to execute in sequence. Processes are C++ functions registered with the kernel during elaboration (static processes) or during simulation (dynamic processes), and called from the kernel during simulation.

The *sensitivity* of a process identifies the set of events that would cause the scheduler to execute that process should those events be notified. Both *static* and *dynamic* sensitivity are provided. Static sensitivity is created at the time the process instance is created, whereas dynamic sensitivity is created during the execution of the function associated with the process during simulation. A process may be sensitive to named events or to events buried within channels or behind ports and located using an *event finder*. Furthermore, dynamic

sensitivity may be created with a *time-out*, meaning that the scheduler executes the process after a given time interval has elapsed (see 4.2.1 and 5.2.13 through 5.2.17).

Channels serve to encapsulate the mechanisms through which processes communicate and hence to model the communication aspects or protocols of a system. Channels can be used for inter-module communication or for inter-process communication within a module.

Interfaces provide a means of accessing channels. An interface proper is an abstract class that declares a set of pure virtual functions (interface methods). A channel is said to *implement* an interface if it defines all of the methods (that is, member functions) declared in that interface. The purpose of interfaces is to exploit the object-oriented type system of C++ in order that channels can be refined independently from the modules that use them. Specifically, any channel that implements a particular interface can be interchanged with any other such channel in a context that names that interface type.

The methods defined within a channel are typically called through an interface. A channel may implement more than one interface, and a single interface may be implemented by more than one channel.

Interface methods implemented in channels may create dynamic sensitivity to events contained within those same channels. This is a typical coding idiom and results in a so-called blocking method in which the process calling the method is suspended until the given event occurs. Such methods can only be called from certain kinds of processes known as thread processes (see 5.2.10 and 5.2.11).

Because processes and channels may be encapsulated within modules, communication between processes (through channels) may cross boundaries within the module hierarchy. Such boundary crossing is mediated by ports and exports, which serve to forward method calls from the processes within a module to channels to which those ports or exports are bound. A port specifies that a particular interface is required by a module, whereas an export specifies that a particular interface is provided by a module. Ports allow interface method calls within a module to be independent of the context in which the module is instantiated in the sense that the module need have no explicit knowledge of the identity of the channels to which its ports are bound. Exports allow a single module to provide multiple instances of the same interface.

Ports belonging to specific module instances are bound to channel instances during elaboration. The port binding policy can be set to control whether a port need be bound but the binding cannot be changed subsequently. Exports are bound to channel instances that lie within or below the module containing the export. Hence, each interface method call made through a port or export is directed to a specific channel instance in the elaborated module hierarchy - the channel instance to which that port is bound.

Ports can only forward method calls *up* or *out* of a module, whereas exports can only forward method calls *down* or *into* a module. Such method calls always originate from processes within a module and are directed to channels instantiated elsewhere in the module hierarchy.

Ports and exports are instances of a templated class that is parameterized with an interface type. The port or export can only be bound to a channel that implements that particular interface or one derived from it (see 5.11 through 5.13).

There are two categories of channel: hierarchical channels and primitive channels. A hierarchical channel is a module. A primitive channel is derived from a specific base class (**sc\_prim\_channel**) and is not a module. Hence, a hierarchical channel can contain processes and instances of modules, ports, and other channels, whereas a primitive channel can contain none of these. It is also possible to define channels derived from neither of these base classes, but every channel implements one or more interfaces.

A primitive channel provides unique access to the update phase of the scheduler, enabling the very efficient implementation of certain communication schemes. This standard includes a set of predefined channels, together with associated interfaces and ports, as follows:

- `sc_signal` (see 6.4)
- `sc_buffer` (see 6.6)
- `sc_clock` (see 6.7)
- `sc_signal_resolved` (see 6.13)
- `sc_signal_rv` (see 6.17)
- `sc_fifo` (see 6.23)
- `sc_mutex` (see 6.27)
- `sc_semaphore` (see 6.29)
- `sc_event_queue` (see 6.29)

Class **sc\_signal** provides the semantics for creating register transfer level or pin-accurate models of digital hardware. Class **sc\_fifo** provides the semantics for point-to-point FIFO-based communication appropriate for models based on networks of communicating processes. Classes **sc\_mutex** and **sc\_semaphore** provide communication primitives appropriate for software modeling.

This standard includes a set of data types for modeling digital logic and fixed-point arithmetic, as follows:

- `sc_int<>` (see 7.5.4)
- `sc_uint<>` (see 7.5.5)
- `sc_bigint<>` (see 7.6.5)
- `sc_biguint<>` (see 7.6.6)
- `sc_logic` (see 7.9.2)
- `sc_lv<>` (see 7.9.6)
- `sc_bv<>` (see 7.9.6)
- `sc_fixed<>` (see 7.10.18)
- `sc_ufixed<>` (see 7.10.19)

Classes **sc\_int** and **sc\_uint** provide signed and unsigned limited-precision integers with a word length limited by the C++ implementation. Classes **sc\_bigint** and **sc\_biguint** provide finite-precision integers. Class **sc\_logic** provides four-valued logic. Classes **sc\_bv** and **sc\_lv** provide two- and four-valued logic vectors. Classes **sc\_fixed** and **sc\_ufixed** provide signed and unsigned fixed-point arithmetic.

The classes **sc\_report** and **sc\_report\_handler** provide a general mechanism for error handling that is used by the SystemC class library itself and is also available to the user. Reports can be categorized by severity and by message type, and customized actions can be set for each category of report, such as writing a message, throwing an exception, or aborting the program (see 8.2 and 8.3).

## Annex B

(informative)

## Glossary

This glossary contains brief, informal descriptions for a number of terms and phrases used in this standard. Where appropriate, the complete, formal definition of each term or phrase is given in the main body of the standard. Each glossary entry contains either the clause number of the definition in the main body of the standard or an indication that the term is defined in ISO/IEC 14882:2003.

**B.1 abstract class:** A class that has or inherits at least one pure virtual function that is not overridden by a non-pure virtual function. (C++ term)

**B.2 application:** A C++ program, written by an end user, that uses the SystemC class library, that is, uses classes, calls functions, uses macros, and so forth. An application may use as few or as many features of C++ as is seen fit and as few or as many features of SystemC as is seen fit. (See 3.1.2.)

**B.3 argument:** An expression in the comma-separated list bounded by the parentheses in a function call (or macro or template instantiation), also known as an actual argument. (See **parameter**.) (C++ term)

**B.4 attach:** To associate an attribute with an object by calling member function **add\_attribute** of class **sc\_object**. (See 5.15.8.)

**B.5 base class sub-object:** A sub-object whose type is the base class type of a given object. (See **sub-object**.) (C++ term)

**B.6 binding, bound:** An asymmetrical association created during elaboration between a port or export on the one hand and a channel (or another port or export) on the other. If a port (or export) is bound to a channel, a process can make an interface method call through the port to a method defined in the channel. Ports can be bound by name or by position. Exports can only be bound by name. (See **Interface Method Call** and 4.1.3.)

**B.7 bit-select:** A class that references a single bit within a multiple-bit data type or an instance of such a class. Bit-selects are defined for each SystemC numeric type and vector class. Bit-selects corresponding to lvalues and rvalues of a particular type are distinct classes. (See 7.2.5.)

**B.8 bit vector:** A class that is derived from class **sc\_bv\_base**, or an instance of such a class. A bit vector implements a multiple-bit data type, where each bit is represented by the symbol ‘0’ or ‘1’. (See 7.1.)

**B.9 body:** A compound statement immediately following the parameter declarations and constructor initializer (if any) of a function or constructor, and containing the statements to be executed by the function. (C++ term)

**B.10 buffer:** An instance of class **sc\_buffer**, which is a primitive channel derived from class **sc\_signal**. A buffer differs from a signal in that an event occurs on a buffer whenever a value is written to the buffer, regardless of whether the write causes a value change. An event only occurs on a signal when the value of the signal changes. (See 6.6.1.)

**B.11 call:** The term *call* is taken to mean that a function is called either directly or indirectly by calling an intermediate function that calls the function in question. (See 3.1.3.)

**B.12 callback:** A member function overridden within a class in the module hierarchy that is called back by the kernel at certain fixed points during elaboration and simulation. The callback functions are `before_end_of_elaboration`, `end_of_elaboration`, `start_of_simulation`, and `end_of_simulation`. (See 4.4.)

**B.13 channel:** A class that implements one or more interfaces or an instance of such a class. A channel may be a hierarchical channel or a primitive channel or, if neither of these, it is strongly recommended that a channel at least be derived from class `sc_object`. Channels serve to encapsulate the definition of a communication mechanism or protocol. (See 3.1.4.)

**B.14 child:** An instance that is within a given module. Module A is a *child* of module B if module A is *within* module B. (See 3.1.4 and 5.15.1.)

**B.15 class template:** A pattern for any number of classes whose definitions depend on the template parameters. The compiler treats every member function of the class as a function template with the same parameters as the class template. A function template is itself a pattern for any number of functions whose definitions depend on the template parameters. (C++ term)

**B.16 clock:** An instance of class `sc_clock`, which is a predefined primitive channel that models the behavior of a periodic digital clock signal. Alternatively, a clock can be modeled as an instance of the class `sc_signal<bool>`. (See 6.7.1.)

**B.17 clocked thread process:** A thread process that is resumed only on the occurrence of a single explicit clock edge. A clocked thread process is created using the `SC_CTHREAD` macro. There are no dynamic clocked threads. (See 5.2.9 and 5.2.12.)

**B.18 complete object:** An object that is not a sub-object of any other object. If a complete object is of class type, it is also called a *most derived object*. (C++ term)

**B.19 concatenation:** An object that references the bits within multiple objects as if they were part of a single aggregate object. (See 7.2.7.)

**B.20 contain:** The inverse relationship to *within* between two modules. Module A *contains* module B if module B is *within* module A. (See 3.1.4.)

**B.21 conversion function:** A member function of the form `operator type_id` that specifies a conversion from the type of the class to the type `type_id`. (See **user-defined conversion**.) (C++ term)

**B.22 copy-constructible type:** A type T for which T(t) is equivalent to t and &t denotes the address of t. This includes fundamental types, as well as certain classes. (C++ term)

**B.23 data member:** An object declared within a class definition. A non-static data member is a sub-object of the class. A static data member is not a sub-object of the class but has static storage duration. Outside of a constructor or member function of the class or of any derived class, a data member can only be accessed using the dot . and arrow -> operators. (C++ term)

**B.24 declaration:** A C++ language construct that introduces a name into a C++ program and specifies how the C++ compiler is to interpret that name. Not all declarations are definitions. For example, a class declaration specifies the name of the class but not the class members, while a function declaration specifies the function parameters but not the function body. (See **definition**.) (C++ term)

**B.25 definition:** The complete specification of a variable, function, type, or template. For example, a class definition specifies the class name and the class members, and a function definition specifies the function parameters and the function body. (See **declaration**.) (C++ term)

**B.26 delta cycle:** A control loop within the scheduler that consists of one evaluation phase followed by one update phase. The delta cycle mechanism serves to ensure the deterministic simulation of concurrent processes by separating and alternating the computation (or evaluation) phase and the communication (or update) phase. (See 4.2.2.)

**B.27 delta notification:** A notification created as the result of a call to function **notify** with a zero time argument. The event is notified one delta cycle after the call to function **notify**. (See 4.2.1 and 5.9.4.)

**B.28 delta notification phase:** The control step within the scheduler during which processes are made runnable as a result of delta notifications. (See 4.2.1.4.)

**B.29 during elaboration, during simulation:** The phrases *during elaboration* and *during simulation* are used to indicate that an action may or may not happen at these times. The meaning of these phrases is closely tied to the definition of the elaboration and simulation callbacks. For example, a number of actions that are permitted *during elaboration* are explicitly forbidden from the **end\_of\_elaboration** callback. (See 3.1.4 and 4.4.)

**B.30 dynamic process:** A process created from the **end\_of\_elaboration** callback or during simulation.

**B.31 dynamic sensitivity:** The set of events or time-outs that would cause a process to be resumed or triggered, as created by the most recent call to the **wait** method (in the case of a thread process) or the **next\_trigger** method (in the case of a method process). (See **sensitivity** and 4.2.)

**B.32 elaboration:** The execution phase during which the module hierarchy is created and ports are bound. The execution of a C++ application consists of elaboration followed by simulation. (See Clause 4.)

**B.33 error:** An obligation on the implementation to generate a diagnostic message using the report-handling mechanism (function **report** of class **sc\_report\_handler**) with a severity of **SC\_ERROR**. (See 3.2.5.)

**B.34 evaluation phase:** The control step within the scheduler during which processes are executed. The evaluation phase is complete when the set of runnable processes is empty. (See **delta cycle** and 4.2.1.2.)

**B.35 event:** An object of class **sc\_event**. An event provides the mechanism for synchronization between processes. The **notify** method of class **sc\_event** causes an event to be notified at a specific point in time. (The notification of an event is distinct from an object of type **sc\_event**. The former is a dynamic occurrence at a unique point in time, the latter an object that can be notified many times during its lifetime.) (See **notification**, 3.1.4, and 5.9.)

**B.36 event finder:** A member function of a port class that returns an event within a channel instance to which the port is bound. An event finder can only be called when creating static sensitivity. (See 5.7.)

**B.37 event list:** A list of events, separated by either operator **&** or operator **|**, and passed as an argument to either the **wait** or the **next\_trigger** method. (See 5.8.)

**B.38 export:** An instance of class **sc\_export**. An export specifies an interface provided by a module. During simulation, a port forwards method calls to the channel to which the export was bound. An export forwards method calls down and into a module instance. (See 3.1.4 and 5.12.)

**B.39 fifo:** An instance of class **sc\_fifo**, which is a primitive channel that models a first-in-first-out buffer. Alternatively, a fifo can be modelled as a module. (See 6.23.)

**B.40 finite-precision fixed-point type:** A class that is derived from class **sc\_fxnum** or an instance of such a class. A finite-precision fixed-point type represents a signed or unsigned fixed-point value at a precision limited only by its specified word length, integer word length, quantization mode, and overflow mode. (See 7.1.)

**B.41 finite-precision integer:** A class that is derived from class **sc\_signed**, class **sc\_unsigned**, or an instance of such a class. A finite-precision integer represents a signed or unsigned integer value at a precision limited only by its specified word length. (See 7.1.)

**B.42 hierarchical channel:** A class that is derived from class **sc\_module** and that implements one or more interfaces; or more informally, an instance of such a class. A hierarchical channel is used when a channel requires its own ports, processes, or module instances. (See **channel**, 3.1.4, and 5.2.22.)

**B.43 hierarchical name:** The unique name of an instance within the module hierarchy. The hierarchical name is composed from the string names of the parent-child chain of module instances starting from a top-level module and terminating with the string name of the instance being named. The string names are concatenated and separated with the dot character. (See 5.3.4 and 5.15.4.)

**B.44 immediate notification:** A notification created as the result of a call to function with an empty argument list. Any process sensitive to the event becomes runnable immediately. (See 4.2.1 and 5.9.4.)

**B.45 implementation:** A specific concrete implementation of the full SystemC class library, only the public shell of which need be exposed to the application (for example, parts may be precompiled and distributed as object code by a tool vendor). (See **kernel** and 3.1.2.)

**B.46 implement:** To create a channel that provides a definition for every pure virtual function declared in the interface from which it is derived. (See 5.13.1.)

**B.47 implicit conversion:** A C++ language mechanism whereby a standard conversion or a user-defined conversion is called implicitly under certain circumstances. User-defined conversions are only applied implicitly where they are unambiguous, and at most one user-defined conversion is applied implicitly to a given value. (See **user-defined conversion**.) (C++ term)

**B.48 initialization phase:** The first phase of the scheduler, during which every process is executed once until it suspends or returns. (See 4.2.1.1.)

**B.49 initializer list:** The part of the C++ syntax for a constructor definition that is used to initialize base class sub-objects and data members. (Related to the C++ term *mem-initializer-list*)

**B.50 instance:** A particular case of a given category. For example, a module *instance* is an object of a class derived from class **sc\_module**. Within the definition of the core language, an *instance* is typically an object of a class derived from class **sc\_object** and has a unique hierarchical name. (See 3.1.4.)

**B.51 instantiation:** The creation of a new object. For example, a module instantiation creates a new object of a class derived from class **sc\_module**. (See 4.1.1.)

**B.52 integer:** A limited-precision integer or a finite-precision integer. (See 7.2.1.)

**B.53 interface:** A class derived from class **sc\_interface**. An interface proper is an interface, and in the object-oriented sense a channel is also an interface. However, a channel is not an interface proper. (See 3.1.4.)



**B.54 Interface Method Call (IMC):** A call to an interface method. An interface method is a member function declared within an interface. The IMC paradigm provides a level of indirection between a method call and the implementation of the method within a channel such that one channel can be substituted with another without affecting the caller. (See 4.1.3 and 5.11.1.)

**B.55 interface proper:** An abstract class derived from class `sc_interface` but not derived from class `sc_object`. An interface proper declares the set of methods to be implemented within a channel and to be called through a port. An interface proper contains pure virtual function declarations, but typically contains no function definitions and no data members. (See 3.1.4 and 5.13.1.)

**B.56 kernel:** The core of any SystemC implementation including the underlying elaboration and simulation engines. The kernel honors the semantics defined by this standard but may also contain implementation-specific functionality outside the scope of this standard. (See **implementation** and Clause 4.)

**B.57 lifetime:** The lifetime of an object starts when storage is allocated and the constructor call has completed, if any. The lifetime of an object ends when storage is released or immediately before the destructor is called, if any. (C++ term)

**B.58 limited-precision fixed-point type:** A class that is derived from class `sc_fxnum_fast`, or an instance of such a class. A limited-precision fixed-point type represents a signed or unsigned fixed-point value at a precision limited by its underlying native C++ floating-point representation and its specified word length, integer word length, quantization mode, and overflow mode. (See 7.1.)

**B.59 limited-precision integer:** A class that is derived from class `sc_int_base`, class `sc_uint_base`, or an instance of such a class. A limited-precision integer represents a signed or unsigned integer value at a precision limited by its underlying native C++ representation and its specified word length. (See 7.1.)

**B.60 logic vector:** A class that is derived from class `sc_lv_base` or an instance of such a class. A logic vector implements a multiple bit data type, where each bit is represented by a four-valued logic symbol '0', '1', 'X', or 'Z'. (See 7.1.)

**B.61 lvalue:** An object reference whose address can be taken. The left-hand operand of the built-in assignment operator must be a non-const lvalue. (C++ term)

**B.62 member function:** A function declared within a class definition, excluding friend functions. Outside of a constructor or member function of the class or of any derived class, a non-static member function can only be accessed using the dot `.` and arrow `->` operators. (See **method**.) (C++ term)

**B.63 method:** A function that implements the behavior of a class. This term is synonymous with the C++ term *member function*. In SystemC, the term *method* is used in the context of an *interface method call*. Throughout this standard, the term *member function* is used when defining C++ classes (for conformance to the C++ standard), and the term *method* is used in more informal contexts and when discussing interface method calls.

**B.64 method process:** A process that executes in the thread of the scheduler and is called (or triggered) by the scheduler at times determined by its sensitivity. An unspawned method process is created using the `SC_METHOD` macro, a spawned method process by calling the function `sc_spawn`. (See 5.2.9 and 5.2.10.)

**B.65 module:** A class that is derived from class `sc_module`; or more informally, an instance of such a class. A SystemC application is composed of modules, each module instance representing a hierarchical boundary. A module can contain instances of ports, processes, primitive channels, and other modules. (See 3.1.4 and 5.2.)

**B.66 module hierarchy:** The set of all instances created during elaboration and linked together using the mechanisms of module instantiation, port instantiation, primitive channel instantiation, process instantiation, and port binding. The module hierarchy is a subset of the object hierarchy. (See 3.1.4 and Clause 4.)

**B.67 multiport:** A port that may be bound to more than one channel or port instance. A multiport is used when an application wishes to bind a port to a set of addressable channels and the number of channels is not known until elaboration. (See 4.1.3 and 5.11.3.)

**B.68 mutex:** An instance of class `sc_mutex`, which is a primitive channel that models a mutual exclusion communication mechanism. (See 6.27.1.)

**B.69 non-abstract class:** A class that is not an abstract class. (C++ term)

**B.70 notification:** The act of scheduling the occurrence of an event as performed by the `notify` method of class `sc_event`. There are three kinds of notification: immediate notification, delta notification, and timed notification. (See `event`, 4.2.1, and 5.9.4.)

**B.71 notified:** An event is said to be *notified* at the control step of the scheduler in which the event is removed from the set of pending events and any processes that are currently sensitive to that event are made runnable. Informally, the event *occurs* precisely at the point when it is notified. (See 4.2.)

**B.72 numeric type:** A finite-precision integer, a limited-precision integer, a finite-precision fixed-point type, or a limited-precision fixed-point type. (See 7.1.)

**B.73 object:** A region of storage. Every object has a type and a lifetime. An object created by a definition has a name, whereas an object created by a new expression is anonymous. (C++ term)

**B.74 object hierarchy:** The set of all objects of class `sc_object`. Each object has a unique hierarchical name. Objects that do not belong to the module hierarchy may be created and destroyed dynamically during simulation. (See 3.1.4 and 5.15.1.)

**B.75 occurrence:** The notification of an event. Except in the case of immediate notification, a call to the `notify` method of class `sc_event` will cause the event to *occur* in a later delta cycle or at a later point in simulation time. Of a time-out: a time-out *occurs* when the specified time interval has elapsed. (See 5.9.1.)

**B.76 overload:** To create two or more functions with the same name declared in the same scope and that differ in the number or type of their parameters. (C++ term)

**B.77 override:** To create a member function in a derived class has the same name and parameter list as a member function in a base class. (C++ term)

**B.78 parameter:** An object declared as part of a function declaration or definition (or macro definition or template parameter), also known as a formal parameter. (See `argument`.) (C++ term)

**B.79 parent:** The inverse relationship to *child*. Module A is the *parent* of module B if module B is a *child* of module A. (See 3.1.4 and 5.15.1.)

**B.80 part-select:** A class that references a contiguous subset of bits within a multiple-bit data type or an instance of such a class. Part-selects are defined for each SystemC numeric and vector class. Part-selects corresponding to lvalues and rvalues of a particular type are distinct classes. (See 7.2.7.)

**B.81 pending:** The state of an event for which a notification has been posted; that is, the `notify` method has been called, but the event has not yet been notified.

**B.82 port:** A class that is derived from class `sc_port`; or more informally, an instance of such a class. A port is the primary mechanism for allowing communication across the boundary of a module. A port specifies an interface required by a module. During simulation, a port forwards method calls made from a process within a module to the channel to which the port was bound when the module was instantiated. A port forwards method calls up and out of a module instance. (See 3.1.4 and 5.11.)

**B.83 portless channel access:** Calling the member functions of a channel directly and not through a port or export. (See 5.11.1.)

**B.84 primitive channel:** A class that is derived from class `sc_prim_channel` and implements one or more interfaces; or more informally, an instance of such a class. A primitive channel has access to the update phase of the scheduler but cannot contain ports, processes, or module instances. (See 3.1.4 and 5.14.)

**B.85 process:** A process instance belongs to an implementation-defined class derived from class `sc_object`. Each process instance has an associated function that represents the behavior of the process. A process may be a static process, a dynamic process, a spawned process, or an unspawned process. The process is the primary means of describing a computation. (See **dynamic process**, **spawned process**, **static process**, **unspawned process**, and 3.1.4.)

**B.86 process handle:** An object of class `sc_process_handle` that provides safe access to an underlying spawned or unspawned process instance. A process handle can be valid or invalid. A process handle continues to exist in the invalid state even after the associated process instance has been destroyed. (See 3.1.4 and 5.6.)

**B.87 proxy class:** A class whose only purpose is to extend the readability of certain statements that would otherwise be restricted by the semantics of C++. An example is to allow an `sc_int` variable to be used as if it were a C++ array of `bool`. Proxy classes are only intended to be used for the temporary (unnamed) value returned by a function. A proxy class constructor shall not be called explicitly by an application to create a named object. (See 7.2.5.)

**B.88 resolved signal:** An instance of class `sc_signal_resolved` or `sc_signal_rv`, which are signal channels that may be written to by more than one process, with conflicting values being resolved within the channel. (See 6.13.1.)

**B.89 resume:** To cause a thread or clocked thread process to continue execution starting with the executable statement immediately following the **wait** method at which it was suspended, dependent upon the sensitivity of the process. (See 5.2.11.)

**B.90 rvalue:** A value that does not necessarily have any storage or address. An rvalue of fundamental type can only appear on the right-hand side of an assignment. (C++ term)

**B.91 scheduled:** The state of an event or of a process as a result of a call to the **notify**, **next\_trigger** or **wait** methods. An event can be *scheduled* to occur, or a process can be *scheduled* to be triggered or resumed, either in a later delta cycle or at a later simulation time.

**B.92 scheduler:** The part of the kernel that controls simulation and is thus concerned with advancing time, making processes runnable as events are notified, executing processes, and updating primitive channels. (See Clause 4.)

**B.93 sensitivity:** The set of events or time-outs that would cause a process to be resumed or triggered. Sensitivity make take the form of static sensitivity or dynamic sensitivity. (See 4.2.)

**B.94 signal:** An instance of class `sc_signal`, which is a primitive channel intended to model relevant aspects of the behavior of a simple wire as appropriate for digital hardware simulation. (See 3.1.4 and 6.4.)

**B.95 signature:** The information about a function relevant to overload resolution, such as the types of its parameters and any qualifiers. (C++ term)

**B.96 simulation:** The execution phase that consists of the execution of the scheduler together with the execution of user-defined processes under the control of the scheduler. The execution of a SystemC application consists of elaboration followed by simulation. (See Clause 4.)

**B.97 spawned process:** A process instance created by calling the function `sc_spawn`. (See **process**, 3.1.4, and 5.5.6.)

**B.98 specialized port:** A class derived from template class `sc_port` that passes a particular type as the first argument to template `sc_port`, and which provides convenience functions for accessing ports of that specific type. (See 6.8.)

**B.99 statement:** A specific category of C++ language construct that is executed in sequence, such as the *if statement*, *switch statement*, *for statement*, and *return statement*. A C++ expression followed by a semicolon is also a statement. (C++ term)

**B.100 static process:** A process created during the construction of the module hierarchy or from the `before_end_of_elaboration` callback.

**B.101 static sensitivity:** The set of events or time-outs that would cause a process to be resumed or triggered, as created using the data member `sensitive` of class `sc_module` (in the case of an unspawned process) or using class `sc_spawn_options` (in the case of a spawned process). (See **sensitivity**, **spawned process**, **unspawned process**, and 4.2.)

**B.102 string name:** A name passed as an argument to the constructor of an instance to provide an identity for that object within the module hierarchy. The string names of instances having a common parent module will be unique within that module and that module only. (See **hierarchical name**, 5.3, and 5.15.4.)

**B.103 sub-object:** An object contained within another object. A sub-object of a class may be a data member of that class or a base class sub-object. (C++ term)

**B.104 terminated:** The state of a thread or clocked thread process when the associated function executes to completion or executes a return statement and thus control returns to the kernel. Calling function `wait` does not terminate a thread process. A method process can never be terminated. (See **thread process**, **clocked thread process**, **method process**, 5.2.11, and 5.6.5.)

**B.105 thread process:** A process that executes in its own thread and is called once only by the scheduler during initialization. A thread process may be suspended by the execution of a `wait` method, in which case it will be resumed under the control of the scheduler. An unspawned thread process is created using the `SC_THREAD` macro, a spawned thread process by calling the function `sc_spawn`. (See **spawned process**, **unspawned process**, 5.2.9, and 5.2.11.)

**B.106 time-out:** The thing that causes a process to resume or trigger as a result of a call to the `wait` or `next_trigger` method with a time-valued argument. The process that called the method will resume or trigger after the specific time has elapsed, unless it has already resumed or triggered as a result of an event being notified. (See 4.2 and 4.2.1.)

**B.107 timed notification:** A notification created as the result of a call to function `notify` with a non-zero time argument. (See 4.2.1 and 5.9.4.)

**B.108 timed notification phase:** The control step within the scheduler during which processes are made runnable as a result of timed notifications. (See 4.2.1.5.)

**B.109 top-level module, top-level object:** A module or object that is not instantiated within any other module or process. Top-level modules are either instantiated within **sc\_main**, or in the absence of **sc\_main**, are identified using an implementation-specific mechanism. (See 3.1.4 and 5.15.1.)

**B.110 trigger:** To cause the member function associated with a method process instance to be called by the scheduler, dependent on its sensitivity. (See **method process**, **sensitivity**, and 5.2.10.)

**B.111 unspawned process:** A process created by invoking one of the three macros **SC\_METHOD**, **SC\_THREAD**, or **SC\_CTHREAD** during elaboration. (See **process**, 3.1.4, and 4.1.2.)

**B.112 update phase:** The control step within the scheduler during which the values of primitive channels are updated. The update phase consists of executing the **update** method for every primitive channel that called the **request\_update** method during the immediately preceding evaluation phase. (See 4.2.1.3.)

**B.113 undefined:** The absence of any obligations on the implementation. Where this standard states that a behavior or a result is *undefined*, the implementation may or may not generate an error or a warning. (See 3.3.5.)

**B.114 user:** The creator of an application, as distinct from an implementor, who creates an implementation. A user may be a person or an automated process such as a computer program. (See 3.1.2.)

**B.115 user-defined conversion:** Either a conversion function or a non-explicit constructor with exactly one parameter. (See **conversion function** and **implicit conversion**.) (C++ term)

**B.116 valid:** The state of a process handle, or of an object returned from a function by pointer or by reference, during any period in which the handle or object is not deleted and its value or behavior remains accessible to the application. A process handle is *valid* when it is associated with a process instance. (See 3.3.3 and 5.6.1.)

**B.117 variable-precision fixed-point type:** Classes **sc\_fxval** and **sc\_fxval\_fast** represent variable-precision fixed-point values that do not model overflow and quantization effects but are used as operand types and return types by many fixed-point operations. These types are not typically used directly by an application.

**B.118 vector:** (See **bit vector**, **logic vector**, and 7.1.)

**B.119 warning:** An obligation on the implementation to generate a diagnostic message using the report-handling mechanism (function **report** of class **sc\_report\_handler**) with a severity of **SC\_WARNING**. (See 3.3.5.)

**B.120 within:** The relationship that exists between an instance and a module if the constructor of the instance is called from the constructor of the module, and also provided that the instance is not *within* a nested module. (See 3.1.4.)



## Annex C

(informative)

### Deprecated features

This annex contains a list of deprecated features. A *deprecated* feature is a feature that was present in version 2.0.1 of the OSCI open source proof-of-concept SystemC implementation but is not part of this standard. Deprecated features may or may not remain in the OSCI implementation in the future. The user is strongly discouraged from using deprecated features, since an implementation is not obliged to support such features. An implementation may issue a warning on the first occurrence of each deprecated feature but is not obliged to do so.

- a) Functions **sc\_cycle** and **sc\_initialize** (Use **sc\_start** instead.)
- b) Class **sc\_simcontext** (Replaced by functions **sc\_delta\_count**, **sc\_is\_running**, **sc\_get\_top\_level\_objects**, **sc\_find\_object**, and member functions **get\_child\_objects** and **get\_parent\_object**)
- c) Type **sc\_process\_b** (Replaced by class **sc\_process\_handle**)
- d) Function **sc\_get\_curr\_process\_handle** (Replaced by function **sc\_get\_current\_process\_handle**)
- e) Member function **notify\_delayed** of class **sc\_event** (Use **notify(SC\_ZERO\_TIME)** instead.)
- f) Non-member function **notify** (Use member function **notify** of class **sc\_event** instead.)
- g) Member function **timed\_out** of classes **sc\_module** and **sc\_prim\_channel**
- h) **operator**, and **operator<<** of class **sc\_module** for positional port binding (Use **operator()** instead.)
- i) **operator()** of class **sc\_module** for positional port binding when called more than once per module instance (Use named port binding instead.)
- j) Constructors of class **sc\_port** that bind the port at the time of construction of the port object
- k) **operator()** of class **sc\_sensitive** (Use **operator<<** instead.)
- l) Classes **sc\_sensitive\_pos** and **sc\_sensitive\_neg** and the corresponding data members of class **sc\_module** (Use the event finders **pos** and **neg** instead.)
- m) Member function **end\_module** of class **sc\_module**
- n) Default time units and all the associated functions and constructors, including:
  - 1) Function **sc\_simulation\_time**
  - 2) Function **sc\_set\_default\_time\_unit**
  - 3) Function **sc\_get\_default\_time\_unit**
  - 4) Function **sc\_start(double)**
  - 5) Constructor **sc\_clock(const char\*, double, double, double, bool)**
- o) Member function **trace** of classes **sc\_object**, **sc\_signal**, **sc\_clock**, and **sc\_fifo** (Use **sc\_trace** instead.)
- p) Member function **add\_trace** of classes **sc\_in** and **sc\_inout** (Use **sc\_trace** instead.)
- q) Member function **get\_data\_ref** of classes **sc\_signal** and **sc\_clock** (Use member function **read** instead.)
- r) Member function **get\_new\_value** of class **sc\_signal**
- s) Typedefs **sc\_inout\_clk** and **sc\_out\_clk** (Use **sc\_out<bool>** instead.)
- t) Typedef **sc\_signal\_out\_if**
- u) Constant **SC\_DEFAULT\_STACK\_SIZE** (Function **set\_stack\_size** is not deprecated.)
- v) Constant **SC\_MAX\_NUM\_DELTA\_CYCLES**

- w) Constant `SYSTEMC_VERSION` (Function `sc_version` is not deprecated.)
- x) Support for the `wif` and `isdb` trace file formats (The `vcd` trace file format is not deprecated.)
- y) Member function `sc_set_vcd_time_unit` of class `vcd_trace_file`
- z) Function `sc_trace_delta_cycles`
- aa) Function `sc_trace` for writing enumeration literals to the trace file (Other `sc_trace` functions are not deprecated.)
- ab) Type `sc_bit` (Use type `bool` instead.)
- ac) Macro `SC_CTHREAD`, except for the case where the second argument is an event finder, which is still supported
- ad) Global and local watching for clocked threads (Use function `reset_signal_is` instead.)
- ae) The reporting mechanism based on integer ids and the corresponding member functions of class `sc_report`, namely `register_id`, `get_message`, `is_suppressed`, `suppress_id`, `suppress_infos`, `suppress_warnings`, `make_warnings_errors`, and `get_id`. (Replaced by a reporting mechanism using string message types)
- af) Utility classes `sc_string`, `sc_pvector`, `sc_plist`, `sc_phash`, and `sc_ppq`



## Annex D

(informative)

### Changes between the different SystemC versions

#### D.1 Significant changes made between SystemC version 2.0.1 and version 2.1 Beta Oct 12 2004

- a) Added the callback functions **before\_end\_of\_elaboration**, **start\_of\_simulation**, and **end\_of\_simulation**.
- b) Added the functions **sc\_start\_of\_simulation\_invoked** and **sc\_end\_of\_simulation\_invoked**.
- c) Added the function **sc\_main\_main**.
- d) Added the functions **sc\_argc** and **sc\_argv**.
- e) Added the function **sc\_stop\_mode**.
- f) Added function **sc\_delta\_count**.
- g) Added class **sc\_process\_handle**.
- h) Added function **sc\_spawn** and class **sc\_spawn\_options**.
- i) Added macros **SC\_FORK** and **SC\_JOIN**.
- j) Added classes **sc\_export\_base** and **sc\_export**.
- k) Added type **sc\_severity** and typedef **sc\_actions**.
- l) Modified the classes **sc\_report** and **sc\_report\_handler**.
- m) Added functions **sc\_interrupt\_here** and **sc\_stop\_here**.
- n) Added classes **sc\_event\_queue\_if** and **sc\_event\_queue**.
- o) Changed base class of class **sc\_clock** from **sc\_module** to **sc\_signal<bool>**.
- p) Added function **reset\_signal\_is** to class **sc\_module**.
- q) Added the function **sc\_release**.
- r) Added classes **sc\_generic\_base** and **sc\_value\_base**.
- s) Removed restrictions concerning the mixing of data types in concatenations.
- t) The process macros (**SC\_METHOD**, **SC\_THREAD**, and **SC\_CTHREAD**) can register the same function multiple times in the same scope without C++ compiler errors or SystemC name clashes.
- u) Changed the return type of function **write** of classes **sc\_inout** and **sc\_out** to **void**.

#### D.2 Changes made between SystemC version 2.1 Beta Oct 12 2004 and this standard

- a) Added the pure virtual function **set\_time\_unit** of class **sc\_trace\_file**.
- b) Added the member functions **start\_time** and **posedge\_first** of class **sc\_clock**.
- c) Added member functions **valid**, **name**, **proc\_kind**, **get\_child\_objects**, **get\_parent\_object**, **get\_process\_object**, **dynamic**, **terminated**, **terminated\_event**, **operator==**, **operator!=** to the class **sc\_process\_handle**.
- d) Removed member function **wait()** from class **sc\_process\_handle**.
- e) Defined function **wait( int )** for thread as well as for clocked thread process, and added member function **wait( int )** to class **sc\_prim\_channel**, and as a non-member function.

- f) Added function **sc\_get\_current\_process\_handle**.
- g) Added member function **kind** to class **sc\_event\_queue** and changed the default constructor name seed to **"event\_queue"**.
- h) Added member functions **get\_parent\_object** and **get\_child\_objects** to class **sc\_object**.
- i) Changed the behavior of **sc\_object** constructors and **sc\_gen\_unique\_name** such that every **sc\_object** is registered in the object hierarchy and has a unique hierarchical name of the form **seed\_N**.
- j) Added functions **sc\_find\_object** and **sc\_get\_top\_level\_objects**.
- k) Added function **sc\_is\_running**.
- l) Disabled copy constructor and assignment operator of class **sc\_spawn\_options**.
- m) Removed the constructors of class **sc\_export** that bind the export at the time of construction.
- n) Added member function **const IF\* operator-> () const** to class **sc\_export**.
- o) Made constructor **sc\_export( const char\* )** explicit.
- p) Each export shall be bound exactly once.
- q) Rename function **sc\_main\_main** to **sc\_elab\_and\_sim**.
- r) Stop mode **SC\_STOP\_IMMEDIATE** does not execute the update phase before stopping.
- s) A call to function **sc\_spawn** during what would have been the final update phase causes the spawned process to run in the next evaluation phase, potentially causing simulation to continue.
- t) Changed the behavior of **sc\_report\_handler::stop\_after(SC\_FATAL,-1)** such that **sc\_stop** is not called on the first fatal error. (Note that simulation may still abort due to the default actions.)
- u) Changed the type of the first parameter to functions **sc\_stop\_here** and **sc\_interrupt\_here** from **int** to **const char\***.
- v) Removed all forms of the second argument to macro **SC\_CTHREAD** except for **sc\_event\_finder**.
- w) Changed the signatures of functions **print** and **dump** to **virtual void print/dump( ostream& = cout ) const;** for class **sc\_object** and derived classes, and to **void print/dump( ostream& = cout ) const;** for all other classes.
- x) Added member functions **get\_count** to class **sc\_report\_handler**.
- y) Removed the member functions **initialize** and **release** of class **sc\_report\_handler**.
- z) Removed the class **sc\_pvector**, and changed the return type of function **get\_child\_objects** from **sc\_pvector** to **std::vector**.
- aa) Removed the class **sc\_string**, changed all occurrences of **sc\_string** to **std::string**.
- ab) Put every declaration in one of the two namespaces **sc\_core** and **sc\_dt**.
- ac) Added a new header file **"systemc"** that does not introduce any names into the global namespace besides **sc\_core** and **sc\_dt**.
- ad) Derived class **sc\_report** from **std::exception**.
- ae) Changed the **default\_handler** to throw an **sc\_report** object.
- af) Removed the class **sc\_exception** and added **typedef std::exception sc\_exception**.
- ag) Allowed process macros to be invoked from the **before\_end\_of\_elaboration** and **end\_of\_elaboration** callbacks.
- ah) Changed the behavior of event finders on multiports so that processes are made sensitive to every channel, not just the first channel.
- ai) Added a binding policy to class **sc\_port** to permit unbound ports.

## Annex E

(informative)

### List of Participants

This entity-based standard was created under the leadership of the following individuals:

**Victor Berman**, *Chair*  
**Stuart Swan**, *Technical Working Group Chair*  
**Oz Levia**, *Vice-Chair*  
**John Aynsley**, *Co-Author (Core)*  
**David Long**, *Co-Author (Data Types)*  
**Sofie Vandeputte**, *Typographical Editor*  
**Dennis Brophy**, *Secretary*

At the time this entity-based standard was completed, the LRM Working Group had the following membership:

|                        |                   |                 |
|------------------------|-------------------|-----------------|
| Cadence Design Systems | CoWare            | Mentor Graphics |
| Calypto                | Jeda Technologies | Synopsys        |
|                        | JEITA             |                 |

The following members of the entity-based balloting committee voted on this standard. Balloting entities may have voted for approval, disapproval, or abstention.

|                        |                   |                 |
|------------------------|-------------------|-----------------|
| Cadence Design Systems | Jeda Technologies | Sony            |
| Calypto                | JEITA             | Sunburst Design |
| CoWare                 | Mentor Graphics   | Synopsys        |

The working group gratefully acknowledges the contributions of the following participants.

|                          |                         |                    |
|--------------------------|-------------------------|--------------------|
| El Mustapha Aboulhamid   | Skip Hovsmith           | César Quiroz       |
| Mike Baird               | Hiroshi Imai            | Adam Rose          |
| Bishnupriya Bhattacharya | Masaharu Imai           | Ray Ryan           |
| David C Black            | Martin Janssen          | Kurt Schwartz      |
| Dundar Dumlogal          | Masaru Kakimoto         | Yasuyuki Shimizu   |
| Abhijit Ghosh            | Masamichi Kawarabayashi | Minoru Shoji       |
| Mark Glasser             | Kevin Kranen            | Bob Shur           |
| Andy Goodrich            | Evan Lavelle            | Kazuyoshi Takemura |
| Serge Goossens           | Mike Meredith           | Yasutaka Tsukamoto |
| Kazunori Goto            | Kazuya Morii            | Vincent Viteau     |
| Robert Graulich          | Wolfgang Mueller        | Masashi Watanabe   |
| Thorsten Groetker        | Kazuhide Nakamura       | Akihisa Yamada     |
| Takashi Hasegawa         | Junji Nakano            | Eugene Zhang       |
|                          | Takashi Ohsaka          |                    |

When the IEEE-SA Standards Board approved this standard on 6 December 2005, it had the following membership:

**Steve M. Mills**, *Chair*  
**Richard H. Hulett**, *Vice Chair*  
**Don Wright**, *Past Chair*  
**Judith Gorman**, *Secretary*

Mark D. Bowman  
Dennis B. Brophy  
Joseph Bruder  
Richard Cox  
Bob Davis  
Julian Forster\*  
Joanna N. Guenin  
Mark S. Halpin  
Raymond Hapeman

William B. Hopf  
Lowell G. Johnson  
Herman Koch  
Joseph L. Koepfinger\*  
David J. Law  
Daleep C. Mohla  
Paul Nikolic

T. W. Olsen  
Glenn Parsons  
Ronald C. Petersen  
Gary S. Robinson  
Frank Stone  
Malcolm V. Thaden  
Richard L. Townsend  
Joe D. Watson  
Howard L. Wolfman

\*Member Emeritus

Also included are the following nonvoting IEEE-SA Standards Board liaisons:

Satish K. Aggarwal, *NRC Representative*  
Richard DeBlasio, *DOE Representative*  
Alan H. Cookson, *NIST Representative*

Jennie Steinhagen  
*IEEE Standards Project Editor*

## INDEX

### A

- abstract class, glossary 393
- add\_attribute, member function
  - class sc\_object 97
- and\_reduce, reduction operator 178
- application
  - definition 4
  - glossary 393
- argument, glossary 393
- attach
  - attribute 97
  - glossary 393
- attr\_cltn, member function
  - class sc\_object 97

### B

- base class sub-object, glossary 393
- basename, member function
  - class sc\_object 94
- before\_end\_of\_elaboration, member function 21
  - class sc\_clock 121
  - class sc\_export 84
  - class sc\_module 44
  - class sc\_port 79
  - class sc\_prim\_channel 90
- begin, member function
  - class sc\_attr\_cltn 101
  - class sc\_fxcast\_context 369
  - class sc\_fxtype\_context 366
  - class sc\_length\_context 363
- binary fixed-point representation 278
- bind, member function
  - class sc\_export 82
  - class sc\_in 123
  - class sc\_port 74
- binding
  - export binding 13
  - glossary 393
  - named binding 43
  - port binding 13
  - positional binding 43
- bit concatenation
  - class sc\_concatref 236
  - vectors 270
- bit vector
  - definition 169
  - glossary 393
- bit-select
  - definition 175
  - glossary 393
- bit-select classes

- finite-precision integer types 227
- fixed-point types 284, 352
- introduction 175
- limited-precision integer types 199
- vectors 261

- body, glossary 393
- Boost, support for the free C++ library 55
- bound
  - glossary 393
  - port to channel 13
- buffer
  - definition 116
  - glossary 393

### C

- C++ header file 27
- C++, relationship with SystemC 1
- call
  - definition 4
  - glossary 393
- callback
  - from kernel 20
  - glossary 394
- called from, definition 4
- can, usage 4
- cancel, member function
  - class sc\_event 67
- cancel\_all, member function
  - class sc\_event\_queue 167
- canonical signed digit 180
- cast\_switch, member function
  - limited-precision fixed-point classes 286
- channel
  - glossary 394
  - hierarchical 5
  - instance 77
  - interface proper 85
  - ordered set of channel instances 75
  - port binding 43
  - primitive 5, 88
  - pure virtual functions 103
  - trace 123
- child
  - definition 5
  - get\_child\_objects, member function 45
  - glossary 394
  - port binding 13
- class template, glossary 394
- classes
  - sc\_attr\_base 99
  - sc\_attr\_cltn 101
  - sc\_attribute 100
  - sc\_bigint 223

## classes (Continued)

|                           |                    |                               |        |
|---------------------------|--------------------|-------------------------------|--------|
| sc_biguint                | 225                | sc_int_base                   | 185    |
| sc_bind_proxy             | 29                 | sc_int_bitref                 | 199    |
| sc_bitref_r               | 261                | sc_int_bitref_r               | 199    |
| sc_buffer                 | 116                | sc_int_subref                 | 203    |
| sc_bv                     | 257                | sc_int_subref_r               | 203    |
| sc_bv_base                | 244                | sc_interface                  | 85     |
| sc_clock                  | 119                | sc_length_context             | 362    |
| sc_concatref              | 236                | sc_length_param               | 361    |
| sc_concref                | 270                | sc_logic                      | 240    |
| sc_concref_r              | 270                | sc_lv                         | 259    |
| sc_context_begin          | 362, 363, 366, 368 | sc_lv_base                    | 251    |
| sc_event                  | 66                 | sc_module                     | 29     |
| sc_event_and_list         | 65                 | sc_module_name                | 47     |
| sc_event_finder           | 62                 | sc_mutex                      | 161    |
| sc_event_finder_t         | 62                 | sc_mutex_if                   | 160    |
| sc_event_or_list          | 65                 | sc_object                     | 92     |
| sc_event_queue            | 166                | sc_out                        | 133    |
| sc_event_queue_if         | 166                | sc_out_resolved               | 139    |
| sc_export                 | 80                 | sc_out_rv                     | 145    |
| sc_export_base            | 80, 81             | sc_plist (deprecated)         | 404    |
| sc_fifo                   | 150                | sc_port_base                  | 71     |
| sc_fifo_blocking_in_if    | 146                | sc_ppq (deprecated)           | 404    |
| sc_fifo_in                | 156                | sc_prim_channel               | 15, 88 |
| sc_fifo_in_if             | 146                | sc_process_handle             | 58     |
| sc_fifo_nonblocking_in_if | 146                | sc_pvector (deprecated)       | 404    |
| sc_fifo_out               | 157                | sc_report                     | 375    |
| sc_fifo_out_if            | 148                | sc_report_handler             | 378    |
| sc_fix                    | 331                | sc_semaphore                  | 164    |
| sc_fix_fast               | 280, 337           | sc_semaphore_if               | 163    |
| sc_fixed                  | 343                | sc_sensitive                  | 50     |
| sc_fixed_fast             | 280, 347           | sc_sensitive_neg (deprecated) | 403    |
| sc_fxcast_context         | 368                | sc_sensitive_pos (deprecated) | 403    |
| sc_fxcast_switch          | 367                | sc_signal                     | 107    |
| sc_fxnum                  | 280, 313           | sc_signal<bool>               | 113    |
| sc_fxnum_bitref           | 352                | sc_signal<sc_logic>           | 113    |
| sc_fxnum_fast             | 317                | sc_signal_in_if               | 103    |
| sc_fxnum_fast_bitref      | 352                | sc_signal_in_if<bool>         | 104    |
| sc_fxnum_fast_subref      | 354                | sc_signal_in_if<sc_logic>     | 104    |
| sc_fxnum_subref           | 354                | sc_signal_inout_if            | 106    |
| sc_fxtype_context         | 366                | sc_signal_resolved            | 134    |
| sc_fxtype_params          | 363                | sc_signal_rv                  | 140    |
| sc_fxval                  | 322                | sc_signal_write_if            | 106    |
| sc_fxval_fast             | 326                | sc_signed                     | 209    |
| sc_generic_base           | 239                | sc_signed_bitref              | 227    |
| sc_in                     | 122                | sc_signed_bitref_r            | 227    |
| sc_in<bool>               | 124                | sc_signed_subref              | 231    |
| sc_in<sc_dt::sc_logic>    | 124                | sc_signed_subref_r            | 231    |
| sc_in_resolved            | 137                | sc_simcontext (deprecated)    | 403    |
| sc_in_rv                  | 142                | sc_spawn_options              | 52     |
| sc_inout                  | 127                | sc_string (deprecated)        | 404    |
| sc_inout_resolved         | 138                | sc_subref                     | 264    |
| sc_inout_rv               | 143                | sc_subref_r                   | 264    |
| sc_int                    | 194                | sc_switch                     | 367    |
|                           |                    | sc_time                       | 68     |

- sc\_trace\_file 371
- sc\_ufix 280, 334
- sc\_ufixed 280, 345
- sc\_ufixed\_fast 350
- sc\_uint 197
- sc\_uint\_base 190
- sc\_uint\_bitref 200
- sc\_uint\_bitref\_r 199
- sc\_uint\_subref 203
- sc\_uint\_subref\_r 203
- sc\_unsigned 216
- sc\_unsigned\_bitref 227
- sc\_unsigned\_bitref\_r 227
- sc\_unsigned\_subref 231
- sc\_unsigned\_subref\_r 231
- sc\_value\_base 182
- clock
  - class sc\_clock 119
  - glossary 394
  - thread processes 36
- clocked thread process
  - arguments for macros 33
  - glossary 394
  - introduction 35
  - reset\_signal\_is, function 36
- complete object
  - glossary 394
  - module name 48
- concat, function
  - integers 237
  - logic and vector types 273
  - overview 176
  - template 277
- concat\_clear\_data, member function
  - class sc\_value\_base 182
- concat\_get\_ctrl, member function
  - class sc\_value\_base 182
- concat\_get\_data, member function
  - class sc\_value\_base 183
- concat\_get\_uint64, member function
  - class sc\_value\_base 183
- concat\_length, member function
  - class sc\_value\_base 183
- concat\_set, member function
  - class sc\_value\_base 183
- concatenation
  - base type 176
  - bits 270
  - class sc\_value\_base 182
  - glossary 394
  - integer 236
  - introduction 176
- const
  - SC\_BIND\_PROXY\_NIL 29

- SC\_LOGIC\_0 244
- SC\_LOGIC\_1 244
- SC\_LOGIC\_X 244
- SC\_LOGIC\_Z 244
- SC\_ZERO\_TIME 69
- const\_iterator, typedef 101
- contain
  - definition 5
  - glossary 394
- conversion, function
  - glossary 394
  - user-defined 172
- co-operative multitasking 16
- copy-constructible type
  - glossary 394
  - sc\_attribute template 100
- co-routine semantics 16

## D

- dagger symbol, usage 6
- data member, glossary 394
- data type classes 169
- data\_read, member function
  - class sc\_fifo\_out 157
- data\_read\_event, member function
  - class sc\_fifo 153
  - class sc\_fifo\_out 157
  - class sc\_fifo\_out\_if 149
- data\_written, member function
  - class sc\_fifo\_in 156
- data\_written\_event, member function
  - class sc\_fifo 153
  - class sc\_fifo\_in 156
  - class sc\_fifo\_in\_if 147
- declaration, glossary 394
- default\_event, member function
  - class sc\_event\_queue 167
  - class sc\_in 123
  - class sc\_inout 128
  - class sc\_interface 50, 86
  - class sc\_signal 110
- default\_value, member function
  - class sc\_fxcast\_context 369
  - class sc\_fxtype\_context 366
  - class sc\_length\_context 363
- definition, glossary 394
- delta cycle
  - class sc\_signal\_inout\_if 106
  - definition 17
  - glossary 395
  - write 110
- delta notification
  - cancel, function 66

- definition 15
- description 17
- glossary 395
- notify, function 66
- delta notification phase
  - glossary 395
  - overview 17
- deprecated features
  - class sc\_phash 404
  - class sc\_plist 404
  - class sc\_ppq 404
  - class sc\_pvector 404
  - class sc\_sensitive\_neg 403
  - class sc\_sensitive\_pos 403
  - class sc\_sim\_context 403
  - class sc\_string 404
  - sc\_bit, type 404
  - sc\_cycle, function 403
  - SC\_DEFAULT\_STACK\_SIZE, const 403
  - sc\_get\_curr\_process, function 403
  - sc\_get\_default\_time\_unit, function 403
  - sc\_initialize, function 403
  - sc\_inout\_clk, typedef 403
  - SC\_MAX\_NUM\_DELTA\_CYCLES, const 403
  - sc\_out\_clk, typedef 403
  - sc\_process\_b, type 403
  - sc\_set\_default\_time\_unit, function 403
  - sc\_set\_vcd\_time\_unit, member function 404
  - sc\_signal\_out\_if, typedef 403
  - sc\_start(double), function 403
  - sc\_trace\_delta\_cycles, function 404
  - set\_simulation\_time, function 403
- derived from, definition 4
- disabled, usage 6
- dont\_initialize, member function
  - class sc\_module 21, 38
  - class sc\_spawn\_options 53
  - semantics 16
- double, member function
  - class sc\_fxval 326
- dump, member function
  - class sc\_fifo 154
  - class sc\_object 95
  - class sc\_signal 111
- during elaboration
  - definition 6
  - glossary 395
- during simulation
  - definition 6
  - glossary 395
- duty\_cycle, member function
  - class sc\_clock 120
- dynamic process

- class sc\_process\_handle 58
  - definition 5
  - description 390
  - dynamic, member function 60
  - glossary 395
  - parent 54
- dynamic sensitivity
  - definition 15
  - glossary 395
  - next\_trigger, function 34, 39
  - wait, function 41
- dynamic spawned process
  - sc\_spawn, function 54
- dynamic, member function
  - class sc\_process\_handle 60

## E

- elaboration 11
  - callback functions 20
  - glossary 395
  - instantiation 11
  - keeping track of module hierarchy 48
  - port binding 14
  - port instantiation 73
  - running 18
  - sc\_main, function 19
  - sc\_set\_time\_resolution, function 69
  - simulation time resolution 14
- elem\_type, typedef 101
- ellipsis, usage 6
- end, member function
  - class sc\_attr\_cltn 102
  - class sc\_fxcast\_context 369
  - class sc\_fxtype\_context 366
  - class sc\_length\_context 363
- end\_of\_elaboration, member function 22
  - class sc\_export 84
  - class sc\_in 123
  - class sc\_in\_resolved 137
  - class sc\_in\_rv 142
  - class sc\_inout 129
  - class sc\_inout\_resolved 138
  - class sc\_inout\_rv 143
  - class sc\_module 44
  - class sc\_port 79
  - class sc\_prim\_channel 90
- end\_of\_simulation, member function 23
  - class sc\_export 84
  - class sc\_module 44
  - class sc\_port 79
  - class sc\_prim\_channel 90
- enumeration
  - sc\_curr\_proc\_kind\_return 58



- sc\_fmt 287
- sc\_logic\_value\_t 240
- sc\_numrep 180, 370
- sc\_o\_mode 286
- sc\_port\_policy 71
- sc\_q\_mode 286
- sc\_severity 375, 376
- sc\_stop\_mode 24
- sc\_time\_unit 68
- error
  - definition 9
  - glossary 395
- evaluation phase
  - definition 16
  - glossary 395
  - sc\_spawn, function 54
  - sc\_stop, function 25
  - update, function 90
- event
  - definition 5, 66
  - glossary 395
  - simulation 14
- event finder
  - class sc\_fifo\_in 156
  - class sc\_fifo\_out 157
  - class sc\_in 123, 126
  - class sc\_inout 128
  - class sc\_inout<bool> 132
  - class sc\_inout<sc\_dt::sc\_logic> 132
  - class sc\_sensitive 51
  - clocked thread process 36
  - description 62
  - glossary 395
- event list
  - class sc\_event 67
  - class sc\_event\_and\_list 65
  - class sc\_event\_or\_list 65
  - glossary 395
- event, member function
  - class sc\_in 123
  - class sc\_inout 128
  - class sc\_signal 110
  - class sc\_signal\_in\_if 103
- execution stack 35, 39
- export
  - class sc\_export 80
  - definition 4
  - glossary 395
- export binding 13

**F**

- fifo
  - class sc\_fifo 150

- glossary 395
- interfaces 104
- file
  - header files 27
  - trace file 372
- finite-precision fixed-point type
  - definition 169
  - glossary 396
- finite-precision integer
  - definition 169
  - glossary 396
  - overview 146
  - typedef 209
- fixed-point type 278
  - context object 174
  - definition 169

## G

- get\_attribute, member function
  - class sc\_object 97
- get\_child\_objects, member function
  - class sc\_module 45
  - class sc\_object 96
  - class sc\_process\_handle 60
  - return value 9
- get\_interface, member function
  - class sc\_export 84
  - class sc\_port 78
- get\_parent\_object, member function
  - class sc\_object 96
  - class sc\_process\_handle 60
  - return value 8
- get\_process\_object, member function
  - class sc\_process\_handle 58
  - return value 8
- get\_value, member function
  - class sc\_semaphore 165

## H

- hierarchical channel
  - class sc\_event\_queue 166
  - class sc\_object 93
  - definition 5
  - glossary 396
- hierarchical name
  - class sc\_module\_name 47
  - description 93
  - glossary 396

## I

- immediate notification
  - cancel, function 66
  - definition 15

- glossary 396
- notify, function 66
- implement
  - glossary 396
  - interface 85
  - update, function 110
- implementation
  - definition 4
  - glossary 396
- implementation-defined, definition 6
- implicit conversion, glossary 396
- initialization phase
  - dont\_initialize, function 38, 53
  - glossary 396
  - overview 16
- initialize, member function
  - class sc\_inout 128
- initializer list, glossary 396
- instance, glossary 396
- instantiation
  - during elaboration 11
  - glossary 396
- int\_type, typedef 184
- int64, typedef 184
- integer concatenation
  - class sc\_concatref 236
  - class sc\_value\_base 182
- integer part-select objects 207
- integer, glossary 396
- interface
  - class sc\_interface 85
  - definition 4
  - glossary 396
- Interface Method Call (IMC)
  - class sc\_module 31
  - class sc\_port 71
  - glossary 397
  - port and export binding 14
- interface proper
  - class sc\_fifo\_in\_if 146
  - class sc\_fifo\_out\_if 148
  - class sc\_interface 85
  - class sc\_mutex\_if 160
  - class sc\_port 72
  - class sc\_semaphore\_if 163
  - class sc\_signal\_in\_if 103
  - class sc\_signal\_inout\_if 106
  - definition 4
  - glossary 397
- invalid, process handle 58
- is\_01, member function
  - bit concatenation classes 274
  - bit-select classes 263
  - class sc\_bv\_base 248

- class sc\_logic 243
- class sc\_lv\_base 254
- part-select classes 268
- is\_neg, member function
  - fixed-point classes 287
- is\_zero, member function
  - fixed-point classes 287
- iterator
  - attributes 101
  - typedef 101
- iw1, member function
  - class sc\_fxtype\_params 364
  - limited-precision fixed-point classes 286

## K

- kernel
  - callbacks 20
  - elaboration and simulation 11
  - glossary 397
  - triggering method process 34
- kind, member function
  - class sc\_buffer 117
  - class sc\_clock 121
  - class sc\_event\_queue 167
  - class sc\_export 81
  - class sc\_fifo 154
  - class sc\_fifo\_in 156
  - class sc\_fifo\_out 157
  - class sc\_in 123
  - class sc\_in\_resolved 137
  - class sc\_in\_rv 142
  - class sc\_inout 128
  - class sc\_inout\_resolved 138
  - class sc\_inout\_rv 144
  - class sc\_module 31
  - class sc\_object 94
  - class sc\_out 133
  - class sc\_out\_resolved 139
  - class sc\_out\_rv 145
  - class sc\_port 74
  - class sc\_prim\_channel 89
  - class sc\_semaphore 165
  - class sc\_signal 111
  - class sc\_signal\_resolved 136
  - class sc\_signal\_rv 141

## L

- len, member function
  - class sc\_length\_param 362
- length context classes
  - class sc\_fxtype\_context 366
  - class sc\_fxtype\_params 363
  - class sc\_length\_context 362

- class `sc_length_param` 361
- introduction 174
- length, member function
  - bit-select classes 203, 230, 235
  - class `sc_bv_base` 250
  - class `sc_concatref` 238
  - class `sc_concref` 276
  - class `sc_generic_base` 239
  - class `sc_int_base` 189
  - class `sc_lv_base` 256
  - class `sc_signed` 216
  - class `sc_unsigned` 222
  - part-select classes 208, 270
- lifetime
  - glossary 397
  - of objects 7
- limited variable-precision fixed-point type
  - definition 169
- limited-precision fixed-point type
  - definition 169
  - glossary 397
- limited-precision integer
  - definition 169
  - glossary 397
- lock, member function
  - class `sc_mutex` 162
- logic vector
  - definition 169
  - glossary 397
- lrotate, member function
  - class `sc_bv_base` 249
  - class `sc_concref` 276
  - class `sc_lv_base` 256
  - part-select classes 269
- lvalue, glossary 397

**M**

macros

- `sc_assert` 380
- `sc_bind` 55
- `sc_cref` 55
- `SC_CTHREAD` 31, 33
- `SC_CTOR` 30, 32, 47
- `SC_DEFAULT_FATAL_ACTIONS` 380
- `SC_DEFAULT_INFO_ACTIONS` 380
- `SC_DEFAULT_WARNING_ACTIONS` 380
- `SC_FORK` 56
- `SC_HAS_PROCESS` 30, 33
- `SC_JOIN` 56
- `SC_METHOD` 13, 30, 33
- `SC_MODULE` 30, 31
- `sc_ref` 55
- `SC_REPORT_ERROR` 380

- `SC_REPORT_FATAL` 380
- `SC_REPORT_INFO` 380
- `SC_REPORT_WARNING` 380
- `SC_THREAD` 13, 31, 33
- may, usage 4
- member functions, glossary 397
- method process
  - arguments for macros 33
  - glossary 397
  - `spawn_method`, function 53
- method, glossary 397
- module
  - class `sc_module` 29
  - definition 4
  - glossary 397
- module hierarchy
  - abnormal usage 73
  - callbacks 21
  - class `sc_prim_channel` 89
  - definition 5
  - elaboration 11, 48
  - glossary 398
- multiport
  - definition 14
  - event finder 62
  - glossary 398
  - port policy 72
  - positional port binding 43
- mutex
  - class `sc_mutex` 161
  - glossary 398

**N**

- `n_bits`, member function
  - class `sc_fxtype_params` 365
  - limited-precision fixed-point classes 286
- name, member function
  - class `sc_attr_base` 99
  - class `sc_object` 94
  - class `sc_process_handle` 59
- namespace
  - `sc_core` 18
  - `sc_dt` 169
  - usage 9
- `nand_reduce`, reduction operator 178
- `nb_read`, member function
  - class `sc_fifo` 152
  - class `sc_fifo_in` 156
  - class `sc_fifo_in_if` 147
- `nb_write`, member function
  - class `sc_fifo` 152
  - class `sc_fifo_out` 157
  - class `sc_fifo_out_if` 149

- negedge, member function
  - class sc\_signal 115
  - class sc\_signal\_in\_if 105
- negedge\_event, member function
  - class sc\_signal 115
  - class sc\_signal\_in\_if 105
- next\_trigger, member function
  - class sc\_module 16, 39, 65
  - class sc\_prim\_channel 16, 90
  - creating dynamic sensitivity 34
- non-abstract class, glossary 398
- nor\_reduce, reduction operator 178
- notes, usage 10
- notification
  - delta notification 67
  - glossary 398
  - immediate 66
  - timed notification 67
- notified
  - events 14
  - glossary 398
  - process reset 37
  - timed notification 17
- notify, member function
  - class sc\_event 15, 66, 90
  - class sc\_event\_queue 167
  - delta notification phase 17
- num\_attributes, member function
  - class sc\_object 97
- num\_available, member function
  - class sc\_fifo 153
  - class sc\_fifo\_in 156
  - class sc\_fifo\_in\_if 147
- num\_free, member function
  - class sc\_fifo 153
  - class sc\_fifo\_out 157
  - class sc\_fifo\_out\_if 149
- numeric type
  - definition 170
  - glossary 398

## O

- o\_mode, member function
  - class sc\_fxtype\_params 365
  - limited-precision fixed-point classes 286
- object
  - class sc\_object 92
  - glossary 398
- object hierarchy
  - class sc\_object 92
  - definition 5
  - glossary 398
  - hierarchical instance name 94

- occurrence
  - class sc\_event 66
  - class sc\_report\_handler 378
  - events 14
  - glossary 398
  - warning 25
- operations, addition
  - limited-precision fixed-point classes 282, 283
  - variable-precision fixed-point classes 283, 284
- operations, arithmetic
  - class sc\_int 196
  - class sc\_int\_base 188
  - class sc\_signed 213
  - class sc\_uint 199
  - class sc\_uint\_base 193
  - class sc\_unsigned 219
  - fixed-point classes 281
- operations, bitwise
  - class sc\_bitref\_r 264
  - class sc\_bv\_base 248
  - class sc\_concref 275
  - class sc\_concref\_r 275
  - class sc\_int 196
  - class sc\_int\_base 188
  - class sc\_logic 243
  - class sc\_lv\_base 255
  - class sc\_signed 214
  - class sc\_subref 269
  - class sc\_subref\_r 268
  - class sc\_uint 199
  - class sc\_uint\_base 193
  - class sc\_unsigned 221
  - fixed-point classes 281
- operations, comparison
  - class sc\_bitref\_r 264
  - class sc\_bv\_base 249
  - class sc\_concref\_r 276
  - class sc\_int\_base 188
  - class sc\_logic 243
  - class sc\_lv\_base 256
  - class sc\_signed 215
  - class sc\_subref\_r 269
  - class sc\_uint\_base 193
  - class sc\_unsigned 222
- operator bool
  - bit-select classes 203
  - class sc\_fxnum\_bitref 354
  - class sc\_fxnum\_fast\_bitref 354
- operator double
  - class sc\_fxnum 317, 321
  - class sc\_fxval\_fast 330
- operator IF&
  - class sc\_export 83
- operator int\_type

- class sc\_int\_base 187
- class sc\_int\_subref\_r 207
- operator sc\_bv\_base
  - class sc\_fxnum\_fast\_subref 360
  - class sc\_fxnum\_subref 360
- operator sc\_logic
  - bit-select classes 263
- operator sc\_unsigned
  - class sc\_signed 238
  - class sc\_signed\_subref\_r 235
  - class sc\_unsigned\_subref\_r 235
- operator uint\_type
  - class sc\_uint\_base 192
  - class sc\_uint\_subref\_r 207
- operator uint64
  - bit-select classes 230
  - class sc\_unsigned 238
- operator!
  - bit-select classes 203, 230
- operator!=
  - class sc\_fxcast\_switch 368
  - class sc\_fxtype\_params 365
  - class sc\_length\_param 362
  - class sc\_process\_handle 59
- operator&
  - class sc\_event 65
- operator()
  - class sc\_export 82
  - class sc\_in 123
  - class sc\_inout 128
  - class sc\_module, port binding 43
  - class sc\_module\_name 48
  - class sc\_port 74
  - part-select classes 176
  - sc\_spawn, function 54
- operator<<
  - class ostream 111, 151
  - class sc\_sensitive 50
  - SystemC data types 179
- operator=
  - class sc\_buffer 117
  - class sc\_fifo 153
  - class sc\_fxcast\_switch 368
  - class sc\_fxtype\_params 365
  - class sc\_inout 128
  - class sc\_length\_param 362
  - class sc\_process\_handle 59
  - class sc\_signal 110
  - class sc\_signal\_resolved 135, 136
- operator==
  - class sc\_fxcast\_switch 368
  - class sc\_fxtype\_params 365
  - class sc\_length\_param 362
  - class sc\_process\_handle 59

- operator->
  - class sc\_export 83
  - class sc\_port 76
- operator>>
  - SystemC data types 179
- operator[]
  - bit-select classes 175
  - class sc\_port 77
- operator|
  - class sc\_event 65
- operator,
  - concatenation overview 176
  - integers 237
  - logic and vector types 273
  - template 277
- operator~
  - bit-select classes 203, 230
- or\_reduce, reduction operator 178
- overflow modes 289
- overflow\_flag
  - fixed-point classes 287
- overload, glossary 398
- override, glossary 398

## P

- parameter, glossary 398
- parent
  - definition 5
  - glossary 398
- part-select classes
  - definition 175
  - fixed-point types 284
  - glossary 398
  - limited-precision integers 203
  - vectors 264
- pending
  - cancel, function 67
  - class sc\_event\_queue 166
  - glossary 398
  - update, function 17
- period, member function
  - class sc\_clock 120
- port 71
  - binding 13, 129
  - class sc\_port 77
  - definition 4
  - glossary 399
  - named binding 43
  - port binding 43
  - positional binding 43
- port policy 14, 72
- portless channel access
  - description 71

- glossary 399
- posedge, member function
  - class sc\_signal 115
  - class sc\_signal\_in\_if 105
- posedge\_event, member function
  - class sc\_signal 114
  - class sc\_signal\_in\_if 105
- posedge\_first, member function
  - class sc\_clock 120, 121
- positional binding 43
- post, member function
  - class sc\_semaphore 165
- primitive channel
  - class sc\_buffer 116
  - class sc\_clock 119
  - class sc\_prim\_channel 88
  - class sc\_signal 107, 113
  - definition 5
  - glossary 399
- print, member function
  - bit-select classes 203, 230, 235, 264
  - class sc\_bv\_base 250
  - class sc\_concatref 238
  - class sc\_concref 276
  - class sc\_fifo 154
  - class sc\_fxcast\_switch 367
  - class sc\_int\_base 189
  - class sc\_length\_param 362
  - class sc\_logic 244
  - class sc\_lv\_base 256
  - class sc\_object 95
  - class sc\_signal 110
  - class sc\_signed 215
  - class sc\_time 69
  - class sc\_unsigned 222
  - part-select classes 178, 208, 270
- proc\_kind, member function
  - class sc\_process\_handle 59
- process
  - associating 34
  - clocked thread 35
  - definition 5
  - dynamic sensitivity 39
  - glossary 399
  - instance 58, 92
  - method 34
  - resumed 35
  - sensitivity 15
  - static sensitivity 37, 50
  - synchronization 66
  - triggered 34
- process handle
  - class sc\_process\_handle 58
  - definition 5

- glossary 399
- proxy class
  - bit-selects 175
  - class sc\_generic\_base 179, 239
  - concatenations 176
  - definition 172
  - glossary 399
  - part-selects 175

## Q

- q\_mode, member function
  - class sc\_fxtype\_params 365
  - limited-precision fixed-point classes 286
- quantization modes, definition 302
- quantization\_flag, member function
  - fixed-point classes 287

## R

- range, member function
  - numeric types and vectors 175
- read, member function
  - class sc\_fifo 152
  - class sc\_fifo\_in 156
  - class sc\_fifo\_in\_if 147
  - class sc\_in 123
  - class sc\_inout 128
  - class sc\_signal 109
  - class sc\_signal\_in\_if 103
- reduction operators 178
- register\_port, member function
  - class sc\_fifo 152
  - class sc\_interface 86
  - class sc\_signal 109
  - class sc\_signal\_resolved 136
- remove\_all\_attributes, member function
  - class sc\_object 97
- remove\_attribute, member function
  - class sc\_object 97
- request\_update, member function
  - class sc\_prim\_channel 16, 89
  - scheduling algorithm 15
- reset\_signal\_is, member function
  - class sc\_module 36
- resolved signal
  - class sc\_in\_resolved 137
  - class sc\_in\_rv 142
  - class sc\_inout\_resolved 138
  - class sc\_inout\_rv 143
  - class sc\_out\_resolved 139
  - class sc\_signal\_resolved 134
  - class sc\_signal\_rv 140
  - definition 134
  - glossary 399

## resume

- class `sc_event` 66
- `dont_initialize`, function 38
- evaluation phase 16
- glossary 399
- lock, function 162
- notify, function 66
- `sc_start`, function 20
- scheduler 14
- thread process 35
- wait, function 41

## reverse, member function

- class `sc_bv_base` 250
- class `sc_concref` 276
- class `sc_lv_base` 256
- part-select classes 270

## reversed, member function

- part-select classes 270

## rounding modes 302

## rrotate, member function

- class `sc_bv_base` 250
- class `sc_concref` 276
- class `sc_lv_base` 256
- part-select classes 269

## rvalue, glossary 399

## S

`SC_ABORT`, value of type `sc_actions` 378, 384

`sc_abs`, function 387

`sc_actions`, typedef 380

`SC_ALL_BOUND`, value of type `sc_port_policy` 71

`sc_argc`, function 19

`sc_argv`, function 19

`sc_assert`, macro 380

`sc_attr_base`, class 99

`sc_attr_cltn`, class 101

`sc_attribute`, class 100

`sc_behavior`, typedef 31, 46

`sc_bigint`, class template 170, 223

`sc_biguint`, class template 170, 225

`SC_BIN`, value of type `sc_numrep` 370

`SC_BIN_SM`, value of type `sc_numrep` 370

`SC_BIN_US`, value of type `sc_numrep` 370

`sc_bind`, macro 55

`sc_bind_proxy`, class 29

`SC_BIND_PROXY_NIL`, const 29

`sc_bit`, type (deprecated) 404

`sc_bitref`, class template 261

`sc_bitref_r`, class template 261

`sc_buffer`, class 116

- derived from `sc_signal` 112

`sc_bv`, class template 170, 257

`sc_bv_base`, class 170, 244

`SC_CACHE_REPORT`, value of type `sc_actions` 375, 378, 384

`sc_channel`, typedef 31, 46

`sc_clock`, class 119

`sc_close_vcd_trace_file`, function 372

`sc_concatref`, class 236

`sc_concref`, class template 270

`sc_concref_r`, class template 270

`sc_context_begin`, class 362, 363, 366, 368

`sc_copyright`, function 387

`sc_core`, namespace 18

`sc_create_vcd_trace_file`, function 372

`sc_cref`, macro 55

`SC_CSD`, value of type `sc_numrep` 180, 370

`SC_CTHREAD`, macro 13, 31, 33

`SC_CTHREAD_PROC_`, `sc_curr_proc_kind` const 58, 59

`SC_CTOR`, macro 30, 32, 47

`sc_curr_proc_kind`, enumeration 58

`sc_cycle`, function (deprecated) 403

`SC_DEC`, value of type `sc_numrep` 370

`SC_DEFAULT_ERROR_ACTIONS`, macro 380

`SC_DEFAULT_FATAL_ACTIONS`, macro 380

`SC_DEFAULT_INFO_ACTIONS`, macro 380

`SC_DEFAULT_STACK_SIZE`, const (deprecated) 403

`SC_DEFAULT_WARNING_ACTIONS`, macro 380

`sc_delta_count`, function 26

`SC_DISPLAY`, value of type `sc_actions` 378, 384

`SC_DO_NOTHING`, value of type `sc_actions` 378, 383

`sc_dt`, namespace 169

`SC_E`, value of type `sc_fmt` 287

`sc_elab_and_sim`, function 18

`sc_end_of_simulation_invoked`, function 21

`SC_ERROR`, `sc_severity` constant 375, 376

`sc_event`, class 14, 15, 66

`sc_event_and_list`, class 65

`sc_event_finder`, class 51, 62, 128

`sc_event_finder_t`, class template 62

`sc_event_or_list`, class 65

`sc_event_queue`, class 166

`sc_event_queue_if`, class 166

`sc_exception`, typedef 386

`sc_export`, class 80

`sc_export_base`, class 80, 81

`SC_F`, value of type `sc_fmt` 287

`SC_FATAL`, `sc_severity` constant 375, 376

`sc_fifo`, class 150

`sc_fifo_blocking_in_if` class 146

`sc_fifo_in`, class 156

`sc_fifo_in_if`, class 146

`sc_fifo_nonblocking_in_if`, class 146

sc\_fifo\_out, class 157  
 sc\_fifo\_out\_if, class 148  
 sc\_find\_object, function 96  
 sc\_fix, class 170, 280, 331  
 sc\_fix\_fast, class 170, 280, 337  
 sc\_fixed, class template 170, 343  
 sc\_fixed\_fast, class template 170, 280, 347  
 sc\_fmt, enumeration 287  
 SC\_FORK, macro 56  
 SC\_FS, value of type sc\_time\_unit 68  
 sc\_fxcast\_context, class 368  
 sc\_fxcast\_switch, class 367  
 sc\_fxnum, class 170, 280, 313  
 sc\_fxnum\_bitref, class 352  
 sc\_fxnum\_fast, class 170, 317  
 sc\_fxnum\_fast\_bitref, class 352  
 sc\_fxnum\_fast\_subref, class 354  
 sc\_fxnum\_subref, class 354  
 sc\_fxtype\_context, class 366  
 sc\_fxtype\_params, class 363  
 sc\_fxval, class 170, 280, 322  
 sc\_fxval\_fast, class 170, 280, 326  
 sc\_gen\_unique\_name, function 31, 45  
 sc\_generic\_base, class 239  
 sc\_get\_curr\_process\_handle, function (deprecated) 403  
 sc\_get\_current\_process\_handle, function 61  
 sc\_get\_default\_time\_unit, function (deprecated) 403  
 sc\_get\_stop\_mode, function 24  
 sc\_get\_time\_resolution, function 69  
 sc\_get\_top\_level\_objects, function 96  
 SC\_HAS\_PROCESS, macro 30, 33  
 SC\_HEX, value of type sc\_numrep 370  
 SC\_HEX\_SM, value of type sc\_numrep 370  
 SC\_HEX\_US, value of type sc\_numrep 370  
 sc\_in, class 122  
     specialized port 112  
 sc\_in<bool>, class 124  
 sc\_in<sc\_dt::sc\_logic>, class 124  
 sc\_in\_clk, typedef 121  
 sc\_in\_resolved, class 137  
 sc\_in\_rv, class 142  
 SC\_INFO, sc\_severity constant 375, 376  
 sc\_initialize, function (deprecated) 403  
 sc\_inout, class 127  
     specialized port 112  
 sc\_inout\_clk, typedef (deprecated) 403  
 sc\_inout\_resolved, class 138  
 sc\_inout\_rv, class 143  
 sc\_int, class template 170, 194  
 sc\_int\_base, class 170, 185  
 sc\_int\_bitref, class 199  
 sc\_int\_bitref\_r, class 199  
 sc\_int\_subref, class 203  
 sc\_int\_subref\_r, class 203  
 sc\_interface, class 85  
 SC\_INTERRUPT, value of type sc\_actions 378, 384  
 sc\_interrupt\_here, function 384  
 sc\_is\_running, function 26  
 SC\_JOIN, macro 56  
 SC\_LATER, constant of type sc\_context\_begin 174, 363, 366, 368  
 sc\_length\_context, class 362  
 sc\_length\_param, class 361  
 SC\_LOG, value of type sc\_actions 378, 384, 385  
 sc\_logic, class 170, 240  
 SC\_LOGIC\_0, const 244  
 SC\_LOGIC\_1, const 244  
 sc\_logic\_value\_t, enumeration 240  
 SC\_LOGIC\_X, const 244  
 SC\_LOGIC\_Z, const 244  
 sc\_lv, class template 170, 259  
 sc\_lv\_base, class 170, 251  
 sc\_main, function 19  
     calling sc\_start 20  
 sc\_main\_main, function 405, 406  
 sc\_max, function 387  
 SC\_MAX\_NUM\_DELTA\_CYCLES, const (deprecated) 403  
 SC\_MAX\_SEVERITY, sc\_severity constant 375  
 SC\_METHOD, macro 13, 30, 33  
 SC\_METHOD\_PROC\_, sc\_curr\_proc\_kind constant 58, 59  
 sc\_min, function 387  
 sc\_module, class  
     definition 29  
     member sensitive 50  
     use of sc\_module\_name 47  
 SC\_MODULE, macro 30, 31  
 sc\_module\_name, class  
     definition 47  
     module instantiation 12  
     usage in sc\_module constructor 31  
 SC\_MS, sc\_time\_unit constant 68  
 sc\_mutex, class 161  
 sc\_mutex\_if, class 160  
 SC\_NO\_PROC\_, sc\_curr\_proc\_kind constant 58, 60  
 SC\_NOBASE, value of type sc\_numrep 370  
 SC\_NOW, constant of type sc\_context\_begin 174, 363, 366, 368  
 SC\_NS, sc\_time\_unit constant 68  
 sc\_numrep, enumeration 180, 370  
 sc\_o\_mode, enumeration 286  
 sc\_object, class  
     definition 92



- usage of `sc_module_name` 48
- `SC_OCT`, value of type `sc_numrep` 370
- `SC_OCT_SM`, value of type `sc_numrep` 370
- `SC_OCT_US`, value of type `sc_numrep` 370
- `SC_OFF`, constant of type `sc_switch` 285, 367
- `SC_ON`, constant of type `sc_switch` 285, 367
- `SC_ONE_OR_MORE_BOUND`, value of type `sc_port_policy` 71
- `sc_out`, class 133
  - specialized port 112
- `sc_out_clk`, typedef (deprecated) 403
- `sc_out_resolved`, class 139
- `sc_out_rv`, class 145
- `sc_phash`, class (deprecated) 404
- `sc_plist`, class (deprecated) 404
- `sc_port`, class
  - definition 72
  - positional port binding 43
- `sc_port_base`, class 71
- `sc_port_policy`, enumeration 71
- `sc_ppq`, class (deprecated) 404
- `sc_prim_channel`, class 15
  - definition 88
  - module hierarchy 48
- `sc_process_b`, type (deprecated) 403
- `sc_process_handle`, class 58
- `SC_PS`, `sc_time_unit` constant 68
- `sc_pvector`, class (deprecated) 404
- `sc_q_mode`, enumeration 286
- `sc_ref`, macro 55
- `sc_release`, function 388
- `sc_report`, class 375
- `SC_REPORT_ERROR`, macro 380
- `SC_REPORT_FATAL`, macro 380
- `sc_report_handler`, class 378
- `sc_report_handler_proc`, typedef 383
- `SC_REPORT_INFO`, macro 380
- `SC_REPORT_WARNING`, macro 380
- `SC_RND`, quantization mode 306
- `SC_RND_CONV`, quantization mode 310
- `SC_RND_INF`, quantization mode 309
- `SC_RND_MIN_INF`, quantization mode 308
- `SC_RND_ZERO`, quantization mode 307
- `SC_SAT`, overflow mode 293
- `SC_SAT_SYM`, overflow mode 294
- `SC_SAT_ZERO`, overflow mode 294
- `SC_SEC`, `sc_time_unit` constant 68
- `sc_semaphore`, class 164
- `sc_semaphore_if`, class 163
- `sc_sensitive`, class
  - data member of `sc_module` 37
  - module instantiation 12
- `sc_sensitive`, class definition 50
- `sc_sensitive_neg`, class (deprecated) 403
- `sc_sensitive_pos`, class (deprecated) 403
- `sc_set_default_time_unit`, function (deprecated) 403
- `sc_set_stop_mode`, function 24
- `sc_set_time_resolution`, function 14, 69
- `sc_set_vcd_time_unit`, member function (deprecated) 404
- `sc_severity`, enumeration 375, 376
- `sc_signal`, class 107
  - `sc_signal<bool>`, class 113
  - `sc_signal<sc_logic>`, class 113
  - `sc_signal_in_if`, class 103
  - `sc_signal_in_if<bool>`, class 104
  - `sc_signal_in_if<sc_logic>`, class 104
  - `sc_signal_inout_if`, class 106
  - `sc_signal_out_if`, typedef (deprecated) 403
  - `sc_signal_resolved`, class 134
    - multiple writers 112
  - `sc_signal_rv`, class 140
  - `sc_signal_write_if`, class 106
- `sc_signed`, class 170, 209
- `sc_signed_bitref`, class 227
- `sc_signed_bitref_r`, class 227
- `sc_signed_subref`, class 231
- `sc_signed_subref_r`, class 231
- `sc_simcontext`, class (deprecated) 403
- `sc_simulation_time`, function (deprecated) 403
- `sc_spawn`, function 52, 54
- `sc_spawn_options`, class 52
- `sc_start(double)`, function (deprecated) 403
- `sc_start`, function 19
- `sc_start_of_simulation_invoked`, function 21
- `sc_stop`, function 25
  - impact on clock 120
- `SC_STOP`, value of type `sc_actions` 378, 384
- `SC_STOP_FINISH_DELTA`, value of type `sc_stop_mode` 24, 25
- `sc_stop_here`, function 384
- `SC_STOP_IMMEDIATE`, value of type `sc_stop_mode` 24, 25, 406
- `sc_stop_mode`, enumeration 24
- `sc_string`, class (deprecated) 404
- `sc_subref`, class template 264
- `sc_subref_r`, class template 264
- `sc_switch`, class 367
- `SC_THREAD`, macro 13, 31, 33
- `SC_THREAD_PROC`, `sc_curr_proc_kind` constant 58, 59
- `SC_THROW`, value of type `sc_actions` 376, 378, 383
- `sc_time`, class 15, 68
- `sc_time_stamp`, function 25
- `sc_time_unit`, enumeration 68
- `sc_trace`, function 123, 128, 372, 374
- `sc_trace_delta_cycles`, function (deprecated) 404

- sc\_trace\_file, class 371
- SC\_TRN, quantization mode 311
- SC\_TRN\_ZERO, quantization mode 312
- sc\_ufix, class 170, 280, 334
- sc\_ufix\_fast, class 170, 280
- sc\_ufixed, class template 170, 345
- sc\_ufixed\_fast, class template 170, 350
- sc\_uint, class template 170, 197
- sc\_uint\_base, class 170, 190
- sc\_uint\_bitref, class 200
- sc\_uint\_bitref\_r, class 199
- sc\_uint\_subref, class 203
- sc\_uint\_subref\_r, class 203
- sc\_unsigned, class 170, 216
- sc\_unsigned\_bitref, class 227
- sc\_unsigned\_bitref\_r, class 227
- sc\_unsigned\_subref, class 231
- sc\_unsigned\_subref\_r, class 231
- SC\_UNSPECIFIED, value of type sc\_actions 378, 383
- SC\_US, sc\_time\_unit constant 68
- sc\_value\_base, class 182
- sc\_version, function 388
- SC\_WARNING, sc\_severity constant 375, 376
- SC\_WRAP, overflow mode 295
- SC\_WRAP\_SM, overflow mode 297
- sc\_write\_comment, function 372
- SC\_ZERO\_OR\_MORE\_BOUND, value of type sc\_port\_policy 71
- SC\_ZERO\_TIME, const 69
- scan, member function
  - bit-select classes 203, 230, 235, 264
  - class sc\_bv\_base 250
  - class sc\_concatref 238
  - class sc\_concref 276
  - class sc\_int\_base 189
  - class sc\_logic 244
  - class sc\_lv\_base 256
  - class sc\_signed 215
  - class sc\_uint\_base 194
  - class sc\_unsigned 222
  - part-select classes 208, 270
  - SystemC data types 178
- scheduled
  - description 390
  - glossary 399
  - multiple event notification 67
- scheduler
  - behavior 14
  - glossary 399
- sensitive, data member
  - class sc\_module 37
- sensitivity
  - dynamic 41
- glossary 399
- method process instance 34, 40
- spawned process instance 53
- static 37
- thread process instance 35
- unspawned process instance 37, 50
- set\_sensitivity, member function
  - class sc\_spawn\_options 53
- set\_stack\_size, member function
  - class sc\_module 39
  - class sc\_spawn\_options 53
- set\_time\_unit, member function
  - class sc\_trace\_file 372
- shall, usage 4
- should, usage 4
- sign extension 173
- signal
  - definition 5
  - glossary 399
  - reading and writing 108
  - resolved 134, 135
  - value of sc\_signal 107
- signature, glossary 400
- simulation
  - behavior 14
  - callbacks 20
  - end\_of\_simulation, function 23
  - glossary 400
  - sc\_elab\_and\_sim, function 18
  - semantics 11
  - start\_of\_simulation, function 23
- simulation time 25
- single-bit logic types, definition 169
- size, member function
  - class sc\_port 75
- spawn\_method, member function
  - class sc\_spawn\_options 53
- spawned process
  - class sc\_spawn\_options 52
  - definition 5
  - glossary 400
  - parent 92
  - SC\_FORK, macro 56
  - SC\_JOIN, macro 56
  - sc\_spawn, function 52
  - sensitivity 15
- specialized port
  - class sc\_fifo\_in 156
  - class sc\_fifo\_out 157
  - class sc\_in 122
  - class sc\_in<bool> 124
  - class sc\_in<sc\_dt::sc\_logic> 124
  - class sc\_in\_resolved 137
  - class sc\_in\_rv 142

- class sc\_inout 127
- class sc\_inout<bool> 130
- class sc\_inout<sc\_dt::sc\_logic> 130
- class sc\_inout\_resolved 138
- class sc\_inout\_rv 143
- get\_interface, function 78
- glossary 400
- start\_of\_simulation, member function 23
  - class sc\_export 84
  - class sc\_module 44
  - class sc\_port 79
  - class sc\_prim\_channel 90
- start\_time, member function
  - class sc\_clock 120
- statement, glossary 400
- static process
  - clocked thread process 36
  - definition 5
  - description 390
  - dynamic, function 60
  - glossary 400
- static sensitivity
  - definition 15
  - glossary 400
- static spawned process
  - sc\_spawn, function 54
- string literal 180
- string name
  - basename, function 94
  - case-sensitivity 45
  - constructor sc\_export 81
  - constructor sc\_module 32
  - constructor sc\_object 93
  - constructor sc\_port 74
  - constructor sc\_prim\_channel 89
  - glossary 400
  - module instance 31
  - sc\_gen\_unique\_name, function 45
  - sc\_module\_name, function 47
  - sc\_spawn, function 55
- sub-object, glossary 400
- SystemC class library 27
- systemc.h, C++ header file 27

## T

- terminated
  - glossary 400
  - method process 34
  - process instance 35
  - sc\_process\_handle 58
- terminated, member function
  - class sc\_process\_handle 60
- terminated\_event, member function

- class sc\_process\_handle 60
- terminology 4
- thread process
  - arguments for macros 33
  - class sc\_process\_handle 58
  - glossary 400
  - initialization phase 16
  - interface methods 391
  - reset\_signal\_is, function 36
  - set\_stack\_size, function 39
  - terminated, function 60
  - wait, function 35, 41
- time resolution 14, 69
- timed notification
  - class sc\_event 67
  - class sc\_event\_queue 167
  - definition 15
  - glossary 400
  - scheduling algorithm 15
- timed notification phase
  - definition 17
  - glossary 400
- time-out
  - definition 16
  - elapsed time interval 15
  - glossary 400
  - timed notification phase 17
- to\_bin, member function
  - class sc\_fxnum 317, 322
  - class sc\_fxval 326
  - class sc\_fxval\_fast 331
- to\_bool, member function
  - bit-select classes 263
  - class sc\_logic 243
- to\_char, member function
  - bit-select classes 263
  - class sc\_logic 243
- to\_dec, member function
  - class sc\_fxnum 317, 322
  - class sc\_fxval 326
  - class sc\_fxval\_fast 331
- to\_double, member function
  - class sc\_fxnum 317, 321
  - class sc\_fxval 326
  - class sc\_fxval\_fast 330
  - class sc\_time 69
- to\_float, member function
  - class sc\_fxnum 317, 321
  - class sc\_fxval 326
  - class sc\_fxval\_fast 330
- to\_hex, member function
  - class sc\_fxnum 317, 322
  - class sc\_fxval 326
  - class sc\_fxval\_fast 331

- to\_int, member function
  - class sc\_fxnum 317, 321
  - class sc\_fxnum\_fast\_subref 360
  - class sc\_fxnum\_subref 360
  - class sc\_fxval 326
  - class sc\_fxval\_fast 330
  - SystemC data types 178
- to\_int64, member function
  - class sc\_fxnum 317, 321
  - class sc\_fxnum\_fast\_subref 360
  - class sc\_fxnum\_subref 360
  - class sc\_fxval 326
  - class sc\_fxval\_fast 330
  - class sc\_generic\_base 239
  - SystemC data types 178
- to\_long, member function
  - class sc\_fxnum 317, 321
  - class sc\_fxnum\_fast\_subref 360
  - class sc\_fxnum\_subref 360
  - class sc\_fxval 326
  - class sc\_fxval\_fast 330
  - SystemC data types 178
- to\_oct, member function
  - class sc\_fxnum 317, 322
  - class sc\_fxval 326
  - class sc\_fxval\_fast 331
- to\_sc\_signed, member function
  - class sc\_generic\_base 239
- to\_sc\_unsigned, member function
  - class sc\_generic\_base 239
- to\_seconds, member function
  - class sc\_time 69
- to\_short, member function
  - class sc\_fxnum 317, 321
  - class sc\_fxval 326
  - class sc\_fxval\_fast 330
- to\_string, member function 370
  - bit concatenation classes 273
  - class sc\_bv\_base 247
  - class sc\_fxcast\_switch 367
  - class sc\_fxnum 317, 321
  - class sc\_fxnum\_fast\_subref 360
  - class sc\_fxnum\_subref 360
  - class sc\_fxval 326
  - class sc\_fxval\_fast 331
  - class sc\_int\_base 188
  - class sc\_length\_param 362
  - class sc\_lv\_base 253
  - class sc\_signed 212
  - class sc\_time 69
  - class sc\_uint\_base 193
  - class sc\_unsigned 218, 238
  - fixed-point classes 287
  - part-select classes 208, 235, 267
  - SystemC numeric types 181
- to\_uint, member function
  - class sc\_fxnum 317, 321
  - class sc\_fxnum\_fast\_subref 360
  - class sc\_fxnum\_subref 360
  - class sc\_fxval 326
  - class sc\_fxval\_fast 330
  - SystemC data types 178
- to\_uint64, member function
  - class sc\_fxnum 317, 321
  - class sc\_fxnum\_fast\_subref 360
  - class sc\_fxnum\_subref 360
  - class sc\_fxval 326
  - class sc\_fxval\_fast 330
  - class sc\_generic\_base 239
  - SystemC data types 178
- to\_ulong, member function
  - class sc\_fxnum 317, 321
  - class sc\_fxnum\_fast\_subref 360
  - class sc\_fxnum\_subref 360
  - class sc\_fxval 326
  - class sc\_fxval\_fast 330
  - SystemC data types 178
- to\_ushort, member function
  - class sc\_fxnum 317, 321
  - class sc\_fxval 326
  - class sc\_fxval\_fast 330
- top-level module
  - definition 6
  - glossary 401
- top-level object
  - definition 92
  - get\_parent\_object, function 96
  - glossary 401
  - sc\_spawn, function 54
- trace file 371
- trigger
  - class sc\_event 66
  - dont\_initialize, function 38
  - evaluation phase 16
  - glossary 401
  - method process 34
  - next\_trigger, function 16, 39
  - process sensitivity 15
  - scheduler 14
- trylock, member function
  - class sc\_mutex 162
- trywait, member function
  - class sc\_semaphore 165
- type of a port
  - definition 43
  - template parameters 72
- type\_params, member function
  - limited-precision fixed-point classes 286

typedef  
    const\_iterator 101  
    elem\_type 101  
    int\_type 184  
    int64 184  
    iterator 101  
    sc\_actions 380  
    sc\_behavior 31, 46  
    sc\_channel 31, 46  
    sc\_exception 386  
    sc\_in\_clk 121  
    sc\_inout\_clk (deprecated) 403  
    sc\_out\_clk (deprecated) 403  
    sc\_report\_handler\_proc 383  
    sc\_signal\_out\_if (deprecated) 403  
    uint\_type 184  
    uint64 184

## U

uint\_type, typedef 184  
uint64, typedef 184  
undefined  
    errors 10  
    glossary 401  
unlock, member function  
    class sc\_mutex 162  
unspawned process  
    class sc\_object 92  
    class sc\_process\_handle 58  
    creating 33  
    definition 5  
    glossary 401  
    instance 13  
    sensitivity 15  
    static sensitivity 15, 37  
update phase  
    definition 17  
    glossary 401  
update request 15, 21, 89, 110  
update, member function 17  
    class sc\_buffer 117  
    class sc\_fifo 153  
    class sc\_prim\_channel 16, 90  
    class sc\_signal 110  
    class sc\_signal\_resolved 135, 136  
user, glossary 401  
user-defined conversion, glossary 401

## V

valid, glossary 401  
valid, member function  
    class sc\_process\_handle 59  
value, member function  
    class sc\_fxcast\_context 369

class sc\_fxtype\_context 366  
class sc\_length\_context 363  
class sc\_logic 243  
class sc\_time 69  
fixed-point classes 287  
value\_changed, member function  
    class sc\_in 123  
    class sc\_inout 128  
value\_changed\_event, member function  
    class sc\_in 123  
    class sc\_inout 128  
    class sc\_signal 110  
    class sc\_signal\_in\_if 103  
variable-precision fixed-point type  
    definition 169  
    glossary 401  
VCD file 371, 372  
vector  
    glossary 401  
    usage 170

## W

wait, member function  
    class sc\_module 16, 35, 41, 65  
    class sc\_prim\_channel 16, 35, 90  
    class sc\_semaphore 165  
    delta notification phase 17  
warning  
    definition 9  
    glossary 401  
within  
    definition 5  
    glossary 401  
    port binding 13  
wl, member function  
    class sc\_fxtype\_params 365  
    limited-precision fixed-point classes 286  
write, member function 106  
    class sc\_buffer 117  
    class sc\_clock 120  
    class sc\_fifo 152  
    class sc\_fifo\_out 157  
    class sc\_fifo\_out\_if 149  
    class sc\_inout 128  
    class sc\_signal 110  
    class sc\_signal\_resolved 135, 136  
    class sc\_signal\_rv 140  
    class sc\_signal\_write\_if 106

## X

xnor\_reduce, reduction operator 178  
xor\_reduce, reduction operator 178

## Z

zero extension 173



INTERNATIONAL  
ELECTROTECHNICAL  
COMMISSION

3, rue de Varembe  
PO Box 131  
CH-1211 Geneva 20  
Switzerland

Tel: + 41 22 919 02 11  
Fax: + 41 22 919 03 00  
[info@iec.ch](mailto:info@iec.ch)  
[www.iec.ch](http://www.iec.ch)